

MySQL

大智小技

五大主题

MySQL 8.0 新特性

技术分享

故障分析

开源产品

数据库开源产品

爱可生开源社区 有深度的 MySQL 社区

大智者，宏观把握，对技术产生的背景以及相应的原理应用了然于胸；

小技者，具体细微，不论什么问题一定要剖析入骨，捋清脉络。

拥有了大智小技，面对问题轻松自如，一目了然，一锤定音。

资料仅供内部学习

推 荐 序

时光荏苒《大智小技》不知不觉陪我们走过了4个年头，团队依然坚持以技术为导向，以客户为目标的宗旨。持续不断的积累经验与分享，同时积极开展对新技术探索与研究。路还很长，愿心不老，志不变。

华夏银行·胡捷、王辉

《大智小技4》今年开源数据库章节部分的内容研究涵盖了对时下火热产品 Oceanbase、TiDB、MongoDB、ClickHouse 等开源数据库的一些实际使用场景及其丰富的解决方案和案例介绍的讨论。在数字化时代的当下，金融需求出现了场景化、多元化、长尾化、碎片化等特点，这些变化不断冲击着传统金融 IT 架构，这些内容的讨论应该能给各位读者提供更多解决场景的选项。另外值得一提的是《大智小技4》仍然没有放松对 MySQL 新特性及各种 BUG 问题的探索和挖掘。可以看出书中每一个案例无不高度浓缩了每一位技术工程师几周甚至数月努力的成果，爱可生的各位老师依然坚持不懈将 MySQL 产品最新的特性通过实战的方式呈现给各位读者。十分有幸能为本书写推荐序，希望各位有志于深耕 MySQL 领域的 DBA 们能够有所收获。

银联数据-高级工程师·陈嘉发

一直是大智小技的忠实读者，也非常喜欢爱可生开源社区，其中汇聚了众多奋斗在开源数据库一线的高手，如华山论剑般，分享出一篇篇高质量的文章，诸多真知灼见，令人受益匪浅

中国电信全渠道运营中心·王健

2022年了，想到曾经在2012年到爱可生上海总部培训MySQL的过往，也是10年前冬天的事情了，作为当时国内唯一的被ORACLE认证的机构，10年时间爱可生从当时的MySQL专业的服务机构，转变为数据库自主研发机构和MySQL开源软件开发机构，紧跟时代的脚步，继续为各大企业提供专业MySQL方面的软件和服务，是推动MySQL在中国持续发展的中坚力量，在拿到第四本MySQL大智小技4，这就是最好的证明！

天财商龙 DBA TEAM LEAD & 数据库架构·刘华阳

持续不断地精进方能大成，对技术的不断积累，让我从《大智小技4》中看到了可能，有幸和爱可生开源社区一起前行。

国泰君安数据库管理员·毛大成

Congratulations! 《大智小技》第四册发布!

作为国内 mysql 领域的技术领导者和分享者,爱可生技术团队认真分析研究每个技术细节,和广大数据库从业者和爱好者分享了很多日常工作中遇到的问题和解决的思路以及方法.如果说数据库是一门理论+实操案例的学科,那么在《大智小技》中每一篇文章中都体现的淋漓尽致!是每个 Mysql DBA 必备的 handbook!感谢爱可生社区对开源生态的贡献!希望这个系列的书籍可以一直延续下去!

捷信金融(中国) DBA • 陈旭

本书是一本面向 MySQL 数据库管理员非常好的问题集,汇集了一年以来全行业数据库运维专家总结的运维经验,解决了一些平时工作中遇到的普遍问题。本书也是一本实用的工具书,不论对于初学者,还是对于具有一定经验的技术人员,在实际的生产运维过程中提供帮助和启发。

最初,开源数据库的应用大家都是摸着石头过河,通过在社区中互相学习,互相借鉴,不断摸索,近年来在生产系统中得到了广泛的应用。随着应用场景的不断丰富并深入,对于技术人员运维能力要求也越来越高,而数据库运维又是一个非常细分的专业领域,需要耐得住寂寞,贵在坚持。期待爱可生开源社区未来为 DBA 们提供更多有价值的知识和技巧,为数据库开源生态贡献更完整的知识体系。

中信登高级专家 • 黄彬

《大智小技》源自社区,再成书回馈大众,很好的诠释了开源精神。也正是因为素材源于真实场景,也让这一系列书籍特别的接地气,可以帮助大家解决工作中的问题,适合放在案头随时查阅。翻看《大智小技-IV》样稿,又发现了数篇感兴趣的文章,感谢爱可生的无私奉献。

某财险数据库负责人 • 董健

今年的大智小计依然嗅觉敏锐,为正在用 MySQL 8.0 版本的同学们提供了故障分析、迁移等日常运维中大量的干货;同时不拘泥于 MySQL 更是广泛涉猎其他明星开源产品,为 DBA 同学打开了又一扇窗。相信读完全书,大智小计定能成为你日常工作中常用的工具书;同时也定能让你在开源的海洋中找到自己的那份热爱。

工商银行数据中心 MySQL 数据库专家 • 顾龚磊

《大智小技》系列出品必是精品,凝集很多技术大咖的文章、思想,是真正在日常工作中可以帮助解决疑难问题的书籍。

蓝月亮(中国) • 马文斌

一年一度的《大智小技》又来了，内容还是一如既往的好，尤其是“故障分析”这部分，对 DBA 们是很好的参考，能很好地学习到优秀同行解决分析问题的思路。

手动点赞爱可生开源社区，望戒骄戒躁，保持优秀！

「3306π」社区联合创始人，Oracle MySQL ACE Director • 叶金荣

《大智小计》是一本优秀的开源数据库技术类书籍，面向数据库专业技术人员及业余爱好发烧友，本书结合实际案例讲解技术原理，深入浅出，生动形象。着重介绍了 MySQL 的最新技术特点，同时还包含其他开源数据库，例如 OB、Tidb、MongoDB 等产品的一些实际使用场景。是一本难得的值得推荐的数据库技术书籍。

某央企信托公司金融科技中心科技运维部总经理 • 徐佳

非常高兴看到爱可生开源社区的《MySQL 大智小技》已经到第四期了，这四年来，爱可生开源社区践行着做有深度的 MySQL 社区的理念，为广大 MySQL 粉丝提供了一个学习和交流的平台。每一期《MySQL 大智小技》都是干货满满，我每篇文章都看过，最喜欢看“技术分享”和“故障解析”栏目，从中受益匪浅，感谢爱可生的付出，祝愿爱可生开源社区越办越好！

Oracle ACE 鼎甲科技数据库专家 • 姚远

很欣喜看到《大智小技》如期发布了第四版，书中内容也是一如既往的高质量，其实我们从往年发布文章向后看，不难看出，DBA 这个岗位需求在不断变化，不单单在要求掌握单一的 MySQL，而是需要掌握更深、更广、更新的技术。

本书中涉及 MySQL 8.0 新特性章节包含了 MySQL 最新的版本特性介绍、开源产品&开源数据库章节包含更广的技术面，技术分享&故障解析章节也更加偏向底层，包含了很多实战的源码解析，这些都是当前 DBA 进一步学习进步不可或缺的知识。

推荐给所有 DBA&运维同学，希望大家看得开心，收货满满，也盼望疫情早日退散，生活回归正常！

国泰新点 • 许天云

作为数据库技术爱好者，不知不觉与《大智小技》相识四年。在爱可生开源社区的精心组织下，各路技术达人的数据库技术汇集于此，大家分享各自生产环境中遇到的故障案例，性能优化以及数据库新特性，k8s，OS 系统等技术知识，相信本期内容依然是数据库技术爱好者的饕餮盛宴，满足大家对数据库的各种技术需求。

阿里云数据库专家 公众号 yangyidba 作者 • 杨奇龙

翘首以盼的《MySQL 大智小技 4》终于集结出版了！在从传统商业数据库向开源数据库转型的浪潮中，MySQL 显然在软件生态、易用性、安全性等诸多方面皆为业界翘楚，也成为了银行业应用数据库的最佳选择之一。团队里的数据库管理员常常向我询问有没有独门秘籍或是灵丹妙药能够快速积攒运维经验，我始终向他们推荐《MySQL 大智小技》。

如今，该书第四卷承袭了其一贯的特色，既有对 MySQL 实战案例的讲解，又深入浅出地介绍了 MySQL 的运维工具。尤为难得的是，数据库性能调优和故障诊断等运维人所关注的场景一直贯穿始终，不管是运维小白还是资深大拿，都能从中获益良多并快速上手实战见效。随着 MySQL 运维实用技能的不断增长，时时翻阅《大智小计》必将有所裨益。最后，关于开源产品经验的分享也没有局限于 MySQL 数据库，拓展了各位读者的知识面。

感谢爱可生开源社区的付出，这本书一定能给正在学习 MySQL 的你带来帮助。

交通银行数据中心系统部高级经理·金恩华

目前 MySQL 是成熟稳重的开源数据库代表，不仅慢慢褪去了侧重开源、免费、简单的标签，而且技术生态也在不断强化。现在 MySQL 技术成为一种新的技术理念，学习 MySQL 而不拘泥于 MySQL，我觉得是现今要保持的一种学习冷静心态。在【大智小技】中我看到了这种理念，里面不仅包含了 MySQL 8.0 里面的新特性解读，也包含大量的诊断分析案例，还有基于开源技术趋势的产品集成思路，值得推荐。

dbaplus 社群发起人, 腾讯云 TVP, 《Oracle/MySQL 工作笔记》作者·杨建荣

很高兴大智小技第四部又要面世了。我经常阅读社区分享的文章，各位作者的分享也是相当精彩，同时我也深知研究一个问题的不易。而在本作中包含了大量的知识点分享，也是各位作者努力的结晶。相信打开本作，你一定会和我一样，收获颇丰。

《MySQL 主从原理》作者·高鹏(八怪)

从 21 年开始接触爱可生云树系列产品，让人眼前一亮，非常符合当下开源数据库管理模式。又进一步接触了爱可生开源社区，如获至宝，《大智小计》系列书籍对 MySQL 爱好者来说是一个很好的学习工具，也是我向集团里每一位 MySQL 数据库管理员推荐的首选。

日日顺供应链数据库负责人·高帅

前言

《大智小技》已经到了第四期，今年社区又收到了很多高价值的内容；这些内容我们通过社区公众号等形式发布出去后，也收到了很多的讨论点评和指正；为了更高效高质量的将内容送给我们的粉丝朋友，社区重新精选了内容，整理成册，将 2022 年的精华重新编排给大家。希望能变成大家日常工作和闲暇之余，随手可得的一本案头册；

今年的内容作者依旧是来自爱可生公司内部产研团队、DBA 团队、项目团队、知名互联网公司的专家、大隐于世的技术高手。内容方面也涵盖了以 MySQL 为主的特性、故障处理、技术原理解析，同时还包括了爱可生开源社区主导的各类开源产品，如分布式中间件 DBLE、数据传输组件 DTLE、SQL 质量管理平台 SQLE 等，另外还囊括了一些主流的开源数据库的讨论，比如：TiDB、Oceanbase、MongoDB、ClickHouse 等。

新的一年，如果您有更好的内容和建议，希望能一起加入与我们共同前行。

爱可生开源社区

2022 年 12 月

致 谢

2022 年 DBLE 和 DTLE、SQLE 等开源软件持续更新，社区共发布各类技术文章 200 余篇，与多家社区合作组织线上线下活动 20 余场，社区用户和文章阅读量持续增长，社区讨论愈发活跃。

感谢每一位内容创作者和每一位 commiter；感谢一起合作的社区兄弟；更要感谢一直关注和支持社区发展的朋友们。

希望越来越多的技术爱好者与社区一起见证 MySQL 生态的蓬勃发展。

此致敬礼！

爱可生团队

开源产品团队

DBA 技术团队

数据库产品研发团队

合作社区

3306 π 社区

IMG 社区

DBAplus

TiDB 社区

*** 致谢名单排名不分先后**

社区作者

许天云 / 国泰新点

刘晨 / 中信建投证券

莫善 / 转转

陈旭 / 捷信金融（中国）

陈俊聪 / 中移信息

范博施 / 云账户

李锡超 / 江苏苏宁银行

杨奇龙 / 阿里云

付祥 / 金山软件

任坤 / 金山软件

杨家鑫 / 多点 Dmall

林浩 / 某国有银行

邓润钦 / 恒峰信息

刘新旺 / 宠胖胖

卢永旺 / 腾讯音乐

目 录

推 荐 序	5
前 言	9
致 谢	10

第 1 章 MySQL 8.0 新特性

1.1 MySQL 8.0 新密码策略（上）	1
1.2 MySQL 8.0 新密码策略（中）	4
1.3 MySQL 8.0 新密码策略（下）	8
1.4 MySQL 8.0 新密码策略（终篇）	11
1.5 MySQL 8.0: explain analyze 分析 SQL 执行过程	14
1.6 MySQL 8.0: 死锁日志改进	17
1.7 MySQL 8.0 对 UNION 的改进	19
1.8 MySQL 8.0.28 用户连接内存限制	25
1.9 MySQL 8.0 多因素身份认证	26
1.10 MySQL 8.0 窗口函数一次疑问解答	31
1.11 MySQL 8.0 对 GTID 的限制解除	34
1.12 MySQL 8.0 GIPK 不可见主键	41
1.13 MySQL 8.0 在线调整 REDO	49
1.14 MySQL 8.0 的交集和差集介绍	51
1.15 MySQL 8.0 对 limit 的优化	54

第 2 章 技术分享

2.1 MySQL 的 TIMESTAMP 类型字段非空和默认值属性的影响	57
2.2 MySQL: 你的 binlog_expire_logs_seconds 可能正在失效	59
2.3 MySQL 索引长度限制的案例	63
2.4 MySQL: 一文看懂时区&time_zone	65
2.5 ProxySQL 搭配 MySQL HA (上)	69
2.6 ProxySQL 搭配 MySQL HA (下)	76
2.7 MySQL Binlog 通过 MySQL 客户端导入数据库效率低的原因	82
2.8 mysqlbinlog 技巧	86
2.9 MySQL 数据库巡检需要做些什么?	89
2.10 MySQL 在线开启 GTID 的每个阶段是要做什么	94
2.11 何时需要手动刷新授权表	100
2.12 MySQL 字符集再探	103
2.13 MySQL 并行 DDL	107
2.14 聊聊 MySQL 关机的故事	111
2.15 MySQL host-cache	118
2.16 你通常选用什么工具在 Linux 中进行数据传输?	121
2.17 MySQL 审计功能实现方案	130
2.18 dbslower 工具学习之探针使用	142
2.19 MySQL 编写脚本时避免烦人的警告	146
2.20 MySQL 中一个聚类增量统计 SQL 的需求	149
2.21 MySQL: caching_sha2_password 快速问答	151
2.22 MySQL: 从库复制半个事务会怎么样?	154

2.23 undo 太大了怎么办	156
2.24 MySQL: max_allowed_packet 影响了什么?	160
2.25 大事务阻塞 show master status	163
2.26 需要小心配置的 gtid_mode	166
2.27 mysql 客户端对配置文件的读取顺序	170
2.28 MySQL 存储过程中的只读语句超时怎么办?	174
2.29 无损半同步复制下, 主从高可用切换后数据一致吗?	176
2.30 MemAvailable 是怎么计算的	185
2.31 基于 Alertmanager 告警系统的改造	187
2.32 MySQL--测试--timestamp 字段默认值测试	197
2.33 MySQL--测试--排序规则 collation	201

第 3 章 故障分析

3.1 undo log 长时间处于清理状态导致备份失败	205
3.2 MySQL 数据”丢失”事件之 binlog 解析应用一则	207
3.3 MySQL 设置 terminology_use_previous 参数导致数据库 Crash	211
3.4 MySQL: 唯一键约束失效	216
3.5 MySQL: 从机故障重启后主从同步报错案例分析	220
3.6 Federated 存储引擎表导致监控线程处于 Opening	226
3.7 MySQL 5.7 使用临时表导致数据库 Crash	228
3.8 MySQL Server 端为什么有大量的 TCP TIME_WAIT ?	236
3.9 大量短时进程导致 cpu 负载过高案例一则	240
3.10 DDL 导致的 Xtrabackup 备份失败	243
3.11 从 data_free 异常说起	256

3.12 show processlist 引起的性能问题	261
3.13 MySQL 耗尽主机内存一例分析	264
3.14 一条 SELECT 语句跑崩了 MySQL ，怎么回事？	267
3.15 Sql_slave_skip_counter 使用不规范对复制的影响	272
3.16 xtrabackup 多表备份报错 “ too many open files ”	279
3.17 密码使用特殊字符	284
3.18 命令行登录 MySQL 报 Segmentation fault 故障解决	287
3.19 MySQL 启动遭遇 Permission denied 失败案例一则	295
3.20 MySQL 节点宕机分析一例	297
3.21 从一则 MGR 异常切换案例，看系统时间对 MGR 的影响	302

第 4 章 开源产品

4.1 MySQL 开源许可研究	308
4.2 SQLE-SQL 审核平台体验报告	311
4.3 SQLE 如何开发一条自定义的规则	315
4.4 如何用 SQLE 自动审核 Java 应用	323
4.5 如何使用 SQLE 进行开发阶段 SQL 审核	326
4.6 如何快速使用 SQLE 审核各种类型的数据库	331
4.7 如何利用 OpenAPI 实现自己的扫描任务	339
4.8 DBLE 监控告警组件	344
4.9 log4j2 漏洞修复方案	347
4.10 中间件是如何处理 Prepare Statement 和游标的	349
4.11 动态调整 DBLE 内线程池的数目	353
4.12 BenchmarkSQL 压测 dble 性能调优指南	357

4.13 数据库连接如何正确处理 TCP 连接三次握手失败	361
4.14 如何与 DBLE 进行“秘密通话”	367
4.15 几步快速拥有读写分离	373
4.16 浅谈 dble 引入 ClickHouse 的配置操作	376
4.17 如何通过 dble 的 split 功能，快速地将数据导入到 dble 中	385
4.18 实战：将业务从 MyCAT 平滑迁移到 dble	392
4.19 dble 启动的时候做了什么之配置检测	395
4.20 利用 TiDB DM 将数据从 MySQL 迁移至 DBLE	402
4.21 从 dble 日志分析到 MySQL 源码学习	414
4.22 Left join ... on shardingColumn = N 优化为整体下发单节点的可行性	419
4.23 如何使用 DTLE 将 Oracle 数据同步到 MySQL	423
4.24 利用 DTLE 将 MySQL 数据同步到 DBLE	433

第 5 章 开源数据库产品

5.1 ClickHouse 集群分布式 DDL 被阻塞案例一则	445
5.2 ClickHouse-Keeper 初探	447
5.3 ClickHouse 物化视图插入时间变为“1970-01-01 08:00:00”问题复盘	450
5.4 Zabbix 监控 TiDB（一）	457
5.5 Zabbix 监控 TiDB（二）	460
5.6 TiDB 上百 T 数据拆分实践	468
5.7 MongoDB 5.0 报错 Illegal instruction 解决	476
5.8 mlaunch-MongoDB 测试环境快速构建工具	480
5.9 MongoDB Sharding QPS 分布不均案例一则	484
5.10 OceanBase 错误日志分析	486

5.11 bgsave 导致 redis 定期卡顿案例一则	492
5.12 Redis Cluster 分片内存异常使用不均问题诊断	494

第 1 章 MySQL 8.0 新特性

截止到定稿，2022 年 MySQL 8.0 已经发版 4 次，最新版本为 8.0.31。今年主题共发表文章 12 篇。每篇的作者时刻关注 8.0 的最新动态，在版本发布后的第一时间测试功能并撰写技术文章，为读者介绍新功能的最新测评。

1.1 MySQL 8.0 新密码策略（上）

作者：杨涛涛

引言

这里来介绍下 MySQL 8.0 版本自带的新密码验证策略。

正文

我们非常熟悉这样的模式：用户想更改自己密码，需要提供原来密码或者追加手机验证码才可以，这种模式在 MySQL 数据库里一直不存在。在 MySQL 8.0 之前的版本，普通用户可以直接更改自己密码，不需要旧密码验证，也不需要知会管理员，比如用户 ytt_admin 需要更改密码，在 MySQL 5.7 下直接敲 alter user 命令即可：

```
root@ytt-ubuntu:~# mysql -uytt_admin -proot1234 -P5734 -h ytt-ubuntu -e "alter user
ytt_admin identified by 'root'"
mysql: [Warning] Using a password on the command line interface can be insecure.
```

这样的密码更改行为其实不是很安全，假设有下面的场景出现：

用户 ytt_admin 登录到 MySQL 服务后，做了些日常操作，完成后忘记退出；此时刚好有一个别有用心用户 ytt_fake 进入 ytt_admin 的登录环境，直接敲命令 alter user 即可更改用户 ytt_admin 的密码，并且退出当前登录环境，用户 ytt_admin 本尊再次登录 MySQL，就会提示密码错误，不允许登录，此时用户 ytt_admin 大脑肯定是懵的。

为了防止这类不安全事件的发生，MySQL 8.0 发布了一系列密码验证策略。这里介绍第一项：当前密码验证策略设置！

当前密码验证策略有两种方法来给到具体用户。

第一种，从管理员侧来设置单个用户的当前密码验证策略。

创建用户或者更改用户设置时使用子句：password require current（表示强制此用户满足当前密码验证策

略）。

```
mysql:(none)>create user ytt_admin identified by 'root123' password require current;
Query OK, 0 rows affected (0.11 sec)
```

之后以用户 `ytt_admin` 登录 MySQL 并且更改密码，提示需要提供旧密码才行。

```
root@ytt-ubuntu:/home/ytt# mysql -h ytt-ubuntu -uytt_admin -proot123
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 33
Server version: 8.0.27 MySQL Community Server - GPL

mysql:(none)>alter user ytt_admin identified by 'root';
ERROR 3892 (HY000): Current password needs to be specified in the REPLACE clause in
order to change it.
```

接下来，`alter user` 跟上子句 `replace` 来让用户 `ytt_admin` 输入旧密码，成功更改新密码。

```
mysql:(none)>alter user ytt_admin identified by 'root' replace 'root123';
Query OK, 0 rows affected (0.00 sec)
```

如果有的场景下需要保持 MySQL 旧版本的密码更改行为，管理员侧可以用子句：`password require current optional` 关闭新特性。

```
-- (optional 关键词可用 default 替代，参考全局密码验证参数设置)
mysql:(none)>alter user ytt_admin password require current optional;
Query OK, 0 rows affected (0.04 sec)
```

来再次验证下用户 `ytt_admin` 更改密码的行为：保持 MySQL 8.0 之前的的规则，不对该用户密码做校验。

```
mysql:(none)>alter user ytt_admin identified by 'root';
Query OK, 0 rows affected (0.01 sec)
```

第二种，设置全局参数，来强制所有用户使用当前密码验证策略。

MySQL 8.0 新版本内置的参数 `password_require_current` 定义一个全局密码策略，默认关闭。开启这个选项时，要求用户更改密码时必须提供旧密码。

开启全局参数：

```
mysql:(none)>set persist password_require_current=on;
Query OK, 0 rows affected (0.00 sec)
```

创建另外一个新用户 `ytt_usage`：

```
mysql:(none)>create user ytt_usage identified by 'root123';
Query OK, 0 rows affected (0.00 sec)
```

以用户 `ytt_usage` 登录 MySQL 更改自己密码：直接拒绝更改，需要提供旧密码。

```
root@ytt-ubuntu:~# mysql -uytt_usage -proot123 -h ytt-ubuntu
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 37
Server version: 8.0.27 MySQL Community Server - GPL

...

mysql:(none)>alter user ytt_usage identified by 'root';
ERROR 3892 (HY000): Current password needs to be specified in the REPLACE clause in
order to change it.
mysql:(none)>
```

replace 子句提供旧密码再次成功更改新密码:

```
mysql:(none)>alter user ytt_usage identified by 'root' replace 'root123';
Query OK, 0 rows affected (0.02 sec)
```

这里有一个需要注意的点: 虽然全局参数开启, 但是 alter user 命令优先级更高, 可以直接覆盖全局参数设置。下面是全局参数开启的环境下, 用 alter user 命令来关闭用户 ytt_usage 的当前密码验证策略。

```
mysql:(none)>alter user ytt_usage password require current optional;
Query OK, 0 rows affected (0.11 sec)
```

接下来用户 ytt_usage 又恢复为 MySQL 旧版本的安全行为:

```
mysql:(none)>alter user ytt_usage identified by 'rootnew';
Query OK, 0 rows affected (0.11 sec)
```

还有另外一个子句: password require current default , 具体行为由全局参数 password_require_current 的设置决定, 全局参数关闭, 这个子句恢复 MySQL 旧版本安全行为; 全局参数开启, 这个子句使用 MySQL 新版本安全行为。

```
mysql:(none)>alter user ytt_usage password require current default;
Query OK, 0 rows affected (0.09 sec)
```

总结:

本文介绍的当前密码验证策略, 使得 MySQL 朝着更加安全的方向努力。

1.2 MySQL 8.0 新密码策略（中）

作者：杨涛涛

本篇继续介绍 MySQL 8.0 的新密码验证策略。

假设有这样的需求：管理员创建了两个用户，职责分别是开发和运维，并且要求这两个用户必须满足如下需求：

1. 开发用户要求定期更改密码，并且密码不能与近期更改过的密码重叠，也即不能复用历史密码，这里限定历史密码个数为 3；
2. 运维用户同样要求定期更改密码，并且密码不能与某段时间内更改过的密码重叠，也即同样不能复用历史密码，这里限定时间段为一个礼拜。

以上两种改密码需求，在数据库侧暂时无法实现，只能拿个“小本子记住历史密码保留个数、历史密码保留天数”，在用户每次更改密码前，先检测小本子上有没有和新密码重叠的历史密码。MySQL 8.0 对以上这两种改密码需求，直接从数据库端实现，用户可以扔掉“小本子”了。我来分两部分讲解在 MySQL 8.0 版本里对以上改密码需求的具体实现。

第一，在配置文件里写上全局参数

参数 `password_history` 表示最近使用的密码保留次数；参数 `password_reuse_interval` 表示最近使用的密码保留天数。

先来实现开发用户的需求：保留历史密码个数为 3。

管理员用户登录，设置全局参数：

```
mysql:(none)>set persist password_history=3;
Query OK, 0 rows affected (0.00 sec)
```

退出重连，创建用户 `ytt_dev`：

```
root@ytt-ubuntu:/home/ytt# mysql -S /opt/mysql/mysqld.sock
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 33
Server version: 8.0.27 MySQL Community Server - GPL

...

mysql:(none)>create user ytt_dev identified by 'root123';
Query OK, 0 rows affected (0.15 sec)
```

退出连接，用户 `ytt_dev` 重新连接数据库，并且更改两次密码：


```

root@ytt-ubuntu:/home/ytt# mysql -uytt_dev -hytt-ubuntu -proot123
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 34
Server version: 8.0.27 MySQL Community Server - GPL

...

mysql:(none)>
mysql:(none)>alter user ytt_dev identified by 'root456';
Query OK, 0 rows affected (0.03 sec)

mysql:(none)>alter user ytt_dev identified by 'root789';
Query OK, 0 rows affected (0.17 sec)

```

加上原始密码，也就是 3 次密码，再来更改一次密码，此时不允许更改密码，错误提示和密码历史策略冲突：

```

mysql:(none)>alter user ytt_dev identified by 'root123';
ERROR 3638 (HY000): Cannot use these credentials for 'ytt_dev@%' because they
contradict the password history policy

```

接下来，选择一个与历史密码不冲突的新密码进行修改，此时密码修改成功：

```

mysql:(none)>alter user ytt_dev identified by 'rootnew';
Query OK, 0 rows affected (0.04 sec)

```

再来实现运维用户的需求：保留密码天数为 7 天。

同样，管理员用户登录 MySQL，并且设置全局参数：

```

mysql:(none)>set persist password_reuse_interval = 7;
Query OK, 0 rows affected (0.00 sec)

mysql:(none)>set persist password_history=default;
Query OK, 0 rows affected (0.00 sec)

```

退出重新连接，创建运维用户 ytt_dba：

```

mysql:(none)>create user ytt_dba identified by 'root123';
Query OK, 0 rows affected (0.01 sec)

mysql:(none)>\q
Bye

```

以用户 ytt_dba 登录数据库，并且改了五次密码：

```

root@ytt-ubuntu:/home/ytt# mysql -uytt_dba -hytt-ubuntu -proot123
mysql: [Warning] Using a password on the command line interface can be insecure.

```

```
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 39
Server version: 8.0.27 MySQL Community Server - GPL

...

mysql:(none)>alter user ytt_dba identified by 'root456';
Query OK, 0 rows affected (0.15 sec)

mysql:(none)>alter user ytt_dba identified by 'root789';
Query OK, 0 rows affected (0.08 sec)

mysql:(none)>alter user ytt_dba identified by 'root000';
Query OK, 0 rows affected (0.02 sec)

mysql:(none)>alter user ytt_dba identified by 'root888';
Query OK, 0 rows affected (0.02 sec)

mysql:(none)>alter user ytt_dba identified by 'root999';
Query OK, 0 rows affected (0.12 sec)
```

接下来验证历史密码验证策略，由于我们设置了密码历史保留天数，任何在设定时间内的历史密码，均不能作为新密码使用：MySQL 拒绝用户更改密码，错误提示与密码历史策略冲突：

```
mysql:(none)>alter user ytt_dba identified by 'root123';
ERROR 3638 (HY000): Cannot use these credentials for 'ytt_dba@%' because they
contradict the password history policy
mysql:(none)>alter user ytt_dba identified by 'root456';
ERROR 3638 (HY000): Cannot use these credentials for 'ytt_dba@%' because they
contradict the password history policy
mysql:(none)>
```

选择一个非最近更改过的新密码，改密成功：

```
mysql:(none)>alter user ytt_dba identified by 'rootnew';
Query OK, 0 rows affected (0.10 sec)
```

如果有一个用户同时需要具备开发用户和运维用户的密码限制条件，可以把两个全局参数一起修改即可：历史密码保留天数为 7 天、同时历史密码保留个数为 3 次。

```
mysql:(none)>set persist password_reuse_interval = 7;
Query OK, 0 rows affected (0.00 sec)

mysql:(none)>set persist password_history=3;
Query OK, 0 rows affected (0.00 sec)
```

第二， 管理员在创建用户或者更改用户属性时可以对单个用户定义密码验证策略

把全局参数重置为默认，也即关闭密码验证策略：

```
mysql:(none)>set persist password_reuse_interval = default;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql:(none)>set persist password_history=default;
Query OK, 0 rows affected (0.00 sec)
```

管理员退出连接重新进入，创建两个用户 ytt_dev1 和 ytt_dba1：

```
mysql:(none)>create user ytt_dev1 identified by 'root123';
Query OK, 0 rows affected (0.04 sec)
```

```
mysql:(none)>create user ytt_dba1 identified by 'root123';
Query OK, 0 rows affected (0.02 sec)
```

更改两个用户的密码历史保留策略：

```
mysql:(none)>alter user ytt_dev1 password history 3;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql:(none)>alter user ytt_dba1 password reuse interval 7 day;
Query OK, 0 rows affected (0.02 sec)
```

检索 mysql.user 表，看看是否更改成功：

```
mysql:(none)>select user,password_reuse_history,password_reuse_time from mysql.user
where password_reuse_history is not null or password_reuse_time is not null;
+-----+-----+-----+
| user | password_reuse_history | password_reuse_time |
+-----+-----+-----+
| ytt_dba1 | NULL | 7 |
| ytt_dev1 | 3 | NULL |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

具体验证方法类似全局参数设置部分，此处省略。

总结：

MySQL 8.0 推出的历史密码验证策略是对用户密码安全机制的另外一个全新的改进，可以省去此类需求非数据库侧的繁琐实现。

1.3 MySQL 8.0 新密码策略（下）

作者：杨涛涛

今天我们来继续介绍 MySQL 8.0 的新密码策略，分别为双密码策略和内置随机密码生成。

第一，双密码策略：

首先来解释下什么是双密码策略？双密码策略就是在日常运维中，需要定期更改指定用户密码，同时又需要旧密码暂时保留一定时长的一种策略。其作用是延迟应用与数据库之间的用户新旧密码对接时间，进而平滑应用的操作感知。可以在如下场景中使用：

在 MySQL 数据库里我们部署最多也是最成熟的架构：一主多从。比如说此架构做了读写分离，主负责处理前端的写流量，读负责处理前端的读流量，为了安全起见，需要定期对应用连接数据库的用户更改密码。有了双密码机制，对用户密码的更改在应用端可以有一定的缓冲延迟，避免业务中断风险以及开发人员的抱怨。应用端依然可以使用旧密码来完成对数据库的检索，等待合适时机再使用管理员发来的新密码检索数据库。

双密码机制包含主密码与备密码，当备密码不再使用时，告知管理员丢弃备密码，此时用户的主密码即是唯一密码。

具体如何使用呢？用法如下：

管理员先创建一个新用户 ytt，密码是 root_old，完了更改他的密码为 root_new。此时 root_new 即为主密码，而 root_old 即为备密码。

```
mysql:(none)>create user ytt identified by 'root_old';
Query OK, 0 rows affected, 2 warnings (0.24 sec)

mysql:(none)>alter user ytt identified by 'root_new' retain current password;
Query OK, 0 rows affected (0.17 sec)
```

接下来用户 ytt 分别使用备密码与主密码连接 MySQL 并且执行一条简单的 SQL 语句：

备密码连接数据库：

```
root@ytt-ubuntu:/home/ytt# mysql -h ytt-ubuntu -P 3306 -uytt -proot_old -e "select
'hello world'"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| hello world |
+-----+
| hello world |
+-----+
```

主密码连接数据库：

```
root@ytt-ubuntu:/home/ytt# mysql -h ytt-ubuntu -P 3306 -uytt -proot_new -e "select
'hello world'"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| hello world |
+-----+
| hello world |
+-----+
```

可以发现在管理员没有丢弃旧密码前，两个密码都能正常使用。

相关业务更改完成后，即可告知管理员丢弃备密码：

```
root@ytt-ubuntu:/home/ytt# mysql -S /opt/mysql/mysql.sock
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 27
Server version: 8.0.27 MySQL Community Server - GPL

...

mysql:(none)>alter user ytt discard old password;
Query OK, 0 rows affected (0.02 sec)

mysql:(none)>\q
Bye
```

双密码策略有以下需要注意的事项：

1. 如果用户本身已经有双密码策略，再次更改新密码时没有带 `retain current password` 子句，那之前的主密码被替换成新改的密码，但是备密码不会被替换。比如更改新密码为 `root_new_new`，此时备密码依然是 `root_old`，并非之前的主密码 `root_new`。下面例子中输入密码 `root_old` 依然可以连接数据库，而输入密码 `root_new` 则被数据库拒绝连接：

```
mysql:(none)>alter user ytt identified by 'root_new_new';
Query OK, 0 rows affected (0.16 sec)

root@ytt-ubuntu:/home/ytt# mysql -h ytt-ubuntu -u ytt -proot_old -e "select 'hello
world'"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| hello world |
+-----+
| hello world |
+-----+

root@ytt-ubuntu:/home/ytt# mysql -h ytt-ubuntu -u ytt -proot_new -e "select 'hello
```

```
world'"
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 1045 (28000): Access denied for user 'ytt'@'ytt-ubuntu' (using password: YES)
```

还有一点需要注意的细节，如果不带 `retain current password` 子句，并且更改新密码为空串，那么主备密码则会统一更改为空串。下面例子中数据库拒绝之前的备密码连接：

```
mysql:(none)>alter user ytt identified by '';
Query OK, 0 rows affected (0.80 sec)

root@ytt-ubuntu:/home/ytt# mysql -h ytt-ubuntu -u ytt -proot_old -e "select 'hello world'"
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 1045 (28000): Access denied for user 'ytt'@'ytt-ubuntu' (using password: YES)
root@ytt-ubuntu:/home/ytt# mysql -h ytt-ubuntu -u ytt -e "select 'hello world'"
+-----+
| hello world |
+-----+
| hello world |
+-----+
```

2. 新密码为空，不允许使用备用密码。

```
mysql:(none)>alter user ytt identified by '' retain current password;
ERROR 3895 (HY000): Current password can not be retained for user 'ytt'@'%' because new password is empty.
```

3. 使用双密码策略时，不能更改用户的认证插件。

```
mysql:(none)>alter user ytt identified with sha256_password by 'root_new' retain current password;
ERROR 3894 (HY000): Current password can not be retained for user 'ytt'@'%' because authentication plugin is being changed.
```

第二，随机密码生成：

以往旧版本有生成随机密码的需求，在 MySQL 端无法直接设定，除非封装用户密码设定逻辑，并且在代码里实现随机密码生成。比如用存储过程，脚本等等。

MySQL 8.0 直接可以设置用户随机密码

```
mysql:(none)>create user ytt_new identified by random password;
+-----+-----+-----+-----+
| user | host | generated password | auth_factor |
+-----+-----+-----+-----+
| ytt_new | % | >h<m3[bnigz%f/SnLfp | 1 |
```

```
+-----+-----+-----+-----+
1 row in set (0.02 sec)
```

也可以用 `set password` 子句来设置随机密码

```
mysql:(none)>set password for ytt_new to random;
+-----+-----+-----+-----+
| user | host | generated password | auth_factor |
+-----+-----+-----+-----+
| ytt_new | % | 5wzZ+0[27cd_CW/]<ua, | 1 |
+-----+-----+-----+-----+
1 row in set (0.04 sec)
```

另外，随机密码的长度由参数 `generated_random_password_length` 调整，默认为 20 个。

总结

双密码策略能让应用和 DBA 沟通起来更加协调；随机密码设置能让数据库系统更加安全。

1.4 MySQL 8.0 新密码策略（终篇）

作者：杨涛涛

我们时常会遇到的场景：用银行卡在 ATM 机取款、在 APP 上转账、网购付款等等环节，因密码连续输错一定的次数，银行卡即被锁定而无法使用，除非拿着有效证件去银行柜台人工解锁才可正常使用。

随着 MySQL 数据库被越来越多的金融场景使用，类似连续输错银行卡密码而导致的锁卡功能呼之欲出。MySQL 从 8.0.19 开始，就推出了类似策略：Failed-Login Tracking and Temporary Account Locking。翻译过来就是 失败登录追踪和临时密码锁定，后面我们简称为：FLTTAL。

和之前几个密码策略不同，FLTTAL 没有全局参数匹配，只能在创建用户或者是更改用户属性时被匹配。有两个选项：

1. `FAILED_LOGIN_ATTEMPTS N`：代表密码失败重试次数。
2. `PASSWORD_LOCK_TIME N | UNBOUNDED`：代表密码连续“`FAILED_LOGIN_ATTEMPTS`”次验证失败后被锁定的天数

FLTTAL 有以下几个需要注意的点：

1. `failed_login_attempts` 和 `password_lock_time` 必须同时不为 0，FLTTAL 才能生效。
2. 创建新用户不指定 `failed_login_attempts` 和 `password_lock_time`，则默认关闭 FLTTAL。

3. 已使用 FLTTAL 的用户，管理员对其 `alter user` 后不改变原有密码验证策略。
4. 一旦账户被锁定，即使输入正确密码也无法登录。
5. 还有最重要的一点：由于 FLTTAL 对密码验证正确与否的连续性，任意一次成功登录，FLTTAL 计数器重置。例如 `failed_login_attempts` 设置为 3，前两次密码连续输错，第三次输入正确的密码，FLTTAL 计数器重置。

那接下来我们来看下如何具体使用这个密码验证策略：

1. 对于普通用户的使用方法：

管理员创建用户 `test1@'localhost'`，并且设置 FLTTAL 策略：失败重试次数为 3，密码锁定时间为 3 天。

```
mysql:(none)>create user test1@'localhost' identified by 'test' failed_login_attempts 3
password_lock_time 3;
Query OK, 0 rows affected (0.14 sec)
```

密码连续输错 3 次，`test1@'localhost'` 账号被锁定：

```
root@ytt-ubuntu:/home/ytt# mysql -utest1 -p -S /opt/mysql/mysqld.sock
Enter password:
ERROR 1045 (28000): Access denied for user 'test1'@'localhost' (using password: NO)
root@ytt-ubuntu:/home/ytt# mysql -utest1 -p -S /opt/mysql/mysqld.sock
Enter password:
ERROR 1045 (28000): Access denied for user 'test1'@'localhost' (using password: NO)
root@ytt-ubuntu:/home/ytt# mysql -utest1 -p -S /opt/mysql/mysqld.sock
Enter password:
ERROR 3955 (HY000): Access denied for user 'test1'@'localhost'. Account is blocked for
3 day(s) (3 day(s) remaining) due to 3 consecutive failed logins.
```

管理员解锁账户方能正常使用：（或者忘记密码，让管理员解锁账号并且重置新密码。）

```
mysql:(none)>alter user test1@'localhost' account unlock;
Query OK, 0 rows affected (0.00 sec)
```

用正确密码再次登录：登录成功。

```
root@ytt-ubuntu:/home/ytt# mysql -utest1 -p -S /opt/mysql/mysqld.sock -e "select
'hello world\!'"
Enter password:
+-----+
| hello world! |
+-----+
| hello world! |
+-----+
```

2. 对于代理用户的使用方法：

对于代理用户来讲，FLTTAL 只影响代理用户本身，并不影响隐藏的真实用户。

代理用户介绍详见我之前的文章：MySQL 8.0 代理用户使用

之前创建的代理用户：

```
mysql:(none)>show grants for ytt_fake;
+-----+
| Grants for ytt_fake@% |
+-----+
| GRANT USAGE ON *.* TO `ytt_fake`@`%` |
| GRANT PROXY ON `ytt_real`@`%` TO `ytt_fake`@`%` |
+-----+
2 rows in set (0.00 sec)
```

把真实用户插件改为 `mysql_native_password` 让其可以正常登录：

```
mysql:(none)>alter user ytt_real identified with mysql_native_password;
Query OK, 0 rows affected (0.10 sec)
```

给代理用户 `ytt_fake` 设定 `FLTTAL` 策略：失败重试次数为 2，密码锁定时间为 7 天。

```
mysql:(none)>alter user ytt_fake failed_login_attempts 2 password_lock_time 7;
Query OK, 0 rows affected (0.14 sec)
```

代理用户连续输错两次密码，账号被锁住：

```
root@ytt-ubuntu:/home/ytt# mysql -u ytt_fake -p -hytt-ubuntu
Enter password:
ERROR 1045 (28000): Access denied for user 'ytt_fake'@'ytt-ubuntu' (using password:
YES)
root@ytt-ubuntu:/home/ytt# mysql -u ytt_fake -p -hytt-ubuntu
Enter password:
ERROR 3955 (HY000): Access denied for user 'ytt_fake'@'ytt-ubuntu'. Account is blocked
for 7 day(s) (7 day(s) remaining) due to 2 consecutive failed logins.
```

使用真实用户登录，不受代理用户影响：真实用户可以正常登录。

```
root@ytt-ubuntu:/home/ytt# mysql -u ytt_real -p -hytt-ubuntu -e "select 'hello
world\!'"
Enter password:
+-----+
| hello world! |
+-----+
| hello world! |
+-----+
```

用户账号被锁定并且禁止登录后，除了管理员通过手动解锁重置计数器外，还可以有以下几种方法重置计数器：

- (1) MySQLD 服务重启。

- (2) 执行 FLUSH PRIVILEGES，对用户权限数据刷盘。
- (3) 一次成功的账户登录。
- (4) 锁定时间过期。例如锁定时间为 7 天，7 天内管理员没做任何处理，FLTTAL 计数器重置。
- (5) 管理员重新更改 failed_login_attempts 或者 password_lock_time 选项，FLTTAL 计数器重置。

总结：

这里讲解了 MySQL 8.0 的失败登录追踪和临时密码锁定策略，结合之前介绍过的其他密码验证策略一起使用，可以弥补 MySQL 数据库在这块领域的不足。

1.5 MySQL 8.0: explain analyze 分析 SQL 执行过程

作者：胡呈清

介绍

MySQL 8.0.16 引入一个实验特性：explain format=tree，树状的输出执行过程，以及预估成本和预估返回行数。在 MySQL 8.0.18 又引入了 EXPLAIN ANALYZE，在 format=tree 基础上，使用时，会执行 SQL，并输出迭代器（感觉这里用“算子”更容易理解）相关的实际信息，比如执行成本、返回行数、执行时间，循环次数。

文档链接：<https://dev.mysql.com/doc/refman/8.0/en/explain.html#explain-analyze>

示例：

```
mysql> explain format=tree SELECT * FROM t1 WHERE t1.a IN (SELECT t2.b FROM t2 WHERE id < 10);
***** 1. row *****
-> Nested loop inner join (cost=4.95 rows=9)
-> Filter: (`<subquery2>`.b is not null) (cost=2.83..1.80 rows=9)
-> Table scan on <subquery2> (cost=0.29..2.61 rows=9)
-> Materialize with deduplication (cost=3.25..5.58 rows=9)
-> Filter: (t2.b is not null) (cost=2.06 rows=9)
-> Filter: (t2.id < 10) (cost=2.06 rows=9)
-> Index range scan on t2 using PRIMARY (cost=2.06 rows=9)
-> Index lookup on t1 using a (a=`<subquery2>`.b) (cost=2.35 rows=1)
1 row in set (0.01 sec)
```

```
mysql> explain analyze SELECT * FROM t1 WHERE t1.a IN (SELECT t2.b FROM t2 WHERE id < 10)\G
***** 1. row *****
-> Nested loop inner join (cost=4.95 rows=9) (actual time=0.153..0.200 rows=9 loops=1)
-> Filter: (`<subquery2>`.b is not null) (cost=2.83..1.80 rows=9) (actual time=0.097..0.100 rows=9 loops=1)
-> Table scan on <subquery2> (cost=0.29..2.61 rows=9) (actual time=0.001..0.002 rows=9 loops=1)
-> Materialize with deduplication (cost=3.25..5.58 rows=9) (actual time=0.090..0.092 rows=9 loops=1)
-> Filter: (t2.b is not null) (cost=2.06 rows=9) (actual time=0.037..0.042 rows=9 loops=1)
-> Filter: (t2.id < 10) (cost=2.06 rows=9) (actual time=0.036..0.040 rows=9 loops=1)
-> Index range scan on t2 using PRIMARY (cost=2.06 rows=9) (actual time=0.035..0.038 rows=9 loops=1)
-> Index lookup on t1 using a (a=`<subquery2>`.b) (cost=2.35 rows=1) (actual time=0.010..0.010 rows=1 loops=9)
1 row in set (0.01 sec)
```

可以看出 `explain format=tree` 与传统的执行计划相比，展示了比较清晰的执行过程。而 `explain analyze` 则会在此基础上多输出实际的执行时间、返回行数和循环次数。

阅读顺序

1. 从右到左：没有遇到并列的迭代器之前，都是从右边开始执行；
2. 从上到下：遇到并列的迭代器，都是上边的先开始执行

上述示例阅读顺序如下图（注意最好不要\G 输出，否则第一行的缩进不准确），SQL 的执行顺序为：

1. 使用 Nested loop inner join 算法；
2. t2 先取数据（Index range scan）、筛选（Filter）、物化成临时表（Materialize），作为驱动表；
3. 将驱动表数据带入到 t1 进行查询（Index lookup on t1），循环执行 9 次。

```
mysql> explain analyze SELECT * FROM t1 WHERE t1.a IN (SELECT t2.b FROM t2 WHERE id < 10);
+-----+
| EXPLAIN
+-----+
|
+-----+
| -> Nested loop inner join (actual time=0.255..0.496 rows=9 loops=1)
|   -> Filter: (<subquery2>`.b is not null) (actual time=0.182..0.246 rows=9 loops=1)
|     -> Table scan on <subquery2> (actual time=0.003..0.018 rows=9 loops=1)
|       -> Materialize with deduplication (actual time=0.178..0.217 rows=9 loops=1)
|         -> Filter: (t2.b is not null) (cost=2.06 rows=9) (actual time=0.087..0.136 rows=9 loops=1)
|           -> Filter: (t2.id < 10) (cost=2.06 rows=9) (actual time=0.080..0.112 rows=9 loops=1)
|             -> Index range scan on t2 using PRIMARY (cost=2.06 rows=9) (actual time=0.068..0.082 rows=9 loops=1)
|               -> Index lookup on t1 using a (a=<subquery2>`.b) (cost=2.35 rows=1) (actual time=0.015..0.019 rows=1 loops=9)
|
+-----+
1 row in set (0.01 sec)
```



重要信息

以下面为例：

```
Index lookup on t1 using a (a=<subquery2>`.b) (cost=2.35 rows=1) (actual
time=0.015..0.017 rows=1 loops=9)
```

cost

预估的成本信息，计算比较复杂。如果想了解，可以查看：[explain format=json 详解](#)

rows

第一个 rows 是预估值，第二个 rows 是实际返回行数。

actual time

“0.015..0.017”，注意这里有两个值，第一个值是获取第一行的实际时间，第二个值获取所有行的时间，如果循环了多次就是平均时间，单位毫秒。

loops

因为这里使用了 Nested loop inner join 算法，按照阅读顺序，t2 是驱动表，先进行查询被物化成临时表；t1 表做为被驱动表，循环查询的次数是 9 次，即 loops=9。

1.6 MySQL 8.0: 死锁日志改进

作者: 胡呈清

重要改进

MySQL8.0 的死锁日志可以看到事务 1 持有的锁信息了:

```

LATEST DETECTED DEADLOCK
-----
2021-12-16 10:51:01 0xf3529ceb700
*** (1) TRANSACTION:
TRANSACTION 15851575, ACTIVE 26 sec starting index read
mysql tables in use 1, locked 1
LOCK WAIT 4 lock struct(s), heap size 1136, 3 row lock(s)
MySQL thread id 111171, OS thread handle 139865592493824, query id 60758724 localhost root statistics
select * from t2 where c2=20 for update

*** (1) HOLDS THE LOCK(S):
RECORD LOCKS space id 1439 page no 5 n bits 72 index c2 of table 'db1'.'t2' trx id 15851575 lock_mode X locks rec but not gap
Record lock, heap no 3 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
 0: len 4; hex 80000004; asc ;;
 1: len 4; hex 80000005; asc ;;

*** (1) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 1439 page no 5 n bits 72 index c2 of table 'db1'.'t2' trx id 15851575 lock_mode X locks rec but not gap waiting
Record lock, heap no 4 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
 0: len 4; hex 80000004; asc ;;
 1: len 4; hex 80000014; asc ;;

*** (2) TRANSACTION:
TRANSACTION 15851628, ACTIVE 8 sec starting index read
mysql tables in use 1, locked 1
LOCK WAIT 4 lock struct(s), heap size 1136, 3 row lock(s)
MySQL thread id 111172, OS thread handle 139865597572864, query id 60758738 localhost root statistics
select * from t2 where c2=4 for update

*** (2) HOLDS THE LOCK(S):
RECORD LOCKS space id 1439 page no 5 n bits 72 index c2 of table 'db1'.'t2' trx id 15851628 lock_mode X locks rec but not gap
Record lock, heap no 4 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
 0: len 4; hex 80000014; asc ;;
 1: len 4; hex 80000014; asc ;;

*** (2) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 1439 page no 5 n bits 72 index c2 of table 'db1'.'t2' trx id 15851628 lock_mode X locks rec but not gap waiting
Record lock, heap no 3 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
 0: len 4; hex 80000004; asc ;;
 1: len 4; hex 80000005; asc ;;

*** WE ROLL BACK TRANSACTION (2)

```

这对我们分析死锁无疑是个很好的帮助, 而在 MySQL5.7 是没有这个信息的, 一直饱受诟病:

```

2020-05-08T11:22:00.153486+08:00 8 [Note] InnoDB: Transactions deadlocks detected, dumping detailed information.
2020-05-08T11:22:00.153560+08:00 8 [Note] InnoDB:
*** (1) TRANSACTION: 事务1的信息
TRANSACTION 9626313, ACTIVE 9 sec inserting
mysql tables in use 1, locked 1
LOCK WAIT 3 lock struct(s), heap size 1136, 3 row lock(s), undo log entries 1
MySQL thread id 9, OS thread handle 140714067699456, query id 176 localhost root update
insert into t3(c2) values(15); 正在执行的SQL
2020-05-08T11:22:00.153705+08:00 8 [Note] InnoDB: *** (1) WAITING FOR THIS LOCK TO BE GRANTED: 等待获取的锁信息
哪个页
RECORD LOCKS space id 301 page no 4 n bits 72 index c2 of table 'hucq'.'t3' trx id 9626313 lock_mode X locks gap before rec insert intention waiting
Record lock, heap no 4 PHYSICAL RECORD: n_fields 2; compact format; info bits 0 哪个表的哪个索引 锁的模式和属性
 0: len 4; hex 80000014; asc ;;
 1: len 4; hex 80000014; asc ;;

2020-05-08T11:22:00.153848+08:00 8 [Note] InnoDB: *** (2) TRANSACTION: 下面是事务2的信息
TRANSACTION 9626312, ACTIVE 18 sec inserting
mysql tables in use 1, locked 1
3 lock struct(s), heap size 1136, 4 row lock(s), undo log entries 1
MySQL thread id 8, OS thread handle 140714260440832, query id 177 localhost root update
insert into t3(c2) values(15)
2020-05-08T11:22:00.153888+08:00 8 [Note] InnoDB: *** (2) HOLDS THE LOCK(S): 事务2持有的锁
RECORD LOCKS space id 301 page no 4 n bits 72 index c2 of table 'hucq'.'t3' trx id 9626312 lock mode S
Record lock, heap no 3 PHYSICAL RECORD: n_fields 2; compact format; info bits 32
 0: len 4; hex 8000000F; asc ;;
 1: len 4; hex 8000000F; asc ;;

Record lock, heap no 4 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
 0: len 4; hex 80000014; asc ;;
 1: len 4; hex 80000014; asc ;;

2020-05-08T11:22:00.154104+08:00 8 [Note] InnoDB: *** (2) WAITING FOR THIS LOCK TO BE GRANTED: 事务2等待获取的锁
RECORD LOCKS space id 301 page no 4 n bits 72 index c2 of table 'hucq'.'t3' trx id 9626312 lock_mode X locks gap before rec insert intention waiting
Record lock, heap no 4 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
 0: len 4; hex 80000014; asc ;;
 1: len 4; hex 80000014; asc ;;

2020-05-08T11:22:00.154211+08:00 8 [Note] InnoDB: *** WE ROLL BACK TRANSACTION (2) 回滚了哪个事务

```

注意事项

但是这在某些情况下可能会产生一些误会，比如事务 1 持有锁和等待锁是同一个锁：

```
LATEST DETECTED DEADLOCK
-----
2021-12-16 10:13:15 0x7f3529ceb700
*** (1) TRANSACTION:
TRANSACTION 15844476, ACTIVE 4 sec inserting
mysql tables in use 1, locked 1
LOCK WAIT 2 lock struct(s), heap size 1136, 1 row lock(s), undo log entries 1
MySQL thread id 111122, OS thread handle 139866214790912, query id 60731430 localhost root update
insert into t2 values(30,10)

*** (1) HOLDS THE LOCK(S):
RECORD LOCKS space id 1439 page no 5 n bits 72 index c2 of table `db1`.`t2` trx id 15844476 lock mode S waiting
Record lock, heap no 5 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
 0: len 4; hex 8000000a; asc ;;
 1: len 4; hex 80000019; asc ;;

*** (1) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 1439 page no 5 n bits 72 index c2 of table `db1`.`t2` trx id 15844476 lock mode S waiting
Record lock, heap no 5 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
 0: len 4; hex 8000000a; asc ;;
 1: len 4; hex 80000019; asc ;;

*** (2) TRANSACTION:
TRANSACTION 15844440, ACTIVE 17 sec inserting
mysql tables in use 1, locked 1
LOCK WAIT 3 lock struct(s), heap size 1136, 2 row lock(s), undo log entries 2
MySQL thread id 111121, OS thread handle 139865598457600, query id 60731474 localhost root update
insert into t2 values(40,9)

*** (2) HOLDS THE LOCK(S):
RECORD LOCKS space id 1439 page no 5 n bits 72 index c2 of table `db1`.`t2` trx id 15844440 lock_mode X locks rec but not gap
Record lock, heap no 5 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
 0: len 4; hex 8000000a; asc ;;
 1: len 4; hex 80000019; asc ;;

*** (2) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 1439 page no 5 n bits 72 index c2 of table `db1`.`t2` trx id 15844440 lock_mode X locks gap before rec insert intention waiting
Record lock, heap no 5 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
 0: len 4; hex 8000000a; asc ;;
 1: len 4; hex 80000019; asc ;;

*** WE ROLL BACK TRANSACTION (1)
```

为什么会出现这种情况？这是不是 bug？

必须不是 bug，我们来复现这一死锁场景：

```
##设置 RC 隔离级别
CREATE TABLE `t2` (
  `c1` int(11) NOT NULL,
  `c2` int(11) DEFAULT NULL,
  PRIMARY KEY (`c1`),
  UNIQUE KEY `c2` (`c2`));
insert into t2 values(1,1),(5,4),(20,20);
```

session1	session2
	begin; insert into t2 values(25,10); (X LOCK but not gap)
begin; insert into t2 values(30,10); (S LOCK wait for X LOCK)	
	insert into t2 values(40,9); (X locks gap before rec insert intention waiting)
(Deadlock found when trying to get lock; try restarting transaction)	
事务1 insert 时发生唯一冲突，需要加S lock，带有gap属性 事务2 再次 insert 时，由于插入的位置有gap lock，所以出现 insert intention lock	

死锁逻辑:

1. session2 插入成功，对 c2 索引 10 这一记录加 X Lock，即死锁日志中的 lock_mode X locks rec but not gap;
2. session1 插入时，发生唯一键冲突，需要对 c2 索引 10 这一记录加 S Lock，带 gap 属性，即锁的范围为 (4,10]。但是由于 session2 已经对记录加了 X Lock，与 S Lock 互斥，所以必须等待 session 2 先释放锁，也就是死锁日志中的 lock mode S waiting;
3. session2 再次插入 9，在 (4,10] 范围内，这个位置有 session1 的 gap 锁（虽然还在锁队列中，没有加上），插入意向锁会被 gap 锁阻塞，即死锁日志中的 lock_mode X locks gap before rec insert intention waiting。session1、session2 互相等待，所以形成死锁。

session1 等待获取的锁 S Lock 阻塞了 session2 将要获取的锁，这在 MySQL8.0 中就会显示成 session1 持有的锁，同时也是 session1 等待的锁。就是这样。

1.7 MySQL 8.0 对 UNION 的改进

作者：张伟

一、UNION 的作用:

UNION 可以将多个 SELECT 查询语句的结果合并成一个结果集，在 MySQL 8.0 中又增添了一些新的功能，我们一起来看下。

二、UNION 操作符的花样展示:

1、温故

- Union 必须由两条或以上的 SELECT 语句组成，才能使用 Union 连接。
- Union 中每个查询必须包含相同数量的列或者聚合函数，列名或者别名需要一致。
- 列数据类型必须兼容，即可以进行隐式类型转换，但可能会出现 SQL 性能问题，建议数据类型相同。

举例如下：

```
create table t1 (id int, name varchar(20));
insert into t1 values(1,"爱可生"),(2,"开源"),(3,"社区"),(5,"MySQL"),(4,"张");
create table t2 (id int, name varchar(20));
insert into t2 values(3,"中国"),(11,"技术"),(15,"开源"),(2,"社区"),(1,"伟大");
```

```
mysql> select id from t1;
```

```
+-----+
```

```
| id |
```

```
+-----+
```

```
| 1 |
```

```
| 2 |
```

```
| 3 |
```

```
| 5 |
```

```
| 4 |
```

```
+-----+
```

```
5 rows in set (0.00 sec)
```

```
mysql> select id from t2;
```

```
+-----+
```

```
| id |
```

```
+-----+
```

```
| 3 |
```

```
| 11 |
```

```
| 15 |
```

```
| 2 |
```

```
| 1 |
```

```
+-----+
```

```
5 rows in set (0.00 sec)
```

// DISTINCT: 删除结果集中重复的数据。

```
mysql> select id from t1 union distinct select id from t2;
```

```
+-----+
```

```
| id |
```

```
+-----+
```

```
| 1 |
```

```
| 2 |
```

```
| 3 |
```

```
| 5 |
```

```
| 4 |
```

```
| 11 |
```

```
| 15 |
```

```
+-----+
```

```
7 rows in set (0.00 sec)
```


// 默认情况下 UNION 操作符已经删除了重复数据，所以 DISTINCT 修饰符对结果没啥影响。

```
mysql> select id from t1 union select id from t2;
```

```
+-----+
```

```
| id |
```

```
+-----+
```

```
| 1 |
```

```
| 2 |
```

```
| 3 |
```

```
| 5 |
```

```
| 4 |
```

```
| 11 |
```

```
| 15 |
```

```
+-----+
```

```
7 rows in set (0.00 sec)
```

// ALL: 返回包括重复数据在内的所有结果集。

```
mysql> select id from t1 union all select id from t2;
```

```
+-----+
```

```
| id |
```

```
+-----+
```

```
| 1 |
```

```
| 2 |
```

```
| 3 |
```

```
| 5 |
```

```
| 4 |
```

```
| 3 |
```

```
| 11 |
```

```
| 15 |
```

```
| 2 |
```

```
| 1 |
```

```
+-----+
```

```
10 rows in set (0.00 sec)
```

使用 Union 组合查询时，只能使用一条 order by 子句对结果集进行排序，而且必须出现在最后一条出现的 SELECT 语句之后。因为 Union 不允许对于部分结果集进行排序，只能针对最终检索出来的结果集进行排序。

注意：由于在多表组合查询时候，可能表字段并不相同。所以在对于结果集排序的时候需要使用检索出来的共同字段。检索的字段 id 必须存在于结果集中。

举例如下：

```
mysql> (select * from t1 order by id) union all (select * from t2);
```

id	name
1	爱可生
2	开源
3	社区
5	MySQL
4	张
3	中国
11	技术
15	开源
2	社区
1	伟大

```
10 rows in set (0.00 sec)
```

```
mysql> (select * from t1 order by id) union all (select * from t2 order by id);
```

id	name
1	爱可生
2	开源
3	社区
5	MySQL
4	张
3	中国
11	技术
15	开源
2	社区
1	伟大

```
10 rows in set (0.01 sec)
```

```
mysql> (select * from t1) union all (select * from t2) order by id;
```

id	name
1	爱可生
1	伟大
2	开源
2	社区
3	社区
3	中国
4	张
5	MySQL
11	技术
15	开源

```
10 rows in set (0.00 sec)
```

2、知新

- 从 MySQL 8.0.19 版本开始，不仅在 SELECT 语句中，也可以在 TABLE 或 VALUES 语句中使用 UNION，只要可以使用等效的 SELECT 语句。
- 可以在 TABLE 中使用 ORDER BY 和 LIMIT，但不支持 WHERE 子句。
- ORDER BY 不能使用包含表名.列名的引用。相反，在第一个 SELECT 语句中提供列别名，并在 ORDER BY 中引用别名。
- 如果要排序的列具有别名，ORDER BY 子句必须引用别名，而不是列名。

举例如下：

// 新增 table 语句的使用，由于取的是全表，对于单一字段的去重就不便使用了

```
mysql> table t1 union select * from t2;
```

```
+-----+-----+
```

```
| id | name |
```

```
+-----+-----+
```

```
| 1 | 爱可生 |
```

```
| 2 | 开源 |
```

```
| 3 | 社区 |
```

```
| 5 | MySQL |
```

```
| 4 | 张 |
```

```
| 3 | 中国 |
```

```
| 11 | 技术 |
```

```
| 15 | 开源 |
```

```
| 2 | 社区 |
```

```
| 1 | 伟大 |
```

```
+-----+-----+
```

```
10 rows in set (0.01 sec)
```

// 新增 values 语句的使用，但会导致字段名失效

```
mysql> values row(15,'开源') union select * from t2;
```

```
+-----+-----+
```

```
| column_0 | column_1 |
```

```
+-----+-----+
```

```
| 15 | 开源 |
```

```
| 3 | 中国 |
```

```
| 11 | 技术 |
```

```
| 2 | 社区 |
```

```
| 1 | 伟大 |
```

```
+-----+-----+
```

```
5 rows in set (0.01 sec)
```

// 使用 table 语句和 values 语句的结合

```
mysql> values row(15,'开源'),row(13,'北京') union table t2;
```

```
+-----+-----+
```

```
| column_0 | column_1 |
```

```
+-----+-----+
```

```

| 15 | 开源 |
| 13 | 北京 |
| 3  | 中国 |
| 11 | 技术 |
| 2  | 社区 |
| 1  | 伟大 |
+-----+-----+
6 rows in set (0.01 sec)

// 如果定义字段名可以使用以下方法
mysql > select * from (values row(15,'开源'),row(13,'北京')) AS t(c1,c2) union table t2;
+-----+-----+
| c1 | c2 |
+-----+-----+
| 15 | 开源 |
| 13 | 北京 |
| 3  | 中国 |
| 11 | 技术 |
| 2  | 社区 |
| 1  | 伟大 |
+-----+-----+
6 rows in set (0.02 sec)

```

三、对比 MySQL 8.0 和 5.7 对 union 的处理

在 MySQL 8.0 中，对 SELECT 和 UNION 的解析器规则被重构进而变得更加一致，且减少了重复。

与 MySQL 5.7 相比，某些语句可能需要重写：

- 对比标准 SQL，NATURAL JOIN 允许一个可选的 INNER 关键字（NATURAL INNER JOIN）。
- 对比标准 SQL，可以使用不带括号的 Right-deep JOIN（例如，...JOIN...JOIN...ON...ON）。
- 与其他 INNER JOIN 类似，STRAIGHT_JOIN 现在允许 USING 子句。
- 解析器接受查询表达式周围的括号。例如：（SELECT ... UNION SELECT...）。
- 以前只允许在子查询中使用 union 的左嵌套，现在允许顶层语句中使用。如：(SELECT 1 UNION SELECT 1) UNION SELECT 1；
- 包含锁定子句的 SELECT 语句必须使用括号。例如：SELECT 1 FOR UPDATE UNION SELECT 1 FOR UPDATE；变更为 (SELECT 1 FOR UPDATE) UNION (SELECT 1 FOR UPDATE)；

四、参考文档：

<https://dev.mysql.com/doc/refman/8.0/en/union.html>

1.8 MySQL 8.0.28 用户连接内存限制

作者：杨涛涛

以往 MySQL 想要限制单个连接的内存，只能小心翼翼的设置各种 SESSION 变量，防止执行某些 SQL 导致单个连接内存溢出！能不能直接在 MySQL 服务端包含这样一个功能，简化数据库的运维呢？

MySQL 最新版本 8.0.28 在前几天发布，其中有一项新功能就是在数据库侧来限制单个连接的内存，着实有点小兴奋。

MySQL 8.0.28 与此功能有关的几个新参数如下：

1. `connection_memory_limit`: 核心参数！用来限制单用户连接的内存上限值，默认为 `BIGINT UNSIGNED` 的最大值：18446744073709551615 字节，最小为 2MB。
2. `global_connection_memory_tracking`: 设置是否开启对连接内存功能的追踪，并且把连接内存数据存入状态变量 `Global_connection_memory`。为了性能考虑，默认关闭。
3. `connection_memory_chunk_size`: 在参数 `global_connection_memory_tracking` 开启的场景下，设置状态变量 `Global_connection_memory` 的更新频率。

接下来我们体验下这个新特性。

管理员端设置内存限制参数上限：为了尽快看到效果，设置为最小值。

```
localhost:(none)>set global connection_memory_limit=2097152;
Query OK, 0 rows affected (0.00 sec)
```

创建一个新用户 `tt1`，并赋予基于库 `ytt` 的只读权限。

```
localhost:(none)>create user tt1 identified by 'tt';
Query OK, 0 rows affected (0.03 sec)
```

```
localhost:(none)>grant select on ytt.* to tt1;
Query OK, 0 rows affected (0.02 sec)
```

创建一张表，插入一行记录：这里使用 `longtext` 数据类型能让查询结果更快内存溢出。

```
localhost:ytt>create table t(id int primary key, r1 longtext);
Query OK, 0 rows affected (2.39 sec)
localhost:ytt>insert t values (1,lpad('mysql',6000000,'database'));
Query OK, 1 row affected (0.63 sec)
```

用户 `tt1` 登录验证：对字段 `r1` 进行简单 `GROUP BY` 检索，报连接内存超出设定限制错误，连接关闭。

```
debian-ytt1:ytt>select count(r1) from t group by r1;  
ERROR 4082 (HY000): Connection closed. Connection memory limit 2097152 bytes exceeded.  
Consumed 7094928 bytes.
```

不过这个新功能对管理员和内置用户不生效。用 ROOT 用户重新登录 MySQL 执行刚才那条 SQL :

```
root@debian-ytt1:~# mysql -S /tmp/mysqld_3306.sock  
...  
  
localhost:(none)>use ytt  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A  
  
Database changed  
localhost:ytt>select count(r1) from t group by r1;  
+-----+  
| count(r1) |  
+-----+  
| 1 |  
+-----+  
1 row in set (0.03 sec)
```

可以看到，管理员可以正常执行这条 SQL。所以我们 DBA 给开发用户赋予权限时，为了避免不必要的运维工作，禁止赋 SUPER 权限。

1.9 MySQL 8.0 多因素身份认证

作者：金长龙

MySQL 8.0.27 增加了多因素身份认证(MFA)功能，可以为一个用户指定多重的身份校验。为此还引入了新的系统变量 `authentication_policy`，用于管理多因素身份认证功能。

我们知道在 MySQL 8.0.27 之前，`create user` 的时候可以指定一种认证插件，在未明确指定的情况下会取系统变量 `default_authentication_plugin` 的值。`default_authentication_plugin` 的有效值有 3 个，分别是 `mysql_native_password`，`sha256_password`，`caching_sha2_password`，这个 3 个认证插件是内置的、不需要注册步骤的插件。

一、系统变量 `authentication_policy`

在 MySQL 8.0.27 中由 `authentication_policy` 来管理用户的身份认证，先启个 `mysql`

```
root@ubuntu:~# docker run --name mysql-1 -e MYSQL_ROOT_PASSWORD=123 -d --ip 172.17.0.2
mysql:8.0.27
```

同时查看下 authentication_policy 和 default_authentication_plugin 的值

```
root@ubuntu:~# docker run -it --rm mysql:8.0.27 mysql -h172.17.0.2 -uroot -p123
.....
```

```
mysql> show global variables like 'authentication_policy';
```

```
+-----+-----+
| Variable_name | Value |
+-----+-----+
| authentication_policy | *,, |
+-----+-----+
1 row in set (0.02 sec)
```

```
mysql> show global variables like 'default_authentication_plugin';
```

```
+-----+-----+
| Variable_name | Value |
+-----+-----+
| default_authentication_plugin | caching_sha2_password |
+-----+-----+
1 row in set (0.00 sec)
```

我们看到 authentication_policy 的默认值是*,,

第 1 个元素值是星号 (*), 表示可以是任意插件, 默认值取 default_authentication_plugin 的值。如果该元素值不是星号 (*), 则必须设置为 mysql_native_password, sha256_password, caching_sha2_password 中的一个。

第 2, 3 个元素值为空, 这两个位置不能设置成内部存储的插件。如果元素值为空, 代表插件是可选的。

建个用户看一下, 不指定插件名称时, 自动使用默认插件 caching_sha2_password

```
mysql> create user 'wei1'@'localhost' identified by '123';
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> select user,host,plugin from mysql.user where user='wei1';
```

```
+-----+-----+
| user | host | plugin |
+-----+-----+
| wei1 | localhost | caching_sha2_password |
+-----+-----+
1 row in set (0.00 sec)
```

指定插件名称时, 会使用到对应的插件

```
mysql> create user 'wei2'@'localhost' identified with mysql_native_password by '123';
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> select user,host,plugin from mysql.user where user='wei2';
+-----+-----+-----+
| user | host | plugin |
+-----+-----+-----+
| wei2 | localhost | mysql_native_password |
+-----+-----+-----+
1 row in set (0.01 sec)
```

尝试变更一下 authentication_policy 第一个元素的值，设置为 sha256_password

```
mysql> set global authentication_policy='sha256_password,,';
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> show global variables like 'authentication_policy';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| authentication_policy | sha256_password,, |
+-----+-----+
1 row in set (0.00 sec)
```

再次创建一个用户，不指定插件的名称

```
mysql> create user 'wei3'@'localhost' identified by '123';
Query OK, 0 rows affected (0.01 sec)

mysql> select user,host,plugin from mysql.user where user='wei3';
+-----+-----+-----+
| user | host | plugin |
+-----+-----+-----+
| wei3 | localhost | sha256_password |
+-----+-----+-----+
1 row in set (0.00 sec)
```

可以看到默认使用的插件是 sha256_password，说明当 authentication_policy 第一个元素指定插件名称时，default_authentication_plugin 被弃用了。

二、多重身份验证的用户

首先我们恢复 authentication_policy 至默认值

```
mysql> set global authentication_policy='*,,';
Query OK, 0 rows affected (0.01 sec)
```



```
mysql> show global variables like 'authentication_policy';
```

```
+-----+-----+
| Variable_name | Value |
+-----+-----+
| authentication_policy | *,, |
+-----+-----+
1 row in set (0.01 sec)
```

创建一个双重认证的用户。如下创建失败了，因为不可以同时用 2 种内部存储插件。

```
mysql> create user 'wei3'@'localhost' identified by '123' and identified with
mysql_native_password by '123';
ERROR 4052 (HY000): Invalid plugin "mysql_native_password" specified as 2 factor during
"CREATE USER".
```

那我们来装一个可插拔插件 Socket Peer-Credential

```
mysql> INSTALL PLUGIN auth_socket SONAME 'auth_socket.so';
```

```
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT PLUGIN_NAME, PLUGIN_STATUS FROM INFORMATION_SCHEMA.PLUGINS WHERE
PLUGIN_NAME LIKE '%socket%';
```

```
+-----+-----+
| PLUGIN_NAME | PLUGIN_STATUS |
+-----+-----+
| auth_socket | ACTIVE |
+-----+-----+
1 row in set (0.00 sec)
```

再创建一个双重认证的用户

```
mysql> create user 'wei4'@'localhost' identified by '123' and identified with auth_socket as 'root';
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> select user,host,plugin,user_attributes from mysql.user where user='wei4';
```

```
+-----+-----+-----+-----+
| user | host | plugin | user_attributes |
+-----+-----+-----+-----+
| wei4 | localhost | caching_sha2_password | {"multi_factor_authentication": [{"plugin": "auth_socket", "passwordless": 0, "authentication_string": "root", "requires_registration": 0}]} |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

创建成功，之后用户'wei4'@'localhost'必须提供正确的密码，且同时本地主机的登录用户为 root 时，才会验证通过。

来试一下，以主机 root 用户身份，提供正确的密码 123，登录成功。

```
root@ubuntu:~# docker exec -it mysql-1 bash
```

```
root@1d118873f98e:/# mysql -uwei4 --password1=123 --password2
```

```
mysql: [Warning] Using a password on the command line interface can be insecure.
```

```
Enter password:
```

```
Welcome to the MySQL monitor. Commands end with ; or \g.
```

```
Your MySQL connection id is 12
```

```
Server version: 8.0.27 MySQL Community Server - GPL
```

Copyright (c) 2000, 2021, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>

修改一下，将'wei4'@'localhost'要求的主机登录用户修改为 wei4

```
mysql> alter user 'wei4'@'localhost' modify 2 factor identified with auth_socket as 'wei4';
Query OK, 0 rows affected (0.16 sec)

mysql> select user,host,plugin,User_attributes from mysql.user where user='wei4';
+-----+-----+-----+-----+
| user | host      | plugin          | User_attributes |
+-----+-----+-----+-----+
| wei4 | localhost | caching_sha2_password | {"multi_factor_authentication": [{"plugin": "auth_socket", "passwordless": 0, "authentication_string": "wei4", "requires_registration": 0}]} |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

再次以主机 root 用户身份，提供正确的密码 123 ，登录失败

```
root@ubuntu:~# docker exec -it mysql-1 bash
root@1d118873f98e:/# mysql -uwei4 --password1=123 --password2
mysql: [Warning] Using a password on the command line interface can be insecure.
Enter password:
ERROR 1698 (28000): Access denied for user 'wei4'@'localhost'
root@1d118873f98e:/#
```

因此可以认定双重身份认证机制是生效的。MySQL8.0.27 最多可以对一个用户设置三重的身份认证，这里不再做展示说明。

简单总结下，已有的密码口令身份验证很适合网站或者应用程序的访问，但是在特定的情况下 如网络在线金融交易方面可能还是不够安全。多因素身份认证(MFA)功能的引入，可以在一定程度上提升数据库系统的安全性。

参考资料：

https://dev.mysql.com/doc/refman/8.0/en/server-system-variables.html#sysvar_authentication_policy

1.10 MySQL 8.0 窗口函数一次疑问解答

作者：杨涛涛

记起来有一次去讲 MySQL 8.0 开发相关特性，在 QA 环节，有人对 MySQL 的几个非常用窗口函数有些困惑，当时现场给了一些示范并且做了详细的解答，今天我用几个简单例子分享下具体的用法。

有困惑的是这四个窗口函数（其实是因为我 PPT 里仅写了这四个窗口函数）：first_value、last_value、nth_value、ntile。

谈到这四个窗口函数的具体用法，特别是前两个，得先熟悉 MySQL 窗口函数的框架用法。这里提到的窗口函数框架，其实就是定义一个分组窗口的边界，边界可以是具体的行号，也可以是具体的行内容，以这个边界为起点或者终点，来展现分组内的过滤数据。详情见我之前的发稿：

<https://opensource.actionsky.com/20210125-mysql/>

接下来我们来看看这四个窗口函数如何使用。

1. first_value：用来返回一个分组窗口里的第一行记录，也即排名第一的那行记录。

我们用表 t1 来示范，这张表里只有 12 行记录，其中每 6 行记录按照字段 r1 来分组。

```
localhost:ytt_new>select id,r1,r2 from t1;
```

```
+-----+-----+-----+
```

```
| id | r1 | r2 |
```

```
+-----+-----+-----+
```

```
| 1 | 10 | 20 |
```

```
| 2 | 10 | 30 |
```

```
| 3 | 10 | 40 |
```

```
| 4 | 10 | 50 |
```

```
| 5 | 10 | 2 |
```

```
| 6 | 10 | 3 |
```

```
| 7 | 11 | 100 |
```

```
| 8 | 11 | 101 |
```

```
| 9 | 11 | 1 |
```

```
| 10 | 11 | 3 |
```

```
| 11 | 11 | 10 |
```

```
| 12 | 11 | 20 |
```

```
+-----+-----+-----+
```

```
12 rows in set (0.00 sec)
```

比如想拿到每个分组里的第一名（升序），可以用 row_number() 函数，我们来回顾下：

```
localhost:ytt_new>select r1,r2 from (select r1,r2,row_number() over(partition by r1
```

```
order by r2) as rn from t1) T where T.rn = 1;
+-----+-----+
| r1 | r2 |
+-----+-----+
| 10 | 2 |
| 11 | 1 |
+-----+-----+
2 rows in set (0.00 sec)
```

此时如果用 `first_value` 来实现，写法会更加简单：

```
localhost:ytt_new>select distinct r1,first_value(r2) over(partition by r1 order by r2)
as first_r2 from t1;
+-----+-----+
| r1 | first_r2 |
+-----+-----+
| 10 | 2 |
| 11 | 1 |
+-----+-----+
2 rows in set (0.00 sec)
```

2. last_value: 和 `first_value` 相反，用来返回分组窗口里的最后一行记录，也即倒数第一的那行记录。

比如我取出对应分组内最后一行 `r2` 的值，如果用 `last_value` 函数，非常好实现，可结果和预期不一致：返回与字段 `r2` 本身等值的记录。

```
localhost:ytt_new>select distinct r1,last_value(r2) over(partition by r1 order by r2)
'last_r2' from t1; +-----+-----+
| r1 | last_r2 |
+-----+-----+
| 10 | 2 |
| 10 | 3 |
| 10 | 20 |
| 10 | 30 |
| 10 | 40 |
| 10 | 50 |
| 11 | 1 |
| 11 | 3 |
| 11 | 10 |
| 11 | 20 |
| 11 | 100 |
| 11 | 101 |
+-----+-----+
12 rows in set (0.01 sec)
```

究其原因是函数 `last_value` 的默认框架是 `rows between unbounded preceding and current row`。这里默认框

架意思是：限制窗口函数的取值边界为当前行和上限无穷大，所以对应的值就是当前行自己。

那正确的框架应该是什么样呢？正确的框架应该是让边界锁定整个分组的上下边缘，也即整个分组的上限与下限之间。所以正确的写法如下：

```
localhost:ytt_new>select distinct r1,last_value(r2) over(partition by r1 order by r2
RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) as 'last_r2' from t1;
+-----+-----+
| r1 | last_r2 |
+-----+-----+
| 10 | 50 |
| 11 | 101 |
+-----+-----+
2 rows in set (0.00 sec)
```

3. nth_value: 用来返回分组内指定行的记录。

比如用 nth_value 函数来求分组内排名第一的记录：

```
localhost:ytt_new>select * from (select distinct r1,nth_value(r2,1) over(partition by
r1 order by r2) 'first_r2' from t1) T where T.first_r2 is not null;
+-----+-----+
| r1 | first_r2 |
+-----+-----+
| 10 | 2 |
| 11 | 1 |
+-----+-----+
2 rows in set (0.00 sec)
```

这个函数的功能基本和函数 row_number 一致。不同的是 row_number 用来展示排名，而 nth_value 用来输入排名。

4. ntile: 用来在分组内继续二次分组。

比如我想取出分组内排名前 50% 的记录，可以这样写：

```
localhost:ytt_new>select id,r1,r2 from (select id,r1,r2, ntile(2) over(partition by r1
order by r2) 'ntile
' from t1) T where T.ntile=1;
+-----+-----+-----+
| id | r1 | r2 |
+-----+-----+-----+
| 5 | 10 | 2 |
| 6 | 10 | 3 |
| 1 | 10 | 20 |
| 9 | 11 | 1 |
```

```
| 10 | 11 | 3 |
| 11 | 11 | 10 |
+----+-----+-----+
6 rows in set (0.00 sec)
```

这四个窗口函数，特别是 `last_value` 需要注意。不过在大多数场景下，记住几个常用的窗口函数即可，比如 `row_number()`，`rank()` 等等。

1.11 MySQL 8.0 对 GTID 的限制解除

作者：杨涛涛

在 MySQL 5.6 以及 MySQL 5.7 上使用 GTID，一直以来都有几个硬性限制，特别是针对开发人员编写 SQL 的两条限制，官方文档对这两条限制详细描述如下：

CREATE TABLE ... SELECT statements. CREATE TABLE ...

SELECT (<https://dev.mysql.com/doc/refman/5.7/en/create-table-select.html>) statements are not allowed when using GTID-based replication. When `binlog_format` (https://dev.mysql.com/doc/refman/5.7/en/replication-options-binary-log.html#sysvar_binlog_format) is set to STATEMENT, a CREATE TABLE ...

SELECT statement is recorded in the binary log as one transaction with one GTID, but if ROW format is used, the statement is recorded as two transactions with two GTIDs. If a source used STATEMENT format and a replica used ROW format, the replica would be unable to handle the transaction correctly, therefore the CREATE TABLE ... SELECT statement is disallowed with GTIDs to prevent this scenario.

Temporary tables. CREATE TEMPORARY TABLE (<https://dev.mysql.com/doc/refman/5.7/en/create-table.html>) and DROP TEMPORARY TABLE (<https://dev.mysql.com/doc/refman/5.7/en/drop-table.html>) statements are not supported inside transactions, procedures, functions, and triggers when using GTIDs (that is, when the `enforce_gtid_consistency` (https://dev.mysql.com/doc/refman/5.7/en/replication-options-gtids.html#sysvar_enforce_gtid_consistency) system variable is set to ON). It is possible to use these statements with GTIDs enabled, but only outside of any transaction, and only with `autocommit=1` (https://dev.mysql.com/doc/refman/5.7/en/server-system-variables.html#sysvar_autocommit).

以上大概意思讲的是对于这两条 SQL 语句，如果想在 GTID 模式下使用，为了不破坏事务一致性，是被严格限制而不允许使用的。为了满足需求，一般我们会通过一些途径来绕过这些限制。这个硬性限制随着 MySQL 8.0 一些新特性的发布，连带着被间接取消掉。比如 MySQL 8.0 的 DDL 原子性！

我们先来看下在 MySQL 5.7 下这一行为对事务的影响以及如何通过变通的方法绕过这些限制。

1. `create table ... select ...`：这条语句本身是懒人写法，语义上分别属于两个隐式事务（一条 DDL 语句，一条 DML 语句）。但在 GTID 开启后，单个语句只能给它分配一个 GTID 事务号，如果强制使用，

会直接报语句违反 GTID 一致性。比如下面例子：直接执行这条语句就会报错。

```
mysql:ytt:5.7.34-log> create table trans1(id int primary key, log_date date);
Query OK, 0 rows affected (0.03 sec)

<mysql:ytt:5.7.34-log> insert trans1 values (1,'2022-01-02');
Query OK, 1 row affected (0.00 sec)

<mysql:ytt:5.7.34-log> create table trans2 as select * from trans1;
ERROR 1786 (HY000): Statement violates GTID consistency: CREATE TABLE ... SELECT.
```

既然理解了需求，就想办法变通下。针对这条语句，拆分为两条语句即可。

需要注意的是拆分后第一条 DDL 语句的后续工作，是延迟建立索引还是根本不需要索引？如果是延迟建立索引，那很简单，使用 MySQL 的 `create table ... like ...` 语法就行。虽然 `create table ... like ...` 语法是直接克隆原表，索引也是立即创建，不过最终目标是一致的。示例如下：

```
<mysql:ytt:5.7.34-log> reset master;
Query OK, 0 rows affected (0.02 sec)

mysql:ytt:5.7.34-log> create table trans2 like trans1;
Query OK, 0 rows affected (0.02 sec)

<mysql:ytt:5.7.34-log> insert trans2 select * from trans1;
Query OK, 1 row affected (0.02 sec)
Records: 1 Duplicates: 0 Warnings: 0
```

对应的 binlog 数据如下，拆分为两个 GTID 事务号：00020135-1111-1111-1111-111111111111:1-2

```
<mysql:ytt:5.7.34-log> show binlog events in 'mysql-bin.000001'\G
***** 1. row *****
...
***** 3. row *****
  Log_name: mysql-bin.000001
    Pos: 154
Event_type: Gtid
Server_id: 100
End_log_pos: 219
  Info: SET @@SESSION.GTID_NEXT= '00020135-1111-1111-1111-111111111111:1'
***** 4. row *****
  Log_name: mysql-bin.000001
    Pos: 219
Event_type: Query
Server_id: 100
End_log_pos: 316
  Info: use `ytt`; create table trans2 like trans1
***** 5. row *****
```

```

Log_name: mysql-bin.000001
Pos: 316
Event_type: Gtid
Server_id: 100
End_log_pos: 381
Info: SET @@SESSION.GTID_NEXT= '00020135-1111-1111-1111-111111111111:2'
***** 6. row *****
Log_name: mysql-bin.000001
Pos: 381
Event_type: Query
Server_id: 100
End_log_pos: 452
Info: BEGIN
***** 7. row *****
Log_name: mysql-bin.000001
Pos: 452
Event_type: Table_map
Server_id: 100
End_log_pos: 501
Info: table_id: 112 (ytt.trans2)
***** 8. row *****
Log_name: mysql-bin.000001
Pos: 501
Event_type: Write_rows
Server_id: 100
End_log_pos: 552
Info: table_id: 112 flags: STMT_END_F
***** 9. row *****
Log_name: mysql-bin.000001
Pos: 552
Event_type: Xid
Server_id: 100
End_log_pos: 583
Info: COMMIT /* xid=54 */
9 rows in set (0.00 sec)

```

如果是后一种，只需要复制表结构和数据，不要索引，那也可以用 `create table ... like ...` 语法创建好表结构，完了手工删除表索引。如果表比较多，可以写个简单脚本对索引批量删除。

2. 对于显式临时表的创建与删除：这样的 DDL 语句在 GTID 模式下也是禁止放在事务块里执行的（显式的 `begin; commit;` 或者存储过程、存储函数、触发器等大事务块）。直接在事务块里执行会报错：

```

<mysql:ytt:5.7.34-log> begin;
Query OK, 0 rows affected (0.00 sec)

```



```
<mysql:ytt:5.7.34-log> create temporary table tmp(id int);
ERROR 1787 (HY000): Statement violates GTID consistency: CREATE TEMPORARY TABLE and
DROP TEMPORARY TABLE can only be executed outside transactional context. These
statements are also not allowed in a function or trigger because functions and triggers
are also considered to be multi-statement transactions.
```

这种如何解决呢？官方也给出建议：把此类 DDL 语句放在事务块外面或者直接使用基于磁盘表的 DDL 语句来替代它。比如下面示例：在事务块外创建临时表，事务块内部引用临时表数据就行。

```
<mysql:ytt:5.7.34-log> create temporary table tmp(id int,log_date date);
Query OK, 0 rows affected (0.00 sec)

<mysql:ytt:5.7.34-log> begin;
Query OK, 0 rows affected (0.01 sec)

<mysql:ytt:5.7.34-log> insert tmp values (100,'2022-10-21');
Query OK, 1 row affected (0.01 sec)

<mysql:ytt:5.7.34-log> insert trans1 select * from tmp;
Query OK, 1 row affected (0.00 sec)
Records: 1 Duplicates: 0 Warnings: 0

<mysql:ytt:5.7.34-log> commit;
Query OK, 0 rows affected (0.00 sec)

<mysql:ytt:5.7.34-log> select * from trans1;
+-----+-----+
| id | log_date |
+-----+-----+
| 1 | 2022-01-02 |
| 100 | 2022-10-21 |
+-----+-----+
2 rows in set (0.00 sec)
```

MySQL 8.0 原生 DDL 原子性，所以连带就解除了这两个 GTID 的限制。

1. 对于 create table ... like ... 语句，在 MySQL 8.0 版本里只会生成一个 GTID 事务号，见下面 binlog 内容：0228ca56-db2f-11ec-83d3-080027951c4a:1。

```
mysql:ytt:8.0.29>create table trans2 as select * from trans1;
Query OK, 1 row affected (0.08 sec)
Records: 1 Duplicates: 0 Warnings: 0

<mysql:ytt:8.0.29>show binlog events in 'binlog.000001'\G
***** 1. row *****
```

```
...
***** 3. row *****
  Log_name: binlog.000001
    Pos: 157
Event_type: Gtid
Server_id: 1
End_log_pos: 236
  Info: SET @@SESSION.GTID_NEXT= '0228ca56-db2f-11ec-83d3-080027951c4a:1'
***** 4. row *****
  Log_name: binlog.000001
    Pos: 236
Event_type: Query
Server_id: 1
End_log_pos: 310
  Info: BEGIN
***** 5. row *****
  Log_name: binlog.000001
    Pos: 310
Event_type: Query
Server_id: 1
End_log_pos: 476
  Info: use `ytt`; CREATE TABLE `trans2` (
  `id` int NOT NULL,
  `log_date` date DEFAULT NULL
) START TRANSACTION
***** 6. row *****
  Log_name: binlog.000001
    Pos: 476
Event_type: Table_map
Server_id: 1
End_log_pos: 528
  Info: table_id: 349 (ytt.trans2)
***** 7. row *****
  Log_name: binlog.000001
    Pos: 528
Event_type: Write_rows
Server_id: 1
End_log_pos: 571
  Info: table_id: 349 flags: STMT_END_F
***** 8. row *****
  Log_name: binlog.000001
    Pos: 571
Event_type: Xid
Server_id: 1
```

```
End_log_pos: 602
      Info: COMMIT /* xid=8833 */
8 rows in set (0.00 sec)
```

2. 对于事务块里有显式临时表的 DDL 语句，可以正常执行：

```
<mysql:ytt:8.0.29>reset master;
Query OK, 0 rows affected (0.02 sec)

<mysql:ytt:8.0.29>begin;
Query OK, 0 rows affected (0.01 sec)

<mysql:ytt:8.0.29>create temporary table tmp(a int,b date);
Query OK, 0 rows affected (0.00 sec)

<mysql:ytt:8.0.29>insert into tmp values (10,'2022-12-31');
Query OK, 1 row affected (0.00 sec)

<mysql:ytt:8.0.29>insert trans1 select * from tmp;
Query OK, 1 row affected (0.00 sec)
Records: 1 Duplicates: 0 Warnings: 0

<mysql:ytt:8.0.29>commit;
Query OK, 0 rows affected (0.01 sec)

<mysql:ytt:8.0.29>table trans1;
+----+-----+
| id | log_date |
+----+-----+
| 1  | 2022-07-07 |
| 10 | 2022-12-31 |
+----+-----+
2 rows in set (0.00 sec)
```

这样生成的 GTID 事务号里（0228ca56-db2f-11ec-83d3-080027951c4a:1）只包含对磁盘表 trans1 的写入记录：

```
mysql:ytt:8.0.29>show binlog events in 'binlog.000001'\G
***** 1. row *****
...
***** 3. row *****
  Log_name: binlog.000001
    Pos: 157
Event_type: Gtid
  Server_id: 1
End_log_pos: 236
```

```
Info: SET @@SESSION.GTID_NEXT= '0228ca56-db2f-11ec-83d3-080027951c4a:1'
***** 4. row *****
Log_name: binlog.000001
Pos: 236
Event_type: Query
Server_id: 1
End_log_pos: 310
Info: BEGIN
***** 5. row *****
Log_name: binlog.000001
Pos: 310
Event_type: Table_map
Server_id: 1
End_log_pos: 362
Info: table_id: 405 (ytt.trans1)
***** 6. row *****
Log_name: binlog.000001
Pos: 362
Event_type: Write_rows
Server_id: 1
End_log_pos: 405
Info: table_id: 405 flags: STMT_END_F
***** 7. row *****
Log_name: binlog.000001
Pos: 405
Event_type: Xid
Server_id: 1
End_log_pos: 436
Info: COMMIT /* xid=9374 */
7 rows in set (0.00 sec)
```

MySQL 8.0 已经发布好几年了，如果有需要这部分的功能改善，建议升级新版本。

1.12 MySQL 8.0 GIPK 不可见主键

作者：杨奇龙

一、前言

作为 MySQL DBA，相信大家都经历过在复制模式下，如果没有主键，遇到 load data，大事务，ddl 等有大量表数据行扫描的行为时，会带来严重的主从延迟，给数据库稳定性和数据一致性带来隐患。

MySQL 8.0.30 已于近日 GA，新版本为我们提供了一个令人惊喜的特性 - (Generated Invisible Primary Keys) 简称 GIPK。一句概况就是：当开启 GIPK 模式后，MySQL 会在没有显示定义主键的 InnoDB 表上自动生成不可见的主键。

对于已经使用云 RDS 的朋友，可能很早就享受到云 RDS MySQL 提供的隐式主键特性。但是对于自建数据库的企业，GIPK 依然是一个比较期待特性，（当然有和用起来是两码事！）

长话短说，本文基于实际测试案例来学校 如何使用 GIPK。

二、实践出真知

2.1 开启

GIPK 由参数 `sql_generate_invisible_primary_key` 控制，默认关闭，表示禁用，如果需要使用该特性，则需显式开启。

```
master [localhost:22031]> show variables like 'sql_generate_invisible_primary_key';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| sql_generate_invisible_primary_key | OFF |
+-----+-----+
1 row in set (0.00 sec)
```

```
master [localhost:22031]> set sql_generate_invisible_primary_key=on;
Query OK, 0 rows affected (0.00 sec)
```

```
master [localhost:22031]> show variables like 'sql_generate_invisible_primary_key';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| sql_generate_invisible_primary_key | ON |
+-----+-----+
1 row in set (0.00 sec)
```

2.2 测试

我们分别在关闭和开启该特性下创建两个无主键表:

```
master [localhost:22031]> create table t1(id int ,c1 int);
Query OK, 0 rows affected (0.00 sec)
master [localhost:22031] {msandbox} (test) > show create table t1 \G
***** 1. row *****
      Table: t1
Create Table: CREATE TABLE `t1` (
  `id` int DEFAULT NULL,
  `c1` int DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

开启 GIPK 并创建无主键表 t3 。

```
master [localhost:22031]> set sql_generate_invisible_primary_key=on;
Query OK, 0 rows affected (0.00 sec)

master [localhost:22031] {msandbox} (test) > show variables like 'sql_generate_invisibl
e_primary_key';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| sql_generate_invisible_primary_key | ON    |
+-----+-----+
1 row in set (0.00 sec)

master [localhost:22031]> create table t3(id int ,c1 int);
Query OK, 0 rows affected (0.01 sec)

master [localhost:22031]>
master [localhost:22031]> show create table t3 \G
***** 1. row *****
      Table: t3
Create Table: CREATE TABLE `t3` (
  `my_row_id` bigint unsigned NOT NULL AUTO_INCREMENT /*!80023 INVISIBLE */,
  `id` int DEFAULT NULL,
  `c1` int DEFAULT NULL,
  PRIMARY KEY (`my_row_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

我们可以通过 `show create table` 发现 t3 的表结构，出现名为 `my_row_id` 的不可见主键。对两个表插入数据查看差异:

```

master [localhost:22031]> insert into t1 values(1,1),(2,2),(3,3);
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0
master [localhost:22031]> select * from t1;
+-----+-----+
| id    | c1    |
+-----+-----+
| 1     | 1     |
| 2     | 2     |
| 3     | 3     |
+-----+-----+
3 rows in set (0.00 sec)
master [localhost:22031]> insert into t3 values(1,1),(2,2),(3,3);
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0
master [localhost:22031]> select * from t3;
+-----+-----+
| id    | c1    |
+-----+-----+
| 1     | 1     |
| 2     | 2     |
| 3     | 3     |
+-----+-----+
3 rows in set (0.00 sec)

```

直接通过 `select * from table` 查询时, `t3` 和普通表 `t1` 无差异。因为 GIPK 是基于不可见列实现的, 如果我们显式指定访问 `my_row_id`, 则可以查看到隐藏的主键 `my_row_id`。

```

master [localhost:22031]> select my_row_id,id,c1 from t3;
+-----+-----+-----+
| my_row_id | id    | c1    |
+-----+-----+-----+
| 1         | 1     | 1     |
| 2         | 2     | 2     |
| 3         | 3     | 3     |
+-----+-----+-----+
3 rows in set (0.00 sec)

```

总的来说, 从业务程序访问数据库的角度来看, 开启 GIPK 对业务是透明的。

2.3 关于 DDL

当开启 GIPK 特性时, MySQL 生成的主键不能更改, 只能在 `VISIBLE` 和 `INVISIBLE` 之间进行切换。比如:

使 GIPK 主键可见: `alter table TABLE_NAME alter column my_row_id set visible;`

```

master [localhost:22031]> alter table t3 alter column my_row_id set visible;
Query OK, 0 rows affected (0.01 sec)
Records: 0 Duplicates: 0 Warnings: 0
master [localhost:22031]> show create table t3 \G
***** 1. row *****
      Table: t3
Create Table: CREATE TABLE `t3` (
  `my_row_id` bigint unsigned NOT NULL AUTO_INCREMENT, ### 显式可见
  `id` int DEFAULT NULL,
  `c1` int DEFAULT NULL,
  PRIMARY KEY (`my_row_id`)
) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)

```

而且可以被直接查询到

```

master [localhost:22031]> select * from t3;
+-----+-----+-----+
| my_row_id | id  | c1  |
+-----+-----+-----+
|          1 |    1 |    1 |
|          2 |    2 |    2 |
|          3 |    3 |    3 |
+-----+-----+-----+
3 rows in set (0.00 sec)

```

关闭起可见性：alter table `TABLE_NAME` alter column `my_row_id` set invisible;

```

master [localhost:22031]> alter table t3 alter column my_row_id set invisible;
Query OK, 0 rows affected (0.00 sec)
Records: 0 Duplicates: 0 Warnings: 0
master [localhost:22031]> show create table t3 \G
***** 1. row *****
      Table: t3
Create Table: CREATE TABLE `t3` (
  `my_row_id` bigint unsigned NOT NULL AUTO_INCREMENT /*!80023 INVISIBLE */,
  `id` int DEFAULT NULL,
  `c1` int DEFAULT NULL,
  PRIMARY KEY (`my_row_id`)
) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)

```

再次通过 select * 查询则看不到 my_row_id

```

master [localhost:22031]> select * from t3;
+-----+-----+
| id  | c1  |
+-----+-----+

```



```
| 1 | 1 |
| 2 | 2 |
| 3 | 3 |
+-----+-----+
3 rows in set (0.00 sec)
```

另外就是开启 GIPK 之后，`my_row_id` 是系统关键字，我们创建无主键的表时，不能包含名为 `my_row_id` 的字段。

```
master [localhost:22031]> create table t6(my_row_id int not null ,c1 int);
ERROR 4108 (HY000): Failed to generate invisible primary key. Column 'my_row_id' already exists.
```

当然如果 MySQL 允许创建包含名为 `my_row_id` 的主键的表：

```
master [localhost:22031]> create table t5(my_row_id int not null auto_increment primary
key ,c1 int);
Query OK, 0 rows affected (0.01 sec)
```

当开启 GIPK 模式时，如不能直接删除不可见主键。必须显式增加一个新的主键然后再删除 GIPK

```
master [localhost:22031]> alter table t3 drop PRIMARY KEY;
ERROR 1235 (42000): This version of MySQL doesn't yet support 'existing primary key drop without adding a new primary key. In @@sql_generate_invisible_primary_key=ON mode table should have a primary key. Please add a new primary key to be able to drop existing primary key.'
```

```
master [localhost:22031]> alter table t3 drop PRIMARY KEY,add primary key(id);
ERROR 4111 (HY000): Please drop primary key column to be able to drop generated invisible primary key.
```

```
master [localhost:22031]> alter table t3 drop column my_row_id,add primary key(id);
Query OK, 0 rows affected (0.02 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

2.4 关于主从复制

需要注意的是 `set sql_generate_invisible_primary_key=on/off` 并不会被复制到从库，主库上开启该特性的话，从库并不会开启 GIPK。也就是说从库也不会为任何在源库上没有创建主键的表创建主键。可能会有读者疑问如果主库关闭该特性，但是从库显示开启呢？做个测试看看

在 master 上关闭该特性并且创建无主键表 t6

```
master [localhost:22031]> set sql_generate_invisible_primary_key=off;
Query OK, 0 rows affected (0.00 sec)
master [localhost:22031]>
master [localhost:22031]> show variables like 'sql_generate_invisible_primary_key';
+-----+-----+
```

```

| Variable_name | Value |
+-----+
| sql_generate_invisible_primary_key | OFF |
+-----+
1 row in set (0.00 sec)

master [localhost:22031]> show tables;
+-----+
| Tables_in_test |
+-----+
| t1 |
| t2 |
| t3 |
| t4 |
| t5 |
+-----+
5 rows in set (0.00 sec)

master [localhost:22031]> create table t6(id int ,c1 int);
Query OK, 0 rows affected (0.01 sec)

master [localhost:22031]> show create table t6\G
***** 1. row *****
      Table: t6
Create Table: CREATE TABLE `t6` (
  `id` int DEFAULT NULL,
  `c1` int DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)

```

在从库上开启该特性

```

slave1 [localhost:22032]> show tables;
+-----+
| Tables_in_test |
+-----+
| t1 |
| t2 |
| t3 |
| t4 |
| t5 |
+-----+
5 rows in set (0.00 sec)

slave1 [localhost:22032]> set sql_generate_invisible_primary_key=on;
Query OK, 0 rows affected (0.00 sec)

```

```

slave1 [localhost:22032]> show variables like 'sql_generate_invisible_primary_key';
+-----+-----+
| Variable_name          | Value |
+-----+-----+
| sql_generate_invisible_primary_key | ON    |
+-----+-----+
1 row in set (0.00 sec)
slave1 [localhost:22032]> show create table t6\G
***** 1. row *****
      Table: t6
Create Table: CREATE TABLE `t6` (
  `id` int DEFAULT NULL,
  `c1` int DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)

```

结果: 主库关闭 GIPK , 从库开启 GIPK , 源库上创建无主键表, 从库上并不会主动为该表创建主键。

2.5 逻辑备份

大多数实例会进行逻辑备份, 如果 开启 GIPK 模式时, MySQL 8.0.30 版本的 `mysqldump` 提供的 `--skip-generated-invisible-primary-key` 选项会忽略 GIPK 信息。简单来说, `mysqldump` 时 不带该参数, 逻辑导出的数据会包含隐式主键, 如果带上该参数, 则不带隐式主键。

```

DROP TABLE IF EXISTS `t3`;
/*!40101 SET @saved_cs_client      = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `t3` (
  `id` int DEFAULT NULL,
  `c1` int DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
/*!40101 SET character_set_client = @saved_cs_client */;

--
-- Dumping data for table `t3`
--

LOCK TABLES `t3` WRITE;
/*!40000 ALTER TABLE `t3` DISABLE KEYS */;
INSERT INTO `t3` VALUES (1,1),(2,2),(3,3);
/*!40000 ALTER TABLE `t3` ENABLE KEYS */;
UNLOCK TABLES;
/*!40103 SET TIME_ZONE=@OLD_TIME_ZONE */;

```

mysqldump 加上 `--skip-generated-invisible-primary-key`

```

--
-- Table structure for table `t3`
--

DROP TABLE IF EXISTS `t3`;
/*!40101 SET @saved_cs_client      = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `t3` (
  `my_row_id` bigint unsigned NOT NULL AUTO_INCREMENT /*!80023 INVISIBLE */ ,
  `id` int DEFAULT NULL,
  `c1` int DEFAULT NULL,
  PRIMARY KEY (`my_row_id`)
) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
/*!40101 SET character_set_client = @saved_cs_client */;

--
-- Dumping data for table `t3`
--

LOCK TABLES `t3` WRITE;
/*!40000 ALTER TABLE `t3` DISABLE KEYS */;
INSERT INTO `t3` (`my_row_id`, `id`, `c1`) VALUES (1,1,1),(2,2,2),(3,3,3);
/*!40000 ALTER TABLE `t3` ENABLE KEYS */;
UNLOCK TABLES;

```

不带参数 skip-generated-invisible-primary-key

2.6 限制

只支持 InnoDB 存储引擎。

支持 row 模式复制，不支持 statement 模式复制。

my_row_id 成为系统关键字。

三、小结

总体而言，该特性绝对是强需求。毕竟林子大了，什么样的情况都可能会发生。运(chu)维(li)经(gu)验(zhang)比较丰富 DBA 而言，MySQL 数据库稳定性深受无主键之苦，对于自建场景尤其是没有审核流程的公司而言，该特性能提升数据库系统稳定性和安全性。

参考文档

<https://dev.mysql.com/doc/refman/8.0/en/create-table-gipks.html>

<https://dev.mysql.com/doc/refman/8.0/en/invisible-columns.html>

1.13 MySQL 8.0 在线调整 REDO

作者：杨涛涛

MySQL 最新版本 8.0.30 的发布带来一个与 REDO 日志文件有关的新功能点：在线调整 REDO 日志文件的大小！极大的简化了运维的工作量（经历过的同学都懂）！

通常一台 MySQL 实例部署完后，REDO 日志文件大小一般不会保持默认值，DBA 同学会根据数据的写入量以及频率来调整其为合适的值。与业务匹配的 REDO 日志文件大小能让数据库获得最佳的性能（如何让 REDO 日志文件的大小匹配现有业务不在本篇讨论范围）。

下面对 MySQL 8.0.30 之前以及之后的版本，对比 REDO 日志文件的更改过程，体验下最新版本的易用性。

8.0.30 之前，如何更改 REDO 日志文件大小？

针对这些版本，修改 REDO 日志文件大小的步骤比较繁琐。假设需要更改其大小为 2G，步骤如下：

1、REDO 日志文件的更改涉及两个传统参数：其最终大小是这两个参数的值相乘。

（1）`innodb_log_files_in_group`：REDO 日志磁盘上的文件个数，默认为 2。

（2）`innodb_log_file_size`：REDO 日志磁盘上单个文件的大小，默认为 48M。

（3）当前的日志大小为单个 48M，两个组，也就是一共 96M。

```
root@ytt-large:~/sandboxes/msb_5_7_34/data# ls -sihl ib_logfile*
3277012  48M -rw-r----- 1 root root  48M 7月 29 16:18 ib_logfile0
3277013  48M -rw-r----- 1 root root  48M 7月 29 16:18 ib_logfile1
```

2、“关闭”快速停实例参数：`innodb_fast_shutdown=0` 以确保 InnoDB 刷新所有脏页到磁盘（需要了解此参数的其他值请参见官方手册）：

```
<mysql:(none):5.7.34-log>set global innodb_fast_shutdown=0;
Query OK, 0 rows affected (0.00 sec)
```

3、等步骤二执行完后，停掉 MySQL 实例。

4、删掉数据目录下旧日志文件：

```
root@ytt-large:~/sandboxes/msb_5_7_34/data# rm -rf ib_logfile*
```

5、在配置文件 `my.cnf` 里修改参数 `innodb_log_file_size`，由于有两个组，设置这个参数为 1G 即可。

```
[mysqld]
innodb_log_file_size=1G
```

6、启动 MySQL 实例（如果没有报错，代表更改成功）。

7、查看新的日志文件大小：

```
root@ytt-large:~/sandboxes/msb_5_7_34/data# ls -sihl ib_logfile*
```

```
3277898 1.1G -rw-r----- 1 root root 1.0G 7月 29 16:31 ib_logfile0
3277923 1.1G -rw-r----- 1 root root 1.0G 7月 29 16:31 ib_logfile1
```

8.0.30 之后，如何更改 REDO 日志文件大小？

最新版本 MySQL 8.0.30 发布后，使用新参数 `innodb_redo_log_capacity` 来代替之前的两个参数（目前设置这两个参数依然有效）。使用新参数调整大小非常简单，直接设置为要调整的值就行。比如调整其大小为 2G：

调整之前，默认 100M：

```
<mysql:(none):8.0.30>select @@innodb_redo_log_capacity;
+-----+
| @@innodb_redo_log_capacity |
+-----+
|          104857600 |
+-----+
1 row in set (0.00 sec)
```

调整其大小为 2G：

```
<mysql:(none):8.0.30>set persist innodb_redo_log_capacity=2*1024*1024*1024;
Query OK, 0 rows affected (0.20 sec)
```

新增对应的状态变量 `innodb_redo_log_capacity_resized`，方便在 MySQL 侧监控当前 REDO 日志文件大小：

```
<mysql:(none):8.0.30>show status like 'innodb_redo_log_capacity_resized';
+-----+-----+
| Variable_name          | Value          |
+-----+-----+
| Innodb_redo_log_capacity_resized | 2147483648 |
+-----+-----+
1 row in set (0.00 sec)
```

同时磁盘文件的存储形式不再是类似 `ib_logfileN` 这样的文件，而是替代为 `#ib_redoN` 这样新文件形式。这些新的文件默认存储在数据目录下的子目录 `#innodb_redo` 里。

1、这样的文件一共有 32 个，按照参数 `innodb_redo_log_capacity` 来平均分配。

```
root@ytt-large:/var/lib/mysql/#innodb_redo# ls |wc -l
32
```

2、有两类文件：一类是不带 `_tmp` 后缀的，代表正在使用的日志文件；带 `_tmp` 后缀的代表多余的日志文件，等正在使用的文件写满后，再接着使用它。

如下所示：正在使用的日志文件有 15 个，未使用的有 17 个。

```
root@ytt-large:/var/lib/mysql/#innodb_redo# ls | grep -v '_tmp' |wc -l
15
50
```

```
root@ytt-large:/var/lib/mysql/#innodb_redo# ls | grep '_tmp' | wc -l
17
```

同时 `performance_schema` 库里新增表 `innodb_redo_log_files`：获取当前使用的 REDO 日志文件 LSN 区间、实际写入大小、是否已满等统计数据。例如当前 15 个 REDO 日志文件的统计数据如下：一目了然！

```
<mysql:performance_schema:8.0.30>select * from innodb_redo_log_files;
+-----+-----+-----+-----+-----+-----+-----+
| FILE_ID | FILE_NAME                | START_LSN | END_LSN   | SIZE_IN_BYTES | IS_FULL | CONSUMER_LEVEL |
+-----+-----+-----+-----+-----+-----+-----+
| 7 | ./#innodb_redo/#ib_redo7 | 552208896 | 619315712 | 67108864 | 1 | 0 |
...
| 21 | ./#innodb_redo/#ib_redo21 | 1491704320 | 1558811136 | 67108864 | 0 | 0 |
+-----+-----+-----+-----+-----+-----+-----+
15 rows in set (0.00 sec)
```

总结：

MySQL 8.0 新版本带来越来越多的功能点，来简化开发和运维的工作，如果可能请尽快升级吧。

1.14 MySQL 8.0 的交集和差集介绍

作者：杨涛涛

MySQL 8.0 最新小版本（8.0.31）支持标准 SQL 的 `intersect`（交集）和 `except`（差集）操作。

交集：也就是返回两个结果集的相交部分，也即左侧和右侧同时存在的记录。

差集：也就是返回两个结果集中左侧存在同时右侧不存在的记录。

之前在做其他数据库往 MySQL 迁移的时候，经常遇到这样的操作。由于 MySQL 一直以来不支持这两类操作符，一般得想办法避开或者是通过其他方法来实现。

比如在 MySQL 5.7.x 中，想要实现如下两个需求：

第一、求表 `t1` 和表 `t2` 的交集，并且结果要去重；

第二、求表 `t1` 和表 `t2` 的差集，并且结果也要去重。

简单创建表 `t1`、表 `t2`，并且插入几条样例数据：

```
<mysql:5.7.34:(ytt)> create table t1(c1 int);
Query OK, 0 rows affected (0.02 sec)
```

```

<mysql:5.7.34:(ytt)> create table t2 like t1;
Query OK, 0 rows affected (0.02 sec)

<mysql:5.7.34:(ytt)> insert t1 values (10),(20),(20),(30),(40),(40),(50);
Query OK, 7 rows affected (0.00 sec)
Records: 7 Duplicates: 0 Warnings: 0

<mysql:5.7.34:(ytt)> insert t2 values (10),(30),(30),(50),(50),(70),(90);
Query OK, 7 rows affected (0.02 sec)
Records: 7 Duplicates: 0 Warnings: 0
<mysql:5.7.34:(ytt)> select * from t1;
+-----+
| c1 |
+-----+
| 10 |
| 20 |
| 20 |
| 30 |
| 40 |
| 40 |
| 50 |
+-----+
7 rows in set (0.00 sec)

<mysql:5.7.34:(ytt)> select * from t2;
+-----+
| c1 |
+-----+
| 10 |
| 30 |
| 30 |
| 50 |
| 50 |
| 70 |
| 90 |
+-----+
7 rows in set (0.00 sec)

```

我们来实现这两个需求：

1. 求去重后的交集：两表内联、去重！

```

<mysql:5.7.34:(ytt)> select distinct t1.c1 from t1 join t2 using(c1);
+-----+
| c1 |
+-----+

```



```
+-----+
|  10 |
|  30 |
|  50 |
+-----+
3 rows in set (0.00 sec)
```

2. 求去重后的差集：两表左外联，去重，并且保留右表关联键为 NULL 的记录。

```
<mysql:5.7.34:(ytt)> select distinct t1.c1 from t1 left join t2 using(c1) where t2.c1
is null;
+-----+
| c1 |
+-----+
| 20 |
| 40 |
+-----+
2 rows in set (0.00 sec)
```

在最新版本 MySQL 8.0.31 中，直接用 intersect 和 except 两个新操作符即可，写起来非常简单。

创建好同样的表结构和数据，用 intersect 来求交集：

```
<mysql:8.0.31:(ytt)>table t1 intersect table t2;
+-----+
| c1 |
+-----+
|  10 |
|  30 |
|  50 |
+-----+
3 rows in set (0.00 sec)
```

用 except 来求差集：

```
<mysql:8.0.31:(ytt)>table t1 except table t2;
+-----+
| c1 |
+-----+
|  20 |
|  40 |
+-----+
2 rows in set (0.00 sec)
```

intersect 和 except 操作符默认去重。比如需要保留原始结果，则可以带上 all 关键词：如下求两表差集的结果会保留所有符合条件的记录。

```
<mysql:8.0.31:(ytt)>table t1 except all table t2;
+-----+
```

```
| c1 |
+-----+
| 20 |
| 20 |
| 40 |
| 40 |
+-----+
4 rows in set (0.00 sec)
```

1.15 MySQL 8.0 对 limit 的优化

作者：杨奇龙

一、前言

提到 limit 优化，大多数 MySQL DBA 都不会陌生，能想到各种应对策略，比如延迟关联，书签式查询等等，之前我也写过一篇优化的文章：<https://mp.weixin.qq.com/s/2nSdHiGsu35Yf3fRStzumQ>，有兴趣的朋友可以复习一下。

二、MySQL 8.0 对 limit 的改进

对于 limit N 带有 group by，order by 的 SQL 语句 (order by 和 group by 的字段有索引可以使用)，MySQL 优化器会尽可能选择利用现有索引的有序性，减少排序--这看起来是 SQL 的执行计划的最优解，但是实际上效果其实是南辕北辙，相信很多 DBA 遇到的相关案例中 sql 执行计划选择 order by id 的索引进而导致全表扫描，而不是利用 where 条件中的索引查找过滤数据。MySQL 8.0.21 版本之前，并没有什么参数来控制这种行为，但是自 MySQL 8.0.21 之后提供一个优化器参数 `prefer_ordering_index`，通过设置 `optimizer_switch` 来开启或者关闭该特性。比如：

```
SET optimizer_switch = "prefer_ordering_index=off";

SET optimizer_switch = "prefer_ordering_index=on";
```

三、实践出真知

测试环境 MySQL 社区版 8.0.30

构造测试数据

```
CREATE TABLE t (
id1 BIGINT NOT NULL PRIMARY KEY auto_increment,
```

```

id2 BIGINT NOT NULL,
c1 VARCHAR(50) NOT NULL,
c2 varchar(50) not null,
INDEX i (id2, c1));

insert into t(id2,c1,c2) values(1,'a','xfvs'),(2,'bbbb','xfvs'),(3,'cddd','xfvs'),(4,'
dfdf','xfvs'),(12,'bbbb','xfvs'),(23,'cddd','xfvs'),(14,'dfdf','xfvs'),
(11,'bbbb','xfvs'),(13,'cddd','xfvs'),(44,'dfdf','xfvs'),(31,'bbbb','xfvs'),(33,'cddd
','xfvs'),(34,'dfdf','xfvs');

```

3.1 默认开启参数

```

mysql (test) > SELECT @@optimizer_switch LIKE '%prefer_ordering_index=on%';
+-----+
| @@optimizer_switch LIKE '%prefer_ordering_index=on%' |
+-----+
|                                                    1 |
+-----+
1 row in set (0.00 sec)

```

查询非索引字段，id2 上有索引，order by 主键 id1，explain 查看执行计划 type index 说明使用索引扫描使用 using where 过滤结果集。这个是优化器的自以为的最优选择，但是实际上遇到数据集合比较大的表，该执行计划就不是最优解，反而导致慢查。

```

mysql (test) > explain select c2 from t where id2>8 ORDER BY id1 ASC LIMIT 2\G
***** 1. row *****
      id: 1
  select_type: SIMPLE
        table: t
  partitions: NULL
         type: index
possible_keys: i
          key: PRIMARY
        key_len: 8
          ref: NULL
         rows: 2
    filtered: 69.23
      Extra: Using where
1 row in set, 1 warning (0.00 sec)

```

3.2 关闭该参数

```

mysql (test) > SET optimizer_switch = "prefer_ordering_index=off";

mysql (test) > explain select c2 from t where id2>8 ORDER BY id1 ASC LIMIT 2\G
***** 1. row *****

```

```
      id: 1
select_type: SIMPLE
      table: t
partitions: NULL
      type: range
possible_keys: i
      key: i
      key_len: 8
      ref: NULL
      rows: 9
      filtered: 100.00
      Extra: Using index condition; Using filesort
1 row in set, 1 warning (0.00 sec)
```

经过调整之后，查看执行计划发现优化器选择 `id2` 索引字段找到记录做过滤，并且使用了 ICP 特性，减少物理 io 请求，而不是选择使用主键 `id1` 遍历索引然后回表查询。显然通过人为介入参数调整优化器的行为能带来更好的优化效果。

四、总结

从不同版本的 MySQL 发展轨迹来看 MySQL 的优化器越来越智能（比如大家期待已久的直方图特性），能更多的减少人为干预，提升执行计划的准确性。

have fun with MySQL 8.0 ^_^

第 2 章 技术分享

本系列的文章是全书最多的一个部分，共收录 33 篇。内容涉及到很多好用的 MySQL 工具。每一篇文章都围绕这一个知识点展开，有大量的代码、操作命令和配图说明。在这里，优秀的作者们将技术分享给读者们。

2.1 MySQL 的 TIMESTAMP 类型字段非空和默认值属性的影响

作者：刘晨

前几天读了一篇文章《故障分析 | MySQL 迁移后 timestamp 列 cannot be null》，没想到这两天就碰到了相近的问题。

同事说他通过某款商业数据同步软件将一个 MySQL 5.7.28 的库同步到 MySQL 5.7.20 的库时，如果表中含有 TIMESTAMP 数据类型、缺省值为 `current_timestamp` 的字段，这些表的同步任务就都失败了，而另外的一些包含了 DATETIME 数据类型的表就同步成功了，不知道这是不是 MySQL 版本差异导致的？

不通过软件，直接手工创建，不会报错，模拟的 SQL，如下所示，一个主键 `id`，外加两个 `timestamp` 类型的字段，都设置了默认值：

```
create table test(  
id int not null auto_increment,  
createtime timestamp null default current_timestamp,  
updatetime timestamp null default current_timestamp on update current_timestamp  
);
```

同步软件报错的日志如下，提示为字段 `updatetime` 设置了无效的默认值，

```
ERROR_GENERAL "Handling new table 'test'. 'flow' failed  
execute create table statement failed, statement CREATE TABLE `test`.`test` ( `id` INT  
NOT NULL, `createtime` TIMESTAMP, `updatetime` TIMESTAMP ) COLLATE utf8_general_ci  
[MySQL][ODBC 5.3(w) Driver][mysqld-5.7.20-log]Invalid default value for 'updatetime'
```

细致的同学，可能会注意到，日志中记录的 SQL 语句显示 `createtime` 和 `updatetime` 都只声明了 `TIMESTAMP` 类型，缺少了原始建表语句中的 `NULL` 和 `DEFAULT` 属性，这会有什么影响？

《故障分析 | MySQL 迁移后 timestamp 列 cannot be null》这篇文章其实介绍过，MySQL 中有个 `explicit_defaults_for_timestamp` 变量，他决定了 MySQL 是否为 `TIMESTAMP` 列的默认值和 `NULL` 值的处理启用某些非标准的行为：

- explicit_defaults_for_timestamp

Command-Line Format	--explicit-defaults-for-timestamp[={OFF ON}]
Deprecated	Yes
System Variable	explicit_defaults_for_timestamp
Scope	Global, Session
Dynamic	Yes
Type	Boolean
Default Value	OFF

<https://dev.mysql.com/doc/refman/5.7/en/server-system-variables.html>

如文档所说，如果 `explicit_defaults_for_timestamp=OFF`，服务器会启用非标准行为，并按以下规则处理 `TIMESTAMP` 列：

- (1) 没有明确使用 `NULL` 属性声明的 `TIMESTAMP` 列会自动使用 `NOT NULL` 属性声明。给这样的列分配一个 `NULL` 的值是允许的，并将该列设置为 `current timestamp`。
- (2) 表中的第一个 `TIMESTAMP` 列，如果没有明确地用 `NULL` 属性，`DEFAULT` 属性或 `ON UPDATE` 属性声明，将自动用 `DEFAULT CURRENT_TIMESTAMP` 和 `ON UPDATE CURRENT_TIMESTAMP` 属性声明。
- (3) 在第一个列之后的 `TIMESTAMP` 列，如果没有明确地用 `NULL` 属性或明确的 `DEFAULT` 属性来声明，就会自动声明为 `DEFAULT '0000-00-00 00:00:00'`。对于插入的行，如果没有为该列指定明确的值，那么该列将被分配为 `'0000-00-00 00:00:00'`，并且不会发生警告。根据是否启用了严格的 `SQL mode` 或包含 `NO_ZERO_DATE` 的 `SQL mode`，默认值 `'0000-00-00 00:00:00'` 可能是不被允许的。

目标库 MySQL 5.7.20 的 `explicit_defaults_for_timestamp` 值是默认的 `OFF`，结合上述规则，就可以模拟复现上述问题了。

虽然原始的建表语句中 `TIMESTAMP` 类型字段包含了 `NULL` 和 `DEFAULT` 属性，但是根据同步软件的错误提示，很明显，不知道什么原因，他在执行的时候忽略了这两个属性，导致真实的执行语句是：

```
create table test(id int not null, createtime timestamp, updatetime timestamp);
```

我们在客户端，执行上述 SQL，就会得到相同的错误信息：

```
Invalid default value for 'updatetime'
```

建表语句第一个 `TIMESTAMP` 类型字段是 `createtime`，按照上述规则(1)，没有明确使用 `NULL` 属性声明的 `TIMESTAMP` 列会自动使用 `NOT NULL` 属性声明，按照上述规则(2)，表中的第一个 `TIMESTAMP` 列，如果没有明确地用 `NULL` 属性，`DEFAULT` 属性或 `ON UPDATE` 属性声明，将自动用 `DEFAULT CURRENT_TIMESTAMP` 和 `ON UPDATE CURRENT_TIMESTAMP` 属性声明，假设这个建表语句中只有一个 `TIMESTAMP` 类型字段 `createtime`，他的结构会改成：

字段名	#	数据类型	非空	自增	键	默认	额外的
123id	1	int(11)	[]	[]			
createtime	2	timestamp	[X]	[]		CURRENT_TIMESTAMP	on update CURRENT_TIMESTAMP

按照上述规则(3)，在第一个列之后的 `TIMESTAMP` 列，即此处的 `updatetime`，如果没有明确地用 `NULL` 属性或明确的 `DEFAULT` 属性来声明，就会自动声明为 `DEFAULT '0000-00-00 00:00:00'`，但是这个 MySQL 5.7.20 的 `sql_mode` 变量包含了 `NO_ZERO_DATE` 规则，因此默认值 `'0000-00-00 00:00:00'` 不被允许，所以提示了错误：

```
Invalid default value for 'updatetime'
```

我们能推断，如果表中存在两个及以上这种情况的 `TIMESTAMP` 类型字段，通过这个软件做同步，建表的时候，就会报错，这个可能是软件的一个 `bug`，或者存在其他的配置控制，但就不在数据库的范畴了。

另外，多说一点，原始语句中 `createtime` 和 `updatetime` 列都指定了默认值，但还是设置 `NULL` 属性，这其实就有些矛盾了，或者说是设计上的不严谨，从规范设计开发的角度，还是应该避免的：

```
create table test(
id int not null auto_increment,
createtime timestamp null default current_timestamp,
updatetime timestamp null default current_timestamp on update current_timestamp
);
```

2.2 MySQL：你的 `binlog_expire_logs_seconds` 可能正在失效

作者：胡呈清

如果你正在使用 MySQL 8.0，并且在使用物理热备工具，那么 `binlog_expire_logs_seconds` 可能不会如你预想的那样生效。

`binlog_expire_logs_seconds`

为了防止 `binlog` 文件过大导致无可用的磁盘空间，MySQL 提供了一个系统变量用来配置过期时间，MySQL 5.7 时变量名为 `expire_logs_days`，精确度为天；MySQL 8.0 使用 `binlog_expire_logs_seconds` 来控制，其效果和名字的变化一样，精确度由天变成了秒。超过这个时间的 `binlog` 会被自动清理，自动清理的触发时机为（注意：并不是以每秒这样的固定频率检查是否有过期日志）：

1. MySQL 启动时；
2. `binlog` 刷新时。

MySQL 启动不用多说，`binlog` 刷新又分两种情况：

1. 当前的 binlog 大小达到 `max_binlog_size`, 写完当前事务的日志后, 就会生成一个新的 binlog;
2. 手工执行了 `flush [binary] logs;` 命令后, 也会生成一个新的 binlog。

下面我们来看一个 `binlog_expire_logs_seconds` 失效的场景。

失效场景&复现步骤

1. 设置 `set global binlog_expire_logs_seconds=60`, 然后执行几次 `flush binary logs` 生成多个 binlog (这样我们 60s 之后就可以看到过期的 binlog 是否被自动删除了):

```
mysql> show global variables like '%expire%';
+-----+
| Variable_name | Value |
+-----+
| binlog_expire_logs_seconds | 60 |
| disconnect_on_expired_password | ON |
| expire_logs_days | 0 |
+-----+
3 rows in set (0.00 sec)

mysql> show master logs;
+-----+
| Log_name | File_size | Encrypted |
+-----+
| mysql-bin.000026 | 242 | No |
| mysql-bin.000027 | 242 | No |
| mysql-bin.000028 | 242 | No |
| mysql-bin.000029 | 242 | No |
| mysql-bin.000030 | 242 | No |
| mysql-bin.000031 | 242 | No |
| mysql-bin.000032 | 242 | No |
| mysql-bin.000033 | 195 | No |
+-----+
8 rows in set (0.00 sec)
```

```
[root@localhost 3308]#
[root@localhost 3308]#
[root@localhost 3308]#
[root@localhost 3308]#
[root@localhost 3308]#
[root@localhost 3308]# ll mysql-bin.*
-rw-r----- 1 mysql mysql 242 9月 1 15:24 mysql-bin.000026
-rw-r----- 1 mysql mysql 242 9月 1 15:24 mysql-bin.000027
-rw-r----- 1 mysql mysql 242 9月 1 15:24 mysql-bin.000028
-rw-r----- 1 mysql mysql 242 9月 1 15:24 mysql-bin.000029
-rw-r----- 1 mysql mysql 242 9月 1 15:24 mysql-bin.000030
-rw-r----- 1 mysql mysql 242 9月 1 15:24 mysql-bin.000031
-rw-r----- 1 mysql mysql 242 9月 1 15:24 mysql-bin.000032
-rw-r----- 1 mysql mysql 195 9月 1 15:24 mysql-bin.000033
-rw-r----- 1 mysql mysql 152 9月 1 15:24 mysql-bin.index
[root@localhost 3308]#
```

2. 等 60s 后, 运行 `xtrabackup` 进行备份 (我这里直接调的脚本, 手工命令完全无影响):

```
[root@localhost backup_script]# /data/packet/backup_script/xtrabackupScript_v2.7.sh full
2021-09-01 15:27:04 [INFO] =====> FULL BACKUP FLAG
2021-09-01 15:27:04 [INFO] /data/backup 备份目录已存在,无需创建
2021-09-01 15:27:04 [INFO] 连接MySQL数据库成功,MySQL版本为: 8.0.18 MySQL Community Server - GPL
2021-09-01 15:27:04 [INFO] Xtrabackup 已安装,当前版本为: 8.0.23-16
2021-09-01 15:27:04 [INFO] Xtrabackup 大版本为 [8.0],与MySQL大版本 [8.0]兼容
2021-09-01 15:27:04 [INFO] Xtrabackup 依赖包 qpress 当前已安装
2021-09-01 15:27:06 [INFO] Xtrabackup 依赖包 libev 当前已安装
2021-09-01 15:27:06 [INFO] /data/backup 备份目录已存在,无需创建
2021-09-01 15:27:06 [INFO] 创建全量备份目录: /data/backup/percona-backup-2021-09-01_152704
2021-09-01 15:27:06 [INFO] 全量备份 操作开始,请等待...
2021-09-01 15:27:13 [INFO] 全量备份 操作完成
2021-09-01 15:27:13 [INFO] 备份文件保存在: /data/backup/percona-backup-2021-09-01_152704
2021-09-01 15:27:13 [INFO] 本次全量备份大小为 16234191 bytes
2021-09-01 15:27:13 [INFO] 开始备份MySQL参数配置文件 /data/mysql/base/mysql-8.0.18/my.cnf.3308
2021-09-01 15:27:13 [INFO] 参数文件备份成功,保存在 /data/backup/percona-backup-2021-09-01_152704/my.cnf
2021-09-01 15:27:13 [INFO] 开始生成全量备份索引文件供增量备份使用
2021-09-01 15:27:13 [INFO] 备份索引创建文件成功,保存在 /data/backup/percona-backup-2021-09-01_152704/index
2021-09-01 15:27:13 [INFO] 拷贝备份临时目录的元数据文件到全量备份目录
2021-09-01 15:27:13 [INFO] 对备份集做MD5校验,生成校验码
2021-09-01 15:27:13 [INFO] 本次全量备份大小为 16234191 bytes
2021-09-01 15:27:13 [INFO] 生成校验码成功,保存在 /data/backup/percona-backup-2021-09-01_152704/md5sum.base
2021-09-01 15:27:13 [INFO] 以下备份文件即将过期,可手工删除 [默认保留30天]
```

3. 备份完之后, 可以看到刷新了 binlog (xtrabackup 运行过程中有一个步骤会执行 `FLUSH NO_WRITE_TO_BINLOG BINARY LOGS` 刷新 binlog), 但是没有删除 60s 之前的 binlog:

```
mysql> show master logs;
+-----+
| Log_name | File_size | Encrypted |
+-----+
| mysql-bin.000026 | 242 | No |
| mysql-bin.000027 | 242 | No |
| mysql-bin.000028 | 242 | No |
| mysql-bin.000029 | 242 | No |
| mysql-bin.000030 | 242 | No |
| mysql-bin.000031 | 242 | No |
| mysql-bin.000032 | 242 | No |
| mysql-bin.000033 | 242 | No |
| mysql-bin.000034 | 195 | No |
+-----+
9 rows in set (0.00 sec)
```

```
[root@localhost 3308]#
[root@localhost 3308]#
[root@localhost 3308]#
[root@localhost 3308]# ll mysql-bin.*
-rw-r----- 1 mysql mysql 242 9月 1 15:24 mysql-bin.000026
-rw-r----- 1 mysql mysql 242 9月 1 15:24 mysql-bin.000027
-rw-r----- 1 mysql mysql 242 9月 1 15:24 mysql-bin.000028
-rw-r----- 1 mysql mysql 242 9月 1 15:24 mysql-bin.000029
-rw-r----- 1 mysql mysql 242 9月 1 15:24 mysql-bin.000030
-rw-r----- 1 mysql mysql 242 9月 1 15:24 mysql-bin.000031
-rw-r----- 1 mysql mysql 242 9月 1 15:24 mysql-bin.000032
-rw-r----- 1 mysql mysql 242 9月 1 15:27 mysql-bin.000033
-rw-r----- 1 mysql mysql 195 9月 1 15:27 mysql-bin.000034
-rw-r----- 1 mysql mysql 171 9月 1 15:27 mysql-bin.index
[root@localhost 3308]#
```


这是因为 MySQL8.0 为了解决备份时的全局锁问题，新引入了 `LOCK INSTANCE FOR BACKUP` 备份锁，而这把锁恰好导致了 `binlog_expire_logs_seconds` 的失效，下面两张图说明这个问题：

```
mysql> LOCK INSTANCE FOR BACKUP;
Query OK, 0 rows affected (0.00 sec)

mysql> show master logs;
+-----+-----+-----+
| Log_name          | File_size | Encrypted |
+-----+-----+-----+
| mysql-bin.000026 | 242       | No       |
| mysql-bin.000027 | 242       | No       |
| mysql-bin.000028 | 242       | No       |
| mysql-bin.000029 | 242       | No       |
| mysql-bin.000030 | 242       | No       |
| mysql-bin.000031 | 242       | No       |
| mysql-bin.000032 | 242       | No       |
| mysql-bin.000033 | 242       | No       |
| mysql-bin.000034 | 242       | No       |
| mysql-bin.000035 | 242       | No       |
| mysql-bin.000036 | 195       | No       |
+-----+-----+-----+
11 rows in set (0.00 sec)

mysql> FLUSH NO_WRITE_TO_BINLOG BINARY LOGS;
Query OK, 0 rows affected (0.00 sec)

mysql> show master logs;
+-----+-----+-----+
| Log_name          | File_size | Encrypted |
+-----+-----+-----+
| mysql-bin.000026 | 242       | No       |
| mysql-bin.000027 | 242       | No       |
| mysql-bin.000028 | 242       | No       |
| mysql-bin.000029 | 242       | No       |
| mysql-bin.000030 | 242       | No       |
| mysql-bin.000031 | 242       | No       |
| mysql-bin.000032 | 242       | No       |
| mysql-bin.000033 | 242       | No       |
| mysql-bin.000034 | 242       | No       |
| mysql-bin.000035 | 242       | No       |
| mysql-bin.000036 | 242       | No       |
| mysql-bin.000037 | 195       | No       |
+-----+-----+-----+
12 rows in set (0.00 sec)
```

释放锁再测试，可以正常触发过期 binlog 的自动删除：

```
mysql> UNLOCK INSTANCE;
Query OK, 0 rows affected (0.00 sec)

mysql> show master logs;
+-----+-----+-----+
| Log_name          | File_size | Encrypted |
+-----+-----+-----+
| mysql-bin.000026 |      242 | No        |
| mysql-bin.000027 |      242 | No        |
| mysql-bin.000028 |      242 | No        |
| mysql-bin.000029 |      242 | No        |
| mysql-bin.000030 |      242 | No        |
| mysql-bin.000031 |      242 | No        |
| mysql-bin.000032 |      242 | No        |
| mysql-bin.000033 |      242 | No        |
| mysql-bin.000034 |      242 | No        |
| mysql-bin.000035 |      242 | No        |
| mysql-bin.000036 |      242 | No        |
| mysql-bin.000037 |      195 | No        |
+-----+-----+-----+
12 rows in set (0.00 sec)

mysql> FLUSH NO_WRITE_TO_BINLOG BINARY LOGS;
Query OK, 0 rows affected (0.00 sec)

mysql> show master logs;
+-----+-----+-----+
| Log_name          | File_size | Encrypted |
+-----+-----+-----+
| mysql-bin.000037 |      242 | No        |
| mysql-bin.000038 |      195 | No        |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

总结

如果 MySQL 每天的数据修改很少，产生的 binlog 很小，而 `max_binlog_size` 设置很大。每次在达到单个 binlog 的最大大小前，执行定时任务调用 xtrabackup 备份，备份时加的备份锁 `LOCK INSTANCE FOR BACKUP` 和 `FLUSH NO_WRITE_TO_BINLOG BINARY LOGS` 会导致：binlog 刷新了，但是无法自动删除过期的 binlog。新的 binlog 写一天没有达到最大大小，又进行备份，每天循环这个逻辑，导致过期的 binlog 越来越多，一直无法被自动删除。

当然，如果你使用的是 MySQL5.7，那并不会会有这个问题，虽然 MySQL5.7 备份时会加全局锁，但是并不影响过期 binlog 的自动删除。从这个角度看，这是个 bug，所以报给官方后很快被确认了：

<https://bugs.mysql.com/bug.php?id=104785>

等待修复的过程是漫长的，如果你恰好遇见了这个冷门的 bug，可以把 `max_binlog_size` 调小，保证在备份前 binlog 就能够达到最大大小，自然的刷新可以正常触发自动删除。

2.3 MySQL 索引长度限制的案例

作者：刘晨

前两天同事提了一个问题，MySQL 5.7 中给某张表字段增加一个单键值索引，提示了如下错误，

 SQL 错误 [1071] [42000]: Specified key was too long; max key length is 1000 bytes

一些背景信息：

1. 表设置的字符集，utf8mb4 。
2. 表的存储引擎，MyISAM 。
3. 表的数据量，1 个亿 。
4. 执行的 SQL，select c1, c2 from test where c1 = :1，c1 数据类型是 varchar(255)，区分度较高，需要创建一个单键值索引。

我们知道，MySQL 和 Oracle 在索引上最大的一个区别，就是索引存在长度的限制。如果是超长键值，可以支持创建前缀的索引，顾名思义，取这个字段的前多少个字符/字节作为索引的键值。

之所以可以定义一个字段前缀作为键值，存储效率是考虑的一个因素，如果列名的前 10 个字符通常都是不同的，检索这 10 个字符创建的索引应该会比检索整个列作为索引的效率更高，使用列前缀作为索引会让索引树更小，不仅节省空间，还可能提高插入操作的速度。

对于非二进制的字符串类型(CHAR、VARCHAR、TEXT)，前缀会按照字符个数计算，对二进制的字符串类型(BINARY、VARBINARY、BLOB)，前缀会按照字节个数计算，因此，当对非二进制的字符串列明确前缀长度的时候，需要考虑多字节字符集的因素。

MySQL 官方手册索引的章节提到了，前缀索引长度限制是和引擎相关的，如果用的是 InnoDB，前缀上限是 767 字节，当启用 innodb_large_prefix 时，上限可以达到 3072 字节。如果用的是 MyISAM，前缀上限是 1000 字节，这正是上述创建索引错误提示的内容，

Prefix support and lengths of prefixes (where supported) are storage engine dependent. For example, a prefix can be up to 767 bytes long for InnoDB tables or 3072 bytes if the innodb_large_prefix option is enabled. For MyISAM tables, the prefix length limit is 1000 bytes.

<https://dev.mysql.com/doc/refman/5.7/en/create-index.html>

utf8mb4 是 MySQL 5.5.3 之后增加的编码，mb4 就是 most bytes 4 的意思，专门用来兼容四字节的 unicode，utf8mb4 是 utf8 的超集。之所以需要 utf8mb4，是因为之前的 utf8 编码最大字符长度为 3 字节，如果遇到 4 字节的宽字符就会插入异常了。三个字节的 UTF-8 最大能编码的 Unicode 字符是 0xffff，也就是 Unicode 中的基本多文种平面(BMP)。因此，任何不在基本多文种平面的 Unicode 字符，都无法使用 MySQL 的 utf8 字符集存储，这就包括 Emoji 表情(Emoji 是一种特殊的 Unicode 编码，常见于手机上)，和很多不常用的汉字，以及任何新增的 Unicode 字符等(这些都是 utf8 的缺点)。

InnoDB，如果需要建索引，就不能超过 767bytes，utf8 编码，2553=765 bytes，是能建索引情况下的最大值，utf8mb4 编码，默认字符长度则应该是 767 除以 4 向下取整，就是 191。如果设置了 innodb_large_prefix，最大长度是 3072 字节，utf8 编码，10243=3072 bytes，utf8mb4 编码，768*4=3072。

2.3 MySQL 索引长度限制的案例

MyISAM ，如果需要建索引，就不能超过 1000bytes ， utf8 编码， $333*3=999$ bytes ，是能建索引情况下的最大值，utf8mb4 编码，默认字符长度则应该是 1000 除以 4，就是 250。

我们可以测下 MyISAM ， utf8mb4 编码表 test1 ， 250 长度的字段 c1 ， 251 长度的字段 c2 ，

```
CREATE TABLE test1 (c1 varchar(250), c2 varchar(251)) ENGINE=MyISAM DEFAULT
CHARSET=utf8mb4 COLLATE=utf8mb4_bin;
```

c1 能创建索引，c2 创建出错，utf8mb4 编码，varchar 最大限制 250 个字符，

```
/* 可执行 */
create index idx_test1_01 on test1(c1);
/*SQL 错误 [1071] [42000]: Specified key was too long; max key length is 1000 bytes*/
create index idx_test1_02 on test1(c2);
```

utf8 编码表 test2 ， 333 长度的字段 c1 ， 334 长度的字段 c2 ，

```
CREATE TABLE test2 (c1 varchar(333), c2 varchar(334)) ENGINE=MyISAM DEFAULT
CHARSET=utf8 COLLATE=utf8_bin;
```

c1 能创建索引，c2 创建出错，utf8 编码，varchar 最大限制 333 个字符，

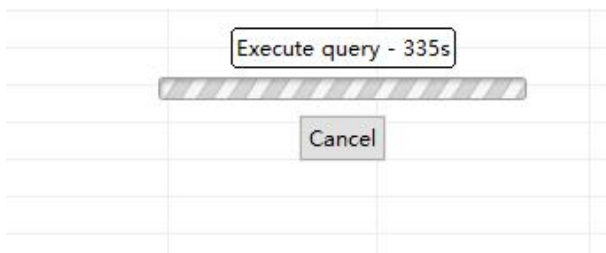
```
/* 可执行 */
create index idx_test2_01 on test2(c1);
/*SQL 错误 [1071] [42000]: Specified key was too long; max key length is 1000 bytes*/
create index idx_test2_02 on test2(c2);
```

其他场景的测试相近，不再展示。

因此，针对这个问题，可以为 c1 字段创建一个前缀索引，

```
create index idx_test_01 on test(c1(250(含)以下的值))
```

但是 1 亿数据量，创建非常慢，



通过沟通，了解到这个需求是检索今年的数据，表中存在时间戳的字段，且今年的数据，是 1000 多万，如果能改应用 SQL ，或者只是通过手工执行 SQL 的前提下，可以有几种解决方案：

1. 如果从原表检索，可以创建一个 c1 和时间戳字段的复合索引，利用索引扫描，定位所需数据。
2. 如果不需要从原表检索，可以使用时间戳作为条件，通过 CTAS ，创建一张今年数据的表，通过 c1 单键值索引，即可定位数据。

2.4 MySQL:一文看懂时区&time_zone

作者：胡呈清

你还在被以下问题困扰吗：

MySQL 的安装规范中应该设置什么时区？

JAVA 应用读取到的时间和北京时间差了 14 个小时，为什么？怎么解决？

已经运行一段时间的业务，修改 MySQL 的时区会影响已经存储的时间类型数据吗？

迁移数据时会有导致时间类型数据时区错误的可能吗？

...

看完这篇文章，你能解决上面所有的疑惑。首先出场的是和时区相关的启动参数和系统变量。

启动参数&系统变量

如果要在 MySQL 启动时就指定时区，则应该使用启动参数：`default-time-zone`，示例：

```
--方法 1：在启动命令中添加
mysqld --default-time-zone='+08:00' &

--方法 2：在配置文件中添加
[mysqld]
default-time-zone='+08:00'
```

启动后我们可以看到控制时区的系统变量，其中 `time_zone` 变量控制时区，在 MySQL 运行时可以通过 `set` 命令修改（注意：不可以写在 `my.cnf` 中）：

```
--查看
mysql> show global variables like '%time%zone%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| system_time_zone | CST |
| time_zone | +08:00 |
+-----+-----+
2 rows in set (0.00 sec)

--修改全局时区，所有新创建的 session 都会被修改
set global time_zone='+00:00';

--修改当前 session 的时区
set session time_zone='+00:00';
```

启动参数和系统变量的可用值遵循相同的格式：

- 'SYSTEM' 表明使用系统时间
- 相对于 UTC 时间的偏移, 比如 '+08:00' 或者 '-6:00'
- 某个时区的名字, 比如 'Europe/Helsinki', 'Asia/Shanghai' 或 'UTC', 前提是已经把时区信息导入到了 mysql 库, 否则会报错。导入方法: `mysql_tzinfo_to_sql /usr/share/zoneinfo | mysql -S /tmp/mysql.sock mysql`

`system_time_zone` 变量只有全局值没有会话值, 不能动态修改, MySQL 启动时, 将尝试自动确定服务器的时区, 并使用它来设置 `system_time_zone` 系统变量, 此后该值不变。当 `time_zone='system'` 时, 就是使用的这个时区, 示例中 `time_zone` 就是 CST, 而 CST 在 RedHat 上就是东八区:

```
mysql> show global variables like '%time%zone%';
```

```
+-----+-----+
| Variable_name | Value |
+-----+-----+
| system_time_zone | CST |
| time_zone | SYSTEM |
+-----+-----+
2 rows in set (0.00 sec)
```

-- 可以看到在当前操作系统上 CST 就是 +08:00 时区

```
[root@localhost ~]# date -R
```

```
Thu, 02 Dec 2021 17:41:46 +0800
```

```
[root@localhost ~]# date
```

```
2021 年 12 月 02 日 星期四 17:41:49 CST
```

时区影响了什么

概括一下就两点:

1. NOW() 和 CURTIME() 系统函数的返回值受当前 session 的时区影响

不仅是 select now(), 包括 insert .. values(now())、以及字段的 DEFAULT CURRENT_TIMESTAMP 属性也受此影响:

```
mysql> set time_zone='+00:00';
```

```
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> select now(),CURTIME();
```

```
+-----+-----+
| now() | CURTIME() |
+-----+-----+
| 2021-12-02 08:45:33 | 08:45:33 |
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> set time_zone='+08:00';
```

```
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> select now(),CURTIME();
```

```
+-----+-----+
| now() | CURTIME() |
+-----+-----+
| 2021-12-02 16:45:39 | 16:45:39 |
+-----+-----+
1 row in set (0.00 sec)
```

2. timestamp 数据类型字段存储的数据受时区影响

timestamp 数据类型会存储当时 session 的时区信息，读取时会根据当前 session 的时区进行转换；而 datetime 数据类型插入的是什么值，再读取就是什么值，不受时区影响。也可以理解为已经存储的数据是不会变的，只是 timestamp 类型数据在读取时会根据时区转换：

```
mysql> set time_zone='+08:00';
```

```
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> create table t(ts timestamp, dt datetime);
```

```
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> insert into t values('2021-12-02 16:45:39','2021-12-02 16:45:39');
```

```
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select * from t;
```

```
+-----+-----+
| ts | dt |
+-----+-----+
| 2021-12-02 16:45:39 | 2021-12-02 16:45:39 |
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> set time_zone='+00:00';
```

```
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> select * from t;
```

```
+-----+-----+
| ts | dt |
+-----+-----+
| 2021-12-02 08:45:39 | 2021-12-02 16:45:39 |
+-----+-----+
1 row in set (0.00 sec)
```

结论

关于时区所有明面上的东西都在上面了，我们前面提到的困扰就是在暗处的经验。

1. MySQL 的安装规范中应该设置什么时区？

对于国内的业务了，在 `my.cnf` 写入 `default-time-zone='+08:00'`，其他地区 and 开发确认取对应时区即可。

为什么不设置为 `system` 呢？使用系统时间看起来也是个不错的选择，比较省事。不建议的原因有两点：

- 操作系统的设置可能不归 DBA 管，万一别人没有设置正确的系统时区呢？把后背交给别人可能会有点发凉；
- 多了一层系统调用，性能有损耗。

2. JAVA 应用读取到的时间和北京时间差了 14 个小时，为什么？怎么解决？

这通常是 JDBC 参数中没有为连接设置时区属性（用 `serverTimezone` 参数指定），并且 MySQL 中没有设置全局时区，这样 MySQL 默认使用的是系统时区，即 CST。这样一来应用与 MySQL 建立的连接的 `session time_zone` 为 CST，前面我们提到 CST 在 RedHat 上是 +08:00 时区，但其实它一共能代表 4 个时区：

- Central Standard Time (USA) UT-6:00 美国标准时间
- Central Standard Time (Australia) UT+9:30 澳大利亚标准时间
- China Standard Time UT+8:00 中国标准时间
- Cuba Standard Time UT-4:00 古巴标准时间

JDBC 在解析 CST 时使用了美国标准时间，这就会导致时区错误。要解决也简单：一是遵守上面刚说到的规范，对 MySQL 显示地设置 +08:00 时区；二是 JDBC 设置正确的 `serverTimezone`。

3. 已经运行一段时间的业务，修改 MySQL 的时区会影响已经存储的时间类型数据吗？

完全不会，只会影响对 `timestamp` 数据类型的读取。这里不得不提一句，为啥要用 `timestamp`？用 `datetime` 不香吗，范围更大，存储空间其实差别很小，赶紧加到开发规范中吧。

4. 迁移数据时会有导致时间类型数据时区错误的可能吗？

这个还真有，还是针对 `timestamp` 数据类型，比如使用 `mysqldump` 导出 `csv` 格式的数据，默认这种导出方式会使用 UTC 时区读取 `timestamp` 类型数据，这意味导入时必须手工设置 `session.time_zone='+00:00'` 才能保证时间准确：

```
--将 test.t 导出成 csv
mysqldump -S /data/mysql/data/3306/mysqld.sock --single-transaction \
--master-data=2 -t -T /data/backup/test3 --fields-terminated-by=',' test t

--查看导出数据
cat /data/backup/test3/t.txt
2021-12-02 08:45:39,2021-12-02 16:45:39
```

如何避免？`mysqldump` 也提供了一个参数 `--skip-tz-utc`，意思就是导出数据的那个连接不设置 UTC 时区，使用 MySQL 的 `global time_zone` 系统变量值。

其实 `mysqldump` 导出 `sql` 文件时默认也是使用 UTC 时区，并且会在导出的 `sql` 文件头部带有 `session time_zone` 信息，这样可以保证导 SQL 文件导入和导出时使用相同的时区，从而保证数据的时区正确（而导出的 `csv` 文件显然不可以携带此信息）。需要注意的是 `--compact` 参数会去掉 `sql` 文件的所有头信息，所以一定要记得：`--compact` 参数得和 `--skip-tz-utc` 一起使用。

```
-- MySQL dump 10.13 Distrib 8.0.18, for linux-glibc2.12 (x86_64)
--
-- Host: 10.186.17.104 Database: sbtest
--
...
/*!40103 SET TIME_ZONE='+00:00' */;
...
```

针对 `--where="date(create_time) < date(now())"` 这样的归档需求。`mysqldump` 如果使用上述条件导出数据，则 `date(now())` 结果会 UTC 时区影响，导致导出的数据不满足要求。也应该使用 `--skip-tz-utc` 规避。

2.5 ProxySQL 搭配 MySQL HA （上）

作者：杨涛涛

ProxySQL 是一个使用非常广泛并且较稳定的中间件，有很多功能点。比如查询缓存，查询重写，读写分离，数据分片等等。

本篇要介绍的是 ProxySQL 和 MySQL Replication 以及 MySQL MGR 的初步结合，初步读写分离以及 failover 功能的体验。

在本机安装一个 ProxySQL 实例，六个 MySQL 实例；ProxySQL 和 MySQL 版本均是最新版。

ProxySQL: 管理端口 6032，流量端口 6033。

MySQL Replication: 流量端口分别为：3340、3341、3342

MySQL MGR: 流量端口分别为：3343、3344、3345

第一，ProxySQL 以及六个 MySQL 部署。

ProxySQL 安装比较简单，官网 `apt/yum`，或者自己下载安装。装好六个 MySQL 实例，并且配置好 MySQL 主从以及组复制环境。

第二，ProxySQL 记录 MySQL 实例相关信息。

进入 ProxySQL 管理端，把以上六个 MySQL 实例信息依次插入到表 `mysql_servers`：主从实例的

hostgroup_id 统一设置为 1，为了不破坏后续 failover 相关 hostgroup_id 的连续性，组复制实例的 hostgroup_id 统一设置为 3。

```
Admin> select hostgroup_id, hostname, port, status from mysql_servers;
+-----+-----+-----+-----+
| hostgroup_id | hostname | port | status |
+-----+-----+-----+-----+
| 1 | 127.0.0.1 | 3340 | ONLINE |
| 1 | 127.0.0.1 | 3341 | ONLINE |
| 1 | 127.0.0.1 | 3342 | ONLINE |
| 3 | 127.0.0.1 | 3343 | ONLINE |
| 3 | 127.0.0.1 | 3344 | ONLINE |
| 3 | 127.0.0.1 | 3345 | ONLINE |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

第三，MySQL 端创建 ProxySQL 所需的两类用户

ProxySQL 与 MySQL 交互，需要两类用户，这两类用户需要同时在主从和组复制环境创建。

1. 监控用户：为了免去后面重复设置监控用户变量的工作，两种架构用户名和密码保持一致，用户名和密码都是 proxysql_monitor。proxysql_monitor 需要以下权限：

```
client,session_variables_admin,system_variables_admin,select
```

在 MySQL 主从以及组复制环境里分别执行下面 SQL：

```
MySQL localhost:3343 ssl SQL > create user proxysql_monitor@'127.0.0.1' identified by
'proxysql_monitor';
Query OK, 0 rows affected (0.0596 sec)
MySQL localhost:3343 ssl SQL > grant replication
client,session_variables_admin,system_variables_admin,select on *.* to
proxysql_monitor@'127.0.0.1';
Query OK, 0 rows affected (0.0103 sec)
```

进入 ProxySQL 管理端，设置监控用户：

```
Admin> set mysql-monitor_username='proxysql_monitor';
Query OK, 1 row affected (0.00 sec)

Admin> set mysql-monitor_password='proxysql_monitor';
Query OK, 1 row affected (0.00 sec)
```

2. 开发用户：对于主从和组复制环境，分别创建此类用户。

主从环境用户创建：用户名 dev_user

```
MySQL localhost:3343 ssl SQL > create user dev_user@'127.0.0.1' identified by
'dev_user';
Query OK, 0 rows affected (0.1221 sec)
```

```
MySQL localhost:3343 ssl SQL > grant insert,delete,update,select,create on ytt.* to
dev_user@'127.0.0.1';
Query OK, 0 rows affected (0.0359 sec)
```

组复制环境用户创建：用户名 dev_user_mgr

```
MySQL localhost:3343 ssl SQL > create user dev_user_mgr@'127.0.0.1' identified by
'dev_user_mgr';
Query OK, 0 rows affected (0.1221 sec)
MySQL localhost:3343 ssl SQL > grant insert,delete,update,select,create on ytt.* to
dev_user_mgr@'127.0.0.1';
Query OK, 0 rows affected (0.0359 sec)
```

进入 ProxySQL 管理端，分别插入主从以及组复制对应的开发用户到表 mysql_users 。字段 transaction_persistent 为 1 代表事务不拆分，统一去主库检索。

```
Admin> insert into
mysql_users(username,password,active,default_hostgroup,transaction_persistent)
values
('dev_user','dev_user',1,1,1),
('dev_user_mgr','dev_user_mgr',1,3,1);
Query OK, 1 row affected (0.00 sec)

Admin> select username,active,default_hostgroup from mysql_users;
+-----+-----+-----+
| username | active | default_hostgroup |
+-----+-----+-----+
| dev_user | 1 | 1 |
| dev_user_mgr | 1 | 3 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

第四，配置读写分离

进入 ProxySQL 管理端，插入之前创建的两个开发用户到表 mysql_query_rules ，定义最基本的读写分离策略，只要是 select 开头的语句都分流到从库。

```
Admin> INSERT INTO
mysql_query_rules(username,schemaname,active,match_pattern,destination_hostgroup,apply)
VALUES
('dev_user','ytt',1,'^select',2,1),
('dev_user_mgr','ytt',1,'^select',4,1);
Query OK, 2 rows affected (0.00 sec)
```

```
Admin> select username,schemaname,active,match_pattern,destination_hostgroup,apply from
mysql_query_rules;
+-----+-----+-----+-----+-----+-----+
| username | schemaname | active | match_pattern | destination_hostgroup | apply |
+-----+-----+-----+-----+-----+-----+
| dev_user | ytt | 1 | ^select | 2 | 1 |
| dev_user_mgr | ytt | 1 | ^select | 4 | 1 |
+-----+-----+-----+-----+-----+-----+
```

```

| username | schemaname | active | match_pattern | destination_hostgroup | apply |
+-----+-----+-----+-----+-----+-----+
| dev_user | ytt | 1 | ^select | 2 | 1 |
| dev_user_mgr | ytt | 1 | ^select | 4 | 1 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

```

设置好相关信息后把以上所有更改加载到内存，并且持久化到磁盘。

```

Admin> load mysql servers to runtime;
Query OK, 0 rows affected (0.01 sec)

Admin> load mysql users to runtime;
Query OK, 0 rows affected (0.00 sec)

Admin> load mysql variables to runtime;
Query OK, 0 rows affected (0.00 sec)

Admin> load mysql query rules to runtime;
Query OK, 0 rows affected (0.00 sec)

Admin> save mysql servers to disk;
Query OK, 0 rows affected (0.18 sec)

Admin> save mysql users to disk;
Query OK, 0 rows affected (0.13 sec)

Admin> save mysql variables to disk;
Query OK, 140 rows affected (0.17 sec)

Admin> save mysql query rules to disk;
Query OK, 0 rows affected (0.11 sec)

```

分别测试下主从和组复制两种架构的读写分离效果：开发用户 `dev_user/dev_user_mgr` 连接端口 6033，创建一张表 `t1`，插入一条记录，并且简单查询一次。

```

-- 主从环境：
root@ytt-ubuntu:/home/ytt/scripts# mysql -u dev_user -p -h 127.0.0.1 -P 6033 -e " \
> use ytt;
> create table t1 (id int primary key,str1 varchar(100));
> insert t1 values (1,'replication');
> select * from t1;
> ";
Enter password:
-- 组复制环境把用户 dev_user 和密码替换为 dev_user_mgr 和对应密码重复执行一次。

```

进入 ProxySQL 管理端，检索审计表 `stats_mysql_query_digest`：写入请求和读取请求根据不同的用户被

成功分发到 `mysql_query_rules` 表里对应的 `hostgroup` 上。

```
Admin> select hostgroup,username,digest_text,count_star from stats_mysql_query_digest where schemaname = 'ytt';
+-----+-----+-----+-----+
| hostgroup | username | digest_text | count_star |
+-----+-----+-----+-----+
| 4 | dev_user_mgr | select * from t1 | 1 |
| 3 | dev_user_mgr | insert t1 values (?,?) | 1 |
| 3 | dev_user_mgr | create table t1 (id int primary key,str1 varchar(?)) | 1 |
| 2 | dev_user | select * from t1 | 1 |
| 1 | dev_user | insert t1 values (?,?) | 1 |
| 1 | dev_user | create table t1 (id int primary key,str1 varchar(?)) | 1 |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

第五，配置主从自动 failover 功能：

进入 ProxySQL 管理端，把主从相关实例信息插入到表 `mysql_replication_hostgroups` 即可。

ProxySQL 通过实时监控 MySQL 系统变量（'read_only','innodb_read_only','super_read_only'）开关与否来探测对应的 MySQL 实例是主库还是从库，完了自动更新 `mysql_server` 表主库对应的 IP 和端口来达到 failover 的目的。

```
Admin> insert into mysql_replication_hostgroups
(writer_hostgroup,reader_hostgroup,check_type,comment)values(1,2,'super_read_only','MySQL Replication failover test');
Query OK, 1 row affected (0.00 sec)
```

```
Admin> select * from mysql_replication_hostgroups;
+-----+-----+-----+-----+
| writer_hostgroup | reader_hostgroup | check_type | comment |
+-----+-----+-----+-----+
| 1 | 2 | super_read_only | MySQL Replication failover test |
+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

```
Admin> load mysql servers to runtime;
Query OK, 0 rows affected (0.01 sec)
```

```
Admin> save mysql servers to disk;
Query OK, 0 rows affected (0.18 sec)
```

我用 MySQL Shell 操作副本集来手动进行主从切换，设置主库为端口 3342。

```
MySQL localhost:3340 ssl Py > rs = dba.get_replica_set()
You are connected to a member of replicaset 'rs1'.

MySQL localhost:3340 ssl Py > rs.set_primary_instance('root@localhost:3342')
```

```
127.0.0.1:3342 will be promoted to PRIMARY of 'rs1'.
The current PRIMARY is 127.0.0.1:3340.
...
127.0.0.1:3342 was promoted to PRIMARY.
```

查看 ProxySQL 日志，已经感知到主从切换，新的主库自动变为 127.0.0.1:3342

```
2021-12-15 16:02:08 [INFO] Regenerating read_only_set1 with 1 servers
2021-12-
15 16:02:08 [INFO] read_only_action() detected RO=0 on server 127.0.0.1:3342 for the fi
rst time after commit(), but no need to reconfigure
```

也可以进入 ProxySQL 管理端来查询表 `mysql_servers` 的字段 `hostgroup_id=1` 的匹配记录是否已经变更为主库：

```
Admin> select hostname,port from mysql_servers where hostgroup_id = 1;
+-----+-----+
| hostname | port |
+-----+-----+
| 127.0.0.1 | 3342 |
+-----+-----+
1 row in set (0.00 sec)
```

第六，配置组复制自动 failover 功能：

和主从配置类似，把组复制实例相关信息插入到表 `mysql_replication_hostgroups` 即可。

这里和主从有点不一样的地方：`writer_hostgroup`,`backup_writer_hostgroup`,`reader_hostgroup`,`offline_hostgroup` 这四个字段代表不同职责的 Hostgroup，最好是设置不一样。

```
Admin> insert into mysql_group_replication_hostgroups
(writer_hostgroup,backup_writer_hostgroup, reader_hostgroup,
offline_hostgroup,active,max_writers,writer_is_also_reader,max_transactions_behind)
values (3,4,5,6,1,1,1,1000);
Query OK, 1 row affected (0.00 sec)

Admin> load mysql servers to runtime;
Query OK, 0 rows affected (0.01 sec)

Admin> save mysql servers to disk;
Query OK, 0 rows affected (0.19 sec)
```

进入 ProxySQL 管理端，查询组复制日志表 `mysql_server_group_replication_log`，可以看到当前的组复制对应的实例数据，其中主库为：127.0.0.1:3343。

```
Admin> select hostname, port,viable_candidate,read_only, transactions_behind, error
```

```

from mysql_server_group_replication_log where port in (3343,3344,3345) order by
time_start_us desc limit 3;
+-----+-----+-----+-----+-----+-----+
| hostname | port | viable_candidate | read_only | transactions_behind | error |
+-----+-----+-----+-----+-----+-----+
| 127.0.0.1 | 3345 | YES | YES | 0 | NULL |
| 127.0.0.1 | 3344 | YES | YES | 0 | NULL |
| 127.0.0.1 | 3343 | YES | NO | 0 | NULL |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

同样用 MySQL Shell 来验证下组复制的主备角色切换后 ProxySQL 是否会自动感知：把实例 127.0.0.1: 3344 提升为主库。

```

MySQL localhost:3343 ssl sys Py > rc = dba.get_cluster();
MySQL localhost:3343 ssl sys Py > rc.set_primary_instance('root@localhost:3344');
Setting instance 'localhost:3344' as the primary instance of cluster 'ytt_mgr'...

Instance '127.0.0.1:3343' was switched from PRIMARY to SECONDARY.
Instance '127.0.0.1:3344' was switched from SECONDARY to PRIMARY.
Instance '127.0.0.1:3345' remains SECONDARY.

WARNING: The cluster internal session is not the primary member anymore. For cluster
management operations please obtain a fresh cluster handle using dba.get_cluster().

The instance 'localhost:3344' was successfully elected as primary.
MySQL localhost:3343 ssl sys Py >

```

进入 ProxySQL 管理端，查询组复制日志：127.0.0.1: 3344 自动变更为主库。

```

Admin> select hostname, port, viable_candidate, read_only, transactions_behind, error
from mysql_server_group_replication_log where port in (3343,3344,3345) order by
time_start_us desc limit 3;
+-----+-----+-----+-----+-----+-----+
| hostname | port | viable_candidate | read_only | transactions_behind | error |
+-----+-----+-----+-----+-----+-----+
| 127.0.0.1 | 3345 | YES | YES | 0 | NULL |
| 127.0.0.1 | 3344 | YES | NO | 0 | NULL |
| 127.0.0.1 | 3343 | YES | YES | 0 | NULL |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

或者查询 mysql_server 表，查找 hostgroup_id 为 3 的记录是否变更为新主库。

```

Admin> select port from mysql_servers where hostgroup_id = 3;
+-----+
| port |

```

```
+-----+
| 3344 |
+-----+
1 row in set (0.00 sec)
```

总结

本篇简单介绍 ProxySQL 配置 MySQL HA 的相关配置与验证测试，更详细的配置与验证策略请关注后续。

2.6 ProxySQL 搭配 MySQL HA （下）

作者：杨涛涛

通过上一章节的介绍，我们已经了解 ProxySQL 如何基于 MySQL 主从以及组复制架构来构建读写分离、故障转移等功能点，但没有涵盖 ProxySQL 相关配置表的工作细节。那本章就对上节遗漏的内容进行一个延伸讲解。

先来了解下 ProxySQL 的内置数据库列表：

```
ytt:admin> show databases;
+-----+-----+-----+-----+
| seq | name | file |
+-----+-----+-----+-----+
| 0 | main | |
| 2 | disk | /var/lib/proxysql/proxysql.db |
| 3 | stats | |
| 4 | monitor | |
| 5 | stats_history | /var/lib/proxysql/proxysql_stats.db |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

以上所列数据库中，main 代表 runtime，也即运行时库；disk 代表持久化库；stats 代表统计数据库；monitor 代表监控数据库；stats_history 代表统计数据库归档。对于储存 MySQL 主从、组复制、读写分离的几张配置表，在每个库里都存在，不同的库代表不同的运行范畴。

第一，后端主机元数据库表

mysql_servers：该表为存储后端数据库相关元数据信息的基础表，所有的后续操作都需要访问并且更新这张表。

其中主要几个字段如下：

hostgroup_id，后端 MySQL 实例的主机组标志，每个实例可以设置为一样的数值或者设置为不相同的数值，推荐一组实例设置为相同。

gtid_port，Proxy Binlog Reader 组件需要监听的端口。不使用此组件可以保持默认即可。

status，实例状态值。

- online，默认选项，在线提供服务，也即正常状态；
- offline_soft，非强制下线状态，也即处理完当前会话后停止接受新请求；
- offline_hard，强制下线，强制关闭目所有会话，并且不再接受新的请求；
- shunned，后端实例由于太多错误连接而暂时关闭的状态或者说由于超过设定的延迟时间而暂停处理新请求。

weight，一个组里的实例优先级，优先级越高的越有几率被选中。比如多个从实例，可以提升一个节点的优先级来保证流量分配优先。

compression，是否压缩连接请求。默认不压缩，可以设置为 1 表示压缩。

max_connections，通过 ProxySQL 流量端口的最大连接数限制。

max_replication_lag，指定实例状态被设置为 shunned 的延迟时间。超过这个时间后，指定实例状态由 online 变为 shunned，直到积压的请求处理完成。

比如下面 runtime 级别的 mysql_servers 表记录：由于这几个节点都没有运行，状态都为 shunned：

```
ytt:admin> select hostgroup_id,hostname,port,status,max_connections from
runtime_mysql_servers where hostgroup_id in (1,2);
```

hostgroup_id	hostname	port	status	max_connections
2	127.0.0.1	3341	SHUNNED	1000
2	127.0.0.1	3342	SHUNNED	1000
2	127.0.0.1	3340	SHUNNED	1000

```
3 rows in set (0.00 sec)
```

我启动这三个主从节点，对应状态自动更新为 online：

```
ytt:admin> select hostgroup_id,hostname,port,status,max_connections from
runtime_mysql_servers where hostgroup_id in (1,2);
```

hostgroup_id	hostname	port	status	max_connections
2	127.0.0.1	3341	ONLINE	1000
1	127.0.0.1	3341	ONLINE	1000
2	127.0.0.1	3342	ONLINE	1000
2	127.0.0.1	3340	ONLINE	1000

```
4 rows in set (0.00 sec)
```

同样，启动组复制实例，三个节点的状态如下：

```
ytt:admin> select hostgroup_id,hostname,port,status from runtime_mysql_servers where
hostgroup_id > 2;
+-----+-----+-----+-----+
| hostgroup_id | hostname | port | status |
+-----+-----+-----+-----+
| 3 | 127.0.0.1 | 3343 | ONLINE |
| 5 | 127.0.0.1 | 3343 | ONLINE |
| 5 | 127.0.0.1 | 3344 | ONLINE |
| 5 | 127.0.0.1 | 3345 | ONLINE |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

第二，用户元数据表

`mysql_users`：此表存储流量用户的授权数据。有几个主要字段：

`transaction_persistent`，用来指定事务整体是否被分流。设置为 1 则代表以事务为粒度分流到默认主机组；为 0 则代表按照事务内部 SQL 为粒度来分流。除了只读事务，其他事务都应该作为一个整体，保持原有事务逻辑。

`default_hostgroup`，默认主机组，没有配置查询规则的 SQL 统一分流到默认主机组。

`frontend`，前端用户，针对 ProxySQL 实例。

`backend`，后端用户，针对 MySQL 实例。

这两个字段默认都为 1，通常定义一个后端 MySQL 实例用户，会自动映射到前端 ProxySQL 实例。

比如下面主从流量用户：从 `mysql_users` 表自动映射到 `runtime_mysql_users` 表，一个用户同时为前后端。

```
ytt:admin> select username,active,default_hostgroup,frontend,backend from mysql_users
where username = 'dev_user';
+-----+-----+-----+-----+-----+
| username | active | default_hostgroup | frontend | backend |
+-----+-----+-----+-----+-----+
| dev_user | 1 | 1 | 1 | 1 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

ytt:admin> select username,active,default_hostgroup,frontend,backend from
runtime_mysql_users where username = 'dev_user';
+-----+-----+-----+-----+-----+
| username | active | default_hostgroup | frontend | backend |
+-----+-----+-----+-----+-----+
| dev_user | 1 | 1 | 0 | 1 |
| dev_user | 1 | 1 | 1 | 0 |
```

```
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

第三，主从元数据表

mysql_replication_hostgroups：此表配置主从实例主机组信息。

ProxySQL 根据这张表的内容来分流前端请求，并且配合 mysql_servers 表来达成主从自动故障转移目标。

writer_hostgroup，写主机组 ID。比如我们的例子里设置为 1，表示主机组 ID 为 1 的处理写请求。

reader_hostgroup，读主机组 ID。比如我们的例子里设置为 2，表示主机组 ID 为 2 的处理读请求。

check_type，检查 MySQL 只读变量的值。在 read_only, innodb_read_only, super_read_only 这几个变量里选。

比如需要检测 super_read_only，如果为 1，代表读；为 0，则为写。

```
ytt:admin> select * from mysql_replication_hostgroups;
+-----+-----+-----+-----+
---+
| writer_hostgroup | reader_hostgroup | check_type | comment |
+-----+-----+-----+-----+
---+
| 1 | 2 | super_read_only | MySQL Replication failover test |
+-----+-----+-----+-----+
---+
1 row in set (0.00 sec)
```

第四，组复制元数据表

mysql_group_replication_hostgroups：此表配置组复制主机组信息，同样配合 mysql_servers 表来完成组复制节点无感知容错功能，类似表 mysql_replication_hostgroups。

writer_hostgroup，reader_hostgroup，这两个分别代表写和读流量组。

offline_hostgroup，下线主机组，状态不正常的节点被放入这个组。

max_writers，backup_writer_hostgroup，这两个用于多写模式，如果写实例数量多过 max_writers 设置，则被放入主机组 backup_writer_hostgroup。

max_transactions_behind，类似主从延迟流量停用功能。设置一个节点落后的事务数量，达到这个数量后，节点状态被设置为 shunned，被完全处理完后，再变更为正常状态。

目前组复制环境的配置表如下：

```
ytt:admin> select writer_hostgroup,backup_writer_hostgroup,reader_hostgroup from
mysql_group_replication_hostgroups;
+-----+-----+-----+
| writer_hostgroup | backup_writer_hostgroup | reader_hostgroup |
```

```

+-----+-----+-----+
| 3 | 4 | 5 |
+-----+-----+-----+
1 row in set (0.00 sec)

```

第五，读写分离元数据表

`mysql_query_rules`：用来配置读写分离模式，非常灵活，可以配置统一端口匹配正则表达式或者根据不同端口来分流。（正则表达式依据的标准由参数 `mysql-query_processor_regex` 设置决定）几个主要的字段如下：

`active`，是否激活这个匹配模式。

`username`，流量用户名。

`schemaname`，数据库名。

`match_pattern`，具体的匹配模式。

除了上一章节介绍的依赖正则表达式来分流读写流量到同一端口外，还可以设置多个端口来区分不同的实例组。比如主从流量走端口 6401，组复制流量走 6402，那么可以直接这样适配：

先把 ProxySQL 要监听的端口添加到变量 `mysql-interfaces` 里，完了重启 ProxySQL 服务：

```

ytt:admin> SET mysql-interfaces='0.0.0.0:6033;0.0.0.0:6401;0.0.0.0:6402';
Query OK, 1 row affected (0.00 sec)

ytt:admin> SAVE MYSQL VARIABLES TO DISK;
Query OK, 140 rows affected (0.02 sec)

```

再把这两个端口插入到这张表：

```

ytt:admin> INSERT INTO mysql_query_rules
(rule_id,active,proxy_port,destination_hostgroup,apply)
-> VALUES (1,1,6401,1,1), (2,1,6402,3,1);
Query OK, 2 rows affected (0.00 sec)

ytt:admin> LOAD MYSQL QUERY RULES TO RUNTIME;
Query OK, 0 rows affected (0.00 sec)

ytt:admin> SAVE MYSQL QUERY RULES TO DISK;
Query OK, 0 rows affected (0.08 sec)

```

这张表现在内容如下：除了按照正则分流外，额外有两条记录来按照指定端口（6401 为主从分流端口，6402 为组复制分流端口）分流。

```

ytt:admin> select rule_id,active,username,schemaname,match_pattern,destination_hostgroup,proxy_port from runtime_mysql_query_rules;
+-----+-----+-----+-----+-----+-----+-----+
| rule_id | active | username | schemaname | match_pattern | destination_hostgroup | proxy_port |
+-----+-----+-----+-----+-----+-----+-----+
| 1       | 1     | NULL     | NULL       | NULL         | 1                     | 6401      |
| 2       | 1     | NULL     | NULL       | NULL         | 3                     | 6402      |
| 13      | 1     | dev_user | ytt        | ^select      | 2                     | NULL      |
| 14      | 1     | dev_user_mgr | ytt      | ^select      | 4                     | NULL      |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

```

来验证下这个分流策略：分别使用用户 `dev_user` 连接端口 6401，用户 `dev_user_mgr` 连接端口 6402。

```

root@ytt-ubuntu:~# mysql -udev_user -pdev_user -P6401 -h ytt-ubuntu -e "use ytt;select count(*) from t1";
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| count(*) |
+-----+
| 5 |
+-----+

root@ytt-ubuntu:~# mysql -udev_user_mgr -pdev_user_mgr -P6402 -h ytt-ubuntu -e "use ytt;select count(*) from t1";
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| count(*) |
+-----+
| 1 |
+-----+

```

进入 ProxySQL 管理端，查看审计表：以上不同用户、不同端口分流到具体的主机组里。

```

ytt:admin> select hostgroup,schemaname,username,digest_text,count_star from stats_mysql_query_digest where schemaname = 'ytt';
+-----+-----+-----+-----+-----+
| hostgroup | schemaname | username | digest_text | count_star |
+-----+-----+-----+-----+-----+
| 3 | ytt | dev_user_mgr | select count(*) from t1 | 1 |
| 1 | ytt | dev_user | select count(*) from t1 | 1 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

```

到这里，对于 ProxySQL 来讲，如何与 MySQL HA 进行搭配，相信已经有了一定的了解。

2.7 MySQL Binlog 通过 MySQL 客户端导入数据库效率低的原因

作者：郭斌斌

一、背景

客户反馈生产环境中，MySQL 5.7 通过 xtrabackup+ Binlog 做基于时间点的恢复操作时，持续卡在 Binlog 的回放阶段，旷日持久，久到离谱。他对于这种旷日持久的操作产生了怀疑，想要确认数据库的这种行为是否合理，因此有了本文的 Binlog 回灌验证操作。

二、复现前提

MySQL Version: 5.7.22 Binlog format: Row 准备 Delete 800 多万记录的 Binlog

三、复现准备

3.1 创建表、构造数据

```
mysql> create table t1(id int primary key,name varchar(10));
Query OK, 0 rows affected (0.02 sec)

mysql> insert into t1 values(1,repeat('a',10));
Query OK, 1 row affected (0.01 sec)

mysql> insert into t1 select (select count(1) from t1)+id,name from t1;
Query OK, 1 row affected (0.01 sec)
Records: 1 Duplicates: 0 Warnings: 0
.....
mysql> insert into t1 select (select count(1) from t1)+id,name from t1;
Query OK, 4194304 rows affected (57.75 sec)
Records: 4194304 Duplicates: 0 Warnings: 0
```

3.2 准备 Delete 800 多万记录的 Binlog 文件

```
mysql> show master status;
+-----+-----+-----+-----+-----+
| File           | Position | Binlog_Do_DB | Binlog_Ignore_DB | Executed_Gtid_Set |
+-----+-----+-----+-----+-----+
| mysql-bin.000002 | 154      |              |                  | 48db8b41-f658-11eb-bcdc-02000aba3c0f:1-29 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql>
mysql> flush logs;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql>
mysql> show master status;
+-----+-----+-----+-----+-----+
| File           | Position | Binlog_Do_DB | Binlog_Ignore_DB | Executed_Gtid_Set |
+-----+-----+-----+-----+-----+
| mysql-bin.000003 | 154      |              |                  | 48db8b41-f658-11eb-bcdc-02000aba3c0f:1-29 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> delete from t1;
Query OK, 8388608 rows affected (1 min 3.42 sec)
```

MySQL Binlog mysql-bin.000003 用于回灌测试

3.3 由于 Binlog 的回灌和造数是在同一个实例上，之前为了构建 Delete 800 多万记录的 Binlog，已经

将数据删除，因此在进行 binlog 回灌前，需要使用之前造数的方法，重新造数

```
mysql> select count(1) from t1;
+-----+
| count(1) |
+-----+
| 8388608 |
+-----+
1 row in set (2.92 sec)
```

3.4 同一个实例上先进行了 Delete，又重新构建新的数据。导致 Delete 操作的 GTID 要比重新造数操作的 GTID 小，为保证可以正常回灌，可以执行 reset master

```
mysql> reset master;
Query OK, 0 rows affected (0.05 sec)
```

```
mysql>
mysql>
mysql> show master status;
+-----+-----+-----+-----+-----+
| File | Position | Binlog_Do_DB | Binlog_Ignore_DB | Executed_Gtid_Set |
+-----+-----+-----+-----+-----+
| mysql-bin.000001 | 154 | | | |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

四、复现测试

4.1 解析 MySQL Binlog mysql-bin.000003

```
[root@10-186-60-15 3456]# /opt/mysql/base/5.7.22/bin/mysqlbinlog mysql-bin.000003 >3.sql
[root@10-186-60-15 3456]#
[root@10-186-60-15 3456]# du -sh mysql-bin.000003 3.sql
129M    mysql-bin.000003
176M    3.sql
```

4.2 导入解析文件

```
[root@10-186-60-15 3456]#
[root@10-186-60-15 3456]# time /opt/mysql/base/5.7.22/bin/mysql -S /opt/mysql/data/3456/mysqld.sock -uroot -p123456 -e "source /tmp/3.sql"
mysql: [Warning] Using a password on the command line interface can be insecure.
```

4.3 查看 processlist，发现导入线程一直处于 Sleep 状态，现象跟客户描述契合。

```
mysql> show processlist;
+-----+-----+-----+-----+-----+-----+-----+
| Id | User | Host | db | Command | Time | State | Info |
+-----+-----+-----+-----+-----+-----+-----+
| 2 | universe_op | 127.0.0.1:55342 | NULL | Sleep | 0 | | NULL |
| 3 | universe_op | 127.0.0.1:55344 | NULL | Sleep | 0 | | NULL |
| 4 | universe_op | 127.0.0.1:55348 | NULL | Sleep | 0 | | NULL |
| 5 | universe_op | 127.0.0.1:55350 | NULL | Sleep | 0 | | NULL |
| 6 | universe_op | 127.0.0.1:55346 | NULL | Sleep | 0 | | NULL |
| 7 | universe_op | 127.0.0.1:55352 | NULL | Sleep | 0 | | NULL |
| 8 | universe_op | 127.0.0.1:55354 | NULL | Sleep | 0 | | NULL |
| 9 | universe_op | 127.0.0.1:55356 | NULL | Sleep | 0 | | NULL |
| 10 | universe_op | 127.0.0.1:55360 | NULL | Sleep | 2 | | NULL |
| 11 | universe_op | 127.0.0.1:55358 | NULL | Sleep | 2 | | NULL |
| 12 | universe_op | 127.0.0.1:55362 | NULL | Sleep | 2 | | NULL |
| 57 | root | localhost | test | Query | 0 | starting | show processlist |
| 86 | root | localhost | NULL | Sleep | 2062 | | NULL |
+-----+-----+-----+-----+-----+-----+-----+
13 rows in set (0.00 sec)
```

4.4 随即中断导入操作，重新发起导入同时使用 strace 记录操作的行为。


```
[root@10-186-60-15 3456]# strace -o /tmp/strace.log -f -tt /opt/mysql/base/5.7.22/bin/mysql -S /opt/mysql/data/3456/mysql.sock -uroot -p1234  
56 -e "source /tmp/3.sql"  
mysql: [Warning] Using a password on the command line interface can be insecure.
```

4.5 通过观测产生的 `strace.log`，发现两个 `read` 的时间间隔不固定，少的也需要 140ms 左右，而读取的大小却只有 4k (4096)，读取效率偏低。

```
30191 16:04:25.999948 mmapmap(0x7f8ba9a6d000, 93519872) = 0
30191 16:04:25.922887 read(4, "\nYFhYwFhYfxLvEEAcMfhYwFhYwFhYWH"... , 4096) = 4096
30191 16:04:25.923373 mmap(NULL, 93528064, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f8baf39a000
30191 16:04:26.050466 munmap(0x7f8ba413c000, 93523968) = 0
30191 16:04:26.064798 read(4, "wIUUEACmFhYwFhYwFhYWH8Cb1BAAPhYwFhYWH"... , 4096) = 4096
30191 16:04:26.065216 mmap(NULL, 93532160, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f8ba9a67000
30191 16:04:26.199135 munmap(0x7f8baf39a000, 93528064) = 0
30191 16:04:26.213671 read(4, "CmfHYwFhYwFhYWH8xL1BAAPhYwFhYwFhYWH"... , 4096) = 4096
30191 16:04:26.214047 mmap(NULL, 93536256, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f8ba4133000
30191 16:04:26.364509 munmap(0x7f8ba9a67000, 93532160) = 0
30191 16:04:26.383248 read(4, "hYWHg8b5BAAPhYwFhYwFhYwFhYIK+QQA"... , 4096) = 4096
30191 16:04:26.383784 mmap(NULL, 93540352, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f8baf397000
30191 16:04:26.521636 munmap(0x7f8ba4133000, 93536256) = 0
30191 16:04:26.537162 read(4, "L9BAAphYwFhYwFhYwFhYwFhYDZ/QQAK\nYwFhYWH"... , 4096) = 4096
30191 16:04:26.537496 mmap(NULL, 93544448, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f8ba9a61000
30191 16:04:26.663744 munmap(0x7f8baf397000, 93540352) = 0
30191 16:04:26.677346 read(4, "YwFhYwFhY/Pq/\nQQAQYwFhYwFhYwFhYwFhYfz"... , 4096) = 4096
30191 16:04:26.677600 mmap(NULL, 93548544, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f8ba412a000
30191 16:04:26.800108 munmap(0x7f8ba9a61000, 93544448) = 0
30191 16:04:26.816101 read(4, "fAQQAQYwFhYwFhYwFhYfy4wEEAcMfhYwFhYWH"... , 4096) = 4096
30191 16:04:26.816262 mmap(NULL, 93552640, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f8baf394000
30191 16:04:26.940000 munmap(0x7f8ba412a000, 93548544) = 0
30191 16:04:26.954520 read(4, "YwFhYwFhYwFhYfzxwUEACmFhYwFhYwFhYWH"... , 4096) = 4096
30191 16:04:26.954706 mmap(NULL, 93556736, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f8ba9a5b000
30191 16:04:27.065955 munmap(0x7f8baf394000, 93552640) = 0
30191 16:04:27.080642 read(4, "hYfWwwkEACmFhYwFhYwFhYwFhYWH8mcJBAAP"... , 4096) = 4096
30191 16:04:27.080982 mmap(NULL, 93560832, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f8ba4121000
30191 16:04:27.212560 munmap(0x7f8ba9a5b000, 93556736) = 0
30191 16:04:27.227579 read(4, "FhYwFhYwFhYwFhYWH87sJBAAphYwFhYwFhYwFhYWH"... , 4096) = 4096
30191 16:04:27.227972 mmap(NULL, 93564928, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f8baf391000
^C
```

五、分析

通过 Google 检索 “MySQL Mem Load Slow” 发现这是一个 BUG，MySQL 5.7 Client 在读取较大事务（涉及多行操作）时，由于内存分配效率比较低，导致消耗大量的时间，已在 MySQL 8.0.13 中修复。

- The **mysql** client was slow to import large multiple-line statements in batch mode. Memory allocation is now more efficient for this situation. Thanks to Sinisa Milivojevic for the patch. (Bug #28116512, Bug #85155)

六、复测

6.1 Mysql 8.0.18 客户端进行 Binlog 解析文件的回灌，提示 MySQL Server has gone away

```
[root@db58 tmp]# time mysql -uroot -h10.186.60.15 -P3456 -p123456 -e "source /tmp/3.sql"
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 2006 (HY000) at line 32869 in file: '/tmp/3.sql': MySQL server has gone away

real    0m3.035s
user    0m1.554s
sys     0m0.814s
```

6.2 导数报错时数据库没触发重启，查看 error 日志，有如下报错：

```
2021-12-27T17:15:35:15.627380+08:00 0 [Note] Event Scheduler: Loaded 0 events
2021-12-27T17:15:35:15.627815+08:00 0 [Note] /opt/mysql/base/5.7.22/bin/mysqld: ready for connections.
Version: '5.7.22-log' socket: '/opt/mysql/data/3456/mysqld.sock' port: 3456 MySQL Community Server (GPL)
2021-12-27T17:06:19.232181+08:00 17 [Note] Aborted connection 17 to db: 'unconnected' user: 'root' host: 'localhost' (Got timeout reading comm
unication packets)
2021-12-27T17:41:25.040417+08:00 14 [Note] Aborted connection 14 to db: 'unconnected' user: 'root' host: 'localhost' (Got timeout reading comm
unication packets)
2021-12-27T17:05:24.929381+08:00 223 [Note] Aborted connection 223 to db: 'unconnected' user: 'root' host: '10.186.63.58' (Got a packet bigger
than 'max_allowed_packet' bytes)
```

6.3 调大 max allowed packet 配置后重新测试


```
[root@db58 tmp]#
[root@db58 tmp]# time strace -o /tmp/strace.log -f -tt mysql -uroot -h10.186.60.15 -P3456 -p123456 -e "source /tmp/3.sql"
mysql: [Warning] Using a password on the command line interface can be insecure.
```

6.4 观测 strace 日志，每次读取 Binlog 大小 16M，远高于原来的 4k

```
6417 17:09:38.734347 read(5, "GwKAaphYwFhYwFhYwFh/P1sCgAKYwFhY...", 16777216) = 16777216
6417 17:09:38.765044 mmap(NULL, 33558528, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe1016b8000
6417 17:09:38.773740 munmap(0x7fe1066bb000, 16781312) = 0
6417 17:09:38.870039 read(5, "H86TcWnAAphYwFhYwFhYwFh/Oo3FgAKY...", 16777216) = 16777216
6417 17:09:38.904068 mmap(NULL, 50335744, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe0fe6b7000
6417 17:09:38.938711 munmap(0x7fe1016b8000, 33558528) = 0
6417 17:09:39.037958 read(5, "FhYwH81gIiAAphYwFhYwFhYwFh/n/NcCI...", 16777216) = 16777216
6417 17:09:39.063734 mmap(NULL, 67112960, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe0fa6b6000
6417 17:09:39.097812 munmap(0x7fe0fe6b7000, 50335744) = 0
6417 17:09:39.191977 read(5, "mFhYwFhYwFhYwFhYwH8wc0tAAphYwFhYwFhY...", 16777216) = 16777216
6417 17:09:39.215980 mmap(NULL, 83890176, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe0fe6b8000
6417 17:09:39.255868 munmap(0x7fe0fa6b6000, 67112960) = 0
6417 17:09:39.354293 read(5, "DkAcMfYwFhYwFhYwFhYwH8rpg5AAphYwFhY...", 16777216) = 16777216
6417 17:09:39.381111 mmap(NULL, 100667392, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe0f86b7000
6417 17:09:39.445705 munmap(0x7fe0fe6b8000, 83890176) = 0
6417 17:09:39.542212 read(5, "WfHYfyYY0UACmFhYwFh\nYwFhYwH8mWNF...", 16777216) = 16777216
6417 17:09:39.566822 mmap(NULL, 117444608, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe0f16b6000
6417 17:09:39.620095 munmap(0x7fe0f86b7000, 100667392) = 0
6417 17:09:39.714574 read(5, "WfHYwFhYfyFLlEACmFhYwFhYwFhYwH8h...", 16777216) = 16777216
6417 17:09:39.739092 mmap(NULL, 134221824, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe0fb6b8000
6417 17:09:39.813659 munmap(0x7fe0f16b6000, 117444608) = 0
6417 17:09:39.918901 read(5, "XAAKYwFhYwFhYwFhYwFhYwXwVwAcMfYwFh...", 16777216) = 16777216
6417 17:09:39.937088 mmap(NULL, 150999040, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe0f26b7000
6417 17:09:40.000082 munmap(0x7fe0fb6b8000, 134221824) = 0
6417 17:09:40.094695 read(5, "FzEaAAK\nYwFhYwFhYwFhYwFhYfXdxGgAcMfH...", 16777216) = 16777216
6417 17:09:40.115713 mmap(NULL, 167776256, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe0e86b6000
6417 17:09:40.215421 munmap(0x7fe0f26b7000, 150999040) = 0
6417 17:09:40.312367 read(5, "WfH/EmPdAAKYwFhYwFhYwFhYwFhYfXK\nj3QA...", 16777216) = 16276432
6417 17:09:40.337023 mmap(NULL, 184553472, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe0f86b8000
6417 17:09:40.446101 munmap(0x7fe0e86b6000, 167776256) = 0
```

6.5 观测线程状态

5	universe_op	127.0.0.1:57956	NULL	Sleep	60	NULL
6	universe_op	127.0.0.1:57950	NULL	Sleep	61	NULL
7	universe_op	127.0.0.1:57954	NULL	Sleep	60	NULL
8	universe_op	127.0.0.1:57952	NULL	Sleep	60	NULL
9	universe_op	127.0.0.1:57958	NULL	Sleep	63	NULL
10	universe_op	127.0.0.1:57960	NULL	Sleep	63	NULL
11	universe_op	127.0.0.1:57962	NULL	Sleep	63	NULL
15	root	localhost	NULL	Query	0	starting show processlist
18	root	10.186.63.58:41522	NULL	Query	8374	System lock BINLOG ' +mLJYRONzTV4MAAAAHkBAAAAAANBAAAAAAEABHRLc3QAAnQxAAIDdwIoAAIUdRL/ +mLJYSCnzTV4EYAAAIwhAAAAAN
21	universe_op	127.0.0.1:57998	NULL	Sleep	60	NULL
22	universe_op	127.0.0.1:58040	NULL	Query	0	starting SELECT @@GLOBAL.datadir
23	universe_op	127.0.0.1:58050	NULL	Query	0	query end SELECT @@max_allowed_packet

6.6 观察执行耗时，MySQL 8.0.18 客户端导数时间变短，效率提升明显。

```
[root@db58 tmp]# time strace -o /tmp/strace.log -f -tt mysql -uroot -h10.186.60.15 -P3456 -p123456 -e "source /tmp/3.sql"
mysql: [Warning] Using a password on the command line interface can be insecure.
```

```
real    3m2.800s
user    0m1.459s
sys     0m0.705s
```

七、结论

目前官方在 MySQL 8.0.13 版本中，解决了“在使用 MySQL Client 进行批量导数时，内存分配效率低”的问题，因此 MySQL 8.0.18 客户端在进行回灌 Binlog 解析后的文件时，读取文件效率明显高于 5.7.22 的客户端，提升了 Binlog 回放的效率。

参考链接

<https://bugs.mysql.com/bug.php?id=85155>

<https://dev.mysql.com/doc/relnotes/mysql/8.0/en/news-8-0-13.html#mysqld-8-0-13-bug>

2.8 mysqlbinlog 技巧

作者：胡呈清

常用命令

1. 解析 binlog 排查问题

如果只是解析出来查看，可以加 `--base64-output=decode-rows` 不显示行格式的内容：`mysqlbinlog --no-defaults -vv --base64-output=decode-rows mysql-bin.000201`

2. 解析指定 GTID 的事务

用来分析某个事务做了什么：`mysqlbinlog --no-defaults -vv --base64-output=decode-rows --include-gtids='b0ca6715-7554-11ea-a684-02000aba3dad:614037' mysql-bin.000199`

3. 解析指定范围的 binlog

a. 时间范围

`--start-datetime`、`--stop-datetime` 解析出指定时间范围内的 binlog，这个只适合粗略的解析，不精准，因此不要用来回放 binlog。有个小技巧：如果只能确定大概的时间范围，而且不确定在哪个 binlog 中，可以直接解析多个 binlog。比如大概在 11:20-12:00 内做了个表删除操作，但这个时间内有多个 binlog，可以这样：

```
mysqlbinlog --no-defaults -vv --base64-output=decode-rows --start-datetime='2020-08-18 11:20:00' --stop-datetime='2020-08-18 12:00:00' mysql-bin.000203 mysql-bin.000204 mysql-bin.000205
```

b. 偏移量范围

`--start-position`、`--stop-position` 解析 binlog 指定偏移量范围内的 binlog。如果同时指定了 `--start-position` 和 `--stop-position`，并且是解析多个 binlog，则 `--start-position` 只对第一个 binlog 生效，`--stop-position` 只对最后一个 binlog 生效。

这个常用场景是：已经解析过一次 binlog 并取得目标事务的 起始 position 后，精确的解析这一段 binlog：

```
mysqlbinlog --no-defaults -vv --base64-output=decode-rows --start-position='537' --stop-position='945' mysql-bin.000204
# at 537 "起始位置是 GTID event 前的这个 position"
#200818 11:29:03 server id 3 end_log_pos 602 CRC32 0x7f07dd8c GTID last_committed=1
sequence_number=2 rbr_only=yes
/*!50718 SET TRANSACTION ISOLATION LEVEL READ COMMITTED/*!*/;
SET @@SESSION.GTID_NEXT= 'b0ca6715-7554-11ea-a684-02000aba3dad:614061'/*!*/;
...
...
#200818 11:29:03 server id 3 end_log_pos 945 CRC32 0xedf2b011 Query thread_id=3
exec_time=0 error_code=0 SET TIMESTAMP=1597721343/*!*/;
COMMIT /*!*/;
```

```
# at 945 "结束位置是 COMMIT event 后的这个 position"
```

c. GTID 范围

--include-gtids、--exclude-gtids 详细看参数解释。

4. 回放 binlog

- 回放一定不能加 --base64-output=decode-rows 参数，因为不会解析出行格式（这是 binlog 真正有效的部分）；
- 回放也可以用上面指定范围的参数；
- 解析 binlog 回放到本实例，不需要修改 server id，但要注意 GTID 是否已存在；
- GTID 已经存在，回放不会报错，但也不会真正回放这些事务，可以通过 --skip-gtids 参数跳过 GTID 的限制；

```
mysqlbinlog --no-defaults --skip-gtids mysql-bin.000203 | mysql -S /data/mysql/data/3306/mysql.sock -proot
```

参数解释

1. --no-defaults

可以避免 my.cnf 里配了 [client] 某些 mysqlbinlog 没有的参数导致 mysqlbinlog 失败

2. -v

不加，只显示行格式（即那一串字符串），无法得到伪 SQL：

```
BEGIN
/*!*/;
# at 331
# at 383
...
BINLOG '
mDQ7XxMCAAAALgAAAK0BAAAAAHAAAAAAAEABGh1Y3EAAQzAAIDAwACwNu2hg==
mDQ7Xx4CAAAALAAAANKBAAAAAHAAAAAAAEAAgAC//xaAAAAWgAAAEXgahg=
'/*!*/;
# at 473
COMMIT/*!*/;
```

加 -v，从行格式中重建伪 SQL(带注释)，不显示 binlog_rows_query_log_events 参数效果：

```
BEGIN
/*!*/;
# at 331
# at 383
...
BINLOG '
mDQ7XxMCAAAALgAAAK0BAAAAAHAAAAAAAEABGh1Y3EAAQzAAIDAwACwNu2hg==
mDQ7Xx4CAAAALAAAANKBAAAAAHAAAAAAAEAAgAC//xaAAAAWgAAAEXgahg=
'/*!*/;
### INSERT INTO `hucq`.`t3`      ##效果
### SET
###   @1=90
###   @2=90
# at 473
COMMIT/*!*/;
```

加 `-vv`，从行格式中重建伪 SQL 并添加字段数据类型的注释，可以显示 `binlog_rows_query_log_events` 参数效果：

```
BEGIN
/*!*/;
# at 331
# insert into t3 values(90,90)    ##效果
# at 383
BINLOG '
mDQ7Xx0CAAAANAAAH8BAACAABxpbnNlcnQgaW50byB0MyB2YWx1ZXMo0TAs0TApBDaHuQ==
mDQ7XxMCAAAALgAAAK0BAAAAHAAAAAAAAEABGh1Y3EAAQzAAIDAwACwNu2hg==
mDQ7Xx4CAAAALAAAANKBAAAAHAAAAAAAAEAAgAC//xaAAAAWgAAAEAgahg=
'/*!*/;
### INSERT INTO `hucq`.`t3`
### SET
###  @1=90 /* INT meta=0 nullable=0 is_null=0 */    ##效果
###  @2=90 /* INT meta=0 nullable=1 is_null=0 */
# at 473
COMMIT/*!*/;
```

3. 加 `--base64-output=decode-rows`

不显示行格式，如果同时加 `-v` 参数，可以从行格式中解码为带注释的伪 SQL：

```
BEGIN
/*!*/;
# at 331
# insert into t3 values(90,90)
# at 383
...
### INSERT INTO `hucq`.`t3`
### SET
###  @1=90 /* INT meta=0 nullable=0 is_null=0 */
###  @2=90 /* INT meta=0 nullable=1 is_null=0 */
# at 473
COMMIT/*!*/;
```

4. `--skip-gtids`

不保留 GTID 事件信息，这样回放 binlog 时会跟执行新事务一样，生成新的 GTID。对比如下：

```
##加 --skip-gtids
[root@localhost 3306]# mysqlbinlog --no-defaults -vv --base64-output=decode-rows \
--skip-gtids mysql-bin.000201 |grep GTID
/*!50616 SET @@SESSION.GTID_NEXT='AUTOMATIC'/*!*/;

##不加 --skip-gtids 可看到 GTID event
[root@localhost 3306]# mysqlbinlog --no-defaults -vv --base64-output=decode-rows \
mysql-bin.000201 |grep GTID
#200818 9:52:53 server id 2 end_log_pos 194 CRC32 0x9a905870 Previous-GTIDs
#200818 9:53:28 server id 2 end_log_pos 259 CRC32 0xd3dd4379 GTID last_committed=0
SET @@SESSION.GTID_NEXT= 'b0ca6715-7554-11ea-a684-02000aba3dad:614045'/*!*/;
SET @@SESSION.GTID_NEXT= 'AUTOMATIC' /* added by mysqlbinlog */ /*!*/;
```

5. `--include-gtids`

只解析出指定的 GTID 的事务：

```
[root@localhost 3306]# mysqlbinlog --no-defaults -vv --base64-output=decode-rows \
> --include-gtids='b0ca6715-7554-11ea-a684-02000aba3dad:614037-614040' mysql-bin.000199
|grep GTID
#200807 17:32:17 server id 2 end_log_pos 194 CRC32 0xc840be04 Previous-GTIDs
#200807 17:32:17 server id 2 end_log_pos 3818435 CRC32 0x9fdea913 GTID last_committed=3
sequence_number=5 rbr_only=yes
SET @@SESSION.GTID_NEXT= 'b0ca6715-7554-11ea-a684-02000aba3dad:614037'/*!*/;
#200807 17:32:17 server id 2 end_log_pos 5726909 CRC32 0x51b51cc1 GTID last_committed=4
sequence_number=6 rbr_only=yes
SET @@SESSION.GTID_NEXT= 'b0ca6715-7554-11ea-a684-02000aba3dad:614038'/*!*/;
#200807 17:32:17 server id 2 end_log_pos 5727523 CRC32 0x758852f1 GTID last_committed=6
sequence_number=7 rbr_only=yes
SET @@SESSION.GTID_NEXT= 'b0ca6715-7554-11ea-a684-02000aba3dad:614039'/*!*/;
#200807 17:32:17 server id 2 end_log_pos 7635997 CRC32 0x47c43f83 GTID last_committed=6
sequence_number=8 rbr_only=yes
SET @@SESSION.GTID_NEXT= 'b0ca6715-7554-11ea-a684-02000aba3dad:614040'/*!*/;
SET @@SESSION.GTID_NEXT= 'AUTOMATIC' /* added by mysqlbinlog */ /*!*/;
```

6. --exclude-gtids

不解析指定的 GTID 的事务

2.9 MySQL 数据库巡检需要做些什么？

作者：陈俊聪

接触 MySQL 数据库 7 年了，专职做 MySQL 数据库运维工作也有 6 个年头了，这 6 年来呆了三家公司，做过很多次数据库巡检工作，从一开始是网上下载个巡检模板应付工作，草草了事，到后来使用公司专门的数据库巡检模板做巡检，经验越来越多，逐渐形成了自己的套路，或称为方法论。今天，想写下这篇文章，把我的这些个人的经验和想法总结下来，也为了证明，即使巡检那么小的一件事，只要你愿意，也能得出个最佳实践。

最佳实践的意义是什么？并不是所有人都对 MySQL 那么熟悉，最佳实践以文档的形式沉淀下来，可以有效避免犯错，也能最大限度的避免因人员流失而带来的巡检质量降低。

我认为巡检有好几种分类，他们的侧重点各不相同，以下是我的分类。

按巡检方式来划分

按巡检方式来划分的话，巡检分为人肉巡检、脚本化巡检、平台化巡检。

人肉巡检也称为手工巡检，在管理的数据库服务器数量少，巡检频率低的情况下，是可以这么做。上去服务器执行 `df -h`、`mysql -u` 等命令检查服务器、数据库的运行状况，查看监控(zabbix、Prometheus 等)来

分析。这种方法除了不效率之外，还有一个缺点，就是非常依赖巡检人员的技能水平，不同的工程师去巡检，可能结论会不一样。

脚本化巡检，这个阶段其实也就是把巡检的命令打包做成一个脚本，工程师登录服务器一台台执行脚本，当然了，如果公司允许的话，可以采用 `ansible` 等批量运维工具，批量跑脚本巡检，脚本生成 `html` 报表或 `csv` 格式的数据，供您分析和汇总。有了这个方法后，我们的经验是，一个人可以在一天内，轻松地在 2000 个实例上做巡检并完成巡检报告。这个方法相对人肉巡检方法在效率上有了质的飞跃。他仍然有缺点，就是他仍然需要耗费人力，仍然需要部分依赖巡检人员的技能水平去分析报告和汇总。

平台化巡检，我们的数据库管理平台解决租户的三大核心诉求（高可用管理、备份恢复管理、监控告警管理）之外，还提供了平台化巡检功能，定时触发巡检生成巡检报告，并且利用融合了我们运维经验的健康度评分模型算法，去给实例打分，对于不满 60 分的实例我们需要马上关注，自动通知数据库管理员，并且自动和智能地分析存在的问题。有了它，极大地减少数据库运维人员登录生产环境的次数，并且不再依赖巡检人员的技能水平，做到巡检的绝对标准化。

按时间维度来划分

按时间维度来划分，数据库巡检分为日常巡检和节前巡检。

日常巡检，就是每天都做的巡检，这个巡检最简单办法就是看监控，主要留意 `warning` 或以上级别的告警，监控基本上可以覆盖百分之 95% 以上的日常巡检需求。

节前巡检，指的是特殊日期的巡检工作，要比日常巡检要深度。狭义上指的是国家法定节假日放假前（例如五一、端午、清明、中秋、国庆、过年等等）的巡检。广义上也包含了双十一等重大活动前的保障级巡检。节前巡检，对于我所在的数据库运维团队来说，主要关注的就是，过节时可能会有部分业务有流量高峰，并且过节时由于人员放假，人员不足，需要提前巡检发现未来的问题，提前解决。而作为运维，主要关注的是数据库可用性，所以节前巡检的检查核心如下：

- 系统层面
 - CPU
 - RAM
 - 磁盘空间
- 应用层面(MySQL 实例)
 - 实例状态
 - 高可用状态
 - 复制状态
 - 监控状态
 - VIP 状态

这里我展开说明一下。

CPU，通过监控，去寻找高 CPU 水位的实例，确认是否和日常监控曲线一致，判断是否有异常，是否有优化的需要。

RAM，通过监控找到 RAM 使用率超过 80% 的实例，检查是否有内存不够用的问题，该扩容的扩容，防止 OOM。这里一并查找是否有实例使用到 `swap`，如果内存充裕而使用到 `swap`，那很大概率是因为未正确设置 `numa` 或 `vm.swappiness` 导致的。

磁盘空间，这个是节前巡检中的重点，平时磁盘使用率的告警线是 80%，而巡检时我们应该去获取大于 90

70% 磁盘使用率的实例，提前扩容，避免放个春节(7 天) 磁盘有告警的风险，这个时候回家过年还加班大家都不好受。

实例状态，一般来说就是检查 `mysqld` 的存活，有条件的话可以分析其是否健康。(至于怎么判断其是否健康，这里不扩展了)

高可用状态，通过巡检证明数据库是"可切换"状态。例如 MHA 架构的话，要检查这三个脚本执行的结果。

```
masterha_check_ssh --conf=/etc/masterha/app.cnf
masterha_check_rep1 --conf=/etc/masterha/app.cnf
masterha_check_status --conf=/etc/masterha/app.cnf
```

复制状态，实际上上述高可用状态检查，一般也会检查复制状态。但有一些复制不在高可用范畴，所以这里要多嘴提一下。异步复制、半同步复制、延迟复制、双向复制、级联复制状态的检查，看是否正常，容灾的 DTS 复制是否正常，DTS 的高可用是否正常等等。

VIP 状态，我们有一些实例是有双网络冗余链路的，这些实例会有双 VIP。如果这时掉了那个冗余网络链路的 VIP 对业务是没有感知的。所以我们应该有一个巡检机制，这个可以开发一个探活任务去定时探测和告警。

按巡检程度来划分

按巡检程度来划分，分为普通巡检和深度巡检。上述的节前巡检虽然比日常巡检要深度，但也只是普通巡检，不能算是深度巡检。那什么样的巡检才算是深度巡检呢？

节前巡检，我们的关注点在于运维，在于数据库可用性，那么深度巡检必然关注点要向外扩展，要关注用户体验，要关注性能，这样的巡检就是深度巡检。

所以深度巡检是解决什么问题呢？深度巡检的目的是对日常巡检和节前巡检的补充，让数据库在未来日子里不单只可用，并且可靠和跑得更快。

我认为，深度巡检 = 可用性巡检 + 可靠性巡检 + 性能巡检 + 分析和建议

可用性巡检

在前面提及的节前巡检，已经大量检查了数据库的可用性，但那些都是从运维角度、从服务、从实例级别来衡量的，从应用角度、从业务角度，其实这个可用性检查是可以扩展的，例如在深度巡检里，我们会检查租户每张表的自增键使用情况，租户常见的自增键类型是 `int unsigned` 和 `int signed`，前者是无符号 `int` 类型，范围是(-2147483648,2147483647)，后者是有符号 `int` 类型 (0,4294967295)，开发人员在建表的时候更常见是直接定义 `int`，没有规定是 `unsigned` 还是 `signed`，那么默认就是 `unsigned` 了，而没有使用更理想的 `int signed` 值，这样自增键可用的范围就会少了一倍。再加上，自增键在插入时并不是连续的，和你插入的方式，参数设置(`innodb_autoinc_lock_mode`、`auto_increment_increment`)有关，自增键在插入前就会分配，所以一旦插入失败事务回滚，这个自增 `id` 也会自然浪费掉。所以 `int unsigned` 的上限虽然看起来很高，有 21 亿之多，但由于刚才说的原因，很有可能你的 `table` 里只有 10 亿行数据时，自增键就满了。自增键满了，你这张表就不可写入了，这就是业务层面的不可用。

我们在生产实践中就遇到过这样的情况，某业务是负责某商城交易账单数据分析的，其账单的日志表，每天入库入表的记录数一般为 500 万条，高峰时可以达到 900 万条以上，当时这张表采用的是 `int`

unsigned 自增 id 作为主键，在业务上线不到 9 个月自增主键就用完了。解决办法就是修改自增利类型，从 int unsigned 修改为 bigint signed，我们知道 MySQL 修改主键列类型是锁表的，只能读不能写，所以当时这个业务受损了，DDL 花了 6 个小时。

所以深度巡检，需要对这些情况，做可用性巡检的扩展，更多的可用性巡检，读者可以自行补充。

可靠性巡检

在说性能巡检之前，我想补充一下，可靠性巡检，前面提到的节前巡检有大量的检查可用性了，但可用性是否等于可靠性呢，这里有很多人会混淆，他们并不相等。可用性指的是 Availability，一般是高可用要解决的问题，而可靠性指的是 Reliability，在数据库里一般指的是数据不错、不丢和数据副本的一致性。

节前巡检，已经包含了不少数据库可靠性检查，例如高可用检查中的"可切换"检查，复制状态检查。但这里并不是万无一失的，在这里我提出深度巡检需要做 "核心参数检查"。这里的"核心参数检查"包括三方面

- 检查数据库里的参数是否满足我们的交维规范要求的核心参数列表
- 检查主备数据库参数是否一致
- 检查数据库运行参数和配置文件(my.cnf)参数是否一致

检查数据库里的参数是否满足我们的交维规范要求的核心参数列表，这个其实是历史遗留问题，因为我们大多数数据库本身不来自于我们的部署交付，而是各业务部门交接给我们的，对于这些新交接的实例，务必检查核心参数，才能保证数据不错不丢和主从数据一致性，相关参数包含并不仅限于以下这些：

```
binlog_format = row
binlog_row_image = full
gtid_mode = on
enforce-gtid-consistency = on
innodb_doublewrite = on
innodb_flush_log_at_trx_commit = 1
log_bin = mysql-bin
master_info_repository = table
sync_binlog = 1
...
```

实际上我们检查的参数高达 80 个。

检查主备数据库参数是否一致，这个主要是避免主备切换后，有使用上的不一致。这里也会检查一些务必设置不一致的参数，例如 server_id，反正目的只有一个，就是检查主备的参数，保证他们正常。

检查数据库运行参数和配置文件(my.cnf)参数是否一致。很多人以为持久化的配置文件一定会和运行参数一致，这个没必要检查，这就错了，在 MySQL 5.7 或之前，没有办法修改参数同时持久化配置文件，所以修改参数通常都是分两步，先在数据库里 set global 参数=值，然后登陆服务器修改 my.cnf 配置文件，因不是原子操作，那么运维人员就有犯错的可能，千万不要相信人，人总是会犯错的。之前我们就发生过好几次运行参数和持久化配置文件不一致产生的故障。例如，动态修改 MySQL 的 innodb_buffer_pool_size = 128G，然后忘记持久化到配置文件了。当时数据库发生了 crash，之后被高可用组件拉回 mysqld 实例，发现性能很差，这个排查了半天，居然是 innodb_buffer_pool_size 被还原了默认值 128M！

还有一个案例，在 mysql 5.6 年代，当时硬件性能不行加上没有好的并行复制技术，从库容易因为 io 瓶颈而复制延迟，临时解决方法是从库设置 `sync_binlog=0`、`innodb_flush_log_at_trx_commit=0` 来追延迟，待延迟追平后，修改回 "双 1"。这个时候 DBA 很容易忘记去执行修改回 "双 1" 操作。如果这个时候有个数据库实例级故障，造成主从切换，那么这个时候就有丢失数据的风险了。

另外，某些租户是持有 `super` 权限，可以修改数据库的参数，但他们是没服务器权限，如果这些租户修改的参数涉及了我们认为的核心参数，造成这个核心参数的运行参数和配置文件(my.cnf)参数不一致，那就有可能埋了雷，后续引发数据库可靠性甚至是可用性问题了。

"核心参数检查" 是可靠性巡检的一个例子，更多的可靠性巡检，读者可以自行补充。

性能巡检

性能巡检上，就有很多细小的项目了，我这里介绍一些常见的。

1、是否存在没有主键的表。

MySQL 的玩法就是需要有主键，最好是业务无关的 `int signed` 自增主键，具体为什么请出门右拐看 "开发规范"，他是如何影响性能的，网上有大量的文章，这里我就没必要过多赘述了。

2、SQL 性能优化

首先，巡检报告里，可以列出 top 10 慢查询，让租户去优化 SQL。其次，在报告里可以抓取提供一些执行全表扫描次数 TOP 30 的 SQL 给租户，因为有些 SQL 他的执行计划本身其实就有问题的，这些 SQL 可能当前跑得很快，但没有评估过数据量增长，当表变得越来越大时，达到一个阈值时，线上可能会爆发 CPU 100% 的性能问题，成为爆发性杀手级慢查询。

再次，索引方面，可以关注冗余索引、无效索引、索引区分度等信息。冗余索引意思是数据库里有重复的索引，对于 a、b、c 列的联合索引 `idx_a_b_c`，他其实同时等于拥有了(a、b、c)、(a、b)、(a) 三个索引的能力，如果这时候你再创建一个 `idx_a`，`idx_a_b` 索引，那他们就是冗余索引了，这些索引应该删掉，因为索引是占存储空间的，并且索引不是越多越好，维护索引是有开销的，他影响了 DML 语句的性能，所以用不到的索引就应该删除。同样的，无效索引，就是从未使用过的索引，巡检报告应该把这些索引列出来，开发去评估一下这些索引是否未来都不会用到，用不到就应该删除他们。索引区分度用于评估列的值是否足够分散，值越多越适合建立索引，如果是性别列，只有男女两个值，是不适用创建索引的。区分度越接近 1，表示区分度越高；低于 0.1，则说明区分度较差，开发者应该重新评估 SQL 语句涉及的字段，选择区分度高的多个字段创建索引。

3、是否有 MyISAM 存储引擎表

MyISAM 基本没有好处，我之前写过一篇文章《交维规范讲解系列——为什么我们禁止使用 MyISAM 存储引擎》来说过这个问题。实际上我们不应该还要检查是否存在 MyISAM 表，99% 的场景应该直接禁用他，请与我一样通过参数防止 MyISAM 表的创建，当然还有很多存储引擎我都建议不要用了，`innodb` 才是永远的王者，参考的参数如

```
下:disabled_storage_engines=ARCHIVE,BLACKHOLE,EXAMPLE,FEDERATED,MEMORY,MERGE,NDB,MyISAM
```

因为 MySQL 5.7 版本仍然有 10 张元数据表使用 MyISAM 存储引擎，可能会影响数据库的升级，如果

5.7 版本使用上了这个参数，运维人员要注意在做数据库升级变更前先禁用此参数，升级完后加回来，参考 <https://mp.weixin.qq.com/s/O9UtGskB3lydkEEMscEo1Q>。8.0 版本则无这个问题。

我们的目标是，检查到 MyISAM 表后，尽量进行整改，加上上述这个参数，哈。

4、TOP 10 大表

大表在做全表扫描时非常耗费性能，在 DDL 方面的话更是灾难，大于 100G 的表都应该评估一下，为什么会有那么大的表？为什么会放在 MySQL 上，是否可以放到 TiDB 上？是否可以拆分为小表，水平拆还是纵向拆？归档，冷热分离？

建议：MySQL 单实例的存储空间大小应控制在 500G，单表行数控制在 1000 万行以内、大小在 30G 以内，单表字段 50 个以内，单表索引 5 个以内。

这是以前的建议，仅供参考。随着硬件的提升，我最新的观点是 MySQL 实例 2T 以内，单表体积 100 G 以内我都可以接受。当然了，我是从运维角度考虑，性能角度的话主要是看业务是否能接受。

分析和建议

暂时列那么多，也就是抛砖引玉，性能巡检的目的是出具尽量多的数据给租户自行做性能分析，这里有 SQL 相关的，有非 SQL 相关的，至于对这些数据的加工和分析方面，我们的报告主要是对非 SQL 相关的加以文字说明，给出建议，SQL 相关的，我们不是这方面的专家，里面有很多门道有很多小技巧，这个交给租户级别高的开发人员去分析优化。

以上就是我个人对 MySQL 数据库巡检需要做什么的总结，欢迎指正。

2.10 MySQL 在线开启 GTID 的每个阶段是要做什么

作者：余振兴

基本概述

我们知道 MySQL 有 2 种方式指定复制同步的方式，分别为：

1. 基于 binlog 文件名及位点的指定方式
 - 匿名事务(Anonymous_gtid_log_event)
2. 基于 GTID (全局事务 ID)的指定方式
 - GTID 事务(Gtid_log_event)

而基于 GTID 的方式在一主多从的架构下主从切换有着明显优势外，对于日常复制异常的故障诊断也更为方便，在日常运维或 MySQL 升级过程中我们免不了要做 GTID 的开启或关闭，从个人角度而言，我也更倾向于大家做在线开启或关闭 GTID 的操作，一方面该操作能尽可能小的影响数据库停机时间，另

一方面在开启或关闭的过程中也顺便可以验证该参数的调整是否会对应用造成影响，从 MySQL 5.7.6 之后便开始支持动态开启和关闭 GTID 模式，其参数 GTID_MODE 有以下取值

- OFF - 只允许匿名事务被复制同步
- OFF_PERMISSIVE - 新产生的事务都是匿名事务，但也允许有 GTID 事务被复制同步
- ON_PERMISSIVE - 新产生的都是 GTID 事务，但也允许有匿名事务被复制同步
- ON - 只允许 GTID 事务被复制同步

其实从该参数的几个取值我们就能看出，在线修改是循序渐进的将匿名事务转化为 GTID 事务过程(反之也一样)，我们先看看在线开启 GTID 分别要做哪些事

在线开启 GTID

1. 设置 GTID 校验 ENFORCE_GTID_CONSISTENCY 为 WARN

该操作的目的是允许在主库执行的 SQL 语句违反 GTID 一致性校验，且只在主库的错误日志中输出 warning 级别日志以作提醒，我们知道 GTID 复制还是有一些限制条件的，其实这里就是为了考虑如果复制方式直接改为 GTID 模式后应用程序因为 GTID 的一些限制导致异常报错，这样做的好处是当我需要开启 GTID 前，我可以把 ENFORCE_GTID_CONSISTENCY 参数开成 WARN 观测一段时间，比如一天，如果观测周期内在错误日志中并没有发现相关的 Warning 信息，那我们再考虑正式开启 GTID 的操作

- 示例：使用 CREATE TABLE AS SELECT 语法在 GTID 模式下不支持（题外话：CTAS 语法在 8.0.21 以后 GTID 模式下也是支持的，该语法变更为了一个特殊的原子性 DDL 操作），而 ENFORCE_GTID_CONSISTENCY 设置为 WARN 时，只会在错误日志提示，不会直接报错，

```
mysql> create table sbtest5 as select * from sbtest1;
Query OK, 100000 rows affected, 1 warning (1.28 sec)
Records: 100000 Duplicates: 0 Warnings: 1

mysql> show warnings;
+-----+-----+-----+
| Level | Code | Message                                     |
+-----+-----+-----+
| Warning | 1786 | Statement violates GTID consistency: CREATE TABLE ... SELECT. |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

```
10.186.63.65
[root@10-186-63-65 ~]# tail -f /data/zhenxing/3310/data/mysql-error.log
2021-11-25T22:14:14.760456+08:00 0 [Note] Server socket created on IP: '::'.
2021-11-25T22:14:14.853588+08:00 0 [Note] Event Scheduler: Loaded 0 events
2021-11-25T22:14:14.853874+08:00 0 [Note] /data/zhenxing/3310/base/bin/mysqld: ready for connections.
Version: '5.7.31-log' socket: '/data/zhenxing/3310/data/mysql.sock' port: 3310 MySQL Community Server (GPL)
2021-11-25T22:14:14.863212+08:00 0 [Note] InnoDB: Buffer pool(s) load completed at 211125 22:14:14
2021-11-25T22:15:09.153529+08:00 2 [Warning] Timeout waiting for reply of binlog (file: mysql-bin.000003, pos: 426), semi-sync up to file , position 0.
2021-11-25T22:15:09.153645+08:00 2 [Note] Semi-sync replication switched OFF.
2021-11-25T22:16:03.041194+08:00 3 [Note] Start binlog_dump to master_thread_id(3) slave_server(63663310), pos(mysql-bin.000001, 4)
2021-11-25T22:16:03.041329+08:00 3 [Note] Start semi-sync binlog_dump to slave (server_id: 63663310), pos(mysql-bin.000001, 4)
2021-11-25T22:16:03.054279+08:00 0 [Note] Semi-sync replication switched ON at (mysql-bin.000003, 426)

2021-11-25T22:51:49.741877+08:00 2 [Note] Changed ENFORCE_GTID_CONSISTENCY from OFF to WARN.
^@2021-11-25T22:53:10.290969+08:00 2 [Warning] Statement violates GTID consistency: CREATE TABLE ... SELECT.
```

该操作在主从库均执行

```
SET @@GLOBAL.ENFORCE_GTID_CONSISTENCY = WARN;
```

2. 设置 GTID 校验 ENFORCE_GTID_CONSISTENCY 为 ON

确认上一个步骤未在错误日志中出现相关 Warning 信息后，正式开启 GTID 一致性校验，当设置为 ON 后，如果再执行 CREATE TABLE AS SELECT 语句则会直接报错

```
mysql> create table sbtest6 as select * from sbtest1;
ERROR 1786 (HY000): Statement violates GTID consistency: CREATE TABLE ... SELECT.
mysql> █
```

3. 设置 GTID_MODE 为 OFF_PERMISSIVE

如前面对 GTID_MODE 可取值的描述，该操作表示新产生的事务依旧是匿名事务，但也允许有 GTID 事务被复制同步，对于在线开启 GTID 模式而言，该步骤就是一个单纯的过渡属性(注意是为在线关闭 GTID 准备的)，执行完后可快速到下一个阶段

```
## 该操作在主从库均执行
SET @@GLOBAL.GTID_MODE = OFF_PERMISSIVE;
```

4. 设置 GTID_MODE 为 ON_PERMISSIVE

该操作依旧是一个过渡属性，其表示的则是新产生的都是 GTID 事务，但也允许有匿名事务被复制，从这个阶段开始就已经是一个正式转化的过程，但依旧是对两种事务做兼容。

```
## 该操作在主从库均执行
SET @@GLOBAL.GTID_MODE = ON_PERMISSIVE;
```

5. (关键点)确保匿名事务回放完毕

该步骤的目的是确保在正式转换为完整的 GTID 模式前，老的匿名事务均已被回放完毕，确保 GTID_MODE 设置为 ON 时，不会因为残留的匿名事务导致复制同步报错，有以下 2 种方式进行校验

```
## 该操作仅在从库执行即可
```

```
## 方式 1: 确保该状态值输出的匿名事务数显示为 0 (注意: 只要出现过 0 即可表示已经转换完成, 即使后续该状态值从 0 变为了大于 0 的值也不影响)
```

```
SHOW STATUS LIKE 'ONGOING_ANONYMOUS_TRANSACTION_COUNT';
```

```
## 在从库上多次执行该语句
```

```
## 方式 2: 查询该视图中 LAST_SEEN_TRANSACTION 可以观测当前同步的事务是否还存在 ANONYMOUS 事务
select * from performance_schema.replication_applier_status_by_worker;
```

确保匿名事务数为 0

```
mysql> SHOW STATUS LIKE 'ONGOING_ANONYMOUS_TRANSACTION_COUNT';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Ongoing_anonymous_transaction_count | 0 |
+-----+-----+
1 row in set (0.01 sec)
```

确保回放线程回放的事务都已是 GTID 事务

```
mysql> select * from performance_schema.replication_applier_status_by_worker;
```

CHANNEL_NAME	WORKER_ID	THREAD_ID	SERVICE_STATE	LAST_SEEN_TRANSACTION	LAST_ERROR_NUMBER	LAST_ERROR_MESSAGE	LAST_ERROR_TIMESTAMP
	1	41	ON	99a9940c-4df9-11ec-8171-02000aba3f41:565457	0		0000-00-00 00:00:00
	2	42	ON	99a9940c-4df9-11ec-8171-02000aba3f41:565458	0		0000-00-00 00:00:00
	3	43	ON	99a9940c-4df9-11ec-8171-02000aba3f41:565454	0		0000-00-00 00:00:00
	4	44	ON	99a9940c-4df9-11ec-8171-02000aba3f41:565444	0		0000-00-00 00:00:00
	5	45	ON	99a9940c-4df9-11ec-8171-02000aba3f41:565445	0		0000-00-00 00:00:00
	6	46	ON	99a9940c-4df9-11ec-8171-02000aba3f41:565446	0		0000-00-00 00:00:00
	7	47	ON	99a9940c-4df9-11ec-8171-02000aba3f41:565395	0		0000-00-00 00:00:00
	8	48	ON	99a9940c-4df9-11ec-8171-02000aba3f41:565396	0		0000-00-00 00:00:00

8 rows in set (0.00 sec)

6. 触发一轮日志切换 FLUSH LOGS

该操作的目的是为了在主库触发 binlog 的轮换，使新生成的 binlog 都是包含 GTID 的事务（防止一个 binlog 中包含 2 种类型的事务日志）

该操作仅在主库执行即可

```
FLUSH LOGS;
```

7. 正式开启 GTID_MODE 为 ON

正式开启 GTID

该操作在主从库均执行

```
SET @@GLOBAL.GTID_MODE = ON;
```

```
SELECT @@GTID_MODE, @@ENFORCE_GTID_CONSISTENCY;
```

8. 修改配置文件确保 GTID 参数持久化

在 my.cnf 配置文件中增加 GTID 参数，确保重启不会失效，该操作也可在第一步进行

该操作在主从库均执行

```
gtid-mode = ON
```

```
enforce-gtid-consistency = 1
```

9. 修改复制模式为 GTID 方式

在开启 GTID 模式后我们也要将复制模式从基于 POS 点改为基于 GTID，操作比较简单

停止复制

```
STOP SLAVE;
```

修改为 GTID 模式

```
CHANGE MASTER TO MASTER_AUTO_POSITION = 1;
```

开启复制

```
START SLAVE;
```

观测复制同步状态

```
SHOW SLAVE STATUS\G
```

在线关闭 GTID

在线关闭的方式基本就类似于在线开启 GTID 的逆向操作,以下只写出步骤和具体命令，不做详细解释

- 先将 GTID 模式的复制改为基于 POS 点的复制
- 设置 GTID_MODE 为 ON_PERMISSIVE
- 设置 GTID_MODE 为 OFF_PERMISSIVE
- 观测 GTID_OWNED 状态变量变为空值及 replication_applier_status_by_worker 表中事务均转为匿名事务
- 触发 FLUSH LOGS
- 设置 GTID_MODE 为 OFF
- 设置 ENFORCE_GTID_CONSISTENCY 为 OFF
- 修改 my.cnf 配置文件中 GTID 相关参数为 OFF

1. 将复制改为基于 POS 点方式

```
stop slave;
show slave status\G

## 取 show slave status\G 中的 Master_Log_File 和 Exec_Master_Log_Pos 填入
## 这里一定不要漏掉 MASTER_AUTO_POSITION = 0 这个配置
CHANGE MASTER TO
MASTER_AUTO_POSITION = 0,
MASTER_LOG_FILE='mysql-bin.000017',
MASTER_LOG_POS=224126137;
start slave;
show slave status\G
```

2. 设置 GTID_MODE 为 ON_PERMISSIVE

```
## 该操作在主从库均执行
SET @@GLOBAL.GTID_MODE = ON_PERMISSIVE;
```

3. 设置 GTID_MODE 为 OFF_PERMISSIVE

```
## 该操作在主从库均执行
SET @@GLOBAL.GTID_MODE = OFF_PERMISSIVE;
```

4. （关键点）确保 GTID 事务回放完毕

观测 GTID_OWNED 状态变量变为空值及 replication_applier_status_by_worker 表中事务均转为匿名事务

```
## 该操作在从库执行即可
SELECT @@GLOBAL.GTID_OWNED;
select * from performance_schema.replication_applier_status_by_worker;
```

5. 触发 FLUSH LOGS

```
## 该操作在主库执行即可
FLUSH LOGS;
```

6. 设置 GTID_MODE 为 OFF

该操作在主从库均执行

```
SET @@GLOBAL.GTID_MODE = OFF;
```

7. 设置 ENFORCE_GTID_CONSISTENCY 为 OFF

该操作在主从库均执行

```
SET @@GLOBAL.ENFORCE_GTID_CONSISTENCY = OFF;
```

8. 修改 my.cnf 配置文件中 GTID 相关参数为 OFF

该操作在主从库均执行

```
gtid-mode                = OFF
enforce-gtid-consistency = 1
```

命令简版

1. 在线开启 GTID

自行判断命令在主库还是从库执行

```
SET @@GLOBAL.ENFORCE_GTID_CONSISTENCY = WARN;
SET @@GLOBAL.ENFORCE_GTID_CONSISTENCY = ON;
SET @@GLOBAL.GTID_MODE = OFF_PERMISSIVE;
SET @@GLOBAL.GTID_MODE = ON_PERMISSIVE;
SHOW STATUS LIKE 'ONGOING_ANONYMOUS_TRANSACTION_COUNT';
select * from performance_schema.replication_applier_status_by_worker;
FLUSH LOGS;
SET @@GLOBAL.GTID_MODE = ON;
```

配置文件修改

```
gtid-mode                = ON
enforce-gtid-consistency = 1
```

将复制模式从基于 POS 点改为基于 GTID

```
STOP SLAVE;
CHANGE MASTER TO MASTER_AUTO_POSITION = 1;
START SLAVE;
SHOW SLAVE STATUS\G
```

2. 在线关闭 GTID

自行判断命令在主库还是从库执行

```
stop slave;
show slave status\G

## 取 show slave status\G 中的 Master_Log_File 和 Exec_Master_Log_Pos 填入
CHANGE MASTER TO
  MASTER_AUTO_POSITION = 0,
  MASTER_LOG_FILE='mysql-bin.000017',
```

```
MASTER_LOG_POS=224126137;
start slave;
show slave status\G

SET @@GLOBAL.GTID_MODE = ON_PERMISSIVE;
SET @@GLOBAL.GTID_MODE = OFF_PERMISSIVE;
SELECT @@GLOBAL.GTID_OWNED;
select * from performance_schema.replication_applier_status_by_worker;
FLUSH LOGS;
SET @@GLOBAL.GTID_MODE = OFF;
SET @@GLOBAL.ENFORCE_GTID_CONSISTENCY = OFF;

## 修改 my.cnf 配置文件中 GTID 相关参数为 OFF
gtid-mode                = OFF
enforce-gtid-consistency = 1
```

技术总结

其实在线开启和在线关闭 GTID 看上去命令较多，但实际基本都是很快的速度就能完成且对业务几乎没有影响，其中更重要的反而是在正式开启之前的一个校验过程。

参考链接

<https://dev.mysql.com/doc/refman/5.7/en/replication-mode-change-online-enable-gtids.html><https://dev.mysql.com/doc/refman/5.7/en/replication-mode-change-online-disable-gtids.html>

2.11 何时需要手动刷新授权表

作者：杨涛涛

本篇来源于客户咨询的问题。问题概要为：记忆中在 MySQL 里对用户进行授权操作后都需要执行 `flush privileges` 才能生效，怎么我在你写的涉及到用户授权相关的文章里没看到执行 `flush privileges` 语句？

对于这个问题的解答，首先得明白语句 `flush privileges` 的作用是什么？`flush privileges` 是 `flush` 语句集合里的一条子项，执行它的作用是用来同步 MySQL 磁盘和内存中的授权数据，让其保持一致。说详细点就是全量读取授权表的磁盘数据并且完全覆盖内存中的授权数据，又或者反着同步磁盘上的授权数据。

接下来用两个简单示例说明执行 `flush privileges` 语句的时机。

第一，何时使用？

当授权数据在磁盘和内存中不一致时，存在两种时机：一是内存数据较新；另一个则是磁盘数据较新。当这两点中的任意一点存在时都需要执行 `flush privileges` 语句。

比如直接对表 `mysql.user` 执行 DML 语句，那么此时磁盘数据较新，需要手动执行 `flush privileges` 语句来覆盖内存中的授权数据。举个例子，用户 `ytt_u1` 拥有的权限数据如下：

```
mysql:(none)>show grants for ytt_u1;
+-----+
| Grants for ytt_u1@% |
+-----+
| GRANT SELECT ON *.* TO `ytt_u1`@`%` |
+-----+
1 row in set (0.00 sec)
```

管理员来直接更新底层用户表数据：取消 `select` 权限。

```
mysql:(none)>update mysql.user set select_priv='N' where user ='ytt_u1';
Query OK, 1 row affected (0.07 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

复查用户 `ytt_u1` 拥有的权限数据：结果没有同步。

```
mysql:(none)>show grants for ytt_u1;
+-----+
| Grants for ytt_u1@% |
+-----+
| GRANT SELECT ON *.* TO `ytt_u1`@`%` |
+-----+
1 row in set (0.00 sec)
```

手动执行 `flush privileges` 语句：再次查看用户 `ytt_u1` 的权限数据，数据已经同步为最新。

```
mysql:(none)>flush privileges;
Query OK, 0 rows affected (0.12 sec)

mysql:(none)>show grants for ytt_u1;
+-----+
| Grants for ytt_u1@% |
+-----+
| GRANT USAGE ON *.* TO `ytt_u1`@`%` |
+-----+
1 row in set (0.00 sec)
```

第二，何时不需要使用？

MySQL 内部命令自动更新或者删除用户授权数据。

比如 `create user`、`grant`、`revoke` 等语句执行后会自动同步授权数据，如无异常，则不需要手动执行 `flush privileges` 语句。举个例子：还是用户 `ytt_u1`，查看最新权限数据，结果显示没有任何权限。

```
mysql:(none)>show grants for ytt_u1;
+-----+
```

```
| Grants for ytt_u1@% |
+-----+
| GRANT USAGE ON *.* TO `ytt_u1`@`%` |
+-----+
1 row in set (0.00 sec)
```

用 `grant` 语句来给用户 `ytt_u1` 赋予数据库 `ytt` 的只读权限：

```
mysql:(none)>grant select on ytt.* to ytt_u1;
Query OK, 0 rows affected (0.20 sec)
```

查看用户 `ytt_u1` 的权限数据：数据已经同步为最新。

```
mysql:(none)>show grants for ytt_u1;
+-----+
| Grants for ytt_u1@% |
+-----+
| GRANT USAGE ON *.* TO `ytt_u1`@`%` |
| GRANT SELECT ON `ytt`.* TO `ytt_u1`@`%` |
+-----+
2 rows in set (0.00 sec)
```

再来看下取消授权操作是否也会同步最新授权数据：

```
mysql:(none)>revoke select on ytt.* from ytt_u1;
Query OK, 0 rows affected (0.09 sec)
```

查看用户 `ytt_u1` 的权限数据：数据也是及时同步的。

```
mysql:(none)>show grants for ytt_u1;
+-----+
| Grants for ytt_u1@% |
+-----+
| GRANT USAGE ON *.* TO `ytt_u1`@`%` |
+-----+
1 row in set (0.00 sec)
```

所以只要按照 MySQL 内置的用户管理语句来操作用户授权信息，则不需要手动执行 `flush privileges` 语句，反之则需要。

2.12 MySQL 字符集再探

作者：傅文辉

本公众号之前发表了一些关于 MySQL 符集的文章: 从 utf8 转换成 utf8mb4、字符集相关概念、有关 SQL 语句、字符集注意事项、乱码问题。

近日, 在为爱可生开源数据传输工具 dtle 增加 UTF32 字符集支持过程中, 笔者又了解了一下 MySQL 字符集机制。下面补充说明一些内容:

1、字符集

- ASCII 是最基础的字符集, 每个 ASCII 字符占用 1 字节, ASCII 仅利用了 8 位编码能力的一半, 最高位恒为 0。
 - Latin1 字符集利用剩下的一半编码了西欧常用字母。
- UCS-2 使用 2 字节编码, 容量为 65536。
 - UTF-16 在这基础上, 增加了临时使用 2 个 2 字节, 编码特殊字符的能力。
- UTF-32 使用定长 4 字节编码。
- GB 系列、UTF-8 等字符集, 0~127 编码和 ASCII 一样, 使用单字节。在最高位不为 0 时, 额外使用 1~3 字节编码。
 - 即它们是变长编码, 每个字符占用 1~4 字节

只表达 ASCII 0~127 字符的话, Latin1、GB 系列、UTF8 等编码是完全相同的。而 UTF16 和 UTF32 则会有额外的空白。

```
mysql> select
    hex(convert("sql" using latin1)) as a,
    hex(convert("sql" using gbk)) as b,
    hex(convert("sql" using utf8mb4)) as c,
    hex(convert("sql" using utf16)) as d,
    hex(convert("sql" using utf32)) as e;
```

a	b	c	d	e
73716C	73716C	73716C	00730071006C	00000073000000710000006C

使用如下 SQL 语句可以看到当前 MySQL 实例支持的字符集(和 collation)。

```
mysql> SELECT CHARACTER_SET_NAME, COLLATION_NAME, ID FROM INFORMATION_SCHEMA.COLLATIONS
    ORDER BY CHARACTER_SET_NAME, ID;
```

```
...
```

gb18030	gb18030_chinese_ci	248
gb18030	gb18030_bin	249

gb18030	gb18030_unicode_520_ci	250
...		
utf8mb4	utf8mb4_general_ci	45
utf8mb4	utf8mb4_bin	46
utf8mb4	utf8mb4_unicode_ci	224
...		

题外：什么是 collation

A collation is a set of rules that defines how to compare and sort character strings.

MySQL 文档（<https://dev.mysql.com/doc/refman/8.0/en/adding-collation.html>）：collation 是进行字符串比较或排序时使用的规则。例如：

- 大小写是否敏感(case sensitive / insensitive)
- 特殊的相等规则: `ij = ji`

2、MySQL 系列参数

执行 `SQL show variables like 'character%'` 可以获得 MySQL 中关于字符集的几个参数。

```
character_set_client
character_set_connection
character_set_results
character_set_server
-- 注：下面 3 个参数一般无需关心
character_set_filesystem
character_set_database
character_set_system
```

2.1 参数 character_set_client

- `character_set_client` 并不是客户端使用的字符集。
- 用户输入什么数据，客户端就发送什么数据。字符集由用户输入决定。
- `character_set_client` 是服务端的参数。
- 服务端认为客户端发来的数据是以 `character_set_client` 编码的。

`character_set_client` 不能被设置为以下几个字符集 (参考链接：

<https://dev.mysql.com/doc/refman/8.0/en/charset-connection.html#charset-connection-impermissible-client-charset>)

```
ucs2
utf16 / utf16le
utf32
```

不难发现其共同点为：都是多字节编码。至于限制的原因本文不作展开。

2.2 参数 character_set_connection

根据 MySQL 文档 (<https://dev.mysql.com/doc/refman/8.0/en/charset-connection.html>) :

(after receiving them) The server converts statements sent by the client from `character_set_client` to `character_set_connection`. Exception: For string literals that have an introducer such as `_utf8mb4` or `_latin2...`

即收到客户端发来的语句后, 服务端(mysql)会将语句从 `character_set_client` 转换为 `character_set_connection`。(除了指明字符集的字符串字面量, 如 `_utf8mb4` 中文字符串".)

2.3 SET NAMES 语句

```
mysql> SET NAMES gb2312;
```

MySQL 文档: 该语句设定 3 个关于 `character set` 的变量:

`character_set_client`, `character_set_connection` 和 `character_set_results`。该语句不设置 `character_set_server`。

注: MySQL 客户端的 `--default-character-set` 参数效果和 `SET NAMES` 语句一样。

```
$ mysql -h ... --default-character-set=gb2312
```

3、一个乱码原因的勘误

之前发表的第 09 期: 有关 MySQL 字符集的乱码问题 “一、转码失败” 一节中有如下案例

```
-- 我的终端字符集是 utf8
...

-- 新建立一个连接, 客户端这边字符集为 gb2312
root@ytt-pc:/home/ytt# mysql --default-character-set=gb2312 ...

...
-- 表的字符集为 utf8
mysql> create database ytt_new10;
mysql> create table ytt_new10.t1(a1 varchar(100)) charset utf8mb4;

-- 插入一条数据, 有两条警告信息
mysql> insert into ytt_new10.t1 values ("病毒滚吧!");
Query OK, 1 row affected, 2 warnings (0.01 sec)
...

-- 那检索出来看到, 数据已经不可逆的乱码了。
mysql> select * from ytt_new10.t1;
+-----+
| a1      |
+-----+
| ???  |
```

该文认为乱码原因是“客户端编码设置成和表编码不一致”。

但可以发现：即使客户端和表的编码都是 `gb2312`，仍然会产生乱码。

```
-- 在上述 gb2312 MySQL session 中
mysql> create table ytt_new10.t2(a1 varchar(100)) charset gb2312;
mysql> insert into ytt_new10.t2 values ("病毒滚吧!");
Query OK, 1 row affected, 2 warnings (0.02 sec)

mysql> select * from ytt_new10.t2;
+-----+
| a1    |
+-----+
|      |
+-----+
```

事实上，MySQL 知道 `character_set_client = gb2312`，也知道 `t1` 表编码为 `utf8`。写入过程中，MySQL 会进行转换，不应当发生乱码。

发生乱码的真正原因是：我们发送给 MySQL 的数据，并不是以 `gb2312` 编码的。是我们欺骗了 MySQL。

`insert ... ("病毒滚吧!")` 这个语句，是以终端字符集 `utf8` 编码的。MySQL 把 `utf8` 数据当作 `gb2312` 数据，转换成 `utf8` 数据，自然产生了乱码。让我们以 `gb2312` 发送数据：

```
# 终端编码为 UTF-8。使用 iconv 转换文件编码。
$ echo 'insert into ytt_new10.t1 val("病毒滚吧!");' | iconv -f utf8 -t gb2312 > insert-gb.txt
$ cat insert-gb.txt # 终端无法正常显示 GB2312 字符
insert into t1 val("???????y?");
$ mysql -h ... --default-character-set=gb2312 < insert-gbk.txt
```

SELECT 时，MySQL 会把数据转换成 `character_set_results` 再返回给客户端。终端可能不支持非 `UTF8` 字符的显示，需要转换。

```
$ mysql ... --default-character-set=utf8 -e 'select * from ytt_new10.t1'
病毒滚吧!
$ mysql ... --default-character-set=gb2312 -e 'select...' | iconv -f gb2312 -t utf8
病毒滚吧!
```

4、字符集 client 到 connection 转换的意义

收到客户端发来的语句后，服务端会将语句从 `character_set_client` 转换为 `character_set_connection`

这个步骤乍一看多此一举

- 直接以 `client` 字符集执行，MySQL 也会正常转换到表字符集。
- 也可以让客户端直接以 `connection` 字符集发送语句。

那么，为什么不将两个参数合并呢？搜索发现，使用转换层的潜在原因如下：

不同字符集的某些行为是完全不同的：

```
mysql> set character_set_connection = 'utf8';
106
```

```
mysql> select length("hello");
|          5 |

mysql> set character_set_connection = 'utf32';
mysql> select length("hello");
|         20 |
```

由于 `character_set_client` 的限制，客户端不能直接以 UTF32 等字符集发送语句。如果我们就是想要 UTF32 下的行为（函数结果、排序规则等），就需要由 MySQL 进行一层转换。

2.13 MySQL 并行 DDL

作者：李鹏博

随着 MySQL 版本的不断更新，对 DDL 操作的支持也在不断的完善和更新：比如从 MySQL 5.6 引入 Online DDL，在 MySQL 5.7 对 Online DDL 进一步完善，到现在的 8.0 版本，则对 DDL 的实现重新进行了设计，比如 DDL 操作支持原子特性，在 MySQL 8.0.27 引入并行 DDL。本篇就来探究一下 MySQL 8.0.27 的并行 DDL 对于 DDL 操作速度的提升。

MySQL 8.0.14 引入了 `innodb_parallel_read_threads` 变量来控制扫描聚簇索引的并行线程。MySQL 8.0.27 引入了 `innodb_ddl_threads` 变量来控制用于创建二级索引时的并行线程数量，此参数一般和一并引入的 `innodb_ddl_buffer_size` 一起使用，`innodb_ddl_buffer_size` 用于指定进行并行 DDL 操作时能够使用的 buffer 大小，buffer 是在所有的 DDL 并行线程中平均分配的，所以一般如果调大 `innodb_ddl_threads` 变量时，也需要调大 `innodb_ddl_buffer_size` 的大小。

`innodb_ddl_threads`、`innodb_ddl_buffer_size` 和 `innodb_parallel_read_threads` 的默认大小分别为：

```
mysql> select @@global.innodb_ddl_threads;
+-----+
| @@global.innodb_ddl_threads |
+-----+
| 4 |
+-----+
1 row in set (0.00 sec)

mysql> select @@global.innodb_ddl_buffer_size;
+-----+
| @@global.innodb_ddl_buffer_size |
+-----+
| 1048576 |
+-----+
```

```

1 row in set (0.00 sec)

mysql> select @@global.innodb_parallel_read_threads;
+-----+
| @@global.innodb_parallel_read_threads |
+-----+
| 4 |
+-----+
1 row in set (0.00 sec)

```

接下来测试一下调大 `innodb_ddl_threads`、`innodb_ddl_buffer_size` 和 `innodb_parallel_read_threads` 参数值对 DDL 操作的性能提升。

首先创建一张 5000 万的表：

```

-- 数据库版本为 8.0.28
mysql> select @@version;
+-----+
| @@version|
+-----+
| 8.0.28 |
+-----+
1 row in set (0.00 sec)

-- buffer pool 大小为 24G
mysql> select @@global.innodb_buffer_pool_size;
+-----+
| @@global.innodb_buffer_pool_size |
+-----+
| 25769803776 |
+-----+
1 row in set (0.001 sec)

mysql> create database action;
Query OK, 1 row affected (0.01 sec)

# sysbench /usr/share/sysbench/oltp_read_write.lua --mysql-
socket=/data/mysql/data/3306/mysql.sock --mysql-user=root --mysql-password='123' --
mysql-db=action --tables=1 --table-size=50000000 --report-interval=1 --threads=8
prepare

mysql> select count(*) from action.sbtest1;
+-----+
| count(*) |
+-----+
| 50000000 |
+-----+

```



```
+-----+
1 row in set (21.64 sec)

-- 表空间大小为 12G
# ll -h
total 12G
-rw-r-----. 1 mysql mysql 12G Jan 20 17:38 sbtest1.ibd
```

分别测试不同的线程数量和缓冲区大小的 DDL 操作时间，例如：

```
-- 设置并发 DDL 线程为 1
mysql> set innodb_ddl_threads = 1;
Query OK, 0 rows affected (0.01 sec)

-- 设置 buffer 大小为 512M
mysql> set innodb_ddl_buffer_size = 536870912;
Query OK, 0 rows affected (0.00 sec)

-- 设置并行索引扫描线程为 1
mysql> set innodb_parallel_read_threads = 1;
Query OK, 0 rows affected (0.01 sec)

-- 执行 DDL 操作
mysql> alter table action.sbtest1 add index idx_c(c);
Query OK, 0 rows affected (6 min 54.21 sec)
Records: 0 Duplicates: 0 Warnings: 0

-- 查看 DDL 的内存最大占用
mysql> select event_name,CURRENT_NUMBER_OF_BYTES_USED/1024/1024 from
performance_schema.memory_summary_global_by_event_name where
event_name='memory/innodb/ddl';
+-----+
| event_name | CURRENT_NUMBER_OF_BYTES_USED/1024/1024 |
+-----+
| memory/innodb/ddl | 513.08750916 |
+-----+
1 row in set (0.00 sec)
```

通过不断调整相关参数得到以下结果：

innodb_ddl_threads	innodb_ddl_buffer_size	innodb_parallel_read_threads	DDL 占用 最大 内存	DDL 时间

innodb_ddl_threads	innodb_ddl_buffer_size	innodb_parallel_read_threads	DDL 占用 最大 内存	DDL 时间
1	512M	1	513M	6 min 54.21 sec
2	1G	2	1230M	4 min 12.08 sec
4	2G	4	2735M	3 min 43.01 sec
8	4G	8	5791M	3 min 19.63 sec
16	8G	16	5975M	3 min 12.33 sec
32	16G	32	6084M	3 min 11.11 sec

可以看到，随着并发线程的增多和 buffer 的增加，DDL 操作所占用的资源也越多，而 DDL 操作所花费的时间则越少。不过通过对比资源的消耗和 DDL 速度的提升比例，最合理的并行线程数量为 4-8 个，而 buffer 大小可以根据情况进行调整。

参考链接：<https://dev.mysql.com/doc/refman/8.0/en/online-ddl-parallel-thread-configuration.html>

2.14 聊聊 MySQL 关机的故事

作者：莫善

背景

这两天看到一个 MySQL 群里在讨论一个有趣的话题，大家平时都是怎么关闭 MySQL 的，一个大佬还发起了一个投票。投票如下：

你是如何关闭 MySQL 数据库的？

- A、mysqladmin shutdown
- B、service mysqld stop (systemctl)
- C、kill mysqld_pid
- D、kill -9 mysqld_pid

投票结果（数据来源 InsideMySQL 公众号）

选项	人数	占比
A	141	33.9%
B	243	58.4%
C	15	3.6%
D	17	4.1%

生产环境中基本都是多实例部署，所以用 A 的方式关闭比较多，偶尔也会贪方便直接采用 C 的关闭方式，如果是单机单实例，用 B 也没毛病，但是为什么会有人选 D 选项呢，发起投票的大佬逐一问过后得知，都是因为当时 MySQL 已经不可用了，迫不得已才采用暴力关闭。

最后，大佬最终公布答案说，生产环境有且只有一种正确的关闭 MySQL 的方式，那就是 D 方式，所以在大佬看来，几乎团灭。

对这个公布结果我是持怀疑态度的，所以我带着怀疑做了下面的测试。

仅对 5.7 的半同步场景做了测试，对于异步场景感觉没什么意义，所以没测。

一、环境介绍

环境架构采用 MySQL 5.7 增强半同步，搭建的一主一从，信息如下：

角色	ip	端口	版本
master	192.168.168.11	6666	5.7.26
slave	192.168.168.12	6666	5.7.26

1、主库配置

```
mysql> show variables like 'rpl%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| rpl_semi_sync_master_enabled | ON |
| rpl_semi_sync_master_timeout | 1000000 |
| rpl_semi_sync_master_trace_level | 32 |
| rpl_semi_sync_master_wait_for_slave_count | 1 |
| rpl_semi_sync_master_wait_no_slave | ON |
| rpl_semi_sync_master_wait_point | AFTER_SYNC |
| rpl_semi_sync_slave_enabled | OFF |
| rpl_semi_sync_slave_trace_level | 32 |
| rpl_stop_slave_timeout | 31536000 |
+-----+-----+
9 rows in set (0.00 sec)

mysql>
```

`rpl_semi_sync_master_timeout = 1000000` 是为了避免半同步降级为异步。

2、从库配置

```
mysql> show variables like 'rpl%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| rpl_semi_sync_master_enabled | OFF |
| rpl_semi_sync_master_timeout | 1000000 |
| rpl_semi_sync_master_trace_level | 32 |
| rpl_semi_sync_master_wait_for_slave_count | 1 |
| rpl_semi_sync_master_wait_no_slave | ON |
| rpl_semi_sync_master_wait_point | AFTER_SYNC |
| rpl_semi_sync_slave_enabled | ON |
| rpl_semi_sync_slave_trace_level | 32 |
| rpl_stop_slave_timeout | 31536000 |
+-----+-----+
9 rows in set (0.00 sec)

mysql>
```

二、测试演示

测试准备

登录主库准备测试数据

```
# /opt/soft/mysql57/bin/mysql -u dba -h 192.168.168.11 -P6666 -p123456

mysql> create database if not exists dbatest;
Query OK, 1 row affected, 1 warning (0.00 sec)

mysql> use dbatest
Database changed
mysql> create table t(id int not null auto_increment primary key,name varchar(10) default '');
Query OK, 0 rows affected (0.01 sec)

mysql> insert into t select 0,'mysql';
Query OK, 1 row affected (0.00 sec)
Records: 1 Duplicates: 0 Warnings: 0

mysql> insert into t select 0,'redis';
Query OK, 1 row affected (0.01 sec)
Records: 1 Duplicates: 0 Warnings: 0

mysql> select * from t;
+----+-----+
| id | name |
+----+-----+
|  1 | mysql |
|  2 | redis |
+----+-----+
2 rows in set (0.00 sec)

mysql>
```

登录从库检查测试数据

```
# /opt/soft/mysql57/bin/mysql -u dba -h 192.168.168.12 -P6666 -p123456

mysql> use dbatest
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select * from t;
+----+-----+
| id | name |
+----+-----+
|  1 | mysql |
|  2 | redis |
```

```
+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

1、安全关闭的场景

(1) 停掉从库的复制

```
# /opt/soft/mysql57/bin/mysql -u dba -h 192.168.168.12 -P6666 -p123456

mysql> stop slave;
Query OK, 0 rows affected (0.01 sec)

mysql>
```

模拟半同步故障，为了更直观查阅测试结果。

(2) 模拟业务更新

这个流程涉及两个操作，一是在主库发起一个更新请求，二是查看主库的 processlist 状态。

● 操作一

```
# /opt/soft/mysql57/bin/mysql -u dba -h 192.168.168.11 -P6666 -p123456 dbatest -
e "update t set name = 'tidb' where id = 1"
```

这个操作回车后会卡着，因为这个请求会等从库的 ACK。具体可以看【操作二】的查询结果。

● 操作二

```
# /opt/soft/mysql57/bin/mysql -u dba -h 192.168.168.11 -P6666 -p123456

mysql> show processlist;
+-----+-----+-----+-----+-----+-----+-----+-----+
| Id | User | Host                | db    | Command | Time | State                | Info                                |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 8  | dba  | 192.168.168.13:49584 | dbatest | Query   | 49   | Waiting for semi-sync ACK from slave | update t set name = 'tidb' where id = 1 |
| 9  | root | localhost           | NULL  | Query   | 0    | starting             | show processlist                   |
+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

(3) 关闭主库的 MySQL

这个流程向主库提交 shutdown 命令来安全关闭 MySQL。

```
# /opt/soft/mysql57/bin/mysql -u dba -h 192.168.168.11 -P6666 -p123456

mysql> shutdown;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

提交完 shutdown 操作后，【操作一】的更新请求会被提交。

- 在业务端看来，id=1 这行数据已经被修改，但是从库的 id=1 这行数据未改变。如果这个时候触发主从切换，那就丢数据了。由此看来，在特殊场景下，安全关闭 MySQL 可能导致数据丢失。
- 这时候将主库重新拉起来，修好主从的半同步，被提交的事务还是会同步到从库的。

2、暴力关闭的场景

（1）停掉从库的复制

```
# /opt/soft/mysql57/bin/mysql -u dba -h 192.168.168.12 -P6666 -p123456

mysql> stop slave;
Query OK, 0 rows affected (0.01 sec)

mysql>
```

模拟半同步故障，为了更直观查阅测试结果。

（2）模拟业务更新

这个流程涉及两个操作，一是在主库发起一个更新请求，二是查看主库的 processlist 状态。

● 操作一

```
# /opt/soft/mysql57/bin/mysql -u dba -h 192.168.168.11 -P6666 -p123456 dbatest -e "update t set name = 'codis' where id = 2"
```

这个操作回车后会卡着，因为这个请求会等从库的 ACK。具体可以看【操作二】的查询结果。

● 操作二

```
# /opt/soft/mysql57/bin/mysql -u dba -h 192.168.168.11 -P6666 -p123456

mysql> show processlist;
+----+-----+-----+-----+-----+-----+-----+
| Id | User | Host                | db    | Command | Time | State |
|----|-----|-----+-----+-----+-----+-----+
| 2  | dba  | 192.168.168.13:34698 | dbatest | Query   | 13   | Waiting for semi-sync ACK from slave | update t set name = 'codis' where id = 2 |
| 3  | root | localhost           | NULL   | Query   | 0    | starting | show processlist |
+----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

（3）暴力关闭主库的 MySQL

这个操作向主库所在服务器的 MySQL 进程发送 kill -9 信号，来暴力关闭 MySQL。

```
# ps -ef|grep mysql|grep 6666
root      123194      1  0 15:45 pts/9    00:00:00 /bin/sh /opt/soft/mysql57/bin/mysqld_
safe --defaults-file=/work/mysql6666/etc/my6666.cnf
mysql     124539 123194  4 15:45 pts/9    00:00:02 /opt/soft/mysql57/bin/mysqld --
defaults-file=/work/mysql6666/etc/my6666.cnf --basedir=/opt/soft/mysql57 --
datadir=/work/mysql6666/var --plugin-dir=/opt/soft/mysql57/lib/plugin --user=mysql --
log-error=/work/mysql6666/log/mysql.err --open-files-limit=65535 --pid-
file=/work/mysql6666/var/mysql.pid --socket=/work/mysql6666/tmp/mysql.sock --port=6666
# kill -9 124539 123194
#
```

kill -9 操作后，【操作一】所在的会话会被终止，并提示【ERROR 2013 (HY000) at line 1: Lost connection to MySQL server during query】

- 在业务端看来，id=2 这行数据没有被修改，所以数据是一致的。

三、停服流程介绍

本小节是通过阅读了 MySQL 官方文档后，简单介绍一下 MySQL 在接收了 SIGINT 信号后会做些什么事情，仅供参考。

该部分文档连接 <https://dev.mysql.com/doc/refman/5.7/en/server-shutdown.html>

1、停止接受新连接

为了预防在关闭系统过程再接收新任务，MySQL 会关闭 TCP/IP 端口、socket 等通道来阻止接受新的客户端连接。

2、处理已经建立的连接

这个过程会处理已经建立的连接，对于空闲连接会马上杀掉。对于当前正在处理任务的线程（会定期检查它们的状态），如果是正在执行的未完成的事务，会回滚（非事务的引擎会导致部分失败部分成功），所以这个过程需要更长的时间。

针对这个我有一个疑问。在【安全关闭的测试场景】中，正在等待 ACK 的任务会被提交，那么跟这个流程就有出入，所以我猜测这个流程之前会先关闭【等待 ACK 的线程】，然后再处理已经建立的连接，这样就能说的通，在【安全关闭的测试场景】中正在等待 ACK 的事务被提交了。当然这个思路仅是通过 MySQL 错误日志（如下展示）记录得出的猜测，仅供参考。

2022-03-

25T17:17:58.165016+08:00 0 [Note] Giving 1 client threads a chance to die gracefully

2022-03-25T17:17:58.165059+08:00 0 [Note] Shutting down slave threads

2022-03-

25T17:17:58.165070+08:00 3 [Warning] SEMISYNC: Forced shutdown. Some updates might not be replicated.

2022-03-25T17:17:58.165092+08:00 3 [Note] Semi-sync replication switched OFF.

2022-03-25T17:17:58.165654+08:00 0 [Note] Forcefully disconnecting 0 remaining clients

2022-03-25T17:17:58.165680+08:00 0 [Note] Event Scheduler: Purging the queue. 0 events


```

2022-03-25T17:17:58.165919+08:00 0 [Note] Binlog end
2022-03-25T17:17:58.167688+08:00 0 [Note] Shutting down plugin 'rpl_semi_sync_slave'
2022-03-25T17:17:58.167719+08:00 0 [Note] Shutting down plugin 'rpl_semi_sync_master'
2022-03-25T17:17:58.167745+08:00 0 [Note] Stopping ack receiver thread
2022-03-25T17:17:58.167831+08:00 0 [Note] unregister_replicator OK
2022-03-25T17:17:58.167843+08:00 0 [Note] Shutting down plugin 'ngram'

省略 shutting down plugin

2022-03-25T17:17:58.167985+08:00 0 [Note] Shutting down plugin 'InnoDB'
2022-03-25T17:17:58.168053+08:00 0 [Note] InnoDB: FTS optimize thread exiting.
2022-03-25T17:17:58.168140+08:00 0 [Note] InnoDB: Starting shutdown...
2022-03-
25T17:18:00.704250+08:00 0 [Note] InnoDB: Shutdown completed; log sequence number 63089
498676
2022-03-
25T17:18:00.705160+08:00 0 [Note] InnoDB: Removed temporary tablespace data file: "ibtm
p1"
2022-03-25T17:18:00.705183+08:00 0 [Note] Shutting down plugin 'MEMORY'

省略 shutting down plugin

2022-03-25T17:18:00.705547+08:00 0 [Note] Shutting down plugin 'binlog'
2022-03-
25T17:18:00.706904+08:00 0 [Note] /opt/soft/mysql57/bin/mysqld: Shutdown complete

```

可以看到 shutdown 的整个过程，其中有一个【Stopping ack receiver thread】

3、关闭存储引擎

在这个阶段，服务器会刷新表缓存并关闭所有打开的表。InnoDB 将其缓冲池刷新到磁盘（除非 `innodb_fast_shutdown = 2`），将当前 LSN 写入表空间，并终止其自己的内部线程。

4、服务器关闭

进程退出，端口关闭。

四、写在最后

测试结果有点惊讶，曾经认为危险的操作命令却是安全的，曾经认为安全的操作命令反而会导致数据异常。即便如此，我觉得也不用太过纠结了，毕竟丢数据的场景还是很苛刻的。

最后需要提醒一下，在 5.7 版本以前，要慎用 `kill -9`。生产环境十分复杂，请时刻保持敬畏之心，任何操作请充分测试。

2.15 MySQL host-cache

作者：高鹏

前言

问题如下：客户生产环境中有一个一主一从半同步的集群，运维同事发现连接主库的时候很快，但是连接从库的时候就很慢，故此咨询原因；

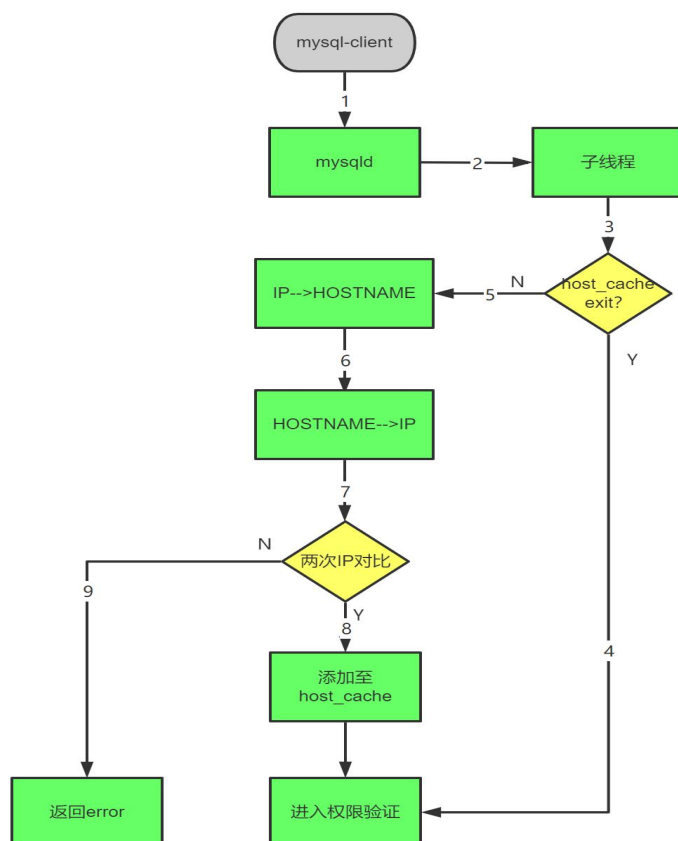
拿到这个问题的时候觉得现象确实挺怪异，所以就进行了抓包对比。连接从库时的抓包结果：

No.	Time	Source	Destination	Protocol	Query	Length	Info
1	2021-04-12 18:58:54.086223	.9	.34	TCP		74	40362 → mysql(3306) [SYN] Seq=0 Win=29
2	2021-04-12 18:58:54.086283	.9	.34	TCP		74	mysql(3306) → 40362 [SYN, ACK] Seq=0 A
3	2021-04-12 18:58:54.086446	.9	.34	TCP		66	40362 → mysql(3306) [ACK] Seq=1 Ack=1
4	2021-04-12 18:59:04.096996	.9	.34	MySQL		148	Server Greeting proto=10 version=5.7.1
5	2021-04-12 18:59:04.097370	.9	.34	TCP		66	40362 → mysql(3306) [ACK] Seq=1 Ack=83
6	2021-04-12 18:59:04.097444	.9	.34	MySQL		253	Login Request user=universe_op
7	2021-04-12 18:59:04.097460	.9	.34	TCP		66	mysql(3306) → 40362 [ACK] Seq=83 Ack=1
8	2021-04-12 18:59:04.097554	.9	.34	MySQL		114	Auth Switch Request
9	2021-04-12 18:59:04.097785	.9	.34	MySQL		90	Auth Switch Response
10	2021-04-12 18:59:04.097855	.9	.34	MySQL		77	Response OK
11	2021-04-12 18:59:04.098143	.9	.34	MySQL	Query	103	Request Query { select @@version_comme
12	2021-04-12 18:59:04.098368	.9	.34	MySQL		158	Response
13	2021-04-12 18:59:04.137722	.9	.34	TCP		66	40362 → mysql(3306) [ACK] Seq=249 Ack=

从抓包结果可以看到三步握手后等待了 10 秒才收到 MySQL 给客户端的响应，那么问题出现在 MySQL 这侧，什么情况下会导致 MySQL 出现该问题？当我们对比了主从配置文件后发现从库比主库少了 skip-name-resolve，从库加上重启后问题得以解决。

正文

1. MySQL 反向解析的过程



1. mysql-client 向 mysqld 发起连接请求;
2. mysqld 创建子线程来处理该连接请求;
3. 子线程首先去检查主机名是否存在于 performance_schema.host_cache 表中;
4. 如果是, 则进入权限验证阶段;
5. 如果否, 那么会尝试解析将 IP 地址解析为主机名;
6. 然后再将主机名解析为 IP 地址;
7. 将解析到的结果和原始 IP 进行比较;
8. 如果相同则添加到 host_cache 中然后进行权限验证;
9. 如果不相同, 那么返回报错

注: 只有非本地主机的 TCP 连接会使用主机缓存; 使用网络回环地址或 socket 建立的连接不会使用主机缓存

2. 主机缓存的作用

- 通过缓存 IP 到主机名查找的结果, 可以避免每次客户端进行连接时都去寻找 DNS 进行解析, 所能缓存条目的数量受 host_cache_size 控制。
- 缓存中包含客户端连接过程中发生的错误信息。如果同一主机连续多次连接错误, 那么将阻止该主机的进一步连接, 该行为受 max_connect_errors 控制。

3. 参数 host_cache_size 与 skip-host-cache 和 skip_name_resolve 的作用

host_cache_size: 控制主机缓存的数量, 设置为 0 时, 禁用主机缓存, 每次服务器连接时, 服务器都会进行 DNS 查找, 相比 skip-host-cache 来说, host_cache_size 更加灵活, 可以随时变更或禁用。

skip-host-cache: 作用类似于 host_cache_size, 但 MySQL 在运行时无法对该参数进行变更, 并且 skip-host-cache 在之后的版本中已弃用。

skip_name_resolve: 检查客户端连接时是否解析主机名, 变量为 off, 则 mysqld 会在检查客户端连接时解析主机名。若变量为 on, mysqld 只使用 IP; 在这种情况下, 授权表中的所有列值都必须是 IP 地址。

4. 如何查看 host_cache 表

```
mysql> select * from performance_schema.host_cache\G
***** 1. row *****
IP: 10.186.61.21 #连接到服务器的 IP 地址
HOST: NULL #客户端 DNS 解析出的主机名, null 代表未知
HOST_VALIDATED: YES #是否执行了 DNS 解析, 如果值为 YES 则主机列将用作与 IP 相对应的主机名, 以便
可以避免对 DNS 的其他调用; 如果值为 NO 则每次连接尝试都会尝试 DNS 解析, 直到最终以有效结果或永久错
误完成解析
SUM_CONNECT_ERRORS: 0 #被视为“阻塞”的连接错误数, 仅统计协议握手错误, 并且仅统计
HOST_VALIDATED=YES 的主机
COUNT_HOST_BLOCKED_ERRORS: 0 #以下为各种连接方式失败的计数
COUNT_NAMEINFO_TRANSIENT_ERRORS: 0
COUNT_NAMEINFO_PERMANENT_ERRORS: 1
COUNT_FORMAT_ERRORS: 0
COUNT_ADDRINFO_TRANSIENT_ERRORS: 0
```

```

COUNT_ADDRINFO_PERMANENT_ERRORS: 0
COUNT_FCRDNS_ERRORS: 0
COUNT_HOST_ACL_ERRORS: 0
COUNT_NO_AUTH_PLUGIN_ERRORS: 0
COUNT_AUTH_PLUGIN_ERRORS: 0
COUNT_HANDSHAKE_ERRORS: 0
COUNT_PROXY_USER_ERRORS: 0
COUNT_PROXY_USER_ACL_ERRORS: 0
COUNT_AUTHENTICATION_ERRORS: 17
COUNT_SSL_ERRORS: 0
COUNT_MAX_USER_CONNECTIONS_ERRORS: 0
COUNT_MAX_USER_CONNECTIONS_PER_HOUR_ERRORS: 0
COUNT_DEFAULT_DATABASE_ERRORS: 0
COUNT_INIT_CONNECT_ERRORS: 0
COUNT_LOCAL_ERRORS: 0
COUNT_UNKNOWN_ERRORS: 0
FIRST_SEEN: 2020-12-18 10:30:03 #客户端第一次尝试连接的时间戳
LAST_SEEN: 2020-12-18 14:50:41 #客户端看到最新的连接尝试的时间戳
FIRST_ERROR_SEEN: 2020-12-18 10:30:03 #第一次连接错误的时间戳
LAST_ERROR_SEEN: 2020-12-18 14:46:37 #最近一次连接错误的时间戳
1 row in set (0.00 sec)

```

提示：执行 `flush hosts` 可以刷新 `host_cache`，刷新后会清除内存中的主机缓存

5. host-cache 和 skip_name_resolve 参数对数据库的影响

环境信息: 4C 4G KVM虚拟机 测试工具: sysbench oltp_read_write.lua --table-size=100000 --tables=2 --threads=30	
skip_name_resolve =OFF 、 host_cache_size=700	host_cache_size=0、 skip_name_resolve =OFF
#在没有DNS与hosts情况下:	TPS:1150.63
TPS: 1234.97	QPS:23036.34
QPS: 24690.34	
#在有DNS或hosts时	
TPS: 1255.04	
QPS: 25097.89	

结论:

结论和想象中一样，当 `skip_name_resolve =OFF` 、 `host_cache_size=700` 时，只有在刚建立连接进行 DNS 解析和 `host_cache` 缓存时会影响 MySQL 性能，之后再次进行压测时，可以看到性能有少许的提升。

2.16 你通常选用什么工具在 Linux 中进行数据传输？

作者：刘开洋

1、问题

前几天跟客户聊天，抱怨服务器之前的数据传输工具不好用，问我哪种传输工具效率高？还真被问住了，毕竟没经过实践嘛，今天就测试下。

有点草率得搜索了下，不过工具还不少。



数据传输工具：ftp sftp scp rsync tftp

2、准备工作

刨除网络带宽等其他因素，我们就 Linux 服务器之间传输文件做个对比，看看到底什么工具才是最快、最适合我的场景。

我们就上面这些传输工具在文件实际大小和文件数量多少的层次上分别做个速度和简便性对比：

首先准备环境，配置两个机器 ssh 互信，创建两个文件夹，以不同的方式创建一个 50G 左右的大文件和 51200 个 1M 的小文件：

```
# 制造 50G 大文件
[root@yang-01 big]# fallocate -l 50G 50g_file
[root@yang-01 big]# ll
total 52428856
-rw-r--r-- 1 root root 53687091200 Apr 10 17:55 50g_file
[root@yang-01 big]# du -sh *
51G 50g_file

# 制造 51200 个 1M 文件
[root@yang-01 many]# seq 51200 | xargs -i dd if=/dev/zero of=1m_file{} bs=1M count=1
1+0 records in
```

```

1+0 records out
1048576 bytes (1.0 MB) copied, 0.00468854 s, 224 MB/s
. . . . .
[root@yang-01 many]# ls | wc -l
51200
[root@yang-01 test]# du -sh many
51G many

```

小提示：这里采用了两种方式制造大文件，大家可以参考使用，生成文件确实是 50G，字节数与之对应，至于为什么 du 工具计算出来是 51G，这里并不是计算机 1024 与 1000 的数值换算的原因，而是因为 du 命令计算的是该文件累计使用到文件系统的总块数，可能出现使用的部分块空间占用并不完整的情况，导致数值偏大。

2.1、测试 SCP

第一个测试的是 scp，这是作者使用频率最多的一个远程传输工具，通常 Linux 系统会安装有该工具。

```

##### 1 * 50G 文件测试
[root@yang-02 big]# scp /opt/test/big/50g_file root@yang-01:/opt/test/re/
50g_file 100% 50GB 135.5MB/s 06:17

##### 51200 * 1M 文件测试
[root@yang-02 many]# time scp /opt/test/many/1m_file* root@yang-01:/opt/test/re/
1m_file1 100% 1024KB 22.5MB/s 00:00
1m_file10 100% 1024KB 35.8MB/s 00:00
1m_file100 100% 1024KB 14.8MB/s 00:00
1m_file1000 100% 1024KB 32.9MB/s 00:00
1m_file10000 100% 1024KB 35.7MB/s 00:00
. . . . .
1m_file9998 100% 1024KB 113.4MB/s 00:00
1m_file9999 100% 1024KB 96.5MB/s 00:00

real 20m43.875s
user 4m2.448s
sys 2m52.604s
[root@yang-01 re]# ls | wc -l
51200

```

说明

- 类似于 cp 命令，由于跨机器文件加密传输导致其速度稍慢于 copy 操作；
- 如果存在生产环境中无法使用 ssh 协议的情况，可以采用 nc 文件传输；
- 优点：该工具对于机器的系统资源占用少、影响较小，使用方便；

2.2、测试 FTP

下面测一下 ftp，看看效果如何。

```
##### 1 * 50G 文件测试
[root@yang-01 re]# ftp yang-02
Connected to yang-02 (192.168.88.72).
220 (vsFTPd 3.0.2)
Name (yang-02:root): root
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> get /opt/test/big/50g_file /opt/test/re/50g_file
local: /opt/test/re/50g_file remote: /opt/test/big/50g_file
227 Entering Passive Mode (192,168,88,72,38,232).
150 Opening BINARY mode data connection for /opt/test/big/50g_file (53687091200 bytes).
226 Transfer complete.
53687091200 bytes received in 150 secs (359091.49 Kbytes/sec)
ftp> quit
221 Goodbye.

##### 51200 * 1M 文件测试
[root@yang-01 re]# time ftp yang-02
Connected to yang-02 (192.168.88.72).
220 (vsFTPd 3.0.2)
Name (yang-02:root):
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> prompt off
Interactive mode off.
ftp> cd /opt/test/many
250 Directory successfully changed.
ftp> mget * .*
local: 1m_file1 remote: 1m_file1
227 Entering Passive Mode (192,168,88,72,156,228).
150 Opening BINARY mode data connection for 1m_file1 (1048576 bytes).
226 Transfer complete.
. . . . .
1048576 bytes received in 0.00337 secs (311057.86 Kbytes/sec)
local: . remote: .
227 Entering Passive Mode (192,168,88,72,213,4).
550 Failed to open file.
Warning: embedded .. in .. (changing to !!)
```

```
local: !! remote: !!
227 Entering Passive Mode (192,168,88,72,223,91).
550 Failed to open file.
ftp> quit
221 Goodbye.

real 14m32.032s
user 0m12.857s
sys 3m21.131s
[root@yang-01 re]# ls | wc -l
51200
```

说明:

- 基于 TCP 传输协议, ftp 客户端发出命令到服务端进行文件下载、上传或者变更目录;
- 适用于内网的文件归集以及公共文件的调阅。
- 偶发性的文件传输, 为了保障各主机之间的安全性, 一般不会使用这种方式进行数据传输。

2.3、测试 SFTP

sftp 就是在 ftp 的基础上补充了 加密/解密 的技术, 我们也一起看下和 ftp 的传输速度差异:

```
##### 1 * 50G 文件测试
[root@yang-01 re]# sftp root@yang-02
Connected to yang-02.
sftp> get ./big/50g_file /opt/test/re/50g_file
Fetching ./big/50g_file to /opt/test/re/50g_file
./big/50g_file 100% 50GB 128.7MB/s 06:37
sftp> quit

##### 51200 * 1M 文件测试
[root@yang-01 re]# time sftp root@yang-02
Connected to yang-02.
sftp> get ./many/1m_file* /opt/test/re/
Fetching ./many/1m_file1 to /opt/test/re/1m_file1
./many/1m_file1 100% 1024KB 77.3MB/s 00:00
. . . . .
Fetching ./many/1m_file9999 to /opt/test/re/1m_file9999
./many/1m_file9999 100% 1024KB 118.0MB/s 00:00
sftp>
sftp> quit

real 19m43.154s
user 4m52.309s
sys 4m47.476s
```



```
[root@yang-01 re]# ls | wc -l
51200
```

说明:

- 相较于 ftp 工具, 在提升安全等级的基础上对 ssh 传输进行加密, sftp 的传输速率降低了 70%左右;
- 使用 sftp 和 ftp 的使用需要搭建服务, 使用麻烦, 且 sftp 的传输速率类似于 scp。

2.4、测试 RSYNC

再看看 rsync 工具的测试:

```
##### 1 * 50G 文件测试
[root@yang-02 big]# time rsync -av ./50g_file root@yang-01:/opt/test/re/50g_file
sending incremental file list
50g_file

sent 53,700,198,488 bytes received 35 bytes 107,940,097.53 bytes/sec
total size is 53,687,091,200 speedup is 1.00

real 8m17.039s
user 5m36.160s
sys 2m41.196s

##### 51200 * 1M 文件测试
[root@yang-02 many]# time rsync -av ./1m_file* root@yang-01:/opt/test/re/
sending incremental file list
1m_file1
1m_file10
1m_file100
. . . . .
1m_file9998
1m_file9999

sent 53,702,886,375 bytes received 972,872 bytes 58,278,740.37 bytes/sec
total size is 53,687,091,200 speedup is 1.00

real 15m21.548s
user 5m46.497s
sys 2m38.581s
```

说明:

- rsync 从比 scp 稍微快一点, Centos 中默认安装;
- 占用带宽少, 因为 rsync 在两端服务器之间传输数据块时会进行解压缩处理。
- 优势: rsync 只会同步发生变化的文件, 如果没有变更, rsync 不会进行覆盖处理, 即 rsync 适合

增量同步（由于场景原因，这里没有过多演示）；

- 如果传输文件数量多的时候，rsync 工具可能会导致磁盘的 I/O 偏高，如果文件系统中存在数据库会产生一定影响。

2.5、测试 TFTP

类似于 sftp，测一下 tftp 的速度怎么样：

```
##### 1 * 50G 文件测试
[root@yang-01 re]# time tftp yang-02
tftp> get ./big/50g_file

real 10m30.114s
user 0m6.029s
sys 1m16.888s
[root@yang-01 re]# ll
total 1805832
-rw-r--r-- 1 root root 1849168384 Apr 11 17:54 50g_file
[root@yang-01 re]# du -sh *
1.8G 50g_file

##### 1 * 1G 文件测试
[root@yang-01 re]# time tftp yang-02
tftp> get ./many/1g_file1
tftp> quit

real 5m54.090s
user 0m29.190s
sys 2m58.866s
[root@yang-01 re]# ll
total 1048576
-rw-r--r-- 1 root root 1073741824 Apr 11 18:09 1g_file1
```

说明：

- tftp 工具是基于 UDP 协议进行数据传输，同样需要配置相关服务，使用过于麻烦；
- 测试 50G 大文件传输仅仅 1.8G，发生超时中断，fail，且时间较长；
- 测试 1 个 1G 文件传输时间为 354s，推测 50 倍时间较长，放弃该工具的测试。

2.6、补充 NC

小编补充 nc 工具肯定是遇到了上述工具无法完成数据传输的场景，我们测下速度：

```
##### 1 * 50G 文件测试
[root@yang-02 big]# nc 192.168.88.71 10086 < /opt/test/big/50g_file
[root@yang-01 re]# time nc -l 10086 > 50G_file
```

```

real 2m30.663s
user 0m9.232s
sys 2m16.370s

##### 51200 * 1M 文件测试
[root@yang-01 many]# tar cfz - *|nc 192.168.88.71 10086
[root@yang-01 re]# time nc -l 10086|tar xfvz -
1m_file1
1m_file10
1m_file100
. . .
1m_file9997
1m_file9998
1m_file9999

real 11m38.400s
user 3m47.051s
sys 2m33.923s

```

说明:

- 这个工具的功能很是强大，可以实现任意 TCP/UDP 端口的监听与扫描；
- 优势：速度相比于 scp 快很多，几乎没有网络协议的开销；
- 跨机器传输文件只是它的功能之一，其他功能等待大家探索，据说可以网络测速哦。

2.7、补充 python 工具

如果还有 nc 无法解决的场景，我们试下使用 python 的 SimpleHTTPServer 模块吧：

```

##### 1 * 50G 文件测试
[root@yang-02 big]# python -m SimpleHTTPServer 10086
Serving HTTP on 0.0.0.0 port 10086 ...
192.168.88.71 - - [13/Apr/2022 16:02:15] "GET /50g_file HTTP/1.1" 200 -

[root@yang-01 re]# wget http://192.168.88.72:10086/50g_file
--2022-04-13 16:02:15-- http://192.168.88.72:10086/50g_file
Connecting to 192.168.88.72:10086... connected.
HTTP request sent, awaiting response... 200 OK
Length: 53687091200 (50G) [application/octet-stream]
Saving to: '50g_file'

100%[=====>] 53,687,091,200 358MB/s in 2m 35s

2022-04-13 16:04:50 (330 MB/s) - '50g_file' saved [53687091200/53687091200]

##### 51200 * 1M 文件测试

```

```
[root@yang-02 many]# python -m SimpleHTTPServer 10086
Serving HTTP on 0.0.0.0 port 10086 ...
192.168.88.71 - - [13/Apr/2022 19:46:02] "GET /1m_file1 HTTP/1.1" 200 -
192.168.88.71 - - [13/Apr/2022 19:46:02] "GET /1m_file2 HTTP/1.1" 200 -
. . . . .
192.168.88.71 - - [13/Apr/2022 19:55:21] "GET /1m_file51200 HTTP/1.1" 200 -

[root@yang-01 re]# cat liu.sh
#!/bin/bash
for ((i=1;i<=51200;i++))
do
echo "http://192.168.88.72:10086/1m_file$i"
done
[root@yang-01 re]# bash liu.sh > liu.list
[root@yang-01 re]# cat liu.list | wc -l
51200
[root@yang-01 re]# wget -i liu.list
--2022-04-13 19:46:02-- http://192.168.88.72:10086/1m_file1
Connecting to 192.168.88.72:10086... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1048576 (1.0M) [application/octet-stream]
Saving to: '1m_file1'

100%[=====>] 1,048,576 --.-K/s in 0.05s

2022-04-13 19:46:02 (19.9 MB/s) - '1m_file1' saved [1048576/1048576]
. . . . .
100%[=====>] 1,048,576 --.-K/s in 0.08s

2022-04-13 19:55:21 (11.9 MB/s) - '1m_file51200' saved [1048576/1048576]

FINISHED --2022-04-13 19:55:21--
Total wall clock time: 9m 19s
Downloaded: 51200 files, 50G in 8m 11s (104 MB/s)
```

说明:

- 使用 python 的 SimpleHTTPServer 模块或者 python3 的 http.server 模块实现了一个轻量级的 HTTP 协议的 Web 服务器;
- Linux 发行版本几乎都内置有 python, 所以该工具使用也较为方便。
- 正常情况下, 多文件传输有些麻烦, 文件名可能无规律, 需要单文件挨个处理。

3、总结

测试工具	1 * 50G 测试时间	1M*51200 测试时间	压缩	加密	速度
SCP	06:17 = 377s	20m43.875s = 1243.875s	-C 选项支持	ssh 协议加密	较快
FTP	150s	14m32.032s = 872.032s	无	无	快
SFTP	06:37 = 397s	19m43.154s = 1183.154s	无	ssh 协议加密	较快
RSYNC	8m17.039s = 657.039s	15m21.548s = 921.548s	-z 选项支持	ssh 协议加密	较快
TFTP	10m30.114s test fail		无	无	慢
NC	2m30.663s = 150.663s	11m38.400s = 698.400s	无	无	快
python 工具	2m 35s = 155s	9m 19s = 559s	无	无	快

1. 大文件传输速率：FTP > NC > python 工具 > SCP > SFTP > RSYNC。
2. 小文件传输速率：python 工具 > NC > FTP > RSYNC > SFTP > SCP。
3. 不同工具具有不同的应用场景，RSYNC 在增量同步或定期归档的情况下优势很大；FTP 工具搭建部署麻烦，适用于内网的文件归集以及公共文件的调阅，安全性低。
4. 如果由于生产安全限制，关闭了 SSH 连接或 22 端口的使用，可以使用基于其他协议的连接访问，推荐 nc 工具。
5. 对比每个工具可能同容量的文件数量测试，小文件会导致 CPU 多次读取匹配信息，增加 CPU 负担，IO 的次数也会增多，存在一定的 IO 瓶颈，同容量的大文件和多个小文件传输对比有一定速度优势。
6. 如果是大量文件传输，可根据文件名特点分批次设置多个传输进程，从而达到源端服务器并发传输的效果，更好的利用网络带宽。

4、备注

- 由于操作限制，以上测试工具均忽略交互式登录的时间。
- 测试为虚拟机环境，无法保证变量绝对可控(网络带宽、吞吐、磁盘性能等影响因素)，因此文中给出的时间节点仅供参考。
- 关于各个工具的使用说明及拓展，这里并没有给出，网上资料很多，各位老板随意爬取。

彩蛋：小编测试时发现各个工具 50 * 1G 的传输速度要快于 1 * 50G 文件，所以在同一环境下，会不会存在一种情况的 50G 容量（单文件容量和数量的交叉匹配）传输速度达到最快？有兴趣的小伙伴可以继续研究下。

2.17 MySQL 审计功能实现方案

作者：刘聪

背景

鉴于目前 MySQL 审计需求不断深入扩张，以提高数据资产的安全，使得数据库愈来愈规范化管理，数据库实现审计功能成为必要安全环节。

众所周知，MySQL 社区版是不带审计插件的（Audit Plugin），那么该如何实现审计功能呢？我们自然会想到使用 binlog 日志做审计，因为 binlog 日志本就记录着数据库上的所有改变。但是无奈，binlog 日志并不会记录用户的登录信息，所以无法做到真正意义上的审计。那使用 general_log 日志呢？

general_log 日志可不仅记录着登录信息，甚至错误的登录信息、select 等所有对数据库的操作都会记录着。也正是因为 general_log 日志记录得过于详细，不好做区分。且当并发访问量过大时，极大的占用空间，影响数据库的性能。所以也不会将之作为实现审计功能的工具。

本文将介绍如下三种方式，实现使用审计功能。

相关说明：本文所使用的工具版本

环境：MySQL - 5.7.25（社区版）

工具：

工具	获取源包名
MariaDB 审计插件	mariadb-10.2.43-linux-systemd-x86_64
McAfee 审计插件	audit-plugin-mysql-5.7-1.1.11-985-linux-x86_64.zip

init_connect 参数

相关说明

init_connect 是社区版 MySQL 自带的参数。MySQL 官方手册释义：“A string to be executed by the server for each client that connects. The string consists of one or more SQL statements, separated by semicolon characters.”

即：在连接客户端时刻，服务器要为每个连接，执行 init_connect 所定义的字符串。这个字符串可以由一个或多个 SQL 语句组成，以分号字符分隔。

那么该如何去利用这个参数呢？要实现审计功能，审计信息至少要包括登录 MySQL 的用户名、登录 IP、登录时间等内容（谁在哪个时刻动了我的数据库啦！）。我们知道，init_connect 参数所定义的字符串在登录连接时刻执行。如果我们提前创建一个表，并且把 init_connect 所定义的字符串写成 insert 语句（登录时刻，插入用户登录的信息）那么利用这个参数的特性，就可以实现简单的审计功能。

实现步骤

1、创建审计日志表

审计日志表记录内容包括：登录数据库的用户、IP、本次登录时间（审计必要信息）：

```
CREATE TABLE `test_db`.`audit` (
  `ID` int(11) NOT NULL AUTO_INCREMENT COMMENT '自增 ID',
  `USER` varchar(64) COLLATE utf8mb4_bin NOT NULL COMMENT '登录用户名',
  `ADRESS` varchar(64) COLLATE utf8mb4_bin DEFAULT NULL COMMENT '登录 IP',
  `LOGIN_TIME` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '登录时间',
  PRIMARY KEY (`ID`),
  KEY `IDX_USER` (`USER`),
  KEY `IDX_HOST` (`ADRESS`),
  KEY `IDX_LOGIN_TIME` (`LOGIN_TIME`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_bin COMMENT='普通用户审计日志表';
```

2、对数据库中所有用户都授予审计日志表的权限

审计原理其实就是用户在登录时刻执行 `init_connect` 所指定的内容（对审计日志表进行 `insert` 操作），那么必须要求所有的用户都要有此表的 `insert` 权限才行。

##为了文章的继续发展，注意这里并没有给普通用户 'user_3'@'%' 授权哦

```
mysql> select user,host from mysql.user;
```

```
+-----+-----+
| user | host |
+-----+-----+
| user_1 | % |
| user_3 | % |
| user_2 | 10.186.61.17 |
| mysql.session | localhost |
| mysql.sys | localhost |
| root | localhost |
+-----+-----+
6 rows in set (0.00 sec)
```

```
mysql> grant insert on test_db.audit to 'user_1'@'%';
```

```
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> grant insert on test_db.audit to 'user_2'@'10.186.61.17';
```

```
Query OK, 0 rows affected (0.01 sec)
```

3、动态设置，开启审计日志表

##动态设置 `init_connect` 参数

```
mysql> set global init_connect="insert into test_db.audit(USER,ADRESS)
values(current_user(),substring_index(user(), '@', -1));";
```

```
Query OK, 0 rows affected (0.00 sec)
```

##补充说明 `current_user()` 可以查到当前登录的用户信息, `substring_index()` 是一个截取函数, 例如使用普通用户 'user_2'@'10.186.61.17'

```
mysql> select current_user();
+-----+
| current_user() |
+-----+
| user_2@10.186.61.17 |
+-----+
1 row in set (0.00 sec)

mysql> select substring_index(user(),'@',-1);
+-----+
| substring_index(user(),'@',-1) |
+-----+
| 10.186.61.17 |
+-----+
1 row in set (0.00 sec)
```

为了永久生效, 必须还要在配置文件中添加如下内容:

```
[mysqld]
init_connect="insert into test_db.audit(USER,ADRESS)
values(current_user(),substring_index(user(),'@',-1));";
```

4、审计日志表结果说明

- (1) 对于普通用户, 需要提前被授予一定权限, 否则不会被记录到审计日志表中, 并且会导致连接失败 ('user_3'@'%'用户因为没有审计表的 insert 权限, 所以没有被记录)。
- (2) 对于普通用户, 如果 `init_connect` 的内容有语法错误, 依然会直接导致连接失败 (无法执行 `init_connect` 的内容)。
- (3) 对于具有 super 权限的用户, 在登录时并不会执行 `init_connect` 的内容 (所以审计日志表里也没有 super 用户的记录)。
- (4) 对于密码过期的普通用户, 登录数据库会直接连接失败, 且不会记录也不会报错 (无法执行 `init_connect` 的内容)。

##模拟对审计表有 insert 权限的普通用户登录数据库

```
[root@10-186-61-17 ~]# /data/mysql/3306/base/bin/mysql -uuser_2 -p -P6666 -h10.186.61.17
[root@10-186-61-17 ~]# /data/mysql/3306/base/bin/mysql -uuser_1 -p -P6666 -h10.186.61.17
[root@10-186-61-55 ~]# /data/mysql/3306/base/bin/mysql -uuser_1 -p -P6666 -h10.186.61.17
```

##模拟对审计表无 insert 权限的普通用户登录数据库

```
[root@10-186-61-17 ~]# /data/mysql/3306/base/bin/mysql -uuser_3 -p -P6666 -h10.186.61.17
```



```

Enter password:
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 5.7.25-log

Copyright (c) 2000, 2019, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
ERROR 2013 (HY000): Lost connection to MySQL server during query

##使用 root 用户登录查看审计日志表记录结果展示
[root@10-186-61-17 ~]# /data/mysql/3306/base/bin/mysql -uroot -p -S
/data/mysql/6666/data/mysql.sock
Enter password:
.....
mysql> select * from test_db.audit;
+----+-----+-----+-----+-----+
| ID | USER | ADRESS | LOGIN_TIME |
+----+-----+-----+-----+
| 1 | user_2@10.186.61.17 | 10.186.61.17 | 2022-05-26 14:17:35 |
| 2 | user_1@% | 10.186.61.17 | 2022-05-26 14:18:12 |
| 3 | user_1@% | 10.186.61.55 | 2022-05-26 14:19:34 |
+----+-----+-----+-----+
3 rows in set (0.02 sec)

```

小结:

优点

- 实现原理和操作步骤简单;
- 几乎不影响 MySQL 性能。

缺点

- 记录的内容简略（仅记录登录时刻执行 `init_connect` 所定义插入的值）;
- 只记录登入行为，不记录登录数据库后的行为;
- 不会记录有 `super` 权限的用户的登录行为;
- 密码过期的用户登录数据库，会直接连接失败，且不会记录也不会报错。

MariaDB 审计插件

相关说明

MariaDB 开发的 MariaDB 审计插件，虽然具有一些仅适用于 MariaDB 的独特功能，但它也可以与 MySQL 一起使用。MariaDB 审计插件所生成的审计日志记录内容包括：用户名和主机、执行了哪些查询、访问了哪些表以及更改了服务器变量等详情信息，下面一起来看看吧。

实现步骤

1、获取 MariaDB 的审计插件—— server_audit.so

```
##下载 MariaDB Server 安装包
##下载地址：
https://mariadb.org/download/?t=mariadb&p=mariadb&r=10.2.43&os=Linux&cpu=x86_64&i=systemd&m=bkns
##本地解压后，在解压目录下找到审计插件`server_audit.so`
./mariadb/mariadb-10.2.43-linux-systemd-x86_64/lib/plugin/server_audit.so
```

2、将审计插件 server_audit.so 复制到 MySQL 的 plugin 目录下，并授权

```
cp ./mariadb/mariadb-10.2.43-linux-systemd-x86_64/lib/plugin/server_audit.so
/data/mysql/3306/base/lib/plugin/
chown -R mysql:mysql server_audit.so
chmod 755 server_audit.so
```

3、进入 MySQL 客户端，安装插件

```
##install审计插件
mysql> install plugin server_audit soname 'server_audit.so';
Query OK, 0 rows affected (0.07 sec)

##查看当前的插件信息
mysql> show plugins;
+-----+-----+-----+-----+-----+
| Name          | Status | Type          | Library          | License |
+-----+-----+-----+-----+-----+
| ngram          | ACTIVE | FTPARSER      | NULL             | GPL     |
| rpl_semi_sync_master | ACTIVE | REPLICATION  | semisync_master.so | GPL     |
| rpl_semi_sync_slave   | ACTIVE | REPLICATION  | semisync_slave.so | GPL     |
| SERVER_AUDIT    | ACTIVE | AUDIT         | server_audit.so  | GPL     |
+-----+-----+-----+-----+-----+
47 rows in set (0.00 sec)
```

4、配置说明

当安装完审计插件之后，MySQL 中将有一些新的全局变量出现。这些变量就是用于配置审计日志的，例如配置审计日志的位置、日志大小、记录的信息格式等。查看相关变量：

```
mysql> show variables like '%server_audit%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| server_audit_events | |
| server_audit_excl_users | |
| server_audit_file_path | server_audit.log |
```

```

| server_audit_file_rotate_now | OFF |
| server_audit_file_rotate_size | 1000000 |
| server_audit_file_rotations | 9 |
| server_audit_incl_users | |
| server_audit_loc_info | |
| server_audit_logging | OFF |
| server_audit_mode | 1 |
| server_audit_output_type | file |
| server_audit_query_log_limit | 1024 |
| server_audit_syslog_facility | LOG_USER |
| server_audit_syslog_ident | mysql-server_auditing |
| server_audit_syslog_info | |
| server_audit_syslog_priority | LOG_INFO |
+-----+-----+
16 rows in set (0.01 sec)

```

部分变量说明：

变量名	获取源包名
server_audit_logging	控制审计功能的开关；
server_audit_events	指定记录的事件类型； 如果设置指定值，则审核日志只记录在设定值范围内的事件类型； 如果未设置，则每个事件类型都会记录到审核日志中（默认值选项）。
server_audit_file_path	当 server_audit_output_type=file 时，设置日志文件的路径和文件名； 默认文件路径为 datadir，默认文件名为 server_audit.log。
server_audit_file_rotate_now	手工触发审计日志轮换； 即：设置为 ON 或 1 来强制日志文件轮换。
server_audit_file_rotate_size	日志大小限制，达到了阈值，审计日志会自动轮换。
server_audit_file_rotations	指定轮换后所保留的日志文件个数； 如果设置为 0，则日志永远不会轮换； 默认值为 9，即保存最多 9 个审计日志。
server_audit_incl_users	指定用户列表，在内的用户，将被记录； 注：CONNECT 记录不受此变量的影响。
server_audit_excl_users	指定用户列表，在内的用户，不会被记录； 如果用户在 server_audit_incl_users 中也指定了，用户活动仍会被记录； 注：CONNECT 记录不受此变量的影响。
server_audit_query_log_limit	限制记录中查询字符串的长度。

2.17 MySQL 审计功能实现方案

配置示例：

```
##开启审计功能
mysql> SET GLOBAL server_audit_logging=ON;

##指定记录内容 'connect,query'，记录用户的连接和查询语句
SET global server_audit_events='connect,query';
##重命名审计日志
mysql> set global server_audit_file_path='mysql_3306_audit.log';
```

为了永久生效，必须还要在配置文件中添加如下内容：

```
##这里仅定义了两项变量值供参考
[server]
server_audit_logging=ON
server_audit_events=connect,query
...
```

5、审计日志结果说明

```
##模拟相关操作（查询结果篇幅过长，省略部分内容）
mysql> show databases;
mysql> use test;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
mysql> select count(*) from sbtest1;
mysql> select * from sbtest1 limit 3;

+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | k     | c     |                                     | pad |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1  | 488670 | 31451372586-15688153734-79729593694-965809299839-83724886275-86711833530-78881337422-36049698573-51724173961-87474696253 | 98986621624-36688827414-84092488557-09587706818-65088859162 |
| 2  | 487778 | 214727878079-78972788322-78018558993-717636568083-89278326047-32417012831-10768856803-14235120402-93898880412-18698312264 | 04776816683-45888822864-77922711547-29857964468-7651426318 |
| 3  | 498956 | 49376827441-24983985829-56844662388-79812577859-48518387141-60588419212-24399138485-42612257832-29494881732-71586824440 | 26843835807-96849339132-53943793991-69741192222-48634174617 |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
mysql> exit
Bye

[root@10-186-65-155 plugin]# /data/mysql/3306/base/bin/mysql -utest -p -S /data/mysql/3306/data/mysql.sock
Enter password:
mysql> show databases;
mysql> use occdb
mysql> show tables;
mysql> create table abc like occ_application_config;
Query OK, 0 rows affected (10.09 sec)
mysql> exit
Bye

##查看datadir下审计日志文件
[root@10-186-65-155 data]# cd /data/mysql/3306/data/ && ll -h | grep 'audit.log'
-rw-r----- 1 mysql mysql 711 Apr 19 15:30 mysql_3306_audit.log
-rw-r----- 1 mysql mysql 329 Apr 19 15:26 server_audit.log

##查看审计日志'mysql_3306_audit.log'记录的内容
20220419 15:26:04,10-186-65-155,root,localhost,6,6,QUERY,, 'set global server_audit_file_path=\'mysql_3306_audit.log\'',0
20220419 15:29:30,10-186-65-155,root,localhost,6,7,QUERY,, 'show databases',0
20220419 15:29:38,10-186-65-155,root,localhost,6,8,QUERY,, 'SELECT DATABASE()',0
20220419 15:29:38,10-186-65-155,root,localhost,6,10,QUERY,test, 'show databases',0
20220419 15:29:38,10-186-65-155,root,localhost,6,11,QUERY,test, 'show tables',0
20220419 15:29:43,10-186-65-155,root,localhost,6,17,QUERY,test, 'show tables',0
20220419 15:30:04,10-186-65-155,root,localhost,6,18,QUERY,test, 'select count(*) from sbtest1',0
20220419 15:30:25,10-186-65-155,root,localhost,6,19,QUERY,test, 'select * from sbtest1 limit 3',0
20220419 15:36:14,10-186-65-155,root,localhost,6,0,DISCONNECT,test,,0
20220419 15:36:24,10-186-65-155,test,localhost,7,0,CONNECT,,0
20220419 15:36:24,10-186-65-155,test,localhost,7,21,QUERY,, 'select @@version_comment limit 1',0
20220419 15:36:28,10-186-65-155,test,localhost,7,22,QUERY,, 'show databases',0
20220419 15:36:36,10-186-65-155,test,localhost,7,23,QUERY,, 'SELECT DATABASE()',0
20220419 15:36:36,10-186-65-155,test,localhost,7,25,QUERY,ocddb, 'show databases',0
20220419 15:36:36,10-186-65-155,test,localhost,7,26,QUERY,ocddb, 'show tables',0
20220419 15:36:39,10-186-65-155,test,localhost,7,28,QUERY,ocddb, 'show tables',0
20220419 15:36:55,10-186-65-155,test,localhost,7,29,QUERY,ocddb, 'create table abc like occ_application_config',0
20220419 15:37:11,10-186-65-155,test,localhost,7,0,DISCONNECT,ocddb,,0
```

补充：下发的错误语句，或者连接失败等语句，审计日志依然会记录在内。

```
##模拟错误语句和连接失败情景
mysql> show creates table abc;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right syntax to use near 'creates
table abc' at line 1
```

```
mysql> exit
Bye
[root@10-186-65-155 plugin]# /data/mysql/3306/base/bin/mysql -utest -p -S
/data/mysql/3306/data/mysql.sock
Enter password:
ERROR 1045 (28000): Access denied for user 'test'@'localhost' (using password: NO)

##查看审计日志记录的相关内容
.....
20220419 15:42:16,10-186-65-155,test,localhost,9,45,QUERY,occdb,'show creates table
abc',1064
20220419 15:42:34,10-186-65-155,test,localhost,9,0,DISCONNECT,occdb,,0
20220419 15:42:48,10-186-65-155,test,localhost,10,0,FAILED_CONNECT,,,1045
```

根据以上记录的内容可以总结审计日志的记录格式：

```
[timestamp],[serverhost],[username],[host],[connectionid],[queryid],[operation],[database],[object],[retcode]
```

小结：

优点

- 使用 xml 格式和内容明确且详细；
- 审计粒度细：例如可通过 `server_audit_events` 和 `server_audit_incl_users` 等参数指定需要审计的内容；
- 审计日志维护管理方便：自带审计日志切割轮换参数；
- 审计日志会记录下发的错误语句、连接失败信息等。

缺点

- 对 MySQL 性能有一定影响。

McAfee 审计插件

相关说明

所属 McAfee 公司 McAfee 审计插件，专门为 MySQL 提供审计功能。其设计重点是安全和审计要求。可以独立的作为审计插件，实现 MySQL 的审计功能，实现步骤与 MariaDB 审计插件类似。

实现步骤

1、获取审计插件——libaudit_plugin.so

```
##下载插件
##下载地址: https://github.com/mcafee/mysql-audit/releases
##本地解压后，在解压目录下找到审计插件 libaudit_plugin.so
./audit-plugin-mysql-5.7-1.1.11-985/lib/libaudit_plugin.so
```

2、将审计插件 server_audit.so 复制到 MySQL 的 plugin 目录下，并授权

```
cp ./audit-plugin-mysql-5.7-1.1.11-985/lib/libaudit_plugin.so
```

```
/data/mysql/6666/base/lib/plugin/  
chown -R mysql:mysql libaudit_plugin.so  
chmod 755 libaudit_plugin.so
```

3、进入 MySQL 客户端，安装插件

```
mysql> INSTALL PLUGIN AUDIT SONAME 'libaudit_plugin.so';  
Query OK, 0 rows affected (1.08 sec)  
  
mysql> show plugins;  
  
+-----+-----+-----+-----+-----+  
| Name          | Status | Type          | Library          | License |  
+-----+-----+-----+-----+-----+  
.....  
| rpl_semi_sync_master | ACTIVE | REPLICATION  | semisync_master.so | GPL     |  
| rpl_semi_sync_slave  | ACTIVE | REPLICATION  | semisync_slave.so  | GPL     |  
| AUDIT              | ACTIVE | AUDIT         | libaudit_plugin.so | GPL     |  
+-----+-----+-----+-----+-----+  
47 rows in set (0.00 sec)
```

4、配置说明

同 Mariadb 的审计插件类似，当安装完审计插件之后，MySQL 中将有一些新的全局变量，用于配置审计日志相关属性。

```
mysql> show variables like '%audit%';  
  
+-----+-----+  
| Variable_name | Value  
+-----+-----+  
| audit_before_after | after  
| audit_checksum      |  
| audit_client_capabilities | OFF  
| audit_delay_cmds    |  
| audit_delay_ms      | 0  
| audit_force_record_logins | OFF  
| audit_header_msg     | ON  
| audit_json_file      | OFF  
| audit_json_file_bufsize | 1  
| audit_json_file_flush | OFF  
| audit_json_file_retry | 60  
| audit_json_file_sync  | 0  
| audit_json_log_file   | mysql-audit.json  
| audit_json_socket     | OFF  
| audit_json_socket_name | /var/run/db-audit/mysql.audit__data_mysql_6666_data_6666  
| audit_json_socket_retry | 10  
| audit_json_socket_write_timeout | 1000  
| audit_offsets         |  
| audit_offsets_by_version | ON  
| audit_password_masking_cmds | CREATE_USER,GRANT,SET_OPTION,SLAVE_START,CREATE_SERVER,ALTER_SERVER,CHANGE_MASTER,UPDATE  
| audit_password_masking_regex |  
| audit_record_cmds     |  
| audit_record_objs     |  
| audit_sess_connect_attrs | ON  
| audit_socket_creds     | ON  
| audit_uninstall_plugin | OFF  
| audit_validate_checksum | ON  
| audit_validate_offsets_extended | ON  
| audit_whitelist_cmds   | BEGIN,COMMIT,PING  
| audit_whitelist_users  |  
+-----+-----+  
30 rows in set (0.01 sec)
```

部分变量说明：

变量名	获取源包名
audit_json_file	是否开启审计 ON/OFF; 默认值：OFF。

变量名	获取源包名
audit_json_log_file	设置日志文件的路径和文件名； 默认值：mysql-audit.json，默认路径：datadir。
audit_record_cmds	设置要记录的语句类型，例如：insert,update,delete ； 默认记录所有的命令。
audit_record_objs	设置要记录到审计跟踪的对象（表），例如：database.table，可以使用通配符。
audit_whitelist_cmds	设置不需要记录的语句类型。
audit_whitelist_users	设置不需要记录的用户。
audit_before_after	控制是在执行当前 SQL 语句之前还是之后写入其日志记录，或两者兼而有之。 可能的值为'before'、'after'或'both'，默认为'after'。
audit_json_file_flush	手工进行轮换日志。
audit_json_file_sync	控制日志刷写周期 json，如果此变量的值为 N（N≠0），则审计日志将在每 N 次写入后同步到磁盘。默认值：0。

配置示例：

```
mysql> set global audit_json_file=on;
Query OK, 0 rows affected (0.01 sec)

mysql> set global audit_json_log_file='mysql_6666_audit.log';
Query OK, 0 rows affected (0.00 sec)
```

为了永久生效，必须还要在配置文件中添加如下内容：

```
##这里仅定义了两项变量值供参考
[mysqld]
audit_json_file=on
audit_record_cmds='query_type' ##自定义需要记录的语句类型
.....
```

5、审计日志结果说明

2.17 MySQL 审计功能实现方案

```
##模拟相关操作（查询结果篇幅过长，省略部分内容）
[root@10-186-65-155 plugin]# /data/mysql/6666/base/bin/mysql -uroot -p -S /data/mysql/6666/data/mysql.sock
Enter password:
Type 'help;' or 'h' for help. Type 'c' to clear the current input statement.

mysql> show databases;
mysql> select count(*) from mysql.user;
mysql> create database test;
mysql> use test;
Database changed
mysql> create table aaa(id int,name varchar(20),primary key(id));
mysql> insert into aaa values (1,'zhangsan'),(2,'lisi');
mysql> exit
Bye
[root@10-186-65-155 plugin]# /data/mysql/6666/base/bin/mysql -utest -p -S /data/mysql/6666/data/mysql.sock
Enter password:
mysql> use test;
mysql> select * from aaa;
+----+-----+
| id | name |
+----+-----+
| 1  | zhangsan |
| 2  | lisi   |
+----+-----+

##查看datadir下的审计日志记录内容（如下结果内容太大，不方便展示，仅显示两条作为参考略）
{
  "msg-type": "activity",
  "date": "1650361443139",
  "thread-id": "12",
  "query-id": "160",
  "user": "test",
  "priv_user": "test",
  "ip": "",
  "host": "localhost",
  "_os": "linux-glibc2.12",
  "_client_name": "libmysql",
  "_pid": "5162",
  "_client_version": "5.7.25",
  "_platform": "x86_64",
  "program_name": "mysql",
  "rows": "1",
  "status": "0",
  "cmd": "show tables",
  "objects": [{"db": "information_schema", "name": "/data/mysql/6666/tmp/#sql_1b0f_0", "obj_type": "TABLE"}],
  "query": "show tables"
},
{
  "msg-type": "activity",
  "date": "1650361454500",
  "thread-id": "12",
  "query-id": "162",
  "user": "test",
  "priv_user": "test",
  "ip": "",
  "host": "localhost",
  "_os": "linux-glibc2.12",
  "_client_name": "libmysql",
  "_pid": "5162",
  "_client_version": "5.7.25",
  "_platform": "x86_64",
  "program_name": "mysql",
  "pid": "5162",
  "os_user": "root",
  "appname": "/data/mysql/6666/base/bin/mysql",
  "rows": "2",
  "status": "0",
  "cmd": "select",
  "objects": [{"db": "test", "name": "aaa", "obj_type": "TABLE"}],
  "query": "select * from aaa"
}
```

根据以上的记录内容可总结审计日志的记录格式：

审计日志使用的是 json 格式记录，如下为一条记录内容

```
{
  "msg-type": "activity",
  "date": "1650361454500",
  "thread-id": "12",
  "query-id": "162",
  "user": "test",
  "priv_user": "test",
  "ip": "",
  "host": "localhost",
  "_os": "linux-glibc2.12",
  "_client_name": "libmysql",
  "_pid": "5162",
  "_client_version": "5.7.25",
  "_platform": "x86_64",
  "program_name": "mysql",
  "pid": "5162",
  "os_user": "root",
  "appname": "/data/mysql/6666/base/bin/mysql",
  "rows": "2",
  "status": "0",
  "cmd": "select",
  "objects": [{
    "db": "test",
    "name": "aaa",
    "obj_type": "TABLE"
  }],
  "query": "select * from aaa"
}
```


小结:

优点

- 使用 json 格式和内容明确且详细;
- 审计粒度细: 例如可通过 `audit_record_cmds` 和 `audit_record_objs` 等指定需要审计的内容;
- 记录连接失败的语句 (`audit_force_record_logins` 参数值为 on 条件下);
- 审计日志刷写策略可控, 可通过 `audit_json_file_sync` 参数控制。

缺点

- 对 MySQL 性能有一定影响;
- 不记录下发错误的语句;
- 审计日志维护管理不便, 无自带审计日志切割轮换参数。

补充说明:

Percona 公司开发的审计插件 `audit_log.so` (可以在 Percona Server 包中获取), 目前无法直接 install 到社区版 MySQL 中使用。

```
mysql> INSTALL PLUGIN audit_log SONAME 'audit_log.so';
ERROR 1126 (HY000): Can't open shared library
'/data/mysql/7777/base/lib/plugin/audit_log.so' (errno: 2
/data/mysql/7777/base/lib/plugin/audit_log.so: undefined symbol:
plugin_thdvar_safe_update)
```

结束语

本文介绍了三种实现审计功能的方案, 各个方案都有自己的优缺点。如果仅仅记录普通用户的登录信息就能够满足审计需求, 那就选择第一种方案 (`init_connect` 参数) 再好不过了。如果想要实现更全面的审计功能, 那么第二种或者第三种方案会更加合适。总之, 根据审计需求来选择自己认为最合适的方案吧。另外, 在实际使用之前, 请务必做好充分的测试工作!

2.18 dbslower 工具学习之探针使用

作者：王悦

最近使用了 bcc 工具集中的 dbslower，这个工具可以探测 MySQL 指定阈值下的慢 query，使用非常方便。

使用举例：

```
root@R740-24:/data/wangyue/binary_tools_man-bundle/bin# dbslower-bpfcc mysql -x /data/wangyue/mysql/mysqllab/mysqld
Tracing database queries for application /data/wangyue/mysql/mysqllab/mysqld slower than 1 ms...
TIME(s)      PID      MS QUERY
5.915514     399865   12.937 select @@version_comment limit 1
15.454449    399865   1000.543 select sleep(1)
```

因为这个工具是一个 python 脚本，于是打开学习一下内部的机制。

首先开始的一段是一些使用案例：

```
examples = """examples:
  dbslower postgres           # trace PostgreSQL queries slower than 1ms
  dbslower postgres -p 188 322 # trace specific PostgreSQL processes
  dbslower mysql -p 480 -m 30  # trace MySQL queries slower than 30ms
  dbslower mysql -p 480 -v     # trace MySQL queries & print the BPF program
  dbslower mysql -x $(which mysqld) # trace MySQL queries with uprobes
"""
```

这里我们关注这两个：

```
dbslower mysql -p 480 -m 30 # trace MySQL queries slower than 30ms
```

```
dbslower mysql -x $(which mysqld) # trace MySQL queries with uprobes
```

同样是检测慢日志，一种是提供 pid，另一种是提供二进制文件地址，粗看之下只是使用方式的不同，其实其中完全走向了不同的机制。

我们接着看下一段

```
mode = "USDT"
if args.path and not args.pids:
    if args.db == "mysql":
        regex = "\\w+dispatch_command\\w+"
        symbols = BPF.get_user_functions_and_addresses(args.path, regex)

        if len(symbols) == 0:
            print("Can't find function 'dispatch_command' in %s" % (args.path))
            exit(1)

        (mysql_func_name, addr) = symbols[0]

        if mysql_func_name.find("COM_DATA") >= 0:
            mode = "MYSQL57"
        else:
            mode = "MYSQL56"
    else:
        # Placeholder for PostgreSQL
        # Look on functions initStringInfo, pgstat_report_activity, EndCommand,
        # NullCommand
        print("Sorry at the moment PostgreSQL supports only USDT")
        exit(1)
```

这一段是用来确认最终使用的机制 `mode`，如果提供 `pid`，那么 `mode=“MYSQL57”`，如果提供的是二进制地址，那么 `mode=“USDT”`。

这里 USDT 指的是(Userland Statically Defined Tracing)用户态静态探针，也就是说，当使用 `pid` 时，脚本实际使用了 USDT 静态探针机制，而使用二进制文件时，则使用了另一种动态探针机制。要继续理解下面的段落，我们先来看一下静态探针和动态探针到底是指什么。

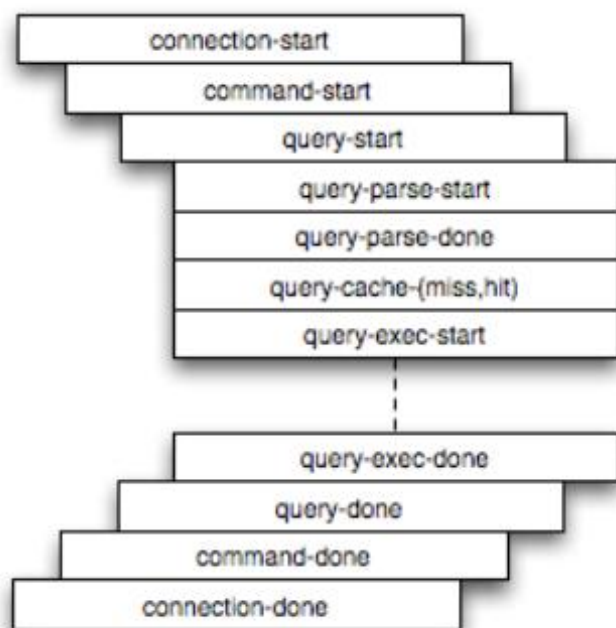
探针的分类

探针大致可以分为两类，静态探针和动态探针。

静态探针就比如我们上文看到的 USDT，对应 MySQL 中，则是 MySQL DTrace 的实现。这类探针需要事先在代码中定义并在编译时开启。

需要说明的是 MySQL DTrace 在 MySQL5.7.18 被弃用，在 MySQL8.0 被彻底删除，所以我们只能在早期的版本中进行 DTrace 检测。MySQL 的 DTrace 包含了很多的探针，比如 `query-start` 和 `query-done` 用于检测一条语句的执行过程，比如 `connection-start` 和 `connection-done` 用于检测客户端连接的过程，这些探针都是需要在代码中实现的。且启用探针需要增加编译参数 `-DENABLE_DTRACE=1`。

Figure 5.1 DTrace Probe Sequence



我们来看一个官方示例直观的了解一下：

<https://dev.mysql.com/doc/refman/5.7/en/dba-DTrace-server.html>

这里使用了 `query-start` 和 `query-done` 这两个探针，来获取语句执行时间

编辑 DTrace 脚本

```
#!/usr/sbin/dtrace -s

#pragma D option quiet

dtrace::BEGIN
{
    printf("%-20s %-20s %-40s %-9s\n", "Who", "Database", "Query", "Time(ms)");
}

mysql*: :query-start
{
    self->query = copyinstr(arg0);
    self->connid = arg1;
    self->db = copyinstr(arg2);
    self->who = strjoin(copyinstr(arg3), strjoin("@", copyinstr(arg4)));
    self->querystart = timestamp;
}

mysql*: :query-done
{
    printf("%-20s %-20s %-40s %-9d\n", self->who, self->db, self->query,
        (timestamp - self->querystart) / 1000000);
}
```

执行效果

```
$> ./query.d
```

Who	Database	Query	Time(ms)
root@localhost	test	select * from t1 order by i limit 10	0
root@localhost	test	set global query_cache_size=0	0
root@localhost	test	select * from t1 order by i limit 10	776
root@localhost	test	select * from t1 order by i limit 10	773
root@localhost	test	select * from t1 order by i desc limit 10	795

可惜静态探针在新版的 MySQL 上已经没有了，我们就不去深究了。

除了静态探针，还有一类是动态探针，就比如 bpf 技术中经常使用到的 uprobes 和 kprobes，这类探针比较灵活，可以在程序运行时动态添加，如果熟悉代码，可谓是神器。

我使用 MySQL8.0 作为实验环境，简单写了一个利用 uprobes 打印 query 的示例。

```
root@R740-24:/data/wangyue/binary_tools_man-bundle/bin#./bpftrace-
e'uprobe: "/data/wangyue/mysql/mysqlslab/mysqld":dispatch_command { printf("%s\n",
str(*arg1)); }'
Attaching 1 probe...
select * from a
select * from b
```

这里的关键在于 `{ printf("%s\n", str(*arg1)); }`，意思是打印出 `dispatch_command` 方法的第二个参数指向的内容。

同样的原理，变更一下函数，能动态打印任何我们想要的变量内容，或者利用探针写一些 bpf 脚本，能够实现许多功能，十分好用。

好了，回到我们的 dbslower 脚本上来。实际上这个脚本同时实现了静态探针和动态探针两种机制。

```
if mode.startswith("MYSQL"):
    # Uprobes mode
    bpf = BPF(text=program)
    bpf.attach_uprobe(name=args.path, sym=mysql_func_name,
                      fn_name="query_start")
    bpf.attach_uretprobe(name=args.path, sym=mysql_func_name,
                        fn_name="query_end")
else:
    # USDT mode
    if not args.pids or len(args.pids) == 0:
        if args.db == "mysql":
            args.pids = map(int, subprocess.check_output(
                "pidof mysqld".split()).split())
        elif args.db == "postgres":
            args.pids = map(int, subprocess.check_output(
                "pidof postgres".split()).split())

    usdts = map(lambda pid: USDT(pid=pid), args.pids)
    for usdt in usdts:
        usdt.enable_probe("query__start", "query_start")
        usdt.enable_probe("query__done", "query_end")
    if args.verbose:
        print('\n'.join(map(lambda u: u.get_text(), usdts)))

    bpf = BPF(text=program, usdt_contexts=usdts)
```

动态探针机制

静态探针机制

当我们使用 pid 时，比如 `dbslower MySQL -p 480 -m 30`，变量 mode 值为 “USDT”，走入了上图的 else 段落，即利用了 `query__start` 和 `query__done` 这两个静态探针，当探针被触发时，挂载到对应的处理函数 `query_start` 和 `query_end`，这两个处理函数中的逻辑很简单，就是获取对应的 query 内容，记录时间等，有兴趣的话可以去脚本中查看，这里就不罗列了。

另一种情况，当我们使用二进制文件地址时，比如 `dbslower MySQL -x $(which MySQLd)`，变量 mode 值为 “MYSQL57”，走入了上图的 if 段落，mysql_func_name 在脚本中被赋予的值是 `dispatch_command` 函数，即利用了 `bpf`，在函数 `dispatch_command` 执行前后插入了动态探针，同样的，当探针被触发时，挂载到对应的处理函数 `query_start` 和 `query_end` 进行时间的计算。

总结

1. dbslower 脚本利用探针检测 MySQL query 执行时间。
2. 当 dbslower 使用 pid 时，实际利用了静态探针机制，需要在编译时开启 `DENABLE_DTRACE=1`，且该机制在 MySQL 新版本中被删除了，需要特别注意。
3. 当 dbslower 使用 mysqld 二进制路径时，实际是利用了动态探针机制，动态探针可以在程序运行时动态添加，无论新旧版本的 MySQL 都可以使用。
4. bpf 的 uprobe 动态探针十分灵活，熟练使用之后，对一些故障排查场景是一大利器，缺点是多数情况下需要对照代码进行配置。

2.19 MySQL 编写脚本时避免烦人的警告

作者：杨涛涛

有客户在编写前期数据库安全规范时，就如何更安全的在 Linux Shell 端操作 MySQL 这一块，让我们帮忙出一份详尽的说明文档。其中有一项内容就是如何在 Linux Shell 下调用 MySQL 各种命令行工具时屏蔽掉烦人的告警信息输出，诸如下面这样：

```
root@ytt-ubuntu18:/home/ytt# mysql -uytt -proot -e "select version()"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| version() |
+-----+
| 8.0.29    |
+-----+
```

其实这是一个非常古老的问题！百度随便一搜，各种解决方法都有，但都写的不是很完善。

这样的告警信息对命令执行结果的输出非常不友好，那么我们如何屏蔽掉它？下面我来罗列下几种我能想到的方法，以供参考。

1、给用户空密码（不推荐）

给用户赋予空密码虽然可以屏蔽掉警告信息，但是极不安全，类似于 MySQL 服务初始化时的 `--initialize-insecure` 选项。

```
root@ytt-ubuntu18:/home/ytt# mysql -u ytt_no_pass -e "select user()"
+-----+
| user() |
+-----+
| ytt_no_pass@localhost |
+-----+
```

2、配置文件不同块加入用户名密码（不推荐）

MySQL 的配置文件有 `my.cnf`、`mysql.cnf`、`mysqld.cnf` 等等，只要在这些配置文件里的不同块下添加对应的用户名和密码即可。

```
root@ytt-ubuntu18:/home/ytt# cat /etc/mysql/conf.d/mysql.cnf
[mysql]
prompt=mysql:\d:\v>
user=ytt
password=root
port=3340
[mysqldump]
user=ytt
password=root
port=3340
146
```

```
[mysqladmin]
user=ytt
password=root
port=3340
```

以上 [mysql] 块下的内容表示对 mysql 命令行生效, [mysqldump] 块下的内容表示对 mysqldump 工具生效, [mysqladmin] 块下的内容表示对 mysqladmin 工具生效。

或者写简单点, 统一加到 [client] 里, 表示对所有客户端生效。注意只能把共享的部分内容加到这里。

```
root@ytt-ubuntu18:/home/ytt# cat /etc/mysql/conf.d/mysql.cnf
[mysql]
prompt=mysql:\d:\v>
[client]
user=ytt
password=root
port=3340
```

由于这些块都是针对客户端设置, 不需要重启 MySQL 服务, 可立即生效。

```
root@ytt-ubuntu18:/home/ytt# mysql -e "select user()"
+-----+
| user() |
+-----+
| ytt@localhost |
+-----+
```

3、设置 MySQL 环境变量（不推荐）

MySQL 有一些内置环境变量, 对所有客户端生效。官方的环境变量列表如下:

<https://dev.mysql.com/doc/refman/8.0/en/environment-variables.html>

给当前用户设置所需的环境变量, 之后再调用命令行工具即可。比如设置密码环境变量 MYSQL_PWD、传统 TCP 端口环境变量 MYSQL_TCP_PORT 等。

```
root@ytt-ubuntu18:/home/ytt# export MYSQL_PWD=root MYSQL_TCP_PORT=3340

root@ytt-ubuntu18:/home/ytt# mysql -uytt -e "select user()"
+-----+
| user() |
+-----+
| ytt@localhost |
+-----+
```

此方法也不推荐使用, 环境变量 MYSQL_PWD 容易被其他用户获取。比如直接用 ps 命令就可以轻易获取 MYSQL_PWD 的值。

用户 1 执行如下命令:

```
root@ytt-ubuntu18:/home/ytt# mysql -uytt -e "select sleep(1000)"
```

用户 2 执行 `ps aex` 就可以打印出环境变量 `MYSQL_PWD` 和 `MYSQL_TCP_PORT` 的值：

```
root@ytt-ubuntu18:/home/ytt# ps aex| grep MYSQL_PWD| grep -v 'grep'
7592 pts/0    S+      0:00 mysql -uytt -
e select sleep(1000) LS_COLORS=rs=0:... MYSQL_PWD=root ...MYSQL_TCP_PORT=3340 ...
```

4、屏蔽标准错误输出内容，重定向到空设备文件（推荐）

```
root@ytt-ubuntu18:/home/ytt# mysql -uytt -proot -P3340 -
e"select version()" 2>/dev/null
+-----+
| version() |
+-----+
| 8.0.29    |
+-----+
```

这里利用 Linux 系统本身的特性来重定向 MySQL 错误信息，其中数字 2 代表错误输出的文件描述符；`/dev/null` 代表空设备。也就是说把执行这条命令的错误信息重定向到空设备而不是标准输出，这样就可以变相的把警告信息屏蔽掉。

5、使用 `mysql_config_editor` 工具生成不同的 `login_path`（推荐）

`mysql_config_editor` 是 MySQL 官方发布的一款工具，专门处理这类必须暴露用户密码的问题，可以进行一次设置，多次安全使用。

使用方法如下：设置一个 `login_path`，名字为 `user_ytt`，密码按提示输入即可。

```
root@ytt-ubuntu18:/home/ytt# mysql_config_editor set -G user_ytt -
S /var/run/mysqld/mysqld.sock -u ytt -p
Enter password:
```

接下来，调用任何 MySQL 命令行工具只需要带上 `--login-path` 选项即可使用。

```
root@ytt-ubuntu18:/home/ytt# mysql --login-path=user_ytt -e 'select user()'
+-----+
| user()          |
+-----+
| ytt@localhost   |
+-----+

root@ytt-ubuntu18:/home/ytt# mysqladmin --login-path=user_ytt ping
mysqld is alive
```

`mysql_config_editor` 工具也有一个缺点：同样的 `login_path` 不能分享给所有系统用户，其他用户得重新添加自己的 `login_path` 才能正常使用。

6、使用 `Unix socket` 插件（推荐，仅限本地）

`auth_socket` 插件只根据本地 OS 登录用户名和本地 `linux socket` 文件来授权认证。比如修改用户

ytt@localhost 插件为 auth_socket :

```
mysql> alter user ytt@localhost identified with auth_socket ;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> \q
```

```
Bye
```

切换到 OS 用户 ytt :

```
root@ytt-pc-big:/home/ytt# su ytt
```

```
ytt@ytt-pc-big:~$ mysql -e "select user(),current_user()"
```

```
+-----+-----+
| user()      | current_user() |
+-----+-----+
| ytt@localhost | ytt@localhost  |
+-----+-----+
```

这里需要提醒一句：为了安全，操作 MySQL 的用户权限一定要做到按需分配。

2.20 MySQL 中一个聚类增量统计 SQL 的需求

作者：刘晨

同事提了一个 MySQL 数据库中 SQL 统计数据的问题，我用测试数据模拟一下，如下所示，表 tt 有三个字段，code 是标识名称，cdate 是对应的日期，ctotal 是个统计值

```
bisal@mysql> select * from tt;
```

code	cdate	ctotal
AAA	2022-01-01 00:00:00	1
AAA	2022-01-02 00:00:00	3
AAA	2022-01-03 00:00:00	6
AAA	2022-02-01 00:00:00	2
AAA	2022-02-02 00:00:00	5
AAA	2022-02-03 00:00:00	2
AAA	2022-03-01 00:00:00	3
AAA	2022-03-02 00:00:00	3
AAA	2022-03-03 00:00:00	5
BBB	2022-01-01 00:00:00	2
BBB	2022-01-02 00:00:00	3
BBB	2022-02-01 00:00:00	1
BBB	2022-02-01 00:00:00	2
BBB	2022-03-01 00:00:00	5
BBB	2022-03-02 00:00:00	5

```
15 rows in set (0.00 sec)
```

原始的统计语句如下所示，按照 code 和 cdate 进行聚类，统计出每个 code 每个月累加的记录数，

```
bisal@mysql> [test]> select code, date_format(cdate, '%Y-%m'),
-> sum(ctotal) from tt
-> group by code, date_format(cdate, '%Y-%m');
+-----+-----+-----+
| code | date_format(cdate, '%Y-%m') | sum(ctotal) |
+-----+-----+-----+
| AAA | 2022-01 | 10 |
| AAA | 2022-02 | 9 |
| AAA | 2022-03 | 11 |
| BBB | 2022-01 | 5 |
| BBB | 2022-02 | 3 |
| BBB | 2022-03 | 10 |
+-----+-----+-----+
6 rows in set (0.00 sec)
```

但是他的需求是能按照 code+cdate 的统计值进行累加显示，例如上图中第一行的 total 是 10，第二行的 total 就显示 10+9=19，第三行的 total 就显示 10+9+11=30，以此类推。

他需要的是 total 逐行累加，MySQL 中可以通过定义变量累加来实现，如下所示，利用 @i 逐行累加，

```
bisal@mysql> [test]> select a.code, a.cdate,
-> @i:=@i+a.total as total
-> from
-> (select code, date_format(cdate, '%Y-%m') as cdate,
-> sum(ctotal) as total from tt
-> group by code, date_format(cdate, '%Y-%m')) a,
-> (select @i:=0) b;
+-----+-----+-----+
| code | cdate | total |
+-----+-----+-----+
| AAA | 2022-01 | 10 |
| AAA | 2022-02 | 19 |
| AAA | 2022-03 | 30 |
| BBB | 2022-01 | 35 |
| BBB | 2022-02 | 38 |
| BBB | 2022-03 | 48 |
+-----+-----+-----+
6 rows in set, 2 warnings (0.00 sec)
```

确实看着实现了，但是有个问题，不同的 code，并未做归零处理，例如上图中，code=BBB 的第一行记录，total 应该是 5，当前的 35 则明显是通过 30+5 得到的。

此处应该再加个逻辑，即按照 code 聚类的时候做 sum 求和计算，如下所示，此时就可以看到，g_total 按照 code 和 cdate 进行累加，不同的 code，就会归零，重新计算，符合实际需求，

```
bisal@mysql> [test]> select t1.code, t1.cdate, t1.total,
-> (select sum(t2.total) from
-> (select code, date_format(cdate, '%Y-%m') as cdate,
-> sum(ctotal) as total from tt
-> group by code, date_format(cdate, '%Y-%m')) t2
-> where t2.code=t1.code and t2.cdate<=t1.cdate) as g_total
-> from (select code, date_format(cdate, '%Y-%m') as cdate,
-> sum(ctotal) as total from tt
-> group by code, date_format(cdate, '%Y-%m')) t1;
+-----+-----+-----+-----+
| code | cdate | total | g_total |
+-----+-----+-----+-----+
| AAA | 2022-01 | 10 | 10 |
| AAA | 2022-02 | 9 | 19 |
| AAA | 2022-03 | 11 | 30 |
| BBB | 2022-01 | 5 | 5 |
| BBB | 2022-02 | 3 | 8 |
| BBB | 2022-03 | 10 | 18 |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

其实这块还可以继续优化，MySQL 8.0 支持 `with`，如上 SQL 中对 `tt` 表读了两次，借助于 `with`，就可以降低为只读一次 `tt` 表，

```
bisal@mysqldb 15:54: [test]> with t as (
-> select code, date_format(cdate, '%Y-%m') as cdate,
-> sum(ctotal) as total from tt
-> group by code, date_format(cdate, '%Y-%m'))
-> select t1.code, t1.cdate, t1.total,
-> (select sum(t2.total) from t t2
-> where t2.code=t1.code and t2.cdate<=t1.cdate) as g_total
-> from t t1;
```

code	cdatetime	total	g_total
AAA	2022-01	10	10
AAA	2022-02	9	19
AAA	2022-03	11	30
BBB	2022-01	5	5
BBB	2022-02	3	8
BBB	2022-03	10	18

6 rows in set (0.00 sec)

另外，结合实际的检索场景需求，考虑为相关字段增加索引，进一步提升数据检索的效率。

当然，以上的 SQL 可能有其他的替代方案或者更好的方案，本文只是给出了其中一种解决的路径。

因此，通过 SQL 实现业务需求，一方面需要充分理解需求的含义，能准确地映射到具体的 SQL 逻辑上，另一方面则要了解所用数据库支持的函数、功能、特性等，是否有能契合到这个需求的实现，除此之外，非功能的因素，也是不可忽视的，合适的索引、避免重复读数据、避免不必要的排序等都是我们可以利用的手段。

2.21 MySQL:caching_sha2_password 快速问答

作者：胡呈清

一个报错

在使用客户端登录 MySQL8.0 时，我们经常会遇到下面这个报错：

```
ERROR 2061 (HY000): Authentication plugin 'caching_sha2_password' reported error: Authentication requires
secure connection.
```

网络上很多帖子教我们将用户认证插件修改成 `mysql_native_password` 来解决，那么事实上这是怎么一回事呢？本文就来探讨一二。

caching_sha2_password 简介

`caching_sha2_password` 是 MySQL 8.0.4 引入的一个新的身份验证插件，它的特点从其命名就可以窥探出

一二:

- **sha2_password**: 其实就是 **sha256_password**, 这是 MySQL5.6 就引入的身份验证插件, 其优点是对加盐密码进行多轮 SHA256 哈希, 以确保哈希转换更安全。其缺点为它要求使用安全连接或使用 RSA 密钥对进行密码交换的未加密连接, 因此其身份验证的效率较低。
- **caching**: 在 **sha256_password** 的基础上增加缓存, 有缓存的情况下不需要加密连接或 RSA 密钥对, 已达到安全和效率并存。

其实上面这个介绍不太容易懂, 下面我们以问答方式来揭开 **caching_sha2_password** 的面纱。

Q: 要求使用安全连接或使用 RSA 密钥对进行密码交换的未加密连接是什么意思?

caching_sha2_password 对密码安全性要求更高, 要求用户认证过程中在网络传输的密码是加密的:

- 如果是 SSL 加密连接, 则使用 SSL 证书和密钥对来完成 "对称加密密钥对 (在 TLS 握手中生成)" 的交换, 后续使用 "对称加密密钥对" 加密密码和数据。具体见: [MySQL:SSL 连接浅析](#);
- 如果是非 SSL 加密连接, 则在连接建立时客户端使用 MySQL Server 端的 RSA 公钥加密用户密码, Server 端使用 RSA 私钥解密验证密码的正确性, 可以防止密码在网络传输时被窥探。

tips: SSL 加密连接会不止会加密用户密码, 还会加密数据 (SQL 请求、返回的结果); 非加密连接只使用 RSA 密钥对进行用户密码的加密。

Q: 未加密连接是怎么使用 RSA 密钥对进行密码交换的?

当用户验证成功后, 会把用户密码哈希缓存起来。新连接客户端发起登录请求时, MySQL Server 端会判断是否命中缓存, 如果没有缓存, 对于未加密的连接, **caching_sha2_password** 插件要求连接建立时使用 RSA 进行加密密码交换, 否则报错, 其过程为:

- 客户端如果拥有服务端的 RSA 公钥, 则使用 `--server-public-key-path` 选项指定 RSA 公钥文件;
- 客户端使用 RSA 公钥对用户密码进行加密, 请求连接;
- 服务端使用 RSA 私钥进行解密, 验证密码的正确性。

如果客户端没有保存服务端的 RSA 公钥文件, 也可以使用 `--get-server-public-key` 选项从服务器请求公钥, 则在建立连接时, 服务端会先将 RSA 公钥发送给客户端。

如果 `--server-public-key-path`、`--get-server-public-key` 都没有指定, 则会报下面这个经典的错误:

```
[root@172-16-21-5 ~] mysql -h172.16.21.4 -utest -ptestpass --ssl-mode=disable
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 2061 (HY000): Authentication plugin 'caching_sha2_password' reported error: Authentication requires secure connection.
```

指定 `--get-server-public-key` 则能成功登录:

```
[root@172-16-21-5 ~] mysql -h172.16.21.4 -utest -ptestpass --ssl-mode=disable --get-server-public-key -e "select 1"
mysql: [Warning] Using a password on the command line interface can be insecure.
+---+
| 1 |
+---+
| 1 |
```

```
+---+
```

如果 test 用户登陆成功，有了缓存，则下次认证时未加密连接不再要求使用 RSA 密钥对：

```
[root@172-16-21-5 ~] mysql -h172.16.21.4 -utest -ptestpass --ssl-mode=disable -
e "select 1"
mysql: [Warning] Using a password on the command line interface can be insecure.
+---+
| 1 |
+---+
| 1 |
+---+
```

注意：上述客户端是指 mysql 默认命令行客户端，--server-public-key-path、--get-server-public-key 参数也只适用于 mysql 客户端

RSA 密钥对保存在哪里？

RSA 钥对默认保存 MySQL datadir 下，用于非 SSL 连接时的密码加密交换：使用 RSA 公钥加密密码，使用 RSA 私钥解密：

```
private_key.pem RSA 公钥
public_key.pem RSA 私钥
```

Q：密码哈希缓存何时失效？

当用户验证成功后，密码哈希会缓存起来，缓存会在以下情况被清理：

- 当用户的密码被更改时；
- 当使用 RENAME USER 重命名用户时；
- 执行 FLUSH PRIVILEGES 时；
- MySQL 重启。

Q：复制用户使用 caching_sha2_password 插件需要注意什么？

对于 MGR，如果设置 group_replication_ssl_mode=DISABLED，则也必须使用下面的变量来指定 RSA 公钥，否则报错：

- group_replication_recovery_get_public_key：向服务端请求 RSA 公钥；
- group_replication_recovery_public_key_path：指定本地 RSA 公钥文件。

设置一个就行，考虑拷贝 RSA 公钥到各节点麻烦，建议设置 group_replication_recovery_get_public_key=ON。

对于异步/半同步复制，需要在 change master 命令中指定：MASTER_PUBLIC_KEY_PATH = 'key_file_path' 或 GET_MASTER_PUBLIC_KEY = {0|1}

含义同上，建议：GET_MASTER_PUBLIC_KEY = 1

参考资料

https://dev.mysql.com/blog-archive/mysql-8-0-4-new-default-authentication-plugin-caching_sha2_password/
<https://dev.mysql.com/doc/refman/8.0/en/caching-sha2-pluggable-authentication.html>

2.22 MySQL:从库复制半个事务会怎么样?

作者: 胡呈清

复制异常

在复制过程中,主库发生故障或者网络中断,都会造成 `slave io thread` 中断,就有可能出现从库只复制了半个事务的情况。比如主库执行的事务如下:

```
begin;
insert 1;
insert 2;
commit;
```

从库接收的 `binlog` 可能只包含事务的一部分,比如:

- 情况 1: 只包含 `begin;`
- 情况 2: 只包含 `begin;insert 1;`
- 情况 3: 只包含 `begin;insert 1;insert 2;`

从库的 `slave sql thread` 回放完这部分 `binlog` 后,会等待 `slave io thread` 从主库读取剩余的 `binlog`,而在此之前 `sql` 线程回放这半个事务,就和我们手工执行半个事务一样,不会提交也不会回滚。

我们应该如何应对这种异常呢?

- 当 `slave io thread` 恢复,应该做什么?
- 当 `slave io thread` 无法恢复,应该做什么?

实验过程测试方法:

##1. 在从库上用 `tc` 模拟网络延迟,意在使读取 `binlog` 的速度变慢

```
tc qdisc add dev eth0 root netem delay 3000ms 3000ms 100%
```

##2. 在主库执行一个多语句事务

```
begin;
update t2 set pad='4' where id < 40;
update t2 set pad='5' where id < 50;
update t2 set pad='6' where id < 60;
commit;
```

##3. 在主库执行 `commit` 成功后,立刻用 `iptables` 切断主从之间的网络

```
iptables -A OUTPUT -d 172.16.21.4 -j DROP
iptables -A INPUT -s 172.16.21.4 -j DROP
```

这样我们可以在从库上观察到的现象为:

- 其中一个 `worker` 线程状态是 `Waiting for an event from Coordinator`,这个状态说明 `work` 线程已经干完活在等 `Coordinator` (协调线程) 分配新的 `relay log event`,但同时又显示它正在执行 `update t2 set pad='5' where id < 50`,这是矛盾 1:

35735	system user		NULL	Connect	6	Slave has read all relay log; waiting for more updates	NULL
35736	system user		NULL	Connect	15	Waiting for an event from Coordinator	update t2 set pad='5' where id < 50
35737	system user		NULL	Connect	156900	Waiting for an event from Coordinator	NULL
35738	system user		NULL	Connect	156900	Waiting for an event from Coordinator	NULL
35739	system user		NULL	Connect	156900	Waiting for an event from Coordinator	NULL
35740	system user		NULL	Connect	156900	Waiting for an event from Coordinator	NULL
35741	system user		NULL	Connect	156900	Waiting for an event from Coordinator	NULL
35742	system user		NULL	Connect	156900	Waiting for an event from Coordinator	NULL
35743	system user		NULL	Connect	156900	Waiting for an event from Coordinator	NULL
35744	system user		NULL	Connect	156900	Waiting for an event from Coordinator	NULL

- show slave status 输出中，Retrieved_Gtid_Set 与 Executed_Gtid_Set 相等（意味着 sql 线程已经回放完所有的 relay log），但是上图 worker 线程又正在回放 SQL，这是矛盾 2:

```
Retrieved_Gtid_Set: f93e2f1b-eb0e-11ec-b9c1-0242ac101503:468-478
Executed_Gtid_Set: f93e2f1b-eb0e-11ec-b9c1-0242ac101503:1-478
Auto_Position: 1
Replicate_Rewrite_DB:
Channel_Name:
Master_TLS_Version:
1 row in set (0.00 sec)
```

最后我们通过 relay log 实锤，可以看到这个事务的 relay log 并不完整，到 update t2 set pad='5' where id < 50; 这个 Rows_query event 就结束了：

```
### UPDATE `test`.`t2`
### WHERE
### @1=39 /* INT meta=0 nullable=0 is_null=0 */
### @2=50152 /* INT meta=0 nullable=0 is_null=0 */
### @3='71910848473-11761789603-92790583834-10164217197-77075823886-45586060395-14611060278-12118272326-27532212058-48023504224' /* STRING(480) meta=61152 nullable=0 is_null=0 */
### @4='00000' /* STRING(240) meta=65264 nullable=0 is_null=0 */
### SET
### @1=39 /* INT meta=0 nullable=0 is_null=0 */
### @2=50152 /* INT meta=0 nullable=0 is_null=0 */
### @3='71910848473-11761789603-92790583834-10164217197-77075823886-45586060395-14611060278-12118272326-27532212058-48023504224' /* STRING(480) meta=61152 nullable=0 is_null=0 */
### @4='4' /* STRING(240) meta=65264 nullable=0 is_null=0 */
# at 178844
#220617 9:55:44 server id 188343572 end_log_pos 14210345 CRC32 0x740aed53 Rows_query
# update t2 set pad='5' where id < 50
# at 178903
#220617 9:55:44 server id 188343572 end_log_pos 14210397 CRC32 0x708ba132 Table_map: `test`.`t2` mapped to number 221
ROLLBACK /* added by mysqlbinlog */ /*!*/;
SET @@SESSION.GTID_NEXT= 'AUTOMATIC' /* added by mysqlbinlog */ /*!*/;
DELIMITER ;
# End of log file
/*!50003 SET COMPLETION_TYPE=OLD_COMPLETION_TYPE*/;
/*!50530 SET @@SESSION.PSEUDO_SLAVE_MODE=0*/;
(END)
```

当 slave io thread 无法恢复

如果 slave io thread 长时间不能恢复，那么 sql 线程会因为等不到剩余的 binlog，一直无法提交或回滚，会一直持有这个事务的锁：

```
mysql> select * from INNODB_TRX\G
***** 1. row *****
      trx_id: 4721
      trx_state: RUNNING
      trx_started: 2022-06-17 09:55:53
      trx_requested_lock_id: NULL
      trx_wait_started: NULL
      trx_weight: 42
      trx_mysql_thread_id: 35736
      trx_query: update t2 set pad='5' where id < 50
      trx_operation_state: NULL
      trx_tables_in_use: 0
      trx_tables_locked: 1
      trx_lock_structs: 3
      trx_lock_memory_bytes: 1136
      trx_rows_locked: 39
      trx_rows_modified: 39
      trx_concurrency_tickets: 0
      trx_isolation_level: READ COMMITTED
      trx_unique_checks: 1
      trx_foreign_key_checks: 1
      trx_last_foreign_key_error: NULL
      trx_adaptive_hash_latched: 0
      trx_adaptive_hash_timeout: 0
      trx_is_read_only: 0
      trx_autocommit_non_locking: 0
1 row in set (0.00 sec)
```

如果是主库故障导致的 slave io thread 异常，那很可能会进行主从切换，这个从库提升为主后，SQL 线程持有的事务锁可能会阻塞业务请求。

此时应该 stop slave 停止 sql 线程，让事务回滚释放锁。需要注意的是：此情况下 stop slave 会等待 60 秒（等 slave io thread 接收事务剩余的 binlog），60 秒超时会才会停止 sql 线程：

```

42048 | root | localhost | information_schema | Query | 0 | starting | show processlist
42050 | root | localhost | test | Query | 60 | Killing slave | stop slave
42066 | system user | NULL | Connect | 67 | Slave has read all relay log; waiting for more updates | NULL
42067 | system user | NULL | Connect | 9237 | Waiting for an event from Coordinator | update t2 set pad='5' where id < 50
42068 | system user | NULL | Connect | 67 | Waiting for an event from Coordinator | NULL
42069 | system user | NULL | Connect | 67 | Waiting for an event from Coordinator | NULL
42070 | system user | NULL | Connect | 67 | Waiting for an event from Coordinator | NULL
42071 | system user | NULL | Connect | 67 | Waiting for an event from Coordinator | NULL
42072 | system user | NULL | Connect | 67 | Waiting for an event from Coordinator | NULL
42073 | system user | NULL | Connect | 67 | Waiting for an event from Coordinator | NULL
42074 | system user | NULL | Connect | 67 | Waiting for an event from Coordinator | NULL
42075 | system user | NULL | Connect | 67 | Waiting for an event from Coordinator | NULL
42076 | system user | NULL | Connect | 67 | Waiting for an event from Coordinator | NULL
42077 | system user | NULL | Connect | 67 | Waiting for an event from Coordinator | NULL
42078 | system user | NULL | Connect | 67 | Waiting for an event from Coordinator | NULL
42079 | system user | NULL | Connect | 67 | Waiting for an event from Coordinator | NULL
42080 | system user | NULL | Connect | 67 | Waiting for an event from Coordinator | NULL
42081 | system user | NULL | Connect | 67 | Waiting for an event from Coordinator | NULL
42082 | system user | NULL | Connect | 67 | Waiting for an event from Coordinator | NULL
-----
35 rows in set (0.00 sec)

mysql>
mysql> stop slave;
Query OK, 0 rows affected (1 min 2.02 sec)

mysql>

```

当 slave io thread 恢复

slave io thread 异常中断后，sql 线程是正常工作的，sql 线程执行了部分事务，并且会等待 io 线程发送新的 binlog。slave io thread 线程恢复后，如果是基于 GTID 的复制，会当前 GTID 事务开始重新获取完整的 binlog，从库会先回滚当前事务，然后再重新回放新收到的 binlog。

2.23 undo 太大了怎么办

作者：王雨晨

问题背景

有用户在使用 MySQL 5.7 的数据库时，遇到 undo 暴涨情况，经排查存在一条慢 SQL 执行了上万秒仍没有结束，导致后续事务产生的 undo 不能清理，越来越多。

在线 truncate undo log 已开启，将慢 SQL kill 掉之后，undo 大小超过 innodb_max_undo_log_size 设置的大小，但 undo 文件没有立即收缩。

测试验证

测试参数如下，开启 innodb_undo_log_truncate

```

mysql> show variables like '%undo%';
+-----+-----+
| Variable_name | Value |
+-----+-----+

```



```
| innodb_max_undo_log_size | 104857600 |
| innodb_undo_directory | ./ |
| innodb_undo_log_truncate | ON |
| innodb_undo_logs | 128 |
| innodb_undo_tablespaces | 3 |
+-----+-----+
5 rows in set (0.00 sec)
```

模拟 undo 增长，超过 innodb_max_undo_log_size 设置大小

```
# du -sh ./undo*
152M ./undo001
296M ./undo002
15M ./undo003
```

查看官方文档 undo 清理策略，简单概括为以下：

- 1、启用 innodb_undo_log_truncate 后，超过 innodb_max_undo_log_size 设置大小的 undo 表空间被标记为截断
- 2、被标记的 undo 表空间的回滚段被设置为不活跃的，不能分配给新的事务；
- 3、purge 线程释放不需要的回滚段
- 4、释放回滚段后，undo 表空间被截断为初始大小 10M

可以看到在收缩 undo 大小前，需要 purge 线程先释放回滚段，这里涉及另一个参数

innodb_purge_rseg_truncate_frequency，默认值 128，表示 purge 线程每调用 128 次，就释放回滚段一次

此次问题背景中，该参数设置的是默认值

```
mysql> show variables like 'innodb_purge_rseg_truncate_frequency';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_purge_rseg_truncate_frequency | 128 |
+-----+-----+
1 row in set (0.01 sec)
```

所以为了尽快收缩 undo 文件，我们可以将 innodb_purge_rseg_truncate_frequency 值调小，提高 purge 线程释放回滚段的频率

```
//调小该值
mysql> show variables like 'innodb_purge_rseg_truncate_frequency';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_purge_rseg_truncate_frequency | 16 |
+-----+-----+
1 row in set (0.01 sec)
```

```
//达到 purge 线程调用次数，释放回滚段，undo 表空间被截断
# du -sh ./undo*
10M ./undo001
10M ./undo002
15M ./undo003
```

MySQL8.0 新增 Manual Truncation

MySQL8.0 新增支持使用 SQL 语句来管理 undo 表空间

1、需要至少三个活跃的 undo 表空间，因为要保证有两个活跃的 undo 表空间来支持 Automated Truncation

手工创建一个 undo 表空间，必须以 .ibu 结尾

```
mysql>
create undo tablespace undo_003 add datafile '/data/mysql/data/3307/undo_003.ibu';
Query OK, 0 rows affected (0.27 sec)
//三个处于 active 状态的 undo 表空间
mysql> SELECT NAME, STATE FROM INFORMATION_SCHEMA.INNODB_TABLESPACES WHERE NAME LIKE
'%undo%';
+-----+-----+
| NAME | STATE |
+-----+-----+
| innodb_undo_001 | active |
| innodb_undo_002 | active |
| undo_003 | active |
+-----+-----+
3 rows in set (0.00 sec)
```

2、手工截断 undo 表空间，需要先将 undo 表空间设置为 inactive

```
//模拟 undo 增长
# du -sh ./undo*
81M ./undo_001
157M ./undo_002
26M ./undo_003.ibu

mysql> ALTER UNDO TABLESPACE innodb_undo_002 SET INACTIVE;
Query OK, 0 rows affected (0.01 sec)
```

3、手工设置 inactive 后，undo 表空间被标记为截断，purge 线程会增加返回频率，快速清空并最终截断 undo 表空间，状态变为 empty

```
mysql> SELECT NAME, STATE FROM INFORMATION_SCHEMA.INNODB_TABLESPACES WHERE NAME LIKE
'%undo%';
+-----+-----+
| NAME | STATE |
+-----+-----+
158
```

```
+-----+-----+
| innodb_undo_001 | active |
| innodb_undo_002 | empty  |
| undo_003        | active |
+-----+-----+
```

//undo 文件收缩

```
# du -sh ./undo*
```

```
81M ./undo_001
```

```
2.1M ./undo_002
```

```
26M ./undo_003.ibu
```

4、empty 状态的 undo 表空间可以重新激活使用

```
mysql> ALTER UNDO TABLESPACE innodb_undo_002 SET ACTIVE;
```

```
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT NAME, STATE FROM INFORMATION_SCHEMA.INNODB_TABLESPACES WHERE NAME LIKE
'%undo%';
```

```
+-----+-----+
| NAME | STATE |
+-----+-----+
| innodb_undo_001 | active |
| innodb_undo_002 | active |
| undo_003        | active |
+-----+-----+
```

```
3 rows in set (0.01 sec)
```

5、MySQL8.0 支持删除表空间，但前提是该表空间为 empty 状态

```
mysql> ALTER UNDO TABLESPACE undo_003 SET INACTIVE;
```

```
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> SELECT NAME, STATE FROM INFORMATION_SCHEMA.INNODB_TABLESPACES WHERE NAME LIKE
'%undo%';
```

```
+-----+-----+
| NAME | STATE |
+-----+-----+
| innodb_undo_001 | active |
| innodb_undo_002 | active |
| undo_003        | empty  |
+-----+-----+
```

```
3 rows in set (0.01 sec)
```

```
mysql> DROP UNDO TABLESPACE undo_003;
```

```
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> SELECT TABLESPACE_NAME, FILE_NAME FROM INFORMATION_SCHEMA.FILES WHERE FILE_TYPE
```

```
LIKE 'UNDO LOG';
+-----+-----+
| TABLESPACE_NAME | FILE_NAME |
+-----+-----+
| innodb_undo_001  | ./undo_001 |
| innodb_undo_002  | ./undo_002 |
+-----+-----+
2 rows in set (0.01 sec)
```

2.24 MySQL: max_allowed_packet 影响了什么？

作者：胡呈清

max_allowed_packet 表示 MySQL Server 或者客户端接收的 packet 的最大大小，packet 即数据包，MySQL Server 和客户端上都有这个限制。

数据包

每个数据包，都由包头、包体两部分组成，包头由 3 字节的包体长度、1 字节的包编号组成。3 字节最多能够表示 $2^{24} = 16777216$ 字节（16M），也就是说，一个数据包的包体长度必须小于等于 16M。

如果要发送超过 16M 的数据怎么办？

当要发送大于 16M 的数据时，会把数据拆分成多个 16M 的数据包，除最后一个数据包之外，其它数据包大小都是 16M。而 MySQL Server 收到这样的包后，如果发现包体长度等于 16M，它就知道本次接收的数据由多个数据包组成，会先把当前数据包的内容写入缓冲区，然后接着读取下一个数据包，并把下一个数据包的内容追加到缓冲区，直到读到结束数据包，就接收到客户端发送的完整数据了。

那怎样算一个数据包？

- 一个 SQL 是一个数据包
- 返回查询结果时，一行数据算一个数据包
- 解析的 binlog，如果用 mysql 客户端导入，一个 SQL 算一个数据包
- 在复制中，一个 event 算一个数据包

下面我们通过测试来讨论 max_allowed_packet 的实际影响。

导入 SQL 文件受 max_allowed_packet 限制吗？

如果 SQL 文件中有单个 SQL 大小超过 max_allowed_packet，会报错：

```
##导出时设置 mysqldump --net-buffer-length=16M，这样保证导出的 sql 文件中单个 multiple-row
INSERT 大小为 16M
mysqldump -h127.0.0.1 -P13306 -uroot -proot --net-buffer-length=16M \
160
```

```
--set-gtid-purged=off sbtest sbtest1 > /data/backup/sbtest1.sql

##设置 max_allowed_packet=1M

##导入报错
[root@localhost data]# mysql -h127.0.0.1 -P13306 -uroot -proot db3 <
/data/backup/sbtest1.sql
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 1153 (08S01) at line 41: Got a packet bigger than 'max_allowed_packet' bytes
```

导入解析后的 binlog 受 max_allowed_packet 限制吗?

row 格式的 binlog，单个 SQL 修改的数据产生的 binlog 如果超过 max_allowed_packet，也会报错。

在恢复数据到指定时间点的场景，解析后的 binlog 单个事务大小超过 1G，并且这个事务只包含一个 SQL，此时一定会触发 max_allowed_packet 的报错。但是恢复数据的任务又很重要，怎么办呢？可以将 binlog 改名成 relay log，用 sql 线程回放来绕过这个限制。

查询结果受 max_allowed_packet 限制吗?

查询结果中，只要单行数据不超过客户端设置的 max_allowed_packet 即可：

```
##插入 2 行 20M 大小的数据
[root@localhost tmp]# dd if=/dev/zero of=20m.img bs=1 count=0 seek=20M
记录了 0+0 的读入
记录了 0+0 的写出
0 字节(0 B)已复制, 0.000219914 秒, 0.0 kB/秒
[root@localhost tmp]# ll -h 20m.img
-rw-r--r-- 1 root root 20M 6 月 6 15:15 20m.img

mysql> create table t1(id int auto_increment primary key,a longblob);
Query OK, 0 rows affected (0.03 sec)

mysql> insert into t1 values(NULL,load_file('/tmp/20m.img'));
Query OK, 1 row affected (0.65 sec)

mysql> insert into t1 values(NULL,load_file('/tmp/20m.img'));
Query OK, 1 row affected (0.65 sec)

##mysql 客户端默认 --max-allowed-packet=16M, 读取失败
mysql> select * from t1;
ERROR 2020 (HY000): Got packet bigger than 'max_allowed_packet' bytes

##设置 mysql 客户端 --max-allowed-packet=22M, 读取成功
[root@localhost ~]# mysql -h127.0.0.1 -P13306 -uroot -proot --max-allowed-
packet=23068672 sbtest -e "select * from t1;" > /tmp/t1.txt
```

```
[root@localhost ~]# ll -h /tmp/t1.txt
-rw-r--r-- 1 root root 81M 6月 6 15:30 /tmp/t1.txt
```

load data 文件大小受 max_allowed_packet 限制吗?

load data 文件大小、单行大小都不受 max_allowed_packet 影响:

```
##将上一个测试中的数据导出, 2 行数据一共 81M
mysql> select * into outfile '/tmp/t1.csv' from t1;
Query OK, 2 rows affected (0.57 sec)

[root@localhost ~]# ll -h /tmp/t1.csv
-rw-r----- 1 mysql mysql 81M 6月 6 15:32 /tmp/t1.csv

##MySQL Server max_allowed_packet=16M
mysql> select @@max_allowed_packet;
+-----+
| @@max_allowed_packet |
+-----+
| 16777216 |
+-----+
1 row in set (0.00 sec)

##load data 成功, 不受 max_allowed_packet 限制
mysql> load data infile '/tmp/t1.csv' into table t1;
Query OK, 2 rows affected (1.10 sec)
Records: 2 Deleted: 0 Skipped: 0 Warnings: 0
```

binlog 中超过 1G 的 SQL , 是如何突破 max_allowed_packet 复制到从库的?

从库 slave io 线程、slave sql 线程可以处理的最大数据包大小由参数 slave_max_allowed_packet 控制。这是限制 binlog event 大小, 而不是单个 SQL 修改数据的大小。

主库 dump 线程会自动设置 max_allowed_packet 为 1G, 不会依赖全局变量 max_allowed_packet。用来控制主库 DUMP 线程每次读取 event 的最大大小。

具体可以参考: https://mp.weixin.qq.com/s/EfNY_UwEthiu-DEBO7TrsA

另外超过 4G 的大事务, 从库心跳会报错: <https://opensource.actionsky.com/20201218-mysql/>

2.25 大事务阻塞 show master status

作者：王福祥

MySQL 主从复制功能可以搭建从库来为 MySQL 创建一套在线的备份系统，但是自身不能独立实现切换；需要借助第三方高可用工具。然而当自身有大事务在运行时阻塞一些 show 语句；例如“show master status”，造成误判。

场景模拟

1、构造两千万行数据，以事务的形式删除

```
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> delete from sbtest1;
Query OK, 20000000 rows affected (38 min 9.07 sec)

mysql>
```

2、新建会话执行 show master status，在 sql 语句运行期间执行成功。在 commit 期间被阻塞。

```
mysql> show master status;
```

3、show master status 处于 starting 状态，也就是语句开始执行的第一个阶段，启动阶段，准备资源。

```
mysql> show processlist;
+-----+-----+-----+-----+-----+-----+-----+-----+
| Id | User | Host | db | Command | Time | State | Info |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 4 | event_scheduler | localhost | NULL | Daemon | 3632 | Waiting on empty queue | NULL |
| 83 | root | localhost | test | Query | 15 | waiting for handler commit | commit |
| 118 | root | localhost | test | Sleep | 26 | | NULL |
| 119 | root | localhost | NULL | Query | 7 | starting | show master status |
| 121 | root | localhost | NULL | Query | 0 | starting | show processlist |
+-----+-----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.01 sec)
```

4、查看 stack 信息，show master status 是在获取 lock_log 锁时被阻塞

```
#6 0x00000000ee8278 in MYSQL_BIN_LOG::get_current_log (this=0x1e839c0 <mysql_bin_log>, linfo=0x7f3ea82e62d0, need_lock_log=<optimized out>) at /export/home/pb2/build/sb_0-32013917-1545390211.74/mysql-5.7.25/sql/binlog.cc:5514
.....
#6 0x00000000eeef824 in MYSQL_BIN_LOG::change_stage (this=<optimized out>, thd=<optimized out>, stage=<optimized out>, queue=<optimized out>, leave_mutex=<optimized out>, enter_mutex=0x1e839c8 <mysql_bin_log+8>) at /export/home/pb2/build/sb_0-32013917-1545390211.74/mysql-5.7.25/sql/binlog.cc:9170
```

5、最终因事务过大，最终 show master status 超时，导致了故障切换。

其原因为 commit 与 show master status 之间的阻塞等待现象，接下来分析一下原因。

原因分析：commit

commit 流程分为以下几个阶段（以 5.7.25 为例），为原子性操作，一次性写入。

1、prepare 阶段主要作用为刷 redo 、undo ；此时 binlog 只涉及到一些准备动作。参照 binlog_prepare 函数。

```
static int binlog_prepare(handlerton *hton, THD *thd, bool all)
{
    DBUG_ENTER("binlog_prepare");
    if (!all)
    {
        thd->get_transaction()->store_commit_parent(mysql_bin_log.
            m_dependency_tracker.get_max_committed_timestamp());
    }
}
```

2、flush 阶段主要作用为持久化 redo ；获取 lock_log 锁阻塞其他组事务写入，以及生成 flush 队列（第一个事务为 leader ，随后为 follower ）以及写入 binlog ，此时是内存写但不刷盘，参照 process_flush_stage_queue 函数

```
int
MYSQL_BIN_LOG::process_flush_stage_queue(my_off_t *total_bytes_var,
                                         bool *rotate_var,
                                         THD **out_queue_var)
{
    DBUG_ENTER("MYSQL_BIN_LOG::process_flush_stage_queue");
    #ifndef DEBUG_OFF
    // number of flushes per group.
    int no_flushes= 0;
    #endif
    DBUG_ASSERT(total_bytes_var && rotate_var && out_queue_var);
    my_off_t total_bytes= 0;
    int flush_error= 1;
    mysql_mutex_assert_owner(&LOCK_log);
```

3、sync 阶段作用为释放 lock_log 锁，获取 lock_sync 锁阻塞其他事务组刷盘，并根据 sync_binlog 决定刷盘策略。

```
if (change_stage(thd, Stage_manager::SYNC_STAGE, wait_queue, &LOCK_log, &LOCK_sync))
{
    DBUG_PRINT("return", ("Thread ID: %u, commit_error: %d",
                          thd->thread_id(), thd->commit_error));
    DBUG_RETURN(finish_commit(thd));
}
```

4、commit 阶段释放 lock_sync 锁，获取 lock_commit 锁并等待 follower 事务提交，提交后释放该锁以

及清空 binlog 缓存以及判断是否刷新并归档 binlog

原因分析：show master status

show master status 其作用为从最后一份 binlog 文件中获取 Executed_Gtid 信息以及该 binlog 所执行的 gtid 信息。其函数入口为：https://github.com/mysql/mysql-server/blob/mysql-5.7.25/sql/rpl_master.cc 624 行并获取 lock_log 锁：

```
int MYSQL_BIN_LOG::get_current_log(LOG_INFO* linfo, bool need_lock_log/*true*/)
{
    if (need_lock_log)
        mysql_mutex_lock(&LOCK_log);
    int ret = raw_get_current_log(linfo);
    if (need_lock_log)
        mysql_mutex_unlock(&LOCK_log);
    return ret;
}
```

结论

案例中的例子是单个大事务，因此不涉及组提交的 sync 阶段，从 commit（flush 阶段）以及 show master status 的步骤中可以看出，两步操作之间的阻塞原因为 lock_log 锁的争用引起的，该锁的性质为 lock_log 日志锁，而并非 MDL 以及引擎层锁，如果事务越大，flush 阶段写入 binlog 期间以及持有锁的时间就会越长；从而阻塞其他事务提交以及 binlog 的 show 操作，因此对应的解决方案为：

- 1、大事务会造成 io 暴涨、主从延时以及事务阻塞等问题，在 MGR 环境中甚至会造成复制中断，mysql 需要避免大事务。
- 2、binlog 是持续在变化的文件，show master status 语句可改为 select @@global.gtid_executed；来获取 GTID 信息。

2.26 需要小心配置的 gtid_mode

作者：王雅蓉

背景信息：

本文章与一次日常运维过程中发现的问题有关。问题如下：

客户给接管数据库进行备份，发生报错：

错

```
误：manual backup error: rpc error: code = Unknown desc = task(manual_backup_manual_mysql-2db0mi_2022_07_09_22_30_33_mysql-2db0mi_2022_07_09_22_30_33) is failed,
reason is parse last gtid from file(/dmp/mysql/backup/3306/backup-616678719/tmp/backup.log) failed\n\n 回滚日志:\n\t
```

分析：

因为报错是原因是从文件中解析最后一个 gtid 失败，所以先检查 gtid 参数

数据库中的 gtid 参数：gtid_mode=OFF_PERMISSIVE（接管后数据库重启过）

```
mysql> show variables like '%gtid%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| binlog_gtid_simple_recovery | ON |
| enforce_gtid_consistency | ON |
| gtid_executed_compression_period | 1000 |
| gtid_mode | OFF_PERMISSIVE |
| gtid_next | AUTOMATIC |
| gtid_owned | |
| gtid_purged | |
| session_track_gtids | OFF |
+-----+-----+
8 rows in set (0.01 sec)
```

检查配置文件中 gtid_mode 相关信息为：

```
gtid_mode = 1
enforce-gtid-consistency=1
```

通过官网查询相关资料，得知 gtid_mode 有 4 个值：OFF；OFF_PERMISSIVE；ON_PERMISSIVE；ON

Logged transactions can be either anonymous or use GTIDs. Anonymous transactions rely on binary log file and position to identify specific transactions. GTID transactions have a unique identifier that is used to refer to transactions. The different modes are:

- OFF: Both new and replicated transactions must be anonymous.
- OFF_PERMISSIVE: New transactions are anonymous. Replicated transactions can be either anonymous or GTID transactions.
- ON_PERMISSIVE: New transactions are GTID transactions. Replicated transactions can be either anonymous or GTID transactions.
- ON: Both new and replicated transactions must be GTID transactions.

Changes from one value to another can only be one step at a time. For example, if gtid_mode is currently set to OFF_PERMISSIVE, it is possible to change to OFF or ON_PERMISSIVE but not to ON.

在此配置下，平台无法开启从库高可用配置，从库复制会报错：

```

Slave_IO_Running: No
Slave_SQL_Running: Yes
.....
      Last_IO_Errno: 1593
      Last_IO_Error: The replication receiver thread cannot start in AUTO_POS
ITION mode: the master has GTID_MODE = OFF_PERMISSIVE instead of ON.
      Last_SQL_Errno: 0
      Last_SQL_Error:

```

而且此时的数据库无法进行备份

本地测试:

1) 将数据库配置文件的 `gtid_mode` 修改为=1, 并重启数据库

```

# GTID #
gtid_mode = 1
enforce_gtid_consistency = 1
binlog_gtid_simple_recovery = 1

```

2) 重启后登录数据库中查看 `gtid_mode` 参数值, 发现重启后的 `gtid_mode=OFF_PERMISSIVE`

```

mysql> show variables like "%gtid%";
+-----+
| Variable_name | Value |
+-----+
| binlog_gtid_simple_recovery | ON |
| enforce_gtid_consistency | ON |
| gtid_executed_compression_period | 1000 |
| gtid_mode | OFF_PERMISSIVE |
| gtid_next | AUTOMATIC |
| gtid_owned | |
| gtid_purged | |
| session_track_gtids | OFF |
+-----+
8 rows in set (0.00 sec)

```

3) 将数据库配置文件的 `gtid_mode` 修改为=ON, 并重启数据库

```

# GTID #
gtid_mode = ON
enforce_gtid_consistency = 1
binlog_gtid_simple_recovery = 1

```

4) 重启后登录数据库中查看 `gtid_mode` 参数值, 发现重启后的 `gtid_mode=ON`

```

mysql> show variables like "%gtid%";
+-----+
| Variable_name | Value |
+-----+
| binlog_gtid_simple_recovery | ON |
| enforce_gtid_consistency | ON |
| gtid_executed_compression_period | 1000 |
| gtid_mode | ON |
| gtid_next | AUTOMATIC |
| gtid_owned | |
| gtid_purged | |
| session_track_gtids | OFF |
+-----+
8 rows in set (0.01 sec)

```

可得出：gtid_mode = 1 效果不等于 gtid_mode = ON，而是等于 gtid_mode = OFF_PERMISSIVE

因为 gtid_mode 参数的类型为 Enumeration，所以变量具有多个值的别名

gtid_mode 变量	配置文件可写值
OFF	0
OFF_PERMISSIVE	1
ON_PERMISSIVE	2
ON	3

```
mysql> show variables like "gtid_mode";
```

```
+-----+-----+
| Variable_name | Value |
```

```
+-----+-----+
| gtid_mode    | OFF   |
```

```
+-----+-----+
```

```
1 row in set (0.01 sec)
```

```
mysql> set global gtid_mode=0;
```

```
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> show variables like "gtid_mode";
```

```
+-----+-----+
| Variable_name | Value |
```

```
+-----+-----+
| gtid_mode    | OFF   |
```

```
+-----+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> set global gtid_mode=1;
```

```
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> show variables like "gtid_mode";
```

```
+-----+-----+
| Variable_name | Value          |
```

```
+-----+-----+
| gtid_mode    | OFF_PERMISSIVE |
```

```
+-----+-----+
```

```
1 row in set (0.01 sec)
```

```
mysql> set global gtid_mode=2;
```

```
Query OK, 0 rows affected (0.01 sec)

mysql> show variables like "gtid_mode";
+-----+-----+
| Variable_name | Value          |
+-----+-----+
| gtid_mode     | ON_PERMISSIVE |
+-----+-----+
1 row in set (0.00 sec)

mysql> set global gtid_mode=3;
Query OK, 0 rows affected (0.01 sec)

mysql> show variables like "gtid_mode";
+-----+-----+
| Variable_name | Value |
+-----+-----+
| gtid_mode     | ON    |
+-----+-----+
1 row in set (0.00 sec)
```

解决方法:

将配置文件中 `gtid_mode = 1` 改为 `gtid_mode = ON`

```
gtid_mode = ON
enforce-gtid-consistency=1
```

在数据库中输入

```
mysql> set global GTID_MODE = ON_PERMISSIVE;
Query OK, 0 rows affected (0.02 sec)

//在主从节点上执行如下命令，检查匿名事务是否消耗完毕，最好多检查几次，以便确认该参数的值是 0
mysql> SHOW STATUS LIKE 'ONGOING_ANONYMOUS_TRANSACTION_COUNT';
+-----+-----+
| Variable_name                | Value |
+-----+-----+
| Ongoing_anonymous_transaction_count | 0     |
+-----+-----+
1 row in set (0.00 sec)

mysql> set global GTID_MODE = ON;
Query OK, 0 rows affected (0.02 sec)
```

修改后就可以成功备份数据库了

总结:

日常运维中,我们习惯将设置为 ON 的参数写成=1,但是 `gtid_mode=1` 却不是将 `gtid_mode` 设置为 ON 所以需注意一定要写 `gtid_mode=ON`,或者 `gtid_mode=3`

参考:

https://dev.mysql.com/doc/mysql-replication-excerpt/8.0/en/replication-options-gtids.html#sysvar_gtid_mode
<https://dev.mysql.com/doc/mysql-replication-excerpt/8.0/en/replication-mode-change-online-enable-gtids.html>

2.27 mysql 客户端对配置文件的读取顺序

作者: 余振兴

我们都知道使用 mysql 客户端去访问 MySQL 数据库时,会以一定的顺序去读取不同位置的配置文件,但在一次做测试时,发现除了按照顺序读取默认的配置文件的默认路径外,mysql 还有额外的读取配置文件的行为,以下是在我本地测试环境做测试时遇到的一个有意思的小知识点

一、场景现象

在本地做测试时,发现一个奇怪的现象,当我使用 socket 打算登录数据库,发现不指定用户时,默认并不是用的 root 用户登录,而是被修改为了 zhenxing 用户

```
[root@10-186-61-162 ~]# mysql -S /data/mysql/3306/data/mysql.sock -p
Enter password:
ERROR 1045 (28000): Access denied for user 'zhenxing'@'127.0.0.1' (using password: NO)
```

通过观测当前 mysql 客户端的默认参数行为,可以看到与报错一致,默认用户确实是变为了 zhenxing

```
[root@10-186-61-162 ~]# mysql --help|egrep "user|host|port"
-h, --host=name      Connect to host.
-P, --port=#         Port number to use for connection or 0 for default to, in
-u, --user=name      User for login if not current user.
host                  127.0.0.1
port                  3306
user                  zhenxing
```

在这里作为 DBA,我们的第一反应是肯定是查看 `/etc/my.cnf` 文件中是否对默认用户做了配置,于是查看该配置文件的客户端配置参数,如下

```
[client]
host          = 127.0.0.1
170
```

```

user          = root
port          = 3306

[mysql]
host          = 127.0.0.1
user          = root
port          = 3306
prompt        = '\U[\d]> '

```

发现配置文件中的反而配置值是 root 用户，并没有对 zhenxing 用户做配置，看来读取的还不是这个配置文件，那是不是读取了其他配置文件呢，继续排查其他的配置文件

二、排查思路

1、获取配置文件读取顺序

我们先打印出所有可能读取的配置文件及其读取的顺序做逐个排查

查看 mysql 客户端读取配置文件的顺序

```

[root@10-186-61-162 ~]# mysql --verbose --help|grep my.cnf
                        order of preference, my.cnf, $MYSQL_TCP_PORT,
/etc/my.cnf /etc/mysql/my.cnf /usr/local/mysql/etc/my.cnf /data/mysql/3306/base/my.cnf
~/my.cnf

```

2、排查 /etc/my.cnf

/etc/my.cnf 在前面已经确认没有做相关配置，这里直接跳过

3、排查 /etc/mysql/my.cnf

查看 `/etc/mysql/my.cnf` 配置，发现不存在相关配置，排除

```

[root@10-186-61-162 ~]# cat /etc/mysql/my.cnf
cat: /etc/mysql/my.cnf: 没有那个文件或目录

```

4、排查 /usr/local/mysql/etc/my.cnf

查看 `/usr/local/mysql/etc/my.cnf` 配置,发现不存在相关配置,排除

```

[root@10-186-61-162 ~]# cat /usr/local/mysql/etc/my.cnf
cat: /usr/local/mysql/etc/my.cnf: 没有那个文件或目录

```

5、排查 /data/mysql/3306/base/my.cnf

查看 `/data/mysql/3306/base/my.cnf` 配置,发现不存在相关配置,排除

```

[root@10-186-61-162 ~]# cat /data/mysql/3306/base/my.cnf
cat: /data/mysql/3306/base/my.cnf: 没有那个文件或目录

```

6、排查 ~/.my.cnf

查看 `~/.my.cnf` 依旧不存在相关配置,排除

```

[root@10-186-61-162 ~]# cat ~/.my.cnf
cat: /root/.my.cnf: 没有那个文件或目录

```

至此按照 `mysql --verbose --help|grep my.cnf` 获取的配置文件读取路径都被排除，都未对用户 `zhenxing` 做配置

7、使用 `no-defaults` 排除配置文件干扰

- 尝试用 `--no-defaults` 不读取任何配置文件排除配置文件的干扰,看是否会恢复正常

```
[root@10-186-61-162 ~]# mysql --help|grep no-defaults
--no-defaults          Don't read default options from any option file

## 查看不读取配置文件时,客户端的默认值
[root@10-186-61-162 ~]# mysql --no-defaults --help|egrep "user|host|port"
-h, --host=name        Connect to host.
-P, --port=#           Port number to use for connection or 0 for default to, in
-u, --user=name        User for login if not current user.
host                    127.0.0.1
port                    3306
user                    zhenxing

## 查看读取的所有客户端配置文件参数设置
[root@10-186-61-162 ~]# mysql --print-defaults
mysql would have been started with the following arguments:
--host=127.0.0.1 --user=root --port=3306 --host=127.0.0.1 --user=root --port=3306 --
prompt=U[\d]> --user=zhenxing --password=***** --host=127.0.0.1 --port=3306
```

从上面输出的结果来看,我们可以得到以下 2 个基本现象:

- 即使指定 `--no-defaults` 不读取任何配置文件,这个 `user` 的默认值依旧是 `zhenxing` 用户
- 当输出 `--print-defaults` 获取实际运行值时,可以看到 `/etc/my.cnf` 下的 `[client]`, 和 `[mysql]` 标签下的属性配置从上到下被正确获取
- 除了 `/etc/my.cnf` 外,在最后还有 `--user=zhenxing --password=***** --host=127.0.0.1 --port=3306` 这 4 个参数被额外添加到了命令最后

ps: mysql 客户端和服务端读取配置的原则都是文件读取从上到下,后面相同参数配置覆盖前面的参数

经过一系列的排除,依旧没找到这个默认值被修改的源头

8、打印 mysql 客户端的系统调用

- 使用 `strace` 直接观测 mysql 客户端在执行时到底调用了哪些配置,以下是调用 `my.cnf` 相关配置的片段(对结果做了精简输出)

```
1. stat("/etc/my.cnf", {st_mode=S_IFREG|0644, st_size=195, ...}) = 0          = 3
2. stat("/etc/mysql/my.cnf", 0x7ffd56813180) = -1 ENOENT (No such file or directory)
3. stat("/usr/local/mysql/etc/my.cnf", 0x7ffd56813180) = -
1 ENOENT (No such file or directory)
4. stat("/data/mysql/3306/base/my.cnf", 0x7ffd56813180) = -
1 ENOENT (No such file or directory)
5. stat("/root/.my.cnf", 0x7ffd56813180) = -1 ENOENT (No such file or directory)
172
```



```
6. stat("/root/.mylogin.cnf", {st_mode=S_IFREG|0600, st_size=336, ...}) = 0
```

通过以上调用顺序可以可到以下结论:

- 1-5 行的调用顺序与我们验证的逻辑基本一致
- 2-5 行显示为 No such file or directory 与我们的验证结果一致
- 第 6 行输出, 增加了一个对 /root/.mylogin.cnf 的读取操作, 并且可以知道当前这个文件是确实存在的

9、排查 /root/.mylogin.cnf

看到这个文件我们一般都知道, 这个是 `mysql_config_editor` 工具用来配置 `login-path` 的生成的文件, 我们可以用以下方式查看当前的配置信息

```
[root@10-186-61-162 ~]# mysql_config_editor print --all
[client]
user = "zhenxing"
password = *****
host = "127.0.0.1"
port = 3306
```

这里可以看到配置中有一个 `client` 标签的连接参数配置, 配置的内容正好是我们文章开头显示的异常默认值, 到这我们基本定位了造成这个奇怪现象的原因, 出现这个故障场景的原因也是刚好这台是测试环境曾经做过一些 `mysql_config_editor` 用法的测试, 导致了该现象的发生

三、场景总结

1、mysql 客户端除了会按照命令 `mysql --verbose --help|grep my.cnf` 输出的常规的顺序读取配置外, 在最后还会额外的读取 `.mylogin.cnf` 文件中配置

2、即使指定了 `--no-defaults`, 依旧会去读取 `.mylogin.cnf` 中的 `[client]`, `[mysql]` 标签的配置值

其中官方文档也在以下链接中给到了明确的说明(以下是关键描述片段)

- <https://dev.mysql.com/doc/refman/8.0/en/option-file-options.html>

```
The mysql client reads [client] and [mysql] from other option files, and [client], [mysql], and [mypath] from the login path file.
```

```
Client programs read the login path file even when the --no-defaults option is used. This permits passwords to be specified in a safer way than on the command line even if --no-defaults is present.
```

个人猜测当时这么设计的目的是考虑如备份脚本等需要连接数据库时, 为了防止非必要的参数文件的干扰, 在指定了 `--no-defaults` 参数后依旧能使用到 `.mylogin.cnf` 文件中配置的加密密码, 提升安全性。

2.28 MySQL 存储过程中的只读语句超时怎么办？

作者：杨涛涛

MySQL 有一个参数叫 `max_execution_time`，用来设置只读语句执行的超时时间，但是仅对单独执行的 `select` 语句有效；对于非单独执行的 `select` 语句，比如包含在存储过程、触发器等内置事务块里则不生效。官方手册上对这个参数解释如下：

`max_execution_time` applies as follows:

- The global `max_execution_time` value provides the default for the session value for new connections. The session value applies to `SELECT` executions executed within the session that include no `MAX_EXECUTION_TIME(*N*)` optimizer hint or for which N is 0.
- `max_execution_time` applies to read-only `SELECT` statements. Statements that are not read only are those that invoke a stored function that modifies data as a side effect.
- `max_execution_time` is ignored for `SELECT` statements in stored programs.

那对这种非单独出现的 `select` 语句，该如何控制超时时间呢？

先来看下参数 `max_execution_time` 设置后的效果。此参数设置后，`select` 语句如果执行时间过长，会直接被 `cancel` 掉，并且报错，如下所示：

```
mysql> set @@max_execution_time=1000;
Query OK, 0 rows affected (0.00 sec)

mysql> select sleep(2) from t1 limit 1;
ERROR 3024 (HY000): Query execution was interrupted, maximum statement execution time exceeded
```

或者是采用直接加 Hint 的方式，也能限制 `select` 语句的执行时间：下面两种方式都能起到限制 `select` 语句执行时间的作用。

```
mysql> select /*+ max_execution_time(1000) */ sleep(2) from t1 limit 2;
ERROR 3024 (HY000): Query execution was interrupted, maximum statement execution time exceeded

mysql> select /*+ set_var(max_execution_time=1000) */ sleep(2) from t1 limit 2;
ERROR 3024 (HY000): Query execution was interrupted, maximum statement execution time exceeded
```

那如果把这条 `select` 语句封装在存储过程内部，按照手册上对参数 `max_execution_time` 的解释，则不生效。比如新建一个存储过程 `sp_test`：

```
DELIMITER $$

USE `ytt`$$
```

```

DROP PROCEDURE IF EXISTS `sp_test`$$

CREATE DEFINER=`admin`@`%` PROCEDURE `sp_test`()
BEGIN
select sleep(2) from t1 limit 1;
END$$

DELIMITER ;

```

重新设置 `max_execution_time` 值为 1 秒：调用存储过程 `sp_test`，可以正常执行，`select` 语句并没有被 `cancel` 掉！

```

mysql> call sp_test;
+-----+
| sleep(2) |
+-----+
| 0 |
+-----+
1 rows in set (2.01 sec)

Query OK, 0 rows affected (2.01 sec)

```

那如何解决这个问题呢？

为了方便大家测试，把语句 `select sleep(2) from t1 limit 1` 改为 `select sleep(2000) from t1 limit 1`。既然 MySQL 层面有这样的限制，那只能从非 MySQL 层面来想办法。最直接有效的就是写个脚本来主动 `cancel` 掉 `select` 语句。脚本如下：

```

root@ytt-normal:/home/ytt/script# cat kill_query
#!/bin/sh
QUERY_ID=`mysql -ss -
e "select id from information_schema.processlist where user='admin' and db='ytt' and ti
me>10 and regexp_like(info,'^select','i')"`
if [ $QUERY_ID ];then
echo "kill query $QUERY_ID"
mysql -e "kill query $QUERY_ID"
fi

```

完后把脚本放到 `crontab` 或者 MySQL 自己的 `event` 里来定时执行即可。单独执行脚本效果如下：

```

root@ytt-normal:/home/ytt/script# ./kill_query
kill query 50

```

除了自己编写脚本，还有一个工具可以实现类似的效果，它包含在我们熟知的 `Percona-toolkit` 工具箱里，叫 `pt-kill`。

`pt-kill` 工具可以根据各种触发条件来执行指定动作。比如 `cancel` 掉指定 SQL 语句、`kill` 掉指定 `session` 等。所以完全可以使用 `pt-kill` 工具来实现 `select` 语句超时被自动 `cancel` 掉。如下所示：`pt-kill` 工具会

在后台一直运行，监听 MySQL 进程，一旦触发条件被激活，即可执行相应动作。

```
root@ytt-normal:/home/ytt/script# pt-kill --match-db=ytt --match-user=admin --match-host=% \--match-info='^select' --victims=all --busy-time='10s' --print --kill-query  
  
# 2022-08-15T17:29:03 KILL QUERY 50 (Query 11 sec) select sleep(2000) from t1 limit 1
```

有一点需要注意：select 语句超时自动 cancel 掉这样的功能不适宜用在生产环境！因为你无法预知其执行结果的时效性、上下文是否相关等特点。

2.29 无损半同步复制下，主从高可用切换后数据一致吗？

作者：陈俊聪

高能警告，因作者能力有限，文章的分析过程没有基于 MySQL 源码，而是根据一些业界大佬的文章和官方文档整理和思考得出的结论，如果觉得说得有道理，请点个赞，如果觉得胡说八道请留言。本文的受众是至少已经入门 MySQL 的 DBA。

前言

1. 我们简单地复习一下半同步的原理：

MySQL5.7 默认参数下我们开启了半同步，在一个事务提交（commit）的过程时，在 MySQL 层的 write binlog 步骤后，Master 节点需要收到至少一个 Slave 节点回复的 ACK（表示收到了 binlog）后，才能继续下一个事务；

如果在一定时间内（Timeout）内没有收到 ACK，则切换为异步复制模式。这能保证数据不丢失的高可用需求，因为他能保证从库确认到这个事务后，再通知主库提交事务。这种模式下，至少一个从库的日志数据和主库保证同步，从而保证主库挂了后，数据不丢失，因为最新数据的是从库。

2. 我先说一下我的半同步保证数据一致性的相关设置

```
sync_binlog=1  
innodb_flush_log_at_trx_commit=1  
...(等等)
```

好了，不列出来了，我保证我半同步设置了最靠谱设置，不会因为参数设置错误而丢失数据。

3. 我再解释一下术语：

lossless semi-sync replication、无损半同步、增强半同步，都是一回事，说的是 MySQL5.7 参数 `rpl_semi_sync_master_wait_point=AFTER_SYNC` 设置下的半同步，他是默认值，解决高可用切换后的幻读问题。MySQL5.6 没有这个参数设置，这个版本在高可用切换后有概率出现幻读，产生数据丢失，那就是数据不一致。如没有特别指出，本文的“半同步”均指的是无损半同步。

4. 然后还有一点要注意的，kill -9 主库 mysqld 和 主动 shutdown 主库 mysqld 的行为引发高可用切换是不一样的，后者行为有概率会导致幻读，也就是数据丢失，具体原因看这两篇文章详细了解。

《如何正确地关闭 MySQL 数据库？99%的 DBA 都是错的！》

《技术分享 | 聊聊 MySQL 关机的故事》

5. 我确保我测试时，半同步复制没有降级为异步复制。

我上述说的，我假设大家都懂了，所以理论上，无损半同步复制下的主从高可用切换后数据一定一致吗？

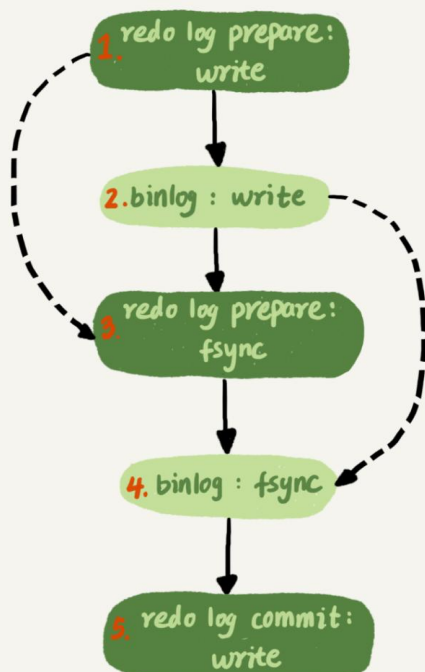
客户角度

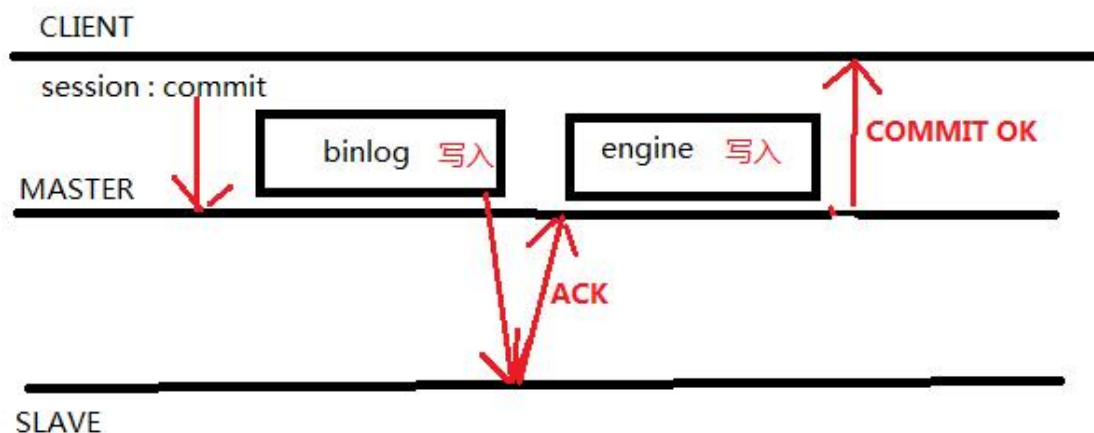
我先说答案，数据是一致的！但我这里说的数据一致，指的是，原主库上原先看到的数据库，在新主库（原备库）也是只能看到这些数据，不多也不少，也就是我指的是不会发生幻读。

主从角度

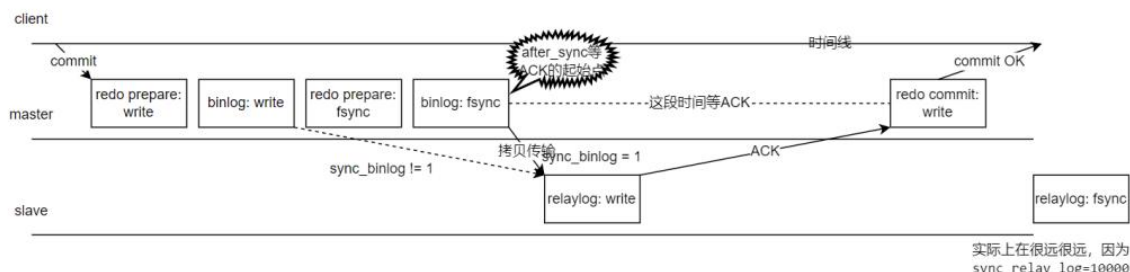
But！可能会有另外一种数据不一致，就是切换后，新主和旧主的数据不一致，旧主数据多了！这篇文章，主要探讨，为什么在主从角度下，数据有可能不一致。

为什么旧主数据多了？





这两张网上流传很广的图，大家如果觉得似曾相识，那必须是资深 DBA 了。我整合了一下，再结合八怪的高清大图（《深入理解 MySQL 主从原理 32 讲》专栏第 15 节），画了以下这张原理图：



- 图中只关注 binlog、redo、relaylog 三个文件写入和刷盘行为。
- 图中 write 代表写入文件，对于 Linux 系统一般就是写入到 OS 的 page cache 了。这个时候服务器发生 crash，日志数据将丢失。
- 图中 fsync 代表 MySQL 调用 Linux 的 fsync 把 page cache 持久化到磁盘，我们称之为"刷盘"。
- 根据《深入理解 MySQL 主从原理 32 讲》专栏第 15 节，sync_binlog 设置不一样，MySQL 的 dump binlog 线程工作的起始点会不一样，我们本文用的是传说中最安全的双一设置，所以 sync_binlog=1，从库到主库拉去 binlog 的时间点发生在 binlog: fsync 后。

这里是步骤解释：

1. 用户在 client 处执行 commit 提交了事务，主库写入了 redo 文件，对应"redo prepare: write"
2. binlog: write—— master 节点写入 binlog 文件。
3. redo prepare: fsync—— 传说中的两阶段提交的第一阶段 prepare 刷盘(从 OS 层 page cache 持久化刷新到硬盘)了。
4. binlog: fsync—— binlog 刷盘。
5. 拷贝 binlog 到 slave 写入到 relaylog 文件，写完则返回 ACK 给 master 节点，表示收到这个 binlog 了，relaylog 根据参数 sync_relay_log=10000 每 10000 个事务刷一次刷盘(因为我们只讨论主库 crash 的情况，这个从库 relay 刷盘行为我们不用管)
6. 根据参数设置 rpl_semi_sync_master_wait_point=AFTER_SYNC，所以在这个位置点开始等待 slave 节点完成"relaylog: write"，然后返回个 ACK 给 master。

7. master 节点收到 ACK 后，知道从库收到这个 binlog 后就可以提交事务了，发起"redo commit: write"，传说中的两阶段提交的第二阶段 commit 阶段，commit 到文件。
8. master 回复客户端 ACK: commit OK。

其中，我设置了双一参数：`sync_binlog=1`，表示事务提交之前，MySQL 都需要先把 BINLOG 刷新到磁盘

`innodb_flush_log_at_trx_commit=1`，表示在每次事务提交的时候，都把 log buffer (内存)刷到文件系统中 (os buffer)去，并且调用文件系统的“flush”操作将缓存刷新到磁盘上去。

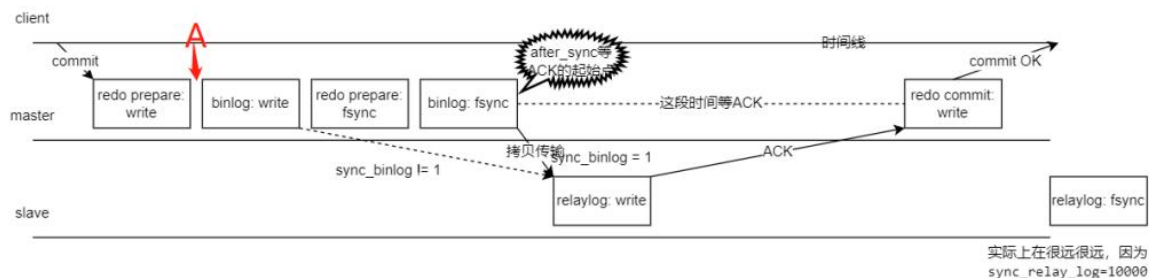
我认为这里指的是刷 redo prepare 状态，redo commit 状态是不需要刷盘的，这是安全的，并且减少一次 io。懂源码的大佬欢迎打脸。

主库崩溃后主从复制的行为，我直接引用丁奇的《MySQL 45 讲》第十五章的文字，有兴趣的同学可以直接去阅读原文。



图 1 两阶段提交示意图

A 阶段对应的是我图的这个位置

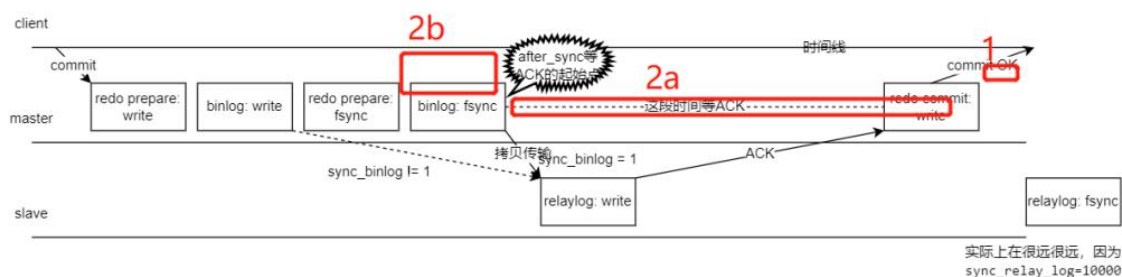


如果主库在 A 阶段发生了 crash，丁奇大佬这么说：

如果在图中时刻 A 的地方，也就是写入 redo log 处于 prepare 阶段之后、写 binlog 之前，发生了崩溃（crash），由于此时 binlog 还没写，redo log 也还没提交，所以崩溃恢复的时候，这个事务会回滚。这时候，binlog 还没写，所以也不会传到备库。到这里，大家都可以理解。

所以 A 阶段主库发生了 crash，主从数据会一致的。

B 阶段对应的是我图的这些位置，按丁奇的说法细分为三种可能，我单独标记了。



如果主库在 B 阶段发生了 crash，丁奇大佬这么说：

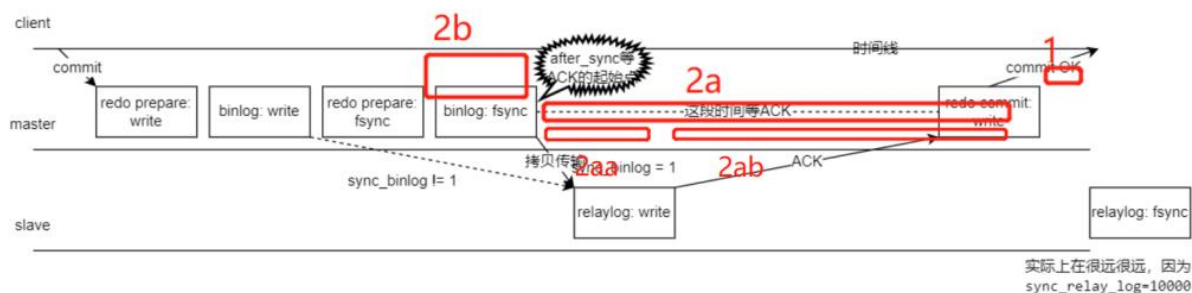
大家出现问题的地方，主要集中在时刻 B，也就是 binlog 写完，redo log 还没 commit 前发生 crash，那崩溃恢复的时候 MySQL 会怎么处理？

我们先来看一下崩溃恢复时的判断规则。

1. 如果 redo log 里面的事务是完整的，也就是已经有了 commit 标识，则直接提交；
2. 如果 redo log 里面的事务只有完整的 prepare，则判断对应的事务 binlog 是否存在并完整：
 - a. 如果是，则提交事务；
 - b. 否则，回滚事务。

这里，时刻 B 发生 crash 对应的就是 2(a) 的情况，崩溃恢复过程中事务会被提交。

2(a) 这个情况下，主库承认 binlog 的数据，但在拷贝到从库时，我对丁奇大佬的说法又细分为两个阶段，分别是 2aa 和 2ab 阶段，如下图：



- 2aa 阶段: 这个阶段从库 relaylog 没有拷贝完, 主库就挂了, 那么从库是不认这些 relaylog 的, 所以主库会多了事务。
- 2ab 阶段: 这个阶段从库 relaylog 已经拷贝完, ack 没有返回成功给主库, 这个时候主库挂了, 大家都基于相同的 binlog 承认相同的事务, 数据是一致的。

数据不一致的最直接证明就是, 高可用切换后, 旧主库起来后 gtid 比新主库(原备库)要多。

老主库的 uuid 下的 gtid 编号要多。

测试

原理上, 我们分析明白了, 我们来实际测试一下, 是不是真的会这样。

实验设计:

- 搭建一主一从半同步复制数据库
- sysbench 压测主库几十秒后
- kill -9 主库 mysqld
- 修改 mysqld_safe 或者 systemd 设置, 确保防止主库自动被拉起来。
- 查看主库 binlog, 查看从库 show slave status\G, 对比看两者 gtid 是否一致。

一开始, 因为虚拟机, 我的 sysbench 压测只能达到 20 tps 时, 测了好几次, 都无法测出问题。然后我调整了虚拟机的配置从 1 vcpu 升级到 8 vcpu, 压测能压到 60 tps 了上不去, 测了好几次, 还是无法测出问题。然后, 我发现了瓶颈在于网络, 因为我的 sysbench 连接数据库走的无线网卡了, 我更换为走有线网卡后, 压测性能达到了 200 tps。我调整压测的 lua 脚本, 从 oltp_read_write 调整为 oltp_write_only, 增加写 binlog 的机会, 那更容易模拟到 gtid 不一致问题, 并且此调整后 tps 涨到了 800。

终于, 我模拟到了现象。

主库崩溃后, 我查看从库情况, 发现 gtid 跑到 283030 这个编号。

```
[root@192-168-199-132 3306]# mysql -uadmin -pGta@2019 -
S /database/mysql/data/3306/mysqld.sock -e "show slave status\G" |grep "ffc43852-1d82-11ed-a65f-000c29375703"
mysql: [Warning] Using a password on the command line interface can be insecure.
      Master_UUID: ffc43852-1d82-11ed-a65f-000c29375703
 Retrieved_Gtid_Set: ffc43852-1d82-11ed-a65f-000c29375703:210837-283030
 Executed_Gtid_Set: ffc43852-1d82-11ed-a65f-000c29375703:1-283030
```

接着, 我解析主库的 binlog, 发现多了红色的三条 gtid 编号。

```
[root@192-168-199-131 3306]# cat 16.txt | grep GTID | grep "ffc43852-1d82-11ed-a65f-000c29375703:28303"
```

```
SET @@SESSION.GTID_NEXT= 'ffc43852-1d82-11ed-a65f-000c29375703:283030'/*!*/;
```

```
SET @@SESSION.GTID_NEXT= 'ffc43852-1d82-11ed-a65f-000c29375703:283031'/*!*/;
```

```
SET @@SESSION.GTID_NEXT= 'ffc43852-1d82-11ed-a65f-000c29375703:283032'/*!*/;
```

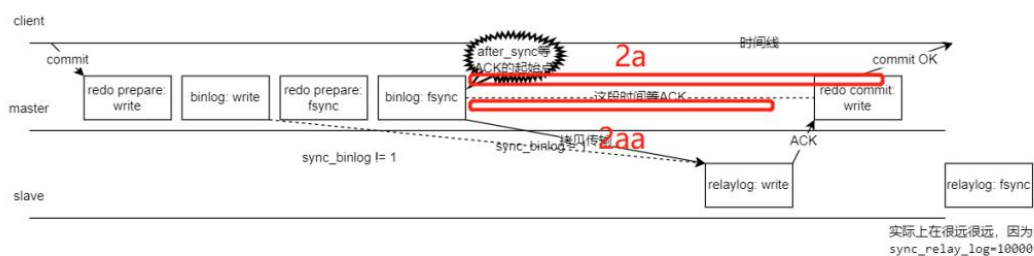
```
SET @@SESSION.GTID_NEXT= 'ffc43852-1d82-11ed-a65f-000c29375703:283033'/*!*/;
```

为什么这么难模拟

说难模拟也不难，只要 `tps` 高是很容易模拟出来的，这么难模拟出来，还有一个原因是，`2aa` 阶段其实持续时间很短。

有一个技巧，使用《第 02 问：怎么模仿磁盘 IO 慢的情况？》大佬黄炎提及的模拟 `io` 慢的方法，可以让你几乎每次都能模拟出来。

- 造一个慢的 `io` 设备 (好像只能造读写都慢的情况，没关系我们仅需要利用他读慢这个特性)
- 把它挂载到 `binlog` 盘
- 利用 `io` 设备的读很慢的特点，使从库拷贝 `binlog` 的持续时间拉长。如图 `2aa` 阶段。



模拟网络慢应该也能达到类似效果

为什么差异了三条 `gtid` 而不是一条？

这个可能和组提交有关，也可能和并发线程有关，这里不方便展开研究，这可能是另一篇文章。

主从数据不一致，如何修复

方法一 重启优先于切换

很多情况下，主库 `crash` 后，会被 `mysqld_safe` 或者 `systemd` 服务自动拉起来，如果数据库重启速度很快，其实不一定要切换，`RTO` 也能得到保证。旧主有多条 `gtid` 没关系，拉起来后，`gtid` 会自动同步到从库，数据就不会不一致啦！

方法二 切换后补数据

我们知道 `MHA` 的切换的大概逻辑是这样的：

1. 卸载旧主库 `VIP`
2. 选主，选出有最新 `gtid` 的从库作为新主(我只有一个从库，就是选他了。。)
3. 等待 `sql thread` 补齐数据(if 有复制延迟)
4. 主从角色切换

5. io thread 级补数, ssh 到旧主拉取 binlog 补数(if 主库 binlog 和从库有 gtid 差异)
6. 新主(原备库) 挂载 VIP, 去掉 read_only, 提供服务

也就是如果你用 MHA 这个高可用, 基于 binlog pos 的传统复制模式下, 发生了主从切换, 并且主库的主机没死, 能 ssh 过去的情况下, 从库会拉去主库 binlog 来补数, 数据不会不一致。

But! 很多 DBA 都知道了, MHA 在 gtid 模式下有 bug, 他 5 这个流程是不走的。我们大多数人用的都是 gtid 复制模式下的 MySQL, 所以这种情况下, 主从数据是有概率不一致的。

方法三 切换后旧主 flashback 数据

我看沃趣和万里开源的技术人写的文章发现, 他们玩法居然是补数据, 如果发现旧主多了 gtid 则 flashback 回退这些 gtid! 这样旧主才可以作为新从, 加回集群。

修复方法, 从库补数, 还是主库回滚, 谁对谁错?

对于主从 gtid 差异的修复方法, 居然分了两个派别:

- 从库补数据
- 主库回退数据

那么谁是对的谁是错的? 答案是—— 都行!

在主库 crash 或 kill -9 这种情况下, mysql 客户端会返回 **【ERROR 2013 (HY000) at line 1: Lost connection to MySQL server during query】**。发生了这个报错, 其实服务器是告诉你 “在执行查询的时候, 你与服务器失去连接了” 当你重新连上服务器后, 你应该发起一次重连确认数据的。我举个例子假设你原本在主库上执行以下 SQL, 一个叫 fander 的人给他老婆转账 10 万元。

```
begin;
update accounts set amount = amount - 100000 where account_id=1 and name='fander';
update accounts set amount = amount + 100000 where account_id=2 and name='fander's wife';
commit;
```

这个时候如果处理这笔业务的 MySQL 服务器主库 crash 了, 数据库给应用程序抛出的异常就类似于我刚才说的 **【ERROR 2013 (HY000) at line 1: Lost connection to MySQL server during query】**, 应用程序重连, 连上了高可用切换后的新主库(原备库), 本身就应该去查一下, 转账成功没, 转账成功就完事了, 没成功那就再执行一次这条事务。

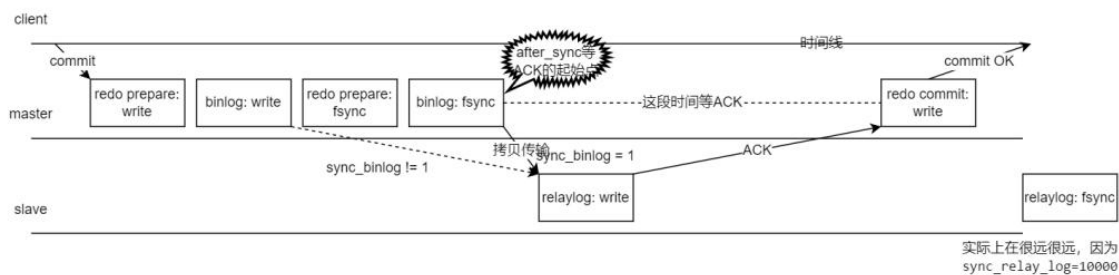
所以这个转账成功与否, 是 MySQL 告诉他。你从库补数, 还是不补数, 是数据库层决定, 然后告诉客户即可。

如果是主库 mysqld crash 了, 在新主提供服务之前, 可以选择基于远程拷贝 binlog, 把旧主日志补到新主。但 server crash 了, 从库已作为新主提供服务了, 这时旧主不能再往新主补数据了, 这会造成数据不一致! 这个场景下, 旧主加回集群只有两种选择:

1. 回退多余 gtid 建立复制, 加回集群
2. 对新主的备份, 重做旧主, 作为从库建立复制, 加回集群。

扩展思考

有没有可能主从高可用切换后，不是主库比从库数据要多 `gtid`，而是从库要比主库要多 `gtid`？在 `sync_binlog=1` 的情况下不会。



因为主库 binlog 持久化后才会同步给从库，不可能从库的 binlog 会比主库要新。

但 `sync_binlog != 1` 的情况下，有两个原因会导致主库的 binlog 没有从库新。

- binlog 不是每次事务都刷盘，主库挂了后，丢失一些事务。
- 即使每次刷盘也没用，因为 dump binlog 起始位置不一样，如果主库挂载在写入 binlog 后，网速很快完成传输到从库，但 binlog 还没有来得及 fsync 这种极限情况下发生了 crash，还是会导致主库丢失了这些 binlog，而从库承认了这些 binlog。

总结

无损半同步复制下，主从高可用切换后对于业务层面来说数据一致，对于运维层面来说，底层主从可能数据不一致，需要知道如何修复主从数据到一致。

参考

https://upload-images.jianshu.io/upload_images/7398834-4714e92068825403.png

<https://mp.weixin.qq.com/s/uZ5cr0-bB6K3GbokBmORqA>

<https://mp.weixin.qq.com/s/1BuNsfVBRwnxe1BNjLeiWg>

<https://mp.weixin.qq.com/s/8JNv17lIDfwMbTdBsTxhxQ>

MySQL 实战 45 讲丁奇(林晓斌)

2.30 MemAvailable 是怎么计算的

作者：胡呈清

背景

前两天安装 OceanBase 时遇到一个小问题：之前也遇到过类似问题，从领导那里求取真经，下面给大家简单分享下真经内容：

```
[root@observer1 ~]# obd cluster autodeploy obtest -c distributed-with-obproxy-example.yaml
Package oceanbase-ce-3.1.4-10000092022071511.el7 is available.
Package obproxy-ce-3.2.3.5-2.el7 is available.
install oceanbase-ce-3.1.4 for local ok
install obproxy-ce-3.2.3.5 for local ok
Cluster param config check ok
Open ssh connection ok
Generate observer configuration x
[ERROR] OBD-2000: (10.186.60.89) not enough memory. (Free: 4.9G, Need: 8.0G)
[ERROR] OBD-2000: (10.186.60.90) not enough memory. (Free: 6.3G, Need: 8.0G)
[ERROR] OBD-2000: (10.186.60.91) not enough memory. (Free: 6.3G, Need: 8.0G)
```

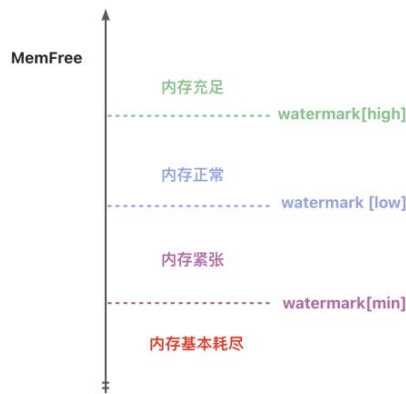
很明显，安装 OB 时要求服务器可用内存至少 8G，不达标就无法安装。为了凑这 3 台 10G 内存的服务器我已经费了不少劲了，free -m 输出中 free 不是有 9G 吗，为什么还报错？

```
[root@observer2 ~]# free -m
              total        used         free      shared  buff/cache   available
Mem:           9836          142          9428           16         265        6431
Swap:              0              0              0
```

那为什么 free -m 输出中：free 有 9.3G，而 available 只有 6.3G 呢？
通常我们会把 MemAvailable 看成是 buffer/cache、free 之和。但实际上不是，它其实还跟 min_free_kbytes 有着密切关系。

min_free_kbytes

kswapd 是专门用来定期回收内存的进程。为了衡量内存的使用情况，定义了三个内存阈值（watermark，也称为水位），分别是 watermark[min/low/high]：



上图基本揭示了几个水位的含义，当 MemFree 低于 watermark[low] 时，kswapd 进行内存回收，直到空闲内存达到 watermark[high] 后停止回收。如果申请内存的速度太快，导致空闲内存降至 watermark[min] 后，内核就会进行 direct reclaim（直接回收），用回收上来的空闲页满足内存申请，这样会阻塞应用程序。而 watermark[min] 的大小等于内核参数 min_free_kbytes 的值，其他几个水位的关系是：

- $\text{watermark}[\text{low}] = \text{watermark}[\text{min}] * 5/4$
- $\text{watermark}[\text{high}] = \text{watermark}[\text{min}] * 3/2$

MemAvailable

显然 $\text{watermark}[\text{min}]$ 以下的内存属于系统的自留内存，不会给普通进程申请使用。而 MemAvailable 意为可以分配使用的内存，因此它不应当包含这一块内存。实际上其计算公式为：

$$\text{MemAvailable} = \text{MemFree} - \text{watermark}[\text{LOW}] + (\text{PageCache} - \min(\text{PageCache} / 2, \text{watermark}[\text{LOW}]))$$

知道了 MemAvailable 是怎么计算的，接下来就很简单了，先查看 `min_free_kbytes` 的设置：

```
[root@observer2 ~]# cat /proc/sys/vm/min_free_kbytes
2097152
```

2G 是 OB 的部署规范，由于是测试环境，将它修改为 64M 后，MemAvailable 就符合要求了：

```
[root@observer2 ~]# free -m
              total        used         free      shared  buff/cache   available
Mem:           9836         142         9428          16         265        6431
Swap:              0              0              0
[root@observer2 ~]# sysctl vm.min_free_kbytes=67584
vm.min_free_kbytes = 67584
[root@observer2 ~]# free -m
              total        used         free      shared  buff/cache   available
Mem:           9836         142         9428          16         265        9419
Swap:              0              0              0
```

min_free_kbytes 设置建议

OB 的部署规范中，规定 `min_free_kbytes=2G`，不得不说这个点很细节，因为：

1. 系统会根据内存大小自动计算出 `min_free_kbytes` 大小，但并不是线性关系，取值范围是 128K-64M，如果系统开启了大页，则最大值通常会超过 64M，但也不会很大，以下面这台服务器为例，256G 内存，`min_free_kbytes` 只有 132M：

```
root@idrc-110:~# cat /proc/sys/vm/min_free_kbytes
135168
root@idrc-110:~# free -m
total used free shared buff/cache available
Mem: 257897 60060 2068 18161 195768 178009
Swap: 616 6 610
```

2. 如果 `min_free_kbytes` 设置的很小，则系统剩余可用内存容易触底，`direct reclaim` 会造成性能严重降低。相反如果设置的很大，则 `watermark[min/low/high]` 这 3 个水位都会很大，经常触发内存回收，使内存利用率降低。

所以为系统预留 2G 内存十分合理，是一个很容易被忽略的优化点。

参考资料：

<https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=34e431b0ae398fc54ea69ff85ec700722c9da773>

2.31 基于 Alertmanager 告警系统的改造

作者：莫善

一、引言

告警跟运维工作息息相关，一个好的告警系统不仅能提升运维的效率，还能提升相关人员的工作舒适度及生活质量。相反，如果告警系统比较拉胯，那运维的工作就比较难受了。比如，半夜收到无关痛痒的告警信息，又比如这个告警正在处理还一直在发，再比如同一时间产生很多告警，然后不重要的告警把重要的告警刷走了，等等。

本文想分享一下在使用 Alertmanager 的过程中遇到的一些困扰，以及分享一下最近在做的告警系统改造的项目，仅做经验交流。

二、前期准备

我们线上采用 Prometheus + Alertmanager 的架构进行监控告警。所以本文主要是基于 Alertmanager 组件进行介绍。

```
alertmanager, version 0.17.0 (branch: HEAD, revision: c7551cd75c414dc81df027f691e2eb21d4fd85b2)
  build user:      root@932a86a52b76
  build date:      20190503-09:10:07
  go version:      go1.12.4
```

1、待处理问题

（1）告警干扰

因历史遗留问题，我们线上环境的环境是一个集群一个 prometheus，然后共享一个告警通道 Alertmanager，有时候会出现 A 集群的告警信息跑到 B 集群的告警中。比如会收到像下面这种情况：

```
cluster: clusterA
instance: clusterB Node
alert_name: xxx
```

这个问题一直没找到原因，也没法稳定复现。

（2）告警升级

现在的告警系统在触发告警时候不会升级。比如优先发给值班人员，其次会根据接收人，告警时间，告警介质等进行升级。

关于告警升级后文有解释说明。

（3）告警恢复

对于已经恢复的告警，Alertmanager 不会发送一份告警恢复的提示。

（4）告警抑制

Alertmanager 针对重复的告警可以做到自定义时间进行抑制，但是不太智能，比如，同一个告警项，前面三次发送间隔短点，超过三次的间隔可以长点，比如第 1, 3, 5, 10, 20, 30 分钟发送。另外也不能做到自适应时间抑制，比如工作时间抑制时间间隔可以长一点（同一个告警十分钟发一次），休息时间的抑制时间短一点（同一个告警五分钟发一次）。

（5）告警静默

Alertmanager 支持告警静默功能，但是需要在 Alertmanager 平台进行配置。如果一个机器宕机后，可能触发很多告警需要静默，所以添加及事后删除静默规则的管理比较麻烦。

另外还有一个比较头疼的问题，正常通过告警页面可以点击【Silence】是可以将待静默的告警信息带到配置告警静默的页面，但是大部分时候都是不行的（空白页面），需要手动填写需要静默的告警信息，这就很头疼了。

（6）语音告警

Alertmanager 目前不支持语音告警。

这个问题不做为单独的问题进行介绍，会放在告警升级部分。

2、发现新问题

为了解决上述提到的【告警干扰】问题，我们采取的方法是将一个 Alertmanager 拆成多个（一个集群一个），这样能解决【告警干扰问题】，那么又带来了新的问题。

- 如何实现告警收敛？
- 如何管理告警静默？

（1）告警收敛

同一时刻产生多条告警，就会导致相关人员收到多条告警信息，这样即浪费告警资源，也对排查问题带来一定的干扰。比如单机多实例的场景，一台机器宕机，同时会产生好几十条告警，或者一个集群出现问题，所有节点都触发告警。针对这种场景如果没有告警收敛，会比较痛苦。

Alertmanager 本身支持收敛，因为我们需要解决【告警干扰】问题，所以我们改造的时候拆成了一个集群一个 Alertmanager，这样我们环境就没法用自带的收敛功能了。

（2）告警静默

本来单个 Alertmanager 的告警静默就比较难管理了，如果多个告警项，可能是多个 Alertmanager 需要静默，静默的管理就更加麻烦了。针对以上问题，下面会逐个介绍一下解决思路，部分解决方案不见得适合所有环境，其他环境或许有更好的解决方案，这里仅供参考。

三、告警改造

1、告警干扰问题

如前文所述，通过将一个 Alertmanager 拆成 n 个，这种方式看起来比较笨，但是却有效，如果有一个上帝视角将所有 Alertmanager 管理起来，就当作一个数据库实例去对待，其实也很方便。这类组件也不需要很多的系统资源，使用虚拟机完全够用。另外其实也有一个好处，加入说 Alertmanager 出现问题，比如进程正常，但是不会发告警了，这样也不至于团灭。

我们平台的监控、告警都已经实现自动化，并且都是通过平台进行管理，用一个 Alertmanager 跟多个，在部署安装上成本差不多，但是不太好管理（这个后文有说明）。

2、告警升级问题

发送告警的介质主要分几种，邮件，企业微信（其他即时通讯工具），短信，电话/语音。从左到右成本依次增高，所以为了告警资源不被浪费，尽可能的节省短信/电话这种告警介质，所以我们希望我们的告警系统是能自适应的进行调整。这个升级分如下几个维度：

- 第一：告警介质的升级。邮件 --> 企业微信 --> 短信 --> 电话（同一个告警项发送超过 3 次就向上升级一个介质，具体可以根据需求定义）。
- 第二：告警接收人升级。一级 --> 二级 --> leader。
- 第三：按照时间升级。工作时间通过邮件/企业微信发送告警，工作时间之外通过短信/电话发送告警。

这个问题想通过 Alertmanager 来解决好像不可行，所以只能通过其他手段进行曲线救国了。我们采取的方式是开发一个脚本读取 Alertmanager 的告警信息，然后通过脚本进行发送告警信息。

其实也可以直接读取 prometheus 的告警信息，原理上差不太多。

```
Send a detailed message to the DBA by Mail      #只要是告警就会通过邮件告知

if now_time > 8 and now_time < 22 :

    Send a simple message to the DBA by WX

else                                             #按照告警时间升级告警介质

    if alert_count > 3 and phone_count < 3 :

        Send a simple simple message to the DBA by phone      #短信告警升级电话告警

    elif alert_count > 3 and phone_count > 3 :

        Send a simple message to the leader by phone #接收告警人员升级

    else

        Send a simple message to the DBA by SMS              #告警介质升级
```

接受告警的人员包含值班 DBA 及项目负责人，如果接收告警的对象无法收到告警信息（联系方式异常，离职），就会读取通讯录获取一位同组的人员进行重新发送告警。

- 可以定义，多次告警后进行接收人员的告警升级，同理也可以定义告警时间，告警介质等告警升级。
- 只要是告警就会通过邮件告知，原因是短信/语音这种介质成本比较高，也不方便查阅，所以只会发送简单的告警提示，详细的需要查阅邮件。

3、告警收敛/抑制问题

首先需要有一个表来保存发送告警的记录，包含告警项（要求全局唯一，建议使用 ip:port），告警状态，累计告警次数，最后告警时间，整个系统的好几个功能都需要这个表。

```
CREATE TABLE `tb_alert_for_task` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT COMMENT '主键',
  `alert_task` varchar(100) DEFAULT '' COMMENT '告警项目',
  `alert_state` tinyint(4) NOT NULL DEFAULT '0' COMMENT '告警状态，0 表示已经恢复，1 表示正在告警',
  `alert_count` int(11) NOT NULL DEFAULT '0' COMMENT '告警的次数，单个告警项发送的告警次数是 10 次（每天至多发送十次）',
  `u_time` datetime NOT NULL DEFAULT '2021-12-08 00:00:00' COMMENT '下一次发送告警的时间',
  `alert_remarks` varchar(50) NOT NULL DEFAULT '' COMMENT '告警内容',
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_alert_task` (`alert_task`)
) ENGINE=InnoDB AUTO_INCREMENT=7049 DEFAULT CHARSET=utf8mb4;
```

通过查阅文档我们可以知道调用下面的告警信息 api 是可以获取当前告警信息。

```
api/v1/alerts?silenced=false&inhibited=false
```

一条告警信息长下面这样，我认为有用的就是【alertname】【cluster】【instance（主机部分）】这三个属性，可以将这三个属性作为收敛维度。

```
{
  'labels': {
    'alertname': 'TiDB_server_is_down',
    'cluster': 'cluster_name_001',
    'env': 'cluster_name_001',
    'expr': 'probe_success{
      group="tidb"
    }==0',
    'group': 'tidb',
    'instance': '192.168.168.1:4000',
    'job': 'tidb_port_probe',
    'level': 'emergency',
    'monitor': 'prometheus'
  }
}
```

这里省略了其他信息，仅保留了 labels 部分

（1）抑制

抑制逻辑是下一次发送告警时间小于当前时间，或者告警发送次数大于 10 次。为什么是这个逻辑？这需要结合收敛部分的代码介绍，所以后面解释。

#这是抑制的逻辑，将当前告警异常项都读出来，如果当前 `alertmanager` 的告警已经在这里面就视为抑制对象，因为这些告警还不满足再次发送的条件

```
select_sql = "select alert_task from tb_tidb_alert_for_task where alert_state = 1 and (u_time > now() or alert_count > 10);"
```

```
state, skip_instance = connect_mysql(opt = "select", sql = {"sql" : select_sql})
```

`skip_instance` 是个列表，在遍历告警信息的时候会用到，如果在这个列表里面就忽略这个告警，以此起到抑制的效果。

（2）收敛

收敛思路大概是这样，遍历每一条告警信息，遍历的时候会在【`tb_tidb_alert_for_task`】表记录一条记录，包含：

- `instance` 信息，这是全局唯一。
- 告警状态，区别是正在告警，还是已经恢复告警，恢复告警的时候用到。
- 发送告警次数，这个值会作为收敛参考数据。
- 下一次发送告警时间，这是方便处理抑制。

```
next_time = 0

if now_time > 8 and now_time < 22 :

    next_time = max_alert_count * 2 + 1

else :

    next_time = max_alert_count

insert_sql[instance_name] = """replace into tb_tidb_alert_for_task(alert_task,alert_
state,alert_count,u_time,alert_remarks)
    select '""' + instance_name + ""',1,""" + str(max_alert_count) + ""', date
_add(now(), INTERVAL + ""' + str(next_time) + ""' MINUTE),
    'tidb 集群告警';"""

state, _ = connect_mysql(opt = "insert", sql = insert_sql)
```

上面这部分逻辑是服务于抑制功能，所以可以解释的通，上文将下一次发送告警时间要求小于当前时间。

- 工作时间，发送一次告警后，下一次告警时间是间隔 $2n+1$ min(n 表示告警次数)。
- 非工作时间，发送一次告警后，下一次告警时间是间隔 $n+1$ min(n 表示告警次数)

下面这部分逻辑是遍历 `Alertmanager` 的所有处于 `active` 状态的告警信息，然后做分析处理。

```
def f_get_alert_to_msg(url) :
```

```

try :

    res = json.loads(requests.get(url, headers = header, timeout = 10).text) #读取
alertmanager 的告警状态

except Exception as err :

    return {"code" : 1, "info" : str(err)}

for temp in res["data"] :

    cluster_name = temp["labels"]["cluster"]

    alert_name = temp["labels"]["alertname"]

    instance_name = cluster_name + ":all" #特殊情况没有 instance 信息，这种时候是集群
告警，而不是某个节点告警

    if "instance" in temp["labels"] : instance_name = temp["labels"]["instance"]

    if len(global_instance_list) == 0 : global_instance_list = []

    if len(instance_name) > 0 : global_instance_list.append(instance_name) #这个
列表在判断是否是告警已经恢复用到

    if instance_name in skip_instance : continue # 符合抑制条件就忽略这个告警

    if cluster_name not in global_alert_cluster.keys() :      global_alert_cluster[c
luster_name] = {}

    if alert_name not in global_alert_name.keys() :      global_alert_name[alert_nam
e] = {}

    if instance[0] not in global_alert_host.keys() :      global_alert_host[instance
[0]] = {}

    if alert_name not in global_alert_cluster[cluster_name].keys() :      global_ale
rt_cluster[cluster_name][alert_name] = []

    if cluster_name not in global_alert_name[alert_name].keys() :      global_alert_
name[alert_name][cluster_name] = []

    if alert_name not in global_alert_host[instance[0]].keys() :      global_alert_h
ost[instance[0]][alert_name] = []

```

```

        if instance_name not in global_alert_cluster[cluster_name][alert_name] :      gl
global_alert_cluster[cluster_name][alert_name].append(instance_name)

        if instance_name not in global_alert_name[alert_name][cluster_name] :      globa
l_alert_name[alert_name][cluster_name].append(instance_name)

        if cluster_name + ":" + instance[1] not in      global_alert_host[instance[0]][a
lert_name] :      global_alert_host[instance[0]][alert_name].append(cluster_name + ":" +
instance[1])

    return {"code" : 0, "info" : "ok"}

```

将【alertname】【cluster】【instance】分别保存到【global_alert_name】【global_alert_cluster】【global_alert_host】这三个字典。

下面这个逻辑是遍历 alertmanager 的 url，根据 url 去扫对应的 alertmanager 的告警信息，可以看到代码中有一个判断 Alertmanager 状态的代码，可以起到监控 Alertmanager 的作用，会将访问异常的发送出来。

```

for url in url_list :

    status = f_get_alert_to_msg(url) #读取 alertmanager 告警信息

    if status == 1 :

        error_list.append(url)

    if len(error_list) > 0 :

        info = "Alertmanager 访问异常 : " + ",".join(error_list)

        status = f_alert_sms_to_user(tel,info)

```

最终根据【global_alert_name】【global_alert_cluster】【global_alert_host】三个字典的长度判断以哪个维度进行收敛，即哪个字典最短就以哪个维度为收敛对象。

```

if len(global_alert_cluster.keys()) < len(global_alert_host.keys()) and len(global_aler
t_cluster.keys()) < len(global_alert_name.keys()) :

    alert = global_alert_cluster

    info_tmp = "告警集群 : "

elif len(global_alert_name.keys()) < len(global_alert_host.keys()) :

    alert = global_alert_name

```

```

        info_tmp = "告警名称 : "

    else :

        alert = global_alert_host

        info_tmp = "告警主机 : "

```

4、告警恢复问题

```

#读取告警状态是 1, 且比当前时间还早的条目
sql = """select alert_task from tb_tidb_alert_for_task
        where alert_state = 1 and alert_remarks = 'tidb 集群告警
' and u_time < date_add(now(), INTERVAL - 1 MINUTE);"""

state, alert_instance = connect_mysql(opt = "select", sql = {"sql" : sql})

alert_ok = []

for instance in alert_instance :

    if instance not in global_instance_list : #global_instance_list 是在遍历告警信息
        的时候记录了所有 instance 信息

        alert_ok.append(instance)

        update_sql[instance] = """update tb_tidb_alert_for_task set alert_state = 0,
        alert_count = 0
        where alert_state = 1 and alert_remarks = 'tidb 集群短信告警
' and alert_task = '""" + instance + ""';"""

        state, _ = connect_mysql(opt = "update", sql = update_sql)

```

如果表里处在告警状态的记录不在当前正在告警的列表中, 就说明告警已经恢复, 这时候就会变更告警状态, 且将告警次数置为 0。

5、告警静默问题

(1) 添加静默

```
/api/v1/silences
```

```
try :
```

```

        expi_time = int(expi_time) #小时

except Exception as err :

    return {"code" : 1, "info" : str(err)}

if expi_time > 24 :

    return {"code" : 1, "info" : "The alarm cannot be silent for more than 24 hours"}

local_time = f_get_time() #当前时间 2022-01-01 00:00:00

local_time = dt.datetime.strptime(local_time, '%Y-%m-%d %H:%M:%S')

start_time = local2utc(local_time).strftime("%Y-%m-%dT%H:%M:%S.000Z") #换成 UTC 时间

end_time = dt.datetime.strptime(((dt.datetime.now() - timedelta(hours = -
expi_time))).strftime("%Y-%m-%d %H:%M:%S")), "%Y-%m-%d %H:%M:%S")

end_time = local2utc(end_time).strftime("%Y-%m-%dT%H:%M:%S.000Z")

dic = {
    "id" : "",
    "createdBy" : user,
    "comment" : comment,
    "startsAt" : start_time ,
    "endsAt" : end_time,
    "matchers" : [
        {
            "name" : name,
            "value" : value
        }
    ]
}

try :

    res = json.loads(requests.post(url, json = dic).text)

except Exception as err :

    res = {"code" : 1, "info" : str(err)}

```

```
return res
```

这里需要注意，添加静默一定需要让用户提供静默超时时间，比如 2h，上限是 24 小时，这样可以避免因时间过长，然后遗忘规则，导致告警一直被静默。

另外用户提供的是静默小时数，而添加静默是需要一个开始时间和结束时间，且需要 UTC 时间，所以需要通过计算特殊处理一下。

最后还需要注意，这里添加静默是单一规则，即只有一个 name-value 对，不支持与条件及正则。如果需要多个条件可以追加 matchers 列表的值。

```
"matchers" : [
  {
    "name" : name1,
    "value" : value1
  },
  {
    "name" : name2,
    "value" : value2
  }
]
```

(2) 删除静默

这部分需要用到两个 api，第一个是先获取规则 id，通过 id 进行删除。

```
/api/v1/silences?silenced=false&inhibited=false
/api/v1/silence/id
```

```
url = "http://xxx/api/v1/silences?silenced=false&inhibited=false"

try :

    id_info = json.loads(requests.get(url).text)

except Exception as err :

    return {"code" : 1, "info" : str(err)}

for item in id_info["data"] :

    if item["status"]["state"] != "active" : continue #非 active 状态的直接忽略

    if item["matchers"][0]["name"] != name or item["matchers"][0]["value"] != value : continue #不满足条件的也直接忽略

    try : #先要获取 id 才能进行删除规则
```



```

url = "http://xxx/api/v1/silence/" + item["id"]

res = json.loads(requests.delete(url).text)

except Exception as err : #访问失败

    res = {"code" : 1, "info" : str(err)}

return res

```

可以看到告警静默的管理还是比较麻烦的，所以我们已经将这部分管理功能整合到平台，可以通过平台的告警管理列表页进行告警静默的管理，可以通过 ip, instance, cluster, role, alert_name 这几个维度进行管理，也支持告警信息展示，可以通过展示页面逐个告警添加静默，也可以将所有告警一键静默，这样就解决了告警静默难管理的问题。

四、注意事项

- 如果没有平台进行管理，不建议使用这样的方式运维告警系统。
- 添加告警静默的时候强烈建议添加超时时间，且不宜过长，避免添加后遗忘。
- 添加静默的时候一定要做到心里有数，避免出现故障告警被顺带添加静默而又未进行处理的情况。

五、写在最后

本文所有内容仅供参考，因各自环境不同，并非通用方案，且在使用文中代码时可能碰上未知的问题。
如有线上环境操作需求，请在测试环境充分测试。

2.32 MySQL--测试--timestamp 字段默认值测试

作者：姚嵩

背景：

客户反馈表定义中含 timestamp 字段(SQL 语句)，定义如下：

```
b timestamp NOT NULL
```

在 MySQL 中执行后，变成如下：

```
b timestamp NOT NULL DEFAULT '0000-00-00 00:00:00'
```

想知道其原因，这里涉及到了两个技术点：

- MySQL 如何给 timestamp 字段设置默认值。

- 00 这种时间格式是否被允许。

MySQL 如何给 timestamp 字段设置默认值:

涉及参数:

`explicit_defaults_for_timestamp`

参数解析:

默认情况, 启用`explicit_defaults_for_timestamp`,

如果启用参数, 表示禁止非标准行为, 此时按如下方式处理 timestamp 列:

- 若要分配当前时间戳, 只能设置列为 `CURRENT_TIMESTAMP` 或同义词 `NOW()`, 不能通过 `NULL`。
- 没有使用 `NOT NULL` 属性显式声明的 timestamp 列将自动使用 `NULL` 属性声明并允许使用 `NULL` 值。
- 对于 `NOT NULL` 属性声明的 timestamp 列不允许使用 `NULL` 值。
- 使用了 `NOT NULL` 属性但未使用 `DEFAULT` 属性的 timestamp 会被认为没有默认值。
- `DEFAULT CURRENT_TIMESTAMP` 或者 `UPDATE CURRENT_TIMESTAMP` 属性不会被自动声明, 只能显式指定。
- 表中第一个 timestamp 列的处理和其他 timestamp 列的处理方式并无不同。

如果禁用参数, 表示开启了非标准行为, 此时按如下方式处理 timestamp 列:

- 没有显式声明 `NULL` 属性的 timestamp 列将自动声明 `NOT NULL` 属性;
- 并允许插入 `NULL` 值(8.0.22 之前), 但会将该列设置设置为当前时间戳。
- 表中第一个 timestamp 列, 如果没有显式声明 `NULL` 属性或者显式的 `DEFAULT` 属性或者 `ON UPDATE` 属性, 则自动使用 `DEFAULT CURRENT_TIMESTAMP` 和 `ON UPDATE CURRENT_TIMESTAMP` 属性声明该列。
- 除第一个 timestamp 列以外的其他 timestamp 列, 如果没有显式声明 `NULL` 属性或者显式的 `DEFAULT` 属性, 则会自动声明为 `DEFAULT '0000-00-00 00:00:00'("zero"时间戳)`。若 `sql_mode` 中包含了 `NO_ZERO_DATE`, 则默认值可能无效。

zero 时间戳是否被允许:

- 通过 `sql_mode` 中的 `NO_ZERO_DATE` 值控制。
- 如果 `sql_mode` 中含 `NO_ZERO_DATE`, 则不允许 `'0000-00-00 00:00:00'("zero"时间戳)`, 否则允许。

MySQL5.7.24 测试:

测试场景:

建表语句:

```
create database if not exists db_test;
use db_test;
drop table if exists tb_a;
create table tb_a(a timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
,

```

```
b timestamp NOT NULL
) engine=InnoDB;
show create table tb_a \G
```

场景 1:

```
set session explicit_defaults_for_timestamp=1;
set session sql_mode='NO_ZERO_DATE';
```

测试截图 1:

```
mysql> set session explicit_defaults_for_timestamp=1;
Query OK, 0 rows affected (0.01 sec)

mysql> set session sql_mode='NO_ZERO_DATE';
Query OK, 0 rows affected, 2 warnings (0.00 sec)

mysql> create database if not exists db_test;
Query OK, 1 row affected (0.00 sec)

mysql> use db_test;
Database changed
mysql> drop table if exists tb_a;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> create table tb_a(a timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP ,
->                        b timestamp NOT NULL
->                        ) engine=InnoDB;
Query OK, 0 rows affected (0.04 sec)

mysql> show create table tb_a \G
***** 1. row *****
Table: tb_a
Create Table: CREATE TABLE `tb_a` (
  `a` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  `b` timestamp NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
1 row in set (0.00 sec)
```

说明 1:

因为 `explicit_defaults_for_timestamp` 值为 1，并不会为 timestamp 字段设置默认值。

场景 2:

```
set session explicit_defaults_for_timestamp=1;
set session sql_mode='';
```

测试截图 2:

```
mysql> set session explicit_defaults_for_timestamp=1;
Query OK, 0 rows affected (0.00 sec)

mysql> set session sql_mode='';
Query OK, 0 rows affected (0.00 sec)

mysql> create database if not exists db_test;
Query OK, 1 row affected, 1 warning (0.00 sec)

mysql> use db_test;
Database changed
mysql> drop table if exists tb_a;
Query OK, 0 rows affected (0.01 sec)

mysql> create table tb_a(a timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP ,
->                        b timestamp NOT NULL
->                        ) engine=InnoDB;
Query OK, 0 rows affected (0.01 sec)

mysql> show create table tb_a \G
***** 1. row *****
Table: tb_a
Create Table: CREATE TABLE `tb_a` (
  `a` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  `b` timestamp NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
1 row in set (0.00 sec)
```

说明 2:

因为 `explicit_defaults_for_timestamp` 值为 1，并不会为 timestamp 字段设置默认值。

场景 3:

```
set session explicit_defaults_for_timestamp=0;
set session sql_mode='NO_ZERO_DATE';
```

测试截图 3:

```
mysql> set session explicit_defaults_for_timestamp=0;
Query OK, 0 rows affected (0.00 sec)

mysql> set session sql_mode='NO_ZERO_DATE';
Query OK, 0 rows affected, 2 warnings (0.00 sec)

mysql> create database if not exists db_test;
Query OK, 1 row affected, 1 warning (0.02 sec)

mysql> use db_test;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> drop table if exists tb_a;
Query OK, 0 rows affected (0.02 sec)

mysql> create table tb_a(a timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP ,
-> b timestamp NOT NULL
-> ) engine=InnoDB;
ERROR 1067 (42000): Invalid default value for 'b'
mysql> show create table tb_a \G
ERROR 1146 (42S02): Table 'db_test.tb_a' doesn't exist
```

说明 3:

因为 `explicit_defaults_for_timestamp` 值为 0，且 b 字段不是第一个 timestamp 字段，没有显示声明 NULL 属性或 DEFAULT 属性，所以会赋予默认值 '0000-00-00 00:00:00'，又因为 sql_mode 中含 NO_ZERO_DATE，不允许 zero 时间戳，所以报错：ERROR 1067 (42000): Invalid default value for 'b'

场景 4:

```
set session explicit_defaults_for_timestamp=0;
set session sql_mode='';
```

测试截图 4:

```
mysql> set session explicit_defaults_for_timestamp=0;
Query OK, 0 rows affected (0.00 sec)

mysql> set session sql_mode='';
Query OK, 0 rows affected (0.00 sec)

mysql> create database if not exists db_test;
Query OK, 1 row affected, 1 warning (0.01 sec)

mysql> use db_test;
Database changed
mysql> drop table if exists tb_a;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> create table tb_a(a timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP ,
-> b timestamp NOT NULL
-> ) engine=InnoDB;
Query OK, 0 rows affected (0.03 sec)

mysql> show create table tb_a \G
***** 1. row *****
Table: tb_a
Create Table: CREATE TABLE `tb_a` (
  `a` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  `b` timestamp NOT NULL DEFAULT '0000-00-00 00:00:00'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
1 row in set (0.00 sec)
```

说明 4:

因为 `explicit_defaults_for_timestamp` 值为 0，且 `b` 字段不是第一个 `timestamp` 字段，没有显示声明 `NULL` 属性或 `DEFAULT` 属性，所以会赋予默认值 `'0000-00-00 00:00:00'`。

总结：

上面测试围绕客户场景设置，还有更多测试场景可供探索。

不过还是建议：

```
explicit_defaults_for_timestamp=1
sql_mode 含 NO_ZERO_DATE
```

2.33 MySQL--测试--排序规则 collation

作者：姚嵩

背景：

客户反应查询返回的结果不符合预期，读取时想要实现大小写区分；

简介：

排序规则 `collation` 是一个字符集的字符进行比较的一组规则；

至少会有两个规则：

- 1) 是否区分大小写；
- 2) 编码；

常用的规则是是否区分大小写。

字符集和排序规则的默认选择：

如果你仅指定字符集，而不指定排序规则，则排序规则为字符集默认的排序规则；

如果你仅指定排序规则，不指定字符集，则字符集为排序规则对应的字符集；

例外项见：

"设置对象的字符集和排序规则" 中 "说明"列中包含的内容。

查看字符集与其默认的排序规则：

```
SHOW CHARACTER SET;
```

或者

```
select * from INFORMATION_SCHEMA.CHARACTER_SETS ;
```

查看字符集包含的排序规则：

```
SHOW COLLATION WHERE Charset = 'utf8mb4';
```

 -- 这里的 utf8mb4 是具体的字符集

查看数据库的默认字符集和排序规则：

```
USE db_name;
SELECT @@character_set_database, @@collation_database;
```

或者

```
SELECT DEFAULT_CHARACTER_SET_NAME, DEFAULT_COLLATION_NAME
FROM INFORMATION_SCHEMA.SCHEMATA WHERE SCHEMA_NAME = 'db_name';
```

设置对象的字符集和排序规则：

对象	字符集	排序规则	说明
server	character_set_server	collation_server	如果create database时未带上字符集和排序规则，则使用server中声明的作为默认值；
database	建库时的CHARACTER SET 子句； character_set_database	建库时的COLLATE 子句； collation_database	如果create table时未带上字符集和排序规则，则使用数据库中声明的作为默认值； 如果load data时未带上character set子句，则使用character_set_database作为默认值； 如果在创建routine时未带上字符集和排序规则，则使用数据库中声明的作为默认值；
table	建表时的CHARACTER SET 子句；	建表时的COLLATE 子句；	如果未在单个列上指定字符集和排序规则，则将表中声明的作为默认值；
column	建表时字段定义上的CHARACTER SET 子句；	建表时字段定义上的COLLATE子句；	
字符串	SELECT _utf8mb4'abc' ;	SELECT 'abc' COLLATE utf8_general_ci;	如果未指定字符集但指定了排序规则，则字符集使用character_set_connection； 如果未指定字符集和排序规则，则使用character_set_connection和collation_connection作为默认值；

说明：

如果客户是查询表中的数据，那么寻找数据时是否忽略大小写，取决于对应字段上的 COLLATE子句中定义的排序规则；

如果客户是直接查询字符串，那么寻找数据时是否忽略大小写，取决于 collation_connection 的值；

数据存入不受 collation 影响，数据查询受 collation 影响。

测试案例：

测试环境：

MySQL Version: 8.0.28

基础环境：

```
create database if not exists test_db ;
```

```
use test_db;
drop table if exists test_db.a ;
create table test_db.a(i char(1)) engine=InnoDB default charset=utf8mb4 ;
```

测试：确认 collate 是否影响数据的存入

查询表中的数据时，值是否返回受字段上的 collation 影响

步骤：

```
alter table test_db.a change i i char(1) collate utf8mb4_0900_ai_ci ; -- 设置列上忽略大小写
insert into test_db.a values('a'),('A');
select * from test_db.a ; -- 存入的值含 "a" "A"
select * from test_db.a where i='a'; -- 返回 "a" "A"
truncate table test_db.a ;
alter table test_db.a change i i char(1) collate utf8mb4_0900_as_cs ; -- 设置列上区分大小写
insert into test_db.a values('a'),('A');
select * from test_db.a ; -- 存入的值含"a" "A"
select * from test_db.a where i='a'; -- 返回 "a"
truncate table test_db.a ;
```

结果截图：

```
mysql> alter table test_db.a change i i char(1) collate utf8mb4_0900_ai_ci ; -- 设置列上忽略大小写
Query OK, 0 rows affected (0.02 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> insert into test_db.a values('a'),('A');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> select * from test_db.a ; -- 存入的值含 "a" "A"
+-----+
| i |
+-----+
| a |
| A |
+-----+
2 rows in set (0.00 sec)

mysql> select * from test_db.a where i='a'; -- 返回 "a" "A"
+-----+
| i |
+-----+
| a |
| A |
+-----+
2 rows in set (0.00 sec)

mysql> truncate table test_db.a ;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql>          alter table test_db.a change i i char(1) collate utf8mb4_0900_as_cs ;    -- 设置列上区分大小写
Query OK, 0 rows affected (0.00 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>          insert into test_db.a values('a'),('A');
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>          select * from test_db.a ;      -- 存入的值含"a" "A"
+-----+
| i     |
+-----+
| a     |
| A     |
+-----+
2 rows in set (0.00 sec)

mysql>          select * from test_db.a where i='a';  -- 返回 "a"
+-----+
| i     |
+-----+
| a     |
+-----+
1 row in set (0.00 sec)

mysql>          truncate table test_db.a ;
Query OK, 0 rows affected (0.02 sec)
```

说明:

数据存入不受表字段上的 collate 影响;

但是搜索记录时会受表字段上的 collate 影响。

第 3 章 故障分析

本章节分享内容都源于用户的现场故障。为您分析故障原因和解决定位故障的过程，将问题的相关知识点加以融合说明。为您呈现一线 DBA 最真实的工作写照。希望在看过作者们的故障分析后，有助于读者快速定位解决现场故障。

3.1 undo log 长时间处于清理状态导致备份失败

作者：王祥

有客户反映有 3 套 MGR 环境全备失败，MySQL 版本为 8.0.18，xtrabackup 版本为 8.0.9。报错信息如下：

```
xtrabackup: Generating a list of tablespaces
Directories to scan './;./;.'
Scanning './'
Completed space ID check of 2 files.
Allocated tablespace ID 12 for zxc/a, old maximum was 0
Undo tablespace number 1 was being truncated when mysqld quit.
Cannot recover a truncated undo tablespace in read-only mode
xtrabackup: error: xb_load_tablespaces() failed with error code 57
```

根据报错到 percona 官方搜索。找到如下链接 xtrabackup issue(<https://jira.percona.com/browse/PXB-1787>):

It becomes a problem for xtrabackup at prepare stage because there is no MLOG_FILE_CREATE record to bind the undo space_id to the file name.

Consider following scenario:

xtrabackup starts the backup, opens InnoDB files and copies undo logs and other tablespaces. Lets say 'undo_001' has the space_id = 1. When xtrabackup completed the copying of 'undo_001' server decides to truncate it. space_id for 'undo_001' becomes 2. xtrabackup doesn't know anything about it, completes the backup.

During prepare, xtrabackup will assign space_id = 1 to the 'undo_001' by looking at the file header. Some redo logs for 'undo_001' will be for the space_id=1 and will be applied, more recent ones will be for space_id=2 and will be skipped. This will cause xtrabackup to produce inconsistent backup or crash at rollback stage.

One possible fix would be to make sure that we apply log record when it appears to be undo log (undo log space ids belong to fixed space id range) record even if the file does not exist. When there is no file for given undo space id, we could create dummy undo log file with the name (something_undo_<space_id>). Then we could use this file as an additional undo tablespace for rollback.

报错原因：undo log 清理会导致 undo log 的 space_id 变化，进而导致 xtrabackup 备份失败。处理方法也很简单重启一下备份的从节点 undo log 恢复正常，重新备份成功。

虽然暂时已经解决了备份的问题，但不能总是重启来解决。接下来重点看看 undo log 清理的问题。

查看 error log 信息,发现在 00:48 已经开始清理 undo log ，备份是在凌晨 1 点开始，参数 innodb_max_undo_log_size 设置为 1G ，正常的话应该也能清理完。

```
2021-10-26T00:48:41.543308+08:00 0 [Note] [MY-012994] [InnoDB] Truncating UNDO tablespace 'innodb_undo_001'.
2021-10-26T00:57:25.369852+08:00 0 [Note] [MY-012994] [InnoDB] Truncating UNDO tablespace 'innodb_undo_002'.
2021-10-26T01:02:01.994594+08:00 11751 [Warning] [MY-012111] [InnoDB] Trying to access missing tablespace 4294967152
2021-10-26T01:02:01.994645+08:00 11751 [Warning] [MY-012111] [InnoDB] Trying to access missing tablespace 4294967151
```

在测试环境复现以上问题：

```
#将 mgr 主节点与从节点参数 innodb_max_undo_log_size 同时修改成 10M
mysql>set global innodb_max_undo_log_size=10485760;
#使用 sysbench 发压
sysbench oltp_read_write --mysql-host=10.186.63.54 --mysql-port=3306 --mysql-user=root --mysql-password=123456 --mysql-db=sysbench --tables=6 --table-size=1000000 --report-interval=1 --threads=1 --time=300 run
```

过几秒就可以看到从节点会生成 undo_1_trunc.log 与 undo_2_trunc.log ，停止 sysbench 发压。等了半小时 undo_1_trunc.log 与 undo_2_trunc.log 依然存在。但主节点 undo log 清理的很快。

执行全备：

```
xtrabackup --defaults-file=/etc/my.cnf \
--target-dir=/opt/xtrabackup/full \
--user=root \
--password=123456 \
--socket=/mysqldata/mysql_3306/tmp/mysql.sock \
--backup
```

得到报错：

```
xtrabackup: Generating a list of tablespaces
Directories to scan './;./;.'
Scanning './'
Completed space ID check of 2 files.
Allocated tablespace ID 12 for ccc/a, old maximum was 0
Undo tablespace number 1 was being truncated when mysqld quit.
Cannot recover a truncated undo tablespace in read-only mode
xtrabackup: error: xb_load_tablespaces() failed with error code 57
```

首先查看是不是由于有大事务占用导致 undo log 无法被清理使用命令：show engine innodb status\G ，未发现有事事务占用 undo log 。难道是 bug 导致的？带着问题去 bug 库搜了一下，果真是一个

bug :<https://bugs.mysql.com/bug.php?id=104573>。

Description:

ON MySQL innodb cluster, One master and one standby instance, and set super_read_only = 1 on the standby machine. When the standby machine triggers undo truncate, it always fails when updating dd. And this problem occurs on multiple platforms, such as centos7, centos8, ubuntu20.04, docker.

undo log 长时间处于清理状态, 该问题是由参数 `super_read_only` 触发的 bug。可以将 `super_read_only` 设置成 0 临时解决。这也解释了为什么在主节点上 undo log 清理的很快。

那有什么办法减少触发这个 bug 呢? 我们可以将 `innodb_max_undo_log_size` 调大。在 InnoDB 存储引擎中, undo 在完成事务回滚和 MVCC 之后, 就可以 purge 掉了, purge 掉文件虽然大小不会变, 但是可以被重新分配使用。在 `innodb_max_undo_log_size` 设置较大的情况下, undo log 大小达到阈值前, 被 purge 掉了, 这样空间可以重用, 很难达到阈值, 所以不会触发 `undo log truncate`, 所以也就不会触发这个 bug。

3.2 MySQL 数据”丢失”事件之 binlog 解析应用一则

作者: 余振兴

事件背景

客户反馈在晚间数据跑批后, 查询相关表的数据时, 发现该表的部分数据在数据库中不存在, 从应用跑批的日志来看, 跑批未报错, 且可查到日志中明确显示当时那批数据已插入到数据库中, 需要帮忙分析这批数据丢失的原因。

备注: 考虑信息敏感性, 以下分析场景测试环境模拟, 相关数据做以下说明

- 涉及的库表为 `demo.t_dba_info` 表
- 丢失的数据为 `insert into t_dba_info(name,age) values('zhenxing',20);` 这条记录

故障分析

1. 先登录数据库确认该条记录是否存在

显然, 数据确实如客户所说, 在数据库中不存在

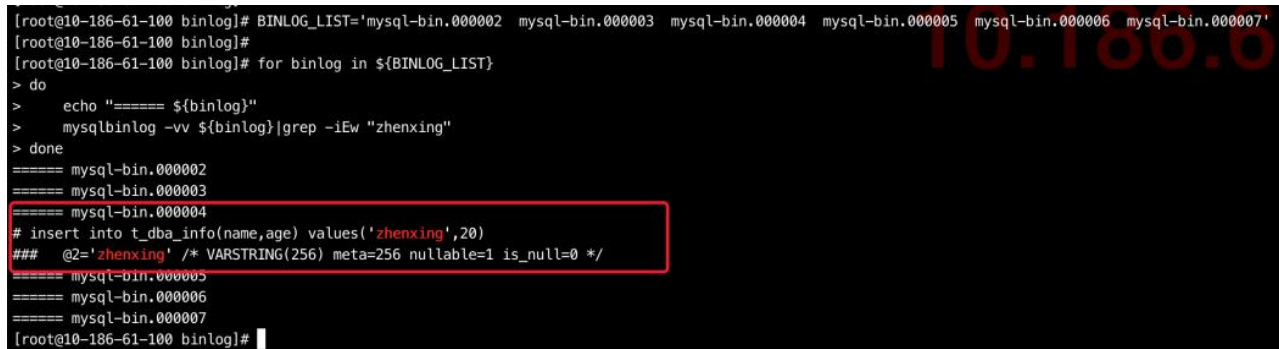
```
localhost demo SQL > select * from t_dba_info where name='zhenxing' and age=20;
Empty set (0.0004 sec)
localhost demo SQL >
```

2. 确认该条数据丢失的时间区间并解析 binlog

这里我为模拟环境,直接在主库解析,生产环境建议都在从库解析避免对主库造成影响

```
BINLOG_LIST='mysql-bin.000002 mysql-bin.000003 mysql-bin.000004 mysql-  
bin.000005 mysql-bin.000006 mysql-bin.000007'
```

```
for binlog in ${BINLOG_LIST}  
do  
    echo "=====${binlog}"  
    mysqlbinlog -vv ${binlog}|grep -iEw "zhenxing"  
done
```



```
[root@10-186-61-100 binlog]# BINLOG_LIST='mysql-bin.000002 mysql-bin.000003 mysql-bin.000004 mysql-bin.000005 mysql-bin.000006 mysql-bin.000007'  
[root@10-186-61-100 binlog]#  
[root@10-186-61-100 binlog]# for binlog in ${BINLOG_LIST}  
> do  
>     echo "=====${binlog}"  
>     mysqlbinlog -vv ${binlog}|grep -iEw "zhenxing"  
> done  
=====${mysql-bin.000002}  
=====${mysql-bin.000003}  
=====${mysql-bin.000004}  
# insert into t_dba_info(name,age) values('zhenxing',20)  
## @2='zhenxing' /* VARSTRING(256) meta=256 nullable=1 is_null=0 */  
=====${mysql-bin.000005}  
=====${mysql-bin.000006}  
=====${mysql-bin.000007}  
[root@10-186-61-100 binlog]#
```

可以看到我们通过解析并搜索 `zhenxing` 这条记录,确实发现数据插入了数据库中,所以接下来从常规的思路来说我们只需要继续解析 binlog,找到是否有对该条记录做 `DELETE` 或 `UPDATE` 操作即可

3. 解析 binlog 查看对这张表的修改操作

过滤出哪些 binlog 对该表做了 `DELETE` 或 `UPDATE`

这里我通过已知的故障时间区间将涉及的 binlog 列出来做循环解析

```
BINLOG_LIST='mysql-bin.000002 mysql-bin.000003 mysql-bin.000004 mysql-  
bin.000005 mysql-bin.000006 mysql-bin.000007'
```

```
for binlog in ${BINLOG_LIST}  
do  
    echo "=====${binlog}"  
    mysqlbinlog --no-defaults --base64-output=decode-rows -  
vv ${binlog} | awk '/###/ {if($0~/UPDATE|INSERT|DELETE/)count[$2] "$NF"}++}END{for(i in  
count) print i,"\t",count[i]}' | column -t | sort -k2nr|grep -i t_dba_info  
done
```

```
[root@10-186-61-100 binlog]# for binlog in $(BINLOG_LIST)
> do
>   echo "=====${binlog}"
>   mysqlbinlog --no-defaults --base64-output=decode-rows -vv ${binlog} | awk '/###/ {if($0~/UPDATE|INSERT|DELETE/)count[$2] "$NF"}++}END{for(i
| column -t | sort -k2nr|grep -i t_dba_info
> done
===== mysql-bin.000002
===== mysql-bin.000003
===== mysql-bin.000004
INSERT `demo`.`t_dba_info` 3
===== mysql-bin.000005
INSERT `demo`.`t_dba_info` 2
===== mysql-bin.000006
UPDATE `demo`.`t_dba_info` 2
===== mysql-bin.000007
[root@10-186-61-100 binlog]#
```

通过解析 binlog 可以看到,对该表的操作只有 mysql-bin.000006 这个 binlog 文件有 2 次 UPDATE 操作,其他都是 INSERT,接下来我们只需要继续解析这个 mysql-bin.000006 文件并搜索看是否对 zhenxing 这条记录是否做了修改即可

4. 解析定位的 binlog

用最简单的命令直接解析并搜索对 demo.t_dba_info 表插入的 zhenxing 这条记录

```
[root@10-186-61-100 binlog]# mysqlbinlog -vv mysql-bin.000006|less
```

```
[root@10-186-61-100 binlog]# mysqlbinlog -vv mysql-bin.000006|grep -i zhenxing
[root@10-186-61-100 binlog]#
```

```
# at 7681087
#211211 14:53:55 server id 2831 end_log_pos 7681153 Rows_query
# update t_dba_info set age=30 where name='dba1'
# at 7681153
#211211 14:53:55 server id 2831 end_log_pos 7681206 Table_map: `demo`.`t_dba_info` mapped to number 124
# at 7681206
#211211 14:53:55 server id 2831 end_log_pos 7681276 Update_rows: table id 124 flags: STMT_END_F
```

```
# at 9384616
#211211 14:53:56 server id 2831 end_log_pos 9384682 Rows_query
# update t_dba_info set age=29 where name='dba3'
# at 9384682
#211211 14:53:56 server id 2831 end_log_pos 9384735 Table_map: `demo`.`t_dba_info` mapped to number 124
# at 9384735
#211211 14:53:56 server id 2831 end_log_pos 9384805 Update_rows: table id 124 flags: STMT_END_F
```

通过解析发现这个 binlog 文件做对 demo.t_dba_info 表的 UPDATE 操作并不是针对 zhenxing 这条记录的,分析到这里发现比较迷惑了,数据明明插入了,也没做修改怎么就不见了,难道做了一些特殊操作

5. 排除一些特殊操作的可能性

1. 在插入这条数据时,主库 binlog 明确有记录,那是否有可能在删除这条记录时做了 set session

sql_log_bin=off 不记录 binlog

- 这个只需在从库查询下这条记录是否存在即可初步排除,客户生产环境是一主多从的架构,从库均没有这条记录存在,可能性被排除

2. 有没有可能这张表除了 DML 行为,还有 DDL 行为,如重建了,但重建后这批数据没有被重新插入该表,于是尝试解析 binlog 看对该表的 DDL 操作行为是否存在

```
BINLOG_LIST='mysql-bin.000002 mysql-bin.000003 mysql-bin.000004 mysql-
bin.000005 mysql-bin.000006 mysql-bin.000007'
```

```
for binlog in ${BINLOG_LIST}
do
    echo "=====${binlog}"
    mysqlbinlog ${binlog}|egrep -iEw "truncate|create|drop"
done
```

```
[root@10-186-61-100 binlog]# BINLOG_LIST='mysql-bin.000002 mysql-bin.000003 mysql-bin.000004 mysql-bin.000005 mysql-bin.000006 mysql-bin.000007'
[root@10-186-61-100 binlog]#
[root@10-186-61-100 binlog]# for binlog in ${BINLOG_LIST}
> do
>     echo "=====${binlog}"
>     mysqlbinlog ${binlog}|egrep -iEw "truncate|create|drop"
> done
==== mysql-bin.000002
==== mysql-bin.000003
==== mysql-bin.000004
truncate table t_dba_info
truncate table t_dba_info
==== mysql-bin.000005
==== mysql-bin.000006
==== mysql-bin.000007
[root@10-186-61-100 binlog]#
```

发现了一些端倪,在 mysql-bin.000004 中有对该表的 2 次 truncate 操作,等等,好像发现了什么,那条丢失的数据也是在这个 mysql-bin.000004 文件中,梳理下逻辑,难道那条记录在 2 次 truncate 之间,于是单独对这个 binlog 做详细解析,得到以下信息

```
#211211 14:52:47 truncate table t_dba_info
#211211 14:53:00 insert into t_dba_info(name,age) values('zhenxing',20)
#211211 14:53:18 truncate table t_dba_info
```

到此基本了解了这条记录为何会诡异丢失了,与客户确认跑批灌数据的逻辑,了解到会对该表做 truncate,但由于误操作,在跑批开始后,又触发了一轮 truncate 行为,导致已经插入到该表的部分数据再次被清理了,也就导致了在解析 binlog 时部分记录丢失了,但并未观测到有删除的行为,而是被 truncate 方式清理。

6. 故障总结

本文是对 binlog 解析的一个实践案例,binlog 记录的信息非常多,可以对 binlog 进行不同维度的解析,同时 binlog 在线上环境的配置使用上也有着一些技巧,如本案例中,线上环境因为是规范化部署,参数设置合理,不会由于单个 binlog 文件过大导致 binlog 解析时间过长,以及如 binlog_rows_query_log_events 参数的开启,使得在 row 模式下也可以明确记录下具体的 SQL 语句

```
max_binlog_size = 250M
binlog_rows_query_log_events = 1
```

备注:虽然 binlog 记录的信息足够多,但当故障原因定位后,由于其并未记录对该操作的 IP 及用户信息,如果不开审计,也只能知道发生了该行为,但无法具体定位触发该行为的“人”。

7. binlog 解析技巧

1. 尽可能在从库解析,避免对主库造成影响
2. 先粗略定位涉及相关的库表操作的 binlog,再单独解析对应的 binlog 中的数据
3. 在解析 DDL 时无需加 -v 输出详细信息(加快解析速度)
4. 如果开启了 binlog_rows_query_log_events 参数,需要用 -vv 参数才可显示具体的 SQL 语句

3.3 MySQL 设置 terminology_use_previous 参数导致数据库

Crash

作者：余振兴

背景信息

由于安全因素,客户需要将 MySQL 升级到 8.0.26 版本,但由于 8.0.26 的一些术语的不兼容性变更,对于监控采集的工具/程序会出现异常,针对这个情况,MySQL 官方也提供了解决方案,那就是新增了一个参数 `terminology_use_previous`,当将该参数设置为 `BEFORE_8_0_26` 时,可以保持 8.0.26 版本之前的术语形式,如依旧保持 `master`, `slave` 的术语形式,以下是官方文档 8.0.26 release note 的描述片段摘要:

```
Incompatible Change: From MySQL 8.0.26, new aliases or replacement names are provided for most remaining identifiers that contain the terms “master”, which is changed to “source”; “slave”, which is changed to “replica”; and “mts” (for “multithreaded slave”), which is changed to “mta” (for “multithreaded applier”). Help text is also changed where applicable to use the new names.

If the incompatible changes do have an impact for you, you can set the new system variable terminology_use_previous to BEFORE_8_0_26 to make MySQL Server use the old versions of the names for the objects specified in the previous list. This enables monitoring tools that rely on the old names to continue working until they can be updated to use the new names. The system variable can be set with session scope to support individual functions, or global scope to be a default for all new sessions. When global scope is used, the slow query log contains the old versions of the names.
```

当升级到 8.0.26 完成后,数据库开启正常监控采集,频繁的触发了 MySQL crash,需要分析是什么原因导致,以下的分析日志均为测试环境模拟。

故障分析

查看 crash 日志我们可以看到 crash 时 MySQL 最后执行的是一条 `select * from information_schema.processlist` 命令,从 crash 的堆栈信息来看,基本就是执行一个普通的 sql 触发了 libstdc 这个库的异常,导致数据库发出信号直接 abort MySQL,个人不太懂代码。从错误信息中

`basic_string::_S_construct null not valid` 的关键字搜索了解到，这是一个 C++ 的报错，大致含义是：一个实例化的逻辑错误导致程序中止。在 `what()` 函数中不能使用 `NULL` 来构造 `basic_string` 的对象。

```
terminate called after throwing an instance of 'std::logic_error'
  what():  basic_string::_S_construct null not valid
10:11:30 UTC - mysqld got signal 6 ;
Most likely, you have hit a bug, but this error can also be caused by malfunctioning hardware.
Thread pointer: 0x7f9924000c20
Attempting backtrace. You can use the following information to find out
where mysqld died. If you see no messages after this, something went
terribly wrong...
stack_bottom = 7f9980596d80 thread_stack 0x46000
/data/mysql/3320/base/bin/mysqld(my_print_stacktrace(unsigned char const*, unsigned long)+0x2e) [0x2251b8e]
/data/mysql/3320/base/bin/mysqld(handle_fatal_signal+0x323) [0x10fc283]
/lib64/libpthread.so.0(+0xf5f0) [0x7f99908d25f0]
/lib64/libc.so.6(gsignal+0x37) [0x7f998eb1d337]
/lib64/libc.so.6(abort+0x148) [0x7f998eb1ea28]
/lib64/libstdc++.so.6(__gnu_cxx::__verbose_terminate_handler()+0x165) [0x7f998f42da95]
/lib64/libstdc++.so.6(+0x5ea06) [0x7f998f42ba06]
/lib64/libstdc++.so.6(+0x5ea33) [0x7f998f42ba33]
/lib64/libstdc++.so.6(+0x5ec53) [0x7f998f42bc53]
/lib64/libstdc++.so.6(std::__throw_logic_error(char const*)+0x77) [0x7f998f480857]
/data/mysql/3320/base/bin/mysqld() [0xea8720]
/lib64/libstdc++.so.6(std::basic_string<char, std::char_traits<char>, std::allocator<char>>::basic_string(char const*, std::allocator<char> const&)+0x38) [0x7f998f48c998]
/data/mysql/3320/base/bin/mysqld(THD::proc_info(System_variables const&) const+0x43) [0xf35d33]
/data/mysql/3320/base/bin/mysqld() [0x10216b0]
/data/mysql/3320/base/bin/mysqld(Fill_process_list::operator()(THD*)+0x2c8) [0x1024b58]
/data/mysql/3320/base/bin/mysqld(Global_THD_manager::do_for_all_thd_copy(Do_THD_Impl*)+0x220) [0xebc6d0]
/data/mysql/3320/base/bin/mysqld() [0x1020a13]
/data/mysql/3320/base/bin/mysqld(do_fill_information_schema_table(THD*, TABLE_LIST*, Item*)+0x83) [0x1025923]
/data/mysql/3320/base/bin/mysqld(MaterializeInformationSchemaTableIterator::Init()+0x91) [0x14550a1]
/data/mysql/3320/base/bin/mysqld(Query_expression::ExecuteIteratorQuery(THD*)+0x345) [0x1085465]
/data/mysql/3320/base/bin/mysqld(Query_expression::execute(THD*)+0x2c) [0x10856cc]
/data/mysql/3320/base/bin/mysqld(Sql_cmd_dml::execute(THD*)+0x2c5) [0x101a885]
/data/mysql/3320/base/bin/mysqld(mysql_execute_command(THD*, bool)+0xac8) [0xfbc2c8]
/data/mysql/3320/base/bin/mysqld(dispatch_sql_command(THD*, Parser_state*)+0x410) [0xfc0b00]
```



```

/data/mysql/3320/base/bin/mysqld(dispatch_command(THD*, COM_DATA const*, enum_server_co
mmand)+0x1f10) [0xfc2e90]
/data/mysql/3320/base/bin/mysqld(do_command(THD*)+0x174) [0xfc3bf4]
/data/mysql/3320/base/bin/mysqld() [0x10ed858]
/data/mysql/3320/base/bin/mysqld() [0x2778d4c]
/lib64/libpthread.so.0(+0x7e65) [0x7f99908cae65]
/lib64/libc.so.6(clone+0x6d) [0x7f998ebe588d]

```

Trying to get some variables.

Some pointers may be invalid and cause the dump to abort.

Query (7f9924019488): select * from information_schema.processlist

Connection ID (thread ID): 9

Status: NOT_KILLED

The manual page at <http://dev.mysql.com/doc/mysql/en/crashing.html> contains

如果是程序逻辑的问题，对此我们就有点无力了，但还是尝试再进一步采集相关信息观测，我们配置在 MySQL crash 时自动生成 coredump，通过整理查看到以下信息(片段有删减,只截取了部分关键片段)，在对 `_S_construct` 结构做定义时出现了 `__throw_logic_error`，基本和 crash 日志中记录的信息一致(能力有限，未能详细分析其中代码逻辑)，coredump 的生成方式可参考链接：

<https://opensource.actionsky.com/20200424-mysql/>

```

Thread 1 (Thread 0x7f1ee80c9700 (LWP 12545)):
#0  0x00007f1efa7e3a01 in pthread_kill () from /lib64/libpthread.so.0
#1  0x00000000010fc2ad in handle_fatal_signal ()
#2  <signal handler called>
#3  0x00007f1ef8a33277 in raise () from /lib64/libc.so.6
#4  0x00007f1ef8a34968 in abort () from /lib64/libc.so.6
#5  0x00007f1ef93427d5 in __gnu_cxx::__verbose_terminate_handler() () from /lib64/libstdc++.so.6
#6  0x00007f1ef9340746 in ?? () from /lib64/libstdc++.so.6
#7  0x00007f1ef9340773 in std::terminate() () from /lib64/libstdc++.so.6
#8  0x00007f1ef9340993 in __cxx_throw () from /lib64/libstdc++.so.6
#9  0x00007f1ef9395597 in std::__throw_logic_error(char const*) () from /lib64/libstdc++.so.6
#10 0x0000000000ea8720 in char* std::string::_S_construct<char const*>(char const*, char const*, std::allocator<char> const&, std::forward_iterator_tag) ()
#11 0x00007f1ef93a1778 in std::basic_string<char, std::char_traits<char>, std::allocator<char> >::basic_string(char const*, std::allocator<char> const&) () from /lib64/libstdc++.so.6
#12 0x0000000000f35d33 in THD::proc_info(System_variables const&) const ()
#13 0x00000000010216b0 in thread_state_info(THD*, THD*) ()
#14 0x0000000001024b58 in Fill_process_list::operator()(THD*) ()
#15 0x0000000000ebc6d0 in Global_THD_manager::do_for_all_thd_copy(Do_THD_Impl*) ()
#16 0x0000000001020a13 in fill_schema_processlist(THD*, TABLE_LIST*, Item*) ()

```

```
#17 0x000000001025923 in do_fill_information_schema_table(THD*, TABLE_LIST*, Item*) ()
#18 0x0000000014550a1 in MaterializeInformationSchemaTableIterator::Init() ()
#19 0x000000001085465 in Query_expression::ExecuteIteratorQuery(THD*) ()
#20 0x0000000010856cc in Query_expression::execute(THD*) ()
#21 0x00000000101a885 in Sql_cmd_dml::execute(THD*) ()
#22 0x000000000fbc2c8 in mysql_execute_command(THD*, bool) ()
#23 0x000000000fc0b00 in dispatch_sql_command(THD*, Parser_state*) ()
#24 0x000000000fc2e90 in dispatch_command(THD*, COM_DATA const*, enum_server_command)
()
#25 0x000000000fc3bf4 in do_command(THD*) ()
#26 0x0000000010ed858 in handle_connection ()
#27 0x000000002778d4c in pfs_spawn_thread ()
#28 0x00007f1efa7dee25 in start_thread () from /lib64/libpthread.so.0
#29 0x00007f1ef8afbbad in clone () from /lib64/libc.so.6
```

经过一系列排除式验证，发现如果不考虑兼容监控采集的正确性，将 `terminology_use_previous` 参数设置为默认值 `NONE` 时，不会触发 `crash` 的现象，也就是说是由于该参数的配置导致的异常行为，并且通过逐个排除，基本确定以下 2 个关键点

- `terminology_use_previous` 必须设置为 `BEFORE_8_0_26`
- 必须有对 `PROCESSLIST` 表的访问请求

接下来关注 `crash` 时记录的对 `processlist` 表的查询操作，我们查看官方文档对 `processlist` 表的说明可以看到，对 `processlist` 表的查询会持有 `global mutex`，频繁的查询其实会对性能有一定影响，但理论上并不会造成 `crash`，当然官方也提供了一个参数 `performance_schema_show_processlist` 来实现无锁版的进程状态采集。

The default SHOW PROCESSLIST implementation iterates across active threads from within the thread manager while holding a global mutex. This has negative performance consequences, particularly on busy systems. The alternative SHOW PROCESSLIST implementation is based on the Performance Schema processlist table. This implementation queries active thread data from the Performance Schema rather than the thread manager and does not require a mutex.

The SHOW PROCESSLIST statement provides process information by collecting thread data from all active threads. The `performance_schema_show_processlist` variable determines which SHOW PROCESSLIST implementation to use:

The alternative SHOW PROCESSLIST implementation is based on the Performance Schema processlist table. This implementation queries active thread data from the Performance Schema rather than the thread manager and does not require a mutex.

尝试开启 `performance_schema_show_processlist=on` 参数及设置 `terminology_use_previous=BEFORE_8_0_26` 来规避 `global mutex`，但依旧能稳定复现 `crash` 现象，而使用 `performance_schema.threads` 表做会话状态的查询则并不会触发 `crash`，所以实际上虽然官方文档中提到设置

`performance_schema_show_processlist=on` 和查询 `performance_schema.threads` 都是不加锁的实现，但两者

的实现方式应该是有差异的。

几番验证下来依旧定位不到问题所在,尝试搜索下 MySQL bug 列表看是否有类似 bug , 以及浏览 release note 看是否新版本有对该问题的修复或描述, 得到的情况是并未有相关的 bug 的描述, 但在 8.0.27 的 release note 中提到一句有相关性的信息, 当并发的访问 INFORMATION_SCHEMA.PROCESSLIST 表时, 会导致 MySQL 故障(该说明后的 Bug #32625376 是 MySQL 内部 BUG 号, 我们无法查看到具体信息)

A SELECT query on the INFORMATION_SCHEMA.PROCESSLIST view with concurrent MySQL Server load caused a failure. (Bug #32625376)

尝试在 8.0.26 版本开多个会话, 同时执行 `select * from INFORMATION_SCHEMA.PROCESSLIST`, 并且关闭监控采集的干扰, 果然稳定复现 crash 现象, (用 `show processlist` 也可复现). 看来确实是这个问题, 于是开始将 MySQL 版本升级到 8.0.27, 再进行验证, 遗憾的是, 当升级到 8.0.27 以后依旧可以复现这个问题, 看来并未彻底解决。

于是尝试给 MySQL 提个 BUG 说明, 很快得到了官方回复, 确认该现象已在内部确认为 BUG, 会在即将发布的 8.0.28 版本做修复。

[7 Jan 6:19] MySQL Verification Team

Hello yu yu,

Thank you for the report and test case.

This is duplicate of Bug #105027 - Crash on 8.0.26 when terminology_use_previous is set to BEFORE_8_0_26

Internal base bug is closed and hence closing this as well:

Fixed as of the upcoming 8.0.28 release:

Executing SHOW PROCESSLIST or accessing INFORMATION_SCHEMA.PROCESSLIST on a server with a specific configuration caused a failure.

Please note that once 8.0.28 is released then change log would appear accordingly here - <https://dev.mysql.com/doc/relnotes/mysql/8.0/en/news-8-0-28.html>

regards,
Umesh

故障总结

- 本次个人分析因能力有限并未彻底了解该 BUG 的深层次原因, 但从最近的几个版本来看, 因为一些术语的变化, 确实对 MySQL 的使用和兼容性上造成了一定影响
- 对于一些比较诡异的现象可以尝试搜索 MySQL BUG 列表或向官方提 issue, 也就能快速得到结果
- 对于监控采集程序, 如果需要采集会话信息, 推荐使用 `performance_schema.threads` 表
- 对会话信息的采集频率不要太高, 尤其会话较多的情况下, 始终会对性能有一定影响
- 本文中涉及的 bug 在单个会话或多个会话低频的访问 `processlist` 表都不会触发 crash

3.4 MySQL: 唯一键约束失效

作者: 胡呈清

最近遇到一个故障: 单主模式 5 节点 MGR 集群, 在使用 `mysqlshell` 导入数据时, 所有 `secondary` 节点报错唯一键冲突退出集群。最终得到的结论是: `mysqlshell importTable()` 导入数据时会做 `SET unique_checks=0` 操作, 在满足一定条件时会导致唯一键约束失效, 主库插入了重复的数据, 但从库会插入失败从而报错。下面我们就来聊聊这是怎么回事。

`unique_checks=0` 的原理 (会导致唯一键约束失效吗?)

看看官方文档是怎么描述的:

If you have UNIQUE constraints on secondary keys, you can speed up table imports by temporarily turning off the uniqueness checks during the import session: `SET unique_checks=0;` ... SQL import statements ... `SET unique_checks=1;` For big tables, this saves a lot of disk I/O because InnoDB can use its change buffer to write secondary index records in a batch. Be certain that the data contains no duplicate keys.

对大数据量的导入, 无论是 `sql` 文件还是 `csv` 文件, 如果表中有唯一索引, 我们可以通过 `SET unique_checks=0;` 来加快导入速度。这里用到的是 `change buffer` 的原理: `change buffer` 的作用对象是普通二级索引, 当插入、修改、删除二级索引记录时, 即使数据不在 `innodb buffer pool` 中, 也不需要先把数据从磁盘读取到内存。只需要在 `change buffer` 中完成 DML 操作, 下次读取时才会从磁盘读取到内存, 并与 `change buffer` 进行 `merge`, 从而得到正确的数据。这个功能减少了 DML 时的随机 IO。

这里要注意对象是普通二级索引, 唯一索引为什么不行? 当执行 DML 时, 数据不在内存中, 为了保证唯一性, 必须先从磁盘读取数据, 这个随机 IO 是无法避免的, `change buffer` 失去了其作用, 因此对唯一索引无效。

但是 `unique_checks=0` 让 `change buffer` 重新对唯一索引有效了。文档还有句暧昧的描述: “Be certain that the data contains no duplicate keys.” (你需要自己确保数据不包含重复值)。这会让唯一键约束失效吗? 再让我们看看文档对于 `unique_checks` 系统变量的描述:

If set to 0, storage engines are permitted to assume that duplicate keys are not present in input data. If you know for certain that your data does not contain uniqueness violations, you can set this to 0 to speed up large table imports to InnoDB. Setting this variable to 0 does not require storage engines to ignore duplicate keys. An engine is still permitted to check for them and issue duplicate-key errors if it detects them.

至始至终没有出现“不进行唯一性检查”的字眼, 写文档的人真是狡猾。要搞清楚这个问题, 我们还得回到 `change buffer` 的原理: `change buffer` 无法支持唯一索引, 但是 `unique_checks=0` 让 `change buffer` 重新对唯一索引有效。结合文档两个描述, 我们作出如下假设:

- 如果 `set unique_checks=0;` DML 操作的数据, 如果主键和唯一索引在 `innodb buffer pool` 中, 则可以通过内存中的数据进行唯一性检测, 这也是我们在测试中都会遇到即使 `set unique_checks=0` 也不能插入重复数据;

- 但如果 DML 操作的数据如果不在 innodb buffer pool 中，则不会通过读取磁盘上的主键、唯一索引进行唯一检测，此时就和普通的二级索引一样，直接在 change buffer 完成数据写入，减少磁盘 IO。

验证

1. 用 sysbench 造点数据

```
sysbench /usr/local/share/sysbench/oltp_read_write.lua --mysql-host=127.0.0.1 \
--mysql-port=3309 --mysql-user=root --mysql-password=root \
--mysql-db=sbtest --tables=4 --table-size=1000000 --report-interval=3 --threads=8 --
time=300 prepare
```

2. 再创建一张表，并导入一些数据

```
CREATE TABLE `t` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `a` int(11) NOT NULL,
  `k` int(11) NOT NULL DEFAULT '0',
  `c` char(120) COLLATE utf8mb4_bin NOT NULL DEFAULT '',
  `pad` char(60) COLLATE utf8mb4_bin NOT NULL DEFAULT '',
  PRIMARY KEY (`id`),
  KEY `k_1` (`k`),
  unique key `a` (`a`)
);
insert into t(a,k,c,pad) select * from sbtest1 where id <= 5000;
```

3. 重启 mysqld

这一步目的是为了保证数据不在 innodb buffer pool 中，注意参数：

```
innodb_buffer_pool_size = 64M
innodb_buffer_pool_load_at_startup = 0
innodb_buffer_pool_dump_at_shutdown = 0
innodb_buffer_pool_dump_pct = 0
```

4. 插入重复数据

```
mysql> set session unique_checks=0;
Query OK, 0 rows affected (0.04 sec)

mysql> insert into t(a,k,c,pad) values(1,439502,'59488447928-69619891934-66520244141-
26804756589-33623161608-43445073345-17979281186-83118017446-98119924781-
27868548476','0000');
Query OK, 1 row affected (0.01 sec)
```

可以看到成功插入了重复数据（这里用 where a=1 等值查询只会输出一行结果，因此用 where a<2）：

```
mysql> select id,a from t where a<2;
+-----+-----+
```

3.4 MySQL: 唯一键约束失效

```
| id | a |
+----+---+
| 1 | 1 |
| 5022 | 1 |
| 5026 | 1 |
| 5035 | 1 |
+----+---+
4 rows in set (0.01 sec)
```

如果想观察的更清楚，可以持续运行 `sysbench run`，用 `sbtest*` 表把 `innodb buffer pool` 占满，这样可以反复插入同一行数据：

```
[root@localhost test4]# sysbench /usr/local/share/sysbench/oltp_read_write
lua -mysql-host=127.0.0.1 -mysql-port=3309 --mysql-user=root --mysql-pa
sword-root --mysql-db=test --tables=4 --table-size=1000000 --report-int
erval=3 --threads=8 --time=300 run
sysbench 1.0.17 (using bundled LuaJIT 2.1.0-beta2)

Running the test with following options:
Number of threads: 8
Report intermediate results every 3 second(s)
Initializing random number generator from current time

Initializing worker threads...

Threads started!

[ 3s ] thds: 8 tps: 313.41 qps: 6306.25 (r/w/o: 4417.77/1258.98/629.49) la
t (ms,95K): 44.98 err/s: 0.00 reconn/s: 0.00
[ 6s ] thds: 8 tps: 367.50 qps: 7334.33 (r/w/o: 5134.67/1465.00/734.67) la
t (ms,95K): 34.95 err/s: 0.00 reconn/s: 0.00
[ 9s ] thds: 8 tps: 343.26 qps: 6878.24 (r/w/o: 4815.00/1376.38/686.86) la
t (ms,95K): 37.56 err/s: 0.00 reconn/s: 0.00
[ 12s ] thds: 8 tps: 354.36 qps: 7076.54 (r/w/o: 4954.04/1413.77/708.72) l
at (ms,95K): 38.25 err/s: 0.00 reconn/s: 0.00
[ 15s ] thds: 8 tps: 377.39 qps: 7557.51 (r/w/o: 5291.49/1511.23/754.78) l
at (ms,95K): 33.12 err/s: 0.00 reconn/s: 0.00
AC

[root@localhost test4]#
[root@localhost test4]#
[root@localhost test4]#
[root@localhost test4]#
mysql> insert into t(a,k,c,pad) values(1,439502,'59488447928-69619891934-66520244141-26804756589-33623161608-434458
73345-17979281186-83118017446-98119924781-27868548476','0000');
Query OK, 1 row affected (0.01 sec)

mysql> insert into t(a,k,c,pad) values(1,439502,'59488447928-69619891934-66520244141-26804756589-33623161608-434458
73345-17979281186-83118017446-98119924781-27868548476','0000');
Query OK, 1 row affected (0.02 sec)

mysql> insert into t(a,k,c,pad) values(1,439502,'59488447928-69619891934-66520244141-26804756589-33623161608-434458
73345-17979281186-83118017446-98119924781-27868548476','0000');
Query OK, 1 row affected (0.01 sec)

mysql> insert into t(a,k,c,pad) values(1,439502,'59488447928-69619891934-66520244141-26804756589-33623161608-434458
73345-17979281186-83118017446-98119924781-27868548476','0000');
Query OK, 1 row affected (0.00 sec)

mysql> insert into t(a,k,c,pad) values(1,439502,'59488447928-69619891934-66520244141-26804756589-33623161608-434458
73345-17979281186-83118017446-98119924781-27868548476','0000');
Query OK, 1 row affected (0.00 sec)

mysql> insert into t(a,k,c,pad) values(1,439502,'59488447928-69619891934-66520244141-26804756589-33623161608-434458
73345-17979281186-83118017446-98119924781-27868548476','0000');
Query OK, 1 row affected (0.00 sec)

mysql> insert into t(a,k,c,pad) values(1,439502,'59488447928-69619891934-66520244141-26804756589-33623161608-434458
73345-17979281186-83118017446-98119924781-27868548476','0000');
Query OK, 1 row affected (0.00 sec)

mysql> insert into t(a,k,c,pad) values(1,439502,'59488447928-69619891934-66520244141-26804756589-33623161608-434458
73345-17979281186-83118017446-98119924781-27868548476','0000');
ERROR 1062 (23000): Duplicate entry '1' for key 't.a'

mysql> []
```

5. 一些干扰验证的现象

有些记录无法顺利插入进去，这个对我困扰非常大，一度让我以为推论是错误的...这个就留给官方解释吧：

```
[root@localhost test4]#
[root@localhost test4]#
[root@localhost test4]#
[root@localhost test4]#
[root@localhost test4]#
[root@localhost test4]# docker stop mysql4
mysql4
[root@localhost test4]# docker start mysql4
mysql4
[root@localhost test4]# docker stop mysql4
mysql4
[root@localhost test4]# docker start mysql4
mysql4
[root@localhost test4]# docker stop mysql4
mysql4
[root@localhost test4]# docker start mysql4
mysql4
[root@localhost test4]# docker stop mysql4
mysql4
[root@localhost test4]# docker start mysql4
mysql4
[root@localhost test4]# docker stop mysql4
mysql4
[root@localhost test4]# docker start mysql4
mysql4
[root@localhost test4]# docker stop mysql4
mysql4
[root@localhost test4]# docker start mysql4
mysql4
[root@localhost test4]# docker stop mysql4
mysql4
[root@localhost test4]# docker start mysql4
mysql4
[root@localhost test4]#
[root@localhost test4]#
[root@localhost test4]#
[root@localhost test4]#
[root@localhost test4]#
[root@localhost test4]#
```

```
mysql> set session unique_checks=0;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into t(a,k,c,pad) values(501,439502,'59488447928-69619891934-66520244141-2680475
6589-33623161608-43445073345-17979281186-83118017446-98119924781-27868548476','0000');
Query OK, 1 row affected (0.01 sec)

mysql> set session unique_checks=0;
ERROR 2013 (HY000): Lost connection to MySQL server during query
No connection. Trying to reconnect...
Connection id: 8
Current database: sbtest

Query OK, 0 rows affected (0.05 sec)

mysql> insert into t(a,k,c,pad) values(1001,439502,'59488447928-69619891934-66520244141-268047
56589-33623161608-43445073345-17979281186-83118017446-98119924781-27868548476','0000');
ERROR 1062 (23000): Duplicate entry '1001' for key 't.a'
mysql> set session unique_checks=0;
ERROR 2013 (HY000): Lost connection to MySQL server during query
No connection. Trying to reconnect...
Connection id: 8
Current database: sbtest

Query OK, 0 rows affected (0.03 sec)

mysql> insert into t(a,k,c,pad) values(1001,439502,'59488447928-69619891934-66520244141-268047
56589-33623161608-43445073345-17979281186-83118017446-98119924781-27868548476','0000');
ERROR 1062 (23000): Duplicate entry '1001' for key 't.a'
mysql> set session unique_checks=0;
ERROR 2013 (HY000): Lost connection to MySQL server during query
No connection. Trying to reconnect...
Connection id: 8
Current database: sbtest

Query OK, 0 rows affected (0.04 sec)

mysql> insert into t(a,k,c,pad) values(501,439502,'59488447928-69619891934-66520244141-2680475
6589-33623161608-43445073345-17979281186-83118017446-98119924781-27868548476','0000');
Query OK, 1 row affected (0.01 sec)

mysql>
```

如果是 `char` 类型的唯一键，则不会有类似问题：

```
CREATE TABLE `t1` (
  `id` int NOT NULL AUTO_INCREMENT,
  `a` char(10) COLLATE utf8mb4_bin DEFAULT NULL,
  `k` int NOT NULL DEFAULT '0',
  `c` char(120) CHARACTER SET utf8mb4 COLLATE utf8mb4_bin NOT NULL DEFAULT '',
  `pad` char(60) CHARACTER SET utf8mb4 COLLATE utf8mb4_bin NOT NULL DEFAULT '',
  PRIMARY KEY (`id`),
  UNIQUE KEY `a` (`a`),
  KEY `k_1` (`k`)
) ENGINE=InnoDB;
```

结论

如果 `set unique_checks=0`; DML 操作的数据，如果主键和唯一索引在 `innodb buffer pool` 中，则可以通过内存中的数据进行唯一性检测，这也是我们在测试中通过都会遇到即使 `set unique_checks=0` 也不能插入重复数据；

但如果 DML 操作的数据不在 `innodb buffer pool` 中，则不会通过读取磁盘上的主键、唯一索引进行唯一检测，此时就和普通的二级索引一样，直接在 `change buffer` 完成数据写入，减少磁盘 IO。

需要注意的是 `mysqldump` 导出的 `sql` 文件头部会自动携带 `set unique_checks=0` 信息，包括开头我们提到的 `mysqlshell importTable()` 也会做同样的设置。

对此我提了个 bug: <https://bugs.mysql.com/bug.php?id=106121>

虽然文档很狡猾的写了：“Be certain that the data contains no duplicate keys.”，我们也知道了 `set unique_checks=0` 的原理就是通过 `change buffer` 加速唯一键的插入，这似乎就不能完全保证唯一性。如果这确实不算 bug，也希望官方能把文档写的更清楚明白些，而不是描述的暧昧不清。

解决方案

如果不能保证导入的数据唯一，在 `mysqlshell importTable()` 导入前关闭 `change buffer`: `set global innodb_change_buffering=none;`，如果是导入 `sql` 文件，则注意将 `set unique_checks=0` 注释。

3.5 MySQL：从机故障重启后主从同步报错案例分析

作者：付祥

一. 环境说明

1. MySQL 版本

```
root@3306 (none)> SELECT @@VERSION;
+-----+
| @@VERSION |
+-----+
| 5.7.34-log |
+-----+
1 row in set (0.01 sec)
```

2. binlog 相关参数

```
root@3306 (none)> select @@log_bin,@@log_slave_updates;
+-----+-----+
| @@log_bin | @@log_slave_updates |
+-----+-----+
| 1 | 1 |
+-----+-----+
1 row in set (0.00 sec)
root@3306 (none)>
```

3. GTID 相关参数

```
root@3306 (none)> select
@@binlog_gtid_simple_recovery,@@enforce_gtid_consistency,@@gtid_mode;
+-----+-----+-----+
| @@binlog_gtid_simple_recovery | @@enforce_gtid_consistency | @@gtid_mode |
+-----+-----+-----+
| 1 | ON | ON |
+-----+-----+-----+
1 row in set (0.01 sec)
root@3306 (none)>
```

4. 半同步相关参数

```
root@3306 (none)> show global variables like '%semi%';
```



```

+-----+-----+
| Variable_name | Value |
+-----+-----+
| rpl_semi_sync_master_enabled | ON |
| rpl_semi_sync_master_timeout | 1000 |
| rpl_semi_sync_master_trace_level | 32 |
| rpl_semi_sync_master_wait_for_slave_count | 1 |
| rpl_semi_sync_master_wait_no_slave | ON |
| rpl_semi_sync_master_wait_point | AFTER_SYNC |
| rpl_semi_sync_slave_enabled | ON |
| rpl_semi_sync_slave_trace_level | 32 |
+-----+-----+
8 rows in set (0.00 sec)
root@3306 (none)>

```

5. 多线程同步相关参数

```

root@3306 (none)> select
@@binlog_transaction_dependency_tracking,@@slave_parallel_type,@@slave_parallel_
workers;
+-----+-----+-----+
-----+
| @@binlog_transaction_dependency_tracking | @@slave_parallel_type |
@@slave_parallel_workers |
+-----+-----+-----+
-----+
| COMMIT_ORDER | LOGICAL_CLOCK |
2 |
+-----+-----+-----+
-----+
1 row in set (0.00 sec)
root@3306 (none)>

```

二、故障现象

MySQL 从库所在主机故障重启后，sql_thread 线程报错：

```

root@3306 (none)> show slave status\G
-- 摘取有用信息如下：
Slave_IO_Running: Yes
Slave_SQL_Running: No
Replicate_Do_DB:
Replicate_Ignore_DB:
Replicate_Do_Table:
Replicate_Ignore_Table:
Replicate_Wild_Do_Table:

```

```

Replicate_Wild_Ignore_Table:
Last_Errno: 1062
Last_Error: Coordinator stopped because there were
error(s) in the worker(s). The most recent failure being: Worker 1 failed
executing transaction '471c2974-f9bb-11eb-afb1-52540010fb89:88313207' at master
log MySQL-bin.000685, end_log_pos 1004756557. See error log and/or
performance_schema.replication_applier_status_by_worker table for more details
about this failure or others, if any.
.....省略.....
Retrieved_Gtid_Set: 471c2974-f9bb-11eb-afb1-
52540010fb89:88313207-88341912
Executed_Gtid_Set: 471c2974-f9bb-11eb-afb1-52540010fb89:1-
88313206,
d4c228df-f9c6-11eb-a2d8-5254006f63b6:1-5
Auto_Position: 1
root@3306 (none)> select * from
performance_schema.replication_applier_status_by_worker\G
***** 1. row *****
CHANNEL_NAME:
WORKER_ID: 1
THREAD_ID: NULL
SERVICE_STATE: OFF
LAST_SEEN_TRANSACTION: 471c2974-f9bb-11eb-afb1-52540010fb89:88313207
LAST_ERROR_NUMBER: 1062
LAST_ERROR_MESSAGE: Worker 1 failed executing transaction '471c2974-f9bb-
11eb-afb1-52540010fb89:88313207' at master log MySQL-bin.000685, end_log_pos
1004756557; Could not execute Write_rows event on table kefumobile.i_sms_proxy;
Duplicate entry '14765130' for key 'PRIMARY', Error_code: 1062; handler error
HA_ERR_FOUND_DUPP_KEY; the event's master log FIRST, end_log_pos 1004756557
LAST_ERROR_TIMESTAMP: 2022-01-24 23:05:02
***** 2. row *****
CHANNEL_NAME:
WORKER_ID: 2
THREAD_ID: NULL
SERVICE_STATE: OFF
LAST_SEEN_TRANSACTION:
LAST_ERROR_NUMBER: 0
LAST_ERROR_MESSAGE:
LAST_ERROR_TIMESTAMP: 0000-00-00 00:00:00
2 rows in set (0.00 sec)

```

通过报错信息可知，worker 线程在回放事务'471c2974-f9bb-11eb-afb1-52540010fb89:88313207'时，由于要插入的记录主键冲突报错。

三、故障分析

主机重启前，主从同步正常，主机重启后，主从同步由于主键冲突报错，对比了冲突主键所在行记录在主从库是一致的，初步分析事务'471c2974-f9bb-11eb-afb1-52540010fb89:88313207'在主机故障前已经在从库进行了回放，那为何事务会重复回放呢？

在开启 GTID 模式下，如果指定 master_auto_position=1，start slave 时，从库会把 Retrieved_Gtid_Set 和 Executed_Gtid_Set 的并集发送给主库，主库将收到的并集和自己的 gtid_executed 比较，把从库 GTID 集合里缺失的事务全都发送给从库。

主机重启后，事务重复回放，表明 Retrieved_Gtid_Set 和 Executed_Gtid_Set 的并集中有 GTID 事务丢失，导致重复获取事务执行引发主键冲突错误。Retrieved_Gtid_Set 和 Executed_Gtid_Set 均为内存变量，MySQL 重启后，Retrieved_Gtid_Set 初始化为空值，从而推断出 Executed_Gtid_Set 有 GTID 事务丢失。

Executed_Gtid_Set 来源于 gtid_executed 变量，gtid_executed 变量持久化介质有 mysql.gtid_executed 表和 binlog 日志，其中 mysql.gtid_executed 表是 MySQL 5.7 后引入的，在 MySQL 5.6 中，从库要使用 GTID，必须要先设置 log_bin=on,log_slave_updates=on，因为从库执行过的 GTID 只保留在 binlog 中。

1. 当 log_bin=on，log_slave_updates=off 时，gtid_executed 变量的更新实时持久化到 mysql.gtid_executed 表中，MySQL 重启后 gtid_executed 变量值从 mysql.gtid_executed 表读取。
2. 当 log_bin=on，log_slave_updates=on 时，只有在 binlog 切换时候才会更新 mysql.gtid_executed 表，保存直到上一个 binlog 执行过的 GTID 集合。MySQL 重启后，在默认参数 binlog_gtid_simple_recovery=1 时，gtid_executed 变量值从最后一个 binlog 文件计算获得。

gtid_executed 变量值陈旧，推断出 binlog 未实时持久化，我们看一下参数 sync_binlog：

```
root@3306 (none)> show variables like 'sync_binlog';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| sync_binlog   | 600   |
+-----+-----+
1 row in set (0.00 sec)
```

通过以上分析，此次故障来龙去脉就清楚了：Worker 线程报 1062 主键冲突错误--> gtid_executed 信息陈旧--> binlog 未实时持久化

四、测试验证

搭建一主一从测试环境，通过 sysbench 模拟主库并发插入，从库主机暴力关机后，故障复现：

```
root@mysql.sock[(none)]> select * from
performance_schema.replication_applier_status_by_worker\G
```

```
***** 1. row *****
CHANNEL_NAME:
WORKER_ID: 1
THREAD_ID: NULL
SERVICE_STATE: OFF
LAST_SEEN_TRANSACTION: 4a0ad3da-b89e-11eb-9d0b-000c299b4d6c:452362
LAST_ERROR_NUMBER: 1062
LAST_ERROR_MESSAGE: Worker 1 failed executing transaction '4a0ad3da-b89e-11eb-9d0b-000c299b4d6c:452362' at master log MySql-bin.000012, end_log_pos 1011339749; Could not execute Write_rows event on table sbtest.sbtest5; Duplicate entry '111305' for key 'PRIMARY', Error_code: 1062; handler error HA_ERR_FOUND_DUPP_KEY; the event's master log FIRST, end_log_pos 1011339749
LAST_ERROR_TIMESTAMP: 2022-01-26 09:56:38
***** 2. row *****
CHANNEL_NAME:
WORKER_ID: 2
THREAD_ID: NULL
SERVICE_STATE: OFF
LAST_SEEN_TRANSACTION:
LAST_ERROR_NUMBER: 0
LAST_ERROR_MESSAGE:
LAST_ERROR_TIMESTAMP: 0000-00-00 00:00:00
2 rows in set (0.00 sec)
[root@mysql.sock][(none)]>
```

五、故障处理

既然错误原因是事务重复执行，那跳过错误就好了，有如下两种方式，根据需要选取其中一种方式执行：

1. 通过空事务替代报错事务执行

```
set gtid_next='xxxxxx';
begin;
commit;
set gtid_next=AUTOMATIC;
start slave sql_thread
```

如果最新 binlog 丢失的 GTID 较多，手工执行比较繁琐，需要不断试错。可写一个存储过程批量执行：

```
set sql_log_bin=0;
DELIMITER $$
create procedure p_fx_gtid_next(i_master_Executed_Gtid_max varchar(100))
begin
declare v_gtid_next varchar(100);
```

```

declare master_Executed_Gtid_max varchar(100);
declare slave_Executed_Gtid_max varchar(100);
-- 主库当前执行了的 gtid 最大值, 做为退出循环条件 show master status
set master_Executed_Gtid_max=i_master_Executed_Gtid_max;
loop_name:loop
SELECT ifnull(min(LAST_SEEN_TRANSACTION),'empty') FROM
performance_schema.replication_applier_status_by_worker WHERE
LAST_ERROR_NUMBER=1062
into v_gtid_next;
if v_gtid_next <> 'empty' then
set gtid_next = v_gtid_next;
start transaction;
commit;
set gtid_next =AUTOMATIC;
start slave sql_thread;
end if;
select max(LAST_SEEN_TRANSACTION) from
performance_schema.replication_applier_status_by_worker into
slave_Executed_Gtid_max;
if slave_Executed_Gtid_max = master_Executed_Gtid_max then
leave loop_name;
end if;
select SLEEP(1);
end loop;
end $$
DELIMITER ;
set sql_log_bin=1;
call p_fx_gtid_next('XXXXX:XXX');

```

2. 带参数 slave_skip_errors=1062 重启 MySQL

待主从同步正常后, 再取消参数 slave_skip_errors 设置重启 MySQL 。

3.6 Federated 存储引擎表导致监控线程处于 Opening

作者：李鹏博

最近发现客户的一台 MySQL 5.7.32 实例的监控线程状态一直处于 Opening table 状态，且都是在对 information_schema.tables 表做相关查询，如图：

```
127.0.0.1:55004 | NULL | Query | 164 | Opening tables | SELECT TABLE_SCHEMA, TABLE_NAME, TABLE_ROWS, DATA_LENGTH, INDEX_LENGTH, DATA_F
127.0.0.1:55008 | NULL | Query | 193 | Opening tables | SELECT TABLE_SCHEMA, TABLE_NAME, TABLE_ROWS, DATA_LENGTH, INDEX_LENGTH, DATA_F
127.0.0.1:55010 | NULL | Query | 223 | Opening tables | SELECT TABLE_SCHEMA, TABLE_NAME, TABLE_ROWS, DATA_LENGTH, INDEX_LENGTH, DATA_F
127.0.0.1:55012 | NULL | Query | 209 | Opening tables | SELECT TABLE_SCHEMA, TABLE_NAME, TABLE_ROWS, DATA_LENGTH, INDEX_LENGTH, DATA_F
127.0.0.1:55016 | NULL | Query | 134 | Opening tables | SELECT TABLE_SCHEMA, TABLE_NAME, TABLE_ROWS, DATA_LENGTH, INDEX_LENGTH, DATA_F
127.0.0.1:55020 | NULL | Sleep | 0 | | NULL
127.0.0.1:55042 | NULL | Query | 178 | Opening tables | SELECT TABLE_SCHEMA, TABLE_NAME, TABLE_ROWS, DATA_LENGTH, INDEX_LENGTH, DATA_F
127.0.0.1:55044 | NULL | Query | 148 | Opening tables | SELECT TABLE_SCHEMA, TABLE_NAME, TABLE_ROWS, DATA_LENGTH, INDEX_LENGTH, DATA_F
127.0.0.1:55052 | NULL | Query | 119 | Opening tables | SELECT TABLE_SCHEMA, TABLE_NAME, TABLE_ROWS, DATA_LENGTH, INDEX_LENGTH, DATA_F
```

通过 show open tables ;语句发现 opened tables 并不算太多：



```
488 rows in set (0.00 sec)
```

相关参数也没有太大的不合理性：

```
mysql> show variables like 'table_open_cache';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| table_open_cache | 2000 |
+-----+-----+
1 row in set (0.01 sec)
```

虽然 ulimit 设置不是很大，但是也不会对此产生什么影响

```
[root@ ~]# ulimit -n
1024
```

查看 MySQL Error 日志也没有发现与此相关的异常。

因此只能使用 pstack 工具对 MySQL 打堆栈来进行分析，堆栈日志如下：

```

Thread 42 (Thread 0x7f32cac73700 (LWP 6502)):
#0 0x00007f321f2d9ed in connect () from /lib64/libpthread.so.0
#1 0x00000000014622fe in inline_mysql_socket_connect (len=16, addr=0x7f32a0065a70, mysql_socket=..., src_line=940, src_file=0x180c478 "/var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/include/mysql/ps/mysql_socket.h:707") at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/include/mysql/ps/mysql_socket.h:707
#2 vio_socket_connect (vio=0x7f32a10baea0, addr=0x7f32a0065a70, len=16, timeout=-1) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/vio/viosocket.cc:940
#3 0x0000000000e234dd in mysql_real_connect (mysql=0x7f32a2021d00, host=0x7f328119e069 "10.10.10.10", user=0x7f328119e058 "root", passwd=0x7f328119e05e "123456", db=0x7f328119e07c "skssp", port=3306, unix_socket=0x0, client_flag=0) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/sql-common/client.cc:4384
#4 0x000000000010d689 in ha_federated::real_connect (this=0x7f32a2050530) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/storage/federated/ha_federated.cc:3217
#5 0x0000000000110d87d in ha_federated::real_query (this=0x7f32a2050530, query=0x0, length=0, length=0, length=0) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/storage/federated/ha_federated.cc:3274
#6 0x0000000000110e2f2 in ha_federated::info (this=0x7f32a2050530, flag=212) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/storage/federated/ha_federated.cc:288
#7 0x00000000000da4961 in get_schema_tables_reord (thd=0x7f32a12a14f0, tables=0x7f32a2134348, table=0x7f32a00564a0, res=0x0, db_name=0x0, table_name=0x0, table_schema=0x0) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/sql/sql_show.cc:5112
#8 0x00000000000d9e0fa in fill_schema_table_by_open (thd=0x7f32a12a14f0, mem_root=0x0, is_show_fields_or_keys=false, table=0x7f32a00564a0, schema_table=0x1ffe800 <schema_table>) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/sql/sql_show.cc:4783
#9 0x00000000000dac25b in get_all_tables (thd=0x7f32a12a14f0, tables=0x7f32a1254020, cond=0x0) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/sql/sql_show.cc:4783
#10 0x00000000000d99bb8 in do_fill_tables (thd=0x0, table_list=0x7f32a1254020, gcp_tab=0x7f32a12bb9c8) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/sql/sql_show.cc:4783
#11 0x00000000000d99e19 in get_schema_tables_result (join=0x7f32a12bb490, executed_place=PROCESSED_BY_JOIN_EXEC) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/sql/sql_show.cc:4783
#12 0x00000000000d8f3dd in JOIN::prepare_result (this=0x7f32a12bb490) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/sql/sql_select.cc:915
#13 0x00000000000d251d0 in JOIN::exec (this=0x7f32a12bb490) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/sql/sql_executor.cc:131
#14 0x00000000000d907a0 in handle_query (thd=0x7f32a12a14f0, lex=0x7f32a123660, result=0x7f32a1255e08, added_options=1, removed_options=0) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/sql/sql_executor.cc:131
#15 0x00000000000d50a73 in execute_sqlcom_select (thd=0x7f32a12a14f0, all_tables=0x0) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/sql/sql_parse.cc:5155
#16 0x00000000000d452a in mysql_execute_command (thd=0x7f32a12a14f0, first_level=true) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/sql/sql_parse.cc:2826
#17 0x00000000000d5609d in mysql_parse (thd=0x7f32a12a14f0, parser_state=0x0) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/sql/sql_parse.cc:5584
#18 0x00000000000d57892 in dispatch_command (thd=0x7f32a12a14f0, com_data=0x7f32cac72de0, command=COM_QUERY) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/sql/sql_parse.cc:1491
#19 0x00000000000d58464 in do_command (thd=0x7f32a12a14f0) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/sql/sql_parse.cc:1032
#20 0x00000000000e2b80e in handle_connection (arg=0x0) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/sql/conn_handler/connection_handler_per_thread.cc:313
#21 0x000000000013fb594 in pfs_spawn_thread (arg=0x5f970e0) at /var/lib/pb2/sb_1-409304-1600863701.97/mysql-5.7.32/storage/perfschema/pfs.cc:2197
#22 0x00007f321f26e25 in start_thread () from /lib64/libpthread.so.0
#23 0x00007f323209e334d in clone () from /lib64/libc.so.6

```

通过分析堆栈日志发现，问题出现在进行查询时会使用 Federated 存储引擎表对远程实例进行查询。

查看数据库使用 Federated 存储引擎的表，发现有两张表使用了 Federated 存储引擎：

```

mysql> select table_name,table_schema,engine from information_schema.tables where engine='federated';
+-----+-----+-----+
| table_name | table_schema | engine |
+-----+-----+-----+
| [redacted] | [redacted] | FEDERATED |
| [redacted] | [redacted] | FEDERATED |
+-----+-----+-----+
2 rows in set (0.17 sec)

mysql> show create table [redacted] \G;
*****
Table [redacted]
CREATE TABLE [redacted] (
  [redacted] [redacted]
) ENGINE=FEDERATED DEFAULT CHARSET=utf8 CONNECTION='mysql://[redacted]:3306/[redacted]'
1 row in set (0.00 sec)

```

通过在实例服务器上 Telnet Federated 服务端的实例 IP 和端口发现是不通的：

```

[redacted]# telnet [redacted] 3306
Trying [redacted]...
AC

```

所以推测问题原因为：监控线程在查询 information_schema.tables 表时，当需要获取 Federated 存储引擎表的信息时需要连接远端 Server，而由于网络或其他原因无法连接时，就会导致本地监控线程处于 Opening table 状态。

接下来设计实验验证我们的想法：

1. 启用 Federated 存储引擎

```

mysql> show engines;
+-----+-----+-----+-----+-----+
| Engine | Support | Comment | Transactions | XA | Savepoints |
+-----+-----+-----+-----+-----+
| InnoDB | DEFAULT | Supports transactions, row-level locking, and foreign keys | YES | YES | YES |
| MRG_MYISAM | YES | Collection of identical MyISAM tables | NO | NO | NO |
| MEMORY | YES | Hash based, stored in memory, useful for temporary tables | NO | NO | NO |
| BLACKHOLE | YES | /dev/null storage engine (anything you write to it disappears) | NO | NO | NO |
| MyISAM | YES | MyISAM storage engine | NO | NO | NO |
| CSV | YES | CSV storage engine | NO | NO | NO |
| ARCHIVE | YES | Archive storage engine | NO | NO | NO |
| PERFORMANCE_SCHEMA | YES | Performance Schema | NO | NO | NO |
| FEDERATED | YES | Federated MySQL storage engine | NO | NO | NO |
+-----+-----+-----+-----+-----+
9 rows in set (0.00 sec)

```

2. 创建一张 Federated 存储引擎的表，连接的 server 不存在


```
mysql> create database action;
Query OK, 1 row affected (0.00 sec)

mysql> create table action.t(id int primary key,name varchar(10))engine=federated CONNECTION='mysql://action:pass@10.10.10.10:3306/db/table';
Query OK, 0 rows affected (0.05 sec)

mysql> show create table action.t\G;
***** 1. row *****
Table: t
Create Table: CREATE TABLE `t` (
  `id` int(11) NOT NULL,
  `name` varchar(10) COLLATE utf8mb4_bin DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=FEDERATED DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_bin CONNECTION='mysql://action:pass@10.10.10.10:3306/db/table'
1 row in set (0.01 sec)
```

3. 查询 information_schema.tables 表，线程卡住

```
mysql> select * from information_schema.tables;
```

4. 线程状态处于 Opening table 状态

```
mysql> show processlist;
+-----+-----+-----+-----+-----+-----+-----+-----+
| Id | User | Host | db | Command | Time | State | Info |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 2 | root | localhost | NULL | Query | 29 | Opening tables | select * from information_schema.tables |
| 3 | root | localhost | NULL | Query | 0 | starting | show processlist |
+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

这刚好验证了我们的想法是正确的。

3.7 MySQL 5.7 使用临时表导致数据库 Crash

作者：雷文霆

背景信息

在 MySQL5.7.30 主从读写分离环境下，从库在某天出现了 MySQL crash.

系统侧：监控显示该从库主机的内存和 CPU 资源使用率在故障前后均正常，磁盘 IO 有 2%的 iowait（读写 200M/s），说明故障前磁盘存在压力。

服务侧：slow-log 中记录了服务重启前，存在使用了临时表和文件排序的慢 SQL 语句。

Error-log 中记录了服务调用到 btr0btr.cc 文件 的 L2165 行，出现了 err == DB_SUCCESS 报错。

```
0x7f2dd49d0700 InnoDB: Assertion failure in thread 139834817316608 in file btr0btr.cc
line 2165
228
```



```
InnoDB: Failing assertion: err == DB_SUCCESS
InnoDB: We intentionally generate a memory trap.
InnoDB: Submit a detailed bug report to http://bugs.mysql.com.
```

故障分析

在系统侧排除了磁盘空间不足和内存不足的因素，服务侧怀疑是慢查询和 BUG 的原因，之后通过 "btr0btr.cc" 关键字查找到了一个类似的 BUG。链接如下：

<https://bugs.mysql.com/bug.php?id=101154>

报告的意思是，MySQL 在执行 `btr_insert_on_non_leaf_level_func()` 函数时，写入临时表会导致带有断言的服务崩溃。

通过查看 `btr0btr.cc` 文件开头的注释了解到的原因是：此文件的调用机制是：对 b 树行操作或记录所做的所有更改。

L2165 行操作内容是：在处理插入到非叶级别的内容时，会检查每个级别的可用空间(需保留 2 倍索引数高度的页空间)，如果在操作之前，叶分裂已经开始，就很难撤销，只能通过崩溃进行前滚。该 BUG 只会在 MySQL5.7 出现

代码查询：<https://github.com/mysql/mysql-server> (通过 Tags 标签选择对应版本)

代码内容：<https://github.com/mysql/mysql-server/blob/mysql-5.7.30/storage/innobase/btr/btr0btr.cc#L2165>

```
inserts a data tuple to a tree on a non-leaf level. It is assumed
that mtr holds an x-latch on the tree. */
void
btr_insert_on_non_leaf_level_func()
    ---
    ut_a(err == DB_SUCCESS);
    ---
```

其中 `ut_a()`

https://dev.mysql.com/doc/dev/mysql-server/latest/ut0dbg_8h.html#ae7aed983dfe98ac872b5a9915fa778fa:



检查数据库关于临时表的参数：

`innodb_temp_data_file_path ibtmp1:12M:autoextend:max:20G`

```
tmp_table_size 64M 和 max_heap_table_size 64M
```

注释：实际限制取二者中的较小者。会话级别的参数，对于 `innodb_buffer_pool_size` 不大且没有用到临时大数据量查询的情况，不建议设置的过大，可能会导致内存溢出的情况。连接数 800+，64M 为推荐值

```
internal_tmp_disk_storage_engine InnoDB
```

注释：用于定义磁盘临时表的存储引擎。超出 InnoDB 行或列限制的磁盘内部临时表的查询会返回 `Row size too large` 或 `Too many columns` 错误。解决方法是设置 `internal_tmp_disk_storage_engine` 为 `MYISAM`，我们的 `error-log` 中无相关报错。初步排查阶段不建议修改

```
created_tmp_disk_tables 2987733
```

```
created_tmp_tables 11049848
```

注释：当在内存或磁盘上创建内部临时表时，服务器会增加该 `Created_tmp_tables` 值。在磁盘上创建内部临时表时，服务器会增加该 `Created_tmp_disk_tables` 值。如果在磁盘上创建了太多内部临时表，请考虑增加 `tmp_table_size` 和 `max_heap_table_size` 设置。从早上 10 点 36 分到 17 点产生较多临时表，结合业务繁忙情况，属于正常现象

小结：通过上面的分析，结合应用架构(无法升级到 MySQL8.0)。初步阶段是建议先优化 SQL 语句，减少对临时表的使用，降低再次发生的概率。

问题复现

上文提到的 BUG 报告中，有个使用 MySQL Test Run 简称 MTR(MySQL 官方的自动化测试框架) 的测试用例。

下文将引用此测试用例，进行复现测试。

参数解释：

`innodb_limit_optimistic_insert_debug` 参数可以限制每个 B 树页面的记录数，在 MySQL 运行过程中动态设置此参数可以导致页分裂。

```
innodb_limit_optimistic_insert_debug
```

限制每个 B 树页面的记录数。默认值 0 表示不施加限制。仅当使用 CMake 选项编译调试支持时，需开启 `DEBUG` 选项。

```
# 依赖
```

```
yum install -y gcc gcc-c++ cmake ncurses ncurses-devel bison openssl openssl-devel
tar -xvf mysql-boost-5.7.30.tar.gz
```

```
--编译安装 MySQL，因为需要设置 innodb_limit_optimistic_insert_debug 参数-----
```

```
tar -xvf mysql-boost-5.7.30.tar.gz
```

```
# 非 BOOST 版本的 Mysql 源码包，需要指定 -DDOWNLOAD_BOOST=1 -DWITH_BOOST=/usr/local/boost
cd mysql-5.7.30
```

```
cmake . -DCMAKE_INSTALL_PREFIX=/tools/mysql-test5.7.30 -DMYSQL_DATADIR=/tools/mysql-test5.7.30/data -DMYSQL_UNIX_ADDR=/tools/mysql-test5.7.30/mysql5.7.30.sock -
```

```

DDEFAULT_CHARSET=utf8mb4 -DDEFAULT_COLLATION=utf8mb4_general_ci -DEXTRA_CHARSETS=all -
DENABLED_LOCAL_INFILE=1 -DWITH_SSL=system -DWITH_BOOST=boost -DWITH_DEBUG=1

make
make install

# Cmake 编译之后会在 DCMAKE_INSTALL_PREFIX 目录中生成 mysql-test 测试框架目录
/tools/mysql-test5.7.30/mysql-test/t
vim my0420.test
cat my0420.test
--source include/have_innodb.inc

--let $restart_parameters = "restart: --
innodb_temp_data_file_path=ibtmp1:12M:autoextend:max:1G --big-tables=1"
--source include/restart_mysql_d.inc

SELECT @@innodb_temp_data_file_path;

drop function if exists func1;
delimiter |;
create function func1(x int) returns int deterministic
begin
    declare z1, z2 int;
    set z1 = x;
    set z2 = z1 + 2;
    return z2;
end|
delimiter ;|

create table t1 (a int, b varchar(20));
insert into t1 values(1, 'a'), (2, 'b'), (3, 'c'), (3, 'c'), (4, 'c');

SET GLOBAL innodb_limit_optimistic_insert_debug=4;
--let $i=1
while ($i <= 15) {
INSERT INTO t1 SELECT * FROM t1;
--inc $i
}

SELECT COUNT(*) FROM t1;

SET GLOBAL innodb_limit_optimistic_insert_debug=2;
select * from t1 order by func1(a);

```

进行测试，

```
/tools/mysql-test5.7.30/mysql-test
./mtr my0420.test
```

其中 `select * from t1 order by func1(a);` 会使用 `Using temporary; Using filesort` 和 业务 SQL 的执行计划一致

将 `my0420.test` 第二行新增`--internal_tmp_disk_storage_engine=MYISAM` 参数后，服务不崩溃。

```
--let $restart_parameters = "restart: --innodb_temp_data_file_path=ibtmp1:12M:autoextend:max:1G --
internal_tmp_disk_storage_engine=MYISAM --big-tables=1"
```

将 `my0420.test` 第二行增大为`---innodb_temp_data_file_path=ibtmp1:12M:autoextend:max:6G` 参数后，服务不崩溃。

```
--let $restart_parameters = "restart: --innodb_temp_data_file_path=ibtmp1:12M:autoextend:max:6G --big-
tables=1"
```

big_tables 如果启用，服务器将所有临时表存储在磁盘上而不是内存中。这可以防止需要大型临时表的操作的大多数错误，但也会减慢内存表就足够的查询。most **The table *tbl_name* is full errors** for **SELECT** operations，如果 `error-log` 中出现此报错，说明 `select` 操作使用了大的磁盘临时表，不推荐启用。

(小提示，客户环境中时常会收到某张临时表 `#sql_tbl_name is full` 的告警邮件，需要考虑是否可以优化 SQL 了)

测试日志

MTR 的执行逻辑为启动一个临时 MySQL 服务，并执行 `t` 目录中 `my0420.test` 文件的内容，执行结果默认会和 `r` 目录中的 `result` 同名文件(也叫标准执行结果文件，一般会在正确的版本中生成)进行对比，用于判断测试文件是否正确。

在 `mysql-test` 目录下：

```
./mtr my0420.test
```

--执行到以下语句时报错--

```
SET GLOBAL innodb_limit_optimistic_insert_debug=2;
```

```
[100%] main.my0420 [ fail ]
```

```
Test ended at 2022-04-20 20:05:39
```

```
CURRENT_TEST: main.my0420
```

```
mysqltest: At line 32: query 'select * from t1 order by func1(a)' failed: 2013: Lost co
nnection to MySQL server during query
```

```
safe_process[7080]: Child process: 7081, exit: 1
```

```
Server [mysqld.1 - pid: 7089, winpid: 7089, exit: 256] failed during test run
```

Server log from this test:

Lost connection , MySQL 服务已停止。之后打印了 error 信息。错误同样出现在 btr0btr.cc line 2165

Server log from this test:

```
-----SERVER LOG START-----
2022-04-20T12:02:42.082135Z 0 [Note] /tools/mysql-test5.7.30/bin/mysqld (mysqld 5.7.30-
debug-log) starting as process 7049 ...
2022-04-20T12:02:43.698812Z 0 [Note] /tools/mysql-
test5.7.30/bin/mysqld: Shutdown complete
2022-04-20T12:02:45.051667Z 0 [Note] /tools/mysql-test5.7.30/bin/mysqld (mysqld 5.7.30-
debug-log) starting as process 7090 ...
2022-04-20T12:02:45.262573Z 0 [Note] /tools/mysql-
test5.7.30/bin/mysqld: ready for connections.
Version: '5.7.30-debug-log' socket: '/tools/mysql-test5.7.30/mysql-
test/var/tmp/mysqld.1.sock' port: 13000 Source distribution
2022-04-
20 15:05:37 0x7fc298bc4700 InnoDB: Assertion failure in thread 140473762858752 in file
btr0btr.cc line 2165
InnoDB: Failing assertion: err == DB_SUCCESS
InnoDB: We intentionally generate a memory trap.
InnoDB: Submit a detailed bug report to http://bugs.mysql.com.
InnoDB: If you get repeated assertion failures or crashes, even
InnoDB: immediately after the mysqld startup, there may be
InnoDB: corruption in the InnoDB tablespace. Please refer to
InnoDB: http://dev.mysql.com/doc/refman/5.7/en/forcing-innodb-recovery.html
InnoDB: about forcing recovery.
12:05:37 UTC - mysqld got signal 6 ;
This could be because you hit a bug. It is also possible that this binary
or one of the libraries it was linked against is corrupt, improperly built,
or misconfigured. This error can also be caused by malfunctioning hardware.
Attempting to collect some information that could help diagnose the problem.
As this is a crash and something is definitely wrong, the information
collection process might fail.

key_buffer_size=1048576
read_buffer_size=131072
max_used_connections=1
max_threads=151
thread_count=1
connection_count=1
It is possible that mysqld could use up to
key_buffer_size + (read_buffer_size + sort_buffer_size)*max_threads = 61093 K bytes of
memory
Hope that's ok; if not, decrease some variables in the equation.
```

```
Trying to get some variables.
Some pointers may be invalid and cause the dump to abort.
Query (7fc234005890): select * from t1 order by func1(a)
Connection ID (thread ID): 2
Status: NOT_KILLED
```

和客户环境的区别之处:

生产中的报错为 Query(7f3be00479d0) is an invalid pointer. 无效指针, 类似磁盘空间不足的报错.

测试中的报错为 Query (7fc234005890): select * from t1 order by func1(a).

测试环境的堆栈信息:

<https://github.com/mysql/mysql-server/blob/mysql-5.7.30/storage/innobase/btr/btr0btr.cc#L2285>

```
2270      /* Get the level of the split pages */
2271      level = btr_page_get_level(buf_block_get_frame(block), mtr);
2272      ut_ad(level
2273      == btr_page_get_level(buf_block_get_frame(new_block), mtr));
2274
2275      /* Build the node pointer (= node key and page address) for the upper
2276      half */
2277
2278      node_ptr_upper = dict_index_build_node_ptr(index, split_rec,
2279      upper_page_no, heap, level);
2280
2281      /* Insert it next to the pointer to the lower half. Note that this
2282      may generate recursion leading to a split on the higher level. */
2283
2284      btr_insert_on_non_leaf_level(flags, index, level + 1,
2285      node_ptr_upper, mtr);
2286
2287      /* Free the memory heap */
2288      mem_heap_free(heap);
2289
2290      /* Update page links of the level */
```

获取上层页信息

插入下层页信息时
可能导致上层页分裂

(上图中文是翻译的代码注释, 可能会有些偏差错误)

检查测试日志文件 error-log, 默认会在当前目录生成 var 目录, 其中包含 my.cnf 文件

/tools/mysql-test5.7.30/mysql-test/var/log/mysqld.1.err

```
Trying to get some variables.
Some pointers may be invalid and cause the dump to abort.
Query (7fc234005890): select * from t1 order by func1(a)
Connection ID (thread ID): 2
Status: NOT_KILLED

The manual page at http://dev.mysql.com/doc/mysql/en/crashing.html contains
information that should help you find out what is causing the crash.
Writing a core file
```

```
safe_process[7089]: Child process: 7090, killed by signal: 6
```

关于编译安装 MySQL 的补充:

Cmake 编译之后会在 DCMAKE_INSTALL_PREFIX 目录中生成 mysql-test 测试框架目录,

不需要以下步骤 (以下步骤在不需要 DEBUG 调试时使用)

1. 此包是 mysql-test-5.7.30-linux-glibc2.12-x86_64.tar.gz

```
mv mysql-5.7.30-linux-glibc2.12-x86_64/ /tools/mysql-test
```

2. 将 mysql 安装包目录下的文件与 mtr 目录合并, mysql 安装包目录下为 basedircp -r /data/mysql/base/5.7.30 /tools/mysql-test

(插播生活日记, make 命令大约需要执行 1 个小时, 刚好 18:27, 又恰逢隔离, 起身洗锅炸厨房了噯)
测试结论

测试结论

此 BUG 可能会出现在 MySQL5.7 版本中

1. 测试中验证了数据库参数 innodb_temp_data_file_path 增大 max_file_size 后不会发生服务崩溃, 如果业务 SQL 无法进行优化时, 可以增大此参数, 可降低触发崩溃的概率。
2. 测试中验证了数据库参数 internal_tmp_disk_storage_engine=MYISAM 时不会发生服务崩溃, 默认 INNODB

如果业务无法升级到 8.0 时, 可以动态调整此参数。

我们建议的变更顺序是:

优化 SQL 语句 -> 增大 innodb_temp_data_file_path 参数的 max_file_size 值 -> 升级到 MySQL 8.0 (使用会话临时表空间) -> 修改 internal_tmp_disk_storage_engine 参数。

其中 internal_tmp_disk_storage_engine 参数, 个人不是很理解, 是否真的要将默认值 INNODB 更改为 MYISAM。之后请教同事了解到, "内部临时表不会被复制, 不会有并发访问, 是可以考虑使用 MYISAM 的"

再次感谢噯。

参考链接

BUG 报告:

<https://bugs.mysql.com/bug.php?id=101154> <https://jira.percona.com/browse/PS-7318?focusedCommentId=268045&page=com.atlassian.jira.plugin.system.issuetabpanels%3Acomment-tabpanel#comment-268045>

编译安装及参数说明:

https://blog.csdn.net/iteye_621/article/details/81959655 <https://baijiahao.baidu.com/s?id=1725289345179642059&wfr=spider&for=pc>

<https://dev.mysql.com/doc/refman/5.7/en/source-configuration-options.html#cmake-general-options>

MTR 文档:

https://dev.mysql.com/doc/dev/mysql-server/latest/PAGE_MYSQLTEST_FRAMEWORK_COMPONENTS.html

数据库参数:

https://dev.mysql.com/doc/refman/5.7/en/server-status-variables.html#statvar_Created_tmp_tables

https://dev.mysql.com/doc/refman/5.7/en/server-status-variables.html#statvar_Created_tmp_disk_tables

https://dev.mysql.com/doc/refman/5.7/en/server-system-variables.html#sysvar_tmp_table_size

https://dev.mysql.com/doc/refman/5.7/en/server-system-variables.html#sysvar_max_heap_table_size

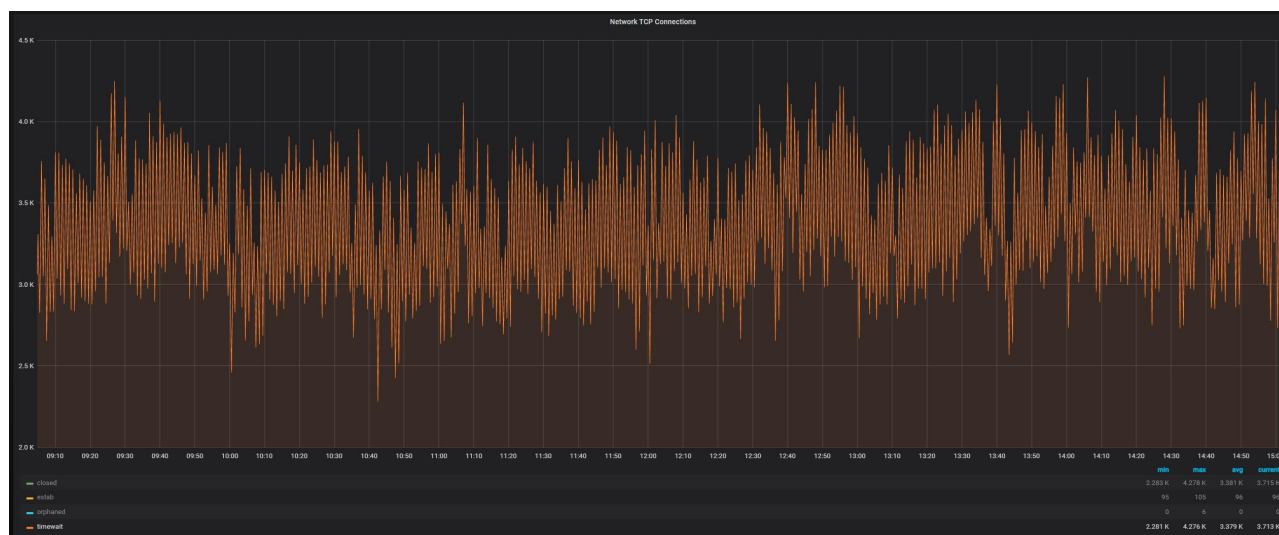
https://dev.mysql.com/doc/refman/5.7/en/server-system-variables.html#sysvar_big_tables

<https://dev.mysql.com/doc/refman/5.7/en/select.html>

3.8 MySQL Server 端为什么有大量的 TCP TIME_WAIT ?

作者: 李鹏博

最近客户的一台 MySQL Server 告警处于 TCP TIME_WAIT 状态的连接过多, 通过查看相关监控发现 TCP Connections 中处于 TIME_WAIT 状态的连接一直在 3000~4000 , 如图:



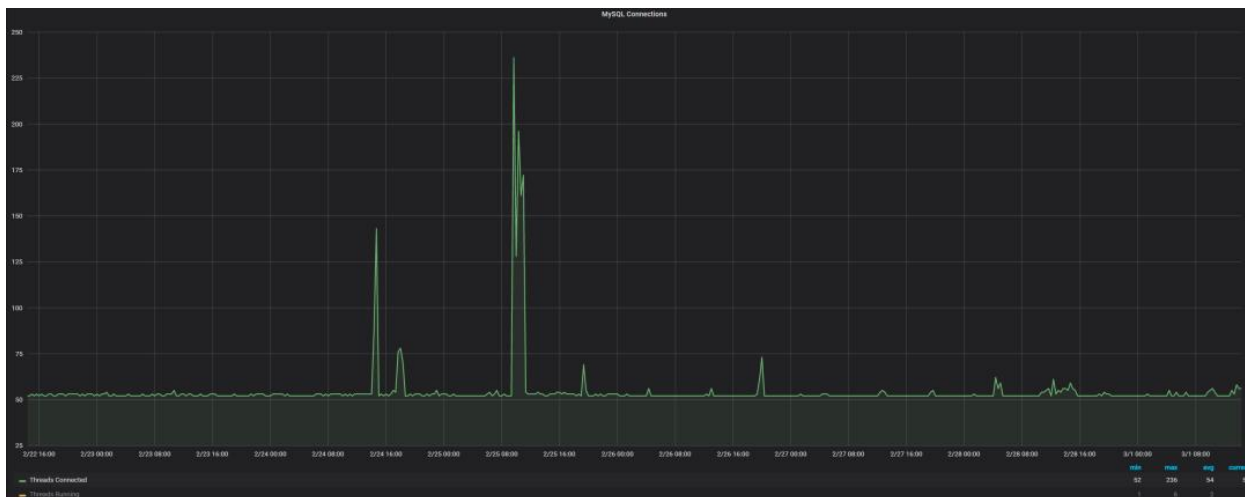
在 OS 上查看 TCP 处于 TIME_WAIT 状态的连接:

tcp6	0	0	161:4125	234:46114	TIME_WAIT
tcp6	0	0	161:4125	234:49550	TIME_WAIT
tcp6	0	0	161:4125	234:47302	TIME_WAIT
tcp6	0	0	161:4125	234:52534	TIME_WAIT
tcp6	0	0	161:4125	234:48114	TIME_WAIT
tcp6	0	0	161:4125	234:47094	TIME_WAIT
tcp6	0	0	161:4125	234:47188	TIME_WAIT
tcp6	0	0	161:4125	234:46180	TIME_WAIT
tcp6	0	0	161:4125	234:48960	TIME_WAIT
tcp6	0	0	161:4125	234:51800	TIME_WAIT
tcp6	0	0	161:4125	234:46316	TIME_WAIT
tcp6	0	0	161:4125	234:53114	TIME_WAIT
tcp6	0	0	161:4125	234:46848	TIME_WAIT
tcp6	0	0	161:4125	234:48760	TIME_WAIT
tcp6	0	0	161:4125	234:45724	TIME_WAIT
tcp6	0	0	161:4125	234:50056	TIME_WAIT
tcp6	0	0	161:4125	234:52180	TIME_WAIT
tcp6	0	0	161:4125	234:49140	TIME_WAIT
tcp6	0	0	161:4125	234:51176	TIME_WAIT
tcp6	0	0	161:4125	234:47180	TIME_WAIT
tcp6	0	0	161:4125	234:46530	TIME_WAIT
tcp6	0	0	161:4125	234:52422	TIME_WAIT
tcp6	0	0	161:4125	234:45762	TIME_WAIT
tcp6	0	0	161:4125	234:51398	TIME_WAIT
tcp6	0	0	161:4125	234:47830	TIME_WAIT
tcp6	0	0	161:4125	234:51904	TIME_WAIT
tcp6	0	0	161:4125	234:52552	TIME_WAIT
tcp6	0	0	161:4125	234:52620	TIME_WAIT
tcp6	0	0	161:4125	234:48848	TIME_WAIT
tcp6	0	0	161:4125	234:48042	TIME_WAIT
tcp6	0	0	161:4125	234:47342	TIME_WAIT
tcp6	0	0	161:4125	234:51174	TIME_WAIT
tcp6	0	0	161:4125	234:49582	TIME_WAIT
tcp6	0	0	161:4125	234:45044	TIME_WAIT
tcp6	0	0	161:4125	234:48060	TIME_WAIT
tcp6	0	0	161:4125	234:51346	TIME_WAIT
tcp6	0	0	161:4125	234:48664	TIME_WAIT
tcp6	0	0	161:4125	234:45322	TIME_WAIT
tcp6	0	0	161:4125	234:50070	TIME_WAIT
tcp6	0	0	161:4125	234:51866	TIME_WAIT
tcp6	0	0	161:4125	234:51806	TIME_WAIT
tcp6	0	0	161:4125	234:51808	TIME_WAIT
tcp6	0	0	161:4125	234:50370	TIME_WAIT
tcp6	0	0	161:4125	234:49528	TIME_WAIT
tcp6	0	0	161:4125	234:48986	TIME_WAIT
tcp6	0	0	161:4125	234:48232	TIME_WAIT
tcp6	0	0	161:4125	234:50698	TIME_WAIT
tcp6	0	0	161:4125	234:46576	TIME_WAIT
tcp6	0	0	161:4125	234:47600	TIME_WAIT
tcp6	0	0	161:4125	234:50610	TIME_WAIT
tcp6	0	0	161:4125	234:49834	TIME_WAIT
tcp6	0	0	161:4125	234:45690	TIME_WAIT
tcp6	0	0	161:4125	234:47534	TIME_WAIT
tcp6	0	0	161:4125	234:51324	TIME_WAIT
tcp6	0	0	161:4125	234:53314	TIME_WAIT
tcp6	0	0	161:4125	234:48730	TIME_WAIT
tcp6	0	0	161:4125	234:46892	TIME_WAIT
tcp6	0	0	161:4125	234:52072	TIME_WAIT
tcp6	0	0	161:4125	234:48434	TIME_WAIT
tcp6	0	0	161:4125	234:47968	TIME_WAIT
tcp6	0	0	161:4125	234:50628	TIME_WAIT
tcp6	0	0	161:4125	234:51408	TIME_WAIT
tcp6	0	0	161:4125	234:45976	TIME_WAIT
tcp6	0	0	161:4125	234:51122	TIME_WAIT
tcp6	0	0	161:4125	234:47492	TIME_WAIT

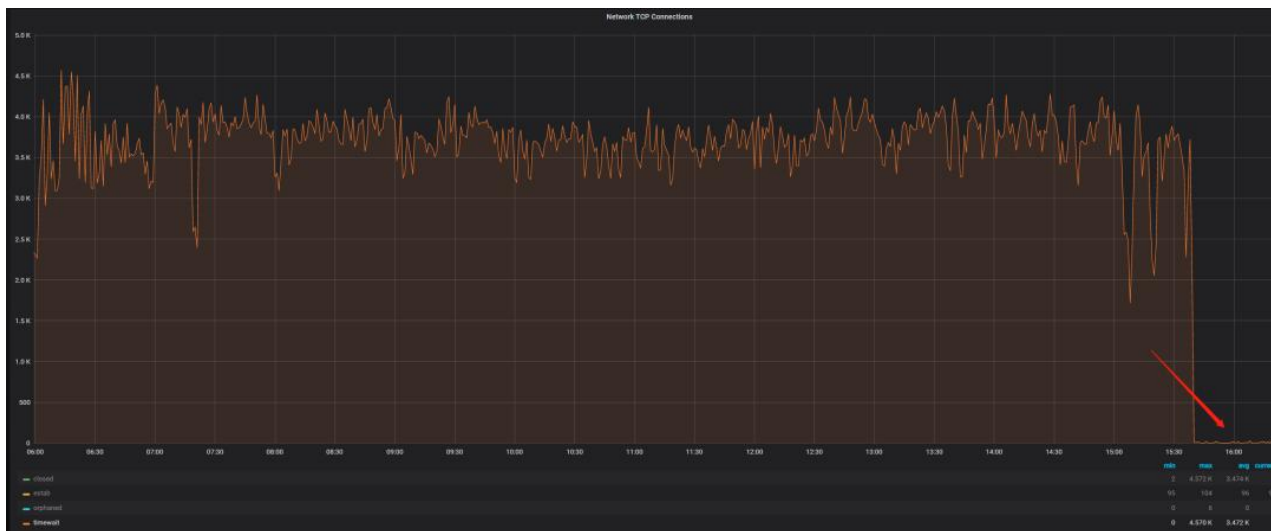
（其中 161 服务器是本地的 MySQL Server，4125 是数据库端口，234 是远程应用服务器）

可以看到所有处于 TIME_WAIT 状态的 TCP 连接都是应用服务器 234 到本地 161 的数据库连接，此时心中有一个疑问：只有主动关闭 TCP 连接的一端才会存在 TIME_WAIT 状态，主观的想法就是大量的 TIME_WAIT 应该位于应用服务器一端，那为何 MySQL Server 这一端有这么多处于 TIME_WAIT 状态的 TCP 连接？难道是有大量由于 wait_timeout 超时的连接，所以 MySQL Server 这一端主动关闭了连接，还是由于网络异常或客户端异常关闭，导致 MySQL Server 一端主动关闭了连接，但是通过观察

MySQL Connections 相关监控发现连接数量一直都是比较少的：



所以就不是由于大量的连接因为 `wait_timeout` 超时而造成的 MySQL Server 一端主动关闭了连接。通过查看 `mysqld error log` 里面也没有太多关于“Aborted connection”或“Got an error reading communication packets”，所以也不是由于网络异常或客户端异常关闭导致的 MySQL Server 一端主动关闭了连接。此时就想从系统角度入手看能不能进行相关调整，之后将 `net.ipv4.tcp_tw_recycle` 系统参数从 0 调整为了 1，然后观察 TCP Connections 相关监控发现效果非常明显，TIME_WAIT 状态的 TCP 连接瞬间就降下去了：



但是查询相关资料发现 `tcp_tw_recycle` 这个参数是有坑的：<https://linux.die.net/man/7/tcp>

`tcp_tw_recycle` (Boolean; default: disabled; since Linux 2.4)

Enable fast recycling of TIME_WAIT sockets. Enabling this option is not recommended since this causes problems when working with NAT (Network Address Translation).

所以虽然效果很明显，但是为了稳妥起见，还是将 `net.ipv4.tcp_tw_recycle` 参数改回了 0。这时就只能换一个思路：通过抓包看看数据库服务器和应用服务器到底是如何通信的，由于没有权限登录应用服务器，就只能在 161 数据库服务器上通过 `tcpdump` 单向抓包并使用 Wireshark 工具进行分析。通过分析发现在抓取的 68000 多个包中存在大量的 FIN 包和 SYN 包，由此说明应用服务器存在大量的短连接，一直在连接数据库，短时间的查询之后又断开连接，并不断反复：

> Chat	Connection finish (FIN)	Sequence	TCP	5439
> Chat	Connection establish acknowledge (SYN+ACK): s...	Sequence	TCP	2719
> Chat	Connection establish request (SYN): server port ...	Sequence	TCP	2719

但是这依旧不能解释为什么 MySQL Server 一端存在大量处于 TIME_WAIT 状态的 TCP 连接。继续分析网络包之后发现当 Client 请求退出之后，Client 和 Server 之间的 TCP 连接有两种断开方式：第一种：234 应用服务器发出退出请求，161 数据库服务器回复“ACK”包，然后由 234 应用服务器主动断开连接。

140 0.219425	234	161	MySQL	71 Request Quit
141 0.219448	161	234	TCP	66 4125 → 53660 [ACK] Seq=1476 Ack=1680 Win=34048 Len=0 TSval=2133308026 TSecr=3383587550
142 0.219454	234	161	TCP	66 53660 → 4125 [FIN, ACK] Seq=1680 Ack=1476 Win=33488 Len=0 TSval=3383587550 TSecr=2133308025
143 0.219491	161	234	TCP	66 4125 → 53660 [FIN, ACK] Seq=1476 Ack=1681 Win=34048 Len=0 TSval=2133308026 TSecr=3383587550

第二种：234 应用服务器发出退出请求，161 数据库服务器主动断开与 234 应用服务器的连接。

166 0.235367	234	161	MySQL	71 Request Quit
167 0.235392	161	234	TCP	66 4125 → 53668 [FIN, ACK] Seq=6828 Ack=1818 Win=34048 Len=0 TSval=2133308042 TSecr=3383587566
168 0.235422	234	161	TCP	66 53668 → 4125 [FIN, ACK] Seq=1818 Ack=6828 Win=42112 Len=0 TSval=3383587566 TSecr=2133308041

此时我们就可以得出结论：在 MySQL 退出会话时，Client 与 MySQL Server 之间，TCP 连接的断开有可能是由 Client 发起的，也有可能是由 Server 发起的，而并不是我们主观认为的都是由 Client 发起的，所以当 Client 与 MySQL Server 之间存在大量的短连接时，MySQL Server 的 OS 上就可能会存在大量处于 TIME_WAIT 状态的 TCP 连接，而解决方法就是由应用侧将大量的短连接由少量的长连接代替。

在 MySQL Internals Manual 上也有相关 Client 退出会话时 MySQL Server 的动作说明：

<https://dev.mysql.com/doc/internals/en/com-quit.html>

14.6.2 COM_QUIT

COM_QUIT:

tells the server that the client wants to close the connection

response: either a connection close or a OK Packet

这也刚好印证了通过抓包看到的现象。

3.9 大量短时进程导致 cpu 负载过高案例一则

作者：任坤

1、背景

某项目的开发环境，单台虚拟机装了 1 套 mongo 集群用于测试，1 个 mongos + 3 节点 config + 1 shard * 3 副本，总计 7 个 mongo 实例。mongo 版本 4.2.19，OS 为 centos 7.9。测试结束后 cpu 负载一直维持在 50% 左右，而此时 mongo 的 qps 已经下降为 0。这台机器上只安装了 mongo，将所有 mongo 实例关闭，cpu 负载立即恢复正常，再将 mongo 实例开启，过了一会 cpu 负载又开始飙升。场景能复现，且确认是跟 mongo 实例有关系。

2、诊断

执行 top 命令，cpu 的 usr 已经达到了 40%，但是前几个进程的 %cpu 加起来远远凑不够数。

top - 16:05:21 up 21 days, 1:57, 10 users, load average: 0.81, 0.53, 0.54											
Tasks: 200 total, 1 running, 199 sleeping, 0 stopped, 0 zombie											
%cpu(s): 39.1 us, 10.7 sy, 0.0 ni, 49.4 id, 0.0 wa, 0.0 hi, 0.8 si, 0.0 st											
KiB Mem : 8009020 total, 3548308 free, 2914748 used, 1545964 buff/cache											
KiB Swap: 8388604 total, 8177792 free, 210812 used. 4528736 avail Mem											
PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2009	root	20	0	472556	49364	2164	S	7.0	0.6	1261:45	titanagent
27366	root	20	0	2511980	713868	17044	S	2.0	8.9	13:09.23	mongod
27053	root	20	0	1999752	101660	17208	S	1.7	1.3	4:30.93	mongod
27299	root	20	0	2561640	713152	17496	S	1.7	8.9	17:06.26	mongod
26990	root	20	0	2008456	104604	17428	S	1.3	1.3	4:42.36	mongod
27173	root	20	0	2659680	779620	17932	S	1.3	9.7	39:17.02	mongod
27106	root	20	0	1932140	99708	16016	S	1.0	1.2	4:20.94	mongod
3448	linuxRo+	20	0	162352	2404	1596	S	0.7	0.0	1:18.52	top
7979	root	20	0	433000	57992	10224	S	0.7	0.7	14:48.17	mongos
10321	zabbix	20	0	83392	2416	1444	S	0.7	0.0	0:00.07	zabbix_agentd
11133	root	20	0	113312	992	848	S	0.7	0.0	0:00.05	sadc
12081	root	20	0	162248	2436	1596	R	0.7	0.0	0:00.05	top

查看 mongos 的 qps，确实没有执行用户命令了。

insert	query	update	delete	getmore	command	flushes	mapped	vsize	res	faults	qrw	arw	net_in	net_out	conn
*0	*0	*0	*0	0	24	0	0B	422M	56.0M	0	0	0	371b	14.4k	16
*0	*0	*0	*0	0	24	0	0B	422M	56.0M	0	0	0	4.27k	52.5k	16
*0	*0	*0	*0	0	1	0	0B	422M	56.0M	0	0	0	214b	14.0k	16
*0	1	*0	*0	0	14	0	0B	422M	56.0M	0	0	0	3.00k	34.5k	16
*0	*0	*0	*0	0	3	0	0B	422M	56.0M	0	0	0	528b	14.7k	16
*0	1	*0	*0	0	15	0	0B	422M	56.0M	0	0	0	2.79k	34.4k	16
*0	*0	*0	*0	0	1	0	0B	422M	56.0M	0	0	0	214b	14.0k	16
*0	*0	*0	*0	0	12	0	0B	422M	56.0M	0	0	0	2.25k	33.3k	16
*0	2	*0	*0	0	15	0	0B	422M	56.0M	0	0	0	3.01k	34.6k	16
*0	*0	*0	*0	0	13	0	0B	422M	56.0M	0	0	0	2.40k	33.7k	16

dstat 查看整体负载(vmstat 格式化做的不好，最后几列总是对不齐整)。

---total-cpu-usage---							-dsk/total-		-net/total-		---paging--		---system--	
usr	sys	idl	wai	hiq	sig		read	writ	recv	send	in	out	int	csw
25	8	67	0	0	0	0	0	48k	5859B	32k	0	0	2911	2094
62	17	20	0	0	1	0	0	72k	5655B	4839B	0	0	5173	2662
48	15	37	0	0	1	0	0	120k	7287B	6749B	0	0	4977	3415
49	13	38	0	0	1	0	0	108k	6820B	32k	0	0	4798	3367
32	11	56	0	0	1	0	0	36k	4614B	3119B	0	0	3616	2237
37	16	47	0	0	0	0	0	48k	6449B	5634B	0	0	4697	3578
47	14	38	0	0	1	0	0	104k	6519B	36k	0	0	4978	3404

除了 cpu 负载不正常，其余指标均正常，中断和上下文切换也不算高，不太可能是这两个引发的。perf record -ag -- sleep 10 && perf report 查看 cpu 执行情况。

Samples: 152K of event 'cpu-clock', Event count (approx.): 38225250000

Children	Self	Command	Shared object	Symbol
+ 28.88%	0.00%	swapper	[kernel.kallsyms]	[k] start_cpu
+ 28.87%	0.02%	swapper	[kernel.kallsyms]	[k] cpu_startup_entry
+ 28.61%	0.01%	swapper	[kernel.kallsyms]	[k] arch_cpu_idle
+ 28.60%	0.01%	swapper	[kernel.kallsyms]	[k] default_idle
- 26.79%	0.00%	js	mongo	[.] nspr::Thread::ThreadRoutine
- nspr::Thread::ThreadRoutine				
- 26.73%	0.00%	mongo::mozjs::MozJSProxyScope::implThread		
+ 23.74%	0.02%	swapper	[kernel.kallsyms]	[k] cpu_startup_entry
+ 2.26%	0.01%	auto mongo::unique_function<void ()>::makeImpl	mongo::mozjs::MozJSProxyScope::exec(mongo::StringData,	
+ 0.71%	0.00%	mongo::mozjs::MozJSImplScope::~MozJSImplScope		
- 26.79%	0.00%	js	mongo	[.] std::thread::_State_impl<std::thread::_Invoker<std::tuple<mongo::std::thread::thread<void* (*) (void*), nspr::Thre
- nspr::Thread::ThreadRoutine				
- 26.73%	0.00%	mongo::mozjs::MozJSProxyScope::implThread		
+ 23.74%	0.02%	swapper	[kernel.kallsyms]	[k] cpu_startup_entry
+ 2.26%	0.01%	auto mongo::unique_function<void ()>::makeImpl	mongo::mozjs::MozJSProxyScope::exec(mongo::StringData,	
+ 0.71%	0.00%	mongo::mozjs::MozJSImplScope::~MozJSImplScope		
+ 26.79%	0.00%	js	mongo	[.] execute_native_thread_routine
+ 26.73%	0.00%	js	mongo	[.] mongo::mozjs::MozJSProxyScope::implThread
+ 26.30%	26.30%	swapper	[kernel.kallsyms]	[k] native_safe_halt
+ 23.75%	0.00%	js	mongo	[.] mongo::mozjs::MozJSImplScope::MozJSImplScope

确实有大量 mongo 调用，但是 API 命名不直观，无法猜测对应的执行逻辑。至此，确认是 mongo 实例引发的问题，但是 mongo 的应用连接为 0，看调用 API 栈也找不到有用信息。回到本文开头，top 进程的 cpu 利用率加起来远远小于 cpu 总体负载，大概率是有频繁短时进程偷走了这部分 CPU 资源，导致 top 命令来不及捕获统计。

sar -w 1 查看每秒生成的进程数，平均每秒新建 80 多个进程，应该就是它了。

	proc/s	cswch/s
19时45分39秒	80.20	3223.76
19时45分41秒	78.22	2479.21
19时45分42秒	70.00	2573.00
19时45分43秒	94.00	3010.00
19时45分44秒	80.00	3074.00
19时45分45秒		

要抓出频繁建立短时进程的应用，可以采用 execsnoop，该工具通过 ftrace 实时监控进程的 exec() 行为，并输出短时进程的基本信息，包括进程 PID/PPID、命令行参数。

```
#下载 execsnoop#
cd /usr/bin
wget https://raw.githubusercontent.com/brendangregg/perf-tools/master/execsnoop
chmod 755 execsnoop
```

以下是输出内容，全是监控系统在执行，不停的连接 mongo 并对输出结果执行 grep 过滤，每个操作都会衍生一个新线程/进程，10s 捕获了 400 多条记录。

```

9459 9458 /bin/echo db.serverStatus().transactions.totalcommitted
9460 9458 /usr/local/mongodb/bin/mongo admin -uzabbix -p[REDACTED] --port 40001
9461 9458 grep -vE session|bye|admin|version
9462 9456 sh-9462 [003] d... 1834707.729700: execsnoop_sys_execve: (Sys_execve+0x0/0x30)
9463 9462 /bin/echo db.serverStatus().opcounters.getmore
9464 9462 /usr/local/mongodb/bin/mongo admin -uzabbix -p[REDACTED] --port 27017
9465 9462 grep -vE session|bye|admin|version
9466 9457 sh-9466 [000] d... 1834707.744717: execsnoop_sys_execve: (Sys_execve+0x0/0x30)
9467 9466 /bin/echo db.serverStatus().opcounters.getmore
9469 9466 grep -vE session|bye|admin|version
9468 9466 /usr/local/mongodb/bin/mongo admin -uzabbix -p[REDACTED] --port 40002
9482 9452 sh-9482 [000] d... 1834708.048090: execsnoop_sys_execve: (Sys_execve+0x0/0x30)
9483 9456 sh-9483 [003] d... 1834708.050600: execsnoop_sys_execve: (Sys_execve+0x0/0x30)
9485 9482 awk -F[,()] {print $2}
9487 9483 awk -F[,()] {print $2}
9488 10322 sh-9488 [001] d... 1834708.063511: execsnoop_sys_execve: (Sys_execve+0x0/0x30)
9489 9457 sh-9489 [000] d... 1834708.064405: execsnoop_sys_execve: (Sys_execve+0x0/0x30)
9491 10323 sh-9491 [001] d... 1834708.065659: execsnoop_sys_execve: (Sys_execve+0x0/0x30)
9493 9489 awk -F[,()] {print $2}
9494 10321 zabbix_agentd-9494 [002] d... 1834708.079199: execsnoop_sys_execve: (Sys_execve+0x0/0x30)
9495 9488 sh-9495 [002] d... 1834708.098887: execsnoop_sys_execve: (Sys_execve+0x0/0x30)
9496 9491 sh-9496 [003] d... 1834708.102738: execsnoop_sys_execve: (Sys_execve+0x0/0x30)
9497 9495 /bin/echo db.serverStatus().opcounters.insert
9498 9495 /usr/local/mongodb/bin/mongo admin -uzabbix -p[REDACTED] --port 27018

```

将 zabbix 进程关闭，cpu 马上恢复正常，找到了元凶。

我们其他环境也采用了 zabbix 监控，但是都没有遇到类似问题。

该节点部署了 7 个 mongo 实例，zabbix 默认对每个 mongo 实例都进行监控，相当于执行损耗放大了 7 倍，而该机器是一台只有 4 核 CPU 的虚拟机。这些因素凑齐了就会爆发问题。

这是个开发环境，暂时关闭了 zabbix 监控，后续要对监控逻辑进行优化，尽量减少连接 db 的次数以及 grep 调用链的长度。

3、小结

当机器 cpu 负载持续高涨却抓取不到 top 进程时，可以采用 execsnoop 抓取短时进程，类似工具还有 iosnoop、opensnoop。

3.10 DDL 导致的 Xtrabackup 备份失败

作者：赵黎明

背景

近日，客户反馈某生产业务系统凌晨的物理备份都失败了（一主二从的集群，仅在两个从库上做 Xtrabackup 全备，主库不参与备份），需排查备份失败的原因。

案例分析

由于客户使用的是我司爱可生的 DMP 数据库管理平台，当备份失败时，在备份目录中会写入一个 FAIL 的标志文件，然后回滚掉残留文件，此时 Xtrabackup 自身的日志已无法查看，不过可以通过 urman-agent 组件（负责备份恢复）日志来获取备份失败的信息，以下是当时两个从库上的报错信息。

● 从库 1 日志

```
220512 00:19:09 >> log scanned up to (126630987919112)
220512 00:19:10 >> log scanned up to (126630987919112)
InnoDB: Last flushed lsn: 126630788171713 load_index lsn 126630987919112
[FATAL] InnoDB: An optimized(without redo logging) DDLoperation has been performed. All modified pages may not have been flushed to the disk yet.
FXB will not be able take a consistent backup. Retry the backup operation
2022-05-12 00:19:11 0x7fbc7d681700 InnoDB: Assertion failure in thread 140447534552832 in file utOut.cc line 916
InnoDB: We intentionally generate a memory trap.
InnoDB: Submit a detailed bug report to http://bugs.mysql.com.
InnoDB: If you get repeated assertion failures or crashes, even
InnoDB: immediately after the mysqld startup, there may be
InnoDB: corruption in the InnoDB tablespace. Please refer to
InnoDB: http://dev.mysql.com/doc/refman/5.7/en/forcing-innodb-recovery.html
InnoDB: about forcing recovery.
16:19:11 UTC - xtrabackup got signal 6 ;
This could be because you hit a bug or data is corrupted.
This error can also be caused by malfunctioning hardware.
Attempting to collect some information that could help diagnose the problem.
As this is a crash and something is definitely wrong, the information
collection process might fail.

Thread pointer: 0x0
Attempting backtrace. You can use the following information to find out
where mysqld died. If you see no messages after this, something went
terribly wrong...
stack_bottom = 0 thread_stack 0x10000
innobackupex(my_print_stacktrace+0x35) [0xd55c45]
innobackupex(handle_fatal_signal+0x273) [0xb0d5a3]
/lib64/libpthread.so.0(+0xf5e0) [0x7fbc898a55e0]
/lib64/libc.so.6(gsignal+0x37) [0x7fbc87e071f7]
/lib64/libc.so.6(abort+0x148) [0x7fbc87e088e8]
innobackupex[0x7bfbe5]
:~
```

● 从库 2 日志

```

220512 02:46:03 >> log scanned up to (138003751883737)
220512 02:46:04 >> log scanned up to (138003751934155)
InnoDB: Last flushed lsn: 137999361579213 load_index lsn 138003751934155
[FATAL] InnoDB: An optimized(without redo logging) DDLOperation has been performed. All modified pages may not have been flushed to the disk yet.
PXBar will not be able take a consistent backup. Retry the backup operation
2022-05-12 02:46:05 0x7f6d53138700 InnoDB: Assertion failure in thread 140107521951488 in file ut0ut.cc line 916
InnoDB: We intentionally generate a memory trap.
InnoDB: Submit a detailed bug report to http://bugs.mysql.com.
InnoDB: If you get repeated assertion failures or crashes, even
InnoDB: immediately after the mysqld startup, there may be
InnoDB: corruption in the InnoDB tablespace. Please refer to
InnoDB: http://dev.mysql.com/doc/refman/5.7/en/forcing-innodb-recovery.html
InnoDB: about forcing recovery.
18:46:05 UTC - xtrabackup got signal 6 :
This could be because you hit a bug or data is corrupted.
This error can also be caused by malfunctioning hardware.
Attempting to collect some information that could help diagnose the problem.
As this is a crash and something is definitely wrong, the information
collection process might fail.

Thread pointer: 0x0
Attempting backtrace. You can use the following information to find out
where mysqld died. If you see no messages after this, something went
terribly wrong...
stack_bottom = 0 thread_stack 0x10000
innobackupex(my_print_stacktrace+0x35) [0xd55c45]
innobackupex(handle_fatal_signal+0x273) [0xb0d5a3]
/lib64/libpthread.so.0(+0xf5e0) [0x7f6d5f35c5e0]
/lib64/libc.so.6(gsignal+0x37) [0x7f6d5d8b1f7]
/lib64/libc.so.6(abort+0x148) [0x7f6d5d8bfe8]
innobackupex[0x7bfb5]
:

```

两个从库虽然报错的时间不同，但报错的内容一致，都指向了“不记录 redo 日志的 DDL 操作”：

[FATAL] InnoDB: An optimized(without redo logging) DDLOperation has been performed. All modified pages may not have been flushed to the disk yet. PXBar will not be able take a consistent backup. Retry the backup operation

经确认，客户的确是在凌晨执行了 DDL 业务变更，变更的内容为创建一张新表，并给现存的两张表添加字段，加字段的表大约有几百万行记录，这一信息与日志给出的内容吻合，看来问题大概率是出在加字段的 DDL 操作上

那什么是不记录 redo 的 DDL 的操作呢？为何会存在不记录 redo 的 DDL ？

首先，我们知道，在 MySQL 5.7 中给表加字段属于 ONLINE DDL，会重建表，但允许并发 DML（PS：MySQL 8.0 加字段不需要重建表）

Column Operations

The following table provides an overview of online DDL support for column operations. An asterisk indicates additional information, an exception, or a dependency. For details, see [Syntax and Usage Notes](#).

Table 14.12 Online DDL Support for Column Operations

Operation	In Place	Rebuilds Table	Permits Concurrent DML	Only Modifies Metadata
Adding a column	Yes	Yes	Yes*	No
Dropping a column	Yes	Yes	Yes	No
Renaming a column	Yes	No	Yes*	Yes
Reordering columns	Yes	Yes	Yes	No

由于 MySQL 采用的是索引组织表（IOT），表上的索引当然也需要重建，由于采用了 ALGORITHM=INPLACE 的方式，允许并发 DML

Syntax and Usage Notes

- Adding a column

```
ALTER TABLE tbl_name ADD COLUMN column_name column_definition, ALGORITHM=INPLACE, LOCK=NONE;
```

Concurrent DML is not permitted when adding an [auto-increment](#) column. Data is reorganized substantially, making it an expensive operation. At a minimum, `ALGORITHM=INPLACE`, `LOCK=SHARED` is required.

在 MySQL 5.7 中，对索引有一个优化，即 **Sorted Index Builds**，会在创建或重建索引的时候通过 **bulk load**、**bottom-up** 的方式来填充索引记录

14.6.2.3 Sorted Index Builds

InnoDB performs a bulk load instead of inserting one index record at a time when creating or rebuilding indexes. This method of index creation is also known as a sorted index build. Sorted index builds are not supported for spatial indexes.

There are three phases to an index build. In the first phase, the [clustered index](#) is scanned, and index entries are generated and added to the sort buffer. When the [sort buffer](#) becomes full, entries are sorted and written out to a temporary intermediate file. This process is also known as a “run”. In the second phase, with one or more runs written to the temporary intermediate file, a merge sort is performed on all entries in the file. In the third and final phase, the sorted entries are inserted into the [B-tree](#).

Prior to the introduction of sorted index builds, index entries were inserted into the B-tree one record at a time using insert APIs. This method involved opening a B-tree [cursor](#) to find the insert position and then inserting entries into a B-tree page using an [optimistic](#) insert. If an insert failed due to a page being full, a [pessimistic](#) insert would be performed, which involves opening a B-tree cursor and splitting and merging B-tree nodes as necessary to find space for the entry. The drawbacks of this “top-down” method of building an index are the cost of searching for an insert position and the constant splitting and merging of B-tree nodes.

Sorted index builds use a “bottom-up” approach to building an index. With this approach, a reference to the right-most leaf page is held at all levels of the B-tree. The right-most leaf page at the necessary B-tree depth is allocated and entries are inserted according to their sorted order. Once a leaf page is full, a node pointer is appended to the parent page and a sibling leaf page is allocated for the next insert. This process continues until all entries are inserted, which may result in inserts up to the root level. When a sibling page is allocated, the reference to the previously pinned leaf page is released, and the newly allocated leaf page becomes the right-most leaf page and new default insert location.

WL#7277: InnoDB: Bulk Load for Create Index

Status: Complete

Description	Requirements	High Level Architecture	Low Level Design
Currently we create an index by repeatedly inserting records. It's very slow when we have a large collection of records. Bulk loading can be done much more efficiently. We sort the records using index key and then insert (exploiting sorted order of the records). The basic idea of bulk load is we builds an index from bottom up (also known as sorted index build). Index entries first go into the leaf page, and when the leaf page is filled up we add the corresponding search-key entry in to the parent non-leaf node and start filling next leaf page. Filling records into leaf page is fast, and we have much less splitting in the building process. Also this approach help in maintaining good locality and page once pinned and unpinned can be flushed to disk as it would not be pinned again for build phase.			

采用 Sorted Index Build 方式创建索引时，不会记录到 redo 日志中，而这就是之前 Xtrabackup 报错中所描述的 “An optimized(without redo logging) DDL operation” 的场景

Sorted Index Builds and Redo Logging

Redo logging is disabled during a sorted index build. Instead, there is a checkpoint to ensure that the index build can withstand an unexpected exit or failure. The checkpoint forces a write of all dirty pages to disk. During a sorted index build, the page cleaner thread is signaled periodically to flush dirty pages to ensure that the checkpoint operation can be processed quickly. Normally, the page cleaner thread flushes dirty pages when the number of clean pages falls below a set threshold. For sorted index builds, dirty pages are flushed promptly to reduce checkpoint overhead and to parallelize I/O and CPU activity.

Percona 称这是 Xtrabackup 2.4.x 的一个 bug，主要是为了避免得到一个错误的备份集，一旦检测到有不记录 redo 的 DDL 操作，就会将备份进程终止，而客户生产环境中的 PXB 版本正是 2.4.5

针对这一问题，Percona 在 Xtrabackup 2.4.8 及之后的版本中，新增了 --lock-ddl, --lock-ddl-timeout, --lock-ddl-per-table 这几个参数，使其可以在备份时加上备份锁，或给表加上 MDL 锁来阻塞其他的 DDL 操作，使备份顺利完成

原因明确了，就知道如何解决问题了，最后分别在 2 个从库上执行手动备份，每个实例（500G 左右）大约耗时 2 小时 40 分钟完成备份

Percona 以增加参数的方式提供了解决备份失败的方法，那如果暂时无法升级 PXB 版本，仅在 MySQL 层面，有没有解决方法呢？其实也是可以的。

对于以上提到的几种场景，我们都来测试一下吧

场景测试

环境准备

创建测试表

```
/usr/local/sysbench/share/sysbench# sysbench oltp_insert.lua --db-driver=mysql --
threads=256 --time=300 --mysql-host=10.186.60.68 --mysql-port=3332 --mysql-user=zlm --
mysql-password=zlm --tables=2 --table-size=2000000 --db-ps-mode=disable --report-
interval=10 prepare
sysbench 1.0.17 (using bundled LuaJIT 2.1.0-beta2)
```

```
Initializing worker threads...
```

```
Creating table 'sbtest2'...
```

```
Inserting 2000000 records into 'sbtest2'
```

```
Creating table 'sbtest1'...
```

```
Inserting 2000000 records into 'sbtest1'
```

```
Creating a secondary index on 'sbtest2'...
```

```
Creating a secondary index on 'sbtest1'...
```

准备脚本 (batch_ddl.sh)

```
-- 该脚本的作用是对测试表进行持续的 DDL 操作（增加/删除字段，模拟客户的业务变更）
dmp2 (master) ~/script# echo > batch_ddl.sh
dmp2 (master) ~/script# cat << EOF > batch_ddl.sh
> #!/bin/bash
> echo "alter table sbtest1 add sid varchar(32);" | /data/mysql/base/5.7.36/bin/mysql -h10.186.60.68 -P3332 -uzlm -pzlm sbtest
> sleep 1
> echo "alter table sbtest2 add sid varchar(32);" | /data/mysql/base/5.7.36/bin/mysql -h10.186.60.68 -P3332 -uzlm -pzlm sbtest
> sleep 10
> echo "alter table sbtest1 drop sid;" | /data/mysql/base/5.7.36/bin/mysql -h10.186.60.68 -P3332 -uzlm -pzlm sbtest
> sleep 1
> echo "alter table sbtest2 drop sid;" | /data/mysql/base/5.7.36/bin/mysql -h10.186.60.68 -P3332 -uzlm -pzlm sbtest
> EOF
```

场景 1：备份时并发执行 DDL (无额外参数)

执行脚本

```
dmp2 (master) ~/script# while true; do bash batch_ddl.sh; done
mysql: [Warning] Using a password on the command line interface can be insecure.
mysql: [Warning] Using a password on the command line interface can be insecure.
... 略
```

验证脚本是否生效（观察测试表的 sid 字段，时而有，时而无）

```
zlm@10.186.60.68 [sbtest]> show create table sbtest1\G
***** 1. row *****
      Table: sbtest1
Create Table: CREATE TABLE `sbtest1` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `k` int(11) NOT NULL DEFAULT '0',
  `c` char(120) NOT NULL DEFAULT '',
  `pad` char(60) NOT NULL DEFAULT '',
  PRIMARY KEY (`id`),
  KEY `k_1` (`k`)
) ENGINE=InnoDB AUTO_INCREMENT=2000001 DEFAULT CHARSET=utf8
1 row in set (0.00 sec)

zlm@10.186.60.68 [sbtest]> show create table sbtest1\G
***** 1. row *****
      Table: sbtest1
Create Table: CREATE TABLE `sbtest1` (
```

```

`id` int(11) NOT NULL AUTO_INCREMENT,
`k` int(11) NOT NULL DEFAULT '0',
`c` char(120) NOT NULL DEFAULT '',
`pad` char(60) NOT NULL DEFAULT '',
`sid` varchar(32) DEFAULT NULL,
PRIMARY KEY (`id`),
KEY `k_1` (`k`)
) ENGINE=InnoDB AUTO_INCREMENT=2000001 DEFAULT CHARSET=utf8
1 row in set (0.00 sec)

```

执行 Xtrabackup 备份

```

-- 检查 Xtrabackup 的版本（客户生产环境为 2.4.5）
dmp2 /data/urman-agent/bin# ./xtrabackup --version
xtrabackup: recognized server arguments: --datadir=/var/lib/mysql
./xtrabackup version 2.4.20 based on MySQL server 5.7.26 Linux (x86_64) (revision id:
c8b4056)

-- 执行备份
./xtrabackup --defaults-file=/data/mysql/mysql3332/my.cnf \
--user=zlm --password=zlm --target-dir=/data/backup/%date +%F_%H-%M-%S` \
--backup --host=127.0.0.1 --port=3332 --parallel 4 \
--check-privileges --no-version-check
... 略
220512 15:11:11 [01] Copying ./sbtest/db.opt to /data/backup/2022-05-12_15-10-
56/sbtest/db.opt
220512 15:11:11 [01]          ...done
220512 15:11:11 [01] Copying ./sbtest/sbtest2.frm to /data/backup/2022-05-12_15-10-
56/sbtest/sbtest2.frm
220512 15:11:11 [01]          ...done
220512 15:11:11 [01] Copying ./sbtest/sbtest1.frm to /data/backup/2022-05-12_15-10-
56/sbtest/sbtest1.frm
220512 15:11:11 [01]          ...done
220512 15:11:11 Finished backing up non-InnoDB tables and files
220512 15:11:11 [00] Writing /data/backup/2022-05-12_15-10-56/xtrabackup_binlog_info
220512 15:11:11 [00]          ...done
220512 15:11:11 Executing FLUSH NO_WRITE_TO_BINLOG ENGINE LOGS...
xtrabackup: The latest check point (for incremental): '250912794909'
xtrabackup: Stopping log copying thread.
.InnoDB: Last flushed lsn: 250912650797 load_index lsn 250913206547
InnoDB: An optimized (without redo logging) DDL operation has been performed. All
modified pages may not have been flushed to the disk yet.
PXB will not be able to make a consistent backup. Retry the backup operation
dmp2 /data/urman-agent/bin#

```

```
## 以上步骤，直接复现了客户生产环境的故障场景
```

终止脚本

```
mysql: [Warning] Using a password on the command line interface can be insecure.
mysql: [Warning] Using a password on the command line interface can be insecure.
mysql: [Warning] Using a password on the command line interface can be insecure.
mysql: [Warning] Using a password on the command line interface can be insecure.
mysql: [Warning] Using a password on the command line interface can be insecure.
mysql: [Warning] Using a password on the command line interface can be insecure.
^C^C -- query aborted
ERROR 1317 (70100) at line 1: Query execution was interrupted
```

小结

默认情况下，即使是 Xtrabackup 高版本，如果备份时并发执行 DDL，并且没有指定 DDL 锁参数（--lock-ddl, --lock-ddl-per-table），会导致备份失败

场景 2：备份时并发执行 DDL (加--lock-ddl 参数)

执行脚本

```
dmp2 (master) ~/script# while true; do bash batch_ddl.sh; done
mysql: [Warning] Using a password on the command line interface can be insecure.
mysql: [Warning] Using a password on the command line interface can be insecure.
... 略
```

执行 Xtrabackup 备份

```
dmp2 /data/urman-agent/bin# ./xtrabackup --defaults-file=/data/mysql/mysql3332/my.cnf -
-user=zlm --password=zlm --target-dir=/data/backup/`date +%F_%H-%M-%S` --backup --
host=127.0.0.1 --port=3332 --parallel 4 --check-privileges --no-version-check --lock-
ddl
xtrabackup: recognized server arguments: --datadir=/data/mysql/mysql3332/data --
tmpdir=/data/mysql/mysql3332/tmp --open_files_limit=65535 --server-id=31782647 --
log_bin=/data/mysql/mysql3332/logs/mysql-bin --innodb_buffer_pool_size=100M --
innodb_data_file_path=ibdata1:100M:autoextend --innodb_flush_log_at_trx_commit=1 --
innodb_log_buffer_size=8M --innodb_log_file_size=100M --innodb_log_files_in_group=3 --
innodb_max_dirty_pages_pct=50 --innodb_file_per_table=1 --innodb_io_capacity=2000 --
innodb_flush_method=O_DIRECT --innodb_flush_log_at_trx_commit=1 --parallel=4
xtrabackup: recognized client arguments: --port=3332 --socket=/tmp/mysql3332.sock --
user=zlm --password=* --target-dir=/data/backup/2022-05-12_16-02-37 --backup=1 --
host=127.0.0.1 --port=3332 --check-privileges=1 --no-version-check=1 --lock-ddl=1
220512 16:02:37 Connecting to MySQL server host: 127.0.0.1, user: zlm, password: set,
port: 3332, socket: /tmp/mysql3332.sock
Using server version 5.7.36-log
220512 16:02:37 Error: LOCK TABLES FOR BACKUP is not supported.
```

```
## MySQL 社区版不支持--lock-ddl 参数，直接报错
```

小结

lock-ddl 其实就是在进行备份的时候，执行备份锁（LOCK TABLES FOR BACKUP）来阻塞 DDL 语句，由于只有 Percona Server 支持该语法，MySQL 社区版暂时无法模拟，MariaDB 分支也同样不支持该语法

场景 3：备份时并发执行 DDL (加--lock-ddl-per-table 参数)

执行脚本

```
dmp2 (master) ~/script# while true; do bash batch_ddl.sh; done
mysql: [Warning] Using a password on the command line interface can be insecure.
mysql: [Warning] Using a password on the command line interface can be insecure.
... 略
```

执行 Xtrabackup 备份

```
dmp2 /data/urman-agent/bin# ./xtrabackup --defaults-file=/data/mysql/mysql3332/my.cnf -
-user=zlm --password=zlm --target-dir=/data/backup/`date +%F_%H-%M-%S` --backup --
host=127.0.0.1 --port=3332 --parallel 4 --check-privileges --no-version-check --lock-
ddl-per-table
xtrabackup: recognized server arguments: --datadir=/data/mysql/mysql3332/data --
tmpdir=/data/mysql/mysql3332/tmp --open_files_limit=65535 --server-id=31782647 --
log_bin=/data/mysql/mysql3332/logs/mysql-bin --innodb_buffer_pool_size=100M --
innodb_data_file_path=ibdata1:100M:autoextend --innodb_flush_log_at_trx_commit=1 --
innodb_log_buffer_size=8M --innodb_log_file_size=100M --innodb_log_files_in_group=3 --
innodb_max_dirty_pages_pct=50 --innodb_file_per_table=1 --innodb_io_capacity=2000 --
innodb_flush_method=O_DIRECT --innodb_flush_log_at_trx_commit=1 --parallel=4
xtrabackup: recognized client arguments: --port=3332 --socket=/tmp/mysql3332.sock --
user=zlm --password=* --target-dir=/data/backup/2022-05-12_16-07-12 --backup=1 --
host=127.0.0.1 --port=3332 --check-privileges=1 --no-version-check=1 --lock-ddl-per-
table=1
220512 16:07:12 Connecting to MySQL server host: 127.0.0.1, user: zlm, password: set,
port: 3332, socket: /tmp/mysql3332.sock
Using server version 5.7.36-log
./xtrabackup version 2.4.20 based on MySQL server 5.7.26 Linux (x86_64) (revision id:
c8b4056)
xtrabackup: uses posix_fadvise().
xtrabackup: cd to /data/mysql/mysql3332/data
xtrabackup: open files limit requested 65535, set to 65535
xtrabackup: using the following InnoDB configuration:
xtrabackup:   innodb_data_home_dir = .
xtrabackup:   innodb_data_file_path = ibdata1:100M:autoextend
xtrabackup:   innodb_log_group_home_dir = ./
```

```

xtrabackup: innodb_log_files_in_group = 3
xtrabackup: innodb_log_file_size = 104857600
xtrabackup: using O_DIRECT
InnoDB: Number of pools: 1
220512 16:07:13 >> log scanned up to (250920385317)
xtrabackup: Generating a list of tablespaces
... 略
220512 16:07:14 Locking MDL for `sbtest`.`sbtest1`  ## 备份 sbtest1 表数据前, 先加上 MDL 锁
220512 16:07:14 [04] Copying ./sbtest/sbtest1.ibd to /data/backup/2022-05-12_16-07-12/sbtest/sbtest1.ibd
220512 16:07:14 [02] Copying ./sbtest/sbtest2.ibd to /data/backup/2022-05-12_16-07-12/sbtest/sbtest2.ibd
220512 16:07:15 >> log scanned up to (250920494721)
220512 16:07:16 >> log scanned up to (250920494832)
220512 16:07:17 >> log scanned up to (250920494841)
220512 16:07:18 >> log scanned up to (250920494841)
220512 16:07:19 [02] ...done
220512 16:07:19 >> log scanned up to (250920494841)
220512 16:07:20 >> log scanned up to (250920494841)
220512 16:07:20 [01] ...done
220512 16:07:20 [04] ...done
220512 16:07:21 Starting to backup non-InnoDB tables and files
... 略
220512 16:07:21 [01] Copying ./sbtest/sbtest2.frm to /data/backup/2022-05-12_16-07-12/sbtest/sbtest2.frm
220512 16:07:21 [01] ...done
220512 16:07:21 [01] Copying ./sbtest/sbtest1.frm to /data/backup/2022-05-12_16-07-12/sbtest/sbtest1.frm
220512 16:07:21 [01] ...done
220512 16:07:21 Finished backing up non-InnoDB tables and files
220512 16:07:21 [00] Writing /data/backup/2022-05-12_16-07-12/xtrabackup_binlog_info
220512 16:07:21 [00] ...done
220512 16:07:21 Executing FLUSH NO_WRITE_TO_BINLOG ENGINE LOGS...
xtrabackup: The latest check point (for incremental): '250920494832'
xtrabackup: Stopping log copying thread.
.220512 16:07:21 >> log scanned up to (250920494841)

220512 16:07:21 Executing UNLOCK TABLES
220512 16:07:21 All tables unlocked
220512 16:07:21 [00] Copying ib_buffer_pool to /data/backup/2022-05-12_16-07-12/ib_buffer_pool
220512 16:07:21 [00] ...done
220512 16:07:21 Backup created in directory '/data/backup/2022-05-12_16-07-12/'
MySQL binlog position: filename 'mysql-bin.000012', position '472261259', GTID of the

```

```

last change '0e795785-0ee5-11ec-9350-02000aba3c44:1-1619'
220512 16:07:21 [00] Writing /data/backup/2022-05-12_16-07-12/backup-my.cnf
220512 16:07:21 [00]      ...done
220512 16:07:21 [00] Writing /data/backup/2022-05-12_16-07-12/xtrabackup_info
220512 16:07:21 [00]      ...done
220512 16:07:21 Unlocking MDL for all tablesxtrabackup: Transaction log of lsn
(250920249371) to (250920494841) was copied.  ## 释放所有表上的 MDL 锁
220512 16:07:22 completed OK!

```

终止脚本

```

... 略
mysql: [Warning] Using a password on the command line interface can be insecure.
^C^C -- query aborted
ERROR 1317 (70100) at line 1: Query execution was interrupted
mysql: [Warning] Using a password on the command line interface can be insecure.
^C^C -- query aborted
ERROR 1317 (70100) at line 1: Query execution was interrupted
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 1060 (42S21) at line 1: Duplicate column name 'sid'
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 1060 (42S21) at line 1: Duplicate column name 'sid'

^C
dmp2 (master) ~/script#

```

小结

备份时使用 `--lock-ddl-per-table` 参数，会在拷贝每个表的 `ibd` 文件前先加一个 MDL 锁，用于阻塞该表上 DDL 语句的执行，直到备份完成后，DDL 才能继续执行

场景 4：备份时并发执行 DDL（设置 MySQL 参数：`old_alter_table=1`）

关闭 ONLINE DDL 特性

```

zlm@10.186.60.68 [(none)]> set global old_alter_table=1;
Query OK, 0 rows affected (0.00 sec)

zlm@10.186.60.68 [(none)]> show variables like 'old_alter_table';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| old_alter_table | OFF   |
+-----+-----+
1 row in set (0.00 sec)

```



```
zlm@10.186.60.68 [(none)]> exit
Bye
dmp2 (master) ~/script# m3332 -Nse "show variables like 'old_alter_table';"
mysql: [Warning] Using a password on the command line interface can be insecure.
old_alter_table ON
```

执行脚本

```
dmp2 (master) ~/script# while true; do bash batch_ddl.sh; done
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 1060 (42S21) at line 1: Duplicate column name 'sid'
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 1060 (42S21) at line 1: Duplicate column name 'sid'
mysql: [Warning] Using a password on the command line interface can be insecure.
mysql: [Warning] Using a password on the command line interface can be insecure.
... 略
```

执行 Xtrabackup 备份

```
dmp2 /data/urman-agent/bin# ./xtrabackup --defaults-file=/data/mysql/mysql3332/my.cnf -
-user=zlm --password=zlm --target-dir=/data/backup/`date +%F_%H-%M-%S` --backup --
host=127.0.0.1 --port=3332 --parallel 4 --check-privileges --no-version-check
xtrabackup: recognized server arguments: --datadir=/data/mysql/mysql3332/data --
tmpdir=/data/mysql/mysql3332/tmp --open_files_limit=65535 --server-id=31782647 --
log_bin=/data/mysql/mysql3332/logs/mysql-bin --innodb_buffer_pool_size=100M --
innodb_data_file_path=ibdata1:100M:autoextend --innodb_flush_log_at_trx_commit=1 --
innodb_log_buffer_size=8M --innodb_log_file_size=100M --innodb_log_files_in_group=3 --
innodb_max_dirty_pages_pct=50 --innodb_file_per_table=1 --innodb_io_capacity=2000 --
innodb_flush_method=O_DIRECT --innodb_flush_log_at_trx_commit=1 --parallel=4
xtrabackup: recognized client arguments: --port=3332 --socket=/tmp/mysql3332.sock --
user=zlm --password=* --target-dir=/data/backup/2022-05-12_16-48-00 --backup=1 --
host=127.0.0.1 --port=3332 --check-privileges=1 --no-version-check=1
220512 16:48:00 Connecting to MySQL server host: 127.0.0.1, user: zlm, password: set,
port: 3332, socket: /tmp/mysql3332.sock
Using server version 5.7.36-log
./xtrabackup version 2.4.20 based on MySQL server 5.7.26 Linux (x86_64) (revision id:
c8b4056)
xtrabackup: uses posix_fadvise().
xtrabackup: cd to /data/mysql/mysql3332/data
xtrabackup: open files limit requested 65535, set to 65535
xtrabackup: using the following InnoDB configuration:
xtrabackup:   innodb_data_home_dir = .
xtrabackup:   innodb_data_file_path = ibdata1:100M:autoextend
xtrabackup:   innodb_log_group_home_dir = ./
xtrabackup:   innodb_log_files_in_group = 3
xtrabackup:   innodb_log_file_size = 104857600
```

```
xtrabackup: using O_DIRECT
InnoDB: Number of pools: 1
220512 16:48:02 >> log scanned up to (251659945145)
xtrabackup: Generating a list of tablespaces
... 略
220512 16:48:03 [02] Copying ./zlm/t.ibd to /data/backup/2022-05-12_16-48-00/zlm/t.ibd
220512 16:48:03 [03]      ...done
220512 16:48:03 [02]      ...done
220512 16:48:03 [03] Copying ./sbtest/sbtest2.ibd to /data/backup/2022-05-12_16-48-00/sbtest/sbtest2.ibd
220512 16:48:03 [04] Copying ./zlm1/t1.ibd to /data/backup/2022-05-12_16-48-00/zlm1/t1.ibd
220512 16:48:03 [02] Copying ./sbtest/sbtest1.ibd to /data/backup/2022-05-12_16-48-00/sbtest/sbtest1.ibd
220512 16:48:03 [04]      ...done
220512 16:48:03 [04] Copying ./sbtest/#sql-c61_30133.ibd to /data/backup/2022-05-12_16-48-00/sbtest/#sql-c61_30133.ibd
220512 16:48:04 >> log scanned up to (251669706184)
220512 16:48:04 [04]      ...done
220512 16:48:05 >> log scanned up to (251672429294)
220512 16:48:06 >> log scanned up to (251675700622)
220512 16:48:07 >> log scanned up to (251678520383)
220512 16:48:08 >> log scanned up to (251681195105)
220512 16:48:09 [02]      ...done
220512 16:48:09 >> log scanned up to (251683273712)
220512 16:48:09 [01]      ...done
220512 16:48:10 >> log scanned up to (251694062575)
220512 16:48:11 >> log scanned up to (251707703017)
220512 16:48:12 [03]      ...done
220512 16:48:12 >> log scanned up to (251713134341)
220512 16:48:12 Executing FLUSH NO_WRITE_TO_BINLOG TABLES...
220512 16:48:13 >> log scanned up to (251721786959)
... 略
220512 16:48:38 [01] Copying ./sbtest/sbtest1.frm to /data/backup/2022-05-12_16-48-00/sbtest/sbtest1.frm
220512 16:48:38 [01]      ...done
220512 16:48:38 [01] Copying ./sbtest/sbtest2.frm to /data/backup/2022-05-12_16-48-00/sbtest/sbtest2.frm
220512 16:48:38 [01]      ...done
220512 16:48:38 Finished backing up non-InnoDB tables and files
220512 16:48:38 [00] Writing /data/backup/2022-05-12_16-48-00/xtrabackup_binlog_info
220512 16:48:38 [00]      ...done
220512 16:48:38 Executing FLUSH NO_WRITE_TO_BINLOG ENGINE LOGS...
xtrabackup: The latest check point (for incremental): '251977260800'
```

```

xtrabackup: Stopping log copying thread.
.220512 16:48:38 >> log scanned up to (252017389382)

220512 16:48:38 Executing UNLOCK TABLES
220512 16:48:38 All tables unlocked
220512 16:48:38 [00] Copying ib_buffer_pool to /data/backup/2022-05-12_16-48-00/ib_buffer_pool
220512 16:48:38 [00] ...done
220512 16:48:38 Backup created in directory '/data/backup/2022-05-12_16-48-00/'
MySQL binlog position: filename 'mysql-bin.000012', position '472262307', GTID of the
last change '0e795785-0ee5-11ec-9350-02000aba3c44:1-1625'
220512 16:48:38 [00] Writing /data/backup/2022-05-12_16-48-00/backup-my.cnf
220512 16:48:38 [00] ...done
220512 16:48:38 [00] Writing /data/backup/2022-05-12_16-48-00/xtrabackup_info
220512 16:48:38 [00] ...done
xtrabackup: Transaction log of lsn (251552488790) to (252017389382) was copied.
220512 16:48:38 completed OK!
dmp2 /data/urman-agent/bin#

```

Xtrabackup 备份日志中没有发现 MDL 锁，备份也顺利地执行完了，说明启用 `old_alter_table` 也能够解决备份失败的问题

终止脚本

```

... 略
mysql: [Warning] Using a password on the command line interface can be insecure.
^C^C -- query aborted
ERROR 1317 (70100) at line 1: Query execution was interrupted
mysql: [Warning] Using a password on the command line interface can be insecure.
^C^C -- query aborted
ERROR 1317 (70100) at line 1: Query execution was interrupted
^C
04:48 PM dmp2 (master) ~/script#

```

小结

当关闭 ONLINE DDL 特性时 (`old_alter_table=1`)，即执行 DDL 时强制其都采用 `ALGORITHM=COPY` 来代替 `ALGORITHM=INPLACE`，此时即使备份时没有使用 DDL 锁的参数，也不会影响备份执行，但需要注意的是，关闭 ONLINE DDL 特性会影响表上的 DML 并发，不推荐使用。

总结

DDL 业务变更操作应尽量避免备份窗口

Sorted Index Builds 对创建索引的优化与 PXB 2.4.x 备份存在冲突，需要替换为 PXB 高版本，并通过加备份锁或 MDL 锁的参数来避免备份失败

MySQL 自身也可以通过关闭 ONLINE DDL 特性来解决备份失败的问题，但是会影响 DML 并发，需权衡利弊

参考链接

<https://dev.mysql.com/doc/refman/5.7/en/sorted-index-builds.html>
<https://dev.mysql.com/doc/refman/5.7/en/innodb-online-ddl-operations.html#online-ddl-column-operations>
https://dev.mysql.com/doc/refman/5.7/en/server-system-variables.html#sysvar_old_alter_table

3.11 从 data_free 异常说起

作者：杨奇龙

一、前言

某个客户反馈查询数据库发现 `information_schema.tables` 的 `data_free` 值突发异常，达到 13G 左右。如图：

```
mysql> select table_schema db, table_name, data_free, engine from information_schema.tables where table_schema in ('information_schema', 'mysql') and data_free > 0;
```

db	table_name	data_free	engine
information_schema	COLUMNS	12914262016	InnoDB
information_schema	EVENTS	12914262016	InnoDB
information_schema	OPTIMIZER_TRACE	12914262016	InnoDB
information_schema	PARAMETERS	12914262016	InnoDB
information_schema	PARTITIONS	12914262016	InnoDB
information_schema	PLUGINS	12914262016	InnoDB
information_schema	PROCESSLIST	12914262016	InnoDB
information_schema	ROUTINES	12914262016	InnoDB
information_schema	TRIGGERS	12914262016	InnoDB
information_schema	VIEWS	12914262016	InnoDB
information_schema	PERF_STATISTICS	12914262016	InnoDB
mysql	help_topic	4194304	InnoDB
mysql	time_zone_transition	4194304	InnoDB

13 rows in set (0.01 sec)

需要排查什么原因导致的，本文梳理排查的过程和和解决问题的方法。

二、排查

2.1 分析

首先 `data_free` 的含义是 表空间 `ibd` 文件经过写入和删除之后，留下的没有回收的碎片空间大小。让现场的同学同时检查主备库，对比有没有文件大小和配置上的差异。发现主库的 `data_free` 值是 13G 左右，备库正常。

```
mysql> select table_schema db, table_name, data_free, engine from information_schema.tables where table_schema in ('information_schema', 'mysql') and data_free > 0;
```

db	table_name	data_free	engine
information_schema	COLUMNS	8388608	InnoDB
information_schema	EVENTS	8388608	InnoDB
information_schema	OPTIMIZER_TRACE	8388608	InnoDB
information_schema	PARAMETERS	8388608	InnoDB
information_schema	PARTITIONS	8388608	InnoDB
information_schema	PLUGINS	8388608	InnoDB
information_schema	PROCESSLIST	8388608	InnoDB
information_schema	ROUTINES	8388608	InnoDB
information_schema	TRIGGERS	8388608	InnoDB
information_schema	VIEWS	8388608	InnoDB
information_schema	PERF_STATISTICS	8388608	InnoDB
mysql	help_topic	4194304	InnoDB
mysql	time_zone_transition	4194304	InnoDB

13 rows in set (0.02 sec)

备库

看结果猜测和主库上的某些请求动作有关，空洞是 MySQL 因为 sql 写入而请求分配的空间没有自动回收的结果。基于前线给的信息，没有其他思路，再看前线发的截图：

```
#du -sh *
```

4.0K	ib_buffer_pool
200M	ibdata1
1001M	ib_logfile0
1001M	ib_logfile1
13G	ibtmp1
26M	master-error.log
4.0K	master-log-bin.index
141M	mysql-bin.000841
5.4M	mysql-bin.000842
4.0K	slave-relay.002172
4.0K	slave-relay.002173
4.0K	slave-relay-log.index
108K	slow_query.log

意外从 截图的 ibtmp1 文件大小找到一些线索，截图显示 ibtmp1 文件大小也是 13G，备库则是初始值大小。

忽略红色的箭头，查看 ibtmp1 文件大小为 13G，似乎有些头绪，data_free 是否和 ibtmp1 有关。

2.2 验证猜想

使用 sysbench 创建测试表 sbtest1，构造 2w 条记录，然后创建 sbtest2，将 sbtest1 的数据 导入到 sbtest2。为何这么操作，后面会说明。

```
mysql > show variables like 'innodb_temp_data_file_path';
```

Variable_name	Value
innodb_temp_data_file_path	ibtmp1:12M:autoextend

1 row in set (0.00 sec)

查看物理 ibtmp1 文件大小：

```
[root@tidb00 data]# du -sm ibtmp1
12 ibtmp1
```

沟通测试用例，让系统自动生成临时表

```
mysql > create table sbtest2 like sbtest1;
Query OK, 0 rows affected (0.01 sec)
mysql > insert into sbtest2(k,c,pad) select k,c ,pad from sbtest1;
Query OK, 200000 rows affected (1.18 sec)
Records: 200000 Duplicates: 0 Warnings: 0
mysql > insert into sbtest2(k,c,pad) select k,c ,pad from sbtest1;
Query OK, 200000 rows affected (1.06 sec)
mysql > insert into sbtest2(k,c,pad) select k,c ,pad from sbtest2;
Query OK, 400000 rows affected (2.49 sec)
Records: 400000 Duplicates: 0 Warnings: 0
mysql > insert into sbtest2(k,c,pad) select k,c ,pad from sbtest2;
Query OK, 800000 rows affected (6.18 sec)
Records: 800000 Duplicates: 0 Warnings: 0
```

再次检查 ibtmp1 文件大小 204MB

```
[root@tidb00 data]# du -sm ibtmp1
204 ibtmp1
```

```
mysql > SELECT FILE_NAME, TABLESPACE_NAME, ENGINE, INITIAL_SIZE, TOTAL_EXTENTS*EXTENT_S
IZE
      ->      AS TotalSizeBytes, DATA_FREE, MAXIMUM_SIZE FROM INFORMATION_SCHEMA.FILES
      ->      WHERE TABLESPACE_NAME = 'innodb_temporary'\G
***** 1. row *****
      FILE_NAME: ./ibtmp1
TABLESPACE_NAME: innodb_temporary
      ENGINE: InnoDB
      INITIAL_SIZE: 12582912
TotalSizeBytes: 213909504
      DATA_FREE: 207618048 ## 和物理文件大小对应
      MAXIMUM_SIZE: NULL
1 row in set (0.00 sec)
```

查看 `I_S.tables` 的 `data_free` 的值:

```
mysql [localhost:5731] {msandbox} (test) > select table_schema, db, table_name, data_free, engine from information_schema.tables where table_schema in ('information_schema');
```

db	table_name	data_free	engine
information_schema	CHARACTER_SETS	0	MEMORY
information_schema	COLLATIONS	0	MEMORY
information_schema	COLLATION_CHARACTER_SET_APPLICABILITY	0	MEMORY
information_schema	COLUMNS	207618048	InnoDB
information_schema	COLUMN_PRIVILEGES	0	MEMORY
information_schema	ENGINES	0	MEMORY
information_schema	EVENTS	207618048	InnoDB
information_schema	FILES	0	MEMORY
information_schema	GLOBAL_STATUS	0	MEMORY
information_schema	GLOBAL_VARIABLES	0	MEMORY
information_schema	KEY_COLUMN_USAGE	0	MEMORY
information_schema	OPTIMIZER_TRACE	207618048	InnoDB
information_schema	PARAMETERS	207618048	InnoDB
information_schema	PARTITIONS	207618048	InnoDB
information_schema	PLUGINS	207618048	InnoDB
information_schema	PROCESSLIST	207618048	InnoDB
information_schema	PROFILING	0	MEMORY
information_schema	REFERENTIAL_CONSTRAINTS	0	MEMORY
information_schema	ROUTINES	207618048	InnoDB

1 能够复现客户的问题 I_S.tables data_free 显示临时表空间的大小

2 207618048 和物理文件大小接近

查看 insert select from table 在执行过程中的确使用了临时表。

```
mysql > explain insert into sbtest2(k,c,pad) select k,c ,pad from sbtest2\G
***** 1. row *****
. .
***** 2. row *****

      id: 1
  select_type: SIMPLE
        table: sbtest2
   partitions: NULL
         type: ALL
possible_keys: NULL
          key: NULL
       key_len: NULL
         ref: NULL
        rows: 1578168
   filtered: 100.00
    Extra: Using temporary  ##
2 rows in set (0.00 sec)

Records: 200000 Duplicates: 0 Warnings: 0
```

至此，可以确定客户的实例因为执行某些 SQL 过程中占用系统临时表空间，使用完之后临时表空间并未被回收导致开头的问题。接下来我们详细了解 MySQL 临时表的相关知识。

三、临时表空间

3.1 介绍

ibtmp1 是非压缩的 innodb 临时表的独立表空间，通过 `innodb temp data file path` 参数指定文件的路径，文件名和大小，默认配置为 `ibtmp1:12M:autoextend`，如果没有指定位置，临时表空间会被创建到 `innodb_data_home_dir` 指定的路径。

需要注意的是：按照默认值，这个文件大小是可以无限增长的。而且 5.7 版本并不会随着 SQL 语句结束主动回收该临时表空间，导致空间资源不足的安全风险。

3.2 什么情况下会用到临时表

当 explain 查看执行计划结果的 extra 列中，如果包含 Using Temporary 就表示会用到临时表,例如如下几种常见的情况通常就会用到：

- insert into tab1 select ... from tab2 。
- group by 无索引字段或 group by order by 的字段不一样。
- distinct 的值和 group by 的值不一样，无法利用稀疏索引。

其他的欢迎补充。

3.3 临时表相关的参数和元数据

5.7 版本:

```
innodb_temp_data_file_path
default_tmp_storage_engine
internal_tmp_disk_storage_engine
```

8.0 版本分为会话级和全局级临时表空间

```
innodb_temp_tablespaces_dir #指定会话级创建临时表到 BASEDIR/data/#innodb_temp
innodb_temp_data_file_path # 全局变量
internal_tmp_disk_storage_engine
```

用户自己创建的临时表可以通过查询 INFORMATION_SCHEMA.INNODB_TEMP_TABLE_INFO

```
mysql > CREATE TEMPORARY TABLE t1 (c1 INT PRIMARY KEY) ENGINE=INNODB;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql > SELECT * FROM INFORMATION_SCHEMA.INNODB_TEMP_TABLE_INFO\G
***** 1. row *****
      TABLE_ID: 54
      NAME: #sqlfd5_b_0
      N_COLS: 4
      SPACE: 36
PER_TABLE_TABLESPACE: FALSE
      IS_COMPRESSED: FALSE
1 row in set (0.00 sec)
```

MySQL 在执行 sql 过程中被优化器创建的表，则无法通过 INFORMATION_SCHEMA.INNODB_TEMP_TABLE_INFO 直接查看。比如本文的案例。

3.4 怎么解决 ibtmp1 文件空间占用的问题

- 万能的重启大法，找个合适的时间，切换数据库，重启老的主库。
- 通过配置 innodb_temp_data_file_path 控制 ibtmp1 文件的最大值，避免表空间大小无限增加。

```
innodb_temp_data_file_path = ibtmp1:12M:autoextend:max:10G
```


12M 是文件的初始大小，10G 是文件的最大值，超过最大值则系统会提示报错

```
ERROR 1114 (HY000): The table '/data/msb_5_7_31/tmp#sql_XXXXX_0' is full
```

参考文章

<https://dev.mysql.com/doc/refman/5.7/en/innodb-temporary-tablespace.html>

<https://dev.mysql.com/doc/refman/8.0/en/innodb-temporary-tablespace.html>

3.12 show processlist 引起的性能问题

作者：王祥

背景信息

业务监控发现交易的平均响应时间比之前慢了近一倍，需要排查一下数据库是不是响应慢了。生产 MySQL 版本为 8.0.18，一主三从半同步复制。

故障分析

首先对比查看了交易正常时段与出现异常的时段各项监控指标（cpu、qps、tps、磁盘 IO 等）都未发现明显的变化。接下来查看 slow log 发现了较多的慢 SQL，而且是普通的 insert 语句，执行时长超过 1 秒。进一步观察对比发现，每次 insert 慢都是出现在同一秒，insert 慢语句条数基本在 30 条左右，而且出现的间隔都是两分钟或两分钟的倍数。根据这个规律第一感觉是不是定时任务引起的问题。经过对定时任务的排查最终定位到监控脚本，监控脚本为两分钟执行一次。接下来需要排查一下，具体是哪部分导致 insert 慢。为了快速复现问题，直接在一个从库上使用 mysqlslap 进行压测。从业务那得知问题 insert 语句每秒会有 60-80 次的写入量，压测语句如下：

```
mysqlslap -h127.0.0.1 -uroot -p --concurrency=80 --iterations=10 --create-
schema=userdb --query=/root/test.sql --engine=innodb --number-of-queries=50000

#test.sql
insert into userdb.ps (clo1, clo2, clo3, clo4, clo4, clo5, clo6) values (substring(MD5(
RAND()),1,20), 'ffffffddddd', '0', '', 'aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaddddd', '2022-06-17 16:00:38.145', 34);
```

在压测期间执行监控脚本，一边查看 slow log，能稳定复现生产的现象。通过排除法，最终定位到几个使用 information_schema.processlist 表的语句导致了 insert 慢。那 information_schema.processlist 为什么会慢呢？带着这个问题去查看一下官方对 information_schema.processlist 的描述。

The default SHOW PROCESSLIST implementation iterates across active threads from within

the thread manager while holding a global mutex. This has negative performance consequences, particularly on busy systems. The alternative SHOW PROCESSLIST implementation is based on the Performance Schema processlist table. This implementation queries active thread data from the Performance Schema rather than the thread manager and does not require a mutex.

根据官方的说明：在使用默认的 show processlist 会持有全局互斥锁，在业务繁忙的系统上会导致性能问题。同时也给出了解决办法，使用 Performance Schema 中的 processlist 代替，此方式不会产生全局互斥锁。

performance_schema_show_processlist 是 MySQL 8.0.22 版本引入的新功能。接下来我们来看看官方对 Performance Schema 中的 processlist 描述。

• performance schema show processlist

Command-Line Format	--performance-schema-show-processlist[={OFF ON}]
Introduced	8.0.22
System Variable	performance_schema_show_processlist
Scope	Global
Dynamic	Yes
SET VAR Hint Applies	No
Type	Boolean
Default Value	OFF

The SHOW PROCESSLIST statement provides process information by collecting thread data from all active threads. The performance_schema_show_processlist variable determines which SHOW PROCESSLIST implementation to use:

The default implementation iterates across active threads from within the thread manager while holding a global mutex. This has negative performance consequences, particularly on busy systems.

The alternative SHOW PROCESSLIST implementation is based on the Performance Schema processlist table. This implementation queries active thread data from the Performance Schema rather than the thread manager and does not require a mutex.

如果开启参数 performance_schema_show_processlist，show processlist 使用 Performance Schema 中的 processlist 避免了全局互斥锁的问题，如果不开启该参数则 show processlist 使用 information_schema.processlist 会产生全局锁。

在配置文件[mysqld]下加上 performance_schema_show_processlist=on 配置。配置完成后，查看 performance_schema 下的 processlist。

```
root@localhost:mysql.sock [(none)]> show variables like 'performance_schema_show_proces
```

```

slist';
+-----+-----+
| Variable_name          | Value |
+-----+-----+
| performance_schema_show_processlist | ON    |
+-----+-----+
#信息与 information_schema.processlist 下保持一致
root@localhost:mysql.sock [(none)]> select * from performance_schema.processlist\G
***** 1. row *****
      ID: 5
    USER: event_scheduler
    HOST: localhost
      DB: NULL
COMMAND: Daemon
      TIME: 354
    STATE: Waiting on empty queue
    INFO: NULL
***** 2. row *****
      ID: 8
    USER: root
    HOST: localhost
      DB: NULL
COMMAND: Query
      TIME: 0
    STATE: executing
    INFO: select * from performance_schema.processlist
2 rows in set (0.00 sec)

```

总结

1. 使用 MySQL 8.0.22 之前的版本，在业务繁忙的敏感系统上执行 `show processlist` 需要谨慎。
2. 使用 MySQL 8.0.22 之后版本，可以开启 `performance_schema_show_processlist` 避免该问题。但依旧不建议频繁查询会话信息。

另外查询 `processlist` 表导致 MySQL 实例 `crash` 问题，请参考文章：【MySQL 设置 `terminology_use_previous` 参数导致数据库 Crash】

参考：

<https://dev.mysql.com/doc/refman/8.0/en/performance-schema-processlist-table.html>
https://dev.mysql.com/doc/refman/8.0/en/performance-schema-system-variables.html#sysvar_performance_schema_show_processlist

3.13 MySQL 耗尽主机内存一例分析

作者：付祥

异常现象

开发人员反馈，有一台服务器内存几乎被 MySQL 耗尽了，执行 top 命令，输出如下：

```
top - 10:20:03 up 189 days, 14:44, 3 users, load average: 0.07, 0.13, 0.12
Tasks: 157 total, 1 running, 156 sleeping, 0 stopped, 0 zombie
Cpu0 :  4.8%us,  2.0%sy,  0.0%ni, 93.2%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu1 :  2.0%us,  0.7%sy,  0.0%ni, 97.3%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu2 :  1.4%us,  1.4%sy,  0.0%ni, 97.3%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu3 :  1.3%us,  0.7%sy,  0.0%ni, 98.0%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu4 :  2.0%us,  1.4%sy,  0.0%ni, 96.6%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu5 :  3.7%us,  3.7%sy,  0.0%ni, 92.6%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu6 :  5.7%us,  1.0%sy,  0.0%ni, 93.0%id,  0.0%wa,  0.0%hi,  0.3%si,  0.0%st
Cpu7 :  0.7%us,  1.3%sy,  0.0%ni, 97.7%id,  0.3%wa,  0.0%hi,  0.0%si,  0.0%st
Mem: 33013152k total, 32352880k used, 660272k free, 162232k buffers
Swap: 0k total, 0k used, 0k free, 920472k cached

  PID USER      PR  NI  VIRT  RES  SHR  S %CPU  %MEM    TIME+  COMMAND
 3244 mysql     20   0 31.7g  29g   0  S 20.6  92.8 2460:25 mysqld
 4175 nginx     20   0 254m 140m  556  S  3.7   0.4 3051:53 nginx
24005 root       20   0 455m  53m 1044  S  6.6   0.2 1526:57 sshd:root@
 4180 nginx     20   0 130m  24m   0  S  0.3   0.1 1527:05 nginx
 4183 nginx     20   0 126m  18m  464  S  0.0   0.1 765:11.50 nginx
17617 root       20   0 125m  6364 4876  S  0.0   0.0 0:00.01 mysql
18468 root       20   0  97.8m 6383 5328  S  0.3   0.2 0:00.04 mysql
```

这台机器是个测试环境，MySQL 是开发自己安装的，数据库版本 5.6.51，机器总内存 32G，MySQL 占了 29G。

分析过程

查看 MySQL 启动时间发现 1 个星期前才启动，猜测之前由于主机内存耗尽触发了 OOM，查看 MySQL 错误日志，发现每隔 10 几天，MySQL 就异常关闭一次：

```
2022-02-24 03:03:42 20981 [Note] InnoDB: Database was not shutdown normally!
2022-03-13 02:31:40 4134 [Note] InnoDB: Database was not shutdown normally!
2022-03-31 02:31:08 6846 [Note] InnoDB: Database was not shutdown normally!
2022-04-12 02:31:41 1159 [Note] InnoDB: Database was not shutdown normally!
2022-04-23 04:41:51 6773 [Note] InnoDB: Database was not shutdown normally!
2022-05-04 02:31:52 2499 [Note] InnoDB: Database was not shutdown normally!
2022-05-13 04:56:06 23010 [Note] InnoDB: Database was not shutdown normally!
2022-05-30 02:31:33 3244 [Note] InnoDB: Database was not shutdown normally!
```

查看操作系统日志，进一步验证了 MySQL 耗尽主机内存，触发 OOM：

```
# grep oom-killer /var/log/messages* /var/log/messages-20220605:May 30 02:31:30 vm10-
136-9-24 kernel: mysqld invoked
oom-killer: gfp_mask=0x201da, order=0, oom_score_adj=0
```

查看 my.cnf 配置文件，发现几乎都是默认配置，innodb_buffer_pool_size 等于默认值 128M：

```
mysql> show variables like 'innodb_buffer_pool_size';
```

```
+-----+
| Variable_name | Value |
+-----+
| innodb_buffer_pool_size | 134217728 |
+-----+
```

当前数据库有 500 多个连接，都是 Sleep 状态，从 MySQL 错误日志发现 performance_schema 下的表结构全是错误的，估计 MySQL 版本做了升级，没有执行 upgrade 升级数据字典，意味着一些内存诊断信息便不能从 PS 获取：

```
2022-06-09 11:19:08 27468 [ERROR] Native table
'performance_schema'. 'cond_instances' has the wrong structure
2022-06-09 11:19:08 27468 [ERROR] Native table
'performance_schema'. 'events_waits_current' has the wrong structure
2022-06-09 11:19:08 27468 [ERROR] Native table
'performance_schema'. 'events_waits_history' has the wrong structure
2022-06-09 11:19:08 27468 [ERROR] Native table
.....省略输出.....
mysql> show variables like 'performance_schema'; +-----+
| Variable_name | Value | +-----+
| performance_schema | ON | +-----+
1 row in set (0.00 sec)
mysql> use performance_schema
Database changed
mysql> show tables;
Empty set (0.00 sec)
```

show engine innodb status ，截取内存信息如下：

```
BUFFER POOL AND MEMORY
-----
Total memory allocated 137363456; in additional pool allocated 0
Dictionary memory allocated 736104382
Buffer pool size 8191
Free buffers 1024
Database pages 6851
Old database pages 2508
Modified db pages 0
Pending reads 0
Pending writes: LRU 0, flush list 0, single page 0
Pages made young 1280745, not young 1337956473
1.59 youngs/s, 0.10 non-youngs/s
Pages read 97753005, created 121179, written 1163360
0.06 reads/s, 0.88 creates/s, 12.12 writes/s
Buffer pool hit rate 1000 / 1000, young-making rate 0 / 1000 not 0 / 1000
```

```
Pages read ahead 0.00/s, evicted without access 0.00/s, Random read ahead 0.00/s
LRU len: 6851, unzip_LRU len: 0 I/O sum[597]:cur[0], unzip sum[0]:cur[0]
```

数据字典分配内存竟然达到 700M，通过 `lsf` 命令发现，MySQL 打开了大量的 MyISAM 表分区文件，每个分区占用 2 个文件句柄，这应该是其内存使用高的原因：

```
lsf|grep "#P#"|grep -E "MYD$|MYI$" |wc -l
29826
```

MyISAM 存储引擎，数据块通过操作系统缓存，索引块由 `key buffer` 缓存，大小通过参数 `key_buffer_size` 控制，当前默认值如下：

```
mysql> show variables like 'key_buffer_size';
+-----+-----+
| Variable_name | Value      |
+-----+-----+
| key_buffer_size | 8388608    |
+-----+-----+
1 row in set (0.00 sec)
```

MySQL 默认使用 GLIBC 内存分配器，通过 `gdb` 调用 `malloc_stats()` 函数分析内存使用情况：

```
gdb -ex "call (void) malloc_stats()" --batch -p $(pidof mysqld)
```

上述命令执行完成后，会将内存使用情况打印到 MySQL 错误日志：

```
Arena 0:
system bytes = 2001301504
in use bytes = 250961264
Arena 1:
system bytes = 12181504
in use bytes = 1000800
Arena 2:
system bytes = 164257792
in use bytes = 8032368
Arena 3:
system bytes = 1363267584
in use bytes = 468958176
Arena 4:
system bytes = 335654912
in use bytes = 708240
.....省略输出.....
Arena 63:
system bytes = 26365952
in use bytes = 693104
Total (incl. mmap):
system bytes = 1166893056
in use bytes = 348358880
```

```
max mmap regions = 55
max mmap bytes = 1919492096
```

将上述输出存入临时文件 /tmp/fx.txt，计算分配给 MySQL 内存和 MySQL 使用内存如下：

```
# awk '{if($1 == "system") total+=$NF; else if ($1 == "in") used+=$NF }END{print
total/1024/1024/1024,used/1024/1024/1024}' /tmp/fx.txt
28.4044 2.87976
```

分配给 MySQL 内存 28.4G，MySQL 使用内存才 2.8G，表明内存碎片化太严重了，使用 google 内存分配器 tcmalloc 重启 MySQL 十几天后，top 命令观察 MySQL 占用内存稳定在 5G：

```
[root@localhost ~]# top -p `pidof mysqld`
top - 13:54:23 up 202 days, 18:19, 1 user, load average: 0.32, 0.17, 0.11
Tasks: 1 total, 0 running, 1 sleeping, 0 stopped, 0 zombie
Cpu(s): 0.5%us, 0.6%sy, 0.0%ni, 99.0%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st
Mem: 33013152k total, 32694152k used, 319000k free, 534444k buffers
Swap: 0k total, 0k used, 0k free, 25819092k cached

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
 27468 mysql    20   0 5814m 5.2g  14m S   3.7  16.5   2991:02 mysqld
```

小结

jemalloc 和 tcmalloc 之类的内存分配器在一些场景，尤其是在多核 CPU 和高并发工作负载下能提供更高效的性能，例如我们熟知的 mongodb 使用 tcmalloc，redis 使用 jemalloc。

3.14 一条 SELECT 语句跑崩了 MySQL，怎么回事？

作者：刘开洋

在很多疑难问题的排查中，小编最近又遇到了一个 select 语句执行就会导致 MySQL 崩溃的问题，特来分享给大家。

先看下报错

一般来讲，只要数据库崩了，那么错误日志一定会留下线索的，先来看下具体的报错：

```
06:08:23 UTC - mysqld got signal 11 ;
Most likely, you have hit a bug, but this error can also be caused by malfunctioning hardware.
Thread pointer: 0x7f55ac0008c0
Attempting backtrace. You can use the following information to find out
where mysqld died. If you see no messages after this, something went
terribly wrong...
```

```

stack_bottom = 7f56f4074d80 thread_stack 0x46000
/usr/local/mysql/bin/mysqld(my_print_stacktrace(unsigned char const*, unsigned long)+0x2e) [0x1f1b71e]
/usr/local/mysql/bin/mysqld(handle_fatal_signal+0x323) [0xfcfcac3]
/lib64/libpthread.so.0(+0xf630) [0x7f5c28c85630]
/usr/local/mysql/bin/mysqld(actual_key_parts(KEY const*)+0xa) [0xef55ca]
/usr/local/mysql/bin/mysqld(calculate_key_len(TABLE*, unsigned int, unsigned long)+0x28) [0x10da428]
/usr/local/mysql/bin/mysqld(handler::ha_index_read_map(unsigned char*, unsigned char const*, unsigned long, ha_rkey_function)+0x261) [0x10dac51]
/usr/local/mysql/bin/mysqld(check_unique_constraint(TABLE*)+0xa3) [0xe620e3]
/usr/local/mysql/bin/mysqld(do_sj_dups_weedout(THD*, SJ_TMP_TABLE*)+0x111) [0xe62361]
/usr/local/mysql/bin/mysqld(WeedoutIterator::Read()+0xa9) [0x1084cd9]
/usr/local/mysql/bin/mysqld(MaterializeIterator::MaterializeQueryBlock(MaterializeIterator::QueryBlock const&, unsigned long long*)+0x17c) [0x10898bc]
/usr/local/mysql/bin/mysqld(MaterializeIterator::Init()+0x1e1) [0x108a021]
/usr/local/mysql/bin/mysqld(SELECT_LEX_UNIT::ExecuteIteratorQuery(THD*)+0x251) [0xf5d241]
/usr/local/mysql/bin/mysqld(SELECT_LEX_UNIT::execute(THD*)+0xf9) [0xf5f3f9]
/usr/local/mysql/bin/mysqld(Sql_cmd_dml::execute_inner(THD*)+0x20b) [0xeedf8b]
/usr/local/mysql/bin/mysqld(Sql_cmd_dml::execute(THD*)+0x3e8) [0xef7418]
/usr/local/mysql/bin/mysqld(mysql_execute_command(THD*, bool)+0x39c9) [0xeab3a9]
/usr/local/mysql/bin/mysqld(mysql_parse(THD*, Parser_state*)+0x31c) [0xead0cc]
/usr/local/mysql/bin/mysqld(dispatch_command(THD*, COM_DATA const*, enum_server_command)+0x156b) [0xeaeb6b]
/usr/local/mysql/bin/mysqld(do_command(THD*)+0x174) [0xeb0104]
/usr/local/mysql/bin/mysqld() [0xfc1a08]
/usr/local/mysql/bin/mysqld() [0x23ffdec]
/lib64/libpthread.so.0(+0x7ea5) [0x7f5c28c7dea5]
/lib64/libc.so.6(clone+0x6d) [0x7f5c26db9b0d]

```

Trying to get some variables.

Some pointers may be invalid and cause the dump to abort.

Query (7f55ac0ca298): SELECT DISTINCT T.CUST_NO FROM testDB.TABLE_TRANSACTION T WHERE EXISTS (SELECT 1 FROM testDB.Table1 T1 WHERE T.CUST_NO = T1.CUST_NO) AND T.AGENT_CERT_NO IS NOT NULL

Connection ID (thread ID): 65

Status: NOT_KILLED

从上述错误日志的输出中可以找到较为明显的几处信息：

1、导致崩溃的 SQL 语句为：

```

SELECT DISTINCT T.CUST_NO FROM testDB.TABLE_TRANSACTION T WHERE EXISTS (SELECT
1 FROM testDB.Table1 T1 WHERE T.CUST_NO = T1.CUST_NO ) AND T.AGENT_CERT_NO IS NOT
NULL

```


2、数据库发出的信号为 `signal 11`，即是 MySQL 访问到了一个错误的内存地址。

分析过程

1、查看 OS 日志以及系统资源使用情况：

OS 日志的输出对排查方向没有影响，无 MySQL OOM 的现象。

查看监控在 MySQL 崩溃时间段没有任何异常输出，且任何时候都可以在环境中执行 `select` 触发数据库 crash。

2、从业务一侧获取完整的 SQL 以及表结构信息。

完整的 SQL 语句：

```
SELECT 'testPA' AS INDIC_KEY, A.CUST_NO AS OBJ_KEY,
CASE WHEN B.CUST_NO IS NULL THEN 1 ELSE END AS INDICVAL1, '2222-06-
06' AS GRADING_DATE
FROM testDB.Table1 A
LEFT JOIN (
SELECT DISTINCT T.CUST_NO
FROM testDB.TABLE_TRANSACTION T
WHERE
EXISTS (SELECT 1 FROM testDB.Table1 T1 WHERE T.CUST_NO = T1.CUST_NO)
AND T.AGENT_CERT_NO IS NOT NULL
) B ON A.CUST_NO = B.CUST_NO;
```

表结构

```
CREATE TABLE `TABLE_TRANSACTION` (
`cert_key` varchar(32) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci DEFAULT NULL,
`cust_no` varchar(32) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci DEFAULT NULL,
...
CREATE TABLE `Table1` (
`CUST_NO` varchar(32) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL,
...

```

3、查看该 select 语句的执行计划

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	A	NULL	index	NULL	idx_cust_no	98	NULL	2	100.00	Using index
1	PRIMARY	<derived2>	NULL	ref	<auto_key0>	<auto_key0>	131	func	2	100.00	Using where; Using index
2	DERIVED	T1	NULL	index	NULL	idx_cust_no	98	NULL	2	100.00	Using index; Using temporary; Start temporary
2	DERIVED	T	NULL	ref	idx_cust_no	idx_cust_no	131	func	1	90.00	Using index condition; Using where; End temporary

4 rows in set, 2 warnings (0.00 sec)

4、堆栈分析

通过堆栈可以看到优化器将 `EXISTS` 子查询转换成了 `semi-join` 操作，由于优化器默认选择了 `DuplicateWeedout` 执行策略，所以会通过建立临时表来实现对外层查询记录进行去重操作。

执行过程可以通过执行计划得到验证：执行计划的 `Extra` 列将驱动表显示 `Start temporary` 提示，被驱动表将显示 `End temporary` 提示。

5、堆栈问题点输出：

该堆栈是在内存地址为 `0xef55ca` 的位置崩溃的，该地址可以通过 `gdb` 分析得到对应的代码位：

```
#4 0x00000000ef55ce in actual_key_parts (key_info=0x7fd5241641b0) at ../../mysql-8.0.19/sql/sql_class.h:1487
```

`sql_class.h:1487` 源码地址为：

C	sql_class.h	2	1485	/** Tells whether the given optimizer_switch flag is on */
G	sql_client.cc		1486	inline bool optimizer_switch_flag(ulonglong flag) const {
G	sql_cmd_ddl_table.cc		1487	return (variables.optimizer_switch & flag);
C	sql_cmd_ddl_table.h		1488	}

通过 `inline` 得知 `optimizer_switch_flag` 函数为 `actual_key_parts` 的内联调用，找到 `actual_key_parts` 函数的位置：

G	sql_rewrite.cc		5411	
C	sql_rewrite.h		5412	uint actual_key_parts(const KEY *key_info) {
G	sql_select.cc	7	5413	return key_info->table->in_use->optimizer_switch_flag(
C	sql_select.h		5414	OPTIMIZER_SWITCH_USE_INDEX_EXTENSIONS)
G	sql_servers.cc		5415	? key_info->actual_key_parts
C	sql_servers.h		5416	: key_info->user_defined_key_parts;
			5417	}

6、使用 `gdb` 进行调试：

6.1. 使用 `gdb` 的 `frame` 下推 4 层堆栈到 `actual_key_parts` 函数：

```
(gdb) frame 4
#4 0x00000000ef55ce in actual_key_parts (key_info=0x7fd5241641b0) at ../../mysql-8.0.19/sql/sql_class.h:1487
1487 ../../mysql-8.0.19/sql/sql_class.h: No such file or directory.
```

6.2. 打印 `actual_key_parts` 函数返回的指针所对应的内存地址

```
(gdb) p key_info
$1 = (const KEY *) 0x7fd5241641b0
(gdb) p key_info->table
$2 = (TABLE *) 0x7fd5241643e0
```

```
(gdb) p *key_info->table
$4 = {s = 0x0, file = 0x0, next = 0x0, prev = 0x0, cache_next = 0x0, cache_prev = 0x0, fields_for_functional_indexes = {bitmap = 0x0, n_bits = 0, last_word_mask = 0, last_word_ptr = 0x0}, in_use = 0x0, field = 0x0, hidden_field_count = 0, record = {0x0, 0x0}, write_row_record = 0x0, insert_values = 0x0, m_record_buffer = {m_max_records = 0, m_record_size = 0, m_count = 0, m_buffer = 0x0, m_out_of_range = 8}, covering_keys = {map = 0}, quick_keys = {map = 0}, merge_keys = {map = 0}, possible_quick_keys = {map = 0}, keys_in_use_for_query = {map = 0}, keys_in_use_for_group_by = {map = 0}, keys_in_use_for_order_by = {map = 0}, key_info = 0x0, next_number_field = 0x0, found_next_number_field = 0x0, vfield = 0x0, gen_def_fields_ptr = 0x0, hash_field = 0x0, fts_doc_id_field = 0x0, triggers = 0x0, pos_in_table_list = 0x0, pos_in_locked_tables = 0x0, group = 0x0, alias = 0x0, null_flags = 0x0, null_flags_saved = 0x0, def_read_set = {bitmap = 0x0, n_bits = 0, last_word_mask = 0, last_word_ptr = 0x0}, def_write_set = {bitmap = 0x0, n_bits = 0, last_word_mask = 0, last_word_ptr = 0x0}, tmp_set = {bitmap = 0x0, n_bits = 0, last_word_mask = 0, last_word_ptr = 0x0}, pack_row_tmp_set = {bitmap = 0x0, n_bits = 0, last_word_mask = 0, last_word_ptr = 0x0}, cond_set = {bitmap = 0x8, n_bits = 0, last_word_mask = 0, last_word_ptr = 0x0}, def_fields_set_during_insert = {bitmap = 0x0, n_bits = 0, last_word_mask = 0, last_word_ptr = 0x0}, read_set = 0x0, write_set = 0x0, fields_set_during_insert = 0x0, query_id = 0, quick_rows = {0, 0, 0, 0, 2, 2, 140553410201152, 0, 0, 0, 0, 8821862827272, 0, 0, 6631429505280, 0 <repeats 19 times>, 2, 2, 140553410201408, 0, 0, 0, 0, 8821862827272, 0, 0, 6631429505280, 0 <repeats 14 times>}, const_key_parts = {0, 0, 0, 0, 0, 1, 1, 140553410201664, 0, 0, 0, 0, 6631429505280, 0 <repeats 49 times>}, quick_key_parts = {0 <repeats 64 times>}, quick_n_ranges = {0 <repeats 64 times>}, quick_condition_rows = 0, lock_position = 0, lock_data_start = 0, lock_count = 0, db_stat = 0, current_lock = 0, table_check_constraint_list = 0x0, nullable = false, m_status = 0 '\000', null_row = false, copy_blobs = false, force_index = false, force_index_order = false, force_index_group = false, is_distinct = false, const_table = false, no_rows = false, key_read = false, no_keyread = false, no_replicate = false, fulltext_searched = false, no_cache = false, open_by_handler = false, autoinc_field_has_explicit_non_null_value = false, alias_name_used = false, get_fields_in_item_tree = false, m_needs_reopen = false, created = false, materialized = false, reginfo = {join_tab = 0x0, qep_tab = 0x0, lock_type = TL_UNLOCK, locked_row_action = THR_DEFAULT, not_exists_optimize = false, impossible_range = false}, mem_root = {static_s_dummy_target = 0 '\000', m_current_block = 0x0, m_current_free_start = 0x0, m_current_free_end = 0x0, m_block_size = 0, m_orig_block_size = 0, m_max_capacity = 0, m_allocated_size = 0, m_error_for_capacity_exceeded = false, m_error_handler = 0x0, m_psi_key = 0}, blob_storage = 0x0, sorting_iterator = 0x0, duplicate_removal_iterator = 0x0, unique_result = {io_cache = 0x0, Python Exception <class 'gdb.error'> There is no member or method named 'M' head impl.: sorted_result_in_fsbuff = false, sorted_result = , sorted_result_end = 0x0, found_records = 0}, part_info = 0x0, all_partitions_pruned_away = false, mdl_ticket = 0x0, m_cost_model = {m_cost_model_server = 0x0, m_se_cost_constants = 0x0, m_table = 0x0}, m_partial_update_columns = 0x0, m_partial_update_info = 0x0, should_binlog_drop_if_temp_flag = false}
```

6.3. 发现在使用 `in_use` 时返回值为空，出现 `0x0`，说明 `table` 内存取址错误。

```

22 = (LHD *) 0x0
(0qp) b k6λ_TU40->f9pJ6->TUN26

```

6.4. 由于 `in_use` 返回为空，在调用 `in_use` 后面的代码 `optimizer_switch_flag` 的时候就会出现非法地址，导致数据库的 crash。

```
return key_info->table->in_use->optimizer_switch_flag(
```

得出结论

经过分析，目前可以确定该问题是 MySQL 函数内存地址错乱引起的 bug，经过对高版本代码的轮询，发现 8.0.24 版本之后对上述代码做出了修订：

<pre> 5259 uint actual_key_parts(const KEY *key_info) { 5260 - return key_info->table->in_use->optimizer_switch_flag(5261 OPTIMIZER_SWITCH_USE_INDEX_EXTENSIONS) 5262 ? key_info->actual_key_parts 5263 : key_info->user_defined_key_parts; 5264 } 5265 5266 uint actual_key_flags(const KEY *key_info) { 5267 - return key_info->table->in_use->optimizer_switch_flag(5268 OPTIMIZER_SWITCH_USE_INDEX_EXTENSIONS) 5269 ? key_info->actual_flags 5270 : key_info->flags; </pre>	<pre> 5259 uint actual_key_parts(const KEY *key_info) { 5260 + return current_thd->optimizer_switch_flag(5261 OPTIMIZER_SWITCH_USE_INDEX_EXTENSIONS) 5262 ? key_info->actual_key_parts 5263 : key_info->user_defined_key_parts; 5264 } 5265 5266 uint actual_key_flags(const KEY *key_info) { 5267 + return current_thd->optimizer_switch_flag(5268 OPTIMIZER_SWITCH_USE_INDEX_EXTENSIONS) 5269 ? key_info->actual_flags 5270 : key_info->flags; </pre>
--	--

以下为该 bug 的相关描述：<https://github.com/mysql/mysql-server/commit/7fde9072e1f62b1b3cf857757a3be41cec5c8e48>

Bug#32299045: Crash after JOIN::generate_derived_keys, concurrent tri...

--ggers

Problem here is that we access the THD pointer for use with getting the mem_root pointer for allocating memory in `add_derived_key()`, as well as in calls to `actual_key_parts()` and `actual_key_flags()`. This happens because we access `TABLE::in_use` from internal temporary tables which are associated with triggers. Those triggers have been transferred from one session to another, but the corresponding `in_use` pointers have not been updated.

The fix is to replace use of `TABLE::in_use` with `current_thd`.

Reviewed by: Chandan Kunal <chandan.kunal@oracle.com>

8.0

mysql-cluster-8.0.29 mysql-cluster-8.0.28 mysql-cluster-8.0.27 mysql-cluster-8.0.26 mysql-cluster-8.0.25 mysql-cluster-8.0.24 mysql-8.0.29 mysql-8.0.28 mysql-8.0.27 mysql-8.0.26 mysql-8.0.25 mysql-8.0.24

解决方案

在上述排查分析中，我们得到这个 bug 是由于使用了 `semi-join` 的 `DuplicateWeedout` 执行策略导致了问题的发生，如果在短时间内无法升级变更数据库，而又想尽量避免这个问题的发生。

一方面肯定是业务侧避免该 SQL 的执行，从 DBA 的角度上考虑的是该 SQL 怎样才能正常执行，那么经过验证：

以下三种解决方案均可解决当前 `select` 查询导致的数据库崩溃问题。

- 1、业务表设置合理统一的字符集 (`utf8mb4`) 和排序规则，避免 `exist` 在半连接中使用了 `DuplicateWeedout` 策略，加快 SQL 执行效率；

2、关闭数据库级别的 DuplicateWeedout 优化策略：

SET [GLOBAL|SESSION] optimizer_switch='duplicateweedout=off';

3、升级 MySQL 版本到 8.0.24 ；

其他解决思路

1、直接在谷歌等平台检索相关堆栈代码，查找 MySQL 类似的 bug ，然后在修复版本进行相关 SQL 的验证，确认该 bug 在对应的已经修复，完成问题排查。

2、数据库开启 core_dump 完成堆栈的辅助验证。

3.15 Sql_slave_skip_counter 使用不规范对复制的影响

作者：唐龙哲

一、背景描述

最近在客户环境遇到一个 sql_slave_skip_counter 使用不规范引起的主从故障问题。

我们知道 mysql 的主从复制是逻辑复制，所以 slave 在应用 relay log 的过程中，经常会遇到错误，而参数 sql_slave_skip_counter 可以设置跳过多少个 event ，让从库 sql 继续回放。但是如果 sql_slave_skip_counter 使用不当，也会造成主从数据不一致。

以下为抓取的部分错误日志截图：

可以看到为了跳过报错的 SQL ，现场有手动设置过 SQL_SLAVE_SKIP_COUNTER=100000 ，表示跳过 10 万的 events ，而 binlog event 事件的多少，与事务表/非事务表、及 binlog 日志模式有关。

```
2022-03-24 16:03:04 63212 [Note] 'SQL_SLAVE_SKIP_COUNTER=100000' executed at relay_log_file='./relay-bin.020367', relay_log_pos='57967860', master_log_name='mysql-bin.003550', master_log_pos='509496708' and new position at relay_log_file='./relay-bin.020367', relay_log_pos='77964392', master_log_name='mysql-bin.003550', master_log_pos='529493240'
2022-03-24 16:03:04 63212 [ERROR] Slave SQL: Could not execute Update_rows event on table newoms_otm18_otm_sub_order; Can't find record in 'otm_sub_order', Error_code: 1032; handler error HA_ERR_KEY_NOT_FOUND; the event's master log mysql-bin.003550, end_log_pos 529498239, Error_code: 1032
2022-03-24 16:03:04 63212 [Warning] Slave: Can't find record in 'otm_sub_order' Error_code: 1032
2022-03-24 16:03:04 63212 [ERROR] Error running query, slave SQL thread aborted. Fix the problem, and restart the slave SQL thread with 'SLAVE START'. We stopped at log 'mysql-bin.003550' position 529493240
2022-03-24 16:03:11 63212 [Note] Slave I/O thread exiting, read up to log 'mysql-bin.003551', position 558007496
2022-03-24 16:03:20 63212 [Warning] Storing MySQL user name or password information in the master info repository is not secure and is therefore not recommended. Please consider using the USER and PASSWORD connection options for START SLAVE; see the 'START SLAVE Syntax' in the MySQL Manual for more information.
2022-03-24 16:03:20 63212 [Note] Slave I/O thread: connected to master 'root@192.168.1.100:3306' replication started in log 'mysql-bin.003551' at position 558007496
2022-03-24 16:03:20 63212 [Warning] Slave SQL: If a crash happens this configuration does not guarantee that the relay log info will be consistent, Error_code: 0
2022-03-24 16:03:20 63212 [Note] Slave SQL thread initialized, starting replication in log 'mysql-bin.003550' at position 529493240, relay log './relay-bin.020367' position: 77964392
2022-03-24 23:07:04 63212 [Note] Error reading relay log event: slave SQL thread was killed
```

常见错误说明：write_rows ： Duplicate entry （1062 错误）主键冲突，主要表现为重复插入已经存在的记录；

update_rows ： Can't find record （1032 错误），无法发现需要更新的行记录。

sql_slave_skip_counter 参数说明：

从官方解释知道，sql_slave_skip_counter 以 event 为单位 skip ，直到 skip 完第 N 个 event 所在的 event group 才停止。对于事务表，一个 event group 对应一个事务；对于非事务表，一个 event group 对应一条 SQL 语句。并且一个 event group 可以包含多个 event 事件。

详细解释，可以参考官方文档：

<https://dev.mysql.com/doc/refman/5.6/en/set-global-sql-slave-skip-counter.html>。<https://dev.mysql.com/doc/refman/8.0/en/replication-administration-skip.html>。

二、测试 sql_slave_skip_counter 对复制的影响

使用显示开启事务方式(begin/commit)，模拟 delete/update 遇到 Can't find record (1032) 错误，insert 中的 Duplicate entry(1062) 错误可以使用类似的方法去分析。此处使用 MySQL8.0.23 版本，基于 Row 日志模式 + Position 方式搭建异步复制架构。

2.1 准备数据

前提条件(主从库应用如下配置)：

```
-- 关闭 GTID
mysql> show global variables like '%gtid%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| binlog_gtid_simple_recovery | OFF |
| enforce_gtid_consistency | OFF |
| gtid_executed | |
| gtid_mode | OFF |
| gtid_owned | |
| gtid_purged | |
| simplified_binlog_gtid_recovery | OFF |
+-----+-----+

-- 禁用 read_only
mysql> show global variables like 'read_only';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| read_only | OFF |
+-----+-----+

-- 设置 row 模式
mysql> show global variables like 'binlog_format';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| binlog_format | ROW |
+-----+-----+
```

在主库创建两张表，一张是事务表，另一张为非事务表，然后往各表中写入 id=1 的记录。

```
-- 主库创建测试表
```

```
create table replica_innodb (id int, msg char(32)) engine=innodb;
create table replica_myisam (id int, msg char(32)) engine=myisam;

-- 往测试表中添加数据
insert into replica_innodb values(1,'innodb_001');
insert into replica_myisam values(1,'myisam_001');
```

2.2 模拟 sql_slave_skip_counter = N 跳过复制故障的场景

说明：该场景将模拟主库更新 replica_myisam 非事务表的记录，导致从库更新失败的操作。

(1) 从库执行 delete 语句，删除非事务表 id=1 的记录

```
set sql_log_bin=0;
delete from replica_myisam where id=1;
set sql_log_bin=1;
```

(2) 主库更新 replica_myisam 非事务表 id=1 的记录，并新增数据到两张表中

```
-- 编写 insert 存储过程
delimiter $$
create procedure p_insert()
begin
declare i int;
set i = 2;
while i < 6 do
insert into replica_innodb values (i, concat('innodb_', repeat(0, 2), i));
insert into replica_myisam values (i, concat('myisam_', repeat(0, 2), i));
set i = i + 1;
end while;
end$$
delimiter ;

-- 更新数据
begin;
update replica_myisam set msg='my01' where id=1;
call p_insert;
commit;
```

(3) 检查复制状态

```
Slave_IO_Running: Yes
Slave_SQL_Running: No
Last_SQL_Errno: 1032
Last_SQL_Error: Could not execute Update_rows event on table testdb.replica_myisam; Can
't find record in 'replica_myisam', Error_code: 1032; handler error HA_ERR_END_OF_FILE;
the event's master log mysql-bin.000001, end_log_pos 1715
```

上述操作先在从库先删除非事务表数据 id=1 的情况下，主库再更新数据，然后通过复制将主库 id=1 的记录传递到从库，在 SQL 线程回放时造成从库无法更新不存的记录（报 1032 错误），导致 SQL 线程故障（此时 SQL 线程已经停止）。

（4）尝试使用 sql_slave_skip_counter 跳过错误，并重启复制

```
-- 从库跳过"10000"个 event，并重启 SQL 线程
set global sql_slave_skip_counter=10000;
start slave sql_thread;

-- 查看数据，可以发现 replica_myisam 以及 replica_innodb 表 id>=2 的记录并未同步到从库
mysql> select * from replica_myisam;
Empty set (0.00 sec)

mysql> select * from replica_innodb\G
***** 1. row *****

id: 1
msg: innodb_001
1 row in set (0.01 sec)

-- 查看复制状态为 Yes，表示已经跳过冲突的 event 事件
mysql> show slave status\G
Slave_IO_Running: Yes
Slave_SQL_Running: Yes
Last_Errno: 0
Last_Error:
Skip_Counter: 9955
Exec_Master_Log_Pos: 4493
Relay_Log_Space: 4854
```

根据观察到的 Skip_Counter 及查询结果显示，所有冲突的 sql 语句及正常的 sql 语句产生的 event 都被跳过了。被跳过的操作以及对应的事件类型，可以通过错误日志中 SQL 线程恢复时记录的 Relay_Log_File 和 Relay_Log_Pos 两个值，确认执行过了哪些操作。

```
2022-08-02T11:35:41.933313+08:00 70 [ERROR] [MY-010584] [Repl] Slave SQL for channel '': Could not execute Update_rows event on table testdb.replica_myisam; Can't find record in 'replica_myisam', Error_code: 1032; handler error HA_ERR_END_OF_FILE; the event's master log mysql-bin.000001, end log_pos 1715, Error_code: MY-001032
2022-08-02T11:35:41.933892+08:00 70 [Warning] [MY-010584] [Repl] Slave: Can't find record in 'replica_myisam' Error_code: MY-001032
2022-08-02T11:35:41.934007+08:00 70 [ERROR] [MY-010586] [Repl] Error running query, slave SQL thread aborted. Fix the problem, and restart the slave SQL thread with "SLAVE START". We stopped at log 'mysql-bin.000001' position 1369
2022-08-02T11:36:42.171845+08:00 71 [Note] [MY-010581] [Repl] Slave SQL thread for channel ' initialized, starting replication in log 'mysql-bin.000001' at position 1369, relay log '/opt/mysql/log/relaylog/3377/mysql-relay.000002' position: 1533
```

```
show relaylog events in "mysql-relay.000002" from 1533 limit 50;
```

```
mysql> show relaylog events in "mysql-relay.000002" from 1533 limit 50;
```

Log_name	Pos	Event_type	Server_id	End_log_pos	Info
mysql-relay.000002	1533	Anonymous_Gtid	337629978	1444	SET @@SESSION.GTID_NEXT= 'ANONYMOUS'
mysql-relay.000002	1608	Query	337629978	1526	BEGIN
mysql-relay.000002	1690	Rows_query	337629978	1593	# update replica_myisam set msg='my01' where id=1
mysql-relay.000002	1757	Table_map	337629978	1657	table_id: 207 (testdb.replica_myisam)
mysql-relay.000002	1821	Update_rows	337629978	1715	table_id: 207 flags: STMT_END_F
mysql-relay.000002	1879	Query	337629978	1798	COMMIT
mysql-relay.000002	1902	Anonymous_Gtid	337629978	1873	SET @@SESSION.GTID_NEXT= 'ANONYMOUS'
mysql-relay.000002	2037	Query	337629978	1946	BEGIN
mysql-relay.000002	2110	Rows_query	337629978	2039	# insert into replica_myisam values (i, concat('myisam_', repeat(0, 2), i))
mysql-relay.000002	2203	Table_map	337629978	2103	table_id: 207 (testdb.replica_myisam)
mysql-relay.000002	2267	Write_rows	337629978	2150	table_id: 207 flags: STMT_END_F
mysql-relay.000002	2314	Query	337629978	2224	COMMIT
mysql-relay.000002	2388	Anonymous_Gtid	337629978	2299	SET @@SESSION.GTID_NEXT= 'ANONYMOUS'
mysql-relay.000002	2463	Query	337629978	2372	BEGIN
mysql-relay.000002	2536	Rows_query	337629978	2465	# insert into replica_myisam values (i, concat('myisam_', repeat(0, 2), i))
mysql-relay.000002	2629	Table_map	337629978	2529	table_id: 207 (testdb.replica_myisam)
mysql-relay.000002	2693	Write_rows	337629978	2576	table_id: 207 flags: STMT_END_F
mysql-relay.000002	2740	Query	337629978	2650	COMMIT
mysql-relay.000002	2814	Anonymous_Gtid	337629978	2725	SET @@SESSION.GTID_NEXT= 'ANONYMOUS'
mysql-relay.000002	2889	Query	337629978	2798	BEGIN
mysql-relay.000002	2962	Rows_query	337629978	2891	# insert into replica_myisam values (i, concat('myisam_', repeat(0, 2), i))
mysql-relay.000002	3055	Table_map	337629978	2955	table_id: 207 (testdb.replica_myisam)
mysql-relay.000002	3119	Write_rows	337629978	3002	table_id: 207 flags: STMT_END_F
mysql-relay.000002	3166	Query	337629978	3076	COMMIT
mysql-relay.000002	3240	Anonymous_Gtid	337629978	3151	SET @@SESSION.GTID_NEXT= 'ANONYMOUS'
mysql-relay.000002	3315	Query	337629978	3224	BEGIN
mysql-relay.000002	3388	Rows_query	337629978	3317	# insert into replica_myisam values (i, concat('myisam_', repeat(0, 2), i))
mysql-relay.000002	3481	Table_map	337629978	3381	table_id: 207 (testdb.replica_myisam)
mysql-relay.000002	3545	Write_rows	337629978	3428	table_id: 207 flags: STMT_END_F
mysql-relay.000002	3592	Query	337629978	3502	COMMIT
mysql-relay.000002	3666	Anonymous_Gtid	337629978	3577	SET @@SESSION.GTID_NEXT= 'ANONYMOUS'
mysql-relay.000002	3741	Query	337629978	3650	BEGIN
mysql-relay.000002	3814	Rows_query	337629978	3743	# insert into replica_innodb values (i, concat('innodb_', repeat(0, 2), i))
mysql-relay.000002	3907	Table_map	337629978	3807	table_id: 190 (testdb.replica_innodb)
mysql-relay.000002	3971	Write_rows	337629978	3854	table_id: 190 flags: STMT_END_F
mysql-relay.000002	4018	Rows_query	337629978	3947	# insert into replica_innodb values (i, concat('innodb_', repeat(0, 2), i))
mysql-relay.000002	4111	Table_map	337629978	4011	table_id: 190 (testdb.replica_innodb)
mysql-relay.000002	4175	Write_rows	337629978	4058	table_id: 190 flags: STMT_END_F
mysql-relay.000002	4222	Rows_query	337629978	4151	# insert into replica_innodb values (i, concat('innodb_', repeat(0, 2), i))
mysql-relay.000002	4315	Table_map	337629978	4215	table_id: 190 (testdb.replica_innodb)
mysql-relay.000002	4379	Write_rows	337629978	4262	table_id: 190 flags: STMT_END_F
mysql-relay.000002	4426	Rows_query	337629978	4355	# insert into replica_innodb values (i, concat('innodb_', repeat(0, 2), i))
mysql-relay.000002	4519	Table_map	337629978	4419	table_id: 190 (testdb.replica_innodb)
mysql-relay.000002	4583	Write_rows	337629978	4466	table_id: 190 flags: STMT_END_F
mysql-relay.000002	4630	Xid	337629978	4493	COMMIT /* xid=258 */

可以看到，基于 ROW 日志模式下，事务表及非事务表至少会产生对应的 4 个 event 事件：
rows_query (binlog_rows_query_log_events=ON 情况下才会记录到 binlog 日志中)、table_map 、
update_rows/write_rows 以及 query(begin)/xid(commit)。实际上所有对事务表的操作是在同一个显式事务
中，所有对非事务表的操作，每条 SQL 语句单独对应一个事务。

按照 binlog 记录的提交顺序，可以看到非事务表会先提交，innodb 事务表在 commit 执行之后提交，所以 1032 报的是 replica_myisam 表相关的错误操作。

各个 event 事件作用：

Event	作用
Query	以文本的形式记录事务，包括 DDL 操作（ROW 格式），begin 等
Rows_query	记录 DML 操作（insert、delete、update）
Table_map	记录事件的库表信息
Write_rows	针对 insert 操作，包含了要插入的数据
Update_rows	针对 update 操作，包含了，修改前后的数据
Xid	在事务的末尾分配一个 Xid（事务 id，唯一），标志事务的结束

（5）模拟主库追加数据

```
-- 主库追加数据
```

```
begin;
```



```

insert into replica_innodb values(6,'innodb_006');
commit;
select * from replica_innodb where id=6\G
***** 1. row *****
  id: 6
msg: innodb_006

-- 从库观察到 IO 及 SQL 线程状态为 yes，复制正常且 Skip_Counter 值递减。
show slave status\G
***** 1. row *****
Slave_IO_Running: Yes
Slave_SQL_Running: Yes
Last_Errno: 0
Last_Error:
Skip_Counter: 9949
Exec_Master_Log_Pos: 4849
Relay_Log_Space: 5210

-- 从库检查事务表数据，未发现 id=6 的记录
select * from replica_innodb\G
***** 1. row *****
  id: 1
msg: innodb_001

```

说明：上述操作并未将主库提交的事务应用到从库。现象说明：

（1）通过上述操作，`sql_slave_skip_counter=N(N>0)` 重启 slave 复制线程后，从库开始跳过 event，每跳过一个 event，N 就减去 1，直到 N 变成 0。在此期间，所有冲突的 sql 语句及正常的 sql 语句产生的 event 都被记录到该变量中，当 N 变成 0 后从库才开始执行当前的语句。

并且当 Skip_Counter 值等于 1，在对事务表操作时，并不是只跳过一个 event，而是跳过一个完整事务，该事务如果存在多条 sql，那么当报错发生时，会将剩余的 sql 操作一并跳过，直到遇到 xid event 为止。例如：报错发生在第一个 sql 语句，如果第二条 sql 语句影响 1000 行数据，那么主从就有 1000 行数据不一致。

（2）观察从库复制正常，IO 及 SQL 线程状态为 yes、Skip_Counter 值递减，且 binlog 无新的日志产生，错误日志无报错信息。

2.3 sql_slave_skip_counter 清零

为了防止 `sql_slave_skip_counter` 对复制的影响，需要在错误跳过之后及时的恢复，将参数置为零。

方式一：重启复制

注意：MySQL 5.7、8.0 重启复制后，Skip_Counter 计数器清零，MySQL 5.6 此方法无效。

```
stop slave;
```

```
start slave;
show global variables like 'sql_slave_skip_counter'\G
***** 1. row *****
Variable_name: sql_slave_skip_counter
Value: 0
```

方式二：手工设置为零，立即生效

```
stop slave;
set global sql_slave_skip_counter=0;
start slave;
show variables like 'sql_slave_skip_counter'\G
***** 1. row *****
Variable_name: sql_slave_skip_counter
Value: 0
```

方式三：重启从库

三、总结

规范使用 `sql_slave_skip_counter`，如果操作不当，极有可能跳过其他 `events`，并造成主从数据不一致。如果数据库数据量较小，大约 60G 左右，建议重做复制。数据量超过 60G 大小，可以在 `sql_slave_skip_counter` 跳过之前，看一下当前 `binlog event group` 执行了哪些操作。对于 1032、1062 错误尽量修补数据，让复制进程在从库应用变更。

建议 MySQL 复制开启 GTID（全局唯一事务 ID），该方式可以根据当前事务的变化快速判断主从数据一致性，也强化故障恢复以及容错能力。需要注意：此方式在 5.6 不支持在线开启 GTID，需要先停库再进行 GTID 相关参数配置。

3.16 xtrabackup 多表备份报错 “ too many open files ”

作者：付祥

一、背景描述

最近使用 xtrabackup 工具对 mysql 实例进行备份时，由于实例的 ibd 文件过多，而备份用户的 open files 参数设置的值太小，在备份实例时打开的文件数量超过了备份用户允许打开的文件数量限制，导致备份失败，其报错如下：

```
220330 08:28:47 >> log scanned up to (328701072168)
InnoDB: Operating system error number 24 in a file operation.
InnoDB: Error number 24 means 'Too many open files'
InnoDB: Some operating system error numbers are described at
http://dev.mysql.com/doc/refman/5.7/en/operating-system-error-codes.html
InnoDB: File ./stage/ts_cg_inteltaxtochanges #P#P_20220322.ibd: 'open' returned
OSerror 124. Cannot continue operation
InnoDB: Cannot continue operation.
```

二、模拟故障场景

1、环境说明

```
mysql 版本：v5.7.35
xtrabackup 版本：v2.4.24
```

2、查看 open files 的已知参数值

(1) 操作系统当前的 open files 数量

```
[root@test ~]# ulimit -a
core file size (blocks, -c) 0
...
open files (-n) 1024
...
```

(2) mysql 实例配置文件 my.cnf 的 innodb_file_per_table、open_files_limit 参数值

```
[root@test ~]# less /opt/mysql/etc/3306/my.cnf |grep innodb_file_per_table
innodb_file_per_table = 1
[root@test ~]# less /opt/mysql/etc/3306/my.cnf |grep open_files_limit
open_files_limit = 20000
```

(3) 备份用户 actiontech-mysql 在 /etc/security/limits.d 中的配置文件的 open files 的限制

```
[root@test ~]# cd /etc/security/limits.d
[root@test limits.d]# cat actiontech-mysql.conf
```

```

actiontech-mysql soft nofile 10240
actiontech-mysql hard nofile 10240
actiontech-mysql soft nproc 65535
actiontech-mysql hard nproc 65535
[root@test limits.d]# su -s $(which bash) actiontech-mysql
bash-4.2$ ulimit -a
core file size (blocks, -c) 0
...
open files (-n) 10240
...

```

3、使用 sysbench 生成大量数据表

```

[root@test ~]# /usr/share/sysbench/oltp_read_write.lua --table-size=1000 --
tables=12000 --mysql-user=root --mysql-password=1 --mysql-host=127.0.0.1 --mysql-
port=3306 --mysql-db=testdb --threads=250 --time=60 --report-interval=3 --skiptrx=on --
mysql-ignore-errors=1062 prepare

```

4、模拟在 root 用户与非 root 用户下备份的报错情况

(1) 在非 root 用户 actiontech-mysql 下执行备份命令，复现报错现象

```

[root@test ~]# su -s $(which bash) actiontech-mysql
bash-4.2$ xtrabackup --defaults-file=/opt/mysql/etc/3306/my.cnf --user=root --
password=1 --socket=/opt/mysql/data/3306/mysqld.sock --backup --target-
dir=/opt/mysql/backup/3306
xtrabackup: recognized server arguments: --server-id=1224415609 --
datadir=/opt/mysql/data/3306 --log_bin=/opt/mysql/log/binlog/3306/mysql-bin --
tmpdir=/opt/mysql/tmp/3306 --
innodb_log_group_home_dir=/opt/mysql/log/redolog/3306 --
innodb_buffer_pool_size=1G --innodb_data_file_path=ibdata1:1G:autoextend --
innodb_file_per_table=1 --innodb_flush_log_at_trx_commit=1 --
innodb_flush_method=O_DIRECT --innodb_io_capacity=1000 --
innodb_log_buffer_size=64M --innodb_log_file_size=2G --
innodb_log_files_in_group=2 --innodb_max_dirty_pages_pct=60 --
innodb_undo_tablespace=3 --innodb_read_io_threads=8 --innodb_write_io_threads=8
--innodb_open_files=4096 --open_files_limit=20000
...
xtrabackup: open files limit requested 20000, set to 10240
...
220413 13:58:50 >> log scanned up to (6550086912)
InnoDB: Operating system error number 24 in a file operation.
InnoDB: Error number 24 means 'Too many open files'
InnoDB: Some operating system error numbers are described at
http://dev.mysql.com/doc/refman/5.7/en/operating-system-error-codes.ht ml

```

```
InnoDB: File ./testdb/sbtest4576.ibd: 'open' returned OS error 124. Cannot
continue operation
InnoDB: Cannot continue operation.
```

(2) 直接以 root 用户备份

前提：将 mysql 配置文件中的 open_files_limit 参数值改小（20000->10000），重启数据库生效

```
[root@test ~]# less /opt/mysql/etc/3306/my.cnf |grep open_files_limit
open_files_limit = 10000
[root@test ~]# systemctl start mysqld_3306
```

在 root 用户下进行备份，复现报错现象

```
[root@test ~]# xtrabackup --defaults-file=/opt/mysql/etc/3306/my.cnf --user=root
--password=1 --socket=/opt/mysql/data/3306/mysqld.sock --backup --
targetdir=/opt/mysql/backup/3306
xtrabackup: recognized server arguments: --server-id=1224415609 --
datadir=/opt/mysql/data/3306 --log_bin=/opt/mysql/log/binlog/3306/mysql-bin --
tmpdir=/opt/mysql/tmp/3306 --
innodb_log_group_home_dir=/opt/mysql/log/redolog/3306 --
innodb_buffer_pool_size=1G --innodb_data_file_path=ibdata1:1G:autoextend --
innodb_file_per_table=1 --innodb_flush_log_at_trx_commit=1 --
innodb_flush_method=O_DIRECT --innodb_io_capacity=1000 --
innodb_log_buffer_size=64M --innodb_log_file_size=2G --
innodb_log_files_in_group=2 --innodb_max_dirty_pages_pct=60 --
innodb_undo_tablespace=3 --innodb_read_io_threads=8 --innodb_write_io_threads=8
--innodb_open_files=4096 --open_files_limit=10000
...
xtrabackup: open files limit requested 10000, set to 10000
...
220413 14:42:33 >> log scanned up to (6550257170)
InnoDB: Operating system error number 24 in a file operation.
InnoDB: Error number 24 means 'Too many open files'
InnoDB: Some operating system error numbers are described at
http://dev.mysql.com/doc/refman/5.7/en/operating-system-error-codes.html
InnoDB: File ./testdb/sbtest10796.ibd: 'open' returned OS error 124. Cannot
continue operation
InnoDB: Cannot continue operation.
```

以上两种情况均已成功复现故障现场。

三、故障分析

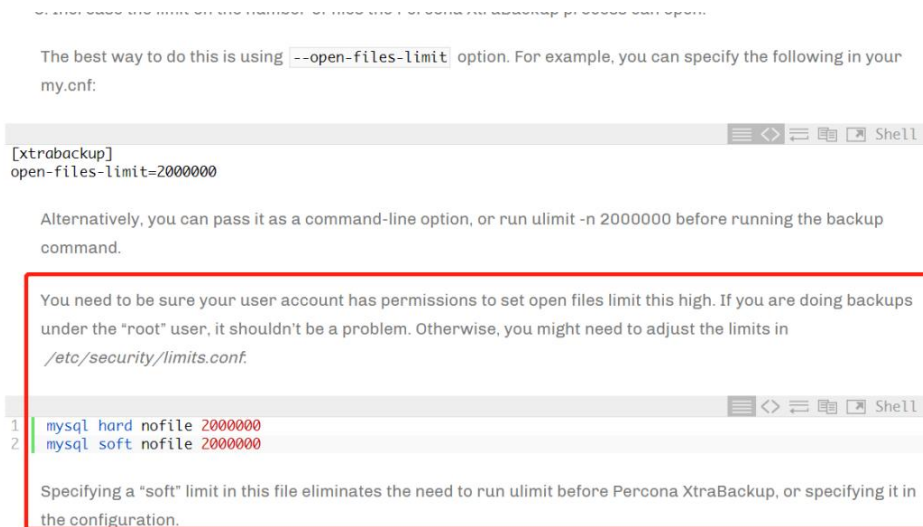
首先我们统计下目前数据库需要打开多少文件

```
[root@test ~]# find /opt/mysql/data/3306 -name "*.ibd" |wc -l
12945
```

3.16 xtrabackup 多表备份报错 “ too many open files ”

根据以上统计结果显示，我们已知实例的 `open_files_limit` 的运行参数值为 20000，且大于备份时需要打开的文件数量，那为什么还会报错呢？

看以下官方给出的说明，意思是当你在非 `root` 用户下执行备份时，用户允许的 `open files` 需要比备份时打开的文件数量要多才可以，而在 `root` 用户下是没有限制的，因此并不会会有此问题。



结合以上官方的说明，我们再来看看使用的非 `root` 用户备份过程中 `open files limit` 实际的取值是多少？

查看备份日志，确认当时 `open files limit` 实际设置的值

```
xtrabackup: open files limit requested 20000, set to 10240
```

查看备份进程实际使用的 `open files` 的值

```
bash-4.2$ ps -ef|grep xtrabackup
actiont+ 19732 19630 0 16:03 pts/1 00:00:00 xtrabackup --
defaultsfile=/opt/mysql/etc/3306/my.cnf --user=root --password=x --
socket=/opt/mysql/data/3306/mysql.sock --backup --targetdir=/opt/mysql/backup/3306
bash-4.2$ cat /proc/19732/limits
Limit Soft Limit Hard Limit Units
Max cpu time unlimited unlimited seconds
...
Max open files 10240 10240 files
..
```

显然 `open files limit` 请求的值为 20000，获取的是 `mysql` 配置文件中的值，但是实际设置的值为备份用户允许的 `open files` 的值 10240，少于上面统计的当前需要打开的文件数量，所以导致备份失败。

我们再来看看直接使用 `root` 用户备份过程中 `open files limit` 实际的取值是怎样的？

查看备份日志，确认当时 `open files limit` 实际设置的值

```
xtrabackup: open files limit requested 10000, set to 10000
```

查看备份进程实际使用的 `open files` 的值

```
[root@test 3306]# ps -ef|grep xtrabackup
```

```

root 20826 8965 3 16:18 pts/1 00:00:00 xtrabackup --
defaultsfile=/opt/mysql/etc/3306/my.cnf --user=root --password=x --
socket=/opt/mysql/data/3306/mysql.sock --backup --targetdir=/opt/mysql/backup/3306
[root@test 3306]# cat /proc/20826/limits
Limit Soft Limit Hard Limit Units
Max cpu time unlimited unlimited seconds
...
Max open files 10000 10000 files
...

```

可看到 open files limit 请求的值为 10000，获取的是 mysql 配置文件中的值，实际设置的值也是 10000，没有变化。这说明在 root 用户下，直接以 mysql 配置文件中的 open files limit 为实际允许打开文件数量的限制。

四、总结

(1) 在非 root 用户下使用 xtrabackup 备份时，用户的 open files 值需要大于备份时打开的文件数量。建议设置的值比配置文件中的 open_files_limit 参数值大，确保备份用户的 open files 是够备份时打开的文件数量。可在 /etc/security/limits.d 下修改 nofile 的值永久生效，也可以直接在用户下 ulimit -n 修改 open files 使其临时生效。

```

# 修改用户的 soft、hard 的 nofile 值永久生效
* soft nofile 20000
* hard nofile 20000
# 或者登录到用户下 ulimit -n 临时生效

```

(2) 在 root 用户下使用 xtrabackup 备份时，因为 root 用户默认对 open files 不做限制，这时需要确保配置文件中的 open_files_limit 的运行值是满足需求的，其备份会直接以配置文件中的参数值来限制允许打开的文件数量。也可以直接命令行选项传递符合需求的值，或直接在执行备份前设置 ulimit -n 20000 使 open files 临时生效。

```

# 1.修改配置文件中的 open_files_limit 值
[mysqld]
open_files_limit=20000
# 2.执行指定命令行选项传递
--open-files-limit=20000
# 3、在执行备份前设置 ulimit 值
ulimit -n 20000

```

(3) 确保系统中保持打开的最大文件数。如果这个数字太小，Percona Xtrabackup 可能会独占系统中打开的文件，导致其他进程尝试打开文件时失败。这可能会导致 MySQL 服务器崩溃，以及其他进程失败。

```

# 查看系统的 file-max
[root@test ~]# cat /proc/sys/fs/file-max
379560
# 修改 file-max
[root@test ~]# sysctl -w fs.file-max=5000000

```

```
[root@test ~]# echo "fs.file-max=5000000" >> /etc/sysctl.conf
```

参考资料:

<https://www.percona.com/blog/2016/12/28/using-percona-xtrabackup-mysql-instance-large-number-tables/>

3.17 密码使用特殊字符

作者: 王祥

背景

最近在使用脚本新建了一批应用用户,发现一个奇怪的问题,有部分用户存下以下问题:应用使用该密码能正常访问,但使用 mysql 客户端登录手动输入密码无法登录。经过与正常用户对比发现存在登录异常的用户使用了特殊字符"\$"。

问题复现

在测试环境使用脚本生成一批用户

```
#新建用户脚本简化后如下
#!/bin/bash
pw="abc$2UY"
mysql --login-path=root -e"create user app@%' identified by '$pw'"
mysql --login-path=root -e"grant insert,update,delete,select on *.* to app@%'"

#测试使用 mysql 客户端登录
[root@node3 ~]# mysql -h127.0.0.1 -uapp -p #手动输入密码无法登录
Enter password:
ERROR 1045 (28000): Access denied for user 'app'@'127.0.0.1' (using password: YES)

[root@node3 ~]# mysql -h127.0.0.1 -uapp -p'abc$2UY' #使用单引号无法登录
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 1045 (28000): Access denied for user 'app'@'127.0.0.1' (using password: YES)

[root@node3 ~]# mysql -h127.0.0.1 -uapp -pabc$2UY #不加单引号或使用双引号都可以登录
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 15389
Server version: 8.0.18 MySQL Community Server - GPL
```



```
Copyright (c) 2000, 2019, Oracle and/or its affiliates. All rights reserved.
```

```
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
```

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

```
mysql>
```

问题分析

根据上面的测试，发现部分能登录部分不能登录。那为什么部分可以部分不可以呢？首先可以确认一下存入数据库的密码是否正确。我们可以手动新建一个用户密码与 app 用户密码保持一致。然后比较 mysql.user 表中 authentication_string 字段是否一致。

```
mysql> create user test@'%' identified by 'abc$2UY';
```

```
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> select user,host,authentication_string from mysql.user where user in ('app','test');
```

user	host	authentication_string
app	%	*06E0B7BA0149152EE2A387A144A2DF9ACC492297
test	%	*7258A15F121DD9F6F7C40C08D34A3DB5ED8C8CB5

通过对比 authentication_string 字段发现两个密码不一致，说明脚本生成的用户密码并不是原来的密码了。为什么会这样呢？我们知道在 shell 中使用单引号与双引号定义的字符串是有区别的。如下：

```
[root@node3 ~]# echo "abc$2UY"
```

```
abcUY
```

```
[root@node3 ~]# echo 'abc$2UY'
```

```
abc$2UY
```

```
[root@node3 ~]# echo abc$2UY
```

```
abcUY
```

单引号定义字符串所见即所得，双引号引用的内容，所见非所得。如果内容中有命令、变量等，会先把变量、命令解析出结果，然后再输出最终内容。分析到这回头看脚本密码使用了双引号导致了 \$2 被解析成了空，最终存入数据库的密码为：abcUY。使用 mysql 登录时密码加单引号或手动输入密码此时 \$2 没有被解析成空，与数据库中的密码不一致无法登录，而密码加双引号或不加时 \$2 被解析成空，刚好和数据库的密码一致此时能正常登录数据库。下面我们新建一个用户密码设置成：abcUY，验证一下。

```
mysql> create user test1@'%' identified by 'abcUY';
```

```
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> select user,host,authentication_string from mysql.user where user in ('app','test','test1');
```

user	host	authentication_string
app	%	*06E0B7BA0149152EE2A387A144A2DF9ACC492297
test	%	*7258A15F121DD9F6F7C40C08D34A3DB5ED8C8CB5
test1	%	*06E0B7BA0149152EE2A387A144A2DF9ACC492297

比较用户 `app` 与 `test1` 的 `authentication_string` 字段发现一致，说明 `app` 密码确实是 `abcUY`。其实不止\$符号会有这个问题，其它特殊字符也会存上以上问题。所以在新建用户或登录数据库时如果密码包含特殊字符需要注意是否会被解析的问题。

另外在使用 `login-path` 免密登录配置时，如果密码包含“#”符号，存在一个 bug

(<https://bugs.mysql.com/bug.php?id=95597>) 正常配置的密码无法登录到数据库，需要加上双引号。测试如下：

```
mysql> create user app2@%' identified by '123#abc';
```

```
Query OK, 0 rows affected (0.01 sec)
```

```
[root@node3 ~]# mysql_config_editor set --login-path=app --user=app2 --host=127.0.0.1 -p
```

```
Enter password: 输入 123#abc
```

```
[root@node3 ~]# mysql --login-path=app
```

```
ERROR 1045 (28000): Access denied for user 'app2'@'127.0.0.1' (using password: YES)
```

#规避这个 bug 也很简单，配置时输入密码加上双引号

```
[root@node3 ~]# mysql_config_editor set --login-path=app --user=app2 --host=127.0.0.1 -p
```

```
Enter password: 输入"123#abc"
```

```
[root@node3 ~]# mysql --login-path=app
```

```
Welcome to the MySQL monitor. Commands end with ; or \g.
```

```
Your MySQL connection id is 15542
```

```
Server version: 8.0.18 MySQL Community Server - GPL
```

```
Copyright (c) 2000, 2019, Oracle and/or its affiliates. All rights reserved.
```

```
Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.
```

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

```
mysql>
```

该 bug 在 MySQL 5.7.33 ， MySQL 8.0.23 修复。

总结

1. 在创建密码时尽量避免使用\$#等特殊字符，以免产生不必要的困扰。如果一定要使用请加上单引号或加上转义字符 \ 。
2. 在 MySQL 5.7.33 与 MySQL 8.0.23 版本前使用 login-path 时密码存在"#"时，配置时输入密码需要加上双引号。

3.18 命令行登录 MySQL 报 Segmentation fault 故障解决

作者：林浩

前段时间遇到一个 MySQL 客户端 crash 的问题，这个 MySQL 客户端是自己源码编译产生的。

为了解决这个问题，查阅了很多资料，涉及终端 ncurses 编程、进程的地址空间（堆和栈）、cmake、gcc 编译等，遇到不少“陷阱”，好在算是比较好的解决了这个问题。

环境：centos8.4 gcc8.4.1 mysql8.0.21 x86_64

问题描述：对 MySQL8.0.21 源码进行 make，由于一开始没安装 ncurses 库，在链接时遇到错误 undefined reference to，后来安装了该库，再次 make 成功。于是将 mysqld 启动，再用 mysql -u root -p 连接，输好密码回车后 MySQL 客户端发生 Segmentation fault。

第一次 make 时有编译警告（第二次 make 时不会有，因为.o 文件在第一次 make 时已经生成），摘要如下：

```
/opt/resource/mysql-8.0.21/extra/libedit/libedit-20190324-3.1/src/terminal.c: In
function 'terminal_set':
/opt/resource/mysql-8.0.21/extra/libedit/libedit-20190324-3.1/src/terminal.c:877:6:
warning: implicit declaration of function 'tgetent'; did you mean 'getenv'? [-Wimplicit-
function
-declaration]
i = tgetent(el->el_terminal.t_cap, term);
^~~~~~
getenv
```

```

/opt/resource/mysql-8.0.21/extra/libedit/libedit-20190324-3.1/src/terminal.c:899:15:
warning:
implicit declaration of function 'tgetflag' ; did you mean 'tigetflag' ? [-Wimplicit-
functi
on-declaration]
Val(T_am) = tgetflag("am");
^~~~~~
tigetflag
/opt/resource/mysql-8.0.21/extra/libedit/libedit-20190324-3.1/src/terminal.c:908:15:
warning:
implicit declaration of function 'tgetnum' ; did you mean 'tigetnum' ? [-Wimplicit-fun
ction-declaration]
Val(T_co) = tgetnum("co");
^~~~~~
tigetnum
/opt/resource/mysql-8.0.21/extra/libedit/libedit-20190324-3.1/src/terminal.c:917:19:
warning:
implicit declaration of function 'tgetstr' ; did you mean 'tigetstr' ? [-Wimplicit-functio
n-declaration]
char *tmp = tgetstr(strchr(t->name, *t->name), &area);
^~~~~~
tigetstr
/opt/resource/mysql-8.0.21/extra/libedit/libedit-20190324-3.1/src/terminal.c:917:19:
warning:
initialization of 'char *' from 'int' makes pointer from integer without a cast
[-Wint-conversion]

```

分析过程:

centos8.4 默认不能生成 core 文件，为了能临时生成 core 文件，需要对操作系统做如下配置：

```
# echo "core-%t.%p" > /proc/sys/kernel/core_pattern
```

意思是在执行程序当前目录生成 core- 为前缀，再带上时间戳和进程号的 core 文件，比如：core-1637149273.2955，其中 1637149273 是时间戳，2955 是进程号。

修改 core_pattern 文件内容后，再次使 MySQL 客户端发生 Segmentation fault，于是就有了 core 文件了。

gdb 查看 core 文件的函数堆栈信息如下：

```

gdb bin/mysql ~/core-1637149273.2955
(gdb) bt
#0 0x00000000004e4eed in terminal_alloc (el=0x286eee0, t=<optimized out>, cap=0x52a9
aaa0 <error: Cannot access memory at address 0x52a9aaa0>)
at /opt/resource/mysql-8.0.21/extra/libedit/libedit-20190324-3.1/src/terminal.c:350
288

```

```
#1 0x0000000004e5da7 in terminal_set (el=el@entry=0x286eee0, term=<optimized out>,
term@entry=0x0)
at /opt/resource/mysql-8.0.21/extra/libedit/libedit-20190324-3.1/src/terminal.c:900
#2 0x0000000004e5ee1 in terminal_init (el=el@entry=0x286eee0) at /opt/resource/mysql-
8.0.21/extra/libedit/libedit-20190324-3.1/src/terminal.c:297
#3 0x0000000004ea220 in el_init_internal (prog=0x7ffd52a9c6c9 "./mysql", fin=0x7fcd6f9
c09c0 <_IO_2_1_stdin_>,
fout=0x7fcd6f9c16e0 <_IO_2_1_stdout_>, ferr=0x7fcd6f9c1600 <_IO_2_1_stderr_>, fdin=
0, fdout=fdout@entry=1, fderr=2, flags=128)
at /opt/resource/mysql-8.0.21/extra/libedit/libedit-20190324-3.1/src/el.c:139
#4 0x0000000004e22d5 in rl_initialize () at /opt/resource/mysql-
8.0.21/extra/libedit/libedi
t-20190324-3.1/src/readline.c:297
#5 0x0000000004e2b55 in read_history (filename=filename@entry=0x286eec0 "/root/.my
sql_history")
at /opt/resource/mysql-8.0.21/extra/libedit/libedit-20190324-3.1/src/readline.c:1359
#6 0x00000000040924a in main (argc=<optimized out>, argv=<optimized out>) at /opt/r
esource/mysql-8.0.21/client/mysql.cc:1403
```

来看看 terminal.c 350 行附近的内容：

```
338 /* terminal_alloc():
339  * Maintain a string pool for termcap strings
340  */
341 static void
342 terminal_alloc(EditLine *el, const struct termcapstr *t, const char *cap)
343 {
344     char termbuf[TC_BUFSIZE];
345     size_t tlen, clen;
346     char **tlist = el->el_terminal.t_str;
347     char **tmp, **str = &tlist[t - tstr];
348
349     (void) memset(termbuf, 0, sizeof(termbuf));
350     if (cap == NULL || *cap == '\0') {
351         *str = NULL;
352         return;
353     } else
```

看这行的内容应该是内存地址非法访问造成 crash 的。再来看看 terminal.c 900 行附近的内容：

```
898 for (t = tstr; t->name != NULL; t++) {
899     /* XXX: some systems' tgetstr needs non const */
900     terminal_alloc(el, t, tgetstr(strchr(t->name, *t->name),
901         &area));
902 }
```

从第 900 行来看，应该是 tgetstr 函数返回值有问题。tgetstr 是 ncurses 库中的一个函数，为了更好的解决这个问题，有必要来了解一些终端编程的基本概念。

输入：man 3 tgetstr 来查看该函数的使用帮助，如下所示，我们可以知道终端能力有 3 种，分别是布尔值、数字值和字符串值，而该函数是用来获取字符串值的终端能力。同时，我们也可以知道，这个函数是给使用 termcap 库的应用使用的，后台会转换为 terminfo 库中的值。termcap 和 terminfo 都是描述终端能力的库，termcap 出现的比较早，已经被 terminfo 取代，但为了兼容性，termcap 的接口仍然保留。

```

mouse@review:~
curs_termcap(3X)                                curs_termcap(3X)

NAME
  PC, UP, BC, ospeed, tgetent, tgetflag, tgetnum, tgetstr, tgoto, tputs -
  direct curses interface to the terminfo capability database

SYNOPSIS
                                     I
#include <curses.h>
#include <term.h>

extern char PC;
extern char * UP;
extern char * BC;
extern unsigned ospeed;

int tgetent(char *bp, const char *name);
int tgetflag(char *id);
int tgetnum(char *id);
char *tgetstr(char *id, char **area);
char *tgoto(const char *cap, int col, int row);
int tputs(const char *str, int affcnt, int (*putc)(int));

DESCRIPTION
  These routines are included as a conversion aid for programs that use
  the termcap library. Their parameters are the same and the routines
  are emulated using the terminfo database. Thus, they can only be used
  to query the capabilities of entries for which a terminfo entry has
  been compiled.

INITIALIZATION
  The tgetent routine loads the entry for name. It returns:
Manual page tgetstr(3x) line 1/183 14% (press h for help or q to quit)

```

在 centos6 中可以使用：cat /etc/termcap 来查看所有终端的能力，/etc/termcap 是一个 ASCII 文件，这个文件在 centos7 和 centos 8 中已不存在。在 centos6/7/8 中可以使用：infocmp 来查看当前终端的能力，位于/usr/share/terminfo，terminfo 数据库保存的是编译后的内容。

这里引用一段 tgetstr 的使用说明 (来源：https://www.gnu.org/software/termutils/manual/termcap-1.3/html_node/termcap_5.html):

tgetstr

Use tgetstr to get a string value. It returns a pointer to a string which is the capability value, or a null pointer if the capability is not present in the terminal description. There are two ways tgetstr can find space to store the string value:

You can ask tgetstr to allocate the space. Pass a null pointer for the argument area, and tgetstr will use malloc to allocate storage big enough for the value. Termcap will never free this storage or refer to it again; you should free it when you are finished with it. This method is more robust, since there is no need to guess how much space is needed. But it is supported only by the GNU termcap library.

You can provide the space. Provide for the argument area the address of a pointer variable of type char *. Before calling tgetstr, initialize the variable to point at available space. Then tgetstr will store the string value in that space and will increment the pointer variable to point after the space that has been used. You can use the same pointer variable for many calls to tgetstr. There is no way to determine how much space is needed for a single string, and no way for you to prevent or handle overflow of the area you have provided. However, you can be sure that the total size of all the string values you will obtain from the terminal description is no greater than the size of the description (unless you get the same capability twice). You can determine that size with strlen on the buffer you provided to tgetent. See below for an example. Providing the space yourself is the only method supported by the Unix version of termcap.

从下图中，我们可以看到第 900 行的 area 指向了 buf，是上面英文提到的第 2 种用法，也即调用方分配好存储。

```

838  /* terminal_set():
839   * Read in the terminal capabilities from the requested terminal
840   */
841  libedit_private int
842  terminal_set(EditLine *el, const char *term)
843  {
844      int i;
845      char buf[TC_BUFSIZE];
846      char *area;
847      const struct termcapstr *t;
848      sigset_t oset, nset;
849      int lins, cols;
850
851      (void) sigemptyset(&nset);
852      (void) sigaddset(&nset, SIGWINCH);
853      (void) sigprocmask(SIG_BLOCK, &nset, &oset);
854
855      area = buf;

```

在 terminal.c 中加上打印来看看 buf、area 和 tgetstr 的值的变化情况：

```

char buf[TC_BUFSIZE];
printf("buf addr:%p\n", buf);

...

for (t = tstr; t->name != NULL; t++) {
    /* XXX: some systems' tgetstr needs non const */
    //terminal_alloc(el, t, tgetstr(strchr(t->name, *t->name),
    // &area));
    char *tmp = tgetstr(strchr(t->name, *t->name), &area);
    printf("area:%p\n", area);
    printf("tgetstr ret val:%p\n", tmp);
    terminal_alloc(el, t, tmp);
}

```

打印结果如下：

```

buf addr:0x7ffe0ec93660
area:0x7ffe0ec93664(第 1 次 for 循环)
tgetstr ret val:0xec93660(第 1 次 for 循环)

```

可以发现第 1 次 for 循环 tgetstr 的返回值是 buf 被截断低 4 个字节后的值，按道理应该和 buf 的值一样，所以会产生内存非法访问的错误，导致 segmentation fault。

问题到这里，令人百思不得其解，为什么就被截断了呢？

这时想起了编译时报的警告错误(写在文章开头)：implicit declaration of function，这个警告是缺少函数原型声明导致的，也就是第一次编译的时候没有安装依赖的 ncurses 库，从而缺少头文件 term.h，从而缺少 tgetstr 的函数原型声明。那么这个警告和函数返回值截断有没有关系呢？

通过下面的程序来测试一下，

foo.h

```
#ifndef __FOO_H__
#define __FOO_H__
void foo();
#endif
```

foo.c

```
#include <stdlib.h>
#include <stdio.h>
void foo()
{
    char buffer[1024];
    printf("buffer:%p\n", buffer);
    char *str = bar(buffer);
    printf("str:%p\n", str);
    printf("sizeof pointer:%d\n", sizeof(str));
    printf("sizeof int:%d\n", sizeof(int));
}
```

bar.c

```
#include <stdio.h>
char *bar(char *buffer)
{
    char *buf = buffer;
    printf("buf:%p\n", buf);
    return buf;
}
```

main.c

```
#include "foo.h"
int main(int argc, char *argv[])
{
    foo();
    return 0;
}
```

编译 foo.c 时报了如下警告 implicit declaration of function ，

```
$ gcc foo.c -c -o foo.o
foo.c: In function ‘foo’:
foo.c:7:15: warning: implicit declaration of function ‘bar’ [-Wimplicit-function-declaration]
    char *str = bar(buffer);
```


^~

```
foo.c:7:15: warning: initialization of 'char *' from 'int' makes pointer from integer
without a cast [-Wint-conversion]
```

执行结果:

```
$ ./main
buffer:0x7ffd563a8720
buf:0x7ffd563a8720
str:0x563a8720
sizeof pointer:8
sizeof int:4
```

通过输出结果不难看出，对于返回指针的 `bar()` 函数，其返回值被截断，只保留了低 32 位。这里对于 64 位系统有“陷阱”，在 64 位系统中，由于 `int` 是 4 字节，指针是 8 字节，存在被截断的问题，容易导致程序 `crash`，32 位系统应该不存在该问题，所以在 64 位系统上要注意该编译警告带来的潜在问题，另外，就是养成一个良好的编译习惯，最好不要有警告。

解决方法:

从该警告“implicit declaration of function”来看是由于缺少函数的原型声明，从 `man` 手册里知道 `tgetstr` 函数在 `term.h` 中有原型声明，只要 `terminal.c` 中 `include` 了这个头文件就可以了。在 `terminal.c` 中有如下代码，也就是说预编译 `if` 条件没成立。

```
66  /* Solaris's term.h does horrid things. */
67  #if defined(HAVE_TERM_H) && !defined(__sun) && !defined(HAVE_TERMCAP_H)
68      #include <term.h>
69  #endif
```

在目录：`extra/libedit/libedit-20191231-3.1/src` 中打开 `CMakeLists.txt`，发现有如下内容：

```
7  CHECK_INCLUDE_FILES (sys/cdefs.h HAVE_SYS_CDEFS_H)
8  CHECK_INCLUDE_FILES (term.h HAVE_TERM_H)
```

也就是说由于第一次 `make` 时没有安装 `ncurses` 依赖库，导致缺少 `term.h`，导致 `HAVE_TERM_H` 没有被定义。所以怎么解决这个问题也就比较清楚了。

1. 安装好 `ncurses` 库

```
sudo yum install -y ncurses-devel
```

2. 清理 `cmake` 缓存

这一步很必要，如果不清理缓存，`cmake` 还是会认为没有相应头文件 `term.h`。进到相应的二进制目录，执行：`rm CMakeCache.txt; rm -rf CMakeFiles`

3. 重新 `cmake` 和 `make`

不再出现编译警告“implicit declaration of function”，MySQL 输入用户名和密码后也不再 `crash`

了。

以上是我的解决方法，我们也来看看其他的处理方法，这种方法在网上能搜到很多，即：编辑文件 `terminal.c`，把 `terminal_set` 方法中的 `char buf [TC_BUFSIZE]`；这一行注释，再把 `area = buf;` 改为 `area = NULL;`。

为什么这种方法可以呢？我们来分析一下：

这其实是那段 `tgetstr` 英文用法的第一种，由 `tgetstr` 函数自己去 `malloc` 一块足够大小的内存，`tgetstr` 函数返回 `malloc` 出来的内存地址，那么同样的问题来了，编译警告“`implicit declaration of function`”会导致 `tgetstr` 函数返回值被截断为低 4 个字节，为什么 MySQL 客户端此时不会 crash 呢？

这里涉及到 Linux 系统中进程的地址空间里的堆(Heap)和栈(Stack)的概念，`malloc` 分配出来的内存是放在堆上，`terminal_set` 方法中的 `char buf [TC_BUFSIZE]` 是放在栈上。堆从进程的地址空间的低地址开始往高地址分配，栈从进程的地址空间的高地址开始往低地址分配。也就是说，堆的地址比栈的地址要低，我们来看一下打印出来的指针内容，

```
buf addr:0x7ffd8cc1e1a0
area:(nil)(第 1 次 for 循环)
tgetstr ret val:0x23e72fa(第 1 次 for 循环)
```

我们看到 `tgetstr` 函数的返回值 `0x23e72fa` 没有 32 位，也即证实了堆从进程的地址空间的低地址开始分配，既然没到 32 位，那么截断后的内容没有改变。

在 MySQL 的官网，我们可以找到这个问题：<https://bugs.mysql.com/bug.php?id=58497>。好了，总结到此完毕。

3.19 MySQL 启动遭遇 Permission denied 失败案例一则

作者：任坤

背景

OS: centos 7.9

MySQL: 5.7

Job for mysqld.service failed because the control process exited with error code. See "systemctl status mysqld.service" and "journalctl -xe" for details.

诊断

journalctl -xe 查看详细日志。

```
Oct 09 06:57:42 mysqld[30507]: 2022-10-09T06:57:42.953987Z 0 [Warning] Can't create test file /data/var/.lower-test
Oct 09 06:57:42 mysqld[30507]: 2022-10-09T06:57:42.954069Z 0 [Note] /usr/sbin/mysqld (mysqld 5.7.34-log) starting as process 30510 ...
Oct 09 06:57:42 mysqld[30507]: 2022-10-09T06:57:42.956924Z 0 [Warning] Can't create test file /data/var/.lower-test
Oct 09 06:57:42 mysqld[30507]: 2022-10-09T06:57:42.956958Z 0 [Warning] Can't create test file /data/var/.lower-test
Oct 09 06:57:42 mysqld[30507]: 2022-10-09T06:57:42.957892Z 0 [ERROR] Could not open file './err.log' for error logging: Permission denied
Oct 09 06:57:42 mysqld[30507]: 2022-10-09T06:57:42.957954Z 0 [ERROR] Aborting
```

查看文件权限，属主都是 mysql。

```
# ll -ld /data/var
drwxr-xr-x. 5 mysql mysql 4096 Oct 9 06:14 /data/var
# ll /data/var/err.log
-rw-r-- -- . 1 mysql mysql 33067 Oct 9 06:14 /data/var/err.log
```

service mysqld start 尝试好几次都以失败告终，手工执行 mysqld --defaults-file=/etc/my.cnf 能成功启动，说明 MySQL 的配置是正常的，问题应该出现在 OS 系统设置。

采用 strace 跟踪一下，

```
strace -tt -T -f -e trace=file -o service.log service mysqld start
```

输出内容

```
socket(AF_UNIX, SOCK_STREAM|SOCK_CLOEXEC|SOCK_NONBLOCK, 0) = 3
```

```
....
connect(3, {sa_family=AF_UNIX, sun_path="/run/systemd/private"}, 22) = 0
getsockopt(3, SOL_SOCKET, SO_PEERCRED, {pid=1, uid=0, gid=0}, [12]) = 0
getsockopt(3, SOL_SOCKET, SO_PEERSEC, "system_u:system_r:init_t:s0 -
s0:c"... , [64 - >40]) = 0
....
recvmsg(3, {msg_namelen=0}, MSG_DONTWAIT|MSG_NOSIGNAL|MSG_CMSG_CLOEXEC) = -
1 EAGAIN (Resource temporarily unavailable)
```

socket 设置出现了 selinux context 相关信息，应该是与 selinux 设置有关。

查看系统日志 /var/log/audit/audit.log 复核，明确记录了 err.log 权限验证失败信息。

```
type=SYSCALL msg=audit(1665296076.671:725): arch=c000003e syscall=2 succe
ss=no exit=-13 a0=1fd7900 a1=441 a2=1b6 a3=1aeb8380 items=0 ppid=1 pid=3616
auid=4294967295 uid=27 gid=27 euid=27 suid=27 fsuid=27 egid=27 sgid=27 fsgi
d=27 tty=(none) ses=4294967295 comm="mysqld" exe="/usr/sbin/mysqld" subj=sy
stem_u:system_r:mysqld_t:s0 key=(null)
type=PROCTITLE msg=audit(1665296076.671:725): proctitle=2F7573722F7362696
E2F6D7973716C64002D2D6461656D6F6E697A65002D2D7069642D66696C653D2F7661722F72
756E2F6D7973716C642F6D7973716C642E706964
type=AVC msg=audit(1665296076.671:726): avc: denied { append } for pid=36
16 comm="mysqld" name="err.log" dev="sdb1" ino=5505026 scontext=system_u:sy
stem_r:mysqld_t:s0 tcontext=unconfined_u:object_r:unlabeled_t:s0 tclass=fil
e permissive=0
```

确认机器开启了 selinux 。

```
# getenforce
Enforcing
# grep -v '^#' /etc/selinux/config
SELINUX=enforcing
SELINUXTYPE=targeted
```

Linux 提供两种鉴权认证体系，DAC(Discretionary Access control) 自主访问控制和 MAC(Mandatory Access control) 强制性访问控制，平时最常使用的是 DAC 。

DAC 将资源访问者分成三类，分别是 Owner 、 Group 、 Other ，针对这三类访问者设置访问权限，权限分成 read、write、execute ，访问者通过自己 uid/gid 和文件权限匹配，来确定是否可以访问。

这种策略有 2 个明显缺陷：

- 某个 OS 账号启动的任意进程，都拥有这个账号的所有权限，可以改动/删除这个账号下的所有文件资源。
- Root 账号拥有最高权限几乎可以做任何事情，一旦入侵者拿到 root 权限，就能完全掌控该机器。

MAC 对权限做了细化，每访问一个文件资源都需要进行验证，并且限制了 Root 账号权限，即使你有

root 权限，如果无法通过 MAC 验证，同样无法执行相关操作。

DAC 的权限访问粒度是用户级别，MAC 则是文件级别。

SELinux context 是一个冒号分隔的字符串，包含 4 个部分：user:role:type:range

上面的 `avc: denied` 报错信息可以做如下解读：

- `denied -> append` //缺少什么权限
- `scontext -> mysqld_t` //谁缺少权限
- `tcontext -> unlabeled_t` //对哪个文件缺少权限
- `tclass -> file` //什么类型的文件

这条信息可以解读为 `mysqld_t` 对 `unlabeled_t` 的 `file` 类型文件 `err.log` 缺少 `append` 权限。

解决方案

我们的机器初始化时会关闭 `selinux`，但是海外新环境没有运行初始化脚本，因此该问题最简单的解决方案就是关闭 `selinux`。

修改 `/etc/selinux/config` 文件，设置 `SELINUX=disabled`，重启机器，`mysqld` 可以正常启动。

3.20 MySQL 节点宕机分析一例

作者：周启超

之前遇到一个问题，用户反馈 MySQL MGR 环境发生了故障切换。作为一名运维，首先应该知道从日志信息里面去找信息。我们很幸运，在对应的时间点，有错误日志信息。而且可以明确的是，切换原因就在这段错误日志信息当中。错误日志中堆栈信息如下：

```
2022-04-12T03:03:36.435579+08:00 0 [ERROR][MY-
013183][InnoDB]Assertion failure: row0pread.cc:727:!page_cur_is_after_last(&page_cursor)
thread 140441717634816
InnoDB: We intentionally generate a memory trap.
InnoDB: Submit a detailed bug report to http://bugs.mysql.com.
InnoDB: If you get repeated assertion failures or crashes, even
InnoDB: immediately after the mysqld startup, there may be
InnoDB: corruption in the InnoDB tablespace. Please refer to
InnoDB: http://dev.mysql.com/doc/refman/8.0/en/forcing-innodb-recovery.html
InnoDB: about forcing recovery.
19:03:36 UTC-mysqld got signal 6;
```

Most likely, you have hit a bug, but this error can also be caused by malfunctioning hardware.

Thread pointer:0x0

Attempting backtrace. You can use the following information to find out where mysqld died. If you see no messages after this, something went terribly wrong...

stack_bottom=0 thread_stack 0x46000

/usr/sbin/mysqld(my_print_stacktrace(unsigned char const*, unsigned long)+0x41) [0x1efd01]

/usr/sbin/mysqld(handle_fatal_signal+0x333)[0xfadec3]

/lib64/libpthread.so.0(+0x12dd0)[0x7fc70518cdd0]

/lib64/libc.so.6(gsignal+0x10f)[0x7fc7030b099f]

/lib64/libc.so.6(abort+0x127)[0x7fc70309acf5]

/usr/sbin/mysqld()[0xd15c02]

/usr/sbin/mysqld(Parallel_reader::Scan_ctx::start_range(unsigned int, mtr_t*, dtuple_t const*, std::vector<std::pair<unsigned long, buf_block_t*>, ut_allocator<std::pair<unsigned long, buf_block_t*>>>&)const+0x27b)[0x211502b]

/usr/sbin/mysqld(Parallel_reader::Scan_ctx::create_ranges(Parallel_reader::Scan_range const&, unsigned int, unsigned long, unsigned long, std::vector<std::pair<std::shared_ptr<Parallel_reader::Scan_ctx::Iter>,

<Parallel_reader::Scan_ctx::Iter>,

std::shared_ptr<Parallel_reader::Scan_ctx::Iter>>,

ut_allocator<std::pair<std::shared_ptr<Parallel_reader::Scan_ctx::Iter>,

std::shared_ptr

<Parallel_reader::Scan_ctx::Iter> >> &, mtr_t*)+0x706) [0x2117476]

/usr/sbin/mysqld(Parallel_reader::Scan_ctx::create_ranges(Parallel_reader::Scan_range const&, unsigned int, unsigned long, unsigned long, std::vector<std::pair<std::shared_ptr<Parallel_reader::Scan_ctx::Iter>,

<Parallel_reader::Scan_ctx::Iter>,

std::shared_ptr<Parallel_reader::Scan_ctx::Iter>>,

ut_allocator<std::pair<std::shared_ptr<Parallel_reader::Scan_ctx::Iter>, std::shared_ptr

<Parallel_reader::Scan_ctx::Iter>>>>&, mtr_t*)+0xbc0)[0x2117930]

/usr/sbin/mysqld(Parallel_reader::Scan_ctx::partition(Parallel_reader::Scan_range const&, unsigned long)+0x17f)[0x2117fbf]

/usr/sbin/mysqld(Parallel_reader::Ctx::split()+0x65)[0x21183a5]

/usr/sbin/mysqld(Parallel_reader::worker(unsigned long)+0x8d) [0x211871d]

/usr/sbin/mysqld(std::thread::_State_impl<std::thread::_Invoker<std::tuple<Runnable, void (Parallel_reader::*)(unsigned long), Parallel_reader*, unsigned long> >>::_M_run()+0xd4)[0x210e2b4]

/lib64/libstdc++.so.6(+0xc2b23)[0x7fc703a98b23]

/lib64/libpthread.so.0(+0x82de)[0x7fc7051822de]

/lib64/libc.so.6(clone+0x43)[0x7fc7031754b3]

The manual page at <http://dev.mysql.com/doc/mysql/en/crashing.html> contains information that should help you find out what is causing the crash.

通过 `mysqld got signal 6`；可以初步判断为 MySQL 逻辑处理相关问题。根据对应堆栈信息找到对应的报错位置。确认问题代码位置为 `row0pread.cc, line 727`（其实本例中错误日志中已经提示了问题发生代码位置）：

```
(gdb) b *0x211502b
Breakpoint 1 at 0x211502b: file
/usr/src/debug/mysql-community-8.0.18-1.el8.x86_64/mysql-
8.0.18/storage/innobase/row/row0pread.cc, line 727.
```

MySQL 8.0.18 的 `row0pread.cc, line 727` 内容：

```
722
723     if (page_rec_is_infimum(page_cur_get_rec(&page_cursor))) {
724         page_cur_move_to_next(&page_cursor);
725     }
726
727     ut_a(!page_cur_is_after_last(&page_cursor));
728
729     return (page_cursor);
730 }
```

对比后续版本 MySQL 8.0.28 和 MySQL 8.0.18 差异，

可以看到 MySQL 8.0.28 中已经去掉了对应 `ut_a(!page_cur_is_after_last(&page_cursor));` 代码。

MySQL 8.0.18:

```
github.com/mysql/mysql-server/blob/mysql-8.0.18/storage/innobase/row/row0pread.cc

694 page_cur_t Parallel_reader::Scan_ctx::start_range(
695     page_no_t page_no, mtr_t *mtr, const dtuple_t *key,
696     Savepoints &savepoints) const {
697     ut_ad(index_s_own());
698
699     auto index = m_config.m_index;
700     page_id_t page_id(index->space, page_no);
701
702     /* Follow the left most pointer down on each page. */
703     for (;;) {
704         auto savepoint = mtr->get_savepoint();
705
706         auto block = block_get_s_latched(page_id, mtr, __LINE__);
707
708         savepoints.push_back({savepoint, block});
709
710         if (!page_is_leaf(buf_block_get_frame(block))) {
711             page_id.set_page_no(search(block, key));
712             continue;
713         }
714
715         page_cur_t page_cursor;
716
717         if (key != nullptr) {
718             page_cur_search(block, index, key, PAGE_CUR_GE, &page_cursor);
719         } else {
720             page_cur_set_before_first(block, &page_cursor);
721         }
722
723         if (page_rec_is_infimum(page_cur_get_rec(&page_cursor))) {
724             page_cur_move_to_next(&page_cursor);
725         }
726
727         ut_a(!page_cur_is_after_last(&page_cursor));
728
729         return (page_cursor);
730     }
731
732     ut_error;
733
734     return (page_cur_t{});
735 }
```

MySQL 8.0.28:

```

github.com/mysql/mysql-server/blob/mysql-8.0.28/storage/innobase/row/row0pread.cc
926 page_cur_t Parallel_reader::Scan_ctx::start_range(
927     page_no_t page_no, mtr_t *mtr, const dtuple_t *key,
928     Savepoints &savepoints) const {
929     ut_ad(index_s_own());
930
931     auto index = m_config.m_index;
932     page_id_t page_id(index->space, page_no);
933     ulint height();
934
935     /* Follow the left most pointer down on each page. */
936     for (;;) {
937         auto savepoint = mtr->get_savepoint();
938
939         auto block = block_get_s_latched(page_id, mtr, __LINE__);
940
941         height = btr_page_get_level(buf_block_get_frame(block), mtr);
942
943         savepoints.push_back({savepoint, block});
944
945         if (height != 0 && height != m_config.m_read_level) {
946             page_id.set_page_no(search(block, key));
947             continue;
948         }
949
950         page_cur_t page_cursor;
951
952         if (key != nullptr) {
953             page_cur_search(block, index, key, PAGE_CUR_GE, &page_cursor);
954         } else {
955             page_cur_set_before_first(block, &page_cursor);
956         }
957
958         if (page_rec_is_infimum(page_cur_get_rec(&page_cursor))) {
959             page_cur_move_to_next(&page_cursor);
960         }
961
962         return (page_cursor);
963     }
964 }

```

查找变更历史，找到官方对去掉 `ut_a(!page_cur_is_after_last(&page_cursor));` 的描述。找到 MySQL Bug#30060690。

涉及到如下 bug，详见 bug 描述。<https://github.com/mysql/mysql-server/commit/911be2d742985f191464c9f2dabcf81e0bac7cbc>

← → ↻ github.com/mysql/mysql-server/commit/911be2d742985f191464c9f2dabcf81e0bac7cbc#diff-b3ca0f9b00a88270faf67e1b658f5a8956d938505aa13e9a3bbc3d81e6c5f7ff

Bug#30060690 MYSQLD CRASH AFTER LOADING DATA

Issue
=====

During parallel read, if the entire load table is rolled back when autocommit = 0 we hit an assertion.

Fix
====

The assertion is bogus as the point that the existing tree structure could be changed during parallel read wasn't considered.

RB: 22847
Reviewed-by: Sunny Bains <sunny.bains@oracle.com>

Change-Id: I3b2a23a0fe0b398b254a76b3f60b284731d170fc

8.0
mysql-cluster-8.0.30 mysql-8.0.20

Darshan M N committed on 11 Dec 2019

Showing 2 changed files with 7 additions and 4 deletions.

Filter changed files

- storage/innobase
 - include
 - row
 - row0pread.cc

storage/innobase/include/row0pread.h

storage/innobase/row/row0pread.cc

```

@@ -539,7 +539,7 @@ dberr_t Parallel_reader::Ctx::traverse_recs(PCursor *pcursor, mtr_t *mtr) {
539 539     if (page_cur_is_after_last(cur)) {
540 540         nen_heap_empty(heap);
541 541
542 -         if (!move_to_next_node(pcursor, mtr)) {
542 +         if (!move_to_next_node(pcursor, mtr) || is_error_set()) {
543 543             break;
544 544         }
545 545         m_first_rec = true;
@@ -771,8 +771,6 @@ page_cur_t Parallel_reader::Scan_ctx::start_range(
771 771     page_cur_move_to_next(&page_cursor);
772 772     }
773 773
774 -     ut_a(!page_cur_is_after_last(&page_cursor));
775 -
776 774     return (page_cursor);
777 775 }

```

根据 bug 描述，可以看到此 bug 触发的原因是在 MySQL 8.0.18 版本，并行读没有考虑到存在的树结构变化的场景。

仅此一例，供大家日后遇到同样问题，查阅参考。

3.21 从一则 MGR 异常切换案例，看系统时间对 MGR 的影响

作者：李锡超

一、问题现象

9月15日，收到告警提示测试环境某系统的 MySQL MGR 集群存在异常。

日志内容如下：

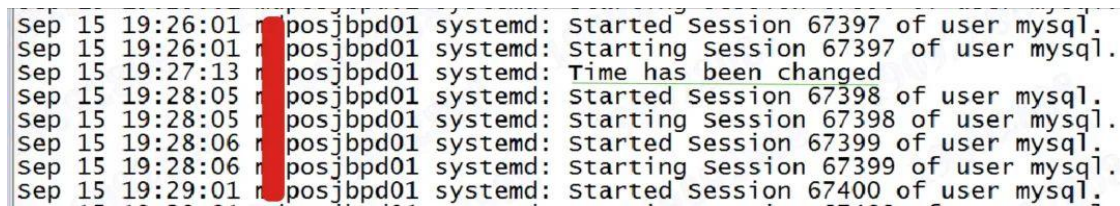
```
// 10.x.y.97 节点 MySQL 错误日志提示节点 10.x.y.95 不可达：
2022-09-15T19:26:14.320181+08:00 0 "Warning" "MY-011493" "Repl" Plugin
group_replication reported: "Member with address 10.x.y.95:3306 has become
unreachable."
// 随后 mgraliver(MGR 探针)日志提示进行了切换：
"2022-09-15 19:26:16" : Exception: Invalid primary_member_ip: or
secondary_node_list:""10.x.y.96", "10.x.y.97"" .
"2022-09-15 19:26:16" : Exception: MGR is likely to be switching, Sleep 1 sec and
continue .
"2022-09-15 19:27:01" : Exception: MGR running with WARN. ONLINE nodes Pri:10.x.y.96
Sec:10.x.y.97 diff from conf_node:10.x.y.95,10.x.y.96,10.x.y.97 .
```

即告警提示应用系统 MySQL MGR 发生了切换，由故障前的三节点 MGR 集群，切换为包括 Pri:10.x.y.96 Sec:10.x.y.97 两个节点集群。

二、初步分析

收到告警后，通过分析 MySQL 错误日志、操作系统系统、监控日志等信息，发现操作系统的的时间存在异常：

具体异常截图，如下图：



```
Sep 15 19:26:01 nposjbpd01 systemd: Started Session 67397 of user mysql.
Sep 15 19:26:01 nposjbpd01 systemd: Starting Session 67397 of user mysql.
Sep 15 19:27:13 nposjbpd01 systemd: Time has been changed
Sep 15 19:28:05 nposjbpd01 systemd: Started Session 67398 of user mysql.
Sep 15 19:28:05 nposjbpd01 systemd: Starting Session 67398 of user mysql.
Sep 15 19:28:06 nposjbpd01 systemd: Started Session 67399 of user mysql.
Sep 15 19:28:06 nposjbpd01 systemd: Starting Session 67399 of user mysql.
Sep 15 19:29:01 nposjbpd01 systemd: Started Session 67400 of user mysql.
```

即：存在【Time has been changed】异常。

结合 mysql 官方文档，其明确说明其故障检测基于时间：

- (1) 如果一个成员在 5 秒内没有收到另一个成员的消息，则怀疑该成员发生故障，并在自己的 Performance Schema 表 replication_group_members 中将该成员的状态列为 UNREACHABLE。
- (2) 如果怀疑持续超过 10 秒，则怀疑成员会尝试将其认为可疑成员有错误的观点传播给该组的其他成员。具体参考：<https://dev.mysql.com/doc/refman/8.0/en/group-replication-responses-failure.html>

由此初步怀疑是操作系统的时间发生了跳变，触发 MySQL MGR 发生故障切换。并将相关异常，反馈系统管理部相关老师进行进一步确认。

三、根本原因

经过系统专家深入分析，确认其根本原因是由于问题时段服务器底层存在异常。

导致 19:27:13 左右

虚拟机发生挂起（此时虚拟机的任何操作都不能进行，包括监控脚本、时间、MGR 的心跳等）。虚拟机挂起后，由于除故障节点外的其它两个节点无法与故障节点进行通信，因此认为该节点存在异常并将其驱逐。并最终产生如上告警信息。

至此，该问题得到最终确认。

四、关于时间对 MGR 的影响

既然该问题最终确认是由于虚拟机挂起导致主节点被驱逐。那么如果不是挂起，只是虚拟机的时间发生跳变，那还会触发故障切换么？

为此，通过测试环境进行了测试，其测试结论为：

当 MGR 集群中一个节点时间发生变化后（比如突然快了 1 小时、慢了 1 小时），MGR 集群的同步状态并不会因此受到影响。error_log 里面也未看到明显的报错！

即：MGR 节点的时间异常，并不会触发 MGR 发生故障切换。

那具体是什么机制呢？？为此结合源码进行确认！！

五、结合源码分析内部逻辑

我们知道，源码中涉及节点间探测，主要包括如下函数(alive_task/detector_task)：

函数调用关系

```
alive_task
  task_now
  may_be_dead
  task_now

detector_task
  check_global_node_set
  DETECT
  task_now
  check_local_node_set
  DETECT
  task_now
```

关键代码说明

```
// alive_task() :
int alive_task(task_arg arg MY_ATTRIBUTE((unused))) {
```

```

while (!xcom_shutdown) {
    ...
    // 超过 0.5 秒，广播我是 alive
    f (server_active(site, get_nodeno(site)) < task_now() - 0.5) {
        replace_pax_msg(&ep->i_p, pax_msg_new(alive_synode, site));
        ep->i_p->op = i_am_alive_op;
        send_to_all_site(site, ep->i_p, "alive_task");
    }
    ...
    {
        double sec = task_now();
        // 超过 4 秒没有心跳，询问你是否活着
        if (i != get_nodeno(site) && may_be_dead(site->detected, i, sec)) {
            replace_pax_msg(&ep->you_p, pax_msg_new(alive_synode, site));
            ep->you_p->op = are_you_alive_op;
            send_server_msg(site, i, ep->you_p);
        }
    }
    TASK_DELAY(1.0);
}

// DETECT
#define DETECTOR_LIVE_TIMEOUT 5.0
// 判断心跳是否超时
#define DETECT(site, i) \
    (i == get_nodeno(site)) || \
    (site->detected[i] + DETECTOR_LIVE_TIMEOUT > task_now())

static void check_global_node_set(site_def *site, int *notify) {
    for (i = 0; i < nodes && i < site->global_node_set.node_set_len; i++) {
        int detect = DETECT(site, i);
        if (site->global_node_set.node_set_val[i]) site->global_node_count++;
        // 本次捕获的心跳状态，则表示有节点心跳突变：无心跳->有心跳，有心跳->无心跳
        if (site->global_node_set.node_set_val[i] != detect) {
            // 需要通知全局状态发生变化
            *notify = 1;
        }
    }
}

static void check_local_node_set(site_def *site, int *notify) {

```

```

for (i = 0; i < nodes && i < site->global_node_set.node_set_len; i++) {
    int detect = DETECT(site, i);
    // 本次捕获的心跳状态，则表示有节点心跳突变：无心跳->有心跳，有心跳->无心跳
    if (site->local_node_set.node_set_val[i] != detect) {
        site->local_node_set.node_set_val[i] = detect;
        // 需要通知本地识别到可疑节点
        *notify = 1;
    }
}
}

// detector_task()
int detector_task(task_arg arg [[maybe_unused]]) {
    while (!xcom_shutdown) {
        {
            site_def *x_site = get_executor_site_rw();
            ...
            //判断是否有节点心跳超时，需要发起全局 view 变更通知
            check_global_node_set(x_site, &ep->notify);
            //更新全局节点个数
            update_global_count(x_site);

            if (ep->notify && iamtheleader(x_site)
                && enough_live_nodes(x_site)) {
                ep->notify = 0;
                //如果有节点心跳异常，且当前节点是 0 号主节点，且多数派的节点存活，则发送视图变更通
                //知，即驱逐心跳异常节点，并处理新节点加入或老节点退出
                send_my_view(x_site);
            }
        }

        if (x_site && get_nodeno(x_site) != VOID_NODE_NO) {
            //更新全局节点个数
            update_global_count(x_site);
            //判断是否有节点心跳超时，需要发起 suspicion 操作
            check_local_node_set(x_site, &ep->local_notify);
            if (ep->local_notify) {
                ep->local_notify = 0;
                //驱逐心跳异常节点
                deliver_view_msg(x_site);
            }
        }
    }
}

```

```

    }
    TIMED_TASK_WAIT(&detector_wait, 1.0);
}
}
}

```

此处为重点*

通过阅读以上代码，可知函数 `alive_task/detector_task` 都通过 `task_now()` 获取当前时间，并进行判断是否超过对应的阈值。那么进一步分析 `task_now()` 逻辑如下：

函数调用关系

```

task_now
  xcom_init_clock
    xcom_monotonic_seconds
seconds

```

关键代码说明

```

double task_now() {
    // 当第一步初始化后，!task_timer.done 始终未 false。
    if (!task_timer.done) {
        xcom_init_clock(&task_timer);
    }
    // 直接返回计算的 task_timer.now
    return task_timer.now;
}

static void xcom_init_clock(xcom_clock *clock) {
    // 调用 Linux 的 clock_gettime 函数，获取从系统启动时开始计时，
    // 以秒为单位（小于 1 秒以小数表示）。该时间不受系统影响，也不会被用户改变。
    clock->monotonic_start = get_monotonic_time();
    // 调用 Linux 的 clock_gettime 函数，获取系统时间（类似 date 命令），
    // 以秒为单位（小于 1 秒以小数表示）。该时间随着系统时间的改变而改变。
    clock->real_start = get_real_time();
    // 计算系统时间与启动计时的差值(offset)
    clock->offset = clock->real_start - clock->monotonic_start;
    // 通过差值+启动计时，得到时间。
    xcom_monotonic_seconds(clock);
    // 修改计算标记为 1。此后，只要 MGR 正常运行，
    // MGR 节点所获取时间等于=此处获取的差值(offset)+启动计时。
    // 因此，无论系统时间如何变化，MGR 都将获取“正确”的时间。
    // 具体逻辑见如下代码：
    clock->done = 1;
}

static double xcom_monotonic_seconds(xcom_clock *clock) {

```

```
// 初始化时获取的差值(offset)+启动计时。
clock->now = get_monotonic_time() + clock->offset;
return clock->now;
}

// 后续其它操作获取时间的函数：
double seconds() {
    // 当第一步初始化后, !task_timer.done 始终未 false
    if (!task_timer.done) { ,
        xcom_init_clock(&task_timer);
    }
    // 因此初始化之后调用 seconds()返回 : xcom_monotonic_seconds(&task_timer)
    return xcom_monotonic_seconds(&task_timer);
}
```

六、总结

基于如上代码分析，总结如下：

MGR 集群中获取时间=初始化时获取的固定差值(offset)+启动计时。由于 offset 是不变的值，启动计时在 OS 正常运行时，是一个恒定增加的数字。即 MGR 集群心跳的时间不受系统时间的控制。因此在步骤 3，进行测试时，未发现 MGR 集群存在异常。也确认本次故障确实不是由于时间发生变化而引起。

第 4 章 开源产品

本章节将介绍爱可生开源社区的开源产品相关的技术文章，通过这些文章能快速掌握 DBLE、DTLE、SQLE 等产品的使用，也希望有更多的社区志愿者加入到开源社区中来，成为我们的 commiter。

4.1 MySQL 开源许可研究

作者：陈书俊

1、Oracle 为什么可以对 MySQL 采用双许可证模式（GPLv2 开源许可证和商业许可证）发布？

在 MySQL 中引用的第三方源代码，全部都基于不具备许可证传染性的宽松开源许可证发布，或者基于双许可证发布，Oracle 在其开源版 MySQL 中对此类双许可证的第三方代码选择使用 GPLv2 许可证，在闭源的商业版 MySQL 中对此类双许可证的第三方代码选择使用更为宽松的许可证。

例如 MySQL 在引用 Memcached.pm 的源代码时有如下声明（Artistic license 就是更为宽松且没有传染性的开源许可证，允许修改或引用其源代码后不开源）：

Memcached.pm

Memcached.pm

Memcached.pm is licensed under the Perl license.

Oracle may use, redistribute and/or modify this code under the terms of either:

- a) the GNU General Public License as published by the Free Software Foundation; either version 1, or (at your option) any later version, or
- b) the "Artistic License" which comes with the Expect/pr code.

Oracle elects to use the GPLv2 for version of MySQL that are licensed under the GPL.

Oracle elects to use the Artistic license for all other (commercial) versions of MySQL.

A copy of the GPLv2 and the Artistic License (Perl) 1.0 must be included with any distribution.

This component is licensed under the GNU GPL license, version 2.0.

This component is licensed under Artistic License (Perl) 1.0

基于以上事实，再加上当第三方向 Oracle MySQL 贡献源代码时，都必须签署 Oracle 公司的 OCA 协议（Oracle Contributor Agreement），声明放弃所贡献的源代码的知识产权，这部分源代码的知识产权归 Oracle 公司所有，于是 Oracle 公司对 MySQL 源代码拥有了绝对的控制权与知识产权，Oracle 公司有权决定 MySQL 基于何种许可证分发。很多人或许有些疑惑：为什么明明 GNU 的 GPL 规定代码必须在较少约束的条款下发布，而版权持有者还可以提供私有许可证。答案是 GPL 的条款是版权持有者为所有其他人设置的；而版权所有者可以自由的决定是否对其本身应用这些条款。对此，一个好的理解方法是想象版权所有者在桶里有无数份软件的拷贝。每次它从桶中取出一个发送到世界上时，它可以决定是采用 GPL、私有或其他许可证。这并不是仅仅与 GPL 或其他任何开源许可证相关，它仅仅是版权法所赋予的权利。

2、Oracle 是否可以更改 MySQL 的开源协议或者干脆直接闭源？

基于问题 1 的结论，可以，但是仅限于后续新发布的版本。已经基于 GPLv2 许可证发布的老版本 MySQL 无法变更开源许可证或闭源。

3、为什么已经发布的老版本 MySQL 无法变更开源协议或闭源？

开源许可证是跟随软件拷贝分发的，每一份 MySQL 软件拷贝都带有一份开源许可证。当用户下载了一份 MySQL 软件拷贝的时候，就获得了这份软件拷贝想对应的许可证，作为软件拷贝发布方的 Oracle 公司无法回收已分发的软件拷贝和许可证，故也不可能变更该软件拷贝的许可证。可以作为上述结论佐证的，是在 GUN 官网上的一份 FAQ，其中有如下解释：

<http://www.gnu.org/licenses/gpl-faq.zh-cn.html#WhyMustIInclude>

为什么 GPL 要求在每个软件拷贝里都要包含一份 GPL 拷贝？

在每个拷贝里都包含许可证是关键性的，这样每个获得拷贝的人都知道他们的权利是什么。

包含一个指向许可证的 URL 而非许可证本身也许看起来很不错。但是你无法保证该 URL 五年或十年之后的有效性。20 年后，今天我们所用的 URL 可能不再存在了。

无论网络发生什么变化，唯一能够确保拥有拷贝的人们还能看到许可证的方法就是在程序中包含许可证的拷贝。

4、Oracle 变更 MySQL 开源许可证或闭源的可能性有多大？

尽管 MySQL 产品本身是开源的，使用 GPLv2 许可证，但是它的开发受到 Oracle 公司的控制，虽然也可能有人确实不满意 Oracle 公司把持着 MySQL 的开发而分叉这个项目（实际已经有人这么做了，比如 Percona、MariaDB 等）。一定程度上，这种威胁迫使 Oracle 公司需要慎重考虑变更许可证或闭源 MySQL 所带来的后果，这有可能导致 Oracle 主动放弃 MySQL 巨大的市场机会，而把这个机会拱手让人。在另一方面，一个完全闭源的 MySQL 是否有可能在目前的开源世界生态或之外的领域获得成功，也是 Oracle 需要考虑的。

我们无法揣测 Oracle 对于 MySQL 的想法，仅是从以上方面分析，我们认为 Oracle 更改目前 MySQL 商业策略的可能性不高。

5、如果后续版本的 MySQL 变更为更不友好的开源许可证或直接闭源，我们应该如何应对？

可以基于业已发布的老版本 MySQL 源代码创建代码分支，继续发展基于 GPLv2 开源许可的 MySQL 分支版本，并基于此分支版本发布商业发行版（必须开源，可以售卖软件许可或对用户提供技术服务作为商业模式）。

我们认为即使 Oracle 后续没有针对 MySQL 变更开源许可证或闭源的打算，发展国内自主的、有权威部门组织领导的 MySQL 分支版本项目也是十分必要的。原因如下：

1. 由于 Oracle 公司对 MySQL 项目的绝对控制，MySQL 并没有真正平等的开发社区，Oracle 只能从外部获得很小规模的 BUG 修复和补丁清理，并且需要投入大量公司内部资源支持 MySQL 项目的发展，一旦 MySQL 项目的收益过小，就有减小投入的可能性进而影响 MySQL 项目的发展。与此同时，不论是满足用户社区对新功能新特性的诉求还是 BUG 修复的响应速度，都不尽如人意，用户对于 MySQL 的期待和诉求无法得到及时的反馈与满足。
2. 由于中美竞争升级，国家对于软硬件自主可控的要求，Oracle MySQL 对于国产化软硬件的适配、国密算法、国内普遍业务量巨大的场景的支持等都是欠缺的，并且 Oracle 公司的 MySQL 需求评估部门，也不可能会为中国国情做适当的考虑。

基于以上两点，发展具有中国特色、符合中国国情、能充分响应国内用户需求的 MySQL 分支版本就显得十分急迫。

6、为什么 MySQL 在整个开源生态和在最终用户的考量中如此重要，我们可以不发展 MySQL 而转到其他路线上吗？

MySQL 是目前世界上最流行的开源数据库软件，市场占有率巨大，这是不可否认的事实。在我国国内，各行各业最终用户也大量使用了 MySQL 数据库，业已形成了巨大的资产投入。这些资产包括但不限于：基础软硬件设施、适配 MySQL 的应用软件开发、MySQL 生态的人才培养。在此基础上，国内已形成了庞大的围绕 MySQL 的软件生态和人才生态，大量最终用户把 MySQL 作为首选数据库软件使用。要替换如此庞大的软件、人才生态，是需要天量投入和强势领导的，其中的风险也是不言自明，这个风险不亚于 Oracle 突然闭源 MySQL 导致的后果。我们认为，与其承担巨大风险彻底放弃 MySQL 并转换路线，还不如牵头各权威部门、联合国内数据库行业内厂商，建立国内自主可控的 MySQL 分支社区，通过强力、科学的领导，形成统一的 MySQL 分支社区生态，谨防 MySQL 在国内的社区分裂化、碎片化。通过这个统一的、强力的 MySQL 分支开源社区，服务好国内庞大的 MySQL 最终用户群。

7、国内发展 MySQL 开源分支，是否会涉及知识产权侵权？

不会。要认清这个问题，首先需要了解 GPLv2 许可证遵循的最基本原则。

GPL 许可社会公众享有：运行、复制软件的自由，发行传播软件的自由，获得软件源码的自由，改进软件并让自己作出的改进版本向社会发行传播的自由。GPL 还规定：只要这种修改在整体上或者某个部分来源于遵循 GPL 的程序，该修改的整体就必须按照 GPL 流通，不仅该修改的源码必须向社会公开，并且对于这种修改的流通不准许附带修改者自己所作出的限制。因此，一项遵循 GPL 流通的程序不能同非自由的软件合并。GPL 所表达的这种流通规则称为 Copyleft，表示于 Copyright（版权）的概念相左。

GPL 协议最主要的几个原则：

1. 确保软件自始至终都以开放源代码形式发布，保护开发成果不被窃取用作商业发售。任何一套软

件，只要其中使用了受 GPL 协议保护的第三方软件的源代码，并向非开发人员发布时，软件本身也就自动成为受 GPL 保护并且约束的实体。也就是说，此时它必须开放源代码。

2. GPL 是 Copyleft，或译为“仅有著作权”的体现，除了著作权归原作者所有以外，其他一切权利均与任何人分享。GPL 的精髓就是，使软件在完整开源的情况下，尽可能使用户得到自由发挥的空间，使软件得到更快更好的发展。
3. 无论软件以何种形式发布，都必须同时附上源代码。例如在互联网上提供下载，就必须在二进制版本（如果有的话）下载的同个页面，清楚地提供源代码下载的链接。如果以光盘形式发布，就必须同时附上源文件的光盘。

从以上描述可以看出，GPL 是一个“自由”大于“版权”为原则的的许可证协议，在“自由”传播面前，其他一切均需让路。用户可以获得著作权人通过 GPL 许可证放弃的权利，但也必须遵守许可证的规定才能行使权利，如果不遵守 GPL 规定，便是侵犯了开源软件著作权，其著作权人就有权要求对方停止相关行为及其他。Oracle 一旦基于 GPLv2 许可证发布了一份 MySQL 软件拷贝，也就立刻放弃了其对这份 MySQL 软件拷贝所拥有的除了著作权之外的其他一切权利，由于 GPL 许可证的保护，这份 MySQL 软件拷贝立即成为了一份“自由”软件拷贝，可以被获得该软件拷贝的人或组织自由使用和修改、传播，并需要继续遵循 GPL 许可证的所有规定和约束。

综上所述，在发展国内 MySQL 开源分支时，只要遵循 GPLv2 的所有规定和约束，就不会触发知识产权层面的争议，我们可以合法的自由使用 Oracle 发布的 MySQL 源代码。

4.2 SQLE-SQL 审核平台体验报告

作者：刘新旺

1、Oracle 为什么可以对 MySQL 采用双许可证模式（GPLv2 开源许可证和商业许可证）发布？

在 MySQL 中引用的第三方源代码，全部都基于不具备许可证传染性的宽松开源许可证发布，或者基于双许可证发布，Oracle 在其开源版 MySQL 中对此类双许可证的第三方代码选择使用 GPLv2 许可证，在闭源的商业版 MySQL 中对此类双许可证的第三方代码选择使用更为宽松的许可证。

产品概述

1、体验环境

- 体验产品：SQLE
- 软件版本：sqle-ce-1.2111.0-pre2
- 部署环境：CentOS Linux release 7.9.2009 (Core)

2、产品介绍

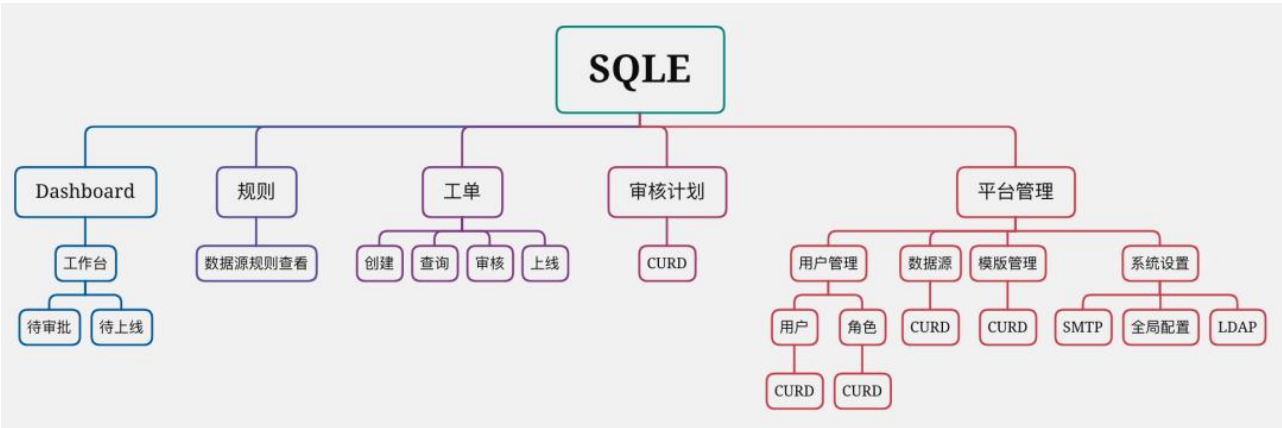
- SQLE (<https://opensource.actionsky.com/sqlc/>) 是由上海爱可生信息技术股份有限公司 开发并开源，支持多场景审核，支持标准化上线流程，原生支持 MySQL 审核且数据库类型可扩展的 SQL 审核工具。

3、需求分析

- 产品定位
 - 互联网时代，一个 app 可能承载成百上千万用户的使用；其业务规模之大，更新迭代之快，众多业务线日常上线 SQL 之多；DBA 对上线 SQL 的审核和执行工作变得十分繁忙，如何高效的保障 SQL 语句的高效执行和质量，对于系统的高效运行和长久稳定有着很大的影响。
- 用户需求分析
 - 核心需求为：开发人员自助、平台初审、DBA 复审、执行上线。
- 用户细分
 - 主要使用人员：开发人员和 DBA ；开发人员希望便捷、高效、自主可控的上线业务 SQL ；DBA 希望便捷、高效、安全、高质量的审核 SQL 上线。

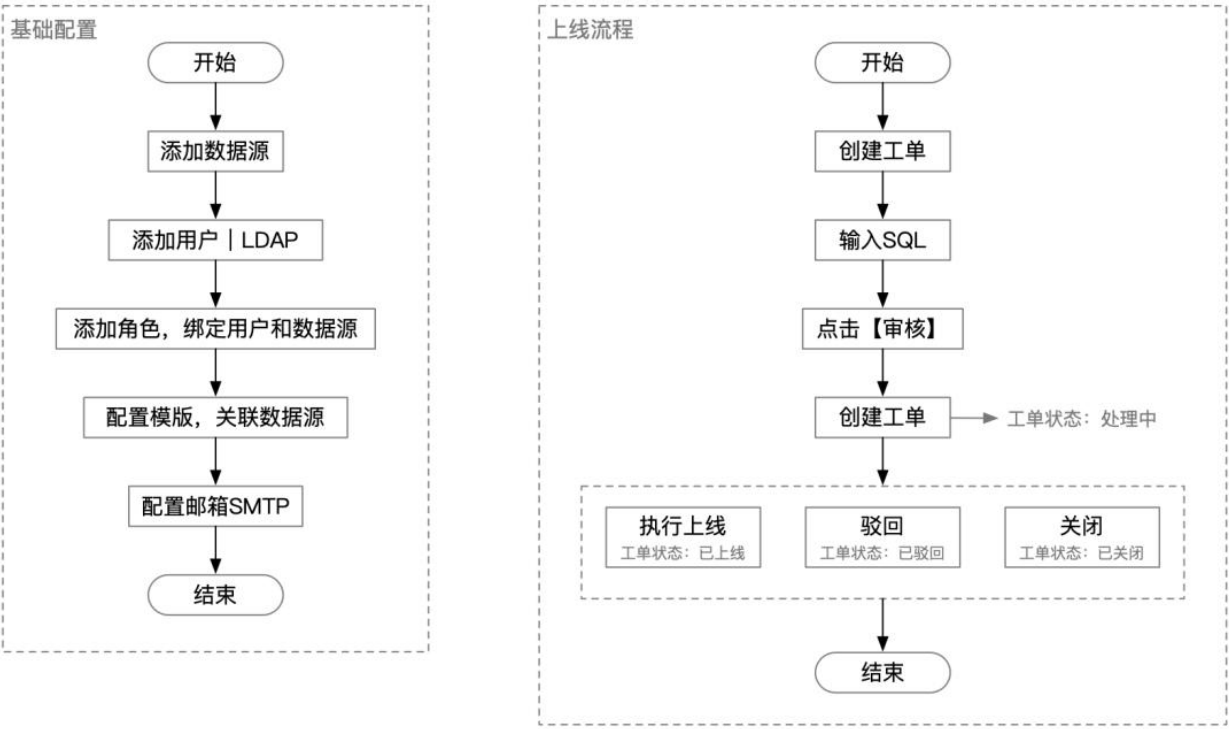
产品分析

1、产品结构图

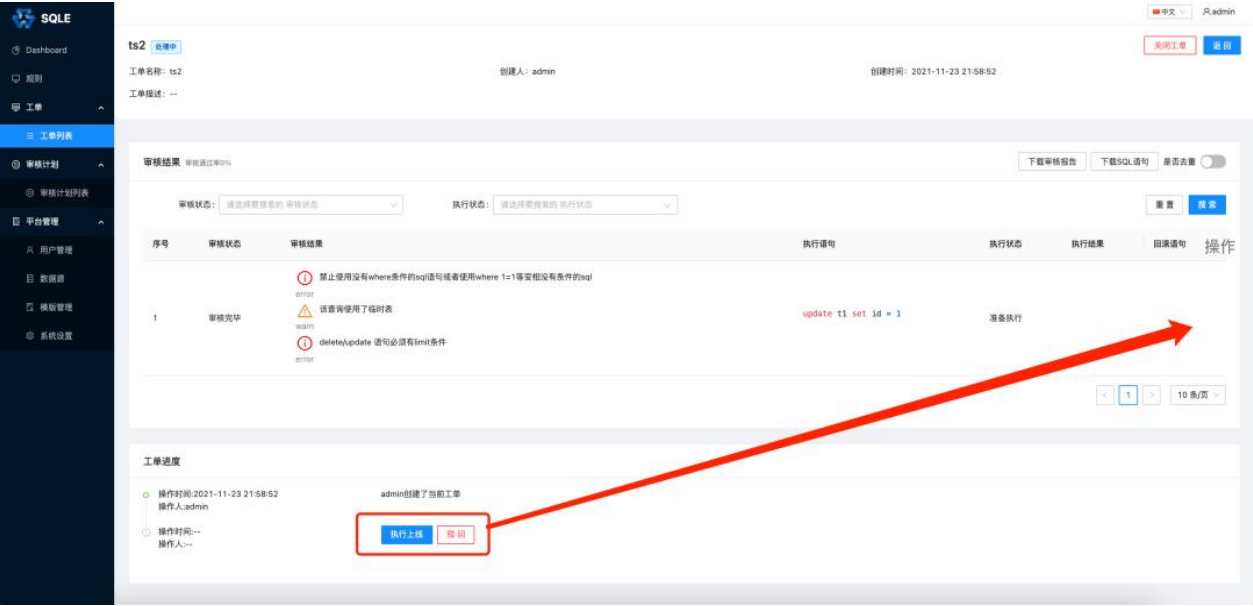


通过 SQLE 的产品结构图我们可以看到，SQLE 的界面设计较简洁，工作台可以看到与自己相关的待办工作；首页列表放置核心功能【工单】审核，其它功能收集到平台管理中；【规则】可以考虑收入到【平台管理】中；【审核计划】也可以考虑收入到【平台管理】中，审核的结果可以集成到工单中，方便统一进行处理。

2、产品使用流程图



通过 SQLE 的流程图可以看到，整体流程分为两个部分，一个是基础配置，配置好了基础配置以后，就可以执行上线流程了；对于审核不合格的功能依旧可以执行工单，没有起到自动审核的意义，对于 ERROR 级别的错误可以自动驳回不与上线；进入工单页面后不能很好的发现审核操作按钮（工单进度部分），需要下拉窗口才能发现；可以把审核操作放到审核结果列表后面加一个操作列，体验会更加友好。下图为缩放 67% 后才能看到下面的审核操作。



3、功能体验分析

- (1) 创建 SQL 工单-SQL 语句不支持输入联想，建议增加输入联想，提高录入效率
- (2) 创建 SQL 工单-SQL 语句对于明显语法错误不能及时提示，建议增加语法错误提示，提前发现明显问题
- (3) 创建 SQL 工单-SQL 语句输入框高度过高，点击审核后不方便查看到结果，建议降低高度增加宽度
- (4) 创建 SQL 工单-工单不支持自定义上线时间，同时也不支持定时上线

4、表现层

平台整体以白灰色色调为主，配色稳重简约，不同状态以不同颜色显示区别，猜测应该是用的 ant design 组件库；页面列表和 CURD 页面不够简洁和清晰，有待优化。

竞品分析

	SQLE	Archery
审核	✓	✓
查询	×	✓
执行	✓	✓
备份	×	✓
通知	email	钉钉、企业微信、邮件通知
流程	✓	✓
白名单	企业版	
慢日志审核	企业版	只有收集展示，暂无审核
审核计划	支持 MyBatis Scanner	×
扩展	支持插件化	支持插件化

使用感受

由于时间有限，仅是个人测试体验；SQLE 整体设计不错，符合现在主流平台的各方面特征；主要几个设计点我比较喜欢：

- DB 类型和实例关联模版这个管理力度很细
- 数据源关联流程，使审批粒度更加细致，管理更加方便
- SQL 白名单，应对特殊情况，避免业务被规则卡死，同时避免管理人员直接操作数据库；sql 指纹相当于提供了统配功能，更加方便
- 审核计划是个好东西；对于想集成 CI/CD 是一个很好的参考
- 审核支持插件化，更易扩展
- 后端使用 go 语言开发，部署兼容性和性能更加友好

4.3 SQLE 如何开发一条自定义的规则

作者: Jason

大家好！本次来分享的是如何开发一条自定义的 review rule 的规则（基于 MySQL 5.7 的语法）。

在开发代码前，我们先从后台 API 调用的角度梳理一下审核 SQL 的具体流程：

我们可以先用 POSTMAN 从调用一遍代码审核这个方法：

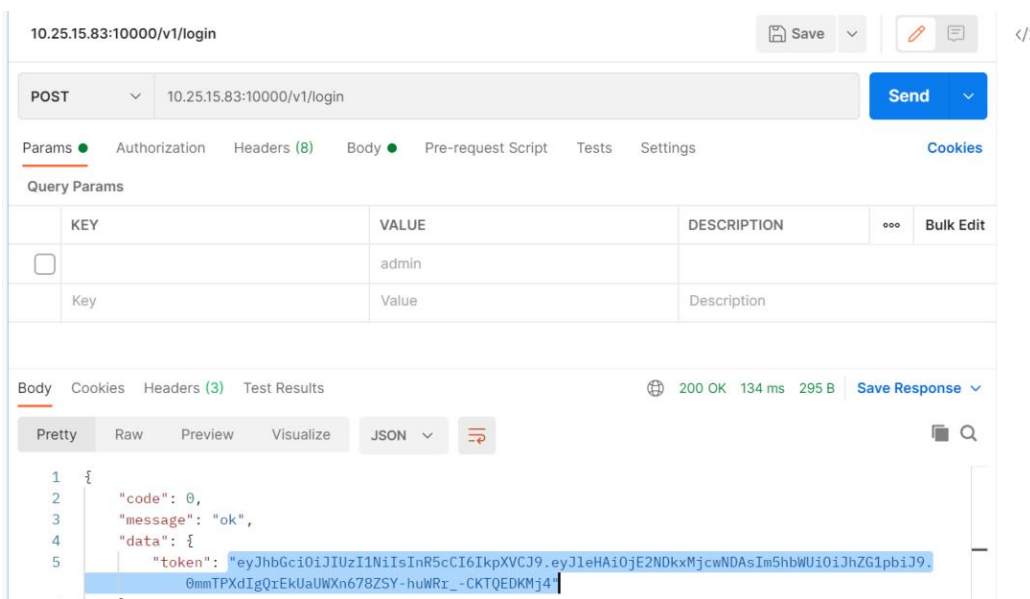
首先，先确定规则验证的 API 接口：`http://10.25.15.83:10000/v1/tasks/audits`

与 API 相对应的源代码方法的入口是：`api.controller.v1.task.go --> func CreateAndAuditTask(c echo.Context)`

由于系统做了 JWT 的登录验证，我们需要先模拟一下登录：

1) 调用登录的 API `10.25.15.83:10000/v1/login`，获得 token: *****

默认的是 `admin/admin`，登录成功后，我们可拿到 token



2) 我们调用一下规则验证的 API 接口：`http://10.25.15.83:10000/v1/tasks/audits`

我们准备了一下简单的 SQL 语句：

```
create table test (id int not null, name varchar(20));
```

这个语句不符合审计的规则如下：1)无主键 2)无列和表级别的 comment 3)无 innodb engine 指定

我们首先设置参数：可以参考 request 的结构体。

```

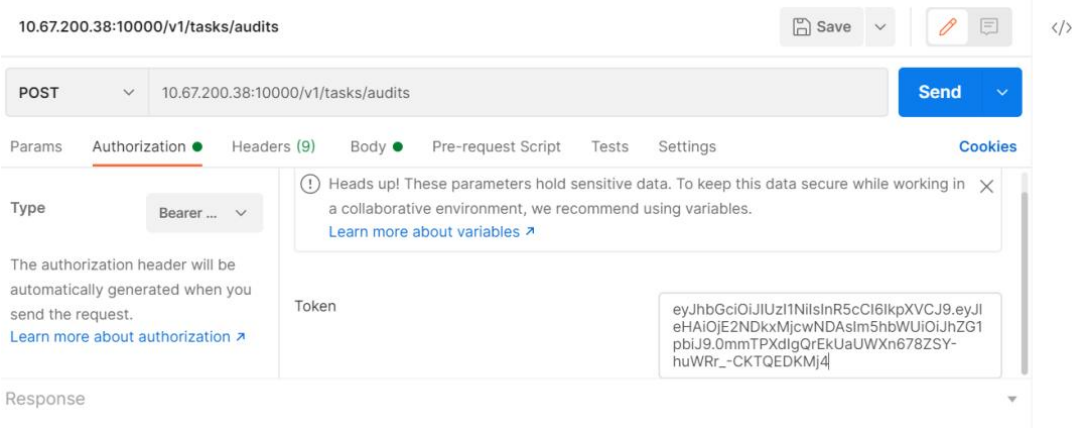
type CreateAuditTaskReqV1 struct {
  InstanceName string `json:"instance_name" form:"instance_name" example:"inst_1"
  valid:"required"`
  InstanceSchema string `json:"instance_schema" form:"instance_schema" example:"db1"`
}

```

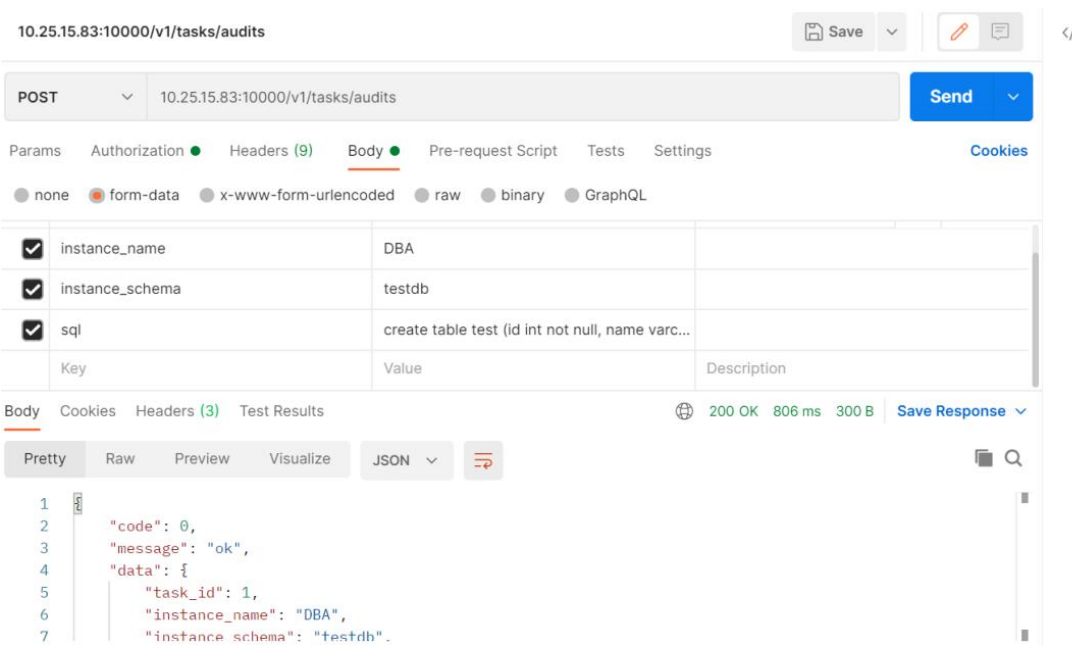
```
Sql string `json:"sql" form:"sql" example:"alter table tb1 drop columns c1"`
}
```

参数 1: InstanceName -》 DBA 参数 2: instance_schema =》 testdb 参数 3: sql =》 create table test (id int not null, name varchar(20));

我们还需要设置一下 token: 这个 token 从登录的 API 返回值可以获取到



下面我们尝试调用一下审计接口: <http://10.25.15.83:10000/v1/tasks/audits>



我们来看一下返回给我们的 response 的含义 :

```
{
"code": 0,
"message": "ok", --表示 api 调用成功
"data": {
"task_id": 1, --返回的审核结果保存在 task_is =1 的任务中
"instance_name": "DBA", --实例名
"instance_schema": "testdb", --DB 名
"audit_level": "error", --ERROR 级别的错误
"pass_rate": 0, --通过率为 0
}
```



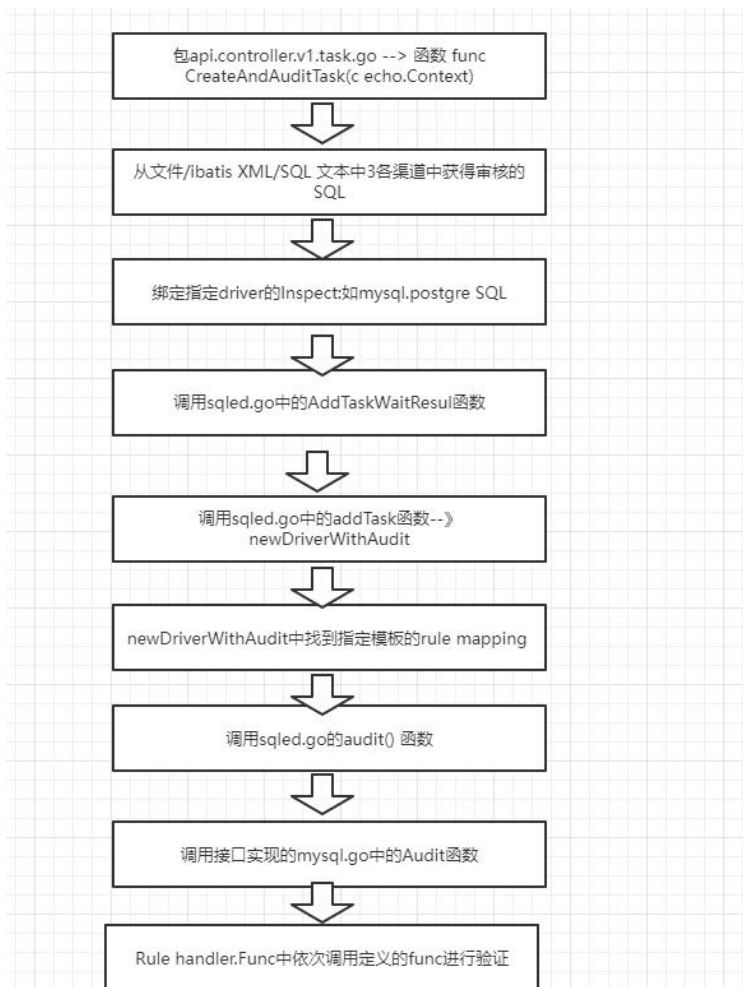
```
"status": "audited", --已经审计的状态
"sql_source": "form_data" --直接获得 SQL 语句的方式
}
}
```

我们要想知道具体的审计返回的结果需要查询数据库：（"task_id": 1, --返回的审核结果保存在 task_id=1 的任务中）

```
mysql> select audit_result from execute_sql_detail where task_id=1\G
***** 1. row *****
audit_result: [notice]列建议添加注释
[error]表必须有主键
[notice]必须使用 Innodb 数据库引擎
[notice]表建议添加注释
1 row in set (0.00 sec)
```

从 response 的信息来看 是完全符合我们之前的预期的结果。

下面我们要从代码的角度追踪一下这个审计的整个流程：SQLE 采用的是类似数据库 driver 插件的开发模式，接口 driver.go 已经定义好了抽象的函数，具体的实现需要每一种具体的数据库来完成 例如 mysql.go, oracle.go



下面是如何添加一条新的 rule 到审核平台的具体步骤：

New Rule 的规则是: 我们想要在新建的 TABLE 表中必须含有指定的列, 像是 cdate, creator, edate, editor 这几个审计字段。

首先找到对应的数据库里面 rule 的表是 rule:

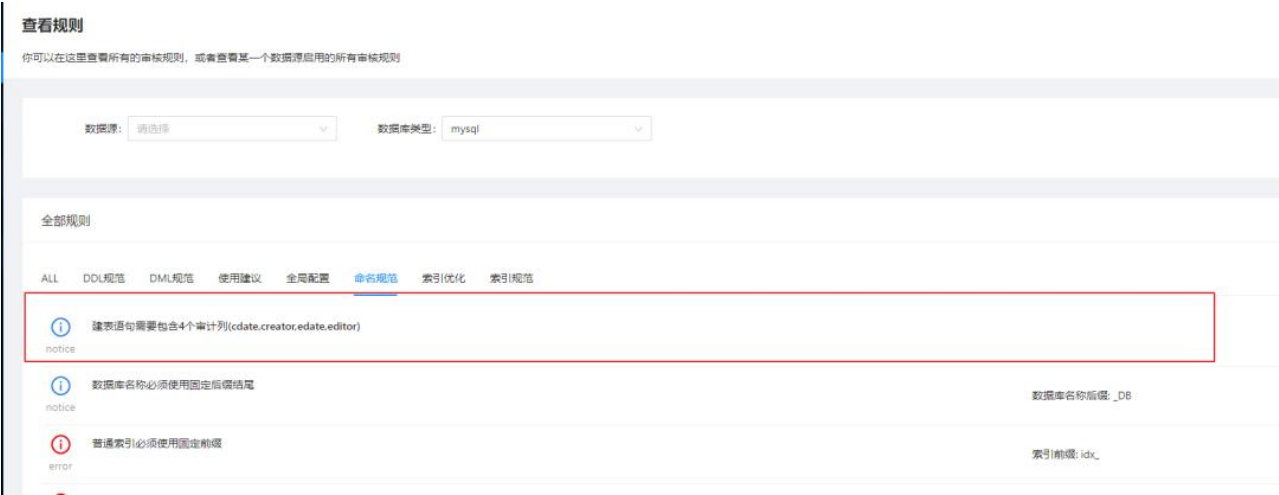
```
mysql> select * from rules \G
***** 28. row *****
name: ddl_check_object_name_using_keyword
db_type: mysql
desc: 数据库对象命名禁止使用保留字
level: error
type: 命名规范
params: NULL
***** 29. row *****
name: ddl_check_pk_name
db_type: mysql
desc: 建议主键命名为"PK_表名"
level: notice
type: 命名规范
params: NULL
....
```

我们来手动插入一条新 rule 的记录:

```
mysql> INSERT INTO sqle.rules
-> (name, db_type, `desc`, `level`, `type`, params)
-> VALUES('ddl_check_audit_column', 'mysql', '建表语句需要包含 4 个审计列
(cdate,creator,edate,editor)', 'notice', '命名规范', NULL);
Query OK, 1 row affected (0.00 sec)

mysql> commit;
Query OK, 0 rows affected (0.00 sec)
```

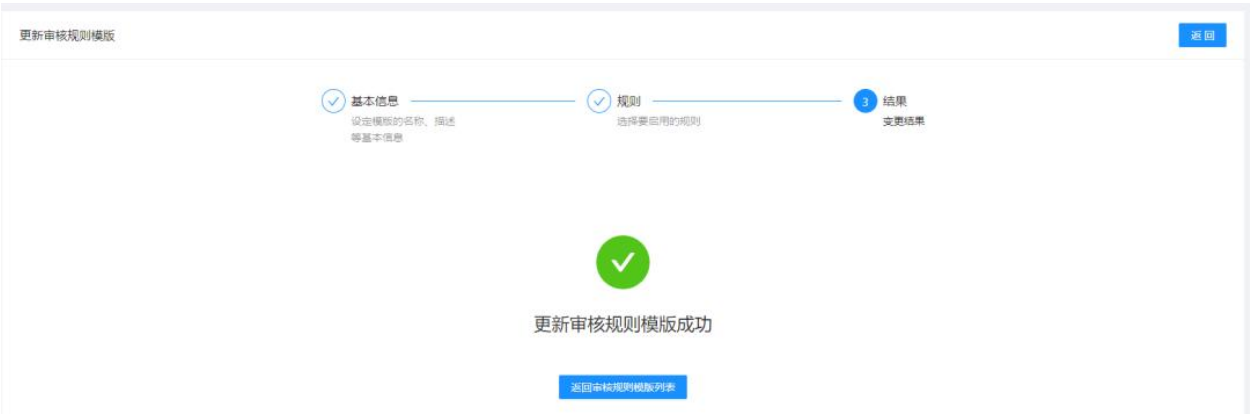
我们可以从页面看到这条新加的规则已经出现在了列表里面:



我们把这条新的规则追加到我们自定义的规则模板中：



最后一步，提交规则的模板信息。



下面我们从页面上测试一下我们新加的规则：（当然了，对应的规则要添加对应的后台代码，这个后面会有介绍~）

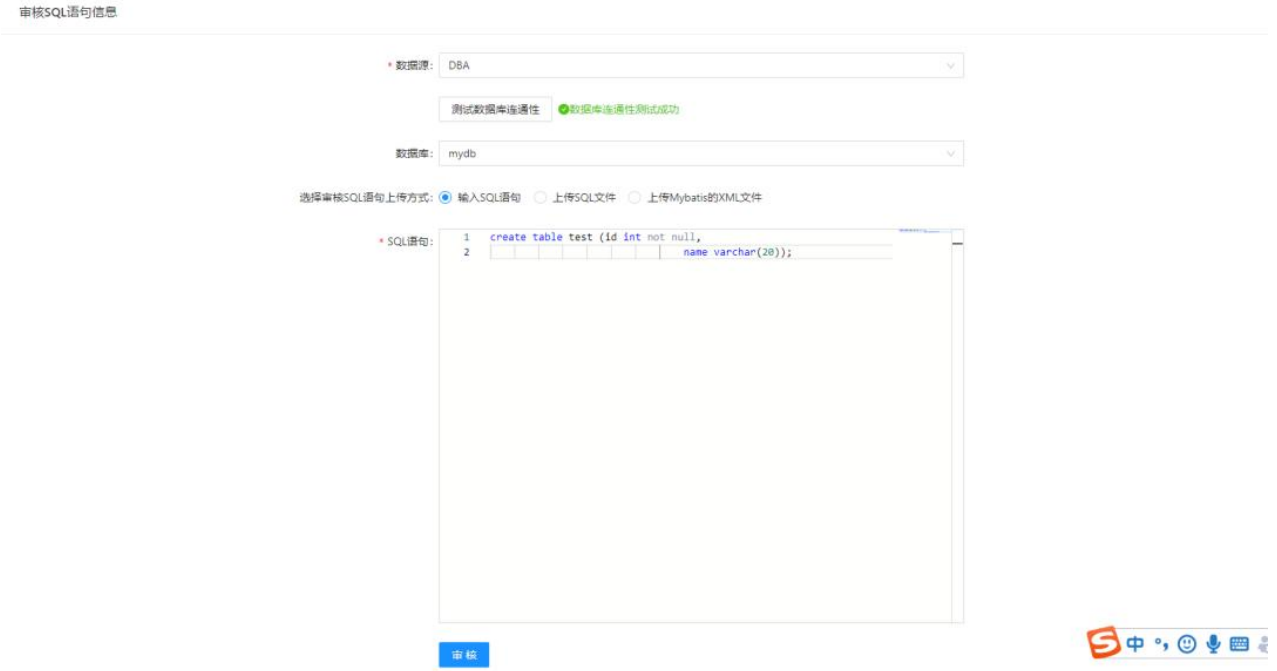
我们新建一个审核任务：

输入建表语句：

```
create table test (id int not null, name varchar(20));
```

点击审核按钮

4.3 SQLE 如何开发一条自定义的规则



我们可以看到我们自定义的审核规则已经生效：



最后我们来看看后台代码是如何开发实现这个规则的：

后台对应的这条规则的逻辑代码是：（入门级别的简单代码）这段代码加入到 `sqlc/driver/mysql/rule/rule.go` 中 用 `Map` 的结构体来判断，必要的列的名字是否存在 `review` 语句的表中

```
//可以在 linux 下进行命令行的代码单元测试
// dlv test github.com/actiontech/sqlc/sqlc/driver/mysql -- -test.run
^TestCheckAuditColumn$
func checkAuditColumn(ctx *session.Context, rule driver.Rule, res *driver.AuditResult,
node ast.Node) error {
var auditCols = [4]string{"cdate","edate","creator","editor"}
var set map[string]struct{}
set = make(map[string]struct{})
for _, value := range auditCols{
set[value] = struct{}{}}
}
```

```

var cnt int = 0
switch stmt := node.(type) {
case *ast.CreateTableStmt:
for _,value := range stmt.Cols {
fmt.Println(value.Name.Name)
if _, ok := set[value.Name.Name.String()];ok {
cnt++
}
}
if cnt != 4{
addResult(res, rule, rule.Name)
}
}

return nil
}

```

同时我们添加 rule 的 mapping 规则: sqle/driver/mysql/rule/rule.go 中

```

{
Rule: driver.Rule{
Name: DDLCheckAuditColumn,
Desc: "建表语句必须包含审计列(cdate,creator,edate,editor)",
Level: driver.RuleLevelError,
Category: RuleTypeUsageSuggestion,
},
Message: "建表语句必须包含审计列(cdate,creator,edate,editor)",
AllowOffline: true,
Func: checkAuditColumn,
},

```

添加常量定义: sqle/driver/mysql/rule/rule.go 中

```
DDLCheckAuditColumn = "ddl_check_audit_column"
```

我们还可以在 sqle/driver/mysql/audit_offline_test.go 进行 unit testing:

```

func TestCheckAuditColumn(t *testing.T) {
    fmt.Println("start.....")
    runSingleRuleInspectCase(rulepkg.RuleHandlerMap[rulepkg.DDLCheckAuditColumn].Rule, t,
        "create table test (id int not null, name varchar(20)); ",
        DefaultMysqlInspectOffline(),
        `create table test (id int not null,

```

```

        name varchar(20),
        cdate datetime,
        edate datetime,
        creator1 varchar(20),
        editor1 varchar(20));`,
    newTestResult().addResult(rulepkg.DDLCheckAuditColumn),
)
}

```

The screenshot shows a Go IDE with several tabs open: `audit_offline_test.go`, `rule.go`, `mysql.go`, `audit_test.go`, `driver.go`, `dml.go`, `rule_test.go`, and `sqled.go`. The main editor displays the `TestCheckAuditColumn` function in `audit_test.go`. The function uses `runSingleRuleInspectCase` to test a rule against a table definition. The table definition includes columns `id`, `name`, `cdate`, `edate`, `creator1`, and `editor1`. The test result is displayed in the bottom panel, showing that the test passed.

```

2182 }
2183
2184 func TestCheckAuditColumn(t *testing.T) {
2185     fmt.Println("start.....")
2186     runSingleRuleInspectCase(rulepkg.RuleHandlerMap[rulepkg.DDLCheckAuditColumn].Rule, t,
2187         desc: "create table test (id int not null, name varchar(20)); ",
2188         DefaultMysqlInspectOffline(),
2189         sql: `create table test (id int not null,
2190             name varchar(20),
2191             cdate datetime,
2192             edate datetime,
2193             creator1 varchar(20),
2194             editor1 varchar(20));`,
2195         newTestResult().addResult(rulepkg.DDLCheckAuditColumn),
2196     )
2197 }

```

TestCheckAuditColumn in github.com/actiontech/sqle... x

» ✓ Tests passed: 1 of 1 test - 14 ms

```

4 ms <3 go setup calls>
/usr/local/go/bin/go tool test2json -t /tmp/___1TestCheckAuditColumn_in_github_com_actiontech_sqle_sqle_driver_mysq
=== RUN    TestCheckAuditColumn
start.....
id
name
cdate
edate

```

最后测试通过后，我们即可提交代码，发布新的程序来从页面上验证我们自己定义的规则了。

总体来说，爱可生的代码是十分规范的，便于二次代码开发和维护。如果你是 `golang` 的开发爱好者，亦可作为学习项目的经典案例。

4.4 如何用 SQLE 自动审核 Java 应用

作者：孙健、张强

一. SQLE 介绍

SQLE 是一款由爱可生开源社区发起，面向数据库开发与管理人员，实现了 SQL 的「开发」-「测试」-「上线」等全流程覆盖，资源与权限精细化管理，兼顾简洁与高效，易维护、易扩展的开源项目，旨在为用户提供一套的安全可靠、自主可控的 SQL 质量管控方案。

在二月份发布的 1.2202.0 版本中：

- 改造了用户系统，新增用户组和操作权限；
- 增加对工单审核结果进行评分的功能；
- 增加审核任务，支持 MySQL 的库表结构、Oracle 的 TopSQL 以及 Java 应用的 SQL 抓取和审核；
- 增加了 Db2 审核插件，支持 Db2 审核工单并上线；
- 新增了 37 条 MySQL 审核规则，如检查 DDL 是否创建了冗余的索引；
- 其他体验优化和 Bug 修复。

其他细节信息，可以访问：<https://github.com/actiontech/sqle/releases/tag/v1.2202.0>。

二. Java 应用审核介绍

（友情提示：此为企业版功能）

考虑到很多用户在实际生产中已经部署了大量的、以 Java 为基础的应用和服务，并且某些已经涉及到了极其重要、不可中断的核心业务。从 1.2202.0 版本开始，SQLE 开始支持 Java 应用的 SQL 审核。并且在完成核心功能的基础上，支持 Java 应用零成本接入。

SQLE 的 Java 审核特性如下：

- 支持无侵入式的业务集成，用户可以直接以环境变量的方式运行来启动 Java 应用，直接接入 SQLE 而无需修改 Java 业务代码，减少了用户接入成本；
- 支持应用 SQL 的自动采集和统计，用户可以在 SQLE 上通过语句池页面统览应用内的 SQL 的分类与统计，迅速掌握应用内 SQL 的情况；
- 支持手动或自动审核 SQL，用户可以在审核报告中知晓应用的 SQL 是否符合预设的审核规则，无论是开发人员还是 DBA 都能实时了解正在运行的业务 SQL 执行状况。

三. 效果展示

1. 事先部署好环境、需要接入的 Java 应用以及对应的数据库，并添加为数据源。为了演示，此处 Java 项目为 <https://gitee.com/surveyking/surveyking>；
2. 创建 Java 应用的审核任务；

4.4 如何用 SQLE 自动审核 Java 应用

创建审核任务

返回

* 审核任务名称:

surveyking_test

数据源名称:

java_app_surveyking

数据库:

surveyking

* 数据库类型:

mysql

* 审核任务类型:

应用程序SQL抓取

* 任务审核周期:

可视化选择

手工填写

每 年 在 每月 月的 每天 日且星期 每天 的 每小时

每分钟

预览:*****

提交

重置

3. 启动应用程序；

```
SQLE_COLLECT_ENABLE=true \SQLE_HOST=XX.XX.XX.XX:10000 \
SQLE_TASK_NAME=surveyking_test \
SQLE_TASK_TOKEN=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcG4iOiJqd19hcHAiLCJleHAiOiJlE2NzcyMjYxNzcsIm5hbWUiOiJhZG1pbiJ9.3d0pA1hiVnFEWJokSFBwCT8d1pKOYV6SViENj4GFqgI \
java -jar surveyking-v0.3.0-beta.4.jar \
--server.port=1991 \
--spring.datasource.url=jdbc:mysql://XX.XX.XX.XX:3306/surveyking \
--spring.datasource.username=root \
--spring.datasource.password=xxxxxxx \
& >>/opt/surveyking/std.log
```

4. 查看 SQLE 审核任务详情界面，可以看到目前应用已经执行的 SQL ；

审核任务: surveyking_test

审核任务类型: 应用程序SQL抓取

您可以在这个查看当前审核任务的SQL语句池中的语句，以及当前审核任务的审核记录

当前审核任务SQL语句池

立即审核

SQL语句	最后一次匹配到该语句的语句	匹配到该语句的数量	最后一次匹配到该语句的时间
SELECT "RES"."* FROM "FLM_CHANNEL_DEFINITION" AS "RES" WHERE "RES"."VERSION"=(SELECT MAX("VERSION") FROM "FLM_CHANNEL_DEFINITION" WHERE "KEY_"="RES"."KEY_" AND ((TENANT_ID_" IS NOT NULL AND "TENANT_ID_"="RES"."TENANT_ID_" OR ("TENANT_ID_" IS NULL AND "RES"."TENANT_ID_" IS NULL))) ORDER BY "RES"."ID_"	SELECT RES.* from FLM_CHANNEL_DEFINITION RES WHERE RES.VERSION_ =(select max(VERSION_) from FLM_CHANNEL_DEFINITION where KEY_ = RES.KEY_ and ((TENANT_ID_ IS NOT NULL and TENANT_ID_ = RES.TENANT_ID_) or (TENANT_ID_ IS NULL and RES.TENANT_ID_ IS NULL))) order by RES.ID_ asc	1	2022-03-01 06:49:07
SELECT "VALUE_" FROM "ACT_GE_PROPERTY" WHERE "NAME_"=?	select VALUE_ from ACT_GE_PROPERTY where NAME_ = 'schema.version'	1	2022-03-01 06:49:04
SELECT "VALUE_" FROM "ACT_ID_PROPERTY" WHERE "NAME_"=?	select VALUE_ from ACT_ID_PROPERTY where NAME_ = 'schema.version'	1	2022-03-01 06:49:06
SELECT * FROM "ACT_GE_PROPERTY" WHERE "NAME_"=?	select * from ACT_GE_PROPERTY where NAME_ = 'cfg.task-related-entities-count'	2	2022-03-01 06:49:05
SELECT * FROM "SURVEYKING"."FLM_EV_DATABASECHANGELOG" ORDER BY "DATEEXECUTED","ORDEREXECUTED"	SELECT * FROM surveyking.FLM_EV_DATABASECHANGELOG ORDER BY DATEEXECUTED ASC, ORDEREXECUTED ASC	1	2022-03-01 06:49:06

审核记录

10条/页

1	生成时间2022-03-01 14:49:34
2	生成时间2022-03-01 14:50:00
3	生成时间2022-03-01 14:50:17
4	生成时间2022-03-01 14:51:00
5	生成时间2022-03-01 14:52:00

5. 查看审核报告，用户可以通过审核任务的审核报告了解到应用的 SQL 是否符合预设审核规则以便及时作出调整。

审核报告: 5

SQL语句	审核结果
<pre>select version()</pre>	
<pre>SELECT USER()</pre>	
<pre>SELECT COUNT(*) FROM surveyking.FLW_EV_DATABASECHANGELOG</pre>	禁止使用没有where条件的sql语句或者使用where 1=1等变相没有条件的sql error
<pre>SELECT * FROM surveyking.FLW_EV_DATABASECHANGELOG ORDER BY DATEEXECUTED ASC, ORDEREXECUTED ASC</pre>	禁止使用没有where条件的sql语句或者使用where 1=1等变相没有条件的sql error
<pre>select VALUE_ from ACT_ID_PROPERTY where NAME_ = 'schema.version'</pre>	
<pre>SELECT COUNT(*) FROM surveyking.ACT_CO_DATABASECHANGELOG</pre>	表 surveyking.ACT_CO_DATABASECHANGELOG 不存在 error 禁止使用没有where条件的sql语句或者使用where 1=1等变相没有条件的sql error
<pre>SELECT COUNT(*) FROM surveyking.ACT_APP_DATABASECHANGELOG</pre>	表 surveyking.ACT_APP_DATABASECHANGELOG 不存在 error 禁止使用没有where条件的sql语句或者使用where 1=1等变相没有条件的sql error
<pre>SELECT RES.* from FLW_CHANNEL_DEFINITION RES WHERE RES.VERSION_ = (select max(VERSION_) from FLW_CHANNEL_DEFINITION where KEY_ = RES.KEY_ and ((TENANT_ID_ IS NOT NULL and TENANT_ID_ = RES.TENANT_ID_) or (TENANT_ID_ IS NULL and RES.TENANT_ID_ IS NULL))) order by RES.ID_ asc</pre>	

<

1

2

>

10 条/页

6. 在示例 Java 应用主页创建一个用户，然后便可以在审核任务对应的 SQLE 语句池内观察到所对应的 INSERT 语句。

卷王

首页

我的问卷

模板广场

系统管理

用户管理

角色管理

组织机构

岗位设置

首页 / 系统管理 / 用户管理

组织机构

卷王

姓名: 请输入

系统用户列表

组织机构

卷王

新建用户

* 用户名

test

* 登录名

test

* 输入密码

....

* 确认密码

....

* 性别

男

* 状态

激活

手机

请输入手机号

邮箱

请输入邮箱

所属部门

卷王

角色列表

Admin

选择岗位

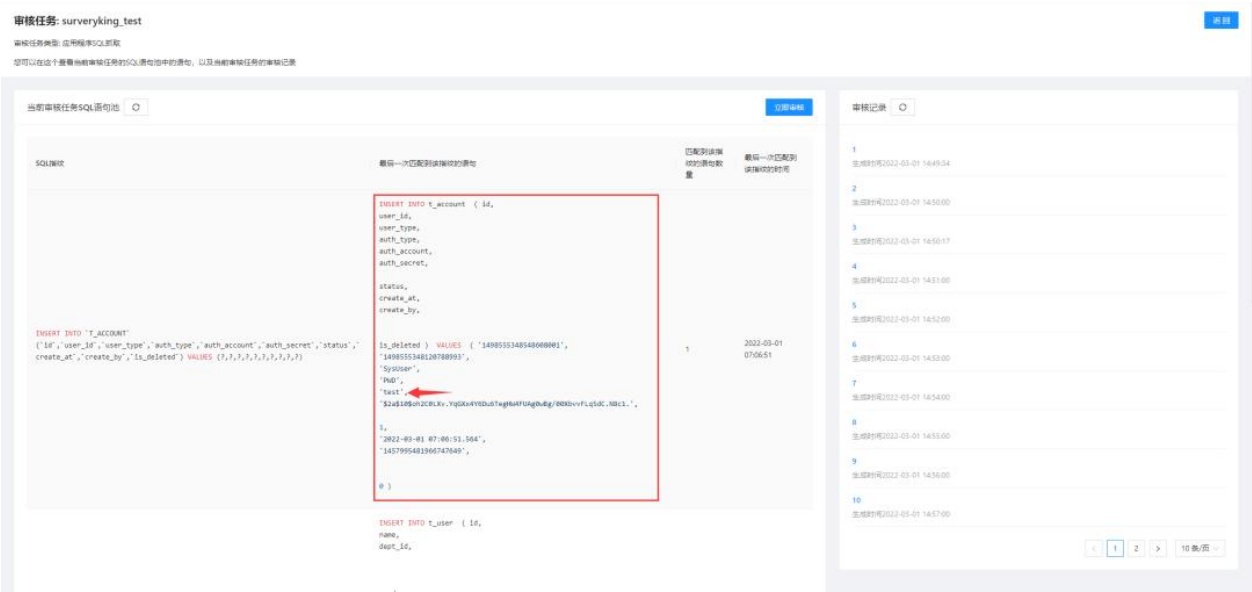
机构名称	岗位名称	操作
+ 添加一行数据		

暂无数据

取消

确认

同时，用户可以从语句池内页面迅速地感知到应用内执行的 SQL 的语句分类和统计情况。



综合以上的示例步骤，Java 应用零成本地接入了 SQLE。开发人员和 DBA 可以通过「审核报告」、「审核任务的 SQL 语句池」等功能完全掌握应用内的 SQL 审核结果、执行情况以及统计信息，完成了从「开发」直至「上线」这一过程中的 SQL 质量管控。

4.5 如何使用 SQLE 进行开发阶段 SQL 审核

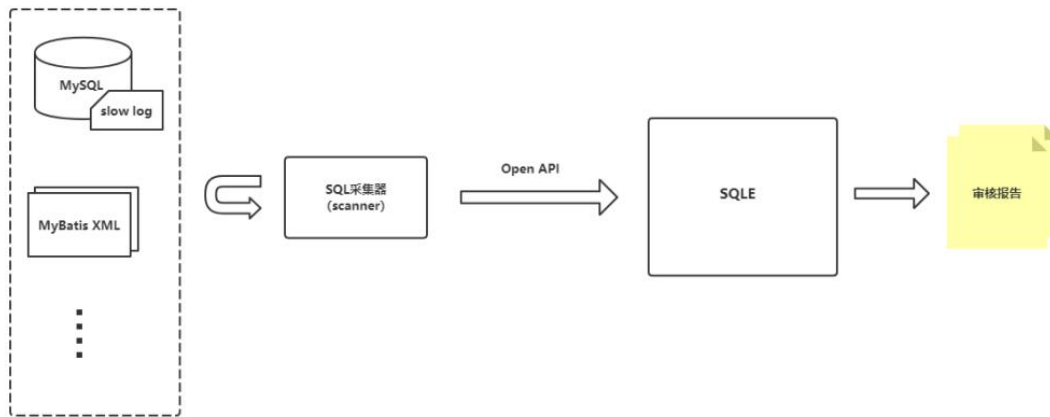
作者：孙健

一、SQLE 介绍

SQLE 是一款由爱可生开源社区发起，面向数据库开发与管理人员，实现了 SQL 的「开发」-「测试」-「上线」等全流程覆盖，资源与权限精细化管理，兼顾简洁与高效，易维护、易扩展的的开源项目，旨在为用户提供一套的安全可靠、自主可控的 SQL 质量管控方案。

二、功能介绍

SQLE 的开发阶段 SQL 审核主要是通过 SQLE 的功能审核任务来完成的，该功能定位是应对全场景 SQL 质量管控解决方案，能够在应用开发、测试、上线、生产等不同阶段对 SQL 进行质量管控。将 SQL 采集与 SQL 审核结合对数据库进行长期审核，目前支持 Mybatis 扫描，慢日志，库表元数据，TopSQL，应用程序 SQL 抓取（java 应用）的场景审核。应用开发阶段可以通过审核任务的 Mybatis 扫描类型的任务对应用代码做实时审核，Mybatis 扫描的审核任务需要通过 SQLE 提供的 Scanner 工具进行 SQL 采集并推送到 SQLE 进行 SQL 审核， 以下是 SQLE 通过 Scanner 采集 SQL 的流程图：



目前开发阶段基本都是使用 CI/CD 平台进行持续集成，例如 Jenkins ， GoCD ， git 平台的 CI/CD 等，该阶段主要用来进行代码的检查，单测，打包。好处是标准化开发流程，减少复杂的工作量。因此在开发阶段也可以将 SQL 审核作为一个检查模块集成进 CI 流程。将 SQL 审核事宜前置到开发阶段能够减少后期 SQL 审核的压力。目前使用 SQLE 的审核任务能够做到：

- SQLE 提供 Scanner 模式可以集成几乎所有 CI/CD，配置简单，可以轻易集成进现有的开发流程；
- SQL 审核结果即时反馈，对于未通过的 SQL 审核，开发可以即时优化；
- 除了 CI 流程之外，也可以在 SQLE 界面进行 SQL 分类统计和审核报告查看。

三、效果演示

1. 创建审核任务

创建审核任务
返回

* 审核任务名称:
app1

数据源名称 ②:
请选择

数据库:
请选择

* 数据库类型:
mysql

* 审核任务类型:
Mybatis 扫描

* 任务审核周期:
☒ 可视化选择
☐ 手工填写

每 年 在 每月 月的 每天 日且星期 每天 的 每小时 :

每分钟

预览: *****

提交
重置

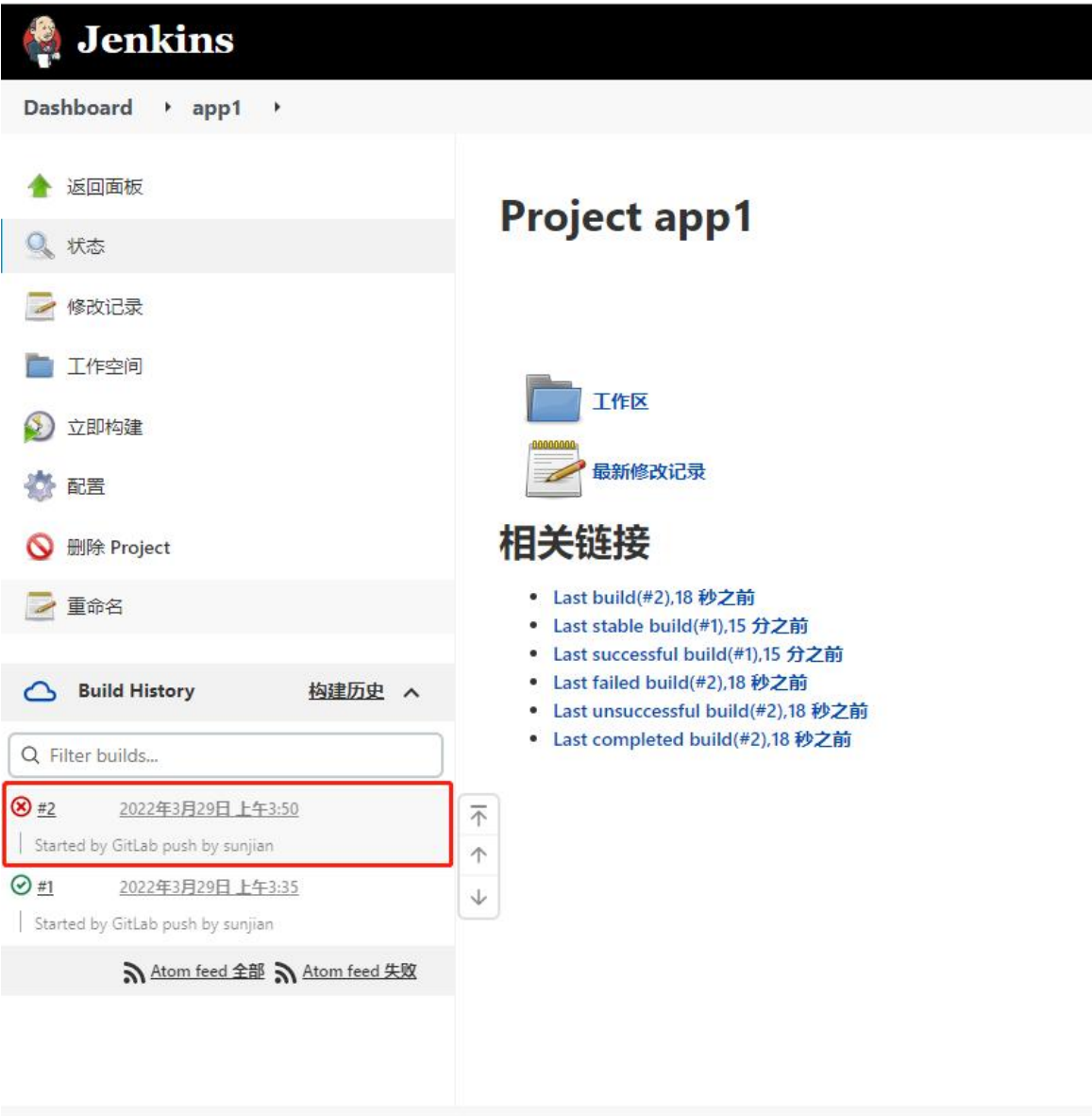
2. 在 jenkins 对应的任务上加一个构建流程

SQLE 的 Scanner 与 CI/CD 的集成方式是通过添加 bash 类型的任务，一般 CI/CD 都支持该方式。SQLE Scanner 本质上是一个二进制命令行工具，该工具集成在 SQLE 内，可以在 SQLE 平台安装目录的 bin 下找到，具体的使用方式和命令行参数解释参考：https://actiontech.github.io/sqle-docs-cn/3.modules/3.6_auditplan/scanner_management.html。



3. 当 jenkins 任务触发时会进行 SQL 审核

Jenkins 集成后进行 SQL 审核的效果如下两张图所示：



⊗ 控制台输出

[illegible]

审核结果会立即通过 Jenkins 的任务返回，当 SQL 不合规触发规则建议，可以看到上图所示的构建失败的结果，此时 CI 流程会通知开发，开发需要优化完不合规 SQL 后再次提交代码触发 CI 构建，如果 SQL 优化成功则 SQL 审核会构建成功。

4. SQLE 页面查看结果

开发负责人可以通过 SQLE 平台查看审核结果和 SQL 统计，提前了解目前项目中 SQL 的开发质量和碰到的问题。SQL 统计信息和审核结果如下两张图所示：

审核任务: app1

审核任务类型: Mybatis 扫描

您可以在这里查看当前审核任务的SQL语句池中的语句, 以及当前审核任务的审核记录

当前审核任务SQL语句池

立即审核

SQL指纹	最后一次匹配到该指纹的语句	匹配到该指纹的语句数量	最后一次匹配到该指纹的时间
SELECT * FROM `users`	SELECT * FROM `users`;	1	2022-03-29T03:50:31Z
SELECT * FROM `users` WHERE `name`=? AND `age`>?	SELECT * FROM `users` WHERE `name`=? AND `age`>;	1	2022-03-29T03:50:31Z
SELECT `id`,`name`,`age`,`score` FROM `users` WHERE `score`>?	SELECT `id`,`name`,`age`,`score` FROM `users` WHERE `score`>60;	1	2022-03-29T03:50:31Z
SELECT `id`,`name`,`age`,`score` FROM `users` WHERE `name` LIKE ?	SELECT `id`,`name`,`age`,`score` FROM `users` WHERE `name` LIKE ?;	1	2022-03-29T03:50:31Z

审核记录

生成时间2022-03-29 11:50:32
审核结果评分 45 审核通过率75%

< 1 > 10条/页

审核任务: app1

返回

审核任务类型: Mybatis 扫描

您可以在这里查看当前审核任务的SQL语句池中的语句，以及当前审核任务的审核记录

审核报告: 1

SQL语句	审核结果
<code>SELECT * FROM `users`;</code>	i error 禁止使用没有where条件的sql语句或者使用where 1=1等变相没有条件的sql
<code>SELECT * FROM `users` WHERE `name`=? AND `age`>?;</code>	
<code>SELECT `id`,`name`,`age`,`score` FROM `users` WHERE `score`>60;</code>	
<code>SELECT `id`,`name`,`age`,`score` FROM `users` WHERE `name` LIKE ?;</code>	

<

1

>

10 条/页

四、总结

以上最简单的 SQL 审核与 CI/CD 流程集成的试用，大家可以下载安装 SQLE 并结合自己公司 CI/CD 试一下，SQLE 的安装参考主页：<https://github.com/actiontech/sqle> ；

更多使用场景可以参考：https://actiontech.github.io/sqle-docs-cn/3.modules/3.6_auditplan/introduction.html 。

4.6 如何快速使用 SQLE 审核各种类型的数据库

作者：孙健

前言

近些年来，数据库产业发展迅猛，各种新兴数据库如雨后春笋般出现，各个公司的技术栈也不再局限于某一种数据库。对于 SQL 质量管理平台来说仅支持某一个类型的数据库（例如 MySQL），那么是会有有一定的局限性，SQLE 在设计之初考虑支持多种数据库，因此产品设计时，将审核流程（业务）的代码和具体 SQL 审核上线的代码进行分离，SQL 审核上线通过插件的形式实现。SQLE 对外提供插件开发所需的接口和库，可以快速创建开启一个审核插件，无需升级软件，导入审核插件即可获对应数据库类型的审核上线能力，使用平台所有功能。

本文将演示如何从零开始创建一个简单可用的审核插件，作为案例。

目标

首先将创建一个 Postgres 数据库审核插件，并添加两条规则，“禁止使用 SELECT *”和“创建的表字段过多”，并在开发过程中结合 SQLE 对 Postgres 数据库进行 SQL 审核上线工单的测试演示。以下过程中的演示代码可从此处 https://github.com/actiontech/sqle/tree/main/example/db_plugin 下载。

实操

提示：SQLE 和插件为 GO 语言开发，如果要进行插件开发，需要对 GO 有一丢丢了解即可。

1. 创建插件项目

首先使用 go mod 初始化一个 go 项目，然后

```
mkdir sqle-pg-plugin
cd sqle-pg-plugin
touch main.go
go mod init sqle-pg-plugin # 初始化 go mod
export GOPROXY=goproxy.cn,goproxy.io,direct # 设置 GoProxy，解决 SQLE 库下载问题，通过 IDEA
开发的可以在 IDEA 软件上设置；
go get github.com/actiontech/sqle@v1.2204.0 # 此版本为该文章编辑时的最新版本。
```

2. 编写最小化插件代码

在项目 main.go 文件内编写如下代码，即可最快的添加一个 Postgres 数据库审核插件，此时插件没有审核规则。

```
package main

import (
    adaptor "github.com/actiontech/sqle/sqle/pkg/driver"
)
```

```
func main() {
plugin := adaptor.NewAdaptor(&adaptor.PostgresDialector{})
plugin.Serve()
}
```

使用`go build`编译后得到二进制文件 `sqle-pg-plugin`，按前言中的插件的使用参考文档，我们部署到 SQLE 服务里。可以正常添加数据源，如下图所示：

添加数据源 返回

* 数据源名称:

数据源描述:

* 数据库类型: PostgreSQL

* 数据库地址:

* 数据库端口:

* 连接用户:

* 密码:

测试数据库连通性 数据库连通性测试成功

运维时间: 添加

可访问的角色:

审核规则模板: 请选择审核规则模板

应用的工作流: 请选择应用的工作流

重置 提交

此时正常进行 SQL 审核上线工单创建并上线，如下图所示：

app1 处理中 关闭工单 返回

工单名称: app1 创建人: admin 创建时间: 2022-05-11 17:16:35

工单描述: --

审核结果 审核通过率100% 审核结果评分100 下载审核报告 下载SQL语句 是否去重

审核状态: 请选择要搜索的 审核状态 执行状态: 请选择要搜索的 执行状态 审核结果: 请选择要搜索的 审核结果 重置 搜索

序号	审核状态	审核结果	执行语句	执行状态	执行结果	回滚语句 (仅提示, 不支持执行回滚)	说明
1	审核完毕		create database db1	准备执行			

< 1 > 10 条/页

工单进度

● 操作时间:2022-05-11 17:16:35

操作人:admin

admin创建了当前工单

○ 操作时间:--

操作人:--

审核通过 驳回

○ 操作时间:--

操作人:--

等待上一步执行

3. 给插件添加一条规则

在刚刚代码的基础上，我们在 main 函数内添加如下代码来新增一条规则“禁止使用 SELECT *”，完整代码如下所示。

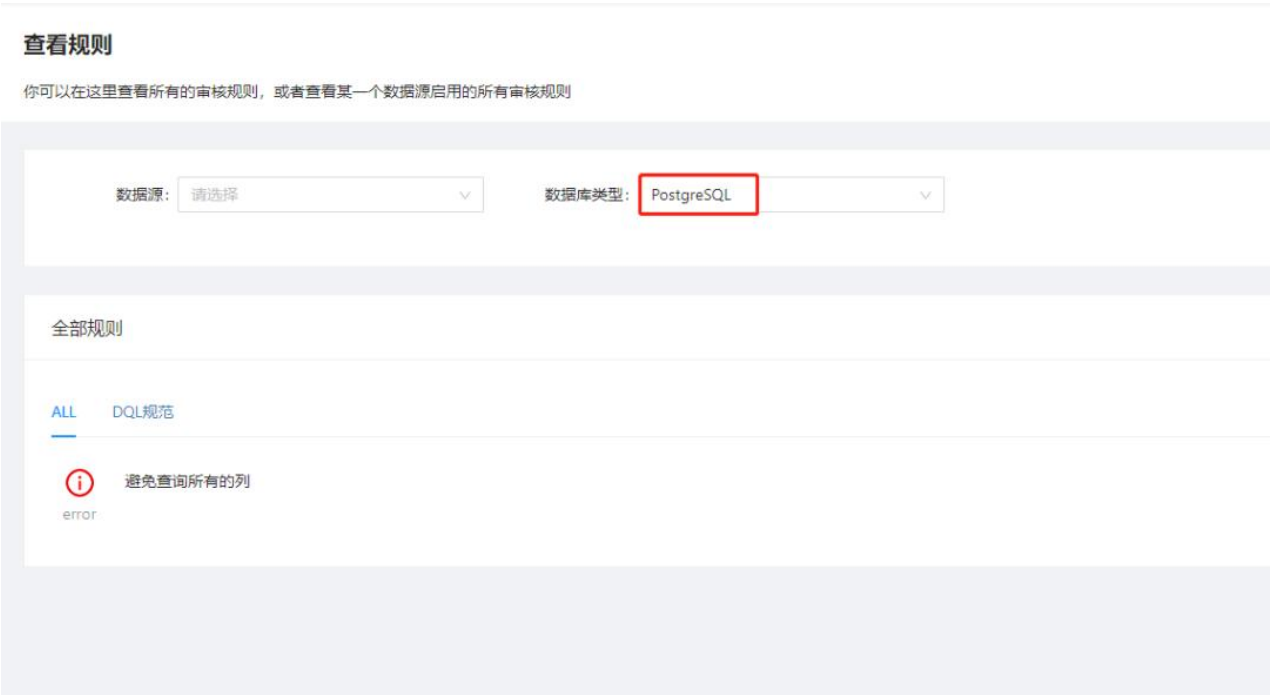
```
package main

import (
    "context"
    "strings"

    "github.com/actiontech/sqlc/sqlc/driver"
    adaptor "github.com/actiontech/sqlc/sqlc/pkg/driver"
)

func main() {
    plugin := adaptor.NewAdaptor(&adaptor.PostgresDialector{})
    rule1 := &driver.Rule{
        Name: "pg_rule_1", // 规则 ID, 该值会与插件类型一起作为这条规则在 SQLC 的唯一标识
        Desc: "禁止使用 SELECT *", // 规则描述
        Category: "DQL 规范", // 规则分类, 用于分组, 相同类型的规则会在 SQLC 的页面上展示在一起
        Level: driver.RuleLevelError, // 规则等级, 表示该规则的严重程度
    }
    //
    rule1Handler := func(ctx context.Context, rule *driver.Rule, sql string) (string, error) {
        {
            if strings.Contains(sql, "select *") {
                return rule.Desc, nil
            }
            return "", nil
        }
        plugin.AddRule(rule1, rule1Handler)
        plugin.Serve()
    }
```

我们按之前的方式编译插件二进制文件，并部署到 SQLC server 内，可以看到新增了一条规则，如下图所示：



此时我们提交一个工单验证一下，可以看到触发了我们刚添加的规则



4. 给插件添加一条可配置的复杂规则

基于上面的代码，我们再添加一条规则“创建的表字段过多”，具备如下特性：

- 上面添加的规则基于字符串匹配进行的，准确性不高，无法匹配到不同的书写格式，比如大小写，换行等。因此我们在这里会基于 SQL 解析器开发一条规则，测试使用的解析库 https://github.com/pganalyze/pg_query_go；
- 为了增加规则的适用性，我们准备给规则加一个动态配置给用户提供可选项。

代码如下：

```

package main

import (
    "context"
    "fmt"
    "strings"

    "github.com/actiontech/sqlc/sqlc/driver"
    adaptor "github.com/actiontech/sqlc/sqlc/pkg/driver"
    "github.com/actiontech/sqlc/sqlc/pkg/params"
    parser "github.com/pganalyze/pg_query_go/v2"
)

func main() {
    plugin := adaptor.NewAdaptor(&adaptor.PostgresDialector{})

    rule1 := &driver.Rule{
        Name: "pg_rule_1", // 规则 ID, 该值会与插件类型一起作为这条规则在 SQLC 的唯一标识
        Desc: "避免查询所有的列", // 规则描述
        Category: "DQL 规范", // 规则分类, 用于分组, 相同类型的规则会在 SQLC 的页面上展示在一起
        Level: driver.RuleLevelError, // 规则等级, 表示该规则的严重程度
    }
    rule1Handler := func(ctx context.Context, rule *driver.Rule, sql string) (string, error) {
        if strings.Contains(sql, "select *") {
            return rule.Desc, nil
        }
        return "", nil
    }

    // 定义第二条规则
    rule2 := &driver.Rule{
        Name: "pg_rule_2",
        Desc: "表字段不建议过多",
        Level: driver.RuleLevelWarn,
        Category: "DDL 规范",
        Params: []*params.Param{ // 自定义参数列表
            &params.Param{
                Key: "max_column_count", // 自定义参数的 ID
                Value: "50", // 自定义参数的默认值
                Desc: "最大字段个数", // 自定义参数在页面上的描述
                Type: params.ParamTypeInt, // 自定义参数的值类型
            },
        },
    },

```

```

}

// 这时处理函数的参数是 interface{} 类型，需要将其断言成 AST 语法树。
rule2Handler := func(ctx context.Context, rule *driver.Rule, ast interface{}) (string,
error) {
node, ok := ast.(*parser.RawStmt)
if !ok {
return "", nil
}
switch stmt := node.GetStmt().GetNode().(type) {
case *parser.Node_CreateStmt:
columnCounter := 0
for _, elt := range stmt.CreateStmt.TableElts {
switch elt.GetNode().(type) {
case *parser.Node_ColumnDef:
columnCounter++
}
}
}
// 读取 SQLE 传递过来的该参数配置的值
count := rule.Params.GetParam("max_column_count").Int()
if count > 0 && columnCounter > count {
return fmt.Sprintf("表字段不建议超过%d个，目前有%d个", count, columnCounter), nil
}
}
return "", nil
}

plugin.AddRule(rule1, rule1Handler)
plugin.AddRuleWithSQLParser(rule2, rule2Handler)

// 需要将 SQL 解析的方法注册到插件中。
plugin.Serve(adaptor.WithSQLParser(func(sql string) (ast interface{}, err error) {
// parser.Parse 使用 PostgreSQL 的解析器，将 sql 解析成 AST 语法树。
result, err := parser.Parse(sql)
if err != nil {
return nil, fmt.Errorf("parse sql error")
}
if len(result.Stmts) != 1 {
return nil, fmt.Errorf("unexpected statement count: %d", len(result.Stmts))
}
// 将 SQL 的语法树返回。
return result.Stmts[0], nil
})))

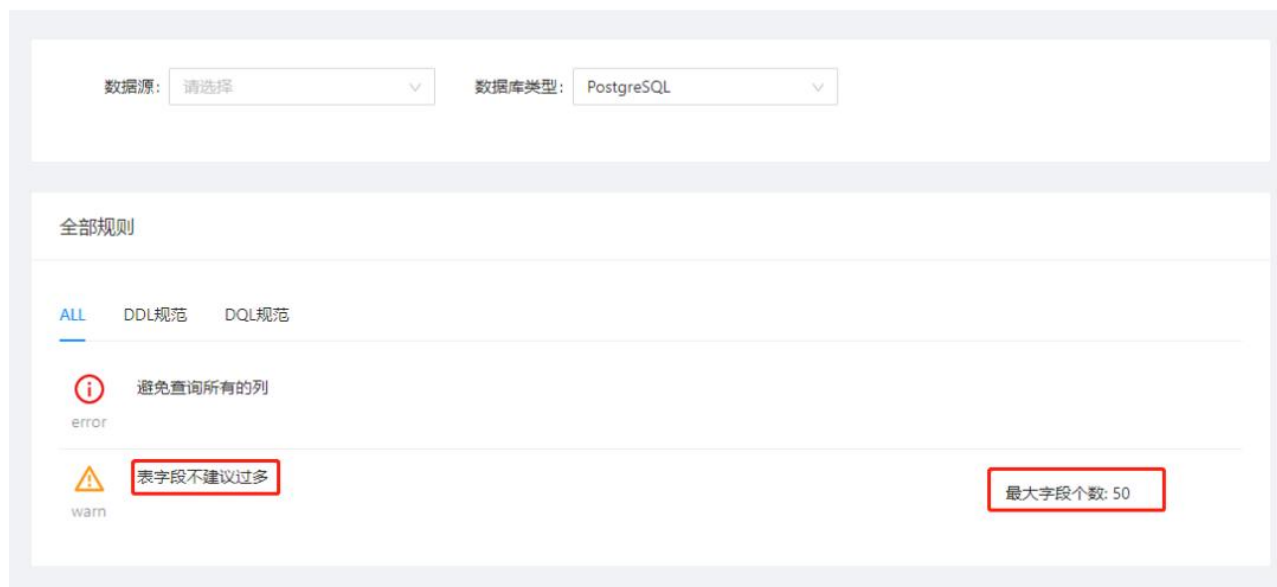
```

```
plugin.Serve()  
}
```

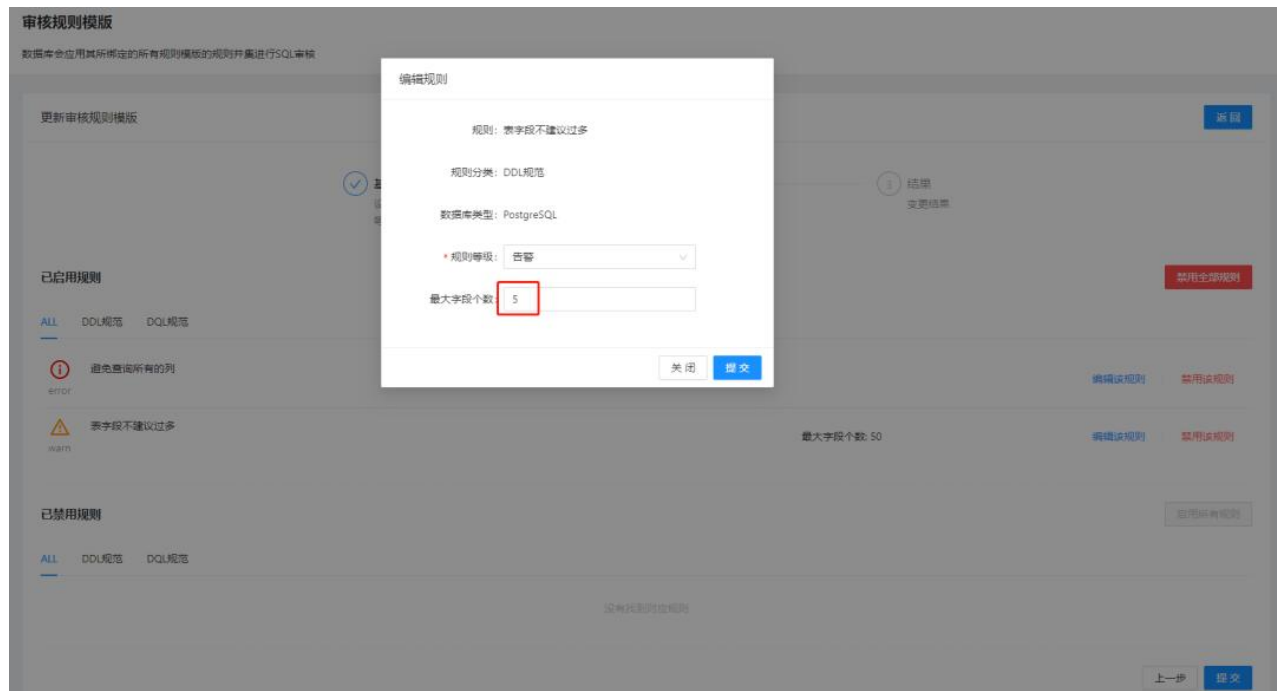
打开 SQLE 规则界面，可以看到该规则已经添加到 SQLE 了，如图：

查看规则

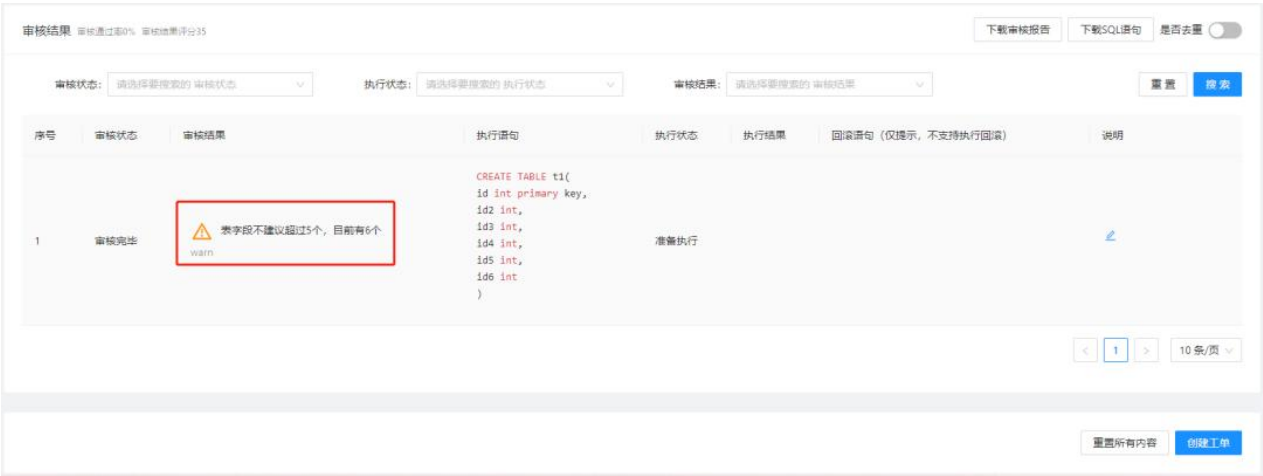
你可以在这里查看所有的审核规则，或者查看某一个数据源启用的所有审核规则



我们将规则模板内该规则的值调小点然后进行测试一下：



首先我们提交一条超过 5 个字段的建表语句，此时 SQLE 会触发该规则并给出预期的提示信息，如下图所示：



然后我们提交一条不超过 5 个字段的建表语句，此时 SQLE 不会触发该规则，如下图所示：



总结

通过上面的演示，大概介绍了 SQLE 数据库审核插件的简单开发测试过程。大家可根据类似步骤开发出一套符合自己公司需求的规则集，结合 SQLE 平台来满足日常使用。我们也提供了一些常见数据库的审核插件，大家也可以在此基础上进行开发，参考文档：https://actiontech.github.io/sqlc-docs-cn/3.modules/3.7_auditplugin/overview.html

4.7 如何利用 OpenAPI 实现自己的扫描任务

作者：蔡传旺

一、SQLE 介绍

SQLE 是一款由爱可生开源社区发起，面向数据库开发与管理人员，实现了 SQL 的「开发」-「测试」-「上线」等全流程覆盖，资源与权限精细化管理，兼顾简洁与高效，易维护、易扩展的的开源项目，旨在为用户提供一套安全可靠、自主可控的 SQL 质量管控方案。

二、功能介绍

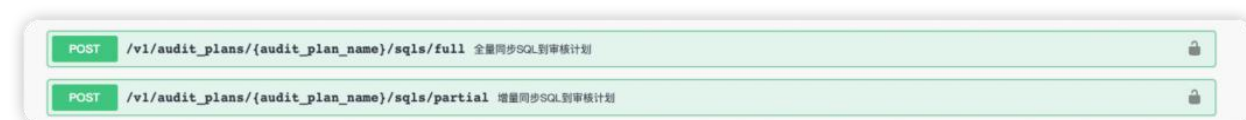
智能扫描任务是我们可以使用一系列的方式调用 sqle 的 openAPI，将数据传输到 sqle 上进行扫描审核的功能，我们可以从数据库中传输建库建表语句审核、可以将慢日志传输到 sqle，由 sqle 解析扫描 sql、也可以将 Mybatis 传输到 sqle 中扫描解析其中的 sql，所以 sqle 的智能扫描任务是非常强大的，那智能扫描能解析那些数据呢？

三、如何使用扫描任务的 OpenAPI

介绍扫描任务的 OpenAPI

智能扫描就是我们可用首先定义一个扫描任务，这个任务是针对某个特定的数据路类型，比如 MySQL，同时我们也会设置任务的执行周期，只要时间一到，sqle 就会自动执行这个扫描任务，刚接触 sqle 的小伙伴可能比较懵，这个扫描任务创建了，但是没有内容，是我使用的方式不对吗？其实不然，sqle 开放了相应的 OpenAPI 接口，我们可以调用 OpenAPI 来将我们要审核的语句发送给 sqle 中相应的任务。那这个 OpenAPI 在哪里？怎么使用这个 OpenAPI 呢？

首先在你按照官方文档的搭建方式 (<https://actiontech.github.io/sqle-docs-cn/2.deploy/overview.html>) 搭建了一个 sqle 后，我们访问 `http://{your address}:10000/swagger/index.html` 就会得到一个 swagger 文档，我们在其中可以找到下图的两个 api，而这两个 api 就是我们可以直接将 sql 语句传给扫描任务的 OpenAPI。



使用方法

1. 比如我在环境中创建了一个自定义类型的扫描任务

任务名称	任务类型	审核周期	数据源名称	审核的数据库	数据库类型	审核规则模板	访问凭证
zidingyi	自定义	0-50/10 ****	local	sqlc	MySQL	default_MySQL	***** 

这样我就得到了这个扫描任务，在任务中我定义了任务类型、审核周期、数据库等，得到了一个访问凭证，这个访问凭证就是我们使用 OpenAPI 访问扫描任务中需要使用到的 token 。

2. 使用类似 postman 的工具，开启一个 post 请求，这里的 zidingyi 是动态参数，你要改成你的扫描任务的名称即可

POST

▼

http://10.186. :10000/v1/audit_plans/zidingyi/sqls/full

请求的 header 是

Content-Type	application/json
Authorization	该扫描任务的访问凭证

请求的 body 是

```
{
  "audit_plan_sql_list": [
    {
      "audit_plan_sql_counter": "1", #匹配到该指纹的语句数量
      "audit_plan_sql_fingerprint": "select * from users where id = ?", #SQL 指纹
      "audit_plan_sql_last_receive_text": "select * from users where id = 1", #最后一次匹配到该指纹的语句
      "audit_plan_sql_last_receive_timestamp": "2022-08-23T19:30:46.00Z", #最后一次匹配到该指纹的时间（RFC3339 格式）
      "audit_plan_sql_schema": "db1" #还不知道
    }
  ]
}
```

3. 经过发送后就可以在该扫描任务的详情页面看到你的 sql

SQL统计信息 				立即审核
SQL指纹	最后一次匹配到该指纹的语句	匹配到该指纹的语句数量	最后一次匹配到该指纹的时间	
<code>select * from users where id = ?</code>	<code>select * from users where id = 1</code>	1	2022-08-23T19:30:46.00Z	

在这个任务到达审核时间，就会自动审核该任务并反馈审核结果

审核记录 	
	生成时间2022-08-23 20:10:00 审核结果评分 100 审核通过率100%
	生成时间2022-08-23 20:00:00 审核结果评分 100 审核通过率100%
	生成时间2022-08-23 19:50:00 审核结果评分 100 审核通过率100%
	生成时间2022-08-23 19:44:00 审核结果评分 100 审核通过率100%

4. 这两个接口分为全量同步和增量同步，全量同步会覆盖之前的记录，而增量同步会在之前的记录下新增
5. 有这个 api 我们就可以将智能扫描任务和我们的日常开发运维连接起来，实现审核 sql 自动化，比如可以 sqle 结合 jenkins 加一个构建流程，在 Jenkins 任务触发时会进行 sql 审核，这里有分享文章

四、利用 OpenAPI 实现自己的扫描任务工具

前面我介绍了这个 OpenAPI 的高度的自定义性，下面我就给大家展示一下我写的一个小工具。就算有了 sqle，但是我还想实现高度的自动化（其实就是比较懒，不想做重复工作），我希望有一个工具直接一键就可以将我的 sql 全部上传到扫描任务里，然后进行审核。于是我写了小工具，可以将文件或者文件夹里的 sql 扫描出来并上传到指定的扫描任务中

自定义配置文件

```
host: your IP and port
path: your sql file path eg: ./test.sql
audit-name: your audit name eg:zidingyi
type: full #full or partial 代表全量同步或者增量同步
isIndependent: true #是否使用独立 sql 语句传入（也就是一个 sql 语句调用一次接口传入）
token: your token
```

自定义配置文件支持命令和参数和配置文件绑定，命令行参数优先级高，也就是优先采用命令行的参数，没有命令行参数则使用配置文件的参数

读取 sql

这一步将分成四种方式读取 sql 文件，分别取是否读取整个文件夹和是否使用独立 sql 语句传入的笛卡

尔积，共 4 个函数

```
package internal

import ...

func GetOneSqlOneFile(path string) error {...}

func GetOneSqlOneDir() error {...}

func GetAllSqlOneFile(path string) error {...}

func GetAllSqlAllDir() error {...}
```

发送请求到 api

根据不用的参数将解析的 sql 生成 OpenAPI 的 body 参数，然后可以发送给 sqlc 服务器，并获取返回参数。

```
client := http.Client{}
req, err := http.NewRequest("POST", fmt.Sprintf("http://%s/v1/audit_plans/%s/sqls/%s", cmd.Host,
cmd.AuditName, cmd.Type), body)
if err != nil {
    fmt.Println(err.Error())
}
req.Header.Set("Content-Type", "application/json")
req.Header.Set("Authorization", cmd.Token)

res, err := client.Do(req)
if err != nil {
    fmt.Println(err.Error())
}
defer res.Body.Close()
b, _ := ioutil.ReadAll(res.Body)
var r = modal.Result{}
_ = json.Unmarshal(b, &r)
fmt.Println(r)
if r.Code != 0 || res.StatusCode != 200 {
    panic(r.Message)
}
}
```

这里只是简单的处理一下错误，还可以加入发生错误自动发送邮件，或者对接第三方办公软件的 api 发送到指定的群等功能。综上，一个简单的小工具就完成了，这样我就可以一键解析 sql 并且将解析的 sql 传送到 sqlc 的扫描任务中了，同时可以根据自己的实际情况增加不同的功能。由此可见，sqlc 的扫描任务的自定义程度是非常高的。这个小工具也是放到 github 上了，喜欢的可以看一看

(<https://github.com/caichuanwang/sqls>)

功能演示

- 将需要上传的 sql 准备成一个 sql 文件，如果需要读取同一文件夹下多个 sql 文件，只需要将配置文件路径设置成文件夹路径即可

```
select * from audit_plans where id = 6;
```

```
select * from execute_sql_detail where id = 5;
```

- 配置自定义配置文件，这个根据自己的实际情况配置，如果没有配置，也可以使用命令行参数配置，命令行参数权重大于配置文件
- 初次运行对项目进行编译后执行

```
$ go run main.go
this need use with sqle,this can parse sqle file or dir and upload to sqle
{0 ok}
```

后期运行只需要直接执行二进制文件并使用命令行参数修改部分参数即可

- 再次查看扫描任务的详情即可

SQL统计信息 🔄				立即审核
SQL指纹	最后一次匹配到该指纹的语句	匹配到该指纹的语句数量	最后一次匹配到该指纹的时间	
SELECT * FROM `AUDIT_PLANS` WHERE `id`=?	select * from audit_plans where id = 6;	0	2022-10-17T14:18:03.48Z	
SELECT * FROM `EXECUTE_SQL_DETAIL` WHERE `id`=?	select * from execute_sql_detail where id = 5;	0	2022-10-17T14:18:03.48Z	
				< 1 > 10 条/页

其他业务场景类型

为了适应不同的业务场景，同时使 sqle 更加强大，扫描任务支持多种类型的 sql 的采集任务，包括有 TopSQL 慢日志 Mybatis 扫描 等场景的处理，这里在我们的文档有介绍

(https://actiontech.github.io/sqle-docs-cn/3.modules/3.6_auditplan/introduction.html)

五、总结

sqle 的智能扫描任务的覆盖面很广，而且可以高度自定义，在日常的开发运维中可以自动帮助我们解析 sql，及时发现错误的 sql。大家可以下载安装 SQLE 并结合自己公司的需求试一下智能扫描，SQLE 的安装参考主页：<https://github.com/actiontech/sqle>；

更多使用场景可以参考：https://actiontech.github.io/sqle-docs-cn/3.modules/3.6_auditplan/introduction.html。

4.8 DBLE 监控告警组件

作者：卢永旺

背景

公司内部使用了大量的 DBLE 实例统一代理众多 mysql 实例，为了增强业务稳定性第一时间可以了解到相关告警。保证各个业务自身的状态,支持不同 dbGroup 实例分发到不同的告警机器人。

实现原理

参考文档：https://actiontech.github.io/dble-docs-cn/1.config_file/1.11_customized_alert.html 通过 DBLE 的回调事件,实现通知下发

使用方式

一、下载并安装

Github 地址：<https://github.com/LuYongwang/dble-alram-webhook/releases>

下载 jar 包,放到 lib 目录下，该 jar 包无任何三方依赖，所有依赖项均为 DBLE 已有依赖不会影响 DBLE 稳定性

二、配置

1、配置文件配置

```
vim $DBLE_HOME/config/dble_alert.properties
```

2、配置文件详解与案例

企业微信

```
# 固定写法
```

```

alert=io.github.luyongwang.dble.WebHookAlarmAlert
# 告警中的节点名称
component_id=DBLE-10.0.142.11
# 企业微信告警（必须）
web_hook.type=WORK_WECHAT
# 企业微信 全局默认机器人 ID（必须）
web_hook.robot_id=xxxx-xxxx-xxxx-xxxx-xxxxxx
# db 负责人以及告警配置 dbGroup1 为 db.xml 中的 dbgroup 节点 name principal 是负责人告警中会@ ,
多个隔开
web_hook.db_config.dbGroup1.principal=xxxxxx,xxxxxx
# robot_id 在一些场景下 想把告警单独分发到某个机器人 可以设置 不设置默认为 全局默认机器人 ID
web_hook.db_config.dbGroup1.robot_id=xxxx-yyyy-zzzz-xxxx-yyyy

```

钉钉配置

```

# 固定写法
alert=io.github.luyongwang.dble.WebHookAlarmAlert
# 告警中的节点名称
component_id=DBLE-10.0.142.11
# 钉钉告警
web_hook.type=DING_TALK
# 钉钉机器人 ID
web_hook.robot_id=xxxxxxxxxxxx
# 这里输入手机号
web_hook.db_config.dbGroup1.principal=150xxxxxxxx,132xxxxxxxx
# robot_id 在一些场景下 想把告警单独分发到某个机器人 可以设置 不设置默认为 全局默认机器人 ID
web_hook.db_config.dbGroup1.robot_id=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

```

飞书配置

```

# 入口
alert=io.github.luyongwang.dble.WebHookAlarmAlert
# 告警中的节点名称
component_id=DBLE-10.0.142.11
# WEBHOOK 配置
web_hook.type=FEI_SHU
# 飞书全局机器人 ID
web_hook.robot_id=xxxx-xxxx-xxxx-xxxx-xxxx
# db 负责人以及告警配置 dbGroup1 为 db.xml 中的 dbgroup 节点 name principal 是负责人告警中会@ ,
多个隔开 这里输入 userid
web_hook.db_config.dbGroup1.principal=xxxxxx,xxxxxx
# robot_id 在一些场景下 想把告警单独分发到某个机器人 可以设置 不设置默认为 全局默认机器人 ID
web_hook.db_config.dbGroup1.robot_id=xxxx-xxxx-xxxx-xxxx-xxxx

```

自定义 WebHook 配置

```

# 入口

```

```

alert=io.github.luyongwang.dble.WebHookAlarmAlert
# 告警中的节点名称
component_id=DBLE-10.0.142.11
# WEBHOOK 配置
web_hook.type=URL
web_hook.hook_url=http://xxxxxxx/api/v1/robot/msg/send
web_hook.hook_params=robot_id=xxxx-xxxx-xxxx-xxxxxx
web_hook.db_config.dbGroup1.principal=150xxxxxx,132xxxxxx
web_hook.db_config.dbGroup1.hook_params=robot_id=xxx-xxx-xxxx-xxxx-xxxx

```

自定义 webHook 请求示例 webHook 接口自行实现

```

curl '{hook_url}?{hook_params}' \
  -H 'Content-Type: application/json' \
  -d '
  {
    "msgtype": "markdown",
    "markdown": {
      "content": "<font color='warning'>DBLE 告警消息</font>:\n >告警代
码:DBLE_HEARTBEAT_FAIL\n >告警解释:心跳后端节点失败\n >告警级别:WARN\n >告警节点:xxx\n >告
警时间:2020-10-15 21:48:11\n >告警详情:|dbInstance:dbGroup1-instanceM1|"
    },
    # 手机号方便@等
    "mobile_list": ["150xxxxxx","132xxxxxx"]
  }'

```

效果图



4.9 log4j2 漏洞修复方案

作者：鲍凤其

db1e 运行依赖许多组件的 jar 包，当遇到某个组件有漏洞时，需要紧急修复。

Apache Log4j2

安全漏洞说明：<https://nosec.org/home/detail/4917.html>

修复方案

- 方案 1 可实施，截止至北京时间 2021 年 12 月 14 日 11 时，log4j 官方已经发布 2.16.0 版本，相关 release notes: <https://github.com/apache/logging-log4j2/blob/rel/2.16.0/RELEASE-NOTES.md>
- 下面提供的方案 2, 3 和 4 方式效果一致，可任取一种，推荐使用第 2 种，全版本 db1e 都适用，影响面确定
- 方案 2 和 3 是临时绕过方案，，不排除有其他问题

1. 升级 log4j2 组件

db1e 版本：2.19.07.x - 3.21.10.x 版本，2.19.07.x 之前的版本需要自行尝试替换方案，官方不再提供支持

影响：需要重启 db1e

步骤：

1.1 停止 db1e

1.2 将 db1e 服务器上 log4j 的 jar 包进行备份并 mv 至 /tmp/ 目录下

/path/to/db1e/lib 下有四个 jar 包分别是：（操作前需要确认一下）

- log4j-1.2-api-2.13.1.jar
- log4j-api-2.13.1.jar
- log4j-core-2.13.1.jar
- log4j-slf4j-impl-2.13.1.jar

执行下面的操作：

```
mv log4j-1.2-api-2.13.1.jar log4j-1.2-api-2.13.1.jar.bak
mv log4j-1.2-api-2.13.1.jar.bak /tmp/
```

1.3 将 log4j 2.16.0 版本的相关 jar 包，上传到该路径下/path/to/db1e/lib，并变更权限

参考链接：<https://repo1.maven.org/maven2/org/apache/logging/log4j/log4j-core/2.16.0/>，其他 jar 在此网站上查找

1.4 重复 1.2，1.3 步骤升级其余三个 jar 包 1.5 启动 db1e

2 添加配置

db1e 版本：理论上全版本 db1e 适配

影响：需要重启 db1e

步骤：

在 db1e 配置文件 /path/to/db1e/conf 下添加配置文件 log4j2.component.properties

修改文件权限：

添加如下配置：

```
log4j2.formatMsgNoLookups=True
```

验证方式：

开发环境验证该变量重启后被加载，不重启情况下，不会被加载。

3. 修改 jvm 参数

db1e 版本：适用于 db1e 版本 < 3.20.07.0

3.20.07.0 及之后的 db1e 版本由于对 JVM 参数进行了限制，因此不支持此种方式，会在近期修复。

影响：需要重启 db1e

步骤：

在 db1e 配置文件 /path/to/db1e/conf/wrapper.cof 中添加如下配置，并重启 db1e 。

配置：

原环境中是否存在 wrapper.java.additional 的参数，下面配置中的 14 在原环境中按需替换

```
wrapper.java.additional.14=-Dlog4j2.formatMsgNoLookups=true
```

执行以下命令判断是否使用该参数启动：

```
ps aux|grep db1e|grep 'formatMsgNoLookups'
```

4. 设置系统环境变量 FORMAT_MESSAGES_PATTERN_DISABLE_LOOKUPS 设置为 true

不推荐

4.10 中间件是如何处理 Prepare Statement 和游标的

作者：董诚怡

PS 语句（预编译语句）

以下用 缩写 PS 代替 Prepare Statement

PS 的优点

- 防止 SQL 注入漏洞
- 标准的 PS 可以 一次编译、多次运行，省去了每次都要解析优化的过程。（被称为预编译，有利于多次执行的结构相同参数不同的语句）
- 更灵活的调用

PS 的缺点

- 普通的查询（即时 SQL ） 需要一次网络开销，但是 PS 需要至少两次网络开销。如果游标的话，次数将更多，和数据量成正比。
- 需要空间开销用于缓存当前的 SQL 预编译后的结构，相对即时 SQL，这块空间不能及时释放。
- 对 in 语句的支持不太好

分类

- server-side PS: 通过 client 发送 PS 协议报文给 server，由 server 完成拼装参数、优化、执行。
- client-side PS: 由 client 来实现 PS 接口，prepare 阶段完成拼装参数，拼装完后，一次性发送即时 SQL 给 server，由 server 来完成优化、执行。这本质上是一个伪预编译，上述的 “省去了每次都要解析优化的过程” 这个优点无法实现。

使用 dble 侧 PS 必要条件

DBLE 端

- 无

客户端

- 如果是 JDBC 需开启 useServerPrepStmts，此时才会使用 server-side prepare，否则属于 client-side prepare。

验证是否开启了

```
PreparedStatement preparedStatement = con.prepareStatement("select t1.id from
no_sharding_t1 t1 where t1.id=?");
//可用于验证是否使用了 dble 侧 prepare
assert preparedStatement instanceof ServerPreparedStatement;
```

协议

- COM_STMT_PREPARE

发送：SQL

作用：创建一个 statement，完成预编译，执行优化等准备工作，等待下一步执行。

响应: statement 的 id 以及 column 的数量、argument 数量和类型

- COM_STMT_EXECUTE

发送: statement 的 id , 每个占位符绑定的值, 以及是否想要开启游标

作用: 执行 SQL

响应: OK 响应或者结果集

- COM_STMT_FETCH

发送: statement 的 id , 期望获取的行数

作用: 分批次获取执行后数据

响应: 指定行数的结果集

注意: 该协议可选, 也可以在 execute 环节上返回结果集, dble 一般仅在开启游标时使用该报文

- COM_STMT_SEND_LONG_DATA

作用: 发送占位符绑定的值, 通常只用于发送 BLOB 数据

- COM_STMT_RESET

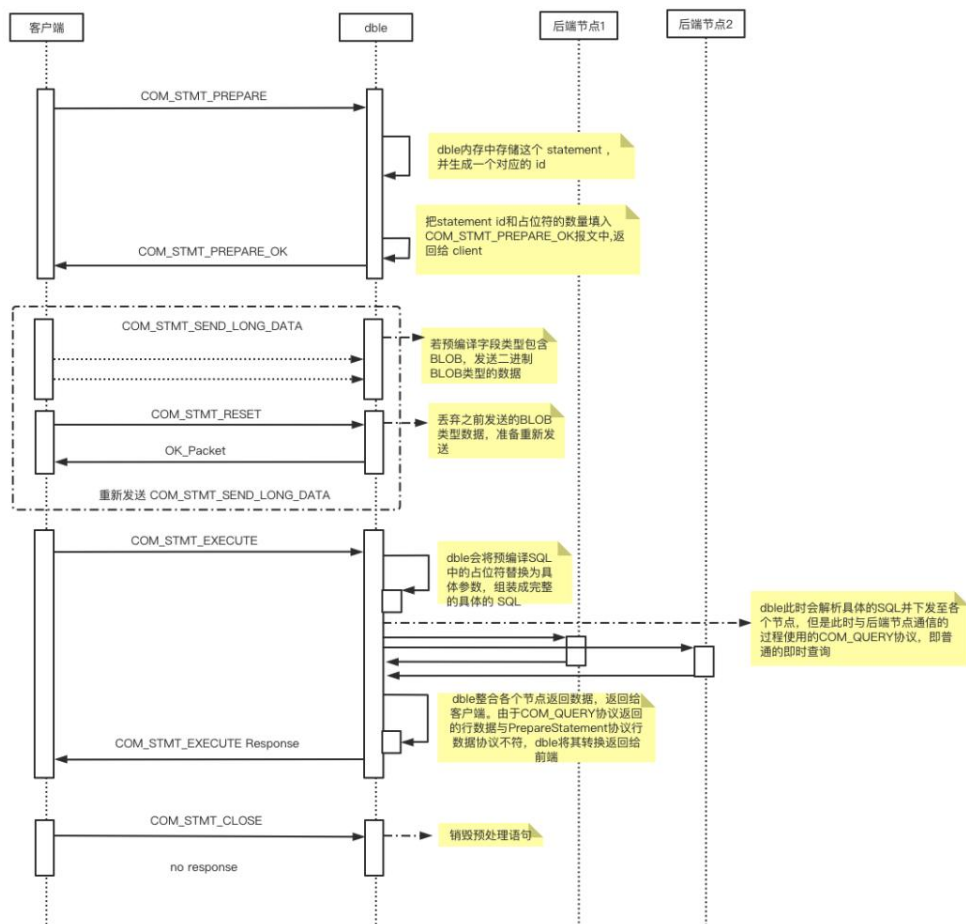
作用: 重置 COM_STMT_SEND_LONG_DATA 设置的值 (通常不用)

- COM_STMT_CLOSE

发送: statement 的 id

作用: 关闭之前的 Prepared Statement , 回收所有资源响应: 无

流程图:



可以看到 client <-> dble 通讯使用了 server-side prepare, dble <-> MySQL 通讯使用了 client-side prepare, 也就是说后端通讯和普通的即时查询无异, 只是需要做一些协议上的包的转换。

原理

- prepare 阶段将语句暂存, 不进行预编译
- execute 阶段拼装参数和语句, 将其下发, 获得结果后转换为 PS 协议并返回

游标

游标的优点

- 对于客户端较友好, 不会因为大查询而 OOM

游标的缺点

- 较慢
- 资源不及时释放

分类

- server-side cursor: server 把结果集暂存起来, 维护一个游标, client 根据需要读取指定的行数
- client-side cursor: client 从 TCP 层面控制报文的读取, 对报文进行流量控制, 当报文太多时暂停读取。(不推荐, 因为 server 需要等待所有数据发送给 client 后, 才能释放资源。)
- 另一种 client-side cursor: client 把所有结果集读取到本地缓存, client 每次从缓存读取指定行数(不推荐, 本质上是个伪 cursor, 只实现了 cursor API。并且在数据量较大时很容易撑爆 client 的内存)

后两种是否支持取决于 client 端的 driver, dble 支持的是第一种 server-side cursor。

游标开启必要条件

DBLE 端

注: 读写分离场景由于不支持 COM_STMT_FETCH 报文, 所以不支持游标。以下描述仅针对分库分表。

- 如果版本<3.21.02, 则不支持。
- 如果版本=3.21.02, 无需设置
- 如果版本>3.21.02, 需在 bootstrap.cnf 开启-DenableCursor=false

客户端

1. 使用支持游标的 driver (MySQL 官方的 JDBC driver 就支持)
2. 如果是 JDBC 需开启 useServerPrepStmts 和 useCursorFetch 选项
3. 执行 prepareStatement 后设置 fetchSize, 必须大于 0.
4. 执行 execute

此时是开启游标的, 如果对结果集 resultSet 进行遍历, 会按 fetchSize 的大小一次次地从 dble 取回数据。

验证是否开启了游标

客户端执行第 4 步后, 调用私有方法 useServerFetch 可验证。

```
final ResultSet resultSet = preparedStatement.executeQuery();
```

//可用于验证是否使用了 server-side 游标

Method method =

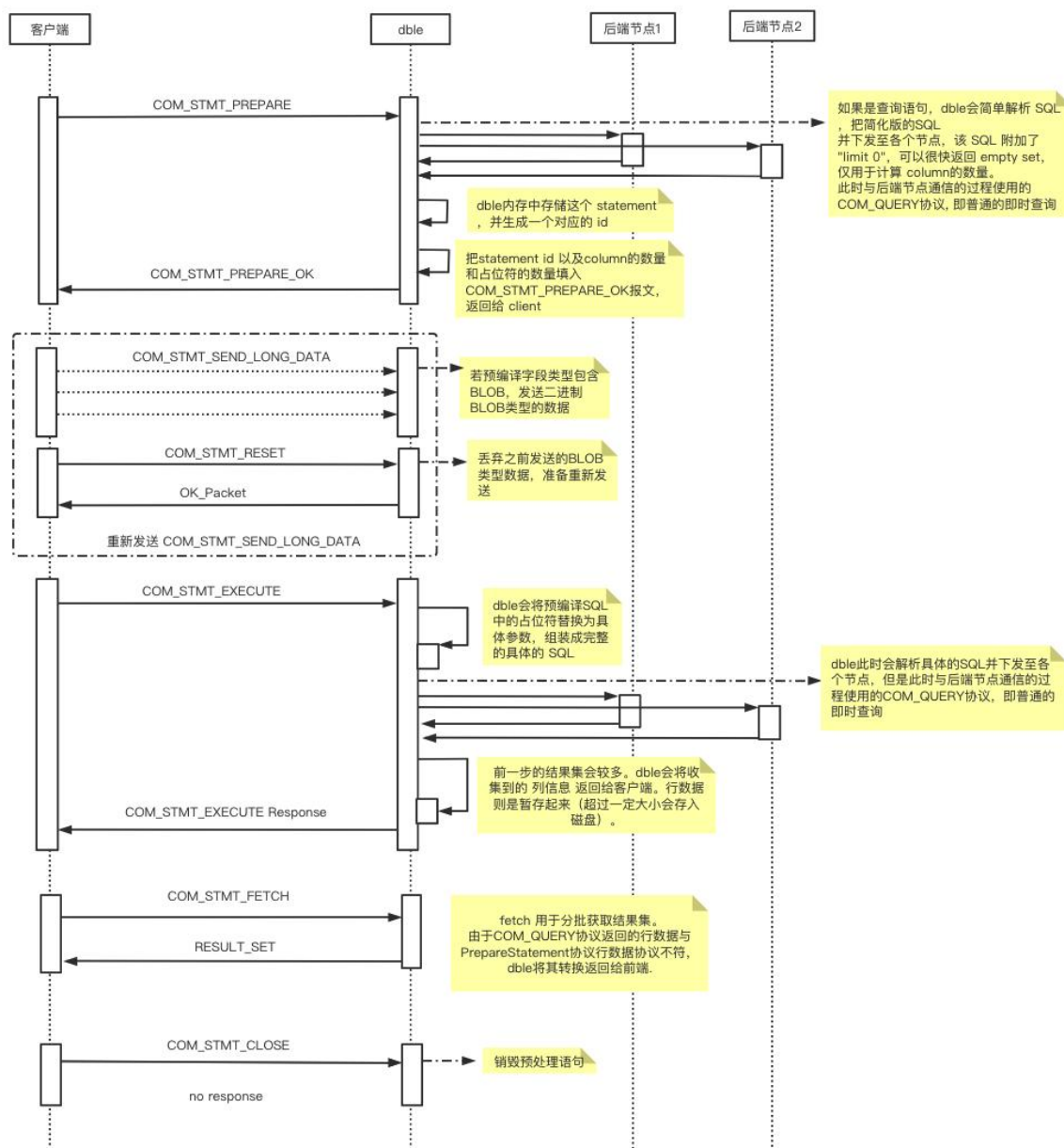
```
com.mysql.cj.jdbc.StatementImpl.class.getDeclaredMethod("useServerFetch");
```

```
method.setAccessible(true);
```

```
Boolean useServerFetch = (Boolean) method.invoke(preparedStatement);
```

```
assert useServerFetch==true;
```

流程图:



原理:

1. prepare 阶段下发特殊语句。用于计算 SQL 中的列数，这是 client 所需的开启游标的必要条件。
2. execute 阶段把结果集存储到临时文件
3. fetch 阶段把结果集分批次取出来

相关参数

maxHeapTableSize

heapTableBufferChunkSize

见文档 https://actiontech.github.io/dble-docs-cn/1.config_file/1.02_bootstrap.cnf.html

4.11 动态调整 DBLE 内线程池的数目

作者：郭奥门

背景

在实际生产环境，项目上线初期流量比较小，等后面项目流量涨上来，db1e 内原有的线程配置可能支撑不了上游的压力，此时可能会遇到一系列性能问题，这时就需要调大 processors、backendProcessors 等线程池参数，并根据预期指标及实际线程使用情况多次调整至最优。

在之前，修改完配置中的线程数之后需要重启才能让配置生效，但这种方式不是很灵活，甚至可能会影响上游的使用。db1e 也是考虑到这一点，在 3.21.06.* 版本提供了不重启调整线程池数目的方式

命令

```
update db1e_information.db1e_thread_pool set core_pool_size = 2 where name = 'BusinessExecutor';
```

注意：

- 支持动态调整的线程池为：businessExecutor 、writeToBackendExecutor 、processors(nio 场景下：bootstrap.cnf 中的 usingAIO 值为 0)、backendProcessors (nio 场景下)、backendBusinessExecutor 、complexQueryExecutor
- 不支持动态调整 AIO 场景下的 processors 、backendProcessors 对应的线程池
- 由于 JDK 原生线程池（ThreadPoolExecutor）扩缩容机制问题，新建的线程和即将被回收的线程需要一定的时机才会被处理，所以设置 core_pool_size 并不会立即通过 db1e_thread_pool 表查到，但此时使用该线程池不受影响
- 尽管线程池数目可以允许不停机的方式调整，但为了防止出现未知问题，建议不要在流量大的时候调整

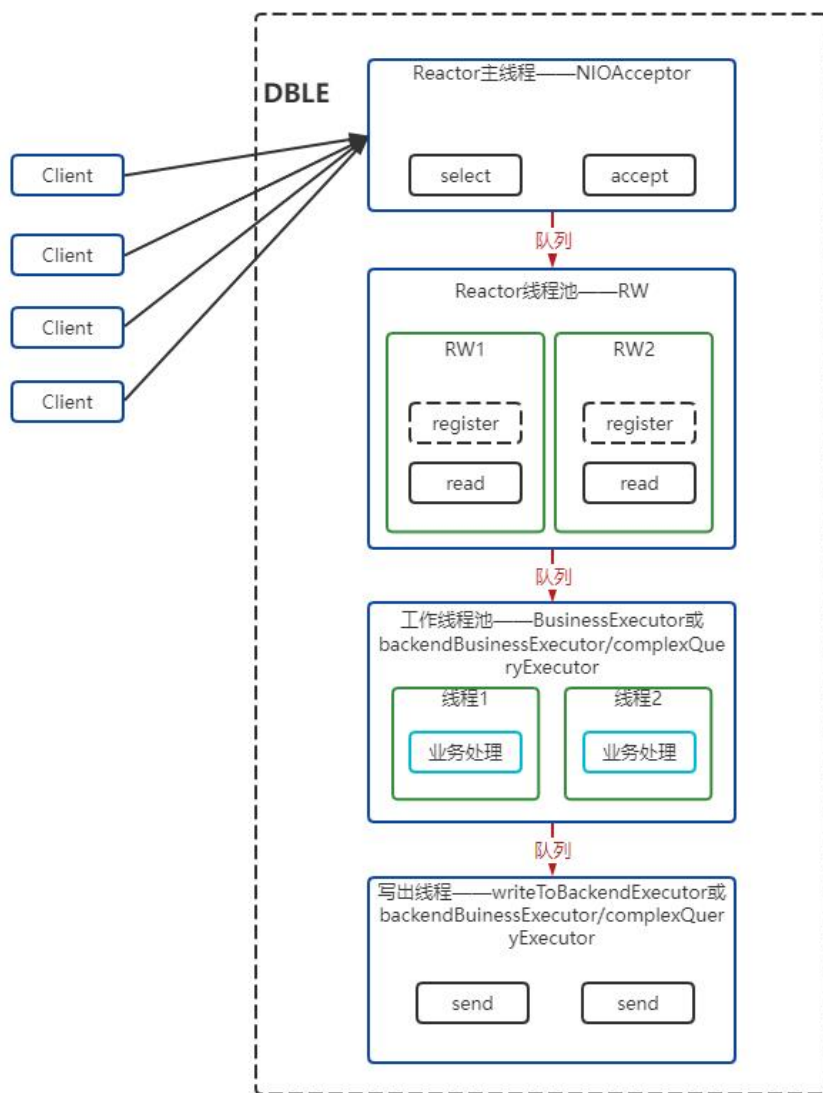
原理解读

dble 内线程的使用方式

当前 dble 内使用线程池主要包含两种方式：一是 JDK 内置线程池，另外是外置队列 + JDK 内置线程池，下面我们简单讲一下两种方式的原理

外置队列+线程池

DBLE 为了高并发、响应快等特点，在网络 IO 层面采用了经典的主从 Reactor 多线程模型，这里只说明 Reactor 模型如何在 DBLE 中落地，不清楚模型原理的同学可以参阅：彻底搞懂 Reactor 模型和 Proactor 模型（文末有链接）



DBLE 目前网络模型如上图所示：

1、Reactor 主线程——NIOAcceptor 通过 `select`、`accept` 事件读取并初步处理 client 连接，并通过 `frontRegisterQueue` 队列传递给 Reactor 子线程

2、Reactor 子线程——RW 从外置队列中（非线程池内部队列，为了区分称此时队列为外置队列）取到连接并注册到当前子线程中，后续通过 `read` 方法读取数据包并通过 `frontHandlerQueue` 队列或本地队列传递

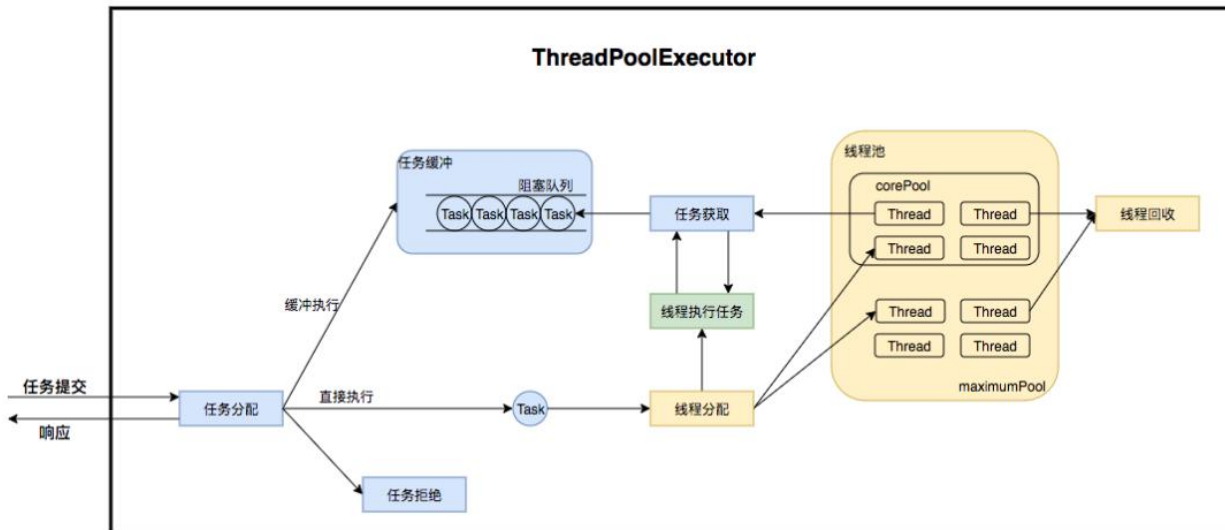
给工作线程

3、工作线程池内的子线程从外置队列中接收到任务，经过后续的一系列分析处理后，将结果经过 writeQueue 队列传递给 writeToBackendExecutor 线程，继而发送给前端 client，或经过本地队列传递给 backendBusinessExecutor/complexQueryExecutor 线程直接将结果返回给前端

从 DBLE 网络模型可以看出其内部使用队列+线程池的方式来分发处理任务，除此之外在业务处理的过程中也是大量使用了线程池来处理一些耗时的任务

JDK 内置线程池

db1e 内线程池是借助了 java 的 ThreadPoolExecutor 类，其运行流程如下：



线程池在内部实际上构建了一个生产者消费者模型，将线程和任务两者解耦，并不直接关联，从而良好的缓冲任务，复用线程。线程池的运行主要分成两部分：任务管理、线程管理。任务管理部分充当生产者的角色，当任务提交后，线程池会判断该任务后续的流转：

- （1）直接申请线程执行该任务；
- （2）缓冲到队列中等待线程执行；
- （3）拒绝该任务。线程管理部分是消费者，它们被统一维护在线程池内，根据任务请求进行线程的分配，当线程执行完任务后则会继续获取新的任务去执行，最终当线程获取不到任务的时候，线程就会被回收。

结合 db1e 中目前的结构，其内部主要线程使用的方式为：

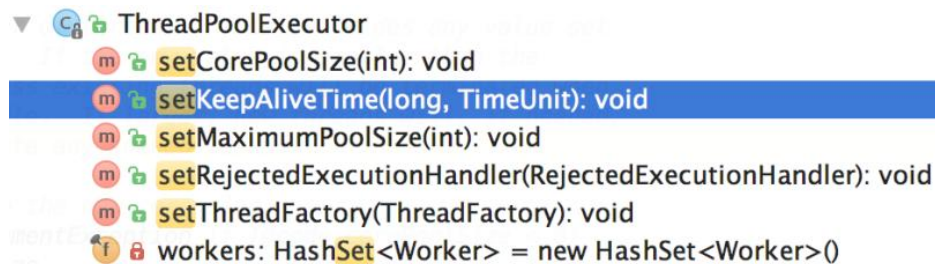
- businessExecutor、writeToBackendExecutor 相关线程通过外置队列+线程池实现调度，在 db1e 启动时初始化线程池，线程内部通过轮询方式获取任务（启动时创建的线程称之为常驻线程）
- processors、backendProcessors 相关线程在 nio 情况下通过外置队列+线程池实现调度；在 aio 情况下通过 AsynchronousChannelGroup 机制（内置线程池）实现线程管理
- backendBusinessExecutor 相关线程在性能模式下（usePerformanceMode=1），通过外置队列+线程池实现调度；反之在后续任务到达时由线程池直接调度
- complexQueryExecutor 相关线程由线程池直接调度（不是在启动时创建的线程称之为非常驻线程，随任务朝生暮死）

具体实现

为了动态的调整线程池的数目，保证扩缩容之后任务都能正常被处理，需要针对以上两种方式作单独处理，具体实现方式如下：

线程池

JDK 原生线程池 `ThreadPoolExecutor` 提供了如下几个 `public` 的 `setter` 方法，如下图所示：



JDK 允许线程池使用方通过 `ThreadPoolExecutor` 的实例来动态设置线程池的核心策略，以 `setCorePoolSize` 为方法例，在运行期线程池使用方调用此方法设置 `corePoolSize` 之后，线程池会直接覆盖原来的 `corePoolSize` 值，并且基于当前值和原始值的比较结果采取不同的处理策略。对于当前值小于当前工作线程数的情况，说明有多余的 `worker` 线程，此时会向当前 `idle` 的 `worker` 线程发起中断请求以实现回收，多余空闲的 `worker` 在任务执行完成后也会被回收（有延迟）；对于当前值大于原始值且当前队列中有待执行任务，则线程池会创建新的 `worker` 线程来执行队列任务，如果当前线程内部队列没有待执行任务，则会在下次任务需要执行时新建 `work` 线程（有延迟）。

外置队列+线程池

线程池可以借助 JDK 提供的 `set` 方法动态设置池的大小，当前场景下扩容时额外需要为新建的线程绑定外置队列，保证后续的任务能通过外置队列被新建的线程接收并处理，那么在代码中新建线程时需要添加外置队列的引用

缩容时，`ThreadPoolExecutor` 线程池通过以上方式对空闲的线程进行回收，针对非常驻线程就只需要设置大小即可；针对正在运行的线程不会主动进行回收，解铃还须系铃人，此时就需要我们手动关闭，先通过 `interrupt` 方法把线程标记为 `interrupted` 状态，同时在线程内部根据状态值判断是否需要退出轮询，后续再借助线程池内部的缩容策略进行回收线程

通过以上两种方式可以实现动态的修改线程池数目，但是为了让 `processors`、`backendProcessors` 相关 IO 线程能在扩缩容时平稳过渡，需要额外的做一些必要的“善后”工作

“善后”

`processors`、`backendProcessors` 所对应的线程为 DBLE 的 IO 线程，负责对注册到当前线程的连接请求的接收和后端结果的接收，那么扩容时就需要保证新建的 IO 线程能处理后续的 IO 请求，相应的缩容时需要对回收的线程所绑定的连接能够转移到其他 IO 线程，保证连接后续请求能被正常处理

在 DBLE 中对于善后工作所做的处理就是先取消当前线程中的连接，再将这些连接重新注册到新的 IO 线程中，此时删除和重新注册的选取策略为：删除时优先选择线程中绑定连接数最小的，重新注册时优先选择线程中连接数最大的，并根据删除的线程往下选择

总结

db1e 在 3.21.06.* 版本及之后提供了可以不重启来修改线程参数的命令，由于 db1e 内不只是简单的使用 JDK 内置的线程池，那么动态修改的命令也不只是使用 JDK 内置的方法去实现，同时也为了兼容外置队列、IO 连接而做了额外的工作。

虽然我们在并发测试中动态调整线程池数目并未发现异常情况，但是仍旧建议在并发量小的时候进行调整，不仅为了线程间切换平稳过渡，也是为了减少线程调整时资源的使用。实际在使用该命令的时候遇到一些问题，可以反馈在 [actiontech/db1e: A High Scalability Middle-ware for MySQL Sharding \(github.com\)](https://github.com/actiontech/db1e)，帮助我们改善

参考

彻底搞懂 Reactor 模型和 Proactor 模型：<https://cloud.tencent.com/developer/article/1488120>

Java 线程池实现原理及其在美团业务中的实践：<https://tech.meituan.com/2020/04/02/java-pooling-practice-in-meituan.html>

4.12 BenchmarkSQL 压测 db1e 性能调优指南

作者：蔡玮

背景介绍

BenchmarkSQL 是一个支持众多关系型数据库的基准测试工具，通过使用 BenchmarkSQL 对数据库进行 TPC-C 标准测试，即模拟多种事务处理：新订单、支付操作、订单状态查询、发货、库存状态查询等，从而获得最终的压测值。相较于 Sysbench 的单一，它更能贴切的模拟出真实的应用场景，因此越来越多的客户在对数据库进行压测时，更多的选择使用 BenchmarkSQL。

作为一款数据库中间件，db1e 同样也是可以使用 BenchmarkSQL 进行压测的，但是压测过程中，往往如何调优，从而测得产品得最大压测值则是更值得关注的，本文将从可观测角度，绕开内部逻辑，介绍我们内部在使用 BenchmarkSQL 压测 db1e 时的一些调优经验总结以及观测手法，以供大家参考。这种调优手段主要集中在配置以及 db1e 内部线程数上。

配置建议

本文主要针对的是调优借鉴，在这里就不再过分的介绍 BenchmarkSQL 安装以及 db1e 安装等过程，也将不再就数据准备以及压测步骤展开进行叙述。

但是有必要就一些必要配置信息做一些必要以及建议值的简要说明。

后端连接数

需要确保压测时，dble 用户连接数 > 压测并发数 dble 中存储节点最大连接数 = 对应后端 MySQL 的最大连接数 >> (并发数 / 存储接节点数量)

日志级别

建议将 dble 的日志级别设置为 WARN，设置的对应配置文件为 dble/conf/log4j2.xml

隔离级别

dble 和后端数据库的隔离级别设置为 RC 隔离级别，如果是 RR 隔离级别会在压测过程中出现锁等待超时的报错，原因可参考历史公众号文章：MySQL 中间件性能测试

关闭 dble 可能会影响性能的参数开关

建议关闭 dble 中一些可能会影响性能的参数开关，其他的开关如果非必要均可以关闭，如：
useSqlStat=0, enableSlowLog=0

dble 的配置

至于 dble 的配置信息，需要额外关注的是，在压测时需要将 config 以及 item 表设置为全局表，其他表全部设置为分片表，同时为了确保后端数据库压力的均匀分配，应当确保分片表的数据均匀分布在后端数据库上，这里建议采用求模算法，以便于达到目的

MySQL 的配置

至于 MySQL 的性能参数配置信息，需要根据机器配置自行调优，在这里不再赘述

观测手段

系统资源观测工具

在使用压测工具进行压测时，选择一款简单好用的系统观测工具有时候也是很重要的，以便于我们观测瓶颈所在并进行一些可尝试的调优。这里推荐使用 dstat 工具，其可以实时地看到所有系统资源，例如，CPU 使用情况、磁盘 IO 以及 网络 IO 情况。

dstat 更可以将观测到的结果输入到 csv 文件，以便于后续绘图等分析。

以下为 dstat 的实时输出结果，详情可参考 <https://linux.cn/article-3215-1.html>

```
--total-cpu-usage--  -dsk/total-  -net/total-
usr sys idl wai stl  _read  _writ  _recv  _send
45  14  41   0   0   0      0      0    72M    73M
46  14  40   0   0   0      0      0    69M    70M
43  14  42   0   0   0      0    44k    70M    71M
47  14  40   0   0   0      0      0    73M    74M
46  14  40   0   0   0      0      0    73M    74M
44  13  43   0   0   0      0      0    72M    72M
45  14  41   0   0   0      0      0    70M    71M
45  14  40   0   0   0      0    36k    72M    73M
46  14  40   0   0   0      0      0   103k   103k
44  13  43   0   0   0      0    36k    72M    72M
42  14  44   0   0   0      0      0    70M    71M
45  14  41   0   0   0      0      0    72M    72M
45  14  41   0   0   0      0      0    71M    72M
```

dble 侧观测手段

这里提供两种 dble 侧可以使用的观测手法：

开启查询耗时统计需要在 `dble bootstrap.cnf` 中开启 `useCostTimeStat=1`，并重启 `dble` 生效，以下为观测样图

```
profiling:
```

Block	Invocations	SelfTime.Total	SelfTime.Avg	SelfTime.Min	SelfTime.Max	WallTime.Total	WallTime.Avg	WallTime.Min	WallTime.Max
request->1.startProcess	9073	-638142734	-70334	-1051058	493260	202952071	22368	10565	493260
request->2.endParse	9073	234134936	25805	13206	523393	437087007	48174	23771	1016653
request->3.endRoute	9073	404389553	44570	20123	121474	841476560	92745	43894	1075553
request->4.resFromBack	9073	592398	65	-649019	1602901	4805691043	529669	261612	1602901
request->5.startExecuteBackend	9073	-56808823	-6261	-1749483	2020297	5047581273	556329	350530	2020297
request->6.response	9073	59150286	6519	3045	366620	5107315454	562913	353575	2386917

DBLE 将查询分为了六个阶段：

- 1) 开始梳理
- 2) 完成解析
- 3) 完成路由分配
- 4) 从数据库回收结果
- 5) 后置处理
- 6) 反馈处理

我们只需要重点关注 `WallTime.Avg`，其指代了每个阶段的平均耗时，这样我们可知道压力到底在中间件的哪一个阶段变慢。然后根据不同阶段的耗时异常，去调整 `dble` 内部的线程数或者排查后端 MySQL。详情可参考：https://actiontech.github.io/dble-docs-cn/2.Function/2.18_performance_observation.html

● 查看线程使用率

`dble` 内部提供了查看各类线程使用率的命令：`show @@thread_used`，借助此命令可以在压测过程中查看各类线程的使用情况，然后进行相应的线程数量调整，以下为命令展示样图。

```
mysql> show @@thread_used;
```

THREAD_NAME	LAST_QUARTER_MIN	LAST_MINUTE	LAST_FIVE_MINUTE
\$ _NIO_REACTOR_BACKEND-0-RW	0%	0%	0%
\$ _NIO_REACTOR_BACKEND-1-RW	0%	0%	0%
\$ _NIO_REACTOR_FRONT-0-RW	0%	0%	0%
\$ _NIO_REACTOR_FRONT-1-RW	0%	0%	0%
BusinessExecutor0	0%	0%	0%
BusinessExecutor1	0%	0%	0%
backendBusinessExecutor0	0%	0%	0%
backendBusinessExecutor1	0%	0%	0%
writeToBackendExecutor0	0%	0%	0%
writeToBackendExecutor1	0%	0%	0%

10 rows in set (0.01 sec)

压测经验

在我们刚开始去使用 `BenchmarkSQL` 去压测 `dble` 时，同样也并不是很清楚各类线程数的大小对于性能的正负影响，都是在一步步的摸索中，才慢慢得到了初步的结论，在此给大家介绍一下相关的情况。

关于 dstat

由于 `dstat` 主要用于资源占用观测，所以在测试的过程中，作为辅助性工具使用一直打开，用于观察资源使用情况。并未作为调整的依据。

关于查询耗时统计

我们首先使用了 `dble` 内部的耗时统计功能【`useCostTimeStat=1`】，去测试不同阶段的耗时情况，然后根据 `SQL` 执行不同阶段的耗时情况调整线程数。但是测试时发现，调整了线程数并不能有效解决观测到的耗时异常情况，说明这些耗时异常情况并不适用于通过线程数上的调整去解决问题，这也是通过 `BenchmarkSQL` 压测 `dble` 时所发现的一问题之一，对此我们也有记录，并会持续的跟进调查。

关于 show 命令

由于依据查询耗时统计去调整 dble 的线程数量，并未起到良好的效果。所以，在压测过程中，管理端命令 `show @@thread_used` 观测线程使用情况，就成了我们调整线程数量的主要依据。按照其他压测工具压测 dble 的经验，在压测之初我们也想着确保各类线程的使用率在 80% 以下，确保线程不那么繁忙，可是测试中发现，在调整各类线程数将各类线程使用率降至 80% 之后，有时得到了压测 `tpmC` 值反而不升反降了，猜想，难道是线程数量过多，频繁的线程切换影响性能，那反之又该如何？所以下一步我们的方向，则是将 dble 内部的线程按照类型分为三类：

- 前后端 IO 线程：processors、backendProcessors
- 前后端业务处理线程：processorExecutor、backendProcessorExecutor
- 其他线程：complexExecutor、writeToBackendExecutor

参照 `show @@thread_used` 查询出来的线程使用率，按照不同类型的线程进行数量上的上下调整。

在调整的过程中我们发现，不同类型线程数量的上下调整，对于压测结果，效果是不一样的。经过多轮测试得到以下结论：

- 前后端 IO 线程 及 complexExecutor、writeToBackendExecutor 的数量对于压测 `tpmC` 的值没有明显影响
- 对于 `tpmC` 值影响最大的为前后端业务处理线程数量，且前后端业务处理线程使用率与 `tpmC` 的值正向相关，即前后端业务处理线程的线程使用率越高，`tpmC` 的值越高
- dble 所在机器的网络 IO 与 `tpmC` 的值正向相关，即网络 IO 越高，`tpmC` 的值越高【通过观察 `dstat` 的结果而得】
- 前后端业务处理线程数量调整，对于 dble 所在机器的网络 IO 与前后端业务处理线程使用率影响一致，即前后端业务处理线程数量下调，网络 IO 上升、线程使用率上升，且 `tpmC` 值越高

总结

通过以上的调整手段，在配置不变的情况下，基于 dble 3.20.10.0 版本，我们内部在压测时得到压测结果，dble 【后端挂有 4 个 MySQL 节点，1000 数仓、1000 线程并发】：MySQL 【250 数仓，250 线程并发】 = 2.8 : 1 左右。当然，在理想情况下，两者的 `tpmC` 的值应该可以达到 4: 1 左右，可以看到，应该还是有可以提升的空间。除去必要的网络损耗，在其他 dble 内部可以优化的地方，我们的团队仍在努力尝试进行其他的优化，大家可以期待一下后续的版本表现。

至此，建议大家在使用 BenchmarkSQL 压测 dble，使用可观测手段进行调优时，目前可主要针对前后端业务处理线程的数量进行调整（调低），调整的依据即为：

- 理应保持前后端业务处理线程的使用率维持在一个比较高的水平，建议在 95% 左右
- 通过 `dstat` 命令观测到 dble 所在机器的网络 IO 数值越高越好

此外，前后端 IO 线程数量和其他类型的线程数量可依据机器配置按照 dble 官方推荐值进行适配。另外需要注意的是，如果是多次进行压测，建议每次压测时都要将历史数据删除并重新进行数据准备，以免首次压测后造成数据上的污染，导致后续压测结果与首次压测结果偏差较大的问题。

4.13 数据库连接如何正确处理 TCP 连接三次握手失败

作者：鲍凤其

背景

在稳定性环境中，当 dble 初始化后端连接池后，后端连接池会出现连接计数器（totalConnections）和实际连接（allConnections）数量不符合的情况，理论情况下两个变量会保持最终一致性。

后续通过查阅网上相关文档，找到了相关文档：<https://mp.weixin.qq.com/s/FEybf-jRL8gMUYPBDH3aQg>，详细分析过程可参考此文章。

简单来说，在 dble 初始化后端连接池的过程中，瞬时创建的连接数量可能过大，导致部分 TCP 连接握手时触发了 TCP 的 syn_cookie 机制并且第三次 TCP 握手的 ACK 报文丢失了，从而导致了上述的情况。

后续，在稳定性环境中将 TCP 的 syn_cookie 关闭之后暂时解决了此种情况。

但假设正常 TCP 三次握手出现如下三种异常情况：

- TCP 第一次握手包 SYN 丢包了
- TCP 第二次握手包 SYN、ACK 丢包了
- TCP 第三次握手包 ACK 包丢了

客户端和服务端是如何处理的，如果重传，重传多少次？每次间隔时长多少？

实验环境

在一台服务器上启动 MySQL 服务，端口是 3306，IP 地址：10.186.60.69

在一台服务器上使用 MySQL client 连接 MySQL 服务，IP 地址：10.186.60.60

第一种场景

TCP 第一次握手包 SYN 报文丢包了，会发生什么？

在 MySQL 服务器上执行，通过 iptables 阻断客户端的发送过来的所有 TCP 报文：

```
$ iptables -i eth0 -A INPUT -p tcp --dport 3306 -j DROP
```

在 MySQL 服务器上开始抓包：

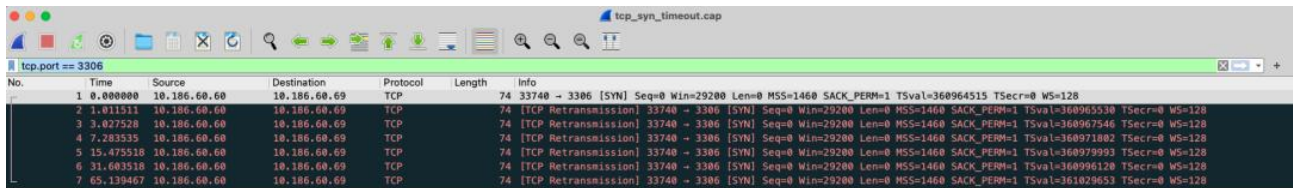
```
$ tcpdump -i eth0 tcp and port 3306 -w tcp_syn_timeout.cap
```

通过 MySQL client 连接 MySQL 服务端，过了一分多钟后，客户端返回报错，此时停止抓包，如下图：

```
$ mysql -h10.186.60.69 -uroot -p123456 -P3306
mysql: [Warning] Using a password on the command line interface can be insecure.
```

```
ERROR 2003 (HY000): Can't connect to MySQL server on '10.186.60.69' (110)
```

wireshark 分析抓包文件：



The image shows a Wireshark packet capture window titled 'tcp_syn_timeout.cap'. The filter is 'tcp.port == 3306'. The packet list shows seven packets. Packets 1, 3, 5, and 7 are SYN packets from 10.186.60.60 to 10.186.60.69. Packets 2, 4, 6, and 7 are retransmissions of the SYN packet. The packet details for packet 7 show: '74 33740 -> 3386 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=360964515 TSecr=0 WS=128'.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.186.60.60	10.186.60.69	TCP	74	33740 -> 3386 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=360964515 TSecr=0 WS=128
2	1.011511	10.186.60.60	10.186.60.69	TCP	74	[TCP Retransmission] 33740 -> 3386 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=360965530 TSecr=0 WS=128
3	3.027528	10.186.60.60	10.186.60.69	TCP	74	[TCP Retransmission] 33740 -> 3386 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=360967546 TSecr=0 WS=128
4	7.283535	10.186.60.60	10.186.60.69	TCP	74	[TCP Retransmission] 33740 -> 3386 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=360971002 TSecr=0 WS=128
5	15.475518	10.186.60.60	10.186.60.69	TCP	74	[TCP Retransmission] 33740 -> 3386 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=360979993 TSecr=0 WS=128
6	31.603518	10.186.60.60	10.186.60.69	TCP	74	[TCP Retransmission] 33740 -> 3386 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=360996120 TSecr=0 WS=128
7	65.139467	10.186.60.60	10.186.60.69	TCP	74	[TCP Retransmission] 33740 -> 3386 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=361029653 TSecr=0 WS=128

客户端在收不到 TCP SYN 报文的 ACK 报文后，会不断进行重试，示例中会进行六次重试并且每次 RTO 是不同的：

- 第一次是 1 秒后重试
- 第二次是 3 秒后重试，和第一次相差 2s 左右
- 第三次是 7 秒后重试，和第二次相差 4s 左右
- 第四次是 15 秒后重试，和第三次相差 8s 左右
- 第五次是 31 秒后重试，和第四次相差 16s 左右
- 第六次是 65 秒后重试，和第五次相差 32s 左右

每次超时时间 RTO 是指数上涨的。另外，这里的重试次数可以配置，由客户端机器的如下内核参数指定：

```
$ cat /proc/sys/net/ipv4/tcp_syn_retries
6

# 不同的发行版本，参数可能不同
$ uname -a
Linux ubuntu 4.15.0-36-generic #39-Ubuntu SMP Mon Sep 24 16:19:09 UTC 2018 x86_64
x86_64 x86_64 GNU/Linux
```

可以尝试修改此参数看看效果：

```
$ echo 2 > /proc/sys/net/ipv4/tcp_syn_retries
```

因此：TCP 第一次握手包 SYN 报文丢包了，会发生什么？客户端会重传 SYN 报文，直到收到 ACK 或者达到最大次数，每次重试的时间是翻倍上涨的。

第二种场景

TCP 第二次握手的 SYN + ACK 报文丢包了，会发生什么？

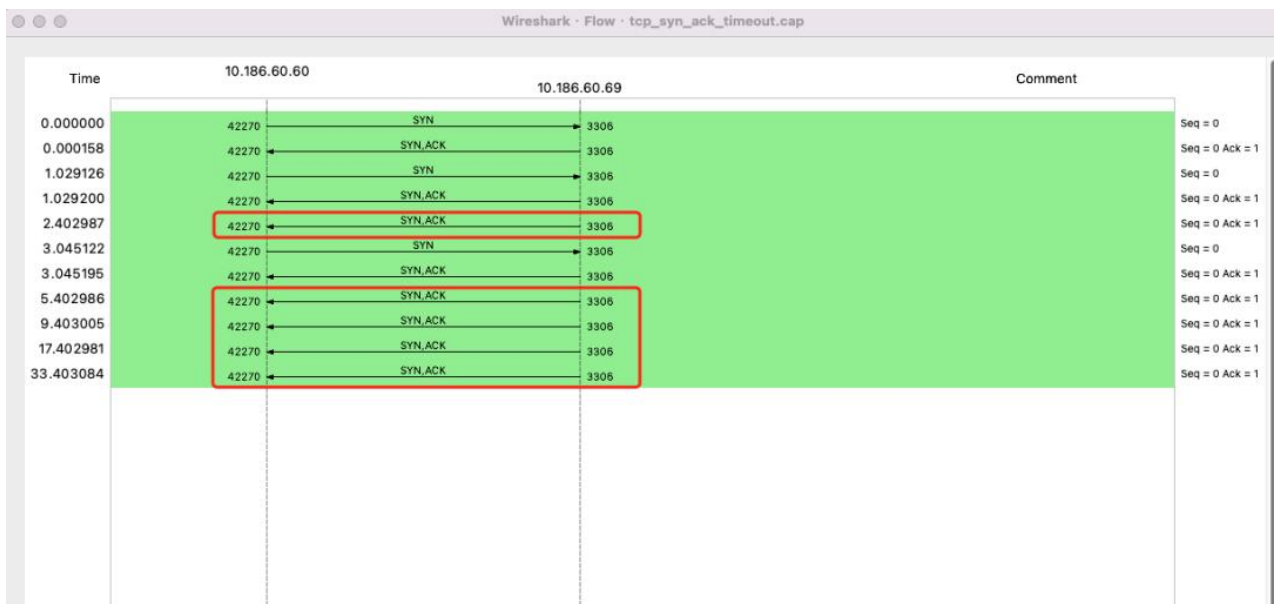
为了模拟 SYN + ACK 的丢包情形，在客户端设置防火墙，将 MySQL 服务端的报文全部拦截：

```
$ iptables -A INPUT -p tcp -s 10.186.60.69 -j DROP
```

在 MySQL 服务器端抓包：

```
$ tcpdump -i eth0 tcp and port 3306 -w tcp_syn_ack_timeout.cap
```

在 wireshark Statistics 下面 flow graph 功能分析 tcp 流：



从上图来看，可以分为两个视角：

客户端视角：客户端发出 SYN 报文之后，由于设置了防火墙，没有收到 SYN + ACK 报文，因此，客户端会不断进行重试，直到收到 SYN + ACK 或者达到最大重试次数

服务器视角：服务器端在收到 SYN 报文之后，发送 SYN + ACK 报文，但是收不到最后一次握手的 ACK 报文，因此服务器端会不断进行重试发送 SYN + ACK 报文。

客户端超时重传的 SYN 包抵达了服务端后，服务端然后回了 SYN、ACK 包，但是 SYN、ACK 包的重传定时器并没有被重置，仍然持续在重传。

从图中可以看出，服务端在收到第三次的 SYN 报文并发出的 SYN + ACK 报文之后，后面重试了四次，将之前重试的一次也算在内。

这个重试次数也由内核参数控制：

```
$ cat /proc/sys/net/ipv4/tcp_synack_retries
5
```

将客户端内核参数 tcp_synack_retries 设置成 1 之后，TCP 交互图：



这样重试次数就更加明显了。

第三种场景

TCP 第三次握手的 ACK 丢包了

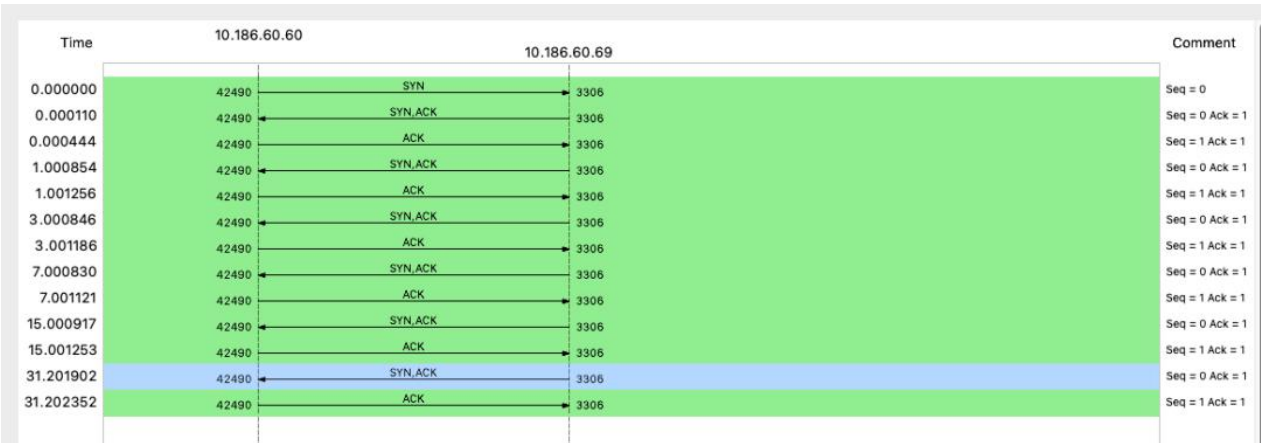
在 MySQL 服务器端设置防火墙，拦截 TCP 第三次握手的 ACK 报文：

```
$ iptables -A INPUT -p tcp --tcp-flag ack ack --dport 3306 -j DROP
```

在 MySQL 服务器端抓包：

```
$ tcpdump -i eth0 tcp and port 3306 -w tcp_3th_ack_timeout.cap
```

分析抓包文件：



从上图来看，可以分为两个视角：

客户端视角：对于客户端来说，其实连接已经建立好了

通过 netstat 命令查看连接的状态：

```
$ netstat -napt|grep 3306
tcp 0 0 10.186.60.60:42490 10.186.60.69:3306 ESTABLISHED 14391/mysql
```

此时连接的状态是 ESTABLISHED 状态。

服务器视角：由于没有收到第三次的 ACK 报文，和第二种场景类似，服务器会一直重新发送 SYN + ACK 报文，直到达到最大次数

在重试期间，服务端连接的状态一直处于 SYN_RECV 状态：

```
$ netstat -napt|grep 3306
tcp 0 0 10.186.60.69:3306 10.186.60.60:42868 SYN_RECV -
```

过了半分钟左右，也就是服务器端达到重试次数之后，服务端刚才处于 SYN_RECV 的状态的 TCP 连接不见了。

可是此时客户端的连接却依然存在。

客户端的连接之后怎么处理？

此时分场景讨论：

一种场景是，客户端在 TCP 连接建立完成之后，直接发送数据。

另一个场景是，客户端没有任何操作。下面对这两种情况进行讨论。

客户端发送数据

为了模拟这种场景，我们预先通过客户端连接 MySQL 服务器。

连接上之后在 MySQL 服务器端通过防火墙隔离客户端的报文：

```
$ iptables -A INPUT -p tcp -s 10.186.60.60 -j DROP
```

在 MySQL 服务端进行抓包：

```
$ tcpdump -i eth0 tcp and port 3306 -w tcp_data.cap
```

之后通过刚才建立的连接，下发 `use test;` 语句。

下面分析一下抓包文件：



分析：

我们发现客户端一共重传了十一次。

TCP 建立连接后的数据包传输，最大超时重传次数是由 `tcp_retries2` 指定，默认值是 15 次，这里为了便于观测，将数值调整成了 10 次，如下：

```
$ cat /proc/sys/net/ipv4/tcp_retries2
10
```

可是这里的抓包文件中传输了十一次报文，这里参考文章：<https://perthcharles.github.io/2015/09/07/wiki-tcp-retries/>

无任何操作

在 MySQL 的协议中，TCP 建立完成之后，MySQL 服务端会发送握手包，由于 MySQL 服务端连接已经不在，因此不会下发握手包，客户端会一直 hang 住。

```
$ mysql -h10.186.60.69 -uroot -p123456 -P3306
mysql: [Warning] Using a password on the command line interface can be insecure.
```

此时客户端连接的存活由 TCP 的保活机制确保。

keep-alive 机制:

1. 首先, 有个前提: 在特定的时间段内, 连接如果没有任何动作, TCP 保活机制会开始作用。
2. 保活机制会每过一个固定时间发送一个「探测报文」, 如果连续几个探测报文都没有得到响应, 则认为该 TCP 连接已经死亡, 系统内核将错误信息通知给上层应用程序。

在 Linux 内核可以有对应的参数可以设置保活时间、保活探测的次数、保活探测的时间间隔, 以下都为默认值:

```
$ sysctl -a|grep keepalive
net.ipv4.tcp_keepalive_intvl = 75
net.ipv4.tcp_keepalive_probes = 9
net.ipv4.tcp_keepalive_time = 7200
```

也可以通过下面的方式来查看:

```
$ cat /proc/sys/net/ipv4/tcp_keepalive_intvl
$ cat /proc/sys/net/ipv4/tcp_keepalive_probes
$ cat /proc/sys/net/ipv4/tcp_keepalive_time
```

参数解释:

- tcp_keepalive_intvl=75: 表示每次检测间隔 75 秒;
- tcp_keepalive_probes=9: 表示检测 9 次无响应, 认为对方是不可达的, 从而中断本次的连接。
- tcp_keepalive_time=7200: 表示保活时间是 7200 秒(2 小时), 也就 2 小时内如果没有任何连接相关的活动, 则会启动保活机制

我们可以修改参数看下效果:

```
$ echo 10 > /proc/sys/net/ipv4/tcp_keepalive_intvl
$ echo 40 > /proc/sys/net/ipv4/tcp_keepalive_time
$ echo 2 > /proc/sys/net/ipv4/tcp_keepalive_probes
```

通过抓包文件查看:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.0000	10.186.60.69	10.186.60.69	TCP	74	42868 → 3306 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1 TSval=455188792 TSecr=0 WS=128
2	0.0001	10.186.60.69	10.186.60.69	TCP	74	3306 → 42868 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 SACK_PERM=1 TSval=1575808 TSecr=455188792 WS=128
3	0.0003	10.186.60.69	10.186.60.69	TCP	66	42868 → 3306 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=455188792 TSecr=1575808
4	1.2012	10.186.60.69	10.186.60.69	TCP	74	[TCP Retransmission] 3306 → 42868 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 SACK_PERM=1 TSval=1577010 TSecr=455188792 WS=128
5	1.2015	10.186.60.69	10.186.60.69	TCP	66	[TCP Dup ACK 3#1] 42868 → 3306 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=455189994 TSecr=1575808
6	3.2012	10.186.60.69	10.186.60.69	TCP	74	[TCP Retransmission] 3306 → 42868 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 SACK_PERM=1 TSval=1579010 TSecr=455188792 WS=128
7	3.2015	10.186.60.69	10.186.60.69	TCP	66	[TCP Dup ACK 3#2] 42868 → 3306 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=455191994 TSecr=1575808
8	7.2012	10.186.60.69	10.186.60.69	TCP	74	[TCP Retransmission] 3306 → 42868 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 SACK_PERM=1 TSval=1583010 TSecr=455188792 WS=128
9	7.2015	10.186.60.69	10.186.60.69	TCP	66	[TCP Dup ACK 3#3] 42868 → 3306 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=455195993 TSecr=1575808
10	15.201	10.186.60.69	10.186.60.69	TCP	74	[TCP Retransmission] 3306 → 42868 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 SACK_PERM=1 TSval=1591010 TSecr=455188792 WS=128
11	15.201	10.186.60.69	10.186.60.69	TCP	66	[TCP Dup ACK 3#4] 42868 → 3306 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=455203003 TSecr=1575808
12	31.201	10.186.60.69	10.186.60.69	TCP	74	[TCP Retransmission] 3306 → 42868 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 SACK_PERM=1 TSval=1607010 TSecr=455188792 WS=128
13	31.201	10.186.60.69	10.186.60.69	TCP	66	[TCP Dup ACK 3#5] 42868 → 3306 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=455219992 TSecr=1575808
14	41.723	10.186.60.69	10.186.60.69	TCP	66	[TCP Keep-Alive] 42868 → 3306 [ACK] Seq=0 Ack=1 Win=29312 Len=0 TSval=455238513 TSecr=1575808
15	51.963	10.186.60.69	10.186.60.69	TCP	66	[TCP Keep-Alive] 42868 → 3306 [ACK] Seq=0 Ack=1 Win=29312 Len=0 TSval=455240752 TSecr=1575808
16	62.204	10.186.60.69	10.186.60.69	TCP	66	42868 → 3306 [RST, ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=455250992 TSecr=1575808

可以看到在 40s 时候, 开始探活包, 探测了两次, 每次间隔 10s 中, 符合参数修改的定义

修改参数之后, client 过了大约一分钟后, 就报错了:

```
$ mysql -h10.186.60.69 -uroot -p123456 -P3306
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 2013 (HY000): Lost connection to MySQL server at 'reading initial communication packet', system error: 110
```

4.14 如何与 DBLE 进行“秘密通话”

作者：蔡玮

SSL 协议简介

众所周知，如果我们在网络上传输数据时信息都是使用明文的话，会很容易出现数据被监听和窃取的情况，从而造成一定的安全性的问题，这对于一些个人敏感信息乃至公司的数据安全无疑造成了很大的风险。

基于此，势必有一定的需求，对网络上传输的数据进行“包裹化”处理，而 SSL 即在此背景之下应运而生。Netscape 公司于 1996 年提出了安全协议 SSL，其是工作于应用层和传输层之间的一款协议，设计即全面，其涉及的概念众多，不仅仅“包裹化”了数据【数据加密】，更是提供了身份验证和消息完整性验证机制，为网络数据传输安全性建设做出了巨大贡献，从而非常大程度上改善了互联网的安全性问题。对于数据库层面，加密通信同样显得很重要，毕竟任何业务的数据存储最终都要落实到数据库上，其重要性不言而喻。所以对于 MySQL 而言，SSL 已经是一个成熟的功能并广泛应用。

对于协议实现原理以及加密算法不再是本文介绍的重点，在此就不再赘述，可参考历史公众号文章：[MySQL：SSL 连接浅析](#)

SSL 之 DBLE 篇

概述

作为一款数据库中间件产品，在使用 DBLE 时，将 MySQL 挂载到 DBLE 后端后，完全可以脱离 MySQL 而与 DBLE 进行直接建立连接。那么问题来了，如何确保与 DBLE 进行通信时数据的安全性呢？显然，在这方面 DBLE 需要向 MySQL 学习，使用 SSL 武装自己，以确保通信时用户数据的安全性。

在即将要发版的 DBLE 版本中，我们将会支持 SSL 加密连接，需要注意的是目前加密处理是处于 Client — DBLE 通信阶段，DBLE — MySQL 通信阶段暂未涉及。同时在已经发版的 DBLE 3.22.01.1 的小版本中也已率先支持了 SSL，感兴趣的同学可以下载相关的版本进行试用。

使用说明

对于 DBLE 的 SSL 连接配置和 MySQL 有一定的相似性，但是并不尽相同，下面就 DBLE 对于 SSL 加密的使用进行简要的配置使用介绍。

熟悉 SSL 的同学应该知道，使用 SSL 的前提必然是各种证书【涉及各种密钥信息】，DBLE 也并不例外。MySQL 中使用的是自签名证书，自签名证书是由不受信的 CA 机构颁发的数字证书，也就是自己签发的证书。与受信任的 CA 签发的传统数字证书不同，自签名证书是由一些公司或软件开发商创建、颁发和签名的。DBLE 同样采用和 MySQL 一样的方式：使用自签名证书方式制作 SSL 证书。

证书制作

证书制作需要借助 OpenSSL 来进行，如果机器上并未安装可手动进行安装 OpenSSL。

1、制作 CA 自签名证书(包含公钥)和私钥

```
openssl genrsa 2048 > ca-key.pem
openssl req -new -x509 -nodes -days 3600 -key ca-key.pem -out ca.pem
```

2、创建私钥和签发服务端的数字证书

```
openssl req -newkey rsa:2048 -days 3600 -nodes -keyout server-key.pem -out server-req.pem
openssl rsa -in server-key.pem -out server-key.pem
openssl x509 -req -in server-req.pem -days 3600 -CA ca.pem -CAkey ca-key.pem -set_serial 01 -out server-cert.pem
```

3、创建私钥和签发客户端的数字证书（与上相似）

```
openssl req -newkey rsa:2048 -days 3600 -nodes -keyout client-key.pem -out client-req.pem
openssl rsa -in client-key.pem -out client-key.pem
openssl x509 -req -in client-req.pem -days 3600 -CA ca.pem -CAkey ca-key.pem -set_serial 01 -out client-cert.pem
```

4、验证服务端和客户端数字证书是否可信，当输出的结果为 OK，表示通过

```
openssl verify -CAfile ca.pem server-cert.pem client-cert.pem
```

值得一提的是，MySQL 自带一键生成证书的 `mysql_ssl_rsa_setup` 命令中内部也是大致按照以上生成证书的，所以更方便的做法是直接使用 `mysql_ssl_rsa_setup` 生成相应的证书文件【当然用于 DBLE 处也需要再进行证书类型转换，见下文】。

证书类型转换

由于 DBLE 是基于 JAVA 语言进行开发的，OpenSSL 生成的证书格式 `pem`、`crt` 等格式，在 JAVA 语言并不能正确识别，需要额外使用 `keytool` 工具【java 原生自带，安装 java 后不需要再进行安装】转换成 `p12`、`jks` 格式，同时如果使用的客户端是 JDBC 时，相关的 URL 中用到的证书也需要使用格式转换后的证书文件，其他 Driver 则均适用于 `pem` 证书文件。

1、将 `ca.pem` 导入 Java 平台的密钥库中，java 支持密钥库类型有：JKS、JCEKS、PKCS12、PKCS11 和 DKS，这里生成 JKS 扩展名的 `truststore.jks` 密钥库，密码可自定义，此处定义为 123456

```
keytool -import -noprompt -file ca.pem -keystore truststore.jks -storepass 123456
```

2、将 `server-cert.pem` 和 `server-key.pem` 转成 `p12` 类型的密钥库，然后在转成 JKS 类型的密钥库，密码可自定义，此处定义为 123456

```
openssl pkcs12 -export -in server-cert.pem -inkey server-key.pem -out serverkeystore.p12 -passout pass:123456
keytool -importkeystore -srckeystore serverkeystore.p12 -srcstoretype PKCS12 -destkeystore serverkeystore.jks -srcstorepass 123456 -deststorepass 123456
```

3、同样，将客户端用到的证书文件转换为 JKS 类型的密钥库，密码可自定义，此处定义为 123456

```
openssl pkcs12 -export -in client-cert.pem -inkey client-key.pem -out
```

```
clientkeystore.p12 -passout pass:123456
keytool -importkeystore -srckeystore clientkeystore.p12 -srcstoretype PKCS12 -
destkeystore clientkeystore.jks -srcstorepass 123456 -deststorepass 123456
```

至此，我们一共得到了以下密钥文件信息：

证书	说明
ca.pem	自签名 CA 证书；用于验证数字证书的可信度
server-cert.pem、server-key.pem	服务端数字证书和私钥；作为服务端身份，适用于除 java 以外的语言
client-cert.pem、client-key.pem	客户端数字证书和私钥；作为客户端身份，适用于除 java 以外的语言
truststore.jks	包含自签名 CA 证书的 JKS 密钥库；适用于 java 语言
serverkeystore.jks	包含服务端数字证书和私钥的 JKS 密钥库；适用于 java 语言
clientkeystore.jks	包含客户端数字证书和私钥的 JKS 密钥库；适用于 java 语言

服务端 DBLE 配置

在使用 SSL 时，DBLE 作为服务端需要手动进行配置相关的文件信息，并开启相关的功能。和 MySQL 一致，我们提供了一个开关 `supportSSL`，用于标识 SSL 是否启用，默认值为 `false`，如果需要使用 SSL 连接时，首先需要确保此开关处于打开的状态。同时需要配置使用到的一些证书信息，在 `bootstrap.cnf` 中进行如下配置：

```
-DsupportSSL=true
-DserverCertificateKeyStoreUrl=${path}/serverkeystore.jks
-DserverCertificateKeyStorePwd=123456
-DtrustCertificateKeyStoreUrl=${path}/truststore.jks
-DtrustCertificateKeyStorePwd=123456
```

配置完成之后，重启 `dbld` 即可。

为了便于查询 SSL 的一些状态信息，我们在 DBLE 的管理端 `dbld_information` 库中新增了一些用于维护相关的 SSL 的元数据信息，确保配置无误并重启 `dbld` 之后，可在 DBLE 管理端查询到对应的 SSL 配置信息以及状态：

```
mysql> select * from dbld_variables where comment like '%SSL%';
+-----+-----+-----+-----+
| variable_name | variable_value | comment | read_only |
+-----+-----+-----+-----+
| isSupportSSL | true | isSupportSSL in configuration | true |
| isSupportOpenSSL | true | Whether OpenSSL is actually supported | true |
| serverCertificateKeyStoreUrl | /opt/ssl_tmp/serverkeystore.jks | Service certificate required of OpenSSL | true |
| trustCertificateKeyStoreUrl | /opt/ssl_tmp/truststore.jks | Trust certificate required of OpenSSL | true |
+-----+-----+-----+-----+
```

客户端连接配置

在使用 SSL 连接 MySQL 时区分了多种连接模式，此方式同样适用于 DBLE，以下提供两种常见的 Client 加密连接时的客户端配置：

模式	连接参数配置
DISABLE D	MySQL client: mysql -uroot -proot --ssl-mode=DISABLED JDBC: jdbc:mysql://ip:port/schema?useSSL=false
PREFERR ED	MySQL client: mysql -uroot -proot --ssl-mode=PREFERRED JDBC: jdbc:mysql://ip:port/schema?requireSSL=false&useSSL=true&verifyServer Certi
REQUIRE D	MySQL client: mysql -uroot -proot --ssl-mode=REQUIRED JDBC: jdbc:mysql://ip:port/schema?requireSSL=true&useSSL=true&verifyServ
VERIFY_ CA	单向认证（客户端会认证服务端的身份） MySQL client: mysql -uroot -proot --ssl-mode=VERIFY_CA --ssl-ca='{自签名CA证书}' JDBC: jdbc:mysql://ip:port/schema?requireSSL=true&useSSL=true&verifyServerCertific ate=true&trustCertificateKeyStoreUrl=file:\${自签名CA的JKS格式证书}&trustCertificateKe yStorePassword=\${自签名CA证书的JKS密钥库的密码} 双向认证 MySQL client: mysql -uroot -proot --ssl-mode=VERIFY_CA --ssl-ca='{自签名CA证书}' - -ssl-cert='{客户端数字证书}' --ssl-key='{客户端私钥}' JDBC: jdbc:mysql://ip:port/schema?requireSSL=true&useSSL=true&verifyServerCertific ate=true&trustCertificateKeyStoreUrl=file:\${自签名CA证书的JKS密钥库}&trustCertificate KeyStorePassword=\${自签名CA证书的JKS密钥库password}&clientCertificateKeyStoreU rl=file:\${客户端数字证书和私钥的JKS密钥库}&clientCertificateKeyStorePassword=fil e:\${客户端数字证书和私钥的JKS密钥库password}
VERIFY_I DENTITIY	基于VERIFY_CA模式，追加了证书中服务器的主机验证；但上面自签名证书不适宜此模 式，因此这里不推荐

实验

disabled 模式

在使用 SSL 加密连接 DBLE 之前，让我们先借助抓包工具 Wireshark 来看看未使用加密连接 DBLE 时，数据传输是怎么样的。在这里使用 JDBC 作为客户端为例。在进行查询之前，笔者已先行按照上述步骤在 DBLE 侧配置并开启了 SSL，创建好了 user 表，并准备了相关的数据，在此不作为重点进行赘述。

1、非加密连接 DBLE，以下为 JDBC Demo 可供参考，与 DBLE 建立连接并查询 user 表数据：

```
public class SslTest {

    private static final String JDBC_DRIVER = "com.mysql.jdbc.Driver";

    public static void main(String[] args) throws SQLException, IOException, ClassNotFo
undException {
        List<User> res = disabled();
        System.out.println(res);
    }

    public static List<User> disabled() throws ClassNotFoundException, IOException, SQL
```



```

Exception {

    List<User> usersList = new ArrayList<>();
    Properties pro = new Properties();
    FileInputStream fis = new FileInputStream("E:\\jdbc\\src\\main\\resources\\db.properties");
    pro.load(fis);

    Class.forName(JDBC_DRIVER);
    String url = "jdbc:mysql://" + pro.getProperty("host") + ":" + pro.getProperty("port") + "/" + pro.getProperty("db");
    String fullUrlString = url + "?useSSL=false";    // 非加密连接

    Connection conn = DriverManager.getConnection(fullUrlString, pro.getProperty("user"), pro.getProperty("password"));
    PreparedStatement ps = conn.prepareStatement("select username from user");
    ResultSet rs = ps.executeQuery();
    while(rs.next()){
        String name = rs.getString("username");
        usersList.add(new User(name));
    }
    ps.close();
    rs.close();
    conn.close();
    return usersList;
}
}

```

2、开启抓包后，执行相关 demo 进行查询，将数据包过滤、解析后如下所示：

No.	Time	Source	Destination	Protocol	Length	Info
126	4.527015	192.168.2.209	10.186.60.23	TCP	66	61437 → 8066 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM=1
127	4.528467	10.186.60.23	192.168.2.209	TCP	66	8066 → 61437 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460 SACK_PERM=1 WS=1
128	4.528520	192.168.2.209	10.186.60.23	TCP	54	61437 → 8066 [ACK] Seq=1 Ack=1 Win=131328 Len=0
129	4.531118	10.186.60.23	192.168.2.209	MySQL	204	Server Greeting: proto=10 version=5.7.11-dble-3.21.10.99-31f73ba43c43a1e845aa0bf9e073d63aa08757ba-20220613132727
131	4.574998	192.168.2.209	10.186.60.23	MySQL	61	Login Request user=test db=schema1
132	4.577245	10.186.60.23	192.168.2.209	TCP	68	8066 → 61437 [ACK] Seq=151 Ack=260 Win=30016 Len=0
133	4.577719	10.186.60.23	192.168.2.209	MySQL	65	Response OK
134	4.581631	192.168.2.209	10.186.60.23	MySQL	1060	Request Query (/* mysql-connector-java-5.1.49-SNAPSHOT (Revision: \${revinfo.commit}) */SELECT @@session.auto_
135	4.585240	10.186.60.23	192.168.2.209	MySQL	1164	Response TABULAR Response
137	4.604941	192.168.2.209	10.186.60.23	MySQL	76	Request Query (SET NAMES utf8mb4)
138	4.607661	10.186.60.23	192.168.2.209	MySQL	65	Response OK
139	4.607877	192.168.2.209	10.186.60.23	MySQL	91	Request Query (SET character_set_results = NULL)
140	4.609515	10.186.60.23	192.168.2.209	MySQL	65	Response OK
141	4.609700	192.168.2.209	10.186.60.23	MySQL	75	Request Query (SET autocommit=1)
142	4.611481	10.186.60.23	192.168.2.209	MySQL	65	Response OK
143	4.620182	192.168.2.209	10.186.60.23	MySQL	77	Request Query (select * from user)
144	4.624740	10.186.60.23	192.168.2.209	MySQL	227	Response TABULAR Response
145	4.628144	192.168.2.209	10.186.60.23	MySQL	59	Request Quit
146	4.628263	192.168.2.209	10.186.60.23	TCP	54	61437 → 8066 [FIN, ACK] Seq=1374 Ack=1478 Win=131328 Len=0

> Frame 144: 227 bytes on wire (1816 bits), 227 bytes captured (1816 bits) on interface \Device\NPF_{3F40BA37-F3D1-4D35-850C-797F45E5F203}, id 0 > Ethernet II, Src: Routerbo,65:93:15 (2c:c8:1b:65:93:15), Dst: IntelCor_0b:8a:ec (d8:f8:83:0b:8a:ec) > Internet Protocol Version 4, Src: 10.186.60.23, Dst: 192.168.2.209 > Transmission Control Protocol, Src Port: 8066, Dst Port: 61437, Seq: 1305, Ack: 1369, Len: 173 > MySQL Protocol > MySQL Protocol > MySQL Protocol > MySQL Protocol > MySQL Protocol		Packet Length: 14 Packet Number: 5 text Prefix: 6 Length: 6 text: calwei text Prefix: 6 Length: 6 text: 123456
---	--	---

以以明文的信息查询到。

required 模式

在此仅以某一种 SSL 加密模式为例进行测试演示——required，在以上的 JDBC Demo 中稍加修改，将 URL 参数变更为相应的模式参数【如下所示】，即可进行加密通信：

```
String fullUrlString = url
+ "?useSSL=true&requireSSL=true&verifyServerCertificate=false";
```

然后再次抓包并执行 Demo 进行查询，解析数据包并过滤得到：

tcp.port==8066

No.	Time	Source	Destination	Protocol	Length	Info
137	4.997183	192.168.2.209	10.186.60.23	TCP	66	61510 → 8066 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM=1
138	4.999367	10.186.60.23	192.168.2.209	TCP	66	8066 → 61510 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460 SACK_PERM=1 WS=1
139	4.999454	192.168.2.209	10.186.60.23	TCP	54	61510 → 8066 [ACK] Seq=1 Ack=1 Win=131328 Len=0
140	5.003805	10.186.60.23	192.168.2.209	MySQL	204	Server Greeting proto=10 version=5.7.11-dble-3.21.10.99-31f73ba43c43a1e845aa0bf9
141	5.011152	192.168.2.209	10.186.60.23	MySQL	90	Login Request user=
142	5.014195	10.186.60.23	192.168.2.209	TCP	60	8066 → 61510 [ACK] Seq=151 Ack=37 Win=29200 Len=0
189	5.743986	192.168.2.209	10.186.60.23	TLSv1.1	198	Client Hello
190	5.746805	10.186.60.23	192.168.2.209	TCP	60	8066 → 61510 [ACK] Seq=151 Ack=181 Win=30016 Len=0
191	5.753353	10.186.60.23	192.168.2.209	TLSv1.1	1004	Server Hello, Certificate, Certificate Request, Server Hello Done
192	5.806360	192.168.2.209	10.186.60.23	TCP	54	61510 → 8066 [ACK] Seq=181 Ack=1101 Win=130048 Len=0
194	6.016375	192.168.2.209	10.186.60.23	TLSv1.1	328	Certificate, Client Key Exchange
195	6.020079	192.168.2.209	10.186.60.23	TLSv1.1	60	Change Cipher Spec
196	6.021096	192.168.2.209	10.186.60.23	TLSv1.1	123	Encrypted Handshake Message
197	6.024032	10.186.60.23	192.168.2.209	TCP	60	8066 → 61510 [ACK] Seq=1101 Ack=461 Win=31088 Len=0
198	6.038618	10.186.60.23	192.168.2.209	TLSv1.1	60	Change Cipher Spec
199	6.038618	10.186.60.23	192.168.2.209	TLSv1.1	123	Encrypted Handshake Message
200	6.038718	192.168.2.209	10.186.60.23	TCP	54	61510 → 8066 [ACK] Seq=530 Ack=1176 Win=130048 Len=0
201	6.078008	192.168.2.209	10.186.60.23	TLSv1.1	363	Application Data
202	6.094543	10.186.60.23	192.168.2.209	TLSv1.1	107	Application Data
203	6.098665	192.168.2.209	10.186.60.23	TLSv1.1	1115	Application Data
204	6.104019	10.186.60.23	192.168.2.209	TLSv1.1	1211	Application Data
205	6.122005	192.168.2.209	10.186.60.23	TLSv1.1	123	Application Data
206	6.125910	10.186.60.23	192.168.2.209	TLSv1.1	107	Application Data
207	6.126314	192.168.2.209	10.186.60.23	TLSv1.1	139	Application Data
208	6.128722	10.186.60.23	192.168.2.209	TLSv1.1	107	Application Data
209	6.129234	192.168.2.209	10.186.60.23	TLSv1.1	123	Application Data
210	6.133510	10.186.60.23	192.168.2.209	TLSv1.1	107	Application Data
211	6.142203	192.168.2.209	10.186.60.23	TLSv1.1	123	Application Data
212	6.154325	10.186.60.23	192.168.2.209	TLSv1.1	283	Application Data

> Frame 206: 107 bytes on wire (856 bits), 107 bytes captured (856 bits) on interface \Device\NPF_{3F40BA37-F3D1-4D35-850C-797F45E5F203}, id 0

> Ethernet II, Src: Routerbo_65:93:15 (2c:c8:1b:65:93:15), Dst: IntelCor_0b:8a:ec (d8:f8:83:0b:8a:ec)

> Internet Protocol Version 4, Src: 10.186.60.23, Dst: 192.168.2.209

> Transmission Control Protocol, Src Port: 8066, Dst Port: 61510, Seq: 2386, Ack: 1969, Len: 53

> Transport Layer Security

- TLV1.1 Record Layer: Application Data Protocol: mysql
 - Content Type: Application Data (23)
 - Version: TLS 1.1 (0x0302)
 - Length: 48
 - Encrypted Application Data: a7a43fd19cd99853202d24a0b0c0564826714158d7cfb92d871fa23b4184ed6e92b8cbcd...
 - [Application Data Protocol: mysql]

可以发现在建立 TCP 连接之后，SSL 协议随之进行双方的认证过程，具体协议分析可参考：

<https://www.jianshu.com/p/8028bcbc4e05>，经过认证之后，随即以 TLS 加密协议的标准将数据包进行加密后传输，即便经过初步的解析后也无法得到传输的数据信息，最终确保了数据的安全性。当然，如果有服务端的 SSL 密钥文件，在 wireshark SSL 协议设置中添加相关的密钥信息，也是可以成功解析出传输的具体数据包信息的，在此不再过多演示，感兴趣的读者可自行测试。

总结

凡是都有两面性，加密连接虽然确保了数据的安全性，但是另一方面无疑是牺牲了部分性能。从 SSL 实现方式来看，建立连接时需要进行握手、加密、解密等操作。所以耗时基本都在建立连接的阶段，这对于使用短连接的应用程序可能并不太友好，因为会产生较大的性能损耗。不过对于使用连接池或者长连接的应用程序可能会好许多。所以，对于要求高性能的应用，或者不产生核心敏感数据的应用，性能及可用性才是首要，建议不要采用 SSL 方式。

同时需要注意区别的是，DBLE 侧在进行 SSL 设置时，并没有像 MySQL 一样设置了

【require_secure_transport】类似的强制要求使用安全连接参数设置，也没有按照用户去区分 SSL 配置的适用对象，只要 DBLE 服务端开启并正确配置了 SSL 选项，所有用户与 DBLE 建立连接时均可自主选择是否需要采用 SSL 加密连接。

4.15 几步快速拥有读写分离

作者：王娟

db1e 从 3.20.10 版本开始支持单纯的读写分离功能，可以和分库分表功能分开使用。

如何快速拥有读写分离呢？

第一步，准备好一组 mysql 实例，并确保这组 mysql 实例的主从复制关系正常。如下使用一主二从的 mysql：

主：172.100.9.6:3307

从：172.100.9.2:3307、172.100.9.3:3307

分别到 2 个从实例上执行 show slave status，检查复制关系是否正常。

```
mysql> show slave status \G;
***** 1. row *****
      Slave_IO_State: Waiting for master to send event
      Master_Host: 172.100.9.6
      Master_User: repl
      Master_Port: 3307
      Connect_Retry: 60
      Master_Log_File: mysql-bin.000002
      Read_Master_Log_Pos: 2498
      Relay_Log_File: mysql-relay.000002
      Relay_Log_Pos: 2711
      Relay_Master_Log_File: mysql-bin.000002
      Slave_IO_Running: Yes
      Slave_SQL_Running: Yes
      Replicate_Do_DB:
      Replicate_Ignore_DB:
      Replicate_Do_Table:
      Replicate_Ignore_Table:
      Replicate_Wild_Do_Table:
      Replicate_Wild_Ignore_Table:
      Last_Errno: 0
      Last_Error:
      Skip_Counter: 0
      Exec_Master_Log_Pos: 2498
      Relay_Log_Space: 2914
      Until_Condition: None
```

第二步，在 db.xml 配置 mysql 实例。如下：

```
<dbGroup rwSplitMode="1" name="ha_group1" delayThreshold="100">
<heartbeat>show slave status</heartbeat>
<dbInstance name="hostM1" password="*****" url="172.100.9.6:3307" user="test"
maxCon="1000" minCon="10" primary="true"/>
<dbInstance name="hostS1" password="*****" url="172.100.9.2:3307" user="test"
maxCon="1000" minCon="10" readWeight="1"/>
<dbInstance name="hostS2" password="*****" url="172.100.9.3:3307" user="test"
maxCon="1000" minCon="10" readWeight="1"/>
</dbGroup>
```

配置 dbGroup 时需要注意以下参数：

rwSplitMode: 读操作的负载均衡模式，可选值 0/1/2/3

在进行读负载均衡的时候会根据这个配置进行

- 0: 不做均衡，直接分发到主实例，从实例将被忽略，不会尝试建立连接池，但会有心跳连接
- 1: 读操作在所有从实例中均衡，当所有从实例都不可用时，下发语句会报错。
- 2: 读操作在所有实例中均衡。
- 3: 读操作在所有从实例中均衡，当所有从实例都不可用时，将语句发往主实例。

delayThreshold: 指定主从延迟阈值，单位秒，默认 -1，表示无延迟

在进行读取负载均衡的时候会根据最近一次的心跳状态以及读库和主库的延迟进行判断，如果主从复制不工作或者复制延迟超过 delayThreshold 配置，则认为此节点不适合进行读取，依赖于心跳语句为 show slave status。如果 delayThreshold=-1 那么读负载均衡选取的时候不会进行延迟检测。

readWeight: 节点权重(负载均衡时候使用)

负载均衡过程中会查看所有节点的权重是否相等，如果不相等，那么就会根据权重来配置压力。该值需是大于等于 0 的整数。如果配为 0 表示该节点不参与读。需注意，总权重（所有节点权重之和）必须大于 0。

如何区分读节点与写节点？

写节点: primary="true"

读节点: primary 没配置或者 primary="false"

第三步，user.xml 增加读写分离用户。通过 dbGroup 指定到 db.xml 里 dbGroup 的 name。

```
<rwSplitUser name="rwSplit1" password="111111" dbGroup="ha_group1" />
```

第四步，到 dble 管理端执行 reload @@config_all，使配置生效。这时就可以使用新创建的读写分离用户 rwSplit1 登录 dble 了。

如何验证读写分离配置是否生效呢？

分别到 3 个 mysql 实例执行：set global general_log = on，开启 general log。

```
mysql> show variables like '%general_log%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| general_log   | OFF   |
| general_log_file | /root/sandboxes/msb_5_7_25/data/dble-2.log |
+-----+-----+
2 rows in set (0.00 sec)

mysql> set global general_log = on;
Query OK, 0 rows affected (0.01 sec)

mysql> show variables like '%general_log%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| general_log   | ON    |
| general_log_file | /root/sandboxes/msb_5_7_25/data/dble-2.log |
+-----+-----+
2 rows in set (0.00 sec)
```

使用 `rwSplit1` 用户登录 `db1e 8066` 端口，执行以下 `sql`：

```
insert into test_table values (1, 'name1'),(2, 'name2');
```

```
select * from test_table;
```

```
mysql>
mysql> create table test_table (id int, name varchar(10));
Query OK, 0 rows affected (0.02 sec)

mysql> insert into test_table values (1, 'name1'),(2, 'name2');
Query OK, 2 rows affected (0.04 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> select * from test_table;
+-----+-----+
| id | name |
+-----+-----+
| 1 | name1 |
| 2 | name2 |
+-----+-----+
2 rows in set (0.01 sec)
```

查看写节点的 `general log`，`insert` 语句发到写节点上。

```
[root@mysql-master2 opt]# tail -15 /root/sandboxes/msb_5_7_25/data/mysql-master2.log
2022-06-14T07:40:45.439795Z 34 Query /*# from=1 reason=heartbeat*/show slave status
2022-06-14T07:40:46.730217Z 36 Query create table test_table (id int, name varchar(10))
2022-06-14T07:40:55.439731Z 34 Query /*# from=1 reason=heartbeat*/show slave status
2022-06-14T07:41:05.439603Z 34 Query /*# from=1 reason=heartbeat*/show slave status
2022-06-14T07:41:15.439788Z 34 Query /*# from=1 reason=heartbeat*/show slave status
2022-06-14T07:41:25.439616Z 34 Query /*# from=1 reason=heartbeat*/show slave status
2022-06-14T07:41:35.439469Z 34 Query /*# from=1 reason=heartbeat*/show slave status
2022-06-14T07:41:39.686377Z 36 Query insert into test_table values (1, 'name1'),(2, 'name2')
2022-06-14T07:41:45.439601Z 34 Query /*# from=1 reason=heartbeat*/show slave status
```

分点查看 2 个读节点的 `general log`，`select` 语句发到其中一个读节点上。

```
[root@db1e-2 opt]# tail -15 /root/sandboxes/msb_5_7_25/data/db1e-2.log
2022-06-14T07:42:05.440126Z 17 Query /*# from=1 reason=heartbeat*/show slave status
2022-06-14T07:42:15.440049Z 17 Query /*# from=1 reason=heartbeat*/show slave status
2022-06-14T07:42:25.439952Z 17 Query /*# from=1 reason=heartbeat*/show slave status
2022-06-14T07:42:35.440064Z 17 Query /*# from=1 reason=heartbeat*/show slave status
2022-06-14T07:42:45.440786Z 17 Query /*# from=1 reason=heartbeat*/show slave status
2022-06-14T07:42:55.440087Z 17 Query /*# from=1 reason=heartbeat*/show slave status
2022-06-14T07:42:58.476297Z 18 Query use 'test_db';
2022-06-14T07:42:58.476927Z 18 Query SET CHARACTER_SET_CLIENT = utf8, CHARACTER_SET_RESULTS = utf8, COLLATION_CONNECTION = utf8_general_ci
2022-06-14T07:42:58.477227Z 18 Query select * from test_table
2022-06-14T07:43:05.440255Z 17 Query /*# from=1 reason=heartbeat*/show slave status
2022-06-14T07:43:15.440398Z 17 Query /*# from=1 reason=heartbeat*/show slave status
2022-06-14T07:43:25.440344Z 17 Query /*# from=1 reason=heartbeat*/show slave status
```

补充:

- 1、同时开启 dble 读写分离和分库分表的功能时，分库分表引用的 dbGroup 和 读写分离引用的 dbGroup 必须相互独立。rwSplitUser 引用的 dbGroup，仅需在 db.xml 中定义即可。
- 2、多个 rwSplitUser 可以引用同一个 dbGroup。
- 3、dble 读写分离哪些语句发往主实例，哪些语句发往从实例呢？参考：
<https://github.com/actiontech/dble/discussions/3145>

4.16 浅谈 dble 引入 ClickHouse 的配置操作

作者：阙秀平

一、功能背景

我们知道，dble 是基于 MySQL 的高可扩展性的分布式中间件，而 MySQL 擅长的是联机事务处理（OLTP），那么面对越来越多的联机分析(OLAP)需求，MySQL 就显得有些捉襟见肘了。为了能够提供良好的联机分析(OLAP)能力，dble 在 3.22.01 版本提供了解决方案，可以在后端节点支持 ClickHouse，借助 ClickHouse 的能力，提供强大的联机分析(OLAP)服务。

那什么是 ClickHouse？为什么选择 ClickHouse 呢？ClickHouse 是一个用于联机分析(OLAP)的列式数据库管理系统(DBMS)。相对于行式数据库 MySQL，对于大多数查询而言，列式数据库处理速度至少提高 100 倍。

那 dble 是怎么实现的呢？下面来简单操作一下。

二、环境准备

服务器上部署四个 docker 容器：

7. 部署两个 5.7.25 版本的 MySQL 数据库。
8. 部署 3.22.01 版本 dble 环境，安装可参考：【0.3 快速开始 • dble manual】（https://actiontech.github.io/dble-docs-cn/0.overview/0.3_dble_quick_start.html）
9. 部署 22.6.1 版本 ClickHouse 数据库，安装可参考：【安装 | ClickHouse Docs】（<https://clickhouse.com/docs/zh/getting-started/install/>），验证 ClickHouse 可用。

```
# clickhouse-client -u test --password password -h 127.0.0.1
ClickHouse client version 22.6.1.1985 (official build).
Connecting to 127.0.0.1:9000 as user test.
Connected to ClickHouse server version 22.6.1 revision 54455.
```

```
clickhouse-1 :) show databases;
```

```
SHOW DATABASES
```

```
Query id: b2a417e7-7a76-4461-896d-961540eb60a1
```

name
INFORMATION_SCHEMA
default
information_schema
system

```
4 rows in set. Elapsed: 0.003 sec.
```

三、dble+ClickHouse

1. 进入`{install_dir}/dble/conf/user.xml`，新增用户 `analysisUser`。

```
<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE dble:user SYSTEM "user.dtd">
<dble:user xmlns:dble="http://dble.cloud/">
    <managerUser name="root" password="password"/>
    <analysisUser name="ana1" password="password" dbGroup="ha_group3" />
</dble:user>
```

- user.xml 配置注意：
 - a. 一个 `analysisUser` 仅对应一个 `dbGroup`。
 - b. 多个 `analysisUser` 可引用同一个 `dbGroup`。

2. 进入`{install_dir}/dble/conf/db.xml`，新增 `ha_group3`

```
<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE dble:db SYSTEM "db.dtd">
<dble:db xmlns:dble="http://dble.cloud/">
    <dbGroup rwSplitMode="0" name="ha_group3" delayThreshold="100" >
        <heartbeat>select user()/</heartbeat>
        <dbInstance name="hostM3" password="password" url="172.100.9.13:9004" user="tes
t" maxCon="1000" minCon="10" primary="true" databaseType="clickhouse"/>
    </dbGroup>
</dble:db>
```

- db.xml 配置注意：
 - a. `heartbeat` 中下发心跳的语句需要是 ClickHouse 支持的语法。

- b. analysisUser 用户所引用的 dbgroup 中配置的数据库必须和 databaseType 统一。
- c. ClickHosue 的端口需配置 9004，9004 提供 MySQL 的协议。

3. dble 启动成功，使用 analysisUser 用户登录。

```
# mysql -h127.0.0.1 -uana1 -ppassword -P8066
#下发 show databases 语句返回的结果可知，dble 未使用分库分表功能，仅展示后端 clickhouse 的数据
库信息
mysql> show databases;
+-----+
| name          |
+-----+
| INFORMATION_SCHEMA |
| default        |
| information_schema |
| system         |
+-----+
4 rows in set (0.00 sec)

#进入 default 库
mysql> use default;
Database changed, 1 warning

#建表
mysql> CREATE TABLE t1 (x String) ENGINE = TinyLog;;
Query OK, 1 row affected (0.03 sec)

#检查表是否正确
mysql> desc t1;
+-----+-----+-----+-----+-----+-----+-----+
| name | type  | default_type | default_expression | comment | codec_expression | ttl_expression |
+-----+-----+-----+-----+-----+-----+-----+
| x    | String |              |                    |        |                  |                |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

#插入数据
mysql> INSERT INTO t1 VALUES(1);
Query OK, 1 row affected (0.00 sec)

#检查数据是否正确插入
mysql> SELECT x, toTypeName(x) FROM t1;
+-----+-----+
| x    | toTypeName(x) |
+-----+-----+
```

```
| 1 | String |
+-----+-----+
1 row in set (0.00 sec)
```

进入 ClickHouse 中验证。

```
# clickhouse-client -u test --password password -h 127.0.0.1
# 进入默认库
clickhouse-1 :) use default;
USE default
Query id: eac19b96-6da6-4d77-8258-e5a827c31685
Ok.
0 rows in set. Elapsed: 0.002 sec.
# 查看表是否建立成功
clickhouse-1 :) show tables;
SHOW TABLES
Query id: 02f5018d-4b7b-4348-be5a-89fdcdbc3aa5

┌─name─┐
│ t1   │
└─┬──┘

1 row in set. Elapsed: 0.003 sec.
# 查看表是否正确
clickhouse-1 :) desc t1;
DESCRIBE TABLE t1
Query id: 6721fa63-c52c-4236-8c4a-27a1ffdc059

┌─name─┐┌─type─┐┌─default_type─┐┌─default_expression─┐┌─comment─┐┌─codec_expression─┐┌─ttl_expression─┐
│ x    ││ String │              │                    │          │                  │                │
└─┬──┘└─┬──┘└─┬──┘└─┬──┘└─┬──┘└─┬──┘└─┬──┘
1 row in set. Elapsed: 0.002 sec.
# 查看数据是否和 dble 查询的一致
clickhouse-1 :) SELECT x, toTypeName(x) FROM t1;
SELECT
  x,
  toTypeName(x)
FROM t1
Query id: 548ea88e-b1a1-4362-912d-ffa770c7c1be

┌─x─┐┌─toTypeName(x)─┐
│ 1 │ String         │
└─┬──┘└─┬──┘
2 rows in set. Elapsed: 0.002 sec.
```

4. 在 ClickHouse 导入 4.4G 的数据，对比 ClickHouse 和 dble 的查询结果。

```
#clickhouse 容器安装路径下 下载数据，网速好的话五分钟可下载完成
wget http://prod.publicdata.landregistry.gov.uk.s3-website-eu-west-1.amazonaws.com/pp-complete.csv
```

```
#clickhouse-client -u test --password password -h 127.0.0.1, 登录默认库建表
CREATE TABLE uk_price_paid
(
```

```

    price UInt32,
    date Date,
    postcode1 LowCardinality(String),
    postcode2 LowCardinality(String),
    type Enum8('terraced' = 1, 'semi-
detached' = 2, 'detached' = 3, 'flat' = 4, 'other' = 0),
    is_new UInt8,
    duration Enum8('freehold' = 1, 'leasehold' = 2, 'unknown' = 0),
    addr1 String,
    addr2 String,
    street LowCardinality(String),
    locality LowCardinality(String),
    town LowCardinality(String),
    district LowCardinality(String),
    county LowCardinality(String),
    category UInt8
) ENGINE = MergeTree ORDER BY (postcode1, postcode2, addr1, addr2);

```

#clickhouse 容器安装路径下 导入数据，可能耗时 40s 左右

```

clickhouse-local --input-format CSV --structure '
    uuid String,
    price UInt32,
    time DateTime,
    postcode String,
    a String,
    b String,
    c String,
    addr1 String,
    addr2 String,
    street String,
    locality String,
    town String,
    district String,
    county String,
    d String,
    e String
' --query "
    WITH splitByChar(' ', postcode) AS p
    SELECT
        price,
        toDate(time) AS date,
        p[1] AS postcode1,
        p[2] AS postcode2,
        transform(a, ['T', 'S', 'D', 'F', 'O'], ['terraced', 'semi-

```



```
detached', 'detached', 'flat', 'other']) AS type,
    b = 'Y' AS is_new,
    transform(c, ['F', 'L', 'U'], ['freehold', 'leasehold', 'unknown']) AS duration,
    addr1,
    addr2,
    street,
    locality,
    town,
    district,
    county,
    d = 'B' AS category
FROM table" --date_time_input_format best_effort < pp-complete.csv | clickhouse-
client --query "INSERT INTO uk_price_paid FORMAT TSV"
```

登录 dble 和 ClickHouse 查看对比数据。

```
[root@dble-1 opt]# mysql -h127.0.0.1 -uanal -password -P8066
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 15
Server version: 5.7.21-dbl-e-3.22.01.1-6c90fba9b199f2829d8478cb7c6ce8ae2b922925-20220615063052

Copyright (c) 2000, 2019, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use default;
Database changed, 1 warning
mysql> SELECT count() FROM uk_price_paid;
+-----+
| count() |
+-----+
| 27176256 |
+-----+
1 row in set (0.01 sec)

mysql> SELECT formatReadableSize(total_bytes) FROM system.tables WHERE name = 'uk_price_paid';
+-----+
| formatReadableSize(total_bytes) |
+-----+
| 286.64 MiB                       |
+-----+
1 row in set (0.01 sec)

mysql> SELECT toYear(date) AS year, round(avg(price)) AS price, bar(price, 0, 1000000, 80) FROM uk_price_paid GROUP BY year ORDER BY year;
+----+-----+-----+
| year | price | bar(round(avg(price)), 0, 1000000, 80) |
+----+-----+-----+
| 1995 | 67933 | ████████████████████████████████████ |
| 1996 | 71507 | ████████████████████████████████████ |
| 1997 | 78536 | ████████████████████████████████████ |
| 1998 | 85439 | ████████████████████████████████████ |
| 1999 | 96038 | ████████████████████████████████████ |
| 2000 | 107486 | ████████████████████████████████████ |
| 2001 | 118888 | ████████████████████████████████████ |
| 2002 | 137945 | ████████████████████████████████████ |
| 2003 | 155893 | ████████████████████████████████████ |
| 2004 | 178887 | ████████████████████████████████████ |
| 2005 | 189356 | ████████████████████████████████████ |
| 2006 | 203530 | ████████████████████████████████████ |
| 2007 | 219379 | ████████████████████████████████████ |
| 2008 | 217054 | ████████████████████████████████████ |
| 2009 | 213418 | ████████████████████████████████████ |
| 2010 | 236107 | ████████████████████████████████████ |
| 2011 | 232803 | ████████████████████████████████████ |
| 2012 | 238381 | ████████████████████████████████████ |
| 2013 | 256923 | ████████████████████████████████████ |
| 2014 | 279984 | ████████████████████████████████████ |
| 2015 | 297263 | ████████████████████████████████████ |
| 2016 | 313470 | ████████████████████████████████████ |
| 2017 | 346297 | ████████████████████████████████████ |
| 2018 | 350466 | ████████████████████████████████████ |
| 2019 | 351985 | ████████████████████████████████████ |
| 2020 | 375697 | ████████████████████████████████████ |
| 2021 | 379729 | ████████████████████████████████████ |
| 2022 | 370402 | ████████████████████████████████████ |
+----+-----+-----+
28 rows in set (0.05 sec)
```

```

clickhouse-1 :) use default;
USE default
Query id: 2275e192-f9a3-48c7-b291-74c0c6f8ed4b
Ok.
0 rows in set. Elapsed: 0.001 sec.
clickhouse-1 :) SELECT count() FROM uk_price_paid;
SELECT count()
FROM uk_price_paid
Query id: 4b644550-5439-4aa0-a9b0-cff24ca47107
count()
27176256
1 row in set. Elapsed: 0.003 sec.
clickhouse-1 :) SELECT formatReadableSize(total_bytes) FROM system.tables WHERE name = 'uk_price_paid';
SELECT formatReadableSize(total_bytes)
FROM system.tables
WHERE name = 'uk_price_paid'
Query id: 0d0a7315-9671-482c-9501-c74953384cbc
formatReadableSize(total_bytes)
235.39 MiB
1 row in set. Elapsed: 0.003 sec.
clickhouse-1 :) SELECT toYear(date) AS year, round(avg(price)) AS price, bar(price, 0, 1000000, 80) FROM uk_price_paid GROUP BY year ORDER BY year;
SELECT
  toYear(date) AS year,
  round(avg(price)) AS price,
  bar(price, 0, 1000000, 80)
FROM uk_price_paid
GROUP BY year
ORDER BY year ASC
Query id: 83a04c67-1f9b-4f05-ace0-ed0d080d9698

```

year	price	bar(round(avg(price)), 0, 1000000, 80)
1995	67933	
1996	71507	
1997	78536	
1998	85439	
1999	96038	
2000	107486	
2001	118888	
2002	137945	
2003	155893	
2004	178887	
2005	189356	
2006	203530	
2007	219379	
2008	217054	
2009	213418	
2010	236107	
2011	232803	
2012	238381	
2013	256923	
2014	279984	
2015	297263	
2016	313470	
2017	346297	
2018	350486	
2019	351985	
2020	375697	
2021	379729	
2022	370402	

```

28 rows in set. Elapsed: 0.064 sec. Processed 27.18 million rows, 163.06 MB (423.68 million rows/s., 2.54 GB/s.)

```

clickhouse 侧结果

四、dbler+MySQL+ClickHouse

1. 修改 user.xml 的配置

```

<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE dbler:user SYSTEM "user.dtd">
<dbler:user xmlns:dbler="http://dbler.cloud/">
  <managerUser name="root" password="password"/>
  <shardingUser name="test" password="password" schemas="schema1"/>
  <analysisUser name="ana1" password="password" dbGroup="ha_group3" />
</dbler:user>

```

2. 修改 db.xml 的配置

```
<!DOCTYPE dble:db SYSTEM "db.dtd">
<dble:db xmlns:dble="http://dble.cloud/">
  <dbGroup rwSplitMode="0" name="ha_group1" delayThreshold="100" >
    <heartbeat>select user()</heartbeat>
    <dbInstance name="hostM1" password="password" url="172.100.9.5:3307" user="test"
maxCon="1000" minCon="10" primary="true">
    </dbInstance>
  </dbGroup>

  <dbGroup rwSplitMode="0" name="ha_group2" delayThreshold="100" >
    <heartbeat>select user()</heartbeat>
    <dbInstance name="hostM2" password="password" url="172.100.9.6:3307" user="test"
maxCon="1000" minCon="10" primary="true">
    </dbInstance>
  </dbGroup>

  <dbGroup rwSplitMode="0" name="ha_group3" delayThreshold="100" >
    <heartbeat>select user()</heartbeat>
    <dbInstance name="hostM3" password="password" url="172.100.9.13:9004" user="test"
maxCon="1000" minCon="10" primary="true" databaseType="clickhouse"/>
    </dbInstance>
  </dbGroup>
</dble:db>
```

- db.xml 配置注意

- a. shardingUser 用户所引用的 dbgroup 中 databaseType 的参数值只能是 mysql，当 databaseType 未设置时，默认是 mysql。
- b. shardingUser 引用的 dbGroup，需要被配置的 schemas 对应的 sharding.xml 中的 shardingNode 所引用

3. 修改 sharding.xml 配置

```
<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE dble:sharding SYSTEM "sharding.dtd">
<dble:sharding xmlns:dble="http://dble.cloud/">
  <schema shardingNode="dn1" name="schema1" sqlMaxLimit="100">
    <shardingTable name="sharding_4_t1" shardingNode="dn1,dn2,dn3,dn4" function="hash-four" shardingColumn="id"/>
  </schema>
  <shardingNode dbGroup="ha_group1" database="db1" name="dn1" />
  <shardingNode dbGroup="ha_group2" database="db1" name="dn2" />
  <shardingNode dbGroup="ha_group1" database="db2" name="dn3" />
  <shardingNode dbGroup="ha_group2" database="db2" name="dn4" />
  <function class="Hash" name="hash-four">
```

```

    <property name="partitionCount">4</property>
    <property name="partitionLength">1</property>
  </function>

</dbld:sharding>

```

4. dble 启动成功，分别使用 **shardingUser** 用户和 **analysisUser** 用户登录。

```
#mysql -h127.0.0.1 -utest -ppassword -P8066 分库分表用户
```

```
mysql> show databases;
+-----+
| DATABASE |
+-----+
| schema1  |
+-----+
1 row in set (0.01 sec)
```

```
#mysql -h127.0.0.1 -uana1 -ppassword -P8066 analysisUser 用户
```

```
mysql> show databases;
+-----+
| name          |
+-----+
| INFORMATION_SCHEMA |
| default       |
| information_schema |
| system        |
+-----+
4 rows in set (0.00 sec)
```

一种可能的业务架构：

1. OLTP: client 端会发送请求至 dble，dble (shardingUser, rwSplitUser) 会将语句发送至 MySQL，然后返回请求。
2. 复制: dble 后端的 MySQL 的数据会同步至 MySQL 汇总数据库，为了使用 ClickHouse 分析数据，使用工具把 MySQL 汇总数据库的数据同步至 ClickHouse。
3. OLAP: client 端会发送请求至 dble，dble (analysisUser) 会将语句发送至 ClickHouse，然后返回请求。实现 dble+MySQL+ClickHouse 的数据分析。

4.17 如何通过 dble 的 split 功能，快速地将数据导入到 dble 中

作者：马莹乐

split 功能的介绍

有一天，我们一位社区用户提了这样一个问题：“我们前期对 dble 的调研和验证性测试都做好了，老大很满意，我来问一下，如果把我们的业务从旧的单机 MySQL 改造成用 dble 的话，历史数据应该怎么导入呢？”

这是个好问题，dble 支持的导入导出方式有多种，具体详见 [文档 3.11.1](https://actiontech.github.io/dble-docs-cn/3.SQL_Syntax/3.11_Export_and_Import.html) (https://actiontech.github.io/dble-docs-cn/3.SQL_Syntax/3.11_Export_and_Import.html)，本次我们介绍的 split 功能可以理解为导入过程加速器，那它是怎么加速的呢？

大家可以考虑这样一个场景：一份原始数据通过 mysqldump 工具 dump 下了一个 sql 文件（下称“dump 文件”），正常情况下，这个 dump 文件也不会太小，直接拿着这个 dump 文件通过 dble 的业务端去导入，可能需要等上一段时间才能完成，而且这个过程一定会比直接往 MySQL 里导入数据慢一些，万一导入数据期间发生了什么错误，也会难以排查。

那当我拿到 dump 文件后，就只能通过直连 dble 业务端导入数据才能实现历史数据的拆分和导入吗？有没有一种可能的方式，是先把 dump 文件按照配置拆分成 dump 子文件，然后我们就可以拿着这些拆分后的 dump 子文件，分别到与之对应的后端 MySQL 上直接导入数据呢？

于是 dble 的 split 功能应运而生。在这里它就是做 dump 文件拆分工作的。它工作的大致过程就是先根据分库分表的配置，对 dump 文件按照分片进行处理，配置有多少分片就会产生多少个拆分后的 dump 子文件。当然，任凭你 sharding.xml 里配置的有 shardingTable、有 globalTable、有 singleTable，还是全都有，都会体现在生成的 dump 子文件里，我们拿着这些 dump 子文件，就可以直接导入到各自分片对应的后端 MySQL 中，当完成后端数据的导入操作后，只需要再同步一下 dble 的元数据信息，这样就完成了历史数据的拆分和导入。

经 split 处理后，生成的 dump 子文件示例：

```
[root@db1e-1 tmp]# ll dump
total 40
-rw-r--r-- 1 root root 1566 Apr 2 16:53 test4.sql-dn1-1648889591863.dump
-rw-r--r-- 1 root root 1375 Apr 2 16:53 test4.sql-dn10-1648889591863.dump
-rw-r--r-- 1 root root 1566 Apr 2 16:53 test4.sql-dn2-1648889591863.dump
-rw-r--r-- 1 root root 1566 Apr 2 16:53 test4.sql-dn3-1648889591863.dump
-rw-r--r-- 1 root root 1566 Apr 2 16:53 test4.sql-dn4-1648889591863.dump
-rw-r--r-- 1 root root 1566 Apr 2 16:53 test4.sql-dn5-1648889591863.dump
-rw-r--r-- 1 root root 1375 Apr 2 16:53 test4.sql-dn6-1648889591863.dump
-rw-r--r-- 1 root root 1743 Apr 2 16:53 test4.sql-dn7-1648889591863.dump
-rw-r--r-- 1 root root 1743 Apr 2 16:53 test4.sql-dn8-1648889591863.dump
-rw-r--r-- 1 root root 1375 Apr 2 16:53 test4.sql-dn9-1648889591863.dump
[root@db1e-1 tmp]# ll dump3
total 56
```

mysqldump文件名-分片名-时间戳.dump

不过在使用 mysqldump 导出历史数据 sql 文件时，需要按照以下格式进行导出，否则可能出现错误，因为有些 mysqldump 参数，dble 不支持。

```
./mysqldump -h127.0.0.1 -utest -P3306 -p111111 --default-character-set=utf8mb4 --
master-data=2 --single-transaction --set-gtid-purged=off --hex-blob --
databases schema1 --result-file=export.sql
```

其他注意点参见 [文档 3.11.2](https://actiontech.github.io/dble-docs-cn/3.SQL_Syntax/3.11_Export_and_Import.html) (https://actiontech.github.io/dble-docs-cn/3.SQL_Syntax/3.11_Export_and_Import.html)

基本使用介绍

语法与示例

登录 dble 的管理端口 9066 执行 split 命令，语法如下：

```
mysql > split src dest [-sschema] [-r500] [-w512] [-l10000] [--ignore] [-t2]
```

src: 表示原始 dump 文件名

dest: 表示生成的 dump 文件存放的目录

-s: 表示默认逻辑数据库名，当 dump 文件中不包含 schema 的相关语句时，会默认导出到该 schema。如：当 dump 文件中包含 schema 时，dump 文件中的优先级高于 -s 指定的；若文件中的 schema 不在配置中，则使用 -s 指定的 schema，若 -s 指定的 schema 也不在配置中，则返回报错

-r: 表示设置读文件队列大小，默认 500

-w: 表示设置写文件队列大小，默认 512，且必须为 2 的次幂

-l: 表示 split 后一条 insert 中最多包含的 values，只针对分片表，默认 4000

--ignore: insert 时，忽略已存在的数据

-t: 表示多线程处理文件中 insert 语句时线程池的大小

使用示例：

```
mysql > split /path-to-mysqldump-file/mysqldump.sql /tmp/dump-dir-name;
mysql > split /path-to-mysqldump-file/mysqldump.sql /tmp/dump-dir-name -sdatabase1;
mysql > split /path-to-mysqldump-file/mysqldump.sql /tmp/dump-dir-name -sdatabase1 -
r600;
mysql > split /path-to-mysqldump-file/mysqldump.sql /tmp/dump-dir-name -sdatabase1 -
r600 -w1024;
mysql > split /path-to-mysqldump-file/mysqldump.sql /tmp/dump-dir-name -sdatabase1 -
r600 -w1024 -l10000;
mysql > split /path-to-mysqldump-file/mysqldump.sql /tmp/dump-dir-name -sdatabase1 -
r600 -w1024 -l10000 --ignore;
mysql > split /path-to-mysqldump-file/mysqldump.sql /tmp/dump-dir-name -sdatabase1 -
r600 -w1024 -l10000 --ignore -t4;
```

一个 split 执行的示例：


```

test@820-107:/opt/splitTest$ mysql -uroot -p111111 -h127.0.0.1 -P9066
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 3
Server version: 5.7.11-dble-9.9.9-3d3612c9b31c2798b782b07066bc4615253920d4-20220321060316 dble Server (ActionTech)

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> split /opt/splitTest/benchmarksql_with_data.sql /tmp/split40k -sbenchmarksql -l40000;
Query OK, 0 rows affected (16 min 41.70 sec)
please see detail in dump.log.

```

根据此示例，则可以在指定的目录 `/tmp/split40k` 下查看到 `split` 生成的 `dump` 子文件：

```

test@820-107:/tmp/split40k$ pwd
/tmp/split40k
test@820-107:/tmp/split40k$ ll
total 77669644
-rw-r--r-- 1 root root 7954018928 Jun 28 14:01 benchmarksql_with_data.sql-an10-1656394913077.dump
-rw-r--r-- 1 root root 7955100275 Jun 28 14:01 benchmarksql_with_data.sql-an1-1656394913077.dump
-rw-r--r-- 1 root root 7952857380 Jun 28 14:01 benchmarksql_with_data.sql-an2-1656394913077.dump
-rw-r--r-- 1 root root 7953643554 Jun 28 14:01 benchmarksql_with_data.sql-an3-1656394913077.dump
-rw-r--r-- 1 root root 7953431335 Jun 28 14:01 benchmarksql_with_data.sql-an4-1656394913077.dump
-rw-r--r-- 1 root root 7952739769 Jun 28 14:01 benchmarksql_with_data.sql-an5-1656394913077.dump
-rw-r--r-- 1 root root 7953553523 Jun 28 14:01 benchmarksql_with_data.sql-an6-1656394913077.dump
-rw-r--r-- 1 root root 7953240373 Jun 28 14:01 benchmarksql_with_data.sql-an7-1656394913077.dump
-rw-r--r-- 1 root root 7952658305 Jun 28 14:01 benchmarksql_with_data.sql-an8-1656394913077.dump
-rw-r--r-- 1 root root 7952409411 Jun 28 14:01 benchmarksql_with_data.sql-an9-1656394913077.dump
test@820-107:/tmp/split40k$

```

更加详细的使用说明，可查阅文档 `split` 命令的介绍（https://actiontech.github.io/dble-docs-cn/2.Function/2.1_manager_cmd/2.1.19_split.html）部分。

split 性能测评

讲了那么多，那它到底快了没快，快了多少呢？拆好的数据会不会缺斤少两？

我们准备做个试验来看看。在同样的测试环境下，准备了 3 组测试，具体如下：

- 对照组 1：同一 `dump` 文件，在不使用 `dble` 的情况下，直连 `MySQL` 整体导入 `MySQL` 的耗时，并获取各个 `table` 的总行数，用来作为其他测试组导入的数据是否存在问题的标杆。
- 对照组 2：同一 `dump` 文件，同一环境下，直连 `dble` 导入数据的耗时，以及各个 `table` 的总行数，各个分片上每张表的行数和 `checksum` 值
- 实验组：同一 `dump` 文件，同一环境下，`split` 处理+导入的耗时，以及各个 `table` 的总行数，各个分片上每张表的行数和 `checksum` 值

注：由于数据经过了拆分，`dble` 业务端暂不支持 `checksum table` 的语法，所以难以从 `table checksum` 值这个层面去对比原始 `MySQL` 中各个 `table` 总体的 `checksum` 值，所以本次试验只对比了这 3 组测试中各个 `table` 总行数，以及对照组 2 和实验组各个分片对应 `table` 的行数和 `checksum` 值。

试验环境

- 测试使用 `dble`：单节点 `dble`，版本为 `5.7.11-dble-3.22.01-0-e5d394e5994a004cd11b28dc5c171904769adad3-20220427091228`
- 数据文件来源：

使用 `benchmarksql` 造 1000 个 `warehouse` 的数据作为数据源（本次测试使用的表结构未添加外键关

系)，mysqldump 获取的 dump 文件约 75G 本次试验采用了 10 个分片的测试，同时由于每个分片的数据导入的时间和数据量的大小成正比，所以采用求模的拆分算法，使得数据可均匀分布在每个分片上。

- split 命令：实验组采用 split 命令 `/opt/splitTest/benchmarksql_with_data.sql /tmp/splittest - sbenchmarksql;`
- dble 配置：

本次试验拿 4 个 MySQL 实例作为后端 MySQL，并统一采用 10 个分片作为对照组 2 和实验组的配置，相关配置如下：

sharding.xml

```
<?xml version="1.0"?>
<!DOCTYPE dble:sharding SYSTEM "sharding.dtd">
<dble:sharding xmlns:dble="http://dble.cloud/" version="4.0">

<schema name="benchmarksql">
    <globalTable name="bmsql_config" shardingNode="an$1-10" checkClass="CHECKSUM" cron="0 0 0 * * ?"></globalTable>
    <globalTable name="bmsql_item" shardingNode="an$1-10" checkClass="CHECKSUM" cron="0 0 0 * * ?"></globalTable>
    <shardingTable name="bmsql_warehouse" shardingNode="an$1-10" function="benchmarksql-mod" shardingColumn="w_id"></shardingTable>
    <shardingTable name="bmsql_district" shardingNode="an$1-10" function="benchmarksql-mod" shardingColumn="d_w_id"></shardingTable>
    <shardingTable name="bmsql_customer" shardingNode="an$1-10" function="benchmarksql-mod" shardingColumn="c_w_id"></shardingTable>
    <shardingTable name="bmsql_history" shardingNode="an$1-10" function="benchmarksql-mod" shardingColumn="h_w_id"></shardingTable>
    <shardingTable name="bmsql_new_order" shardingNode="an$1-10" function="benchmarksql-mod" shardingColumn="no_w_id"></shardingTable>
    <shardingTable name="bmsql_oorder" shardingNode="an$1-10" function="benchmarksql-mod" shardingColumn="o_w_id"></shardingTable>
    <shardingTable name="bmsql_order_line" shardingNode="an$1-10" function="benchmarksql-mod" shardingColumn="ol_w_id"></shardingTable>
    <shardingTable name="bmsql_stock" shardingNode="an$1-10" function="benchmarksql-mod" shardingColumn="s_w_id"></shardingTable>
</schema>

<shardingNode name="an1" dbGroup="ha_group1" database="dh_dn_1"/>
<shardingNode name="an2" dbGroup="ha_group2" database="dh_dn_2"/>
<shardingNode name="an3" dbGroup="ha_group3" database="dh_dn_3"/>
<shardingNode name="an4" dbGroup="ha_group4" database="dh_dn_4"/>
<shardingNode name="an5" dbGroup="ha_group1" database="dh_dn_5"/>
<shardingNode name="an6" dbGroup="ha_group2" database="dh_dn_6"/>
<shardingNode name="an7" dbGroup="ha_group3" database="dh_dn_7"/>
```



```

<shardingNode name="an8" dbGroup="ha_group4" database="dh_dn_8"/>
<shardingNode name="an9" dbGroup="ha_group1" database="dh_dn_9"/>
<shardingNode name="an10" dbGroup="ha_group2" database="dh_dn_10"/>

<function name="benchmarksql-mod" class="Hash">
  <property name="partitionCount">10</property>
  <property name="partitionLength">1</property>
</function>
</db:sharding>

```

试验过程

在同一测试环境下，分别测试 对照组 1、2 和实验组。实验组并未将 dump 子文件转移至它对应的后端执行本机导入，而是在 dump 子文件所在机器上远程连接到各自后端 MySQL 服务，同时并发导入，并开始计时，由于是并发导入，所以导入的耗时取决于耗时最长的后端 MySQL 节点。

当各个分片的 dump 子文件均导入完成后，可在 dble 管理端执行 `reload @@metadata;` 重新加载所有元数据信息。

接着可以：

- 获取 3 组测试各自导入数据的耗时
- 查看 10 张 table 各自的总行数在 3 组测试中是否完全一致，其中对照组 2 和实验组(即直连 dble 执行的导入和 split 执行的导入)，则可以通过 dble 业务端直接查询 `select count(*) from tb_name;`
- 对照组 2 和实验组，还可通过后端 MySQL，检查每个分片对应的各个 table 的 checksum 值 `checksum table tb_name;` 和行数 `select count(*) from tb_name;`

试验记录

对照组

对照组 1

同一 mysqldump 文件(75G)，在不使用 dble 的情况下，直连 MySQL 整体导入，耗时统计：13181s

```

[2022-06-22 15:29:54] <===== loadin 108 mysql startTime
[2022-06-22 15:29:54] <===== startTime
在dble所在机器上，执行原始75Gmysqldump数据导入原生mysql耗时为： 13181s
[2022-06-22 19:09:35] <===== endTime
[2022-06-22 19:09:35] <===== loadin 108 mysql finishTime

```

mysqldump文件不经过dble，整体导入MySQL耗时

对照组 2

同一 mysqldump 文件(75G)，未经 split 分片，(mysqldump 文件在 dble 所在主机本机)直连 dble 导入耗时统计：50883s

```
Now load dump file </opt/splittest/benchmarksql_with_data.sql> to <127.0.0.1> into db <benchmarksql>
[2022-06-07 14:18:36] <=====> startTime
耗时: 50883s
total time is 50883s
[2022-06-08 04:26:39] <=====> endTime
```

75G的mysqldump文件直接导入dble的耗时

实验组

同一 mysqldump 文件(75G), 经过 split 分片处理+(在 dble 本机远程连接后端 MySQL)并发导入到后端 MySQL 的耗时统计: 912s+1839s=2751s

```
NOW we are testing loadin on dble server locally, and shardingnode num is 10-----
[2022-06-23 10:19:01] <=====> start exec split cmd on dble manager 9066
[2022-06-23 10:19:01] <=====> startTime
split /opt/splittest/benchmarksql_with_data.sql /tmp/splittest -sbenchmarksql;
dble管理端执行split的耗时: 912s
total time is 912s
[2022-06-23 10:34:13] <=====> endTime
[2022-06-23 10:34:13] <=====> finish exec split cmd on dble manager 9066

[2022-06-23 10:34:13] <=====> loadAll_shardingDumpFiles_on_dble_server startTime
[2022-06-23 10:34:13] <=====> startTime
/usr/bin/mysql -uaction -p -P3306 -h10.186.17.108 dh dn_8 < /tmp/splittest/benchmarksql_with_data.sql-an8.sql
/usr/bin/mysql -uaction -p -P3306 -h10.186.17.101 dh dn_1 < /tmp/splittest/benchmarksql_with_data.sql-an1.sql
/usr/bin/mysql -uaction -p -P3306 -h10.186.17.102 dh dn_2 < /tmp/splittest/benchmarksql_with_data.sql-an2.sql
/usr/bin/mysql -uaction -p -P3306 -h10.186.17.102 dh dn_6 < /tmp/splittest/benchmarksql_with_data.sql-an6.sql
/usr/bin/mysql -uaction -p -P3306 -h10.186.17.102 dh dn_10 < /tmp/splittest/benchmarksql_with_data.sql-an10.sql
/usr/bin/mysql -uaction -p -P3306 -h10.186.17.108 dh dn_4 < /tmp/splittest/benchmarksql_with_data.sql-an4.sql
/usr/bin/mysql -uaction -p -P3306 -h10.186.17.105 dh dn_7 < /tmp/splittest/benchmarksql_with_data.sql-an7.sql
/usr/bin/mysql -uaction -p -P3306 -h10.186.17.101 dh dn_5 < /tmp/splittest/benchmarksql_with_data.sql-an5.sql
/usr/bin/mysql -uaction -p -P3306 -h10.186.17.105 dh dn_3 < /tmp/splittest/benchmarksql_with_data.sql-an3.sql
/usr/bin/mysql -uaction -p -P3306 -h10.186.17.101 dh dn_9 < /tmp/splittest/benchmarksql_with_data.sql-an9.sql
split后的dumpfile文件从dble本机并发导入,总时长(即最长耗时)为: 1839s
[2022-06-23 11:04:52] <=====> endTime
[2022-06-23 11:04:52] <=====> loadAll_shardingDumpFiles_on_dble_server finishTime

NOW we have finished the testing of loadin on dble server locally, and shardingnode num is 10-----
```

在dble机器上远程连接后端mysql导入数 耗时

数据对比:

3 组测试中, benchmarksql 相关的 10 个 table 总行数完全一致, 其中对照组 2 和实验组 (即直连 dble 执行的导入和 split 执行的导入) 后端的各个分片上对应的每张 table 的 checksum 值和行数均是一致的。

```
test@r820-107:/tmp/splittest$ mysql -uaction -p -h 10.186.17.108 -P3306 -Dbenchmarksql;
mysql: [Warning] Using a password on the command line interface can be insecure.
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 154
Server version: 5.7.21 MySQL Community Server (GPL)

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SELECT "bmsql_config", COUNT(*) FROM bmsql_config UNION ALL
-> SELECT "bmsql_customer", COUNT(*) FROM bmsql_customer UNION ALL
-> SELECT "bmsql_district", COUNT(*) FROM bmsql_district UNION ALL
-> SELECT "bmsql_history", COUNT(*) FROM bmsql_history UNION ALL
-> SELECT "bmsql_item", COUNT(*) FROM bmsql_item UNION ALL
-> SELECT "bmsql_new_order", COUNT(*) FROM bmsql_new_order UNION ALL
-> SELECT "bmsql_oorder", COUNT(*) FROM bmsql_oorder UNION ALL
-> SELECT "bmsql_order_line", COUNT(*) FROM bmsql_order_line UNION ALL
-> SELECT "bmsql_stock", COUNT(*) FROM bmsql_stock UNION ALL
-> SELECT "bmsql_warehouse", COUNT(*) FROM bmsql_warehouse;
```

实验组	对照组2
bmsql_config count(*) 4	bmsql_config count(*) 4
bmsql_customer count(*) 30000000	bmsql_customer count(*) 30000000
bmsql_district count(*) 10000	bmsql_district count(*) 10000
bmsql_history count(*) 30000000	bmsql_history count(*) 30000000
bmsql_item count(*) 100000	bmsql_item count(*) 100000
bmsql_new_order count(*) 9000000	bmsql_new_order count(*) 9000000
bmsql_oorder count(*) 30000000	bmsql_oorder count(*) 30000000
bmsql_order_line count(*) 300011145	bmsql_order_line count(*) 300011145
bmsql_stock count(*) 100000000	bmsql_stock count(*) 100000000
bmsql_warehouse count(*) 1000	bmsql_warehouse count(*) 1000

各个table的行数

```
10 rows in set (2 min 42.49 sec)
total_lines_checking_from_dble8066 line_756_without_split (END)
```

```

dh_dn_9.bmsql_stock      670696209
count
10000000

<9> an9 -- dh_dn_9, table -- bmsql_warehouse
Table Checksum
dh_dn_9.bmsql_warehouse 1884849480
count
100

table infos of shardingNode --an10--backend schema is dh_dn_10
-----
<0> an10 -- dh_dn_10, table -- bmsql_config
Table Checksum
dh_dn_10.bmsql_config    1394662018
count
4

<1> an10 -- dh_dn_10, table -- bmsql_customer
Table Checksum
dh_dn_10.bmsql_customer 562142890
count
3000000

<2> an10 -- dh_dn_10, table -- bmsql_district
Table Checksum
dh_dn_10.bmsql_district 855310496
count
1000

<3> an10 -- dh_dn_10, table -- bmsql_history
Table Checksum
dh_dn_10.bmsql_history   2545056971
count
3000000

<4> an10 -- dh_dn_10, table -- bmsql_item
Table Checksum
dh_dn_10.bmsql_item      1883200466
count
100000

<5> an10 -- dh_dn_10, table -- bmsql_new_order
Table Checksum
dh_dn_10.bmsql_new_order 2112736336
count
900000

<6> an10 -- dh_dn_10, table -- bmsql_oorder
Table Checksum
dh_dn_10.bmsql_oorder    4144316668
count
3000000

<7> an10 -- dh_dn_10, table -- bmsql_order_line
Table Checksum
dh_dn_10.bmsql_order_line 2910395363
count
30009847

<8> an10 -- dh_dn_10, table -- bmsql_stock
Table Checksum
dh_dn_10.bmsql_stock     631555598
count
10000000

<9> an10 -- dh_dn_10, table -- bmsql_warehouse
Table Checksum
dh_dn_10.bmsql_warehouse 2444320566
count
100

dh_dn_9.bmsql_stock      670696209
count
10000000

<9> an9 -- dh_dn_9, table -- bmsql_warehouse
Table Checksum
dh_dn_9.bmsql_warehouse 1884849480
count
100

table infos of shardingNode --an10--backend schema is dh_dn_10
-----
<0> an10 -- dh_dn_10, table -- bmsql_config
Table Checksum
dh_dn_10.bmsql_config    1394662018
count
4

<1> an10 -- dh_dn_10, table -- bmsql_customer
Table Checksum
dh_dn_10.bmsql_customer 562142890
count
3000000

<2> an10 -- dh_dn_10, table -- bmsql_district
Table Checksum
dh_dn_10.bmsql_district 855310496
count
1000

<3> an10 -- dh_dn_10, table -- bmsql_history
Table Checksum
dh_dn_10.bmsql_history   2545056971
count
3000000

<4> an10 -- dh_dn_10, table -- bmsql_item
Table Checksum
dh_dn_10.bmsql_item      1883200466
count
100000

<5> an10 -- dh_dn_10, table -- bmsql_new_order
Table Checksum
dh_dn_10.bmsql_new_order 2112736336
count
900000

<6> an10 -- dh_dn_10, table -- bmsql_oorder
Table Checksum
dh_dn_10.bmsql_oorder    4144316668
count
3000000

<7> an10 -- dh_dn_10, table -- bmsql_order_line
Table Checksum
dh_dn_10.bmsql_order_line 2910395363
count
30009847

<8> an10 -- dh_dn_10, table -- bmsql_stock
Table Checksum
dh_dn_10.bmsql_stock     631555598
count
10000000

<9> an10 -- dh_dn_10, table -- bmsql_warehouse
Table Checksum
dh_dn_10.bmsql_warehouse 2444320566
count
100

各个分片上每张table的checksum比对, 总体对比无差异

test@r820-107:/tmp/splittest$ diff checksum_and_lins_each_shardingnodes_0622 keep_checksum_and_lins_each_shardingnodes_1
test@r820-107:/tmp/splittest$

```

试验结果:

在本次试验中:

- 导入速率对比: 同一 mysqldump 文件(75G), split 导入的速率是直接整体 MySQL 导入速率的 5 倍, 是直接通过 dble 整体导入速率的 18 倍。split 的导入速度达到 98G/h。
- 导入正确性对比: 通过 split 导入数据的方式和通过直连 dble 业务端导数据的最终结果是一致的。

小结

理论上，执行 `split` 命令的机器性能足够好，且 MySQL 服务器充足时，导入的速度可进一步提高，如，我们可以尝试以下策略：

- 适当增加分片数
- 拆分算法的选择，规划数据更加均匀分布在每个后端 MySQL 服务器上
dump 子文件也可先传输至后端 MySQL 本机，再执行导入，以减小网络上的消耗

强大如 `split`，也还是存在一些使用上的限制，如：

- 不支持显式配置的 `childTable`（但支持含有外键关系的表以智能 ER 关系配置在 `sharding.xml` 中）
- 不支持 `view`
- 对于使用全局序列的表，表原先全局序列中的值会被擦除，替换成全局序列，需要注意

更多详细信息可参考 [split 命令的介绍](https://actiontech.github.io/dble-docs-cn/2.Function/2.1_manager_cmd/2.1.19_split.html) (https://actiontech.github.io/dble-docs-cn/2.Function/2.1_manager_cmd/2.1.19_split.html)

4.18 实战：将业务从 MyCAT 平滑迁移到 dble

作者：肖亚洲

背景介绍

客户环境近期出现了几次问题，经过讨论后决定进行架构变更，要将 `mycat` 迁移到 `dble`。要求是：最小变动。

问题整理

结合客户情况与要求罗列了下需要考虑的事情：

- 参数设置
- 分片函数
- 数据节点的数据
- 业务 SQL

问题处理

1、参数设置 `mycat` 端参数如下：

```
<system>
<property name="defaultSqlParser">druidparser</property>
<property name="processors">4</property>
```



```
<property name="processorBufferPool">2048000</property>
<property name="processorBufferLocalPercent">100</property>
<property name="frontSocketSoRcvbuf">10485760</property>
<property name="frontSocketSoSndbuf">41943040</property>
<property name="frontSocketNoDelay">1</property>
<property name="backSocketSoRcvbuf">41943040</property>
<property name="backSocketSoSndbuf">10485760</property>
<property name="backSocketNoDelay">1</property>
<property name="maxPacketSize">2048576000</property>
<property name="memoryPageSize">100m</property>
</system>
```

针对该参数 DBLE 侧建议如下：

- defaultSqlParser 、 memoryPageSize 、 processorBufferLocalPercent 已于 dble 中被废弃，可不必配置
- processorBufferPool 参数名变更为 bufferPoolPageSize
- 其他的参数在 dble 中可在 server.xml 中与 mycat 保持一致配置信息，部分参数信息详情可见：
https://github.com/actiontech/dble-docs-cn/blob/2.19.11.0/tag/1.config_file/1.03_server.xml.md

注意：mycat 和 dble 的内存管理配置有较大不同。比如：没有 threadlocal 概念。建议阅读以下文档：
https://actiontech.github.io/dble-docs-cn/2.Function/2.07_memory_manager.html

2、分片函数

查看 mycat 分片规则：

```
<function name="mod-long" class="io.mycat.route.function.PartitionByMod">
<!-- how many data nodes -->
<property name="count">8</property>
</function>
```

直接采用 DBLE 的 hash 算法测试发现部分数据查询报错：

```
where
  user_code = '202108101432111570000000000036'
[Err] 1064 - columnValue 202108101432111570000000000036 Please eliminate any quote and non number within it
```

查看表内数据情况：

样例数据：202108101432111570000000000036

字段：user_code 为 varchar(32)

查看 mycat 环境分片规则：采用的分片算法为 mod-long 。而 long 的取值范围是 - 9223372036854774808~9223372036854774807 ， 查询出错的 202108101432111570000000000036 超过了 long 的取值范围。所以可以得出 mycat 针对这一特殊情况做了特殊处理。

针对该情况 DBLE 侧建议如下：

- (1)、建议分片算法换成 HashString

分片列值超过 Long 的最大值(9223372036854774807), 需要换成列为 String 型的分片算法;

注意: 历史数据需要在更换完算法分片后重新导入

(2)、采用 DBLE 的自定义算法

引用 mycat 的 PartitionByMod 算法, 加入[dble 的自定义算法 https://actiontech.github.io/dble-docs-cn/1.config_file/1.09_dble_route_function_spec.html?q=]中。

最终配置如下:

```
<function name="test_func" class="com.actiontech.dble.custom.sharding.algorithm.PartitionByMod">
  <property name="count">8</property>
</function>
```

3、数据节点数据

通过引入 DBLE 的自定义拆分算法功能, 兼容了原环境 mycat 上的拆分算法, 因此原 mycat 环境的存储节点数据也不需要再做任何变动。这极大的减少了我们迁移的工作量。

4、业务 SQL

经过几轮测试下来, 我们发现 mycat 在很多时候会将 SQL 直接下发到后端节点, 这就造成了我们在测试时碰到了很多问题, 因为 DBLE 对多种情况做了细分。

为了最大程度保证业务查询数据的准确性 DBLE 砍了很多此类 SQL 的支持, 但为了业务的变更最小, DBLE 也做了适当开放。

如下是部分记录:

- 对 insert into ... select ... 语法的支持目前 DBLE 放开了垂直表该语法的支持。
- 对 rownum 透传的支持目前 DBLE 放开了指定表 rownum 的透传。
- UPDATE query with sub-query is not supported
 1. update 多节点更新会触发分布式事务的问题, 难以保证一致性
 2. 按照目前的执行计划, mycat 对于 update+子查询采用的是将源 sql 进行广播下发(下发至表关联的所有节点), 这种机制在某些情况下是无法保证正确性的
 3. 多表关联更新不支持, dble 作为一个中间件产品, 处理的某一类的 sql, 对于这类 sql 正确的处理方法是需要先将子查询下发, 回收结果后再拼接外层 sql 进行下发, 目前对于这种处理方式解析、实现比较困难, 而且性能问题也是需要考量。
 4. 为了保证 update+子查询这类 sql 的正确性, 肯定不能用 mycat 这种广播下发来做, 所以对于这类 sql 目前是不支持的
- 全局表+分片表类型 SQL 支持
ERROR 4004 (HY000): Unknown function FN_TREE_PATHNAME
 1. dble 在 2.20.04.x 版本及其之后版本已经做过优化可以直接执行这种情况的 sql.
 2. 全局表+分片表之前没有处理是因为直接下发是可能存在问题, 对于全局表+分片表中特殊的场景: 全局表+垂直表, 并未计算出来这些节点最后使用的是同一个节点。(其实可以计算, dble 在 2.20.04.x 版本及之后已优化)

3. mycat 会直接下发语句到节点中，全局表在每个配置的节点都会存储相同的数据，如果将每个节点和拆分表 Left Join 的结果进行简单的 UNION ALL 合并，会造成数据的重复，不能保证数据的准确性。所以不能直接透传。

更多

针对 DBLE 对 MyCat 做的增强更多记录于：https://actiontech.github.io/dble-docs-cn/0.overview/0.2_dble_enhance_MyCat.html

4.19 dble 启动的时候做了什么之配置检测

作者：吴金玲

本文谨献给 dble 的新粉。在初次使用 dble 的时候，新手们经常会遭遇 dble 启动失败的情况，而且启动失败一般都是由于配置不正确导致的。众所周知，dble 启动失败了，我们可以去 wrapper.log 里面查找启动失败的原因，并且可喜的是一般都能从 wrapper.log 里很直观的找到启动失败的原因。但少数启动失败的原因就不是那么直观，要想快速的定位及解决问题，那就需要我们对 dble 启动时具体都做了些什么有所了解。本文以 3.22.01.0 版本为例，简单介绍一下 dble 在启动的时候是如何对配置文件的正确性进行校验的。

具体如何配置请参考 dble 的官方文档：https://actiontech.github.io/dble-docs-cn/1.config_file/1.00_config_file.html。

一、首先让我们简单了解一下 dble 都有哪些配置文件，以及他们的主要作用是什么

下载 3.22.01.0 版本的安装压缩包，解压后进入 dble 安装目录的根目录，里面有个 conf 的配置文件目录（常用的配置文件里面都有模板文件），从配置文件目录可知，dble 主要分为 4 种类型的配置文件（.cnf、.xml、.properties、.txt），每个配置文件的主要作用如下表：

文件名	主要作用	注意点	参考链接
cluster.cnf	集群参数配置 文件	修改此文件需要停止多个 dble 实例，重新初始化集群元数据，然后再逐个启动 dble	https://actiontech.github.io/dble-docs-cn/1.config_file/1.01_cluster.cnf.html
bootstrap.cnf	dble 实例启动时加载	文件分为两部分，第一部分为 JVM 启动参数，第二部分	https://actiontech.github.io/dble-docs-cn/1.config_file/1.02_bootstrap.cnf.html

文件名	主要作用	注意点	参考链接
	的系统参数配置文件	为 dble 系统参数	
bootstrap.dynamic.cnf	一般为可以通过管理命令热生效的系统参数	当 dble 重启时会替换 bootstrap.cnf 中对应的值来生效配置	https://github.com/actiontech/dbble-docs-cn/blob/master/1.config_file/1.02_bootstrap.cnf.md
user.xml	登录用户配置文件（dble 主要分四种用户）	主要注意 sql 权限和 ip 白名单的配置	https://actiontech.github.io/dbble-docs-cn/1.config_file/1.03_user.xml.html
db.xml	后端数据库和组的配置文件	为了 dble 能正常工作，后端 MySQL 用户需开启特定的权限	https://actiontech.github.io/dbble-docs-cn/1.config_file/1.04_db.xml.html
sharding.xml	虚拟库、表、算法的配置文件	注意全局序列的配置方法	https://actiontech.github.io/dbble-docs-cn/1.config_file/1.05_sharding.xml.html
sequence_conf.properties	分布式 offset-step 方式的全局序列配置文件	结合 cluster.cnf 配置，sequenceHandlerType=4	https://actiontech.github.io/dbble-docs-cn/1.config_file/1.7_global_sequence/1.7.4_distribute_offset-step.html
sequence_db_conf.properties	MySQL-offset-	结合 cluster.cnf 配置，sequenceHandlerType	https://actiontech.github.io/dbble-docs-cn/1.config_file/1.7_global_sequence/1.7.1_MySQL-offset-setp.html

文件名	主要作用	注意点	参考链接
	step 方式的全局序列配置文件	e=1	
.txt 文件	不同分片算法对应的配置文件	常用，需要时可参照文档配置	https://actiontech.github.io/dble-docs-cn/1.config_file/1.05_sharding.xml.html
log4j2.xml	日志归档规则及日志记录级别配置	当出错时可将日志级别调整为 debug 来定位详细的错误信息	https://actiontech.github.io/dble-docs-cn/1.config_file/1.06_log4j2.xml.html
cacheservice.properties	dbale 的 cache 的总配置文件	不常用，需要时可自行参照文档配置	https://actiontech.github.io/dble-docs-cn/1.config_file/1.8_cache/1.8.1_cache_setting.html
ehcache.xml	dbale 的缓存实现配置文件	不常用，需要时可自行参照文档配置	https://actiontech.github.io/dble-docs-cn/1.config_file/1.8_cache/1.8.2_ehcache_setting.html
\$DBLE_HOME/lib 目录下 jar 包	自定义拆分算法文件	不常用，需要时可自行参照文档配置	https://actiontech.github.io/dble-docs-cn/1.config_file/1.09_dble_route_function_spec.html
dble_alert.properties	自定义告警配置文件	不常用，需要时可自行参照文档配置	https://actiontech.github.io/dble-docs-cn/1.config_file/1.11_customized_alert.html

二、下面通过列举几个新粉经常遇到的 dble 启动失败的例子，来说明 dble 在加载配置文件时做了哪些检测

1. 首先以以下 bootstrap.cnf 配置为例启动 dble (其他配置文件请按需配置，此处省略):

```
[root@db1e-1 db1e]# cat conf/bootstrap.cnf
#encoding=UTF-8
-agentlib:jdwp=transport=dt_socket,server=y,address=8088,suspend=n
-server
-XX:+AggressiveOpts
-Dfile.encoding=UTF-8
-Dcom.sun.management.jmxremote
-Dcom.sun.management.jmxremote.port=22
-Dcom.sun.management.jmxremote.authenticate=false
-Dcom.sun.management.jmxremote.ssl=false
-Dcom.sun.management.jmxremote.host=127.0.0.1
-Xmx1G
-Xms1G
-XX:MaxDirectMemorySize=1G
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/tmp/java_error_in_idea.hprof
# base config
-DhomePath=.
-DinstanceName=1
# valid for sequenceHandlerType=2 or 3
-DinstanceId=1
-DserverId=server_1

# processor
-DNIOFrontRW=1
-DfrontWorker=1
```

进入 dble 安装的根目录，执行 dble 启动命令 `bin/db1e start`，然后执行 `bin/db1e status` 查看 dble 的状态，发现启动失败了：

```
[root@db1e-1 db1e]# bin/db1e status
db1e-server is not running.
```

进入 `wrapper.log` 中查看，发现报错如下：

```
--> Wrapper Started as Daemon
Java Service Wrapper Community Edition 64-bit 3.5.40
Copyright (C) 1999-2019 Tanuki Software, Ltd. All Rights Reserved.
http://wrapper.tanukisoftware.com

Launching a JVM...
Listening for transport dt_socket at address: 8088
错误：代理抛出异常错误：java.rmi.server.ExportException: Port already in use: 22; nested exception is:
    java.net.BindException: Address already in use (Bind failed)
JVM exited while loading the application.
JVM process is gone.
Launching a JVM...
Listening for transport dt_socket at address: 8088
错误：代理抛出异常错误：java.rmi.server.ExportException: Port already in use: 22; nested exception is:
    java.net.BindException: Address already in use (Bind failed)
JVM exited while loading the application.
JVM process is gone.
Launching a JVM...
Listening for transport dt_socket at address: 8088
错误：代理抛出异常错误：java.rmi.server.ExportException: Port already in use: 22; nested exception is:
    java.net.BindException: Address already in use (Bind failed)
JVM exited while loading the application.
JVM process is gone.
Launching a JVM...
Listening for transport dt_socket at address: 8088
错误：代理抛出异常错误：java.rmi.server.ExportException: Port already in use: 22; nested exception is:
    java.net.BindException: Address already in use (Bind failed)
JVM exited while loading the application.
JVM process is gone.
Launching a JVM...
Listening for transport dt_socket at address: 8088
错误：代理抛出异常错误：java.rmi.server.ExportException: Port already in use: 22; nested exception is:
    java.net.BindException: Address already in use (Bind failed)
JVM exited while loading the application.
There were 5 failed launches in a row, each lasting less than 300 seconds. Giving up.
There may be a configuration problem: please check the logs.
<-- Wrapper Stopped
```

从日志中可以看出，报错是由于 22 端口被占用。这是由于 dble 在启动的时候会初始化一个虚拟机，从 `bootstrap.cnf` 读取 JVM 参数。dble 启动过程中会占用三个端口：业务端口（默认 8066），管理端口 398

(默认 9066)，jvm 对外提供 jmx 服务端口，所以配置的时候这三个端口不能相同，且不能被其他程序占用。具体参考：<https://actiontech.github.io/dble-docs-cn/A.Faq/ErrorCode/Content/PortAlreadyInUse1984.html>。所以将端口改为未被占用的端口即可启动成功。

2. 再以以下 `bootstrap.cnf` 配置启动(其他配置文件请按需配置，此处省略):

```
[root@db1e-1 dble]# cat conf/bootstrap.cnf
#encoding=UTF-8
-agentlib:jdwp=transport=dt_socket,server=y,address=8088,suspend=n
-server
-XX:+AggressiveOpts
-Dfile.encoding=UTF-8
-Dcom.sun.management.jmxremote
-Dcom.sun.management.jmxremote.port=1987
-Dcom.sun.management.jmxremote.authenticate=false
-Dcom.sun.management.jmxremote.ssl=false
-Dcom.sun.management.jmxremote.host=127.0.0.1
-Xmx1G
-Xms1G
-XX:MaxDirectMemorySize=1G
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/tmp/java_error_in_idea.hprof
# base config
-DhomePath=.
-DinstanceName=1
# valid for sequenceHandlerType=2 or 3
-DinstanceId=1
-DserverId=server_1

# processor
-D NIOFrontRW=1
-DfrontWorkers=1
-DsqlExecuteTimeout=30s
-DfakeMySQLVersion=5.7
```

发现 dble 启动报错，报错信息如下：

```
--> Wrapper Started as Daemon
Java Service Wrapper Community Edition 64-bit 3.5.40
Copyright (C) 1999-2019 Tanuki Software, Ltd. All Rights Reserved.
http://wrapper.tanukisoftware.com

The value 'NIOFrontRW=1' is not a valid argument to the JVM. Skipping.
Launching a JVM...
Listening for transport dt_socket at address: 8088
WrapperManager: Initializing...
property [ sqlExecuteTimeout ] '30s' data type should be long
These properties in bootstrap.cnf or bootstrap.dynamic.cnf are not recognized: ,frontWorkers
The specified MySQL Version (5.7) is not valid, the version should look like 'x.y.z'.
<-- Wrapper Stopped
```

从日志中可以看出，此处有 4 个错误：

- `-D` 与参数名“`NIOFrontRW`”之间有个空格，这不符合 JSW 对配置文件的要求，也就是不能有空格，具体请参见 <https://wrapper.tanukisoftware.com/doc/english/prop-app-parameter-n.html#file>；
- `sqlExecuteTimeout` 的属性值类型错误，此处应该是 `long` 类型，错配成了字符类型，将“`30s`”改成“`30`”即可；
- 误将“`frontWorker`”写成了“`frontWorkers`”。注意，`bootstrap.cnf` 文件里的属性名要严格按照文档里的写法，大小写也需要严格一致，否则报错；
- `fakeMySQLVersion` 的值不正确，将“`5.7`”改写成“`5.7.26`”即可。

3. 针对 `xml` 文件，`dble` 启动时同样也会对配置文件格式、属性名、属性值、属性类型等做校验，此处不再赘述。另外，针对 `db.xml`，`dble` 在启动时还会对后端 `MySQL` 节点的可连接性做检测，还会获取所有后端 `MySQL` 节点的版本号和大小写敏感参数值。

`dble` 用以下 `db.xml` 配置启动(其他配置文件请按需配置，此处省略):

```
<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE dbml:db SYSTEM "db.dtd">
<dbml:db xmlns:dbml="http://dbml.cloud/">
  <dbGroup rwSplitMode="0" name="ha_group1" delayThreshold="100">
    <heartbeat>select user()</heartbeat>
    <dbInstance name="hostM1" password="111111" url="172.100.9.5:3306" user="test" maxCon="1000" minCon="10" primary="true">
    </dbInstance>
  </dbGroup>
  <dbGroup rwSplitMode="0" name="ha_group2" delayThreshold="100">
    <heartbeat>select user()</heartbeat>
    <dbInstance name="hostM2" password="111111" url="172.100.9.6:3306" user="test" maxCon="1000" minCon="10" primary="true">
    </dbInstance>
  </dbGroup>
</dbml:db>
```

发现 dble 启动失败，检查 wrapper.log 日志，发现如下错误：

```
--> Wrapper Started as Daemon
Java Service Wrapper Community Edition 64-bit 3.5.40
Copyright (C) 1999-2019 Tanuki Software, Ltd. All Rights Reserved.
http://wrapper.tanukisoftware.com

Launching a JVM...
Listening for transport dt_socket at address: 8088
WrapperManager: Initializing...
java.io.IOException: Can't get variables from all dbGroups
    at com.actiontech.dble.DbleServer.pullVarAndMeta(DbleServer.java:491)
    at com.actiontech.dble.DbleServer.startup(DbleServer.java:249)
    at com.actiontech.dble.DbleStartup.initClusterAndServerStart(DbleStartup.java:62)
    at com.actiontech.dble.DbleStartup.main(DbleStartup.java:41)
    at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
    at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
    at java.lang.reflect.Method.invoke(Method.java:498)
    at org.tanukisoftware.wrapper.WrapperSimpleApp.run(WrapperSimpleApp.java:349)
    at java.lang.Thread.run(Thread.java:745)
Server execute ShutdownHook.
You use OuterHa or Cluster, no need to clean up ha process
2022-07-11 14:51:10.210 0-NIOFrontRW WARN Unable to register Log4j shutdown hook because JVM is shutting down. Using SimpleLogger
<-- Wrapper Stopped
```

这个错误表示后端 MySQL 节点连接检测失败。针对这种报错，一般常见的排查手法就是用 MySQL 客户端按照配置里的连接参数登录后端 MySQL 节点，如下：

EMPOK 5003 (H4000): can't connect to M207 server on 125.100.0.2. (111)
 m207: [M9LWUD] n2ud 9 b922m01q on the command line interface can be traced.
 [L00f@qprj6-1 qprj6]# m207 -b111111 -n1e2f -b330e -M125.100.0.2

此处是由于连接端口配置错误，将数据库的连接端口“3306”修改成“3307”即可启动成功。这种报错还有可能是用户和密码不对，也有可能是用户没有权限，具体可能原因及解决办法请对照参考：

<https://actiontech.github.io/dble-docs->

cn/A.Faq/ErrorCode/Content/Can%27tGetVariablesFromShardingNode.html。

4. dble 用以下 db.xml 配置启动(其他配置文件请按需配置, 此处省略):

```
root@db1e-1 db1e:~# cat conf/db.xml
<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE db1e:db SYSTEM "db.dtd">
<db1e:db xmlns:db1e="http://db1e.cloud/">
  <dbGroup rwSplitMode="0" name="ha_group1" delayThreshold="100">
    <heartbeat>select user()</heartbeat>
    <dbInstance name="hostM1" password="111111" url="172.100.9.5:3307" user="test" maxCon="1000" minCon="10" primary="true">
    </dbInstance>
  </dbGroup>

  <dbGroup rwSplitMode="0" name="ha_group2" delayThreshold="100">
    <heartbeat>select user()</heartbeat>
    <dbInstance name="hostM2" password="111111" url="172.100.9.6:3307" user="test" maxCon="1000" minCon="10" primary="true">
    </dbInstance>
  </dbGroup>
</db1e:db>
```

dbie 启动报错，wrapper.log 中报错如下：


```

--> Wrapper Started as Daemon
Java Service Wrapper Community Edition 64-bit 3.5.40
Copyright (C) 1999-2019 Tanuki Software, Ltd. All Rights Reserved.
http://wrapper.tanukisoftware.com

Launching a JVM...
Listening for transport dt_socket at address: 8088
WrapperManager: Initializing...
java.io.IOException: The values of lower_case_table_names for backend MySQLs are different. These MySQL's value is not 0 :ha_group1:hostM1. And these MySQL's value is 0 :ha_group2:hostM2.
    at com.actiontech.dble.config.util.ConfigUtil.getAndSyncKeyVariables(ConfigUtil.java:147)
    at com.actiontech.dble.config.ServerConfig.getAndSyncKeyVariables(ServerConfig.java:159)
    at com.actiontech.dble.DbleServer.startup(DbleServer.java:229)
    at com.actiontech.dble.DbleStartup.initClusterAndServerStart(DbleStartup.java:62)
    at com.actiontech.dble.DbleStartup.main(DbleStartup.java:41)
    at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
    at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
    at java.lang.reflect.Method.invoke(Method.java:498)
    at org.tanukisoftware.wrapper.WrapperSimpleApp.run(WrapperSimpleApp.java:349)
    at java.lang.Thread.run(Thread.java:745)
Server execute ShutdownHook.
You use OuterHa or Cluster, no need to clean up ha process
<-- Wrapper Stopped

```

以上报错说明后端 MySQL 节点大小写敏感参数值不一致。dble 启动时，会根据 db.xml 的配置，获取所有节点的大小写敏感参数值，dble 要求此参数值在所有 MySQL 节点中必须保持一致，否则报错。将 hostM1 中的大小写改成敏感后 dble 即可启动成功。

5. dble 用以下 bootstrap.cnf 配置启动(其他配置文件请按需配置，此处省略):

```

[root@dble-1 dble]# cat conf/bootstrap.cnf
#encoding=UTF-8
-agentlib:jdwp=transport=dt_socket,server=y,address=8088,suspend=n
-server
-XX:+AggressiveOpts
-Dfile.encoding=UTF-8
-Dcom.sun.management.jmxremote
-Dcom.sun.management.jmxremote.port=1987
-Dcom.sun.management.jmxremote.authenticate=false
-Dcom.sun.management.jmxremote.ssl=false
-Dcom.sun.management.jmxremote.host=127.0.0.1
-Xmx1G
-Xms1G
-XX:MaxDirectMemorySize=1G
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/tmp/java_error_in_idea.hprof
# base config
-DhomePath=
-DinstanceName=1
# valid for sequenceHandlerType=2 or 3
-DinstanceId=1
-DserverId=server_1

# processor
-DNIOFrontRW=1
-DfrontWorker=1
-DfakeMySQLVersion=8.0.16

```

发现 dble 启动失败，且 wrapper.log 里报如下错误：

```

--> Wrapper Started as Daemon
Java Service Wrapper Community Edition 64-bit 3.5.40
Copyright (C) 1999-2019 Tanuki Software, Ltd. All Rights Reserved.
http://wrapper.tanukisoftware.com

Launching a JVM...
Listening for transport dt_socket at address: 8088
WrapperManager: Initializing...
com.actiontech.dble.config.util.ConfigException: the dble version[8.0.16] cannot be higher than the minimum version of the backend mysql node,pls check the backend mysql node.
    at com.actiontech.dble.config.util.ConfigUtil.getAndSyncKeyVariables(ConfigUtil.java:126)
    at com.actiontech.dble.config.ServerConfig.getAndSyncKeyVariables(ServerConfig.java:159)
    at com.actiontech.dble.DbleServer.startup(DbleServer.java:229)
    at com.actiontech.dble.DbleStartup.initClusterAndServerStart(DbleStartup.java:62)
    at com.actiontech.dble.DbleStartup.main(DbleStartup.java:41)
    at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
    at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
    at java.lang.reflect.Method.invoke(Method.java:498)
    at org.tanukisoftware.wrapper.WrapperSimpleApp.run(WrapperSimpleApp.java:349)
    at java.lang.Thread.run(Thread.java:745)
Server execute ShutdownHook.
You use OuterHa or Cluster, no need to clean up ha process
<-- Wrapper Stopped

```

错误提示 dble 里配置的 version 值高于后端节点的版本值。这是由于 dble 在启动时会获取 db.xml 里配置的所有节点的 MySQL 版本，然后和 bootstrap.cnf 里配置的 fakeMySQLVersion 进行比较，要求配置的 fakeMySQLVersion 的值不能高于后端 MySQL 节点的最低版本号，否则报错。

除了以上列举的例子外，dble 还会对其他配置文件做各自特殊的校验，比如分片算法正确性，用户类型和实例组的正确配对，maxPacketSize 同步等等，由于篇幅原因，本文不再一一举例，因为一般 dble 启动失败都可以通过 wrapper.log 里的错误提示得到解决。

简单总结来说就是，dble 在启动的时候会对所有配置文件进行合法性校验，包括配置文件的格式，属性名，属性类型、属性值等。还会对后端 MySQL 节点做可连性测试，大小写一致性检测，fakeMySQLVersion 合法性检测，maxPacketSize 同步等。还会校验各配置文件之间是否正确配置等等。

4.20 利用 TiDB DM 将数据从 MySQL 迁移至 DBLE

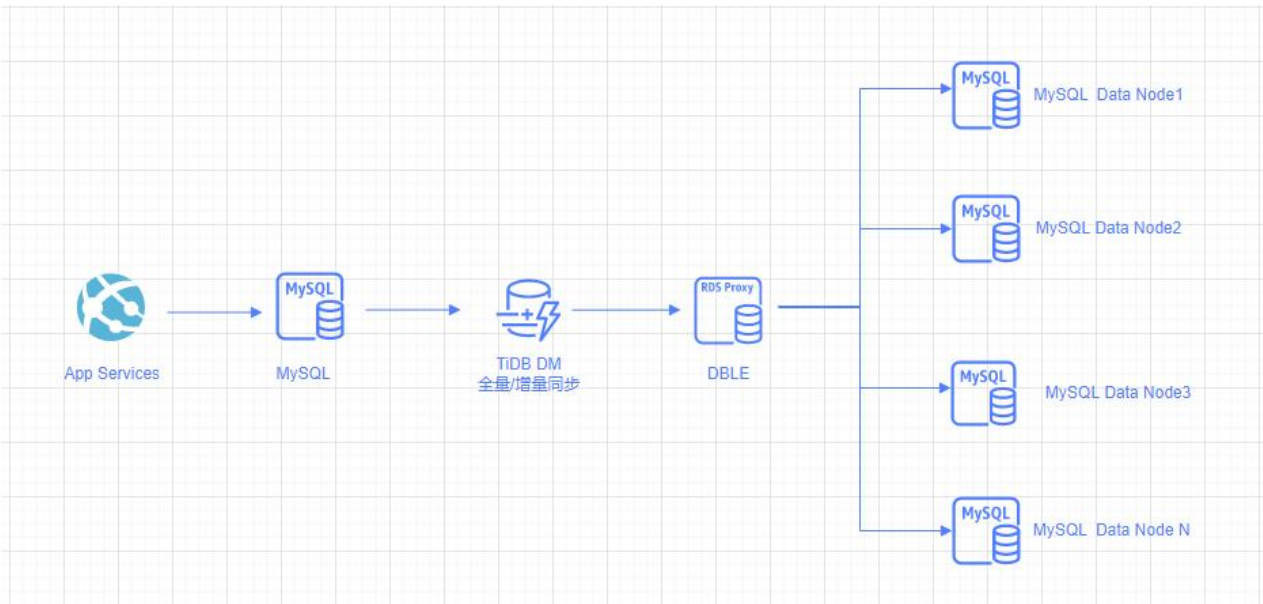
作者：沈光宇

一、背景

某客户 MySQL 实例中数据较大(上 T 的数据量)，需要将 MySQL 中数据全量/增量快速迁移至 DBLE 。TiDB DM 支持多线程数据导出导入及增量同步，因此选择了 TiDB DM 为作数据迁移工具。本文以此案例为背景，介绍使用 TiDB DM 将数据从 MySQL 迁移至 DBLE 的方法及遇到的一些问题。

二、数据迁移示意图及服务器环境

(1) 数据迁移示意图如下：



(2) 服务器环境如下：

服务器 IP	角色	端口	版本	备注
10.186.65.83	dm master	8261/8291	V2.0.7	tiup 安装节点
10.186.65.118	dm worker	8262	V2.0.7	
10.186.65.14	SRC MySQL	4490	MySQL 8.0.18	
10.186.65.4	DBLE	8066	DBLE 3.21.10.6	
10.186.65.61	DBLE datanode	4408	MySQL 8.0.25	
10.186.65.65	DBLE datanode	4408	MySQL 8.0.25	

三、安装 TiDB DM

(1) 准备环境及安装 tiup

```
#分别在 dm master 和 worker 节点主机创建 tidb 用户,密码为 dmadmin,并配置 sudo
shell> useradd tidb
shell> echo "dmadmin" | passwd tidb --stdin
shell> echo "tidb ALL=(ALL) NOPASSWD: ALL" > /etc/sudoers.d/tidb

#切换至 tidb 用户,生成密钥, 并做 ssh 信任,dm 集群内机器都需要做 ssh 信任
shell> su - tidb
shell> ssh-keygen -t rsa
shell> ssh-copy-id -i ~/.ssh/id_rsa.pub tidb@10.186.65.118
shell> ssh-copy-id -i ~/.ssh/id_rsa.pub tidb@10.186.65.83

#安装 tiup,在 tidb 用户下
shell> curl --proto 'https' --tlsv1.2 -sSf https://tiup-
mirrors.pingcap.com/install.sh | sh
shell> source .bash_profile
shell> which tiup
~/tiup/bin/tiup

#安装 dmctl
shell> tiup install dm dmctl:v2.0.7
component dm version v1.10.2 is already installed
download https://tiup-mirrors.pingcap.com/dmctl-v2.0.7-linux-
amd64.tar.gz 26.92 MiB / 26.92 MiB 100.00% 10.01 MiB/s
```

(2) 安装 dm-cluster

```
#生成配置模版
shell> tiup dm template > dm_topology.yaml
#修改后配置文件如下:
shell> cat dm_topology.yaml |grep -v ^[#-] |grep -v ^$
global:
  user: "tidb"
  ssh_port: 22
  deploy_dir: "/home/tidb/dm/deploy"
  data_dir: "/home/tidb/dm/data"
master_servers:
  - host: 10.186.65.83
worker_servers:
  - host: 10.186.65.118
monitoring_servers:
  - host: 10.186.65.83
grafana_servers:
```

```

- host: 10.186.65.83
alertmanager_servers:
- host: 10.186.65.83

#安装、并启动 DM 集群
shell> tiup dm deploy dm-test v2.0.7 ./dm_topology.yaml --user tidb -
i /home/tidb/.ssh/id_rsa
shell> tiup dm start dm-test
#查看 DM 集群
[tidb@10_186_65_83 ~]$ tiup dm display dm-test
tiup is checking updates for component dm ...
Starting component `dm`: /home/tidb/.tiup/components/dm/v1.10.2/tiup-dm display dm-test
Cluster type:      dm
Cluster name:      dm-test
Cluster version:   v2.0.7
Deploy user:      tidb
SSH type:         builtin
Grafana URL:      http://10.186.65.83:3000
ID                Role          Host          Ports         OS/Arch        Status      Da
ta Dir           Deploy Dir
--              ----
-----
10.186.65.83:9093  alertmanager  10.186.65.83  9093/9094    linux/x86_64  Up          /h
ome/tidb/dm/data/alertmanager-9093  /home/tidb/dm/deploy/alertmanager-9093
10.186.65.83:8261  dm-
master          10.186.65.83  8261/8291    linux/x86_64  Healthy|L    /home/tidb/dm/data/dm-
master-8261      /home/tidb/dm/deploy/dm-master-8261
10.186.65.118:8262  dm-
worker          10.186.65.118  8262        linux/x86_64  Free         /home/tidb/dm/data/dm-
worker-8262      /home/tidb/dm/deploy/dm-worker-8262
10.186.65.83:3000  grafana       10.186.65.83  3000         linux/x86_64  Up          -
/home/tidb/dm/deploy/grafana-3000
10.186.65.83:9090  prometheus    10.186.65.83  9090         linux/x86_64  Up          /h
ome/tidb/dm/data/prometheus-9090    /home/tidb/dm/deploy/prometheus-9090
Total nodes: 5

```

四、DBLE 端配置

(1) db.xml

```

<?xml version="1.0"?>
<dbledb xmlns:dble="http://dble.cloud/">
  <dbGroup name="dbGroup1" rwSplitMode="0" delayThreshold="-1">
    <heartbeat>select 1</heartbeat>
    <dbInstance name="host_1" url="10.186.65.61:4408" user="sgy" password="QfnHoIeI

```



```
YL7ZT+EdJMYNuiLw6glbx2hEyrqxK+uxFPU8vja8vZHCLEKvKB0t1vOJdXCCgW7wNidJaTYWORDaUg==" maxCon="2000" minCon="50" primary="true" usingDecrypt="true"></dbInstance>
</dbGroup>
<dbGroup name="dbGroup2" rwSplitMode="0" delayThreshold="-1">
    <heartbeat>select 1</heartbeat>
    <dbInstance name="host_2" url="10.186.65.65:4408" user="sgy" password="d+rG/82+4h21ARS8D6Gu5MIFQ2UBC0h+ZS0EI7bvWAtTJUBsKuFY8+AVZtW1pZk+qtISoI2WpVt29Z9eqMRP5A==" maxCon="2000" minCon="50" primary="true" usingDecrypt="true"></dbInstance>
</dbGroup>
</dble:db>
#注：用户密码为加密后的字符串
```

(2) sharding.xml

```
<?xml version="1.0"?>
<dble:sharding xmlns:dble="http://dble.cloud/" >
    <schema name="dm_meta" sqlMaxLimit="-1">
        <singleTable name="mysql_sync_to_dble_loader_checkpoint" shardingNode="dm_meta"
        />
        <singleTable name="mysql_sync_to_dble_syncer_checkpoint" shardingNode="dm_meta"
        />
    </schema>
    <schema name="sbtest" sqlMaxLimit="-1">
        <shardingTable name="sbtest1" shardingNode="dn1,dn2,dn3,dn4" sqlMaxLimit="-1" shardingColumn="id" function="pro_func_jumpHash"></shardingTable>
        <shardingTable name="sbtest2" shardingNode="dn1,dn2,dn3,dn4" sqlMaxLimit="-1" shardingColumn="id" function="pro_func_jumpHash"></shardingTable>
        <shardingTable name="t1" shardingNode="dn1,dn2,dn3,dn4" sqlMaxLimit="-1" shardingColumn="c1" function="pro_func_jumpHash"></shardingTable>
        <shardingTable name="t2" shardingNode="dn1,dn2,dn3,dn4" sqlMaxLimit="-1" shardingColumn="c1" function="pro_func_jumpHash"></shardingTable>
    </schema>

    <shardingNode name="dn1" dbGroup="dbGroup1" database="dh_dn1"></shardingNode>
    <shardingNode name="dn2" dbGroup="dbGroup1" database="dh_dn2"></shardingNode>
    <shardingNode name="dn3" dbGroup="dbGroup2" database="dh_dn3"></shardingNode>
    <shardingNode name="dn4" dbGroup="dbGroup2" database="dh_dn4"></shardingNode>
    <shardingNode name="dm_meta" dbGroup="dbGroup1" database="dm_meta"/>

    <function name="pro_func_jumpHash" class="jumpStringHash">
        <property name="partitionCount">4</property>
        <property name="hashSlice">0:0</property>
    </function>
</dble:sharding>
```

注：需要提前配置好 DM 同步时保存数据的 schema 及两张表，表名分别是

dm 同步任务名 + _loader_checkpoint 即表 mysql_sync_to_dble_loader_checkpoint

dm 同步任务名 + _syncer_checkpoint 即表 mysql_sync_to_dble_syncer_checkpoint

(3) user.xml

```
<?xml version="1.0"?>
<dble:user xmlns:dble="http://dble.cloud/">
  <managerUser name="root" password="RYQdYYnzbCZlDuhV4LhJxFTM8kbU3d0y183LU+FgRWzNscvv
vFPcdmeEKmdswMyzIaA+k0bcozliHEYA1T0AjA==" usingDecrypt="true"></managerUser>
  <shardingUser name="sz" schemas="sbtest,dm_meta" password="jyaXmGFU+mdTgbUZIVbdEcVw
ItLPI+Yjxdq4wk0zhAYRB29WGZJd0/PkTJh3ky/v4E2yYoqgUzJXPzPRPiQF0Q==" usingDecrypt="true" r
eadOnly="false" maxCon="1000"></shardingUser>
</dble:user>
```

#注：用户密码为加密后的字符串

(4) 登陆 DBLE 管理端执行创建物理数据库命令

```
shell> /data/mysql/base/5.7.36/bin/mysql -uroot -padmin -h 10.186.65.4 -P 9066
dble> create database @@shardingNode = 'dn$1-4';
Query OK, 1 row affected (0.06 sec)
```

```
dble> create database @@shardingNode = 'dm_meta';
Query OK, 1 row affected (0.01 sec)
```

五、在源端 MySQL 压测创建测试数据

#执行 prepare，初始化 100W 行数据

```
/usr/share/sysbench/oltp_insert.lua --mysql-db=sbtest --mysql-host=10.186.65.14 --
mysql-port=4490 --mysql-user=sgy --mysql-password=admin --tables=2 --table-
size=1000000 --threads=16 --time=60 --report-interval=1 --max-requests=0 --
percentile=95 --mysql-ignore-errors=1062 prepare
sysbench 1.0.17 (using system LuaJIT 2.0.4)
```

Initializing worker threads...

Creating table 'sbtest1'...

Creating table 'sbtest2'...

Inserting 1000000 records into 'sbtest2'

Inserting 1000000 records into 'sbtest1'

Creating a secondary index on 'sbtest2'...

Creating a secondary index on 'sbtest1'...

#利用 sysbench 不断写入数据

```
shell> /usr/share/sysbench/oltp_read_write.lua --mysql-db=sbtest --mysql-
host=10.186.65.14 --mysql-port=4490 --mysql-user=sgy --mysql-password=admin --
```

```
tables=2 --table-size=2000000 --threads=16 --time=6000 --report-interval=1 --max-requests=0 --percentile=95 --mysql-ignore-errors=1062 run
sysbench 1.0.17 (using system LuaJIT 2.0.4)
```

Running the test with following options:

Number of threads: 16

Report intermediate results every 1 second(s)

Initializing random number generator from current time

Initializing worker threads...

Threads started!

```
[ 1s ] thds: 16 tps: 585.44 qps: 11916.49 (r/w/o: 8360.33/1282.63/2273.53) lat (ms,95%):
55.82 err/s: 0.00 reconn/s: 0.00
[ 2s ] thds: 16 tps: 744.18 qps: 14819.02 (r/w/o: 10379.16/1609.36/2830.50) lat (ms,95%):
41.85 err/s: 0.00 reconn/s: 0.00
[ 3s ] thds: 16 tps: 605.11 qps: 12144.20 (r/w/o: 8500.54/1321.24/2322.42) lat (ms,95%):
47.47 err/s: 0.00 reconn/s: 0.00
[ 4s ] thds: 16 tps: 773.91 qps: 15558.16 (r/w/o: 10881.71/1737.79/2938.65) lat (ms,95%):
41.10 err/s: 0.00 reconn/s: 0.00
```

六、启动数据同步任务

(1) 配置数据源

#使用 dmctl 工具生成密码加密字符串,每次执行都会产生不同的加密字符串

```
shell> tiup dmctl encrypt 'admin'
dmXgktpuF18RP1mDN/B2UkA6pDN9PQ==
```

#mysql 数据库配置文件如下

```
shell> cat mysql_source_14_4490.yaml
source-id: "mysql_source_14_4490"
from:
  host: "10.186.65.14"
  port: 4490
  user: "sgy"
  password: "dmXgktpuF18RP1mDN/B2UkA6pDN9PQ=="
```

#创建数据源

```
shell> tiup dmctl --master-addr 10.186.65.83:8261 operate-
source create ./mysql_source_14_4490.yaml
Starting component `dmctl`: /home/tidb/.tiup/components/dmctl/v2.0.7/dmctl/dmctl --
master-addr 10.186.65.83:8261 operate-source create ./mysql_source_14_4490.yaml
```

```
{
  "result": true,
  "msg": "",
  "sources": [
    {
      "result": true,
      "msg": "",
      "source": "mysql_source_14_4490",
      "worker": "dm-10.186.65.118-8262"
    }
  ]
}
```

#查看数据源

```
shell> tiup dmctl --master-addr 10.186.65.83:8261 operate-source show
```

```
{
  "result": true,
  "msg": "",
  "sources": [
    {
      "result": true,
      "msg": "",
      "source": "mysql_source_14_4490",
      "worker": "dm-10.186.65.118-8262"
    }
  ]
}
```

(2) 配置同步任务

#同步配置文件如下

```
shell> cat mysql_sync_to_dble.yaml
```

```
name: "mysql_sync_to_dble"      #同步任务名
task-mode: "all"                #全量 + Binlog 实时同步
clean-dump-file: true
timezone: "Asia/Shanghai"
ignore-checking-items: ["auto_increment_ID"]
target-database:
  host: "10.186.65.4"           #DBLE 主机 IP 地址
  port: 8066                    #DBLE 流量端口
  user: "sz"                    #连接 DBLE 的用户
  password: "KRfSNtdxe63tehpnCYoCz0ABdUGivg==" #连接 DBLE 用户密码, 用
tiup dmctl encrypt 'password'生成
```

```
mysql-instances:
```

```

-
source-id: "mysql_source_14_4490"
block-allow-list: "global"
mydumper-config-name: "global"
loader-config-name: "global"
syncer-config-name: "global"

block-allow-list:
  global:
    do-dbs: ["sbtest"]          #只迁移 sbtest 库

mydumpers:
  global:
    extra-args: "-B sbtest"    #只 dump sbtest 库

loaders:
  global:
    pool-size: 8

syncers:
  global:
    worker-count: 8

#更详细配置说明参考官方文档:
https://docs.pingcap.com/zh/tidb-data-migration/v2.0/task-configuration-file-
full#%E5%AE%8C%E6%95%B4%E9%85%8D%E7%BD%AE%E6%96%87%E4%BB%B6%E7%A4%BA%E4%BE%8B

#检查配置文件
shell> tiup dmctl:v2.0.7 --master-addr=10.186.65.83:8261 check-
task ./mysql_sync_to_dble.yaml
Starting component `dmctl`: /home/tidb/.tiup/components/dmctl/v2.0.7/dmctl/dmctl --
master-addr=10.186.65.83:8261 check-task ./mysql_sync_to_dble.yaml
{
  "result": true,
  "msg": "check pass!!!"
}

#启动同步任务
shell> tiup dmctl:v2.0.7 --master-addr=10.186.65.83:8261 start-
task ./mysql_sync_to_dble.yaml
Starting component `dmctl`: /home/tidb/.tiup/components/dmctl/v2.0.7/dmctl/dmctl --
master-addr=10.186.65.83:8261 start-task ./mysql_sync_to_dble.yaml
{
  "result": true,

```

```

    "msg": "",
    "sources": [
      {
        "result": true,
        "msg": "",
        "source": "mysql_source_14_4490",
        "worker": "dm-10.186.65.118-8262"
      }
    ]
  }
}

```

```

shell> head /home/tidb/dm/deploy/dm-worker-
8262/dumped_data.mysql_sync_to_dble/sbtest.sbtest1.00000000000000000000.sql
/*!40101 SET NAMES binary*/;
INSERT INTO `sbtest1` (`id`,`k`,`c`,`pad`) VALUES
(1,498670,'31451373586-15688153734-79729593694-96509299839-83724898275-86711833539-
78981337422-35049690573-51724173961-87474696253','98996621624-36689827414-04092488557-
09587706818-65008859162'),
(2,497778,'21472970079-70972780322-70018558993-71769650003-09270326047-32417012031-
10768856803-14235120402-93989080412-18690312264','04776826683-45880822084-77922711547-
29057964468-76514263618'),
(3,498956,'49376827441-24903985029-56844662308-79012577859-40518387141-60588419212-
24399130405-42612257832-29494881732-71506024440','26843035807-96849339132-53943793991-
69741192222-48634174017'),
(4,518727,'85762858421-36258200885-10758669419-44272723583-12529521893-95630803635-
53907705724-07005352902-43001596772-53048338959','37979424284-37912826784-31868864947-
42903702727-96097885121'),
(5,502480,'24805466175-85245528617-94635882649-46305216925-28637832581-03224489581-
68883711727-95491561683-91969681472-12022277774','19288959552-55556468076-14192290426-
55457672510-18043372364'),
(6,500774,'52892836230-54177743992-01821871718-48412537487-30066596248-87215430797-
00375777469-64498831720-58542556455-90784765418','59487960480-08453890592-99628797439-
16757639138-29377916560'),
(7,501466,'85820931248-14475640036-11980694501-86588543167-31029306229-09626867980-
90685354565-02350460358-25863585366-53793794448','26081374730-86321700986-51212137094-
30635959762-03880194434'),
(8,497838,'81578049255-33453976301-67096870761-27658738403-30546242249-53677469854-
26594573136-34292002037-52736825353-99165193170','64289062455-51067794311-09919261228-
11533354367-07401173317'),

```

(3) 查看同步任务状态

```

shell> tiup dmctl:v2.0.7 --master-addr=10.186.65.83:8261 query-
status ./mysql_sync_to_dble.yaml
Starting component `dmctl`: /home/tidb/.tiup/components/dmctl/v2.0.7/dmctl/dmctl --

```

```

master-addr=10.186.65.83:8261 query-status ./mysql_sync_to_dble.yaml
{
  "result": true,
  "msg": "",
  "sources": [
    {
      "result": true,
      "msg": "",
      "sourceStatus": {
        "source": "mysql_source_14_4490",
        "worker": "dm-10.186.65.118-8262",
        "result": null,
        "relayStatus": null
      },
      "subTaskStatus": [
        {
          "name": "mysql_sync_to_dble",
          "stage": "Running",
          "unit": "Sync",
          "result": null,
          "unresolvedDDLlockID": "",
          "sync": {
            "totalEvents": "425355",
            "totalTps": "1203",
            "recentTps": "2386",
            "masterBinlog": "(mysql-bin.000027, 76114275)",
            "masterBinlogGtid": "c7827165-bf89-11ec-92e6-02000aba410e:1-5091258",
            "syncerBinlog": "(mysql-bin.000026, 229462770)",
            "syncerBinlogGtid": "",
            "blockingDDLs": [
            ],
            "unresolvedGroups": [
            ],
            "synced": false,
            "binlogType": "remote",
            "secondsBehindMaster": "38"
          }
        }
      ]
    }
  ]
}

```

七、验证数据

#暂停先前执行的 sysbench 程序，确保 dm 同步与源端无延迟时，对表作 count(*)

#在源端 MySQL，对表进行 count(*)

```
mysql> select count(*) from sbtest.sbtest1;
```

```
+-----+
```

```
| count(*) |
```

```
+-----+
```

```
| 1116471 |
```

```
+-----+
```

```
1 row in set (0.68 sec)
```

```
mysql> select count(*) from sbtest.sbtest2;
```

```
+-----+
```

```
| count(*) |
```

```
+-----+
```

```
| 1117020 |
```

```
+-----+
```

```
1 row in set (0.54 sec)
```

#通过 dble 对表进行 count(*)

```
dble> select count(*) from sbtest.sbtest1;
```

```
+-----+
```

```
| count(*) |
```

```
+-----+
```

```
| 1116471 |
```

```
+-----+
```

```
1 row in set (1.04 sec)
```

```
dble> select count(*) from sbtest.sbtest2;
```

```
+-----+
```

```
| count(*) |
```

```
+-----+
```

```
| 1117020 |
```

```
+-----+
```

```
1 row in set (1.58 sec)
```

#在源端数据进行更新操作

```
mysql> update sbtest.sbtest1 set c=uuid() where id=20;
```

```
Query OK, 1 row affected (0.01 sec)
```

```
Rows matched: 1 Changed: 1 Warnings: 0
```

```
mysql> select * from sbtest.sbtest1 where id=20;
```

```
+-----+
```



```
| id | k      | c                                     | pad |
+-----+-----+-----+-----+
| 20 | 501448 | 24649f0f-14d1-11ed-b4f2-02000aba410e | 91052688950-96415657187-00012408429-12357288330-41295735957 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

#通过 dble 查询数据

```
dble> select * from sbtest.sbtest1 where id=20;
```

```
| id | k      | c                                     | pad |
+-----+-----+-----+-----+
| 20 | 501448 | 24649f0f-14d1-11ed-b4f2-02000aba410e | 91052688950-96415657187-00012408429-12357288330-41295735957 |
+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

八、注意事项

- (1) TiDB DM 版本要求：TiDB DM v2.0.7
- (2) 在进行数据量导入时需要关闭 DBLE 慢查询日志

#由于数据同步在全量导入阶段会产生大量慢查询，需要在执行同步任务之前先关闭 DBLE 的慢查询日志

```
shell> mysql --prompt='dble>' -uroot -p -h 10.186.65.4 -P 9066
```

#查看慢查询日志状态，1 表示开启，0 表示关闭

```
dble> show @@slow_query_log;
```

```
+-----+
```

```
| @@slow_query_log |
```

```
+-----+
```

```
| 1 |
```

```
+-----+
```

```
1 row in set (0.00 sec)
```

#全量数据导入前，关闭 dble 慢查询日志

```
dble> disable @@slow_query_log;
```

```
Query OK, 1 row affected (0.01 sec)
```

```
disable slow_query_log success
```

#待全量数据导入完成后，开启 dble 慢查询日志

```
dble> enable @@slow_query_log;
```

```
Query OK, 1 row affected (0.01 sec)
```

```
enable slow_query_log success
```

- (3) 检查源端 MySQL 实例上是否有运行类似 pt-kill 的程序

#如果在源端 MySQL 实例上运行有 pt-kill 或类似的程序，DM 同步任务在执行全量数据 dump 阶段，导出线程可能会被程序 kill 掉，导致全备失败。dm-worker.log 日志如下：

```
[ERROR] [subtask.go:311] ["unit process error"] [subtask=oms-sync-
```

```
receiver] [unit=Dump] [{"error information"="{\"ErrCode\":32001,\"ErrClass\":\"dump-unit\",\"ErrScope\":\"internal\",\"ErrLevel\":\"high\",\"Message\":\"mydumper/dumpling runs with error, with output (may empty): \",\"RawCause\":\"invalid connection\"}"}]
```

解决方法：先将 pt-kill 实用工具停止，然后再重新运行 TiDM 同步任务，待全量数据 dump 完成后在开启 pt-kill。

(4) 迁移时如需对目标表表结构进行修改，如源表有建立分区而目标表通过分片后不需要建立分区、修改字符集(建议使用 utf8mb4)等，可先将表结构从源库导出，修改后导入 DBLE 或导入 DBLE 后在修改表结构，最后再开启 DM 同步任务。

(5) 源、目标表字段顺序必须要一致，否则可能会导致数据不一致，两字段同为 varchar 并且长度不相等时，同步任务并不会报错，但写入目标表的值可能会被截断，如下图：

```
mysql>desc t2;
+-----+-----+-----+-----+-----+-----+
| Field | Type      | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| c1     | int(11)   | NO   | PRI | NULL    | auto_increment |
| c2     | varchar(10) | YES  |     | NULL    |                |
| c3     | varchar(20) | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>insert into t2(c2,c3) values(repeat('a',10),repeat('b',20));
Query OK, 1 row affected (0.01 sec)

mysql>select * from t2;
+-----+-----+-----+
| c1 | c2          | c3                |
+-----+-----+-----+
| 2 | aaaaaaaaaa | bbbbbbbbbbbbbbbbbbbb |
+-----+-----+-----+
1 row in set (0.00 sec)
```

```
dble>desc t2;
+-----+-----+-----+-----+-----+-----+
| Field | Type      | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| c1     | int       | NO   | PRI | NULL    | auto_increment |
| c3     | varchar(10) | YES  |     | NULL    |                |
| c2     | varchar(20) | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

dble>select * from t2;
+-----+-----+-----+
| c1 | c3          | c2                |
+-----+-----+-----+
| 2 | bbbbbbbbbb | aaaaaaaaaa |
+-----+-----+-----+
1 row in set (0.00 sec)
```

目标端c3字段值已被截断

DM 任务启动后连接目标端时，会将 session 级别 sql_mode 设置成下面值：
sql_mode='IGNORE_SPACE,NO_AUTO_VALUE_ON_ZERO,NO_ENGINE_SUBSTITUTION,ALLOW_INVALID_DATES,ONLY_FULL_GROUP_BY';

4.21 从 dble 日志分析到 MySQL 源码学习

作者：袁琳铸

背景

在客户的生产环境中，dble.log 时常出现 no handler 日志。虽然没有影响客户业务的正常使用，但是需要调查下这个日志打印的原因是什么以及什么样的场景才会打印该日志。

分析

首先对 dble 和 mysql 的日志进行分析，看看能不能从日志中找到可用信息

dble 日志

日志中有 no handler 日志和 dble 收到 mysql 发送的 fin 包打印的 stream closed by peer 的日志，并无其他异常。

```
2022-09-02 03:25:23.450 INFO [0-backendWorker] (com.actiontech.dble.net.response.DefaultResponseHandler.closeNoHandler(DefaultResponseHandler.java:116)) -
    no handler bind in this service MySQLResponseService[isExecuting = false attachment = null autocommitSynced = true isolationSynced = true xaStatus = 0 isDDL = false complexQuery = false] with response handler [null] with rrs = [null] with connection BackendConnection[id = 342 host = 10.186.62.41 port = 3312 localPort = 51886 mysqlId = 677 db config = dbInstance[name=M1,disabled=false,maxCon=1000,minCon=10]
2022-09-02 03:25:23.450 INFO [0-backendWorker] (com.actiontech.dble.net.connection.BackendConnection.close(BackendConnection.java:125)) - connection id 342 mysqlId 677 close for reason no handler
2022-09-02 03:25:23.450 INFO [1-NIOBackendRW] (com.actiontech.dble.net.service.AbstractService.beforeInsertServiceTask(AbstractService.java:170)) -
    prepare close for conn.conn id 342,reason [stream closed by peer]
2022-09-02 03:25:23.450 INFO [0-backendWorker] (com.actiontech.dble.net.connection.AbstractConnection.closeImmediatelyInner(AbstractConnection.java:169)) -
    connection id close for reason [no handler] with connection BackendConnection[id = 342 host = 10.186.62.41 port = 3312 localPort = 51886 mysqlId = 677 db config = dbInstance[name=M1,disabled=false,maxCon=1000,minCon=10]
```

mysql 日志

存在连接超时而关闭连接的日志，并无其他异常

```
2022-09-02T03:25:23.375174Z 672 [Note] [MY-013730] [Server] 'wait_timeout' period of 20 seconds was exceeded for `root`@`%`. The idle time since last command was too long.
2022-09-02T03:25:23.375252Z 677 [Note] [MY-010914] [Server] Aborted connection 677 to db: 'unconnected' user: 'root' host: '10.186.62.148' (The client was disconnected by the server because of inactivity.).
```

dble 相关逻辑

下发语句

- dble 收到客户端发送的语句
- dble 从连接池中选择处理语句的连接

- 根据语句类型 dble 选择合适的 responseHandler (responseHandler 用来处理 mysql 发送的报文, 空闲连接的 responseHandler 为 null)
- 下发语句到 mysql

处理 mysql 报文

- dble 接收 mysql 发送的报文
- dble 的 connection 使用下发语句时设置的 responseHandler 处理该报文 (找到 responseHandler 的情况下这里结束)
- dble 的 connection 无法找到 responseHandler, 无法处理该报文, 打印 no handler 日志

后端连接 close 处理

- dble 收到 fin 包或者检测到该连接存在异常
- 打印连接 close 相关信息 (close 原因, 被关闭的连接的信息)
- 清空连接绑定的内容 (比如 responseHandler, service 等信息)
- 设置该连接不可用
- 从连接池中移除该连接

理论情况下只有 dble 执行语句的后端连接进行 close 处理, dble 依旧在使用这个被 close 的后端连接处理 mysql 发送的数据才会出现 no handler 日志, 从日志中可以发现 dble 先处理 no handler 日志关闭连接, 然后才收到了 mysql 发送的 fin 包, 这与 dble 的逻辑不符 (不应该先打印 no handler 日志), 这是需要排查的疑惑点。然后开始翻阅 dble 的源码看看能不能用肉眼 debug 出一个符合日志打印的场景和对日志中的疑惑点进行解释, 经过翻来覆去的 review 没有发现可能场景也无法解释 dble 日志中的现象。

这时候可以换个方式, 借助 arthas 去客户环境中看看打印日志时 dble 的调用链和当时 dble 后端 connection 的信息, 然后在尝试分析问题。

arthas 分析

- arthas 命令

```
stack com.actiontech.dble.net.response.DefaultResponseHandler closeNoHandler -
n 1000 >> closeNoHandler_stack.log
watch com.actiontech.dble.net.response.DefaultResponseHandler closeNoHandler '{params,returnObj,throwExp,target}' -n 1000 -x 3 -b >> closeNoHandler.log
```

- 从 closeNoHandler_stack.log 可以看到 dble 收到了 mysql 传过来的 error 报文, 调用了 error 方法, 然后再调用 closeNoHandler 方法打印了 no handler 日志

```
@com.actiontech.dble.net.response.DefaultResponseHandler.closeNoHandler()
  at com.actiontech.dble.net.response.DefaultResponseHandler.error(DefaultResponseHandler.java:73)
  at com.actiontech.dble.services.BackendService.handleInnerData(BackendService.java:209)
  at com.actiontech.dble.net.service.AbstractService.consumeSingleTask(AbstractService.java:189)
  at com.actiontech.dble.services.BackendService.handleTaskQueue(BackendService.java:173)
  at com.actiontech.dble.services.BackendService.access$300(BackendService.java:48)
  at com.actiontech.dble.services.BackendService$BackendOnetimeRunnable.run(BackendService.java:647)
  at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149)
  at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
  at java.lang.Thread.run(Thread.java:748)
```

- 从 closeNoHandler.log 中可以看到是一个空闲连接收到了 mysql 发送的 error 报文

dble 中 responseHandler = null 只有两种可能 1. 执行 sql 语句的 connection 执行 close 后会把 responseHandler 置为 null 2. 空闲连接的 responseHandler 默认为 null，因为 connection 执行 close 会在日志中会打印 close 相关日志，而 dble 日志中没有打印所以排除可能 1

```
LOGGFR=@l og4j logger[org.apache.logging.slf4j.Log4jLogger@2c27c201],
responseHandler=null,
attachment=null,
session=null,
session2=null,
complexQuery=@Boolean[false],
```

结合上述日志分析，看起来是 mysql 因为连接超时关闭该连接，这时候 dble 不仅收到 mysql 发送的 fin 包还收到了 mysql 发送的 error 报文，然后 dble 处理 error 报文打印了 no handler 日志。使用和客户端相同的 mysql 版本（8.0.25）按照猜想进行尝试。

实验

- mysql 设置超时时间

```
mysql> set global interactive_timeout=20;
Query OK, 0 rows affected (0.00 sec)
```

- 在 mysql 服务器上开始抓包

```
tcpdump port 3312 -w time_wait.cap
```

- 启动 dble，然后等待连接池中的连接超时，停止抓包
- dble 日志中检索打印 no handler 日志的后端连接，查看 mysqlid（本实验中为 667）和 localPort（本实验中为 51886）
- 拿着 mysqlid 在 mysql error log 中检索，确认为因为连接超时 mysql 主动关闭了该连接
- 使用 tcp.port == 51886 在抓包文件中过滤信息

wireshark 分析抓包文件

从抓包内容中可以看到 mysql 的连接超时后，mysql 先向客户端（dble）发送了一个 error 报文然后在发送一个 fin 包，符合我们的猜想

No.	Time	Source	Destination	Protocol	Length	Transmission Control Pr	Info
4	0.017067	10.106.62.41	10.106.62.148	MySQL	215	✓	Server Greeting Error: 4031
5	0.017229	10.106.62.41	10.106.62.148	TCP	66	✓	3312 → 51886 [FIN, ACK] Seq=150 Ack=1 Win=510 Len=0 TSval=3842548932 TSecr=4215493366
6	0.018357	10.106.62.148	10.106.62.41	TCP	66	✓	51886 → 3312 [FIN, ACK] Seq=1 Ack=151 Win=16652 Len=0 TSval=4215513377 TSecr=3842548911

Frame 4: 215 bytes on wire (1720 bits), 215 bytes captured (1720 bits)

Ethernet II, Src: 02:00:0a:ba:3e:29 (02:00:0a:ba:3e:29), Dst: 02:00:0a:ba:3e:94 (02:00:0a:ba:3e:94)

Internet Protocol Version 4, Src: 10.106.62.41 (10.106.62.41), Dst: 10.106.62.148 (10.106.62.148)

Transmission Control Protocol, Src Port: 3312, Dst Port: 51886, Seq: 1, Ack: 1, Len: 149

MySQL Protocol

Packet Length: 145

Packet Number: 0

Error Code: 4031

SQL state: HY000

Error message: The client was disconnected by the server because of inactivity. See wait_timeout and interactive_timeout for configuring this behavior.

这里就忍不住想看看这个抓包结果中的 Server Greeting Error 4031 是什么然后在 mysql 文档中找到了相关信息

- Error 4031 报文详细信息

- Error number: 4031; Symbol: `ER_CLIENT_INTERACTION_TIMEOUT`; SQLSTATE: HY000

Message: The client was disconnected by the server because of inactivity. See `wait_timeout` and `interactive_timeout` for configuring this behavior.

`ER_CLIENT_INTERACTION_TIMEOUT` was added in 8.0.24.

- 发现是在 `mysql8.0.24` 中加入的，然后去 `mysql` 的 `8.0.24 Release Notes` 查看是否有详细说明。可以看到 `Release Notes` 中提到了因为 `wait_timeout` 闭的连接 `mysql` 在关闭该连接之前会先向客户端发送一个包含关闭该连接的详细原因的报文

Connection Management Notes

- Previously, if a client did not use the connection to the server within the period specified by the `wait_timeout` system variable and the server closed the connection, the client received no notification of the reason. Typically, the client would see `Lost connection to MySQL server during query (CR_SERVER_LOST)` or `MySQL server has gone away (CR_SERVER_GONE_ERROR)`.

In such cases, the server now writes the reason to the connection before closing it, and client receives a more informative error message, `The client was disconnected by the server because of inactivity. See wait_timeout and interactive_timeout for configuring this behavior. (ER_CLIENT_INTERACTION_TIMEOUT)`.

The previous behavior still applies for client connections to older servers and connections to the server by older clients.

- 再看看具体代码处理

```
my_error(net->last_errno, MYF(0));
/* First packet always wait for net_wait_timeout */
if (net->pkt_nr == 0 && vio_was_timeout(net->vio)) {
    net->last_errno = ER_NET_WAIT_ERROR;
    net->last_errno = ER_CLIENT_INTERACTION_TIMEOUT;
    /* Socket should be closed after trying to write/send error. */
    LogErr(INFORMATION_LEVEL, net->last_errno);
    LogErr(INFORMATION_LEVEL, ER_NET_WAIT_ERROR);
}
net->error = NET_ERROR_SOCKET_NOT_READABLE;
/*
    Attempt to send error message to client although the client won't be
    expecting messages. If later the client tries to send a command and fail
    it will instead check if it can read an error message.
*/
my_error(net->last_errno, MYF(0));
#else
/* Socket should be closed. */
net->error = NET_ERROR_SOCKET_UNUSABLE;
```

源码简要说明

- `pr` 提交之前的逻辑（红色部分和灰色部分）
 - 直接向客户端发送 `last_errno` 的内容（这里向客户端发送的是 `psh` 包，不是 `mysql` 协议的报文）
 - `net->last_errno` 置为 `ER_NET_WAIT_ERROR`
 - 使用 `LogErr` 方法将 `ER_NET_WAIT_ERROR` 的内容写入 `error log`
 - 连接等 `write` 日志和发送数据结束后关闭
- `pr` 提交之后的逻辑（绿色部分和灰色部分）

- net->last_errno 置为 ER_CLIENT_INTERACTION_TIMEOUT
- 使用 LogErr 方法将 ER_NET_WAIT_ERROR 的内容写入 error log
- 向客户端发送 last_errno 的内容 (Error 4031)
- 连接等 write 日志和发送数据结束后关闭

结论

mysql8.0.24 及其之后的版本在因为 mysql 连接超时的情况下会向客户端先发送一个 error 报文再关闭连接。如果 mysql 关闭的是 dble 连接池中的空闲连接, dble 的空闲连接会处理这个报文, 由于空闲连接不存在 responseHandler (responseHandler 是 dble 用来处理 mysql 发送的报文, 所以只有执行语句的连接才会有 responseHandler, 空闲连接默认 responseHandler 为 null), 所以 dble 就直接打印了 no handler 日志, 这是一个正常现象, 当然对连接超时的 error 报文 dble 会在之后进行调整。

参考

<https://arthas.aliyun.com/doc/https://dev.mysql.com/doc/relnotes/mysql/8.0/en/news-8-0-24.html><https://dev.mysql.com/doc/mysql-errors/8.0/en/server-error-reference.html><https://github.com/mysql/mysql-server/commit/14508bbb1790697c28659dd051fbc855cd3b5da9>

4.22 Left join ... on shardingColumn = N 优化为整体下发单节点的可行性

作者: 文韵涵

背景

SQL-1: select a.name from tabler a Left Join gtable1 b on a.name = b.name and a.id = 2; (tabler、gtable1 分别为分片表、全局表, 其中 tabler.id 为分片列; 两个表配置的节点均为 dn1~4)

查看 SQL-1 在 DBLE 中执行计划 Explain-1:

```
mysql> explain select a.name from tabler a Left Join gtable1 b on a.name = b.name and a.id = 2;
+-----+-----+-----+
| SHARDING_NODE | TYPE | SQL/REF |
+-----+-----+-----+
| dn1_0 | BASE SQL | select `a`.`name`,`a`.`id` from `tabler` `a` ORDER BY `a`.`name` ASC |
| dn2_0 | BASE SQL | select `a`.`name`,`a`.`id` from `tabler` `a` ORDER BY `a`.`name` ASC |
| dn3_0 | BASE SQL | select `a`.`name`,`a`.`id` from `tabler` `a` ORDER BY `a`.`name` ASC |
| dn4_0 | BASE SQL | select `a`.`name`,`a`.`id` from `tabler` `a` ORDER BY `a`.`name` ASC |
| merge_and_order_1 | MERGE_AND_ORDER | dn1_0; dn2_0; dn3_0; dn4_0 |
| shuffle_field_1 | SHUFFLE_FIELD | merge_and_order_1 |
| dn2_1 | BASE SQL | select `b`.`name` from `gtable1` `b` order by `b`.`name` ASC |
| merge_1 | MERGE | dn2_1 |
| join_1 | JOIN | shuffle_field_1; merge_1 |
| shuffle_field_2 | SHUFFLE_FIELD | join_1 |
+-----+-----+-----+
10 rows in set (0.01 sec)
```


从 explain 中，将分片表 `tabler` 分别下发各个节点，全局表 `gtable1` 单独下发一个节点分别获取数据；另外，没有 “on a.name = b.name and a.id = 2 ” 条件的身影，想必是在 DBLE 层面对数据进行的 join 合并和筛选处理了。

提出 Suppose-1

SQL-1 on 中 “a.id = 2”，a.id 作为分片列；看起来可以根据 a.id 明确路由单节点下发 `tabler` 表，然后 `gtable1` 作为全局表可以随意选一个路由节点下发；

具体假设：根据 “a.id = 2” 确定路由节点为 `dn1`，那么 SQL-1 貌似可以整体下发(直接透传)到 `dn1` 节点中了，这样以来可以减少在 DBLE 层面 join 数据的处理。

After a while:咦....啊...为啥....好像不行诶.....

经过一顿胡乱操作，发现 Suppose-1 的优化方案并不适合在 left/right join 场景下。

SQL-2: select a.name from `tabler` a Left Join `gtable1` b on a.name = b.name where a.id = 2;

提出的 Suppose-1 的实现方案几乎与在 DBLE 中处理 SQL-2 的思维雷同；反思：误认为 SQL-2 中 “where a.id = 2” 与 SQL-1 中 “on ... a.id = 2 ” 属于同一种过滤。

正视 On 和 Where 的定义

- On: 作为两表在做笛卡尔积关联时，附加的关联条件，生成虚拟表 V。
- Where: 对虚拟表 V（两表关联后的结果）筛选时用的过滤条件，只保留符合条件的数据行

MySQL 中执行 Left Join

1、Employee 为左表，Info 为右表，On 关联条件：a.name = b.name

```
mysql> select * from Employee a left join Info b on a.name = b.name;
+-----+-----+-----+-----+-----+-----+-----+-----+
| name | empid | deptname | level | name | age | country | deptname |
+-----+-----+-----+-----+-----+-----+-----+-----+
| Harry | 3415 | Finance | P7 | Harry | 25 | China | Finance |
| Sally | 2242 | Sales | P7 | Sally | 30 | USA | Sales |
| Harriet | 2202 | Sales | P8 | Harriet | 35 | Japan | Sales |
| Mary | 1257 | Human Resources | P7 | Mary | 22 | China | Human Resources |
| Lili | 9527 | Human Resources | P9 | Lili | 33 | Kream | Human Resources |
| Jessi | 7948 | Finance | P8 | Jessi | 27 | Kream | Finance |
| George | 3401 | Finance | P8 | NULL | NULL | NULL | NULL |
| Tom | 7012 | Market | P9 | NULL | NULL | NULL | NULL |
| Tony | 3052 | Market | P10 | NULL | NULL | NULL | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+
9 rows in set (0.00 sec)
```

Result-1: 左表数据全部展示，右表符合关联条件的则显示对应的信息，若没有关联信息，则右表填充 null。(图中，左表的全部信息有 10 个 name，而右表的 name 没有 George,Tom,Tony，所以对应右表显示 null)

2、在 Result-1 中追加 On 关联条件：b.country = 'China'。


```
mysql> select * from Employee a left join Info b on a.name = b.name and b.country = 'China';
+-----+-----+-----+-----+-----+-----+-----+-----+
| name | empid | deptname | level | name | age | country | deptname |
+-----+-----+-----+-----+-----+-----+-----+-----+
| Harry | 3415 | Finance | P7 | Harry | 25 | China | Finance |
| Mary | 1257 | Human Resources | P7 | Mary | 22 | China | Human Resources |
| Sally | 2242 | Sales | P7 | NULL | NULL | NULL | NULL |
| George | 3401 | Finance | P8 | NULL | NULL | NULL | NULL |
| Harriet | 2202 | Sales | P8 | NULL | NULL | NULL | NULL |
| Lili | 9527 | Human Resources | P9 | NULL | NULL | NULL | NULL |
| Tom | 7012 | Market | P9 | NULL | NULL | NULL | NULL |
| Tony | 3052 | Market | P10 | NULL | NULL | NULL | NULL |
| Jessi | 7948 | Finance | P8 | NULL | NULL | NULL | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+
9 rows in set (0.01 sec)
```

Result-2: 左表数据依旧全部展示, 在 Result-1 基础上的数据行找到符合 “b.country = 'China' ” 关联条件, 如果不符合, 则右表填充 null。(图中, 右表中只有两条 country 为 China 的数据, 其余的都不符合关联条件, 所以右表显示 null)

3、在 Result-1 中追加 Where 过滤条件: b.country = 'China' 。

```
mysql> select * from Employee a left join Info b on a.name = b.name where b.country = 'China';
+-----+-----+-----+-----+-----+-----+-----+-----+
| name | empid | deptname | level | name | age | country | deptname |
+-----+-----+-----+-----+-----+-----+-----+-----+
| Harry | 3415 | Finance | P7 | Harry | 25 | China | Finance |
| Mary | 1257 | Human Resources | P7 | Mary | 22 | China | Human Resources |
+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

Result-3: 在 Result-1 的基础上, 进行 “b.country = 'China' ” 过滤条件的筛选, 剔除不符合的数据行。(图中, 只显示符合 country 为 China 过滤条件的两条数据)

总结

- 从上面的 Result-2 和 Result-3 , 可以很明显的看出 Left join 下使用 On 和 Where 的区别:
 - On : 关联条件不影响左表全部信息显示, 它是作为右表是否符关联条件, 不符合的, 右边将由 NULL 填充。
 - Where : 不关心 JOIN 类型, 在虚拟表 V 基础上再进行条件过滤, 只返回符合条件的数据行。
- LEFT JOIN
 - 水平视角, 等价于左表的全部信息+(右边符合关联条件的信息+不符合关联条件的 NULL 填充)
 - 垂直视角, 等价于 INNER JOIN + 补足左表 + 右表 NULL 填充
 - 左外连接 ($\bowtie$) : 左外连接写成 $R \bowtie S$, 其中 R 与 S 为关系。左外连接的结果包含 R 中所有元组, 对每个元组, 若在 S 中有在公共属性名字上相等的元组, 则正常连接, 若在 S 中没有在公共属性名字上相等的元组, 则依旧保留此元组, 并将对应其他列设为 NULL 。

$$(R \bowtie S) \cup ((R - \pi_{r_1, r_2, \dots, r_n}(R \bowtie S)) \times \{(\omega, \dots, \omega)\})$$

- 在 LEFT JOIN 中, 左表的数据需全部显示; 根据提出的 Suppose-1 方案, SQL-1 中只会下发 dn1 一个节点, 并不符合 on 关联条件的定义, 所以 Suppose-1 优化方案不可行; (目前的 Explain-1 是符合预期的)

（Right Join 与 Left Join 相似，Right Join 的左边作为右表，右边作为左表，可套用以上的描述，然后在显示的两表调换顺序即可）。

MySQL Left Join 中 on 关联条件的实现逻辑

数据准备

```
create table A(c1 int, c2 int); create table B(c3 int, c4 int);  
  
insert into A values(1,1),(22,22),(333,333); insert into B values(0,1),(1,1);
```

查询

```
mysql> select * from A left join B on A.c1 > 0;  
+-----+-----+-----+-----+  
| c1    | c2    | c3    | c4    |  
+-----+-----+-----+-----+  
| 1     | 1     | 0     | 1     |  
| 22    | 22    | 0     | 1     |  
| 333   | 333   | 0     | 1     |  
| 1     | 1     | 1     | 1     |  
| 22    | 22    | 1     | 1     |  
| 333   | 333   | 1     | 1     |  
+-----+-----+-----+-----+  
6 rows in set (0.00 sec)
```

（图 1）

```
mysql> select * from A left join B on A.c1 = 333;  
+-----+-----+-----+-----+  
| c1    | c2    | c3    | c4    |  
+-----+-----+-----+-----+  
| 333   | 333   | 0     | 1     |  
| 333   | 333   | 1     | 1     |  
| 1     | 1     | NULL  | NULL  |  
| 22    | 22    | NULL  | NULL  |  
+-----+-----+-----+-----+  
4 rows in set (0.00 sec)
```

（图 2）

提问

图 1 结果符合预期，但图 2 的结果有点不理解，不是说 on 条件不会过滤么，这里好像过滤了吧？

解答

图 2 中没有对数据过滤；需要了解 MySQL 中采用嵌套循环连接的 join 算法，这里列举 Left join 实现的伪代码：

```
for(leftTable :leftRow) { // 遍历左表的每一行  
    boolean b = false;
```

```

for(rightTable: rightRow) { // 遍历右表每一行
    if(满足 on 关联条件){
        System.out.println(leftRow + rightTable); // 则左、右行进行合并输出
        b=true;
    }
}
if(!b) { // 遍历完 rightTable, 发现 leftRow 没有关联对应的行, 则用 null 补一行
    System.out.println(leftRow + NULL); // 则左行与 NULL 合并输出
}
}

```

具体解读:

图 1, “A.c1 > 0” 作为关联条件总是为 true, 查询结果等于两个表的笛卡尔积。

图 2, “A.c1 = 333” 也还是关联, 套用伪代码: 左表 c1 列为 333 的时候, 已经满足了 on 条件, 左、右边数据合并; 左表 c1 列为 1 和 22 时, 不符合关联条件, 所以各自就用左边数据+NULL 合并补了一行;

参考文档

<https://dev.mysql.com/doc/refman/5.6/en/nested-join-optimization.htm>

[https://zh.wikipedia.org/wiki/%E5%85%B3%E7%B3%BB%E4%BB%A3%E6%95%B0_\(%E6%95%B0%E6%8D%AE%E5%BA%93\)](https://zh.wikipedia.org/wiki/%E5%85%B3%E7%B3%BB%E4%BB%A3%E6%95%B0_(%E6%95%B0%E6%8D%AE%E5%BA%93))

4.23 如何使用 DTLE 将 Oracle 数据同步到 MySQL

作者: 刘安

一、现状

1. 目前只支持增量同步
 - a. 可以根据 SCN 节点开启增量复制
 - b. 从任务启动时间开启增量复制
2. 类型映射
 - a. 已支持类型

Oracle	MySQL	限制
BINARY_DOUBLE	float	mysql 不支持 Inf/-Inf/Nan 数据，用 NULL 来存储
CHAR(n), CHARACTER(n)	CHAR(n), CHARACTER(n)	
DATE	datetime	
DECIMAL(p,s), DEC(p,s)	DECIMAL(p,s), DEC(p,s)	
DOUBLE PRECISION	DOUBLE PRECISION	
FLOAT(p)	DOUBLE	
INTEGER, INT	INT	极值问题 (https://github.com/actiontech/dtle/issues/825)
INTERVAL DAY(p) TO SECOND(s)	VARCHAR(30)	
INTERVAL YEAR(p) TO MONTH	VARCHAR(30)	
NCHAR VARYING(n)	NCHAR VARYING(n)	
NCHAR(n)	NCHAR(n)/NVARCHAR(n)	
NUMBER(p,0), NUMBER(p)	TINYINT/SMALLINT/INT/BIGINT/DECIMAL(p)	
NUMBER(p,s)	DECIMAL(p,s)	
NUMBER, NUMBER(*)	DOUBLE	
NUMERIC(p,s)	NUMERIC(p,s)	
NVARCHAR2(n)	NVARCHAR(n)	
RAW(n)	VARBINARY(n)	
REAL	DOUBLE	

Oracle	MySQL	限制
ROWID	CHAR(100)	
SMALLINT	DECIMAL(38)	
TIMESTAMP(p)	datetime	
VARCHAR2(n)	VARCHAR(n)	
VARCHAR(n)	VARCHAR(n)	

b. 待支持类型

Oracle	MySQL	当前不支持原因
BINARY_FLOAT	float	MySQL 不支持 Inf/-Inf/Nan 数据, MySQL float 类型无法精确匹配, 导致更新失败
BLOB	BLOB	当前实现逻辑, 无法从 redoSQL 获取足够的值
CLOB	CLOB	当前实现逻辑, 无法从 redoSQL 获取足够的值
LONG	LONGTEXT	只支持 insert
LONG RAW	LOB	只支持 insert
NCLOB	TEXT	无法从 redoSQL 获取足够的值
TIMESTAMP(p) WITH TIME ZONE	datetime	时区问题未处理

c. 不支持类型

Oracle	MySQL	不支持原因
BFILE	VARCHAR(255)	logminer 不支持
UROWID(n)	VARCHAR(n)	logminer 读取的数据不足以构造新 SQL
XMLTYPE	VARCHAR(30)	logminer 不支持

3. DML 支持度

a. DML 类型

DML 类型	Oracle SQL	MySQL SQL
INSERT	INSERT INTO ACTION_DB.CHAR_255_COLUMNS VALUES (0, NULL)	replace into ACTION_DB.CHAR_255_COLUMNS (COL1, COL2) values ('0', NULL)
UPDATE	UPDATE ACTION_DB.CHAR_255_COLUMNS SET COL2='abcdefghijklmnopqrst uvwxyz' WHERE COL1=0	update ACTION_DB.CHAR_255_COLUMNS set COL1='0', C OL2='abcdefghijklmnopqrstuvwxy' where ((COL1 = '0') and (COL2 is NULL)) limit 1
DELETE	DELETE FROM ACTION_DB.CHARACTE R_255_COLUMNS WHERE COL1=0	delete from ACTION_DB.CHAR_255_COLUMNS where ((COL1 = '0') and (COL2 = 'abcdefghijklmnopqrstuvwxy')) limit 1

b. DML 函数支持度

函数名	是否支持	其他
CURRENT_TIMESTAMP	是	
DATE	是	
EMPTY_BLOB	是	函数支持解析为 NULL
EMPTY_CLOB	是	函数支持解析为 NULL
HEXTORAW	是	
LOCALTIMESTAMP	是	
RAWTOHEX	是	
RAWTOHEX(CHR())	是	
SYSTIMESTAMP	是	
TO_DATE	是	
TO_DSINTERVAL	是	
TO_TIMESTAMP	是	

函数名	是否支持	其他
TO_YMINTERVAL	是	
UNISTR	是	

4. DDL 支持度

DDL	Target	Option
CREATE	表 TABLE	DEFAULT CREATE
ALTER	表 TABLE	增加字段
		删除字段
		重命名字段 （当前仅支持 MySQL 8.0 语法）
		变更字段类型
DROP	表 TABLE	DEFAULT DROP

二、环境准备

1. Oracle 数据库开启归档日志

```

shell> su oracle
shell> mkdir /u01/app/oracle/oradata/archive_log
shell> sqlplus sys/oracle as sysdba

SQL> alter system set log_archive_dest_1='location=/u01/app/oracle/oradata/archive_log'
scope=spfile;
SQL> alter system set db_recovery_file_dest_size = 10G;
SQL> shutdown immediate;
SQL> startup mount;
SQL> alter database add logfile group 3 '/u01/app/oracle/fast_recovery_area/XE/onlinelo
g/redo01.log' size 500m;
SQL> alter database add logfile group 4 '/u01/app/oracle/fast_recovery_area/XE/onlinelo
g/redo02.log' size 500m;
SQL> alter database add logfile group 5 '/u01/app/oracle/fast_recovery_area/XE/onlinelo
g/redo03.log' size 500m;
SQL> alter database archivelog;
SQL> alter database add supplemental log data (all) columns;
SQL> alter database open;
SQL> archive log list;

```

```
# Archive Mode 表示已开启归档模式, Archive destination 表示归档日志储存路径
```

2. 安装 LogMiner 工具 (Oracle 安装时默认安装)

可查看系统中是否存在运行 LogMiner 所需要的 dbms_logmnr、dbms_logmnr_d 包, 如果没有安装 LogMiner 工具需要的包, 需要运行下面两个命令:

```
shell> cat $ORACLE_HOME/rdbms/admin/dbmslsm.sql | sqlplus sys/oracle as sysdba
shell> cat $ORACLE_HOME/rdbms/admin/dbmslmd.sql | sqlplus sys/oracle as sysdba
```

3. 创建 logminer 需要角色权限

```
shell> su oracle
shell> sqlplus sys/oracle as sysdba

SQL> create user roma_logminer identified by oracle default tablespace users;

SQL> GRANT CREATE SESSION TO roma_logminer;
SQL> GRANT SET CONTAINER TO roma_logminer;
SQL> GRANT SELECT ON V_$DATABASE TO roma_logminer;
SQL> GRANT FLASHBACK ANY TABLE TO roma_logminer;
SQL> GRANT SELECT ANY TABLE TO roma_logminer;
SQL> GRANT SELECT_CATALOG_ROLE TO roma_logminer;
SQL> GRANT EXECUTE_CATALOG_ROLE TO roma_logminer;
SQL> GRANT SELECT ANY TRANSACTION TO roma_logminer;
SQL> GRANT CREATE TABLE TO roma_logminer;
SQL> GRANT LOCK ANY TABLE TO roma_logminer;
SQL> GRANT CREATE SEQUENCE TO roma_logminer;
SQL> GRANT EXECUTE ON DBMS_LOGMNR TO roma_logminer;
SQL> GRANT EXECUTE ON DBMS_LOGMNR_D TO roma_logminer;

SQL> GRANT SELECT ON V_$LOG TO roma_logminer;
SQL> GRANT SELECT ON V_$LOG_HISTORY TO roma_logminer;
SQL> GRANT SELECT ON V_$LOGMNR_LOGS TO roma_logminer;
SQL> GRANT SELECT ON V_$LOGMNR_CONTENTS TO roma_logminer;
SQL> GRANT SELECT ON V_$LOGMNR_PARAMETERS TO roma_logminer;
SQL> GRANT SELECT ON V_$LOGFILE TO roma_logminer;
SQL> GRANT SELECT ON V_$ARCHIVED_LOG TO roma_logminer;
SQL> GRANT SELECT ON V_$ARCHIVE_DEST_STATUS TO roma_logminer;

SQL> alter user roma_logminer quota unlimited ON users;
```

4. 部署一个单节点的 4.22.01.0 版本 DTLE

https://github.com/actiontech/dtle/releases/download/v4.22.01.0/dtle-ce-4.22.01.0.x86_64.rpm

三、创建 Oracle-MySQL 任务

1. 获取 Token

```
shell> curl -s -X POST "http://172.100.9.11:8190/v2/loginWithoutVerifyCode" -
H "accept: application/json" -H "Content-Type: application/json" -
d '{"password": "admin", "tenant": "platform", "username": "admin"}' | jq
{
  "message": "ok",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE2NDYxOTE2MzksImdyb3VwIjoiYXNjaW4ifQ.-c_jVkxd_vP5Ka3gocdfGZIudWPujutdHpQYx8srX00"
  }
}
```

2. 准备 job 文件

```
shell> cat job.json
{
  "job_id": "test_oracle",
  "src_task": {
    "connection_config": {
      "database_type": "Oracle",
      "host": "172.100.9.31",
      "port": 1521,
      "user": "roma_logminer",
      "password": "oracle",
      "service_name": "XE"
    },
    "node_id": "96d28881-f91b-19f4-1614-4d8a0e718e2f",
    "binlog_relay": false,
    "repl_chan_buffer_size": 120,
    "group_max_size": 1,
    "group_timeout": 100,
    "oracle_src_task_config": {
      "scn": 0
    },
    "task_name": "src",
    "replicate_do_db": [
      {
        "table_schema": "ACTION_DB"
      }
    ]
  },
  "is_password_encrypted": false,
  "dest_task": {
    "connection_config": {
      "database_type": "MySQL",
```

```

    "host": "172.100.9.1",
    "port": 3306,
    "user": "test_dest",
    "password": "test_dest"
  },
  "node_id": "96d28881-f91b-19f4-1614-4d8a0e718e2f",
  "task_name": "dest",
  "mysql_dest_task_config": {}
},
"task_step_name": "job_stage_full",
"failover": true,
"retry": 2
}

```

3. 创建 Oracle-MySQL job

```

shell> curl -s -X POST "http://172.100.9.11:8190/v2/job/migration/create" -
H "accept: application/json" -
H "Authorization: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE2NDYxOTE2MzksImdyb3VwIjoicGxhdGZvcn0iLCJuYW11IjoieWRtaW4ifQ.-c_jVkd_vP5Ka3gocdfGZIudWPujutdHpQYx8srX00" -
H "Content-Type: application/json" -d @job.json | jq
{
  "job": {
    "job_id": "test_oracle-migration",
    "task_step_name": "job_stage_full",
    "reverse": false,
    "failover": true,
    "is_password_encrypted": false,
    "src_task": {
      "task_name": "src",
      "node_id": "96d28881-f91b-19f4-1614-4d8a0e718e2f",
      "replicate_do_db": [
        {
          "table_schema": "ACTION_DB",
          "table_schema_regex": "",
          "table_schema_rename": "",
          "tables": null
        }
      ],
      "replicate_ignore_db": null,
      "skip_create_db_table": false,
      "drop_table_if_exists": false,
      "mysql_src_task_config": null,
      "oracle_src_task_config": {
        "scn": 0
      }
    }
  }
}

```

```

    },
    "group_max_size": 1,
    "group_timeout": 100,
    "repl_chan_buffer_size": 120,
    "chunk_size": 2000,
    "connection_config": {
        "host": "172.100.9.31",
        "port": 1521,
        "user": "roma_logminer",
        "password": "*",
        "service_name": "XE",
        "database_type": "Oracle"
    }
},
"dest_task": {
    "task_name": "dest",
    "node_id": "96d28881-f91b-19f4-1614-4d8a0e718e2f",
    "database_type": "",
    "mysql_dest_task_config": {
        "parallel_workers": 1,
        "use_my_sql_dependency": false,
        "dependency_history_size": 2500
    },
    "connection_config": {
        "host": "172.100.9.1",
        "port": 3306,
        "user": "test_dest",
        "password": "*",
        "service_name": "",
        "database_type": "MySQL"
    }
},
"retry": 2
},
"eval_create_index": 12,
"job_modify_index": 12,
"message": "ok"
}

```

4. 源端 Oracle 写入数据

```

SQL> create tablespace ACTION_DB datafile 'ACTION_DB.dbf' size 100M;
SQL> create user ACTION_DB identified by ACTION_DB default tablespace ACTION_DB;
SQL> grant unlimited tablespace to ACTION_DB;
SQL> CREATE TABLE ACTION_DB.CHAR_255_COLUMNS(col1 INT, col2 CHAR(255));

```

```
SQL> INSERT INTO ACTION_DB.CHAR_255_COLUMNS VALUES (0, NULL);
SQL> INSERT INTO ACTION_DB.CHAR_255_COLUMNS VALUES (1, 'abcdefghijklmnopqrstuvwxy');
SQL> INSERT INTO ACTION_DB.CHAR_255_COLUMNS VALUES (2, 'ABCDEFGHIJKLMNOPQRSTUVWXYZ');
SQL> INSERT INTO ACTION_DB.CHAR_255_COLUMNS VALUES (3, '1234567890');
SQL> INSERT INTO ACTION_DB.CHAR_255_COLUMNS VALUES (4, 1234567890);
SQL> INSERT INTO ACTION_DB.CHAR_255_COLUMNS VALUES (5, '~`!@#%^&*()-
=_+{ } [ ] | \ : ; < > , . ? / ' );
SQL> INSERT INTO ACTION_DB.CHAR_255_COLUMNS VALUES (6, '中文测试 abc');
SQL> INSERT INTO ACTION_DB.CHAR_255_COLUMNS VALUES (7, '•~!@#¥%……&*() -=—+{ }
【】、|; ‘: “, 。 / 《》? ');
```

5. 目标端 MySQL 检查同步情况

```
mysql> SHOW CREATE TABLE ACTION_DB.CHAR_255_COLUMNS\G
***** 1. row *****
      Table: CHAR_255_COLUMNS
Create Table: CREATE TABLE `CHAR_255_COLUMNS` (
  `COL1` int(11) DEFAULT NULL,
  `COL2` char(255) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
1 row in set (0.00 sec)

mysql> SELECT * FROM ACTION_DB.CHAR_255_COLUMNS;
+-----+-----+
| COL1 | COL2                                     |
+-----+-----+
| 0    | NULL                                   | |
| 1    | abcdefghijklmnopqrstuvwxyz           |
| 2    | ABCDEFGHIJKLMNOPQRSTUVWXYZ           |
| 3    | 1234567890                           |
| 4    | 1234567890                           |
| 5    | ~`!@#%^&*()-=_+{ } [ ] | \ : ; < > , . ? / |
| 6    | 中文测试 abc                         |
| 7    | •~!@#¥%……&*() -=—+{ } 【】、|; ‘: “, 。 /
《》? |
+-----+-----+
8 rows in set (0.01 sec)
```

四、使用限制

因为 Oracle 和 MySQL 是异构数据库，所以在源端 Oracle 能执行的 Oracle SQL 语句通过 DTLE 转换到目标端的 MySQL SQL 语句后有可能无法正确执行。比如 Oracle 支持数值范围 MySQL 不支持，Oracle 的 DATE 类型支持公元前的年份而 MySQL 的 datetime 只能为公元后的年份等等。遇到这些情况，DTLE 的默认行为是报错并且停止同步。

如果需要跳过这种阻塞情况，可以修改 DTLE 的环境变量然后重新启动 DTLE 服务。

```

shell> vi /etc/systemd/system/multi-user.target.wants/dtle-nomad.service
# 添加
[Service]
Environment="SkipErr=true"

shell> systemctl daemon-reload
shell> systemctl restart dtle-nomad

```

4.24 利用 DTLE 将 MySQL 数据同步到 DBLE

作者：任仲禹

背景

源于某客户的需求，存在线上某业务 MySQL 库因为数据量及业务读写压力较大，需要将业务数据迁移到 DBLE 分布式数据库，但同时因为业务为 7x24h，能够停机的时间窗口较短，所以需要考虑数据实时同步的方案。

过往 DBLE 的业务上线基本为全新部署，数据实时同步的情况极少实施，去年 DTLE 发布后这一问题得到了些改善，今天我们来实践下。

环境准备

1. 目标端 DBLE 集群部署

- 安装 DBLE 软件、后端分片 MySQL 库过程略
 - DBLE 版本 3.20.10.8、MySQL 版本 5.7.25
- sharding.xml

```

<?xml version="1.0"?>
<!DOCTYPE dble:sharding SYSTEM "sharding.dtd">
<dble:sharding xmlns:dble="http://dble.cloud/" version="4.0">
  <schema name="dtle" sqlMaxLimit="-1" shardingNode="dn_01">
    <singleTable name="gtid_executed_v4" shardingNode="dn_01" sqlMaxLimit="-1"></singleTable>
  </schema>
  <schema name="ren" sqlMaxLimit="-1" shardingNode="dn_01">
    <shardingTable name="test" shardingNode="dn_01,dn_02,dn_03,dn_04" sqlMaxLimit="-1" shardingColumn="id" function="func_jumphash"></shardingTable>
  </schema>
  <shardingNode name="dn_03" dbGroup="dh-mysql-

```

```

cluster02" database="dh_dn_03"></shardingNode>
    <shardingNode name="dn_04" dbGroup="dh-mysql-
cluster02" database="dh_dn_04"></shardingNode>
    <shardingNode name="dn_02" dbGroup="dh-mysql-
cluster01" database="dh_dn_02"></shardingNode>
    <shardingNode name="dn_01" dbGroup="dh-mysql-
cluster01" database="dh_dn_01"></shardingNode>
    <function name="func_jumphash" class="jumpStringHash">
        <property name="partitionCount">4</property>
        <property name="hashSlice">0:-1</property>
    </function>
</db:sharding>

```

- db.xml

```

<?xml version="1.0"?>
<!DOCTYPE db:db SYSTEM "db.dtd">
<db:db xmlns:db="http://db:cloud/" version="4.0">
    <dbGroup name="dh-mysql-cluster02" rwSplitMode="0" delayThreshold="-1">
        <heartbeat timeout="0" errorRetryCount="0">show slave status</heartbeat>
        <dbInstance name="10.186.61.13-3326-dh-
1" url="10.186.61.13:3326" user="dbuser" password="jpfmxIeMt1vxAJ6zd6Q10PGRRi+Qj023Dl
+YXu0r3C4VXTdV5+GJa0Iv5iVmWCwpXcucn/zi02HVlT7ADX+m6Q==" maxCon="100" minCon="10" primar
y="true" readWeight="0" id="mysql-i63009" usingDecrypt="true"></dbInstance>
    </dbGroup>
    <dbGroup name="dh-mysql-cluster01" rwSplitMode="0" delayThreshold="-1">
        <heartbeat timeout="0" errorRetryCount="0">show slave status</heartbeat>
        <dbInstance name="10.186.61.11-3316-dh-
1" url="10.186.61.11:3316" user="dbuser" password="QQWRF80AGNb4jIAx/b2Ww7Myol1+ntlyz
GmA1A3PXVISmRD/i5pgRnLLwYsXoLmH0jiv1qZAKqIBHv6Yg/XAg==" maxCon="100" minCon="10" primar
y="true" readWeight="0" id="mysql-47vn84" usingDecrypt="true"></dbInstance>
    </dbGroup>
</db:db>

```

- user.xml

```

<?xml version="1.0"?>
<!DOCTYPE db:user SYSTEM "user.dtd">
<db:user xmlns:db="http://db:cloud/" version="4.0">
    <managerUser name="root" password="CrjplhvVJkHk0EPW35Y07dUeTimf52zMqClYQkIAN3/dqi
G1DVUe9Zr4JLh8Kl+1KH1zd7YTKu5w04QgdyQeDw==" usingDecrypt="true"></managerUser>
    <shardingUser name="ren" schemas="ren,dtle" password="P+C2KazQiS3ZZ6uoJBj91MZIqYq
GczspQ/ebyBZOC9xKAAkAFrqEDC90Pn/v0bAy04P8Zu3vHqJ+rljM040Kdg==" usingDecrypt="true" read
Only="false" maxCon="0" blacklist="default_black_list"></shardingUser>
</db:user>

```

2. 源端和目标端测试表创建

- 源端 MySQL 数据库软件安装略
- 源端 MySQL 与目标端 DBLE 都需要创建测试表名：test

```
use ren;
CREATE TABLE `test` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `name` varchar(20) COLLATE utf8mb4_bin DEFAULT NULL,
  `city` varchar(20) COLLATE utf8mb4_bin DEFAULT NULL,
  `dt` datetime DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  KEY `idx_ctiy` (`city`)
) ENGINE=InnoDB;
```

3. 部署单节点 DTLE

- DTLE 社区版本 GitHub 下载地址：<https://github.com/actiontech/dtle/>
- 下载完成后直接 rpm 安装（本示例使用内部 QA 验证版本）
 - rpm -ivh --prefix=/data/dtle dtle-ee-9.9.9.x86_64.rpm
- 安装完成确认启动正常

```
# curl -XGET "127.0.0.1:4646/v1/nodes" -s | jq
[
  {
    "Address": "127.0.0.1",
    "ID": "223c31b4-05cd-a763-b3e7-dbea6d416576",
    "Datacenter": "dc1",
    "Name": "nomad0",
    "NodeClass": "",
    "Version": "1.1.2",
    "Drain": false,
    "SchedulingEligibility": "eligible",
    "Status": "ready",
    "StatusDescription": "",
    "Drivers": {
      "dtle": {
        "Attributes": {
          "driver.dtle.full_version": "9.9.9.9-master-a65ee13",
          "driver.dtle": "true",
          "driver.dtle.version": "9.9.9.9"
        },
        "Detected": true,
        "Healthy": true,
        "HealthDescription": "Healthy",
        "UpdateTime": "2022-02-28T07:45:15.650289984Z"
      }
    }
  }
]
```

]

创建 MySQL-To-DBLE 任务

一、全量同步

1. 准备 job 文件

```
# cat job.json
{
  "Job": {
    "ID": "mysqlToDBLE",
    "Datacenters": ["dc1"],
    "TaskGroups": [{
      "Name": "src",
      "Tasks": [{
        "Name": "src",
        "Driver": "dtle",
        "Config": {
          "Gtid": "",
          "ReplicateDoDb": [{
            "TableSchema": "ren",
            "Tables": [{
              "TableName": "test"
            }]
          }]
        },
        "ConnectionConfig": {
          "Host": "10.186.61.11",
          "Port": 3306,
          "User": "root",
          "Password": "root"
        }
      }]
    }], {
      "Name": "dest",
      "Tasks": [{
        "Name": "dest",
        "Driver": "dtle",
        "Config": {
          "ConnectionConfig": {
            "Host": "10.186.61.10",
            "Port": 8066,
            "User": "ren",
            "Password": "ren"
          }
        }
      }]
    }
  }
}
```



```

        }
    }
}
}
}

```

2. 准备全量复制数据

● 源端 MySQL 库执行

```

mysql> insert into test values(1,'ren','sh',now());
mysql> insert into test values(2,'jack','bj',now());
mysql> insert into test values(3,'tom','sz',now());

```

3. 启动同步任务

```

# curl -XPOST "http://127.0.0.1:4646/v1/jobs" -d @job.json -s | jq
{
  "EvalID": "88ab4a42-98b7-696e-0f98-08c1fe3ee4bd",
  "EvalCreateIndex": 12310,
  "JobModifyIndex": 12310,
  "Warnings": "",
  "Index": 12310,
  "LastContact": 0,
  "KnownLeader": false
}

```

4. 检查同步情况

● 确认全量数据同步完成

```

# 目标端 DBLE 中执行
mysql> use ren;
Database changed
mysql> show tables;
+-----+
| Tables_in_ren |
+-----+
| test          |
| gtid_executed_v4 |
+-----+
2 rows in set (0.01 sec)

mysql> select * from test;
+-----+-----+-----+-----+
| id | name | city | dt                |
+-----+-----+-----+-----+
| 1  | ren  | sh   | 2022-03-07 06:53:30 |
| 2  | jack | bj   | 2022-03-07 06:53:41 |
+-----+-----+-----+-----+

```

```
| 3 | tom | sz | 2022-03-07 06:53:59 |
+----+-----+-----+-----+
3 rows in set (0.01 sec)
```

源端 MySQL 写入增量测试数据

```
mysql> insert into test select null,'mike','nj',now();
Query OK, 1 row affected (0.01 sec)
Records: 1 Duplicates: 0 Warnings: 0
.....
mysql> insert into test select null,'mike4','nj',now();
Query OK, 1 row affected (0.01 sec)
Records: 1 Duplicates: 0 Warnings: 0
```

```
mysql> update test set city = 'sh' where name like 'mike%';
Query OK, 4 rows affected (0.01 sec)
Rows matched: 4 Changed: 4 Warnings: 0
```

```
mysql> select * from test;
+----+-----+-----+-----+
| id | name | city | dt                |
+----+-----+-----+-----+
| 1 | ren  | sh   | 2022-03-07 06:53:30 |
| 2 | jack | bj   | 2022-03-07 06:53:41 |
| 3 | tom  | sz   | 2022-03-07 06:53:59 |
| 45 | mike | sh   | 2022-03-07 08:03:57 |
| 46 | mike2 | sh  | 2022-03-07 08:04:02 |
| 47 | mike3 | sh   | 2022-03-07 08:04:05 |
| 48 | mike4 | sh   | 2022-03-07 08:04:09 |
+----+-----+-----+-----+
7 rows in set (0.01 sec)
```

目标端 DBLE 检查增量同步情况

```
mysql> select * from test;
+----+-----+-----+-----+
| id | name | city | dt                |
+----+-----+-----+-----+
| 1 | ren  | sh   | 2022-03-07 06:53:30 |
| 2 | jack | bj   | 2022-03-07 06:53:41 |
| 3 | tom  | sz   | 2022-03-07 06:53:59 |
| 45 | mike | sh   | 2022-03-07 08:03:57 |
| 46 | mike2 | sh  | 2022-03-07 08:04:02 |
| 47 | mike3 | sh   | 2022-03-07 08:04:05 |
| 48 | mike4 | sh   | 2022-03-07 08:04:09 |
+----+-----+-----+-----+
```

```
7 rows in set (0.04 sec)

mysql> explain select * from test where id = 1;
+-----+-----+-----+
| SHARDING_NODE | TYPE      | SQL/REF          |
+-----+-----+-----+
| dn_01         | BASE SQL  | select * from test where id = 1 |
+-----+-----+-----+
1 row in set (0.03 sec)
```

二、基于 GTID 位点增量同步

1. 销毁全量同步任务

```
# cd /data/dtle/usr/bin/
# ll
total 188836
-rwxr-xr-x 1 root root 107811060 Mar 17 2020 consul
-rwxr-xr-x 1 root root 85550512 Jun 22 2021 nomad
# ./nomad job status
ID                Type      Priority  Status  Submit Date
mysqlToDBLE       service   50       running 2022-03-07T15:47:31+08:00
mysqltoMysql-sync service   50       running 2022-03-03T16:06:10+08:00
# ./nomad job stop -purge mysqlToDBLE
.....
  :: Deployment "433ed3d4" successful
.....
# ./nomad job status
ID                Type      Priority  Status  Submit Date
mysqltoMysql-sync service   50       running 2022-03-03T16:06:10+08:00
```

2. 记录源端 GTID 位点

```
# 记录源端 MySQL 需要开始的 GTID 位点
mysql> show master status\G
***** 1. row *****
                File: mysql-bin.000178
                .....
Executed_Gtid_Set: 442dbe92-00c3-11ec-a0cf-02000aba3d0b:1-49705119,
cdc6fb62-00c2-11ec-a259-02000aba3d0a:1-3555
1 row in set (0.01 sec)
# 插入增量数据（模拟业务新增数据）
mysql> insert into test select 88,'sync01','wh',now();
mysql> insert into test select 99,'sync02','wh',now();
# 源端 MySQL 确认数据已插入
mysql> select * from test;
```

```

+----+-----+-----+-----+
| id | name  | city | dt                |
+----+-----+-----+-----+
|  1 | ren   | sh   | 2022-03-07 06:53:30 |
.....
| 48 | mike4 | sh   | 2022-03-07 08:04:09 |
| 88 | sync01 | wh   | 2022-03-07 08:24:20 |
| 99 | sync02 | wh   | 2022-03-07 08:24:31 |
+----+-----+-----+-----+
9 rows in set (0.00 sec)
# 目标端 DBLE 数据因同步 job 已销毁，新插入数据未同步过来
mysql> select * from test;
+----+-----+-----+-----+
| id | name  | city | dt                |
+----+-----+-----+-----+
|  1 | ren   | sh   | 2022-03-07 06:53:30 |
.....
| 48 | mike4 | sh   | 2022-03-07 08:04:09 |
+----+-----+-----+-----+
7 rows in set (0.00 sec)

```

3. 准备增量同步 job 文件

```

# cat job.json
{
  "Job": {
    "ID": "mysqlToDBLE",
    "Datacenters": ["dc1"],
    "TaskGroups": [{
      "Name": "src",
      "Tasks": [{
        "Name": "src",
        "Driver": "dtle",
        "Config": {
          "Gtid": "442dbe92-00c3-11ec-a0cf-02000aba3d0b:1-49705119,cdc6fb62-00c2-11ec-a259-02000aba3d0a:1-3555",
          "ReplicateDoDb": [{
            "TableSchema": "ren",
            "Tables": [{
              "TableName": "test"
            }]
          }],
          "ConnectionConfig": {
            "Host": "10.186.61.11",
            "Port": 3306,

```

```

        "User": "root",
        "Password": "root"
    }
}
]]
}, {
    "Name": "dest",
    "Tasks": [{
        "Name": "dest",
        "Driver": "dtle",
        "Config": {
            "ConnectionConfig": {
                "Host": "10.186.61.10",
                "Port": 8066,
                "User": "ren",
                "Password": "ren"
            }
        }
    }]
}]
}
}

```

4. 开始增量同步任务

```

# curl -XPOST "http://127.0.0.1:4646/v1/jobs" -d @job.json -s |jq
{
  "EvalID": "cad6fb19-62d3-67aa-6f5c-fbb79f8016d2",
  "EvalCreateIndex": 12855,
  "JobModifyIndex": 12855,
  "Warnings": "",
  "Index": 12855,
  "LastContact": 0,
  "KnownLeader": false
}

```

5. 检查同步情况

```

# 目标端 DBLE 中查看到 GTID 位点之后的数据已同步过来
mysql> select * from test;
+-----+-----+-----+-----+
| id  | name  | city | dt                |
+-----+-----+-----+-----+
| 1   | ren   | sh   | 2022-03-07 06:53:30 |
| 48  | mike4 | sh   | 2022-03-07 08:04:09 |
.....

```

```
| 88 | sync01 | wh    | 2022-03-07 08:24:20 |
| 99 | sync02 | wh    | 2022-03-07 08:24:31 |
+-----+-----+-----+-----+
11 rows in set (0.06 sec)
```

6. 其它 DML 及 DDL 同步

验证下其它 update 、delete 语句及 DDL 语句同步情况

源端 MySQL 执行操作

```
mysql> delete from test where id >= 100;
Query OK, 2 rows affected (0.01 sec)
mysql> delete from test where id > 3;
Query OK, 6 rows affected (0.01 sec)
mysql> update test set name = 'actionsky' where id = 3;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

目标端 DBLE 检查同步情况

```
mysql> select * from test;
+----+-----+-----+-----+
| id | name    | city | dt                |
+----+-----+-----+-----+
| 1  | ren     | sh   | 2022-03-07 06:53:30 |
| 2  | jack    | bj   | 2022-03-07 06:53:41 |
| 3  | actionsky | sz   | 2022-03-07 06:53:59 |
+----+-----+-----+-----+
3 rows in set (0.01 sec)
```

源端 MySQL 执行 DDL 操作

```
mysql> alter table test add column info varchar(20) default 'hello';
mysql> update test set info = 'thanks' where id = 3;
mysql> alter table test add index idx_info(`info`);
```

目标端 DBLE 可以进行 DDL 同步（篇幅所限，实际上 DBLE 兼容的 DDL 语句都能同步成功）

```
mysql> select * from test;
+----+-----+-----+-----+-----+
| id | name    | city | dt                | info  |
+----+-----+-----+-----+-----+
| 1  | ren     | sh   | 2022-03-07 06:53:30 | hello |
| 2  | jack    | bj   | 2022-03-07 06:53:41 | hello |
| 3  | actionsky | sz   | 2022-03-07 06:53:59 | thanks |
+----+-----+-----+-----+-----+
3 rows in set (0.02 sec)
```

```
mysql> show create table test\G
```

```
***** 1. row *****
Table: test
Create Table: CREATE TABLE `test` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `name` varchar(20) COLLATE utf8mb4_bin DEFAULT NULL,
  `city` varchar(20) COLLATE utf8mb4_bin DEFAULT NULL,
  `dt` datetime DEFAULT CURRENT_TIMESTAMP,
  `info` varchar(20) COLLATE utf8mb4_bin DEFAULT 'hello',
  PRIMARY KEY (`id`),
  KEY `idx_ctiy` (`city`),
  KEY `idx_info` (`info`)
) ENGINE=InnoDB AUTO_INCREMENT=89 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_bin
1 row in set (0.01 sec)
```

配置 MySQL-To-DBLE 注意事项

1. 检测连接失败问题

- 问题描述：DTLE 配置 JOB 过程中，“检测连接”失败，nomad 日志报错 ERROR 1064 (HY000): java.sql.SQLException: illegal value[TRUE]
- 原因：DTLE 下发的检测客户端语句 set autocommit=true，在 DBLE 某些版本中不支持
- 解决：升级 DBLE 到 3.20.10.6 版本及之后

2. 任务启动后同步失败报'dtle'不存在

- 问题描述：DTLE 同步任务启动后报错，nomad 日志出现 Can't create database 'dtle' that doesn't exists.
- 原因：
 - DTLE To MySQL，不会出现该种报错
 - DTLE To DBLE，由于 DBLE 中间件中 schema 的创建方式与普通 MySQL 不一致，所以该 create 语法不支持
- 解决：
 - 需要对 DBLE 进行额外的 Schema/Table 配置，参考前文 sharding.xml 和 user.xml 中相关配置

3. 任务启动后同步失败报'Data too long'

- 问题描述：DTLE 同步任务启动后报错，nomad 日志出现“applier error/restart: insert gno: Error 1406: Data too long for column 'source_uuid' at row 1”
- 原因：
 - DTLE 在 DBLE 中创建的表 gtid_executed_v4 中，字段 source_uuid 的 Binary 数据类型长度不够
 - 也可通过排查 DBLE 中间件日志（core/log/dble.log），报错信息为“execute sql err : errNo:1406 Data too long for column 'source_uuid' at row 1”
- 解决：
 - DBLE 中，修改字段
 - alter table gtid_executed_v4 modify column source_uuid binary(60);

结论

- DTLE 目前功能基本可以满足 MySQL -> DBLE 间数据实施同步需求，不过需要注意的是，不建议采用本文所提到的全量同步方式
 - 生产环境实施由于 MySQL 老库数据量较大，可以先将数据全量逻辑备份出来（需记录 GTID 位点），再通过 DBLE 自带的 `split` 工具进行拆分后进行导入，然后再使用 DTLE 基于 GTID 位点增量同步的方式进行数据同步
- DTLE 创建 To-DBLE 任务前需要关注下前文所示注意事项，尤其是提前准备好 `sharding.xml`、`user.xml` 文件并创建好 DTLE 的元数据表 `gtid_executed_v4`。

第 5 章 开源数据库产品

本章节将介绍目前主流的开源数据库相关的技术文章，通过这些文章能快速掌握 Oceanbase、Tidb、MongoDB、ClickHouse 等开源数据库的一些实际使用场景，希望开源的社区，能给大家带来更多丰富的数据库解决方案。

5.1 ClickHouse 集群分布式 DDL 被阻塞案例一则

作者：任坤

一、背景

线上有套 10 节点 clickhouse 集群，5 分片 * 2 副本，版本 19.7.3。

开发执行一个创建分布式表的操作，9 个节点都成功返回，有个节点报错，返回信息如下：

```
Code: 159. DB::Exception: Received from 127.0.0.1:9000. DB::Exception: Watching task /clickhouse/task_queue/ddl/query-0003271440 is executing longer than distributed_ddl_task_timeout (=180) seconds. There are 1 unfinished hosts (0 of them are currently active), they are going to execute the query in background.
```

二、诊断

登录该节点查看 show processlist，正在执行 1 个分布式 ddl，该 ddl 已经运行 100 多个小时，应该是卡住了。

```
Row 1:
is_initial_query: 0
user:
query_id: f70cddba-56fb-45da-9ec6-73755cd3caa3
address: 0.0.0.0
port: 0
initial_user:
initial_query_id:
initial_address: 0.0.0.0
initial_port: 0
interface: 1
os_user:
client_hostname:
client_name:
client_version_major: 0
client_version_minor: 0
client_version_patch: 0
client_revision: 0
http_method: 0
http_user_agent:
quota_key:
elapsed: 438787.605019313
is_cancelled: 1
read_rows: 0
read_bytes: 0
total_rows_approx: 0
written_rows: 0
written_bytes: 0
memory_usage: 128
peak_memory_usage: 5247051
query: /* ddl_entry=query-0003271416 */ OPTIMIZE TABLE log_catch_data_local
thread_numbers: [44]
```

clickhouse 的分布式 ddl 是串行执行的，每次将任务存储到 zookeeper 的/clickhouse/task_queue/ddl 目

录，按照 FIFO 排列。

对于每个节点，只有当前的 ddl 执行完毕，后面的才能被调用。

```
select * from zookeeper where path='/clickhouse/task_queue/ddl' order by ctime desc\G
```

刚刚发起的创建分布式表 ddl 排在第一位，上述截图中的 optimize table 排在第 23，说明被其阻塞的 ddl 有 22 条之多，开发也确认最近两天的 ddl 任务在该节点上都没有成功。

尝试将其 kill，等待几个小时仍然没有效果。

```
localhost :) KILL QUERY WHERE query_id='e8b4c986-c758-4c72-b8b6-e2e0b19e008a';
KILL QUERY WHERE query_id = 'e8b4c986-c758-4c72-b8b6-e2e0b19e008a' ASYNC
```

kill_status	query_id	user	query
waiting	e8b4c986-c758-4c72-b8b6-e2e0b19e008a	ckuser	OPTIMIZE TABLE 202001071.log_catch_data_local;

```
1 rows in set. Elapsed: 0.002 sec.
```

加 sync 关键字，直接卡住。

```
localhost :) KILL QUERY WHERE query_id='966e8f09-6837-40ca-8308-e3bd5b14e047' sync;
KILL QUERY WHERE query_id = '966e8f09-6837-40ca-8308-e3bd5b14e047' SYNC
✓ Progress: 2.00 rows, 1.74 KB (19.70 rows/s., 17.17 KB/s.) 99% ^C cancelling query.
Ok.
```

尝试重启 ck 实例也卡住，最后只能 Kill -9。重启实例后该任务依然存在，而且执行了 10 多分钟没有要结束的意思，kill 操作仍然无效。

既然这个 ddl 无法绕过，执行的时候又长时间不结束，只能曲线救国，重命名该表让其临时消失一会。

```
rename table log_catch_data_local to log_catch_data_local1;
```

optimize table 当即返回，并且其后 22 个 ddl 也很快执行完毕，最后再将其重命名回原表。此时再次执行 optimize table，只需 5s 便成功返回，该问题解决。

三、总结

clickhouse 的分布式 ddl 在每个节点是 FIFO 串行执行的，任意 1 个 ddl 卡住了都会阻塞后续任务。

本例中的卡住的 ddl 是 optimize table，可以通过重命名表跳过；

如果是增删列，可以在 rename 后手工对该表执行本地 ddl。如果上述方法都不行，可以在出问题的节点将本地表直接 drop，等待所有阻塞 ddl 执行完毕后，重新创建 1 个空表，会自动从另一个副本中同步数据。

最后，19.x 版本已经很旧了，我们在使用过程中遭遇了各种问题，要尽快升级到 20.x 系列

5.2 ClickHouse-Keeper 初探

作者：陈宇

一、ClickHouse-Keeper 是什么：

ClickHouse 社区在 21.8 版本中开始引入了 ClickHouse-Keeper，直至 ClickHouse 21.12 发布公告提到 ClickHouse Keeper 功能基本完成。

ClickHouse Keeper 是 ZooKeeper 的替代品，与 ZooKeeper 不同，ClickHouse Keeper 是用 C++ 编写的，并使用 RAFT 算法实现，该算法允许对读写具有线性化能力。

二、Zookeeper 与 ClickHouse-Keeper 一些场景对比

为什么要引入 ClickHouse-Keeper 呢？主要是 ClickHouse 使用 Zookeeper 有着众多痛点：

- 使用 java 开发
- 运维不便
- 要求独立部署
- zxid overflow 问题
- snapshot 和 log 没有经过压缩
- 不支持读的线性一致性

而 ClickHouse-Keeper 存在着以下优点：

- 使用 C++ 开发，技术栈与 ClickHouse 统一
- 即可独立部署，又可集成到 ClickHouse 中
- 没有 zxid overflow 问题
- 读性能更好，写性能相当
- 支持对 snapshot 和 log 的压缩和校验
- 支持读写的线性一致性

三、配置方式

相较之前的集群配置，没有太大区别，而 ClickHouse-Keeper 仅当<keeper_server>标签存在于配置中时才会运行，配置模板如下：

```

<keeper_server>
  <tcp_port>9181</tcp_port>
  <server_id>1</server_id>
  <log_storage_path>/var/lib/clickhouse/coordination/log</log_storage_path>
  <snapshot_storage_path>/var/lib/clickhouse/coordination/snapshots</snapshot_storage_path>

  <coordination_settings>
    <operation_timeout_ms>10000</operation_timeout_ms>
    <session_timeout_ms>30000</session_timeout_ms>
    <raft_logs_level>trace</raft_logs_level>
  </coordination_settings>

  <raft_configuration>
    <server>
      <id>1</id>
      <hostname>ck1</hostname>
      <port>9234</port>
    </server>
    <server>
      <id>2</id>
      <hostname>ck2</hostname>
      <port>9234</port>
    </server>
    <server>
      <id>3</id>
      <hostname>ck3</hostname>
      <port>9234</port>
    </server>
  </raft_configuration>
</keeper_server>

```

四、启动命令

```
clickhouse-keeper --config /etc/your_path_to_config/config.xml
```

五、参数说明

- tcp_port: 客户端连接的端口 (ZooKeeper 的默认值为 2181)
- tcp_port_secure: 用于客户端和 keeper-server 之间 SSL 连接的安全端口
- server_id: Keeper 集群的每个节点唯一 ID
- log_storage_path: 日志路径, 最好将日志存储在 IO 性能强设备
- snapshot_storage_path: 快照路径

<keeper_server>.<coordination_settings>部分

- operation_timeout_ms: 单个客户端操作超时配置
- min_session_timeout_ms: 客户端会话最小超时
- session_timeout_ms: 客户端会话的最大超时
- dead_session_check_period_ms: 检查过期会话并将其删除的频率
- heart_beat_interval_ms: leader 向 follower 发送心跳的频率
- election_timeout_lower_bound_ms: 如果 follower 在此时间间隔内没有收到 leader 的心跳, 则可以发起 leader 选举
- rotate_log_storage_interval: 在单个文件中存储多少日志记录
- reserved_log_items: 压缩前要存储多少日志记录
- snapshot_distance: 创建快照的频率
- snapshots_to_keep: 快照保留份数
- max_requests_batch_size: 在将请求发送到 RAFT 之前, 请求中的最大批处理请求大小

- raft_logs_level: raft 日志记录级别
- auto_forwarding: 允许将 follower 的写请求转发给 leader
- shutdown_timeout: 等待完成内部连接并关闭的时间

<keeper_server>.<raft_configuration>部分

- Id: 集群的每个节点 ID
- Hostname: 服务器主机名
- Port: 服务器监听端口

六、状态检查

6.1、ruok 主要用于诊断 Keeper 的客户端/服务器

命令如下：

```
echo ruok | nc 127.0.0.1 9181 成功则返回 imok
```

6.2、确保 ClickHouse-Server 知道 keeper 集群，我们可以查询 system.zookeeper 表

```
ck1 :) SELECT * FROM system.zookeeper WHERE path = '/' FORMAT Vertical
SELECT *
FROM system.zookeeper
WHERE path = '/'
FORMAT Vertical

Query id: b300ab49-3a31-4756-9a6e-6ae5be6c4bc7

Row 1:
name:          clickhouse
value:
czxid:         6
mzxid:         6
ctime:         2022-03-01 17:29:56
mtime:         2022-03-01 17:29:56
version:       0
cversion:      1
aversion:      0
ephemeralOwner: 0
dataLength:    0
numChildren:   1
pzxid:         7
path:          /

1 rows in set. Elapsed: 0.013 sec.
```

注：输出以上信息代表安装成功

七、如何迁移 Zookeeper 至 ClickHouse-Keeper

既然 ClickHouse-Keeper 优势这么明显，那么如何将 Zookeeper 中的数据迁移到 ClickHouse-Keeper 中呢？官方提供了迁移工具 ClickHouse-Keeper-Converter，它能够将 Zookeeper 中的数据 dump 成 ClickHouse-Keeper 能够加载的 snapshot。

迁移步骤如下

- 停止所有 Zookeeper 节点
- 找到 Zookeeper-leader 节点并再次停止(这一步是为了让 leader 节点生成一份 snapshot)
- 运行 ClickHouse-Keeper-Converter，生成 Keeper 的 snapshot 文件

命令参考：

```
clickhouse-keeper-converter --zookeeper-logs-dir /var/lib/zookeeper/version-2 --  
zookeeper-snapshots-dir /var/lib/zookeeper/version-2 --output-dir  
/path/to/clickhouse/keeper/snapshots
```

- 启动 ClickHouse-Keeper，使其加载上一步中的 snapshot

参考文献：<https://clickhouse.com/docs/en/operations/clickhouse-keeper/>

5.3 ClickHouse 物化视图插入时间变为“1970-01-01 08:00:00”

问题复盘

作者：大睿

物化视图使用 to 的方式写入到存储表中，即如下：

```
CREATE MATERIALIZED VIEW  
[IF NOT EXISTS]  
[db.]table_name  
[ON CLUSTER] TO  
[db.]nameAS  
SELECT ...
```

指定了存储的表，所以物化视图的创建也不需要指定 `engine`，在查询中，查物化视图和查实际的存储表得到一样的数据，因为都是来自于同一份存储数据。

物化视图是计算每次写入原表的数据，经过聚合之后写入到目标表。比如，有按照 1s 一次记录的明细表，同时需要按照分钟级做数据的聚合统计 `pv`（类似的需要），则可以通过创建物化视图的方式将聚合后的数据写到 1min 的表中（这种感觉有点像触发器）

范例

1s 记录的明细表

```
CREATE TABLE dba_test.t_1s
(
    `ctime` DateTime64(0),
    `pv` Int64
)
ENGINE = MergeTree
PARTITION BY toDate(ctime)
ORDER BY ctime
SETTINGS index_granularity = 8192
```

1min 记录的聚合数据

```
CREATE TABLE dba_test.t_1m
(
    `ctime` DateTime64(0),
    `pv` Int64
)
ENGINE = SummingMergeTree
PARTITION BY toDate(ctime)
ORDER BY ctime
SETTINGS index_granularity = 8192
```

物化视图 t_1m_mv，查询条件是从 1s 的表(t_1s)，按照分钟级(toStartOfMinute)聚合查询结果，重新写入到 1min 的表(t_1m)中

物化视图

```
CREATE MATERIALIZED VIEW dba_test.t_1m_mv TO dba_test.t_1m
(
    `toStartOfMinute(ctime)` DateTime,
    `pv` Int64
) AS
SELECT
    toStartOfMinute(ctime),
    sum(pv) AS pv
FROM dba_test.t_1s
GROUP BY ctime
```

写入测试

```
dba-clickhouse-001 :) insert into t_1s values('2022-01-01 00:10:01',1),('2022-01-01 00:10:01',1),('2022-01-01 00:20:01',2),('2022-01-01 00:20:01',2),('2022-01-01 00:30:01',3);
```

```
INSERT INTO t_1s VALUES
```

```
Query id: 0bf16844-0123-4e25-a3d4-f9b5a5c8db37
```

Ok.

5 rows in set. Elapsed: 0.003 sec.

dba-clickhouse-001 :) select * from t_1s;

```
SELECT *
FROM t_1s
```

Query id: cb442100-37a6-4de7-b6f3-f80f084710dc

	ctime	pv
2022-01-01 00:10:01	1	
2022-01-01 00:10:01	1	
2022-01-01 00:20:01	2	
2022-01-01 00:20:01	2	
2022-01-01 00:30:01	3	

5 rows in set. Elapsed: 0.002 sec.

dba-clickhouse-001 :) select * from t_1m;

```
SELECT *
FROM t_1m
```

Query id: f9d2d05d-8ad7-44a4-b66a-ea8c3c758f1f

	ctime	pv
1970-01-01 08:00:00	9	

1 rows in set. Elapsed: 0.002 sec.

插入的时间竟然是 1970-01-01 08:00:00

开始验证是否是查询语句有误

检查物化视图中的查询结果是否符合预期

```
dba-clickhouse-001 :) SELECT
:-]      toStartOfMinute(ctime),
:-]      sum(pv) AS pv
:-] FROM dba_test.t_1s
:-] GROUP BY ctime;
```



```
SELECT
    toStartOfMinute(ctime),
    sum(pv) AS pv
FROM dba_test.t_1s
GROUP BY ctime
```

Query id: 1ecaf07e-c766-40b7-bfa2-0f87ee54abad

toStartOfMinute(ctime)	pv
2022-01-01 00:20:00	4
2022-01-01 00:30:00	3
2022-01-01 00:10:00	2

3 rows in set. Elapsed: 0.002 sec.

查询结果符合预期

直接通过 `insert ...select...` 方式确认下插入数据是否符合预期

```
dba-clickhouse-001 :) insert into t_1m SELECT
:-]    toStartOfMinute(ctime),
:-]    sum(pv) AS pv
:-] FROM dba_test.t_1s
:-] GROUP BY ctime;
```

```
INSERT INTO t_1m SELECT
    toStartOfMinute(ctime),
    sum(pv) AS pv
FROM dba_test.t_1s
GROUP BY ctime
```

Query id: 5db8279a-ffb1-4174-843c-80cee48b448c

Ok.

0 rows in set. Elapsed: 0.002 sec.

```
dba-clickhouse-001 :) select * from t_1m;
```

```
SELECT *
FROM t_1m
```

Query id: acd79ea7-dc82-49f1-bb71-430a05895f19

```

┌────────────────────────── ctime ─────────── pv ─┐
│ 1970-01-01 08:00:00 | 9 |                        │
└───────────────────────────────────────────────────┘

┌────────────────────────── ctime ─────────── pv ─┐
│ 2022-01-01 00:10:00 | 2 |                        │
│ 2022-01-01 00:20:00 | 4 |                        │
│ 2022-01-01 00:30:00 | 3 |                        │
└───────────────────────────────────────────────────┘

```

```
4 rows in set. Elapsed: 0.002 sec.
```

直接插入，数据正确，时间没有被转化。

可以确认物化视图的查询部分是没有问题，那只能是在写入的时候出现了问题，换个思路去想一下，时间戳的开始时间是 1970-01-01 00:00:00，而这里插入的时间是 1970-01-01 08:00:00 多了 8 小时，也就是说因为时区的原因导致时间推迟了。那会不会是因为插入的数据不规范，或者是“空”被转化了呢。

验证

```
dba-clickhouse-001 :) insert into t_1m values('',100);
```

```
INSERT INTO t_1m VALUES
```

Query id: af1785ef-dca1-467b-84c6-27f9da6547f6

Ok.

```
1 rows in set. Elapsed: 0.002 sec.
```

```
dba-clickhouse-001 :) select * from t_1m;
```

```
SELECT *
FROM t 1m
```

Query id: 34db2057-7274-4859-898e-6132f8df4465

```
|-----ctime-----pv-----|
| 1970-01-01 08:00:00 | 9 |
|-----|
|-----ctime-----pv-----|
| 2022-01-01 00:10:00 | 2 |
| 2022-01-01 00:20:00 | 4 |
| 2022-01-01 00:30:00 | 3 |
|-----|
|-----ctime-----pv-----|
| 1970-01-01 08:00:00 | 100 |
```

```
5 rows in set. Elapsed: 0.002 sec.
```

果然，当插入的数据为空的时候，时间被重置了。

对比下物化视图和目标的聚合表的结构

```
dba-clickhouse-001 :) desc t_1m;

DESCRIBE TABLE  t_1m

Query id: 96c6a5ca-e42a-47e1-8212-cbcfe6a6ffa4
```

name	type	default_type	default_expression	comment	codec_expression	ttl_expression
ctime	DateTime64(0)					
pv	Int64					

```
2 rows in set. Elapsed: 0.001 sec.

dba-clickhouse-001 :) desc t_1m_mv;

DESCRIBE TABLE  t_1m_mv

Query id: a258f6b5-f195-4386-a9bb-4ec86e7e9bd1
```

name	type	default_type	default_expression	comment	codec_expression	ttl_expression
toStartOfMinute(ctime)	DateTime					
pv	Int64					

```
2 rows in set. Elapsed: 0.001 sec.
```

聚合表时间字段名叫 ctime，物化视图的则是 toStartOfMinute(ctime)

重新调整物化视图的写法，并清理 t_1m 表中的数据

```
dba-clickhouse-001 :) show create table t_1m_mv\G
statement: CREATE MATERIALIZED VIEW dba_test.t_1m_mv TO dba_test.t_1m
(
    `ctime` DateTime,
    `pv` Int64
) AS
SELECT
    toStartOfTenMinutes(ctime) AS ctime,
    sum(pv) AS pv
FROM dba_test.t_1s
GROUP BY ctime

dba-clickhouse-001 :) insert into t_1s values('2022-01-01 00:10:01',1),('2022-01-01 00:10:01',1),('2022-01-01 00:20:01',2),('2022-01-01 00:20:01',2),('2022-01-
```

```
01 00:30:01',3);
```

```
INSERT INTO t_1s VALUES
```

```
Query id: 812d1bbd-55f3-4a8f-b9f7-bbbe93e694af
```

```
Ok.
```

```
5 rows in set. Elapsed: 0.003 sec.
```

```
dba-clickhouse-001 :) select * from t_1m;
```

```
SELECT *
FROM t_1m
```

```
Query id: 2d1a045a-4e53-4f94-bb6a-fe5e5d58f5c7
```

	ctime	pv
2022-01-01 00:10:00	2	
2022-01-01 00:20:00	4	
2022-01-01 00:30:00	3	

```
3 rows in set. Elapsed: 0.002 sec.
```

结论

物化视图的字段(t_1m_mv)要与目标表(t_1m)的字段名对齐

(表达不是很严谨，大概是上面的意思)

5.4 Zabbix 监控 TiDB（一）

作者：胡呈清

Zabbix Server 5.4

我们可以在 Zabbix 官网的“解决方案-数据库”页面中发现 Zabbix 对 TiDB 的监控方案：

<https://www.zabbix.com/cn/integrations/tidb>

监控逻辑：

1. 通过 HTTP agent 方式，Zabbix server 主动调用 TiDB 的监控接口（比如 <http://127.0.0.1:10080/metrics>）获取监控数据；
2. Zabbix 使用 Prometheus to JSON 功能（在 4.2 版本引入）对监控数据进行预处理，变成 JSON 格式；
3. 再通过 JSONPath 功能取出任何想要的指标，比如所有 query 的总响应时间。

上述所说的内容都已经做成了模板，安装好就能够使用（部分表达式有错误，后文会指出）。不过要求也比较严格：

Zabbix 版本要求：>=5.4 而 Zabbix 5.4 对操作系统也有要求：

- Zabbix Server RHEL8/CentOS8
- Zabbix agent CentOS6/7/8

Zabbix 的各版本兼容性较差，监控模板导出后基本无法导入到其他版本。如果要在其他版本使用，正确姿势是：

- 版本不低于 4.2（4.0.x 最新版本不支持）；
- 参考 5.4 版本上默认的模板，手工创建监控模板和其中的 item、trigger 等。

我们先来看看 Zabbix 5.4 上的使用方法。

1. 安装

参考官方文档即可，步骤非常简单：

https://www.zabbix.com/cn/download?zabbix=5.4&os_distribution=centos&os_version=8&db=mysql&ws=apache

2. 添加 host

路径：Configuration > Host 注意需要手工添加红框处的接口，其他版本默认就有：

Hosts

All hosts / 10.186.64.59 Enabled ZBX Items 157 Triggers 30 Graphs 13 Discovery rules 13 Web scenarios

Host Templates 3 IPMI Tags Macros Inventory Encryption Value mapping

* Host name 10.186.64.59

Visible name 10.186.64.59

* Groups Linux servers Zabbix servers Select
type here to search

Interfaces	Type	IP address	DNS name	Connect to	Port	Default
Agent		10.186.64.59		IP DNS	10050	☒ Remove

Add

Description

Monitored by proxy (no proxy)

Enabled ☒

Update

Clone

Full clone

Delete

Cancel

3. 关联模板

给 host 关联 TiDB 监控模板：默认就有 TiDB by HTTP 、TiDB PD by HTTP 、TiDB TiKV by HTTP 这 3 个模板，关联到 host 即可（这个 host 装了哪些就关联哪些）

ZABBIX << >>

Monitoring

Inventory

Reports

Configuration

Host groups

Templates

Hosts

Maintenance

Actions

Event correlation

Discovery

Services

Administration

Hosts

All hosts / 10.186.64.59 Enabled ZBX Items 157 Triggers 30 Graphs 13 Discovery rules 13 Web scenarios

Host Templates 3 IPMI Tags Macros Inventory Encryption Value mapping

Linked templates

Name	Action
Linux by Zabbix agent	Unlink Unlink and clear
TiDB by HTTP	Unlink Unlink and clear
TiDB TiKV by HTTP	Unlink Unlink and clear

Link new templates

type here to search Select

Update

Clone

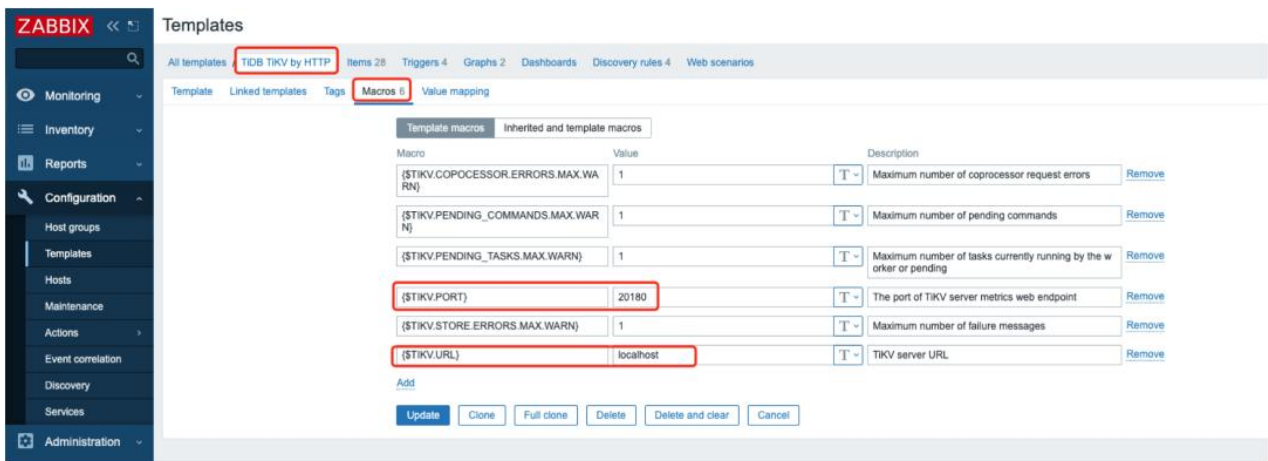
Full clone

Delete

Cancel

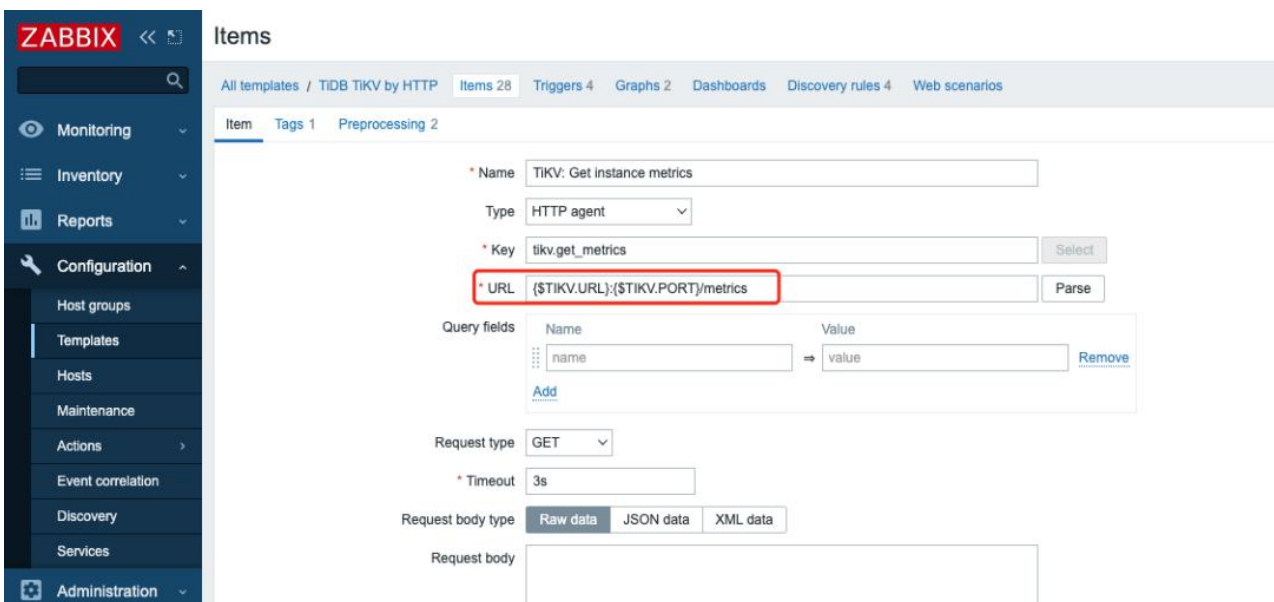
4. 修改宏

模板中的 {STIKV.URL} 、 {STIKV.PORT} 这两个宏需要修改：



{STIKV.PORT} 指定对应的端口就行，如果使用的是默认端口就不需要修改了；
 {STIKV.URL} 在模板中的值是 localhost，它在 master item 中使用（其他 item 类型都是 dependent item）：

Wizard	Name	Triggers	Key	Interval	History	Trends	Type	Status	Tags
☐	*** TiKV: Get instance metrics: TiKV: Available size		tkv.store_size.available	7d	365d		Dependent item	Enabled	Application: TiKV node
☐	*** TiKV: Get instance metrics: TiKV: Bytes read		tkv.engine_flow_bytes.read	7d	365d		Dependent item	Enabled	Application: TiKV node
☐	*** TiKV: Get instance metrics: TiKV: Bytes write		tkv.engine_flow_bytes.write	7d	365d		Dependent item	Enabled	Application: TiKV node
☐	*** TiKV: Get instance metrics: TiKV: Capacity size		tkv.store_size.capacity	7d	365d		Dependent item	Enabled	Application: TiKV node
☐	*** TiKV: Get instance metrics: TiKV: Coprocessor: Errors, rate	Triggers 1	tkv.coprocessor_request_error.rate	7d	365d		Dependent item	Enabled	Application: TiKV node
☐	*** TiKV: Get instance metrics: TiKV: Coprocessor: Requests, rate		tkv.coprocessor_request.rate	7d	365d		Dependent item	Enabled	Application: TiKV node
☐	*** TiKV: Get instance metrics: TiKV: Coprocessor: Response size, rate		tkv.coprocessor_scan_keys.rate	7d	365d		Dependent item	Enabled	Application: TiKV node
☐	*** TiKV: Get instance metrics: TiKV: Coprocessor: RocksDB ops, rate		tkv.coprocessor_rocksdb_perf.rate	7d	365d		Dependent item	Enabled	Application: TiKV node
☐	*** TiKV: Get instance metrics: TiKV: CPU util		tkv.cpu.util	7d	365d		Dependent item	Enabled	Application: TiKV node
☐	*** TiKV: Get instance metrics: TiKV: Regions, count		tkv.get_metrics	1m	0		HTTP agent	Enabled	Application: Zabbix ra...
☐	*** TiKV: Get instance metrics: TiKV: Regions, count		tkv.region_count	7d	365d		Dependent item	Enabled	Application: TiKV node
☐	*** TiKV: Get instance metrics: TiKV: Regions, leader		tkv.region_leader	7d	365d		Dependent item	Enabled	Application: TiKV node

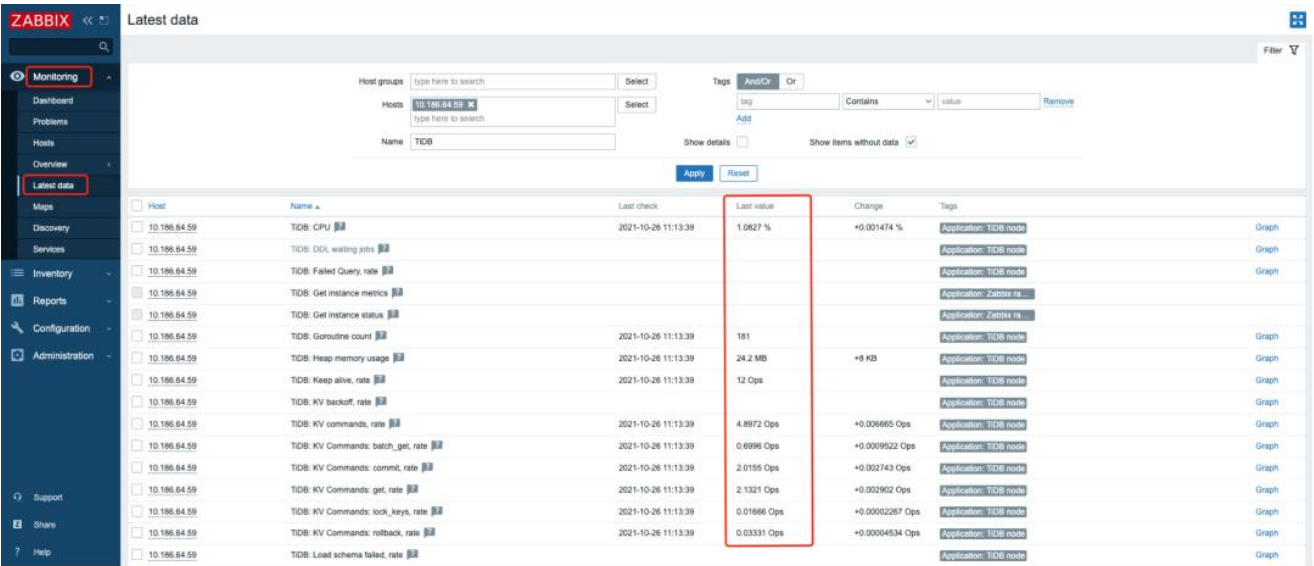


它是通过 HTTP agent 的方式去获取的监控指标，URL 就是用 {STIKV.URL}:{STIKV.PORT}/metrics 表示的，我们要实现的是每个 TiDB、TiKV、PD 都替换成对应的接口，比如 10.186.64.59:20180/metrics，只需要用 {HOST.IP} 替换 {STIKV.URL} 这个宏即可：

- {STIKV.URL} 是个自定义宏，所以需要在模板的宏定义中指定值（注意这个值只能是确定的值，而不能继续用宏表示）；
- {HOST.IP} 是个内部宏，会自动取对应 host 的 IP，所以不再需要指定值。

5. 查看监控数据

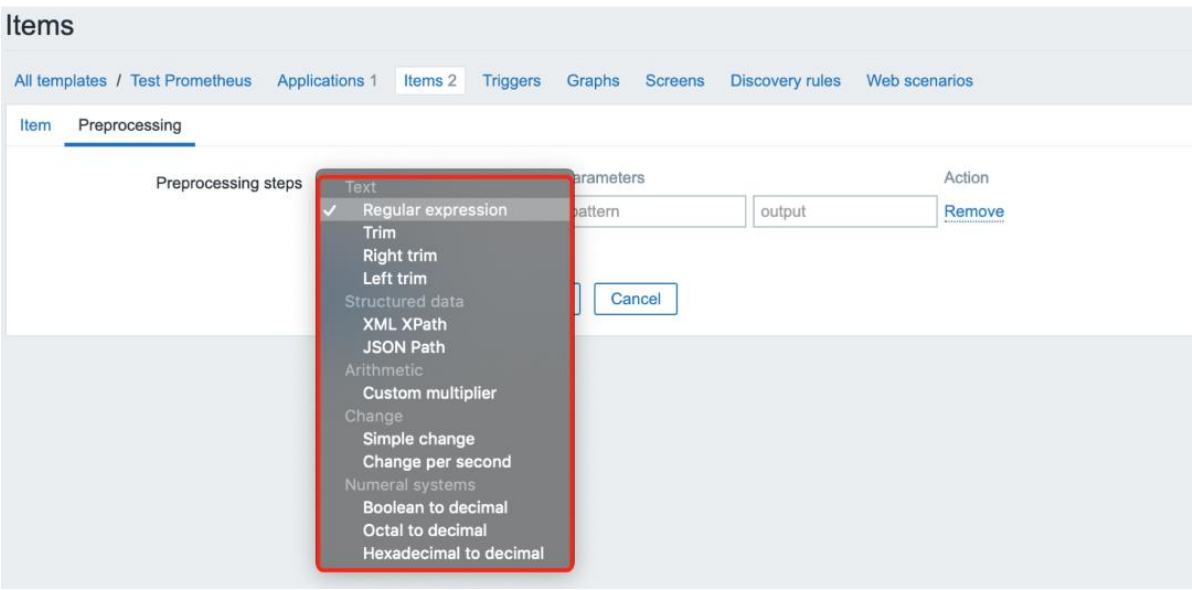
在 Monitoring-Latest data 页面中查看到有监控数据就说明成功了：



5.5 Zabbix 监控 TiDB (二)

作者：胡呈清

如果要使用 Zabbix 监控使用 TiDB，需使用 HTTP agent，主动调用 TiDB 监控接口获取监控数据，然后配置数据预处理：选择使用 Prometheus pattern 或者 Prometheus to JSON 方法，但是这两个功能是在 Zabbix4.2 中加入的，Zabbix4.0.x 没有这个功能（即使是最新的 zabbix4.0.35）：



所以在没法升级到 Zabbix5.4 时，我们可以在大于 4.2 的版本上手工创建监控模板，以下演示环境为 Zabbix5.0.5。

TiDB 监控接口

在开始前，需要先了解 TiDB 的监控接口：<https://docs.pingcap.com/zh/tidb/v5.1/tidb-monitoring-api>

示例：

```
curl http://127.0.0.1:10080/metrics > /tmp/tidb_metrics
```

参考 TiDB 官网上的告警规则（<https://docs.pingcap.com/zh/tidb/v5.1/alert-rules>）中的第一条告警规则：

```
increase(tidb_session_schema_lease_error_total{type="outdated"}[15m]) > 0
```

这个是 prometheus 的语法，我们只需要知道 tidb_session_schema_lease_error_total 是 metrics name 就行，然后我们去监控数据中找到这个 metric（TiDB 的不同版本 metric 可能不一样，示例中的 metric 在 4.0.10 中就没有，在 5.1 中有）：

```
[root@localhost tmp]# grep tidb_session_schema_lease_error_total /tmp/tidb_metrics
# HELP tidb_session_schema_lease_error_total Counter of schema lease error
# TYPE tidb_session_schema_lease_error_total counter
tidb_session_schema_lease_error_total{type="outdated"} 2
```

其数据格式为：

说明

以“# HELP”开头，是对这个 metric 的说明

类型

以“# TYPE”开头，表示这个 metric 的数据类型，一共有 4 种：

指标类型	描述	说明
Counter	只增不减的计数器，其值只能增加或在重启时重置为零	例如，您可以使用计数器来表示服务的总请求数，已完成的任務或错误总数
Gauge	用来存放一个可以任意变大变小的数值	例如温度或当前内存使用情况，或者运行的goroutine数量
Histogram	一段时间范围内对数据进行采样	通常用它计算分位数的直方图
Summary	客户端定义的数据分布统计图	统计事件发生的次数或者大小，以及其分布情况

数据

这里要注意的就是上述示例中的“type”标签，有些 metric 会有多个标签，我们可以根据标签取其中的几个（比如需求是计算所有 Select、Update、Insert 命令的总耗时）：

```
tidb_server_handle_query_duration_seconds_sum{sql_type="Begin"} 0
tidb_server_handle_query_duration_seconds_sum{sql_type="Commit"} 0
```

```

tidb_server_handle_query_duration_seconds_sum{sql_type="Delete"} 0
tidb_server_handle_query_duration_seconds_sum{sql_type="Execute"} 0
tidb_server_handle_query_duration_seconds_sum{sql_type="Insert"} 0
tidb_server_handle_query_duration_seconds_sum{sql_type="Replace"} 0
tidb_server_handle_query_duration_seconds_sum{sql_type="Rollback"} 0
tidb_server_handle_query_duration_seconds_sum{sql_type="Select"} 0
tidb_server_handle_query_duration_seconds_sum{sql_type="Set"} 0
tidb_server_handle_query_duration_seconds_sum{sql_type="Show"} 0
tidb_server_handle_query_duration_seconds_sum{sql_type="Update"} 0
tidb_server_handle_query_duration_seconds_sum{sql_type="Use"} 0
tidb_server_handle_query_duration_seconds_sum{sql_type="general"} 0
tidb_server_handle_query_duration_seconds_sum{sql_type="internal"} 12260.158577597258

```

接下来介绍如何在 Zabbix 中手工添加 TiDB 监控。

创建 items

item 就是监控项，那我们要监控 TiDB 的哪些指标呢？可以参考官方告警规则，需要告警的监控项一定是最优先的。

1. 创建 master item

参数如下图，红色标记是重点。这个 item 定义了调用 TiDB Server 的 metrics 接口获取到所有监控指标的数据：

The screenshot shows the Zabbix web interface for creating a new item. The left sidebar contains navigation menus for Monitoring, Inventory, Reports, Configuration, and Administration. The main panel is titled 'Items' and shows the configuration for a new item named 'TiDB: Get instance metrics'. The 'Type' is set to 'HTTP agent' and the 'URL' is set to '(HOST.IP):(\$TIDB.PORT)/metrics', both highlighted with red boxes. Other settings include 'Key' as 'tidb.get_metrics', 'Request type' as 'GET', 'Timeout' as '3s', and 'Required status codes' as '200'. The 'Follow redirects' checkbox is checked, and the 'Retrieve mode' is set to 'Body'. The 'Convert to JSON' checkbox is unchecked, and the 'HTTP proxy' field is empty. The 'HTTP authentication' is set to 'None', and the 'SSL verify peer' and 'SSL verify host' checkboxes are unchecked.

The screenshot shows the Zabbix Administration interface. On the left is a sidebar with navigation links: Services, Administration, Support, Share, Help, User settings, and Sign out. The main content area is for configuring an item. Key settings include:

- Type of information:** Set to 'Text' (highlighted with a red box).
- Update interval:** Set to '1m'.
- Custom intervals:** A table with columns Type, Interval, Period, and Action. It shows a 'Flexible' type with a 'Scheduling' interval of '50s' and a period of '1-7,00:00-24:00'. There is an 'Add' link below.
- History storage period:** Set to 'Do not keep history' (highlighted with a red box).
- Enable trapping:** An unchecked checkbox.
- New application:** An empty text input field (highlighted with a green box).
- Applications:** A dropdown menu currently showing '-None-'.
- Populates host inventory field:** A dropdown menu currently showing '-None-'.
- Description:** An empty text input field.

注意取到的数据格式为 Text，需要在“Preproceession”(数据预处理)中定义转化成 JSON 格式：

The screenshot shows the Zabbix Items configuration page. The left sidebar has links for Monitoring, Inventory, Reports, and Configuration. The main content area is titled 'Items' and shows a list of items. One item is selected, and its configuration is shown. The 'Preprocessing steps' section has a dropdown menu with '1: Prometheus to JSON' selected (highlighted with a red box). The parameters for this step are set to '<metric name>[<label name>=<label value>,<label value>,...] == <value>'. There are buttons for 'Add', 'Update', 'Clone', 'Test', 'Delete', and 'Cancel'.

2. 创建普通 item

因为 master item 获取的是所有 metrics，所以需要创建子 item 从 master item 中取出单个 metrics：

The screenshot shows the Zabbix Items configuration page. The item is named "TiDB: Schema lease 'outdate' errors, rate". The type is set to "Dependent item". The key is "tidb.session_schema_lease_error.outdate.rate". The master item is "TiDB by HTTP: TiDB: Get instance metrics". The type of information is "Numeric (float)". The history storage period is "Do not keep history" and the trend storage period is "Do not keep trends". The show value is set to "As is". The new application is empty. The applications list is empty. The populates host inventory field is set to "-None-". The description is "The number of schema lease errors per second. 'outdate' errors means that the schema cannot be updated, which is a more serious error and triggers an alert." The item is enabled.

关键在于 “Preprocessing”(数据预处理)中定义 JSONPath，如下图所示：

1. JSONPath 表达式: `$(?(@.name=="tidb_session_schema_lease_error_total" && @.labels.type == "outdated")).value.first()` 表示取 metric name 为 `tidb_session_schema_lease_error_total` 并且其 type 标签为 "outdated" 的值。注意：5.4 的模板中的这个表达式是错误的，可以用 test 功能检测。
2. 因为这个 metric 的类型是 Counter（累计值），所以用 "Change per second" 方法取得其平均每秒的增长值（注意：这是个平均值）。如果类型是 Gauge（瞬时值）就不需要这步处理了。

The screenshot shows the Zabbix Items configuration page with the preprocessing steps section. Step 1 is "JSONPath" with the parameter `$(?(@.name=="tidb_session_schema_lease_error_total" && @.labels.type == "outdated")).value.first()`. Step 2 is "Change per second". The custom on fail actions are "Discard value", "Set value to", and "Set error to".

创建 trigger

trigger 就是定义当指定 item 的值达到什么条件，就触发其状态变成异常。先参考 TiDB 官方告警规则的 Prometheus 语法：`increase(tidb_session_schema_lease_error_total{type="outdated"}[15m]) > 0`

increase([15m]) 函数表示在 15 分钟内的增长值，整个表达式含义：为 15 分钟内的增长大于 0。由于我们在 item 中定义的是 tidb_session_schema_lease_error_total 每秒增长量，所以当一段时间内平均每秒增长量的最大值大于 0 时，说明发生了 error，就需要触发告警，触发器表达式为：{TiDB by HTTP:tidb.session_schema_lease_error.outdate.rate.max(15m)}>0

ZABBIX << Triggers

All templates / TiDB by HTTP Applications Items 36 Triggers 10 Graphs Screens Discovery rules 6 Web scenarios

Trigger Tags Dependencies

* Name: TiDB: Schema lease errors happened for 15m

Operational data:

Severity: Not classified Information Warning Average **High** Disaster

* Expression: {TiDB by HTTP:tidb.session_schema_lease_error.outdate.rate.max(15m)}>0

Expression constructor

OK event generation: Expression Recovery expression None

PROBLEM event generation mode: Single Multiple

OK event closes: All problems All problems if tag values match

Allow manual close: ☐

URL:

Description: The latest schema information is not reloaded in TiDB within one lease.

Enabled: ☒

Update Clone Delete Cancel

附录

数据预处理-JSONPath

示例数据:

```
[
  {
    "name": "tidb_server_handle_query_duration_seconds_sum",
    "value": "100",
    "line_raw": "tidb_server_handle_query_duration_seconds_sum{sql_type=\"Begin\"} 0",
    "labels": {
      "sql_type": "Begin"
    },
    "type": "untyped"
  },
  {
    "name": "tidb_server_handle_query_duration_seconds_sum",
    "value": "50",
```

```

    "line_raw": "tidb_server_handle_query_duration_seconds_sum{sql_type=\"Commit\"} 0",
    "labels": {
      "sql_type": "Commit"
    },
    "type": "untyped"
  }
]

```

表达式: `$[?(@.name=="tidb_server_handle_query_duration_seconds_sum")].value.sum()`

含义: 所有命令的总耗时

测试结果: 150

表达式: `$[?(@.name=="tidb_server_handle_query_duration_seconds_sum" && @.labels.sql_type=="Commit")].value.first()`

含义: 所有 Commit 命令的总耗时

测试结果: 50

表达式: `$[?(@.name=="tidb_server_handle_query_duration_seconds_sum" && @.labels.type =~ "Begin|Commit")].value.sum()`

含义: 所有 Begin、Commit 命令的总耗时 (其他类型的不计算)

测试结果: 150

可计算 item

文档: <https://www.zabbix.com/documentation/5.0/zh/manual/config/items/itemtypes/calculated>

基于其它监控项来创建可计算监控项, 指定新创建的 item 为 "Calculated" 即可, 以下示例先创建了两个 item: query_duration_sum、query_duration_count, 两者相除得到 Query 的平均响应时间:

Items

All templates / TiDB by HTTP Applications Items 35 Triggers 6 Graphs Screens Discovery rules 6 Web scenarios

Item Preprocessing

* Name

Type Calculated

* Key Select

* Formula

Type of information Numeric (float)

Units

* Update interval

Custom intervals

Type	Interval	Period	Action
Flexible Scheduling	50s	1-7,00:00-24:00	Remove

[Add](#)

* History storage period Do not keep history Storage period

* Trend storage period Do not keep trends Storage period

Show value As is show value mappings

New application

Trigger 表达式技巧

1. 内存使用量

表达式: `{TiDB by HTTP:tidb.heap_bytes.min(5m)}>{$TIDB.HEAP.USAGE.MAX.WARN}`

含义:

tidb.heap_bytes 是 key 名, 对应的是 TiDB 监控中的 go_memstats_heap_inuse_bytes 指标, 这是个 Gauge 类型 (即瞬时值); 5 分钟内, 使用的内存最小值超过指定阈值 (也就是持续 5 分钟内, 使用的内存都超过了阈值), 就报警

2.99 响应时间

如何计算 99% 的 SQL 的响应时间?

TiDB 监控接口提供的是直方图数据, 类型为 histogram, prometheus 可以用 histogram_quantile() 函数处理: histogram_quantile(0.99, sum(rate(tidb_server_handle_query_duration_seconds_bucket[1m])) BY (le, instance)) > 1

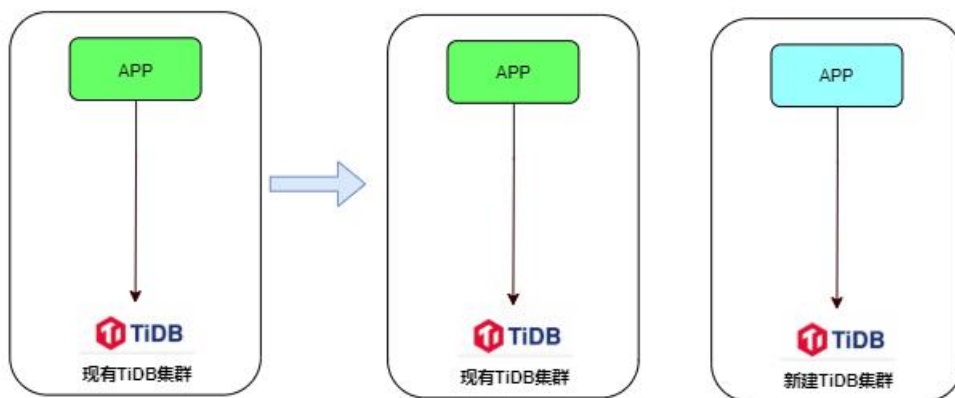
但是 zabbix 处理不了, 可以计算平均响应时间 (用附录中的可计算 item 实现)。

5.6 TiDB 上百 T 数据拆分实践

作者：杨家鑫

背景

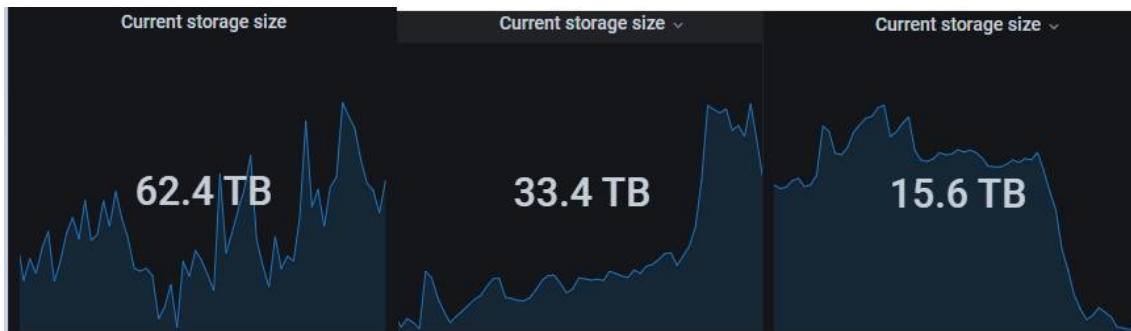
提高 TiDB 可用性，需要把多点已有上百 T TiDB 集群拆分成 2 套



挑战

1. 现有需要拆分的 12 套 TiDB 集群的版本多(4.0.9、5.1.1、5.1.2 都有)，每个版本拆分方法存在不一样
2. 其中 5 套 TiDB，数据量均超过 10T、最大的 TiDB 集群目前数据量 62T、单 TiDB 集群备份集大，消耗大量磁盘空间和带宽资源

空间最大3套集群



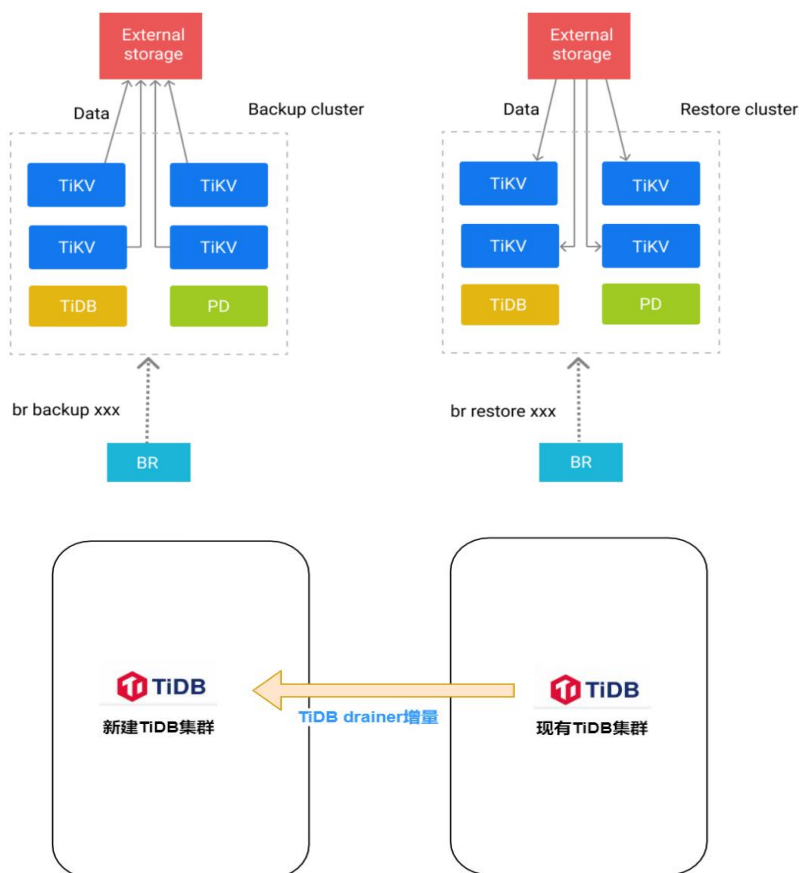
3. tidb 使用方式多样（每种方式拆分方法不同），有直接读写 tidb，也有 mysql->tidb 汇总分析 查询，也有 tidb->cdc->下游 hive
4. 全量备份 TiDB 在业务高峰期是否会产生性能影响
5. 大数据量的拆分数据的一致性保证

方案

目前 TiDB 官方提供的同步工具有：

- DM 全量+增量（该方法无法用于 tidb->tidb，适用于 MySQL->TiDB）

- BR 全量物理备份+CDC 增量同步（CDC 同步在 tidb、tikv 节点 OOM 后修复成本高<https://github.com/pingcap/tiflow/issues/3061>）
- BR 全量物理备份+binlog 增量（类似于 MySQL 记录所有变更的 binlog 日志，TiDB binlog 由 Pump(记录变更日志)+Drainer(回放变更日志)组成，我们采用该方法进行全量+增量同步拆分）



备份与恢复工具 BR: <https://docs.pingcap.com/zh/tidb/stable/backup-and-restore-tool>

TiDB Binlog: <https://docs.pingcap.com/zh/tidb/stable/tidb-binlog-overview>

因 TiDB 拆分 BR 全量物理备份+binlog 增量涉及周期长，我们分为 4 个阶段进行

第一阶段

1、清理现有 TiDB 集群无用数据

按月分表 tidb 库有无用的表，如 3 个月前的 xxxx 日志表

2、升级 GZ 现有 15 套 TiDB 集群（12 套 TiDB 集群需要 1 分为 2）版本至 5.1.2

趁这次拆分统一 GZ tidb 版本，解决挑战 1

第二阶段

1、新机器部署好相同版本 5.1.2 TiDB 集群

set @@global.tidb_analyze_version = 1;

2、目的端，源端所有 tikv tiflash 挂载好 NFS，pd 节点上安装好 BR

External storage 采用腾讯云 NFS 网盘，保障 tikv 备份目的端和还原全量来源端都能在同一目录，NFS 网盘

空间自动动态增加+限速备份以应对挑战 2

3、独立3台机器部署好 12 套 TiDB 集群

pump 收集 binlog(端口区分不同 TiDB 集群) pump, drainer 采用独立16C, 32G 机器保障增量同步最大性能。注意：为保障 tidb 计算节点的可用性，需设置 ignore-errorbinlog 关键参数

(<https://docs.pingcap.com/zh/tidb/v5.1/tidb-binlog-deployment-topology>)

```
server_configs:
tidb:
binlog.enable: true
binlog.ignore-error: true
```

4、修改 pump 组件 GC 时间为 7 天

binlog 保留 7 天保障全量备份->到增量同步过程能接上

```
pump_servers:
- host: xxxxx
config:
gc: 7
#需 reload 重启 tidb 节点使记录 binlog生效
```

5、备份 TiDB 集群全量数据至NFS Backup & Restore 常见问题

(<https://docs.pingcap.com/zh/tidb/v5.1/backup-and-restore-faq>)

注意：每个 TiDB 集群在同一个 NFS 建不同备份目录

注意：源老TiDB 集群分别限速(备份前后对读写延迟时间基本无影响)进行错峰全量备份(存在之前 多个 TiDB 集群同时备份把 NFS 3Gbps网络带宽打满情况)以减轻对现有 TiDB 读写、NFS 的压力以应 对挑战

```
mkdir -p /tidbbr/0110_dfp
chown -R tidb.tidb /tidbbr/0110_dfp
#限速进行全业务应用库备份
./br backup full \
--pd "xxxx:2379" \
--storage "local:///tidbbr/0110_dfp" \
--ratelimit 80 \
--log-file /data/dbatemp/0110_backupdfp.log
#限速进行指定库备份
./br backup db \
--pd "xxxx:2379" \
--db db_name \
--storage "local:///tidbbr/0110_dfp" \
--ratelimit 80 \
--log-file /data/dbatemp/0110_backupdfp.log
```

12.30 号 45T TiDB 集群全量备份耗时 19h, 占用空间 12T

[2021/12/30 09:33:23.768 +08:00] [INFO] [collector.go:66] ["Full backup succes s

```
summary"] [total-ranges=1596156] [ranges-succeed=1596156] [ranges-failed=0] [backup-
checksum=3h55m39.743147403s] [backup-fast-checksum=409.352223ms] [bac kup-total-
ranges=3137] [total-take=19h12m22.227906678s] [total-kv-size=65.13T B] [average-
speed=941.9MB/s] ["backup data size(after compressed)"=12.46TB] [B
ackupTS=430115090997182553] [total-kv=337461300978]
```

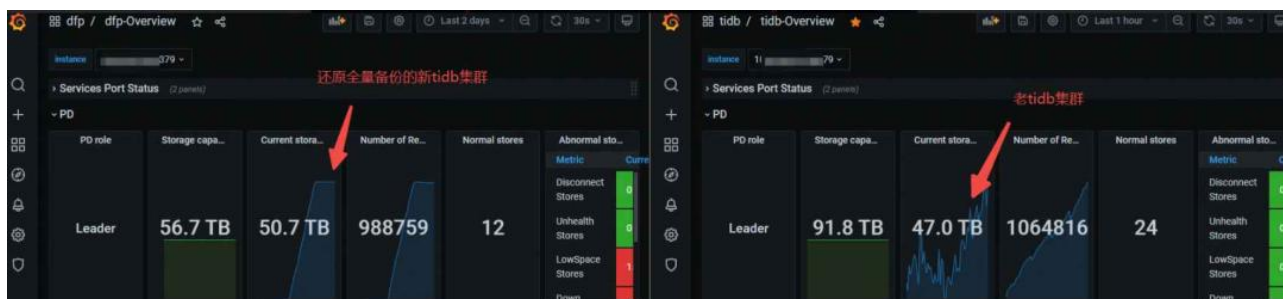
6、每个新建 TiDB 集群单独同步老TiDB 集群用户密码信息

注意：BR 全量备份不备份 tidb mysql 系统库，应用、管理员用户密码信息可用开源 pt-toolkit工具 包 pt-show-grants 导出

7、恢复 NFS 全量备份至新 TiDB 集群

注意：新 TiDB 集群磁盘空间需充裕，全量备份还原后新 TiDB 集群占用空间比老TiDB 集群多几个 T，和官方人员沟通是由于还原时生成 sst 的算法是 lz4，导致压缩率没有老TiDB 集群高

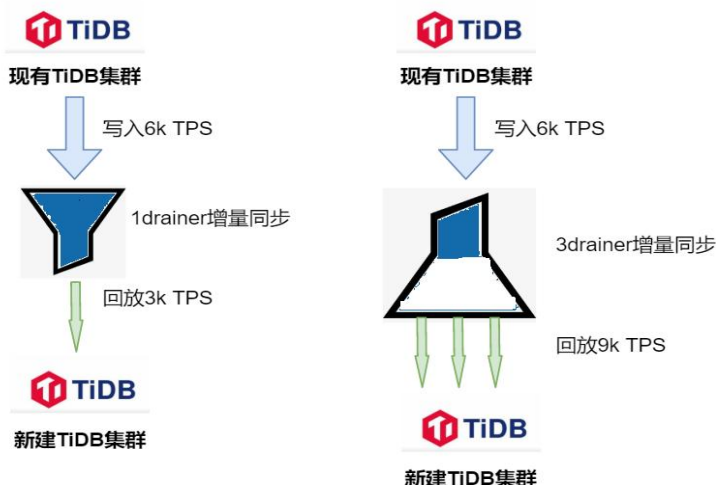
注意：tidb_enable_clustered_index, sql_mode 新老TiDB 集群这 2 参数必须一致



8、tiup 扩容 drainer 进行增量同步

扩容前确认下游 checkpoint 信息不存在或已清理，如果下游之前接过 drainer，相关位点在目标端 tidb_binlog.checkpoint 表中，重做的时候需要清理。

注意：因源最大TiDB 集群长期平均写入TPS 在 6k 左右，在增大worker-count 回放线程数后，尽管目的域名解析到 3 个 tidb 节点，单个 drainer 增量还是无法追上延迟（回放速度最高在 3k TPS），后和 TiDB 官方沟通改成按 3 个 drainer（不同 drainer 同步不同库名）并行增量同步延迟追上（3 个 drainer 增量让“漏斗”没有堆积，源流入端数据能及时到达目标流出端）



注意：多个 drainer 并行增量必须指定目的端 checkpoint.schema 为不同库 drainer 配置说明

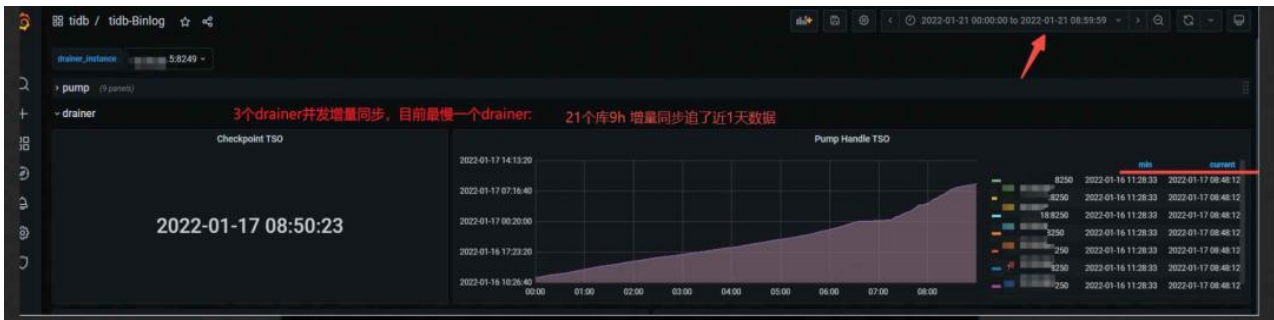
```
# 从备份文件中获取全量备份开始时的位点 TSO
grep "BackupTS=" /data/dbatemp/0110_backupdfp.log
430388153465177629
# 第一次一个 drainer 进行增量同步关键配置
drainer_servers:
- host: xxxxxx
commit_ts: 430388153465177629
deploy_dir: "/data/tidb-deploy/drainer-8249"
config:
syncer.db-type: "tidb"
syncer.to.host: "xxxmall.db.com"
syncer.worker-count: 550 1516

# 第二次多个 drainer 进行并行增量同步
drainer_servers:
- host: xxxxxx
commit_ts: 430505424238936397 # 该位点 TSO 为从第一次 1 个 drainer 增量停止后目的端 checkpoint
表中的 Commit_Ts
config:
syncer.replicate-do-db: [db1,db2,...]
syncer.db-type: "tidb"
syncer.to.host: "xxxmall.db.com"
syncer.worker-count: 550
syncer.to.checkpoint.schema: "tidb_binlog2"
```

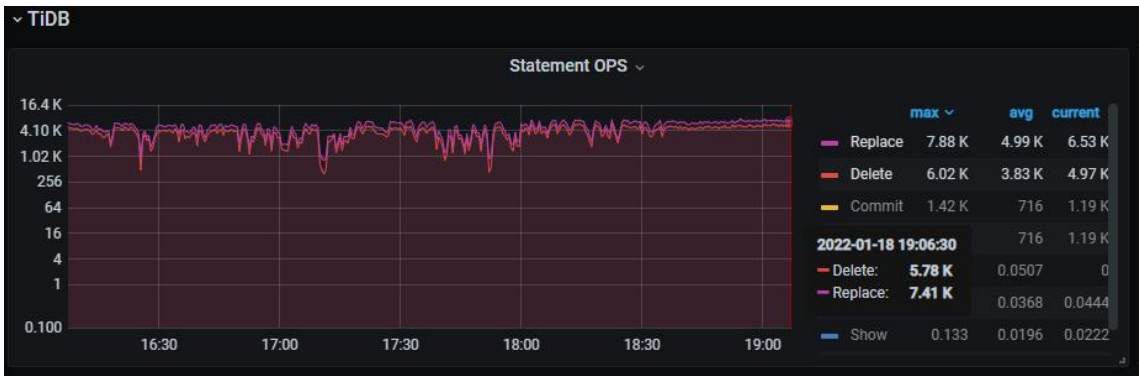
1 个 drainer 进行增量延迟越来越大



3 个 drainer 进行并行增量同步最慢一条增量链路: 9h 追了近 1 天数据

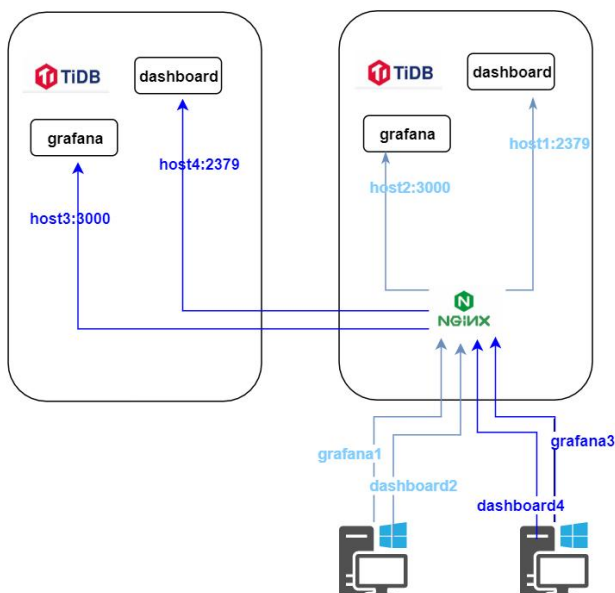


3 个 drainer 并行同步目的端写入1.2w TPS > 源端 6k 写入TPS



9、配置新建 tidb grafana&dashboard 域名

建 grafana、dashboard 的域名指向生产 nginx 代理，由 nginx 代理 grafana 端口，dashboard 端口



第三阶段

1、check 新老TiDB 集群数据同步一致性情况

TiDB 在全量和增量时会自行进行数据一致性校验，我们主要关注增量同步延迟情况，并随机 `count(*)`源目的端表

#延迟检查方法一：在源端 TiDB drainer 状态中获取最新已经回复 TSO 再通过 pd 获取延迟情况

```
mysql> show drainer status;
```

```
+-----+-----+-----+-----+-----+
-----+
| NodeID | Address | State | Max_Commit_Ts | Update_Time |
+-----+-----+-----+-----+-----+
-----+
| xxxxxx:8249 | xxxxxx:8249 | online | 430547587152216733 | 2022-01-21 16:50:58 |
```

```
tiup ctl:v5.1.2 pd -u http://xxxxxx:2379 -i
```

```
> tso 430547587152216733;
```

```
system: 2022-01-17 16:38:23.431 +0800 CST
```

```
logic: 669
```

#延迟检查方法二：在 grafana drainer 监控中观察

tidb-Binlog->drainer->Pump Handle TSO 中 current 值和当前实际时间做延迟比较

曲线越陡，增量同步速率越快

2、tiflash 表建立&CDC 同步在新 TiDB 集群建立&新 mysql->tidb 汇总同步链路闭环(DRC-TIDB)

● tiflash

源端 tidb生成目的端 新建 tiflash 语句

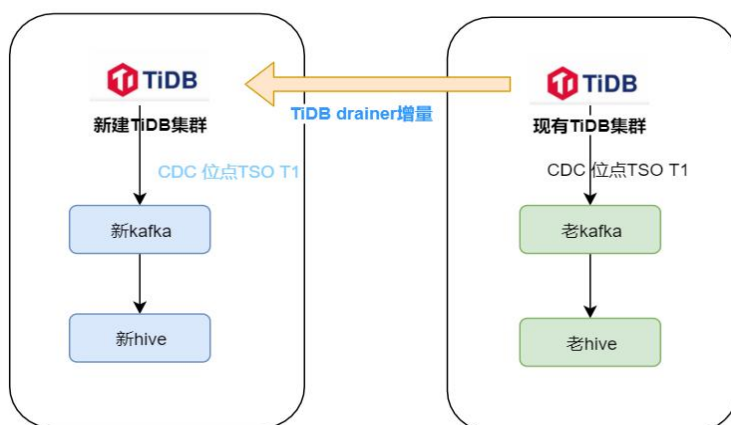
```
SELECT * FROM information_schema.tiflash_replica WHERE TABLE_SCHEMA = '<db_name>' and
TABLE_NAME = '<table_name>'
```

```
SELECT concat('alter table ',table_schema,'.',table_name,' set tiflash replica 1;')
```

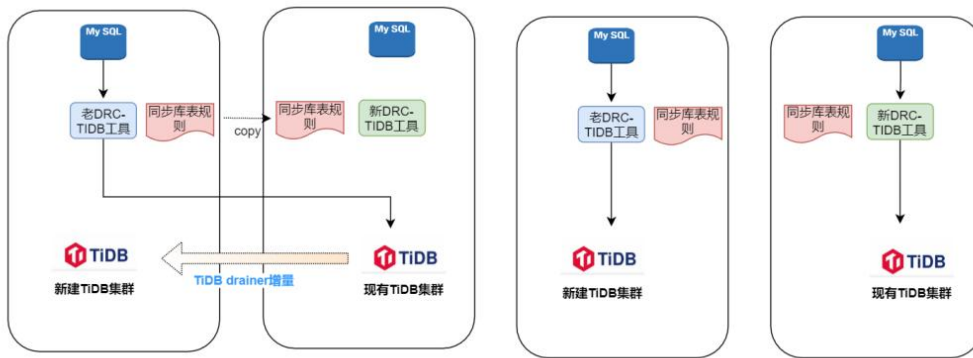
```
FROM information_schema.tiflash_replica where table_schema like 'dfp%';
```

● CDC 链路闭环

在老TiDB CDC 同步中选取 1 个 TSO 位点在新 TiDB 中建立CDC至kafka topic 同步



● DRC-TIDB 链路闭环（自研 mysql->tidb 合库合表同步工具）



上图左右为 DRC-TiDB 拆分前后状态

- (1) 左老drc-tidb 同步规则 copy 到右新 drc-tidb, 不启动 drc-tidb 同步(记录当前时间 T1)
- (2) drainer 同步现有 TiDB 数据至新建 TiDB 链路启用安全模式 replace(syncer.safe-mode: true)插入
- (3) 修改左 drc-tidb 同步源目的地址为闭环, 并启动 drc-tidb (记录当前时间 T2)
- (4) 右 tidb grafana drainer 监控中 check 当前同步时间 checkpoint 是否 $\geq$ T2 (类似于 tikv follower-read), 若没有则等待延迟追上
- (5) 右 tidb 集群增量同步修改 edit-config drainer 配置文件, 去掉 mysql-tidb 同步的库名 (所有库同步增加指定库名同步) 并 reload drainer 节点

```
commit_ts: 431809362388058219
config:
syncer.db-type: tidb
syncer.replicate-do-db:
- dmall_db1 该 DB 为直接读写
- dmall_db2 该 DB 为从 mysql 同步而来, 需去掉
```

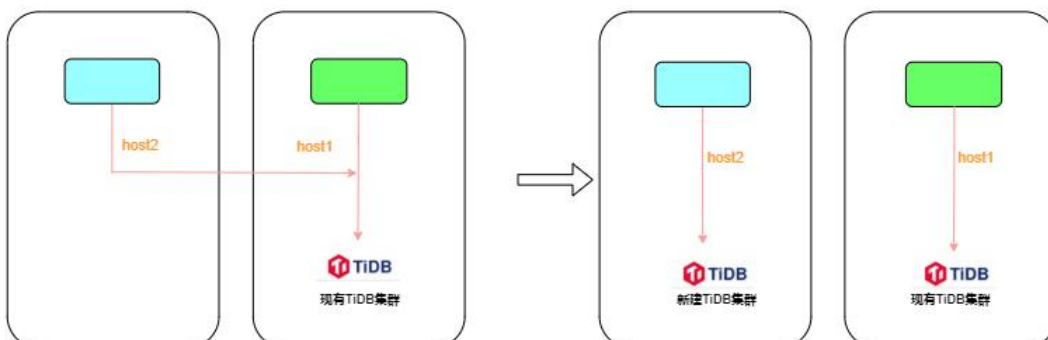
- (6) 修改右 drc-tidb 同步源目的地址为闭环, 并启动右 drc-tidb (drc-tidb 采用幂等同步, 会重复消费 copy 同步规则 T1 时间到现在的 mysql binlog)

3、每个新 TiDB 集群 ANALYZE TABLE 更新表统计信息

不是必须, 更新统计信息为最新可以避免查询 sql 索引选择走错

第四阶段

1、左 tidb 集群应用域名解析至新建 tidb 计算节点



2、批量 kill 右 TiDB 集群左应用的连接

存在脚本多次批量 kill tidb pid;在右 tidb 节点依然有大量左应用的连接，因此左应用滚动重启后右 tidb 节点左应用连接释放

3、移除老TiDB 集群->新 TiDB 集群增量同步 drainer 链路

注意：因多个 TiDB 集群共用的 1 台高配 drainer 机器，node_exporter(采集机器监控 agent)也是多个 TiDB 集群共用，当 A TiDB 集群停止drainer 链路，B C TiDB 集群会报 node_exporter 不存活告警

总结

- 不同 TiDB 版本的升级统一版本很有必要，一是拆分方法的通用，减少拆分的复杂度，二是享受新版本的特性，减低运维管理成本
- 目标 TiDB 集群磁盘空间需足够充裕
- 在源 TiDB 写入压力大时增量同步 binlog 到目的端的延迟保障需要 drainer 按库名进行并发增量同步
- TiDB 拆分涉及步骤多，能提前做的步骤就提前做，真正拆分的时间窗口很短
- 感谢 TiDB 官方社区对我们的技术支持，路漫漫其修远兮，我们将上下而求索

5.7 MongoDB 5.0 报错 Illegal instruction 解决

作者：任仲禹

去年七月的一声炮响，MongoDB Inc 给我们送来了 MongoDB 5.0，该版不仅带来了核心特性—时序集合，但若使用不慎还会给我们埋些小小的“坑”；如果您的环境正准备安装、试用或升级到 MongoDB 5.0，那不妨留步讨论下。

现象

先注明下，我的 Linux 版本为 CentOS Linux release 7.2.1511。

安装完最新的 mongodb-5.0.x 后，执行 mongo 或 mongod 直接报错 **Illegal instruction**。

```
[root@10-186-61-38 mongodb]# cd mongodb-linux-x86_64-rhel70-5.0.5/bin/
[root@10-186-61-38 bin]# ./mongo --help
Illegal instruction
[root@10-186-61-38 bin]# ./mongod --help
Illegal instruction
```

满脸疑惑，使用低版本 MongoDB 4.4.9 是没有任何问题的。

```
[root@10-186-61-38 mongodb]# cd mongodb-linux-x86_64-rhel70-4.4.9/bin/
```



```
[root@10-186-61-38 bin]# ./mongo --help
MongoDB shell version v4.4.9
usage: ./mongo [options] [db address] [file names (ending in .js)]
```

排查

执行 `mongo` 或 `mongod` 命令获取到 2 行 `dmesg` 日志报错：

```
[root@10-186-61-38 bin]# dmesg -T
.....
[Thu Dec 23 18:05:13 2021] traps: mongo[16596] trap invalid opcode ip:7f0ad9fa90da
sp:7ffe9deaa050 error:0 in mongo[7f0ad7f86000+2c8c000]
[Thu Dec 23 18:05:17 2021] traps: mongod[16597] trap invalid opcode ip:7f3b1e329a6a
sp:7ffc8fb540e0 error:0 in mongod[7f3b1a355000+5110000]
```

可以看到，命令执行失败是因为 `invalid opcode` 导致，貌似是跟操作系统的某种指令集有关。

带着疑惑和关键字在 MongoDB 社区进行查找，发现存在类似报错：

MongoDB 5.0 CPU Intel G4650 compatibility

According to the [Production notes](#) (62), mongod and the legacy mongo shell require Sandy Bridge or later microarchitectures.

I tried to execute mongo and mongod with an Intel G4560. This processor belongs to the Kaby Lake Intel CPU generation, that is later than Sandy Bridge, so it should work.

However mongo and mongod fail with an Illegal Instruction message (while mongosh works).

I got exactly the same result with Ubuntu 18.04 and Ubuntu 20.04.

The syslog line corresponding to the event is the following:

```
Jul 25 14:56:18 ubuntu kernel: [ 2725.110420] traps: mongo[24364] trap invalid opcode ip:563cbf6db39a
sp:7ffde34aa360 error:0 in mongo[563cbd6f0000+2c4d000]
```

MongoDB worked without problem for years on this hardware up to and including 4.4.

May I have some advice about the support of this chip by M 5.0 ?

The CPU model name is Intel(R) Pentium(R) CPU G4560 @ 3.50GHz

The flags (from `cpuinfo`) are:

```
fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse
sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc art arch_perfmon pebs bts rep_good nopl
xtopology nonstop_tsc cpuid aperfmperf pni pclmulqdq dtes64 monitor ds_cpl est tm2 ssse3 sdbg cx16
xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave rdand lahf_lm abm
3dnowprefetch cpuid_fault epb invpcid_single pti ssbd ibrs ibpb stibp fsgsbase tsc_adjust smep erms
invpcid mpx rdseed smap clflushopt intel_pt xsaveopt xsavec xgetbv1 xsaves dtherm arat pln pts hwp
hwp_notify hwp_act_window hwp_epp md_clear flush_l1d
```

✓ Solved by [Stennie](#) in [post #2](#)

Hi @Jean-Marie_Gouarne, I looked into this a bit further and the underlying requirement for the MongoDB 5.0 binary server packages is **CPUs with AVX instructions**. These are broadly Sandy Bridge or newer Intel CPUs, but there is a caveat: Not all CPUs from the listed families support AVX. Generally...

该案例虽然操作系统为 Ubuntu，但是系统错误信息类似，报错原因是因为：MongoDB 5.0 版本基本的要求是所在服务器的 CPU 需要支持 AVX 指令集。

仔细查找官方文档，可以看到安装 MongoDB 5.0 版本确实需要依赖支持 AVX 指令集的 CPU：

The screenshot shows the MongoDB Manual website. On the left is a sidebar with a 'MongoDB Manual' header and a 'Version 5.0' dropdown menu. Below the menu are links for Introduction, Installation, MongoDB Shell (mongosh), MongoDB CRUD Operations, Aggregation, Data Models, Transactions, Indexes, Security, and Change Streams. The main content area has a table with operating system compatibility: macOS 10.13 and macOS 10.12, both marked with checkmarks. Below the table, there are two footnotes: [1] stating MongoDB only supports Oracle Linux running the Red Hat Compatible Kernel (RHCK), and [2] stating MongoDB 5.0 requires use of the AVX instruction set, available on select Intel and AMD processors. Below this, there is a section for ARM64, stating that MongoDB on ARM64 requires the ARMv8.2-A or later microarchitecture. At the bottom, it says that starting in MongoDB 5.0, mongod, mongos, and the legacy mongo shell no longer support ARM64 platforms which do not meet this minimum microarchitecture requirement.

目前支持 AVX 指令集的 CPU 型号可以参考链接：

https://en.wikipedia.org/wiki/Advanced_Vector_Extensions#CPUs_with_AVX

检查下我自己服务器的 CPU，确实不支持 AVX（注意：若支持 AVX 指令集，flags 字段会有打印 'avx' 字符串）：

```
[root@10-186-61-38 ~]# cat /proc/cpuinfo
processor : 0
vendor_id : GenuineIntel
cpu family : 6
model : 13
model name : QEMU Virtual CPU version 2.5+
.....
flags : fpu de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pse36 clflush mmx
fxsr sse sse2 syscall nx lm rep_good nopl xtopology eagerfpu pni cx16 x2apic hypervisor
lahf_lm
.....
```

支持 AVX 的 CPU 测试

找台 CPU 支持 AVX 指令集的服务器进行测试。

```
[root@cebl888 bin]# cat /proc/cpuinfo
processor : 0
vendor_id : GenuineIntel
cpu family : 6
model : 79
model name : Intel(R) Xeon(R) Gold 6132 CPU @ 2.60GHz
stepping : 0
microcode : 0x200004d
cpu MHz : 2593.906
cache size : 19712 KB
physical id : 0
siblings : 1
core id : 0
cpu cores : 1
apicid : 0
initial apicid : 0
cpu : yes
cpu_exception : yes
cpuid level : 13
cpuid : yes
flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts mmx fxsr sse sse2 ss syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon pebs bts nopl xtopology tsc_reliable nonstop_tsc eagerfpu pni pclmulqdq ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch invpcid_single ssbd ibrs ibpb stibp fsgsbase tsc_adjust bmi1 hle avx2 smep bmi2 invpcid
bogomips : 5187.81
clflush size : 64
cache alignment : 64
address sizes : 43 bits physical, 48 bits virtual
```

MongoDB 5.0 新版本命令正常执行。

```
[root@ce51888 bin]# ./mongo --version
MongoDB shell version v5.0.5
Build Info: {
  "version": "5.0.5",
  "gitVersion": "d65fd89df3fc039b5c55933c0f71d647a54510ae",
  "opensslVersion": "OpenSSL 1.0.1e-fips 11 Feb 2013",
  "modules": [],
  "allocator": "tcmalloc",
  "environment": {
    "distmod": "rhel70",
    "distarch": "x86_64",
    "target_arch": "x86_64"
  }
}
[root@ce51888 bin]# ./mongod --version
db version v5.0.5
Build Info: {
  "version": "5.0.5",
  "gitVersion": "d65fd89df3fc039b5c55933c0f71d647a54510ae",
  "opensslVersion": "OpenSSL 1.0.1e-fips 11 Feb 2013",
  "modules": [],
  "allocator": "tcmalloc",
  "environment": {
    "distmod": "rhel70",
    "distarch": "x86_64",
    "target_arch": "x86_64"
  }
}
```

AVX 指令集

AVX（Advanced Vector Extensions，高级向量扩展指令集）是 x86 架构微处理器中的指令集

- 由英特尔在 2008 年 3 月提出，并在 2011 年第一季度发布的 Sandy Bridge 系列处理器中首次支持。AMD 在随后的 2011 年第三季度发布的 Bulldozer 系列处理器中开始支持 AVX。所以较老的 CPU 架构中可能并不支持 AVX 指令集。AVX 是 X86 指令集的 SSE 延伸架构，如 IA16 至 IA32 般的把寄存器 XMM 128bit 提升至 YMM 256bit，所以从理论上看 CPU 的运算性能将提升 2 倍。

目前 MongoDB 官方文档中仅说明安装 MongoDB 5.0 需要依赖服务器 CPU 支持 AVX 指令集，但并未说明具体需要支持的原因：

网上仅检索到一篇关于 MongoDB with AVX 的文章《Getting storage engines ready for fast storage devices》提到：可以使用经过高度优化基于 AVX 的 memcpy 方式，实现数据从内存映射区域拷贝到另一块应用的缓冲区；猜测目前 5.0 版本的发布包含了文章提到的 WiredTiger 存储引擎层面的更新（文章提到更新引擎后的读吞吐量提升了 63%），而底层存储引擎的优化更新依赖于 avx 的支持。

<https://engineering.mongodb.com/post/getting-storage-engines-ready-for-fast-storage-devices>

结论

若需要安装或升级到 MongoDB 5.0 新版本，一定要提前确保自己的服务器 CPU 能否支持 AVX 指令集架构，检查命令如下：

```
grep avx /proc/cpuinfo
```

参考文档：

<https://www.mongodb.com/community/forums/t/mongodb-5-0-cpu-intel-g4650-compatibility/116610>

<https://docs.mongodb.com/manual/administration/production-notes/>

5.8 mlaunch-MongoDB 测试环境快速构建工具

作者：余振兴

我们在使用和学习 MongoDB 过程中难免遇到需要部署各类架构进行功能性验证，由于不同的架构和配置。使得我们在搭建环境过程中需要花费大量时间，这里推荐使用一款可快速部署 MongoDB 测试环境各类架构的工具 mlaunch，它是 MongoDB 一组开源工具包 mtools 中的一个(对 mtools 的其他工具有兴趣也可查看文末链接地址了解详情)，可用来在测试环境快速部署不同架构的 MongoDB 服务，支持单节点、副本集以及分片集群，安装及操作均十分简单，以下给到常用场景和架构的部署示例

一、mlaunch 及 MongoDB 软件安装

```
## 安装 python3,及 MongoDB 服务的依赖包
shell> yum install -y python3 python3-devel net-snmp-agent-libs

## 安装 mlaunch 依赖包
shell> pip3 install mtools psutil pymongo

## MongoDB 软件包安装(解压即可),MongoDB 软件包自行下载,当前示例服务器下载了 2 个版本软件包,可以任意指定其中版本
shell> ll -ld /opt/mongodb-linux-x86_64-*
drwxr-xr-x 4 root root      148 8月  16 18:30 /opt/mongodb-linux-x86_64-enterprise-rhel70-4.0.16
-rw-r--r-- 1 root root 120608697 8月  16 18:30 /opt/mongodb-linux-x86_64-enterprise-rhel70-4.0.16.tgz
drwxr-xr-x 3 root root      100 8月  16 18:31 /opt/mongodb-linux-x86_64-rhel70-4.4.14
-rw-r--r-- 1 root root  72399766 8月  16 18:25 /opt/mongodb-linux-x86_64-rhel70-4.4.14.tgz
```

二、mlaunch 部署副本集及运维操作

```
## 初始化一个 3 节点的副本集
## 副本集名称为 demo_replicatset
## 起始端口为 30000
## 指定 30000 端口的优先级最高(优先级默认为 1,主节点调整为 10,值越高,优先级越高)
## 版本为 4.4.14
## 数据目录为 /data/demo_rs
shell> mlaunch init --replicaset --nodes 3 --name "demo_replicatset" --priority --port 30000 --binarypath /opt/mongodb-linux-x86_64-rhel70-4.4.14/bin --dir /data/demo_rs
launching: "/opt/mongodb-linux-x86_64-rhel70-4.4.14/bin/mongod" on port 30000
launching: "/opt/mongodb-linux-x86_64-rhel70-4.4.14/bin/mongod" on port 30001
launching: "/opt/mongodb-linux-x86_64-rhel70-4.4.14/bin/mongod" on port 30002
replica set 'demo_replicatset' initialized.
```

```
## 查看部署运行的副本集状态
shell> mlaunch list --dir /data/demo_rs/
PROCESS    PORT      STATUS    PID
mongod     30000     running   9017
mongod     30001     running   9065
mongod     30002     running   9113

## 登录副本集(默认无密码,可也在初始化时指定)
shell> /opt/mongodb-linux-x86_64-rhel70-4.4.14/bin/mongo --host=localhost:30000
demo_replicatset:PRIMARY>

## 停止该副本集
shell> mlaunch stop --dir /data/demo_rs/
sent signal Signals.SIGTERM to 3 processes.

## 重启启动该副本集
shell> mlaunch start --dir /data/demo_rs/
launching: "/opt/mongodb-linux-x86_64-rhel70-4.4.14/bin/mongod" on port 30000
launching: "/opt/mongodb-linux-x86_64-rhel70-4.4.14/bin/mongod" on port 30001
launching: "/opt/mongodb-linux-x86_64-rhel70-4.4.14/bin/mongod" on port 30002

## 删除该副本集(停止后直接删除目录即可)
shell> rm /data/demo_rs/ -rf
```

三、mlaunch 部署分片集群

```
## 创建 3 节点的分片,每个节点只有 1 个主(考虑服务器配置有限)
## 配置 3 节点 configServer
## 配置 1 个 mongos
shell> mlaunch init --sharded 3 --nodes 1 --replicaset --config 3 --csrs --mongos 1 --
priority --name "demo_shard" --port 30000 --binarypath /opt/mongodb-linux-x86_64-
rhel70-4.4.14/bin --dir /data/demo_shard
launching: "/opt/mongodb-linux-x86_64-rhel70-4.4.14/bin/mongod" on port 30001
launching: "/opt/mongodb-linux-x86_64-rhel70-4.4.14/bin/mongod" on port 30002
launching: "/opt/mongodb-linux-x86_64-rhel70-4.4.14/bin/mongod" on port 30003
launching: config server on port 30004
launching: config server on port 30005
launching: config server on port 30006
replica set 'configRepl' initialized.
replica set 'shard01' initialized.
replica set 'shard02' initialized.
replica set 'shard03' initialized.
launching: /opt/mongodb-linux-x86_64-rhel70-4.4.14/bin/mongos on port 30000
adding shards. can take up to 30 seconds...
```

查看部署的集群架构及基本信息

```
[root@10-186-61-159 ~]# mlaunch list --dir /data/demo_shard/
```

PROCESS	PORT	STATUS	PID
mongos	30000	running	15820
config server	30004	running	15514
config server	30005	running	15567
config server	30006	running	15620
shard01			
mongod	30001	running	15377
shard02			
mongod	30002	running	15422
shard03			
mongod	30003	running	15469

登录集群

```
shell> /opt/mongodb-linux-x86_64-rhel70-4.4.14/bin/mongo --host=localhost:30000
mongos>
```

四、mlaunch 常用部署架构命令示例

单节点副本集

```
mlaunch init \
--replicaset \
--priority \
--nodes 1 \
--name "demo_single_rs" \
--port 30000 \
--binarypath /opt/mongodb-linux-x86_64-rhel70-4.4.14/bin \
--dir /data/demo_single_rs
```

副本集 PSS 架构

```
mlaunch init \
--replicaset \
--priority \
--nodes 3 \
--name "demo_replicatset" \
--port 30000 \
--binarypath /opt/mongodb-linux-x86_64-rhel70-4.4.14/bin \
--dir /data/demo_rs
```

副本集 PSA 架构

```
mlaunch init \
```

```
--replicaset \  
--priority \  
--arbiter \  
--nodes 3 \  
--name "demo_replicaset" \  
--port 30000 \  
--binarypath /opt/mongodb-linux-x86_64-rhel70-4.4.14/bin \  
--dir /data/demo_rs  
  
## 分片集群高可用架构  
## 3 节点 ConfigServer 副本集  
## 3 节点 shard 分片节点 副本集  
## 3 节点 mongos 高可用  
mlaunch init \  
--sharded 3 \  
--nodes 3 \  
--replicaset \  
--config 3 \  
--mongos 3 \  
--csrs \  
--priority \  
--name "demo_shard" \  
--port 30000 \  
--binarypath /opt/mongodb-linux-x86_64-rhel70-4.4.14/bin \  
--dir /data/demo_shard
```

五、参考链接

<http://blog.rueckstiess.com/mtools/mlaunch.html><https://github.com/rueckstiess/mtools>

5.9 MongoDB Sharding QPS 分布不均案例一则

作者：任坤

一、背景

线上某核心业务采用了国内某云厂商 Mongo RDS ，版本为 4.2 ，采用 4 分片集群。

临上线前进行各种场景测试，在运行邮件发送测试时发现 Mongo 集群负载严重不均衡。在 1 分钟的压测时间内，Shard 3 主节点 cpu 被 100%打满，其余 3 个 shard 主库负载均在正常范围，据开发反馈该场景 insert 比较多，可能是 insert 流量倾斜导致的。我们的分片表都是采用 hash 分片，并且分片字段的选择性都很高，理论上不存在数据倾斜的情况。

二、诊断

查看对应时间段监控指标，shard 3 主节点，cpu 在这 1 分钟被打满， 100ms 以上慢查询平均 每秒 1300+ 。



作为对比，查看 shard 4 同一时间段的负载，该分片几乎没有 1 条慢查询，cpu 负载也很低。仔细查看 QPS 细分指标，shard 3 的 read 和 update 分别达到了 1600+和 2200+，而 shard4 只有 500 和 600 左右，insert 两边几乎一致都是 400 左右，可以先排除掉 insert 倾斜的可能性。



查看这个时间段 shard 3 的慢查询日志，总量有几十万，但是目前该厂商 RDS 平台功能还在完善中，不支持在页面直接查看每个 shard 实例的慢查询日志。

只能在群里求助官方人员，让他们从后台导出每个 shard 对应时间段的慢查询文件然后传过来，其中 shard 3 的慢查询文件有几百兆。

文件太大只下载了一部分，打开文件看到前几个慢查询都针对用户表 users 的 update，sql text 都超过 10K，当时判定是 users 的 update 倾斜导致的问题。

该表针对 pid 列进行 hash 分片，目前有 260w 行，db.users.getShardDistribution()显示数据分布很均匀。

本次测试的 pid 是连号的，从 2600001 - 2635000 总共 3.5w 条数据，现在怀疑这 3.5w 条数据分布倾斜。

但是目前该厂商 RDS 不支持直接登录 shard 实例，也就无法直接确认这 3w 多条数据的分布情况。

现在问题演变成了：只允许登录 mongos 的前提下，如何确认某个范围的数据在 shard 节点上的物理分布情况？

社区群里求助，很快有热心群友给出建议：

1、对于每个 pid 值，执行 convertShardKeyToHashed(pid) 计算 hash 值，再根据该 hash 值找出其所属 chunk 和节点。

该方案的难点在于每个 chunk 只记录范围上限和下限，对于给定的 hash 值无法直接判断其落在了哪个 chunk，编写相应脚本的逻辑会比较复杂。

2、新建一个同等结构的分片表，将这 3w 条记录导入，然后查看该表的分布情况。

这个方案很好，对同一批数据采用相同的分片键和分片策略，理论上分布情况也应该是相同的，正在准备操作时热心群友又给出了第 3 个方案。

3、对这批 pid 执行范围查询并查看执行计划 executionStats，每个 shard 会返回 docs 或者 keyexamined 情况。

思路打开了，这个应该是最优解。

以上建议均是由同一位网友提出的，在此感谢@徐靖(公众号"DB 说")。

当即执行了一把

```
db.users.find({ "pid" : { "$gte" : 2600001, "$lt" :
2635000 } }).explain("executionStats")
.....
"totalKeysExamined" : 34994,
"totalDocsExamined" : 34994,
"docsExamined" : 8643, - - shard1
"docsExamined" : 8693, - - shard2
"docsExamined" : 8824, - - shard3
"docsExamined" : 8834, - - shard4
```

.....

确认这 3w 多条数据分布是均匀的，至此时间已经过去了几个小时。

现在确认了所有分片数据都是均匀散列的，接下来只能核对非分片表。

正在准备查询时，云厂商发来消息，他们通过后台查看慢日志确认是某个单表导致的，该表的 `primary shard` 正好是 `shard 3`。

当即和开发确认，每次操作流程确实会涉及该表，调用一次 `findAndModify`。

那为何在官方提供的慢查询日志文件里没有找到，再次查看发现文件开头的时间点不在 CPU 异常时间段内，有点疏忽了。该单表只有 3 条数据，即便开启分片这 3 条数据也只会落到单个 `shard` 节点，不会有任何性能提升，将该表计算逻辑挪到了 `redis`，再次压测问题消失。

3、小结

类似这种 QPS 分布不均匀的案例去年在自建 Mongo 集群遇到过一次，当时通过查看 `shard` 节点的 `oplog` 快速定位并解决了。

这次之所以绕了这么大的弯，原因如下：

1. 该厂商 RDS 平台功能不太完善，不支持在页面上直接查看 `shard` 节点的慢查询，也不支持直接登录 `shard` 实例，好在官方已承诺最快下个月会上线类似功能。
2. 第一次查看官方发来的慢查询文件时，没有仔细看对应的时间点，直接导致排查方向搞偏了。

5.10 OceanBase 错误日志分析

作者：操盛春

OceanBase 运行时会产生很多各种级别的日志，如果出现了错误，想要从数量繁多的错误日志中定位到错误原因，是件不太容易的事。

错误日志是我们定位错误原因的主要途径，本文我们就来聊聊怎么从 OceanBase 错误日志中找到我们想要的错误信息。

1. 日志文件

OceanBase 日志分为 3 类：

- 选举模块日志：存放选举模块产生的日志。
- 总控服务（RootService）模块日志：存放总控服务模块产生的日志。
- 启动、运行日志：存放除选举模块、总控服务模块之外的其它所有模块产生的日志。

每类日志又包含 2 种日志文件：

- .log、.log.YYYYmmDDHHMMSS 文件中包含该类型所有级别的日志。
- .log.wf、.log.wf.YYYYmmDDHHMMSS 文件中包含该类型 WARN、USER_ERROR、ERROR 3 个级别的日志（需要把系统配置 enable_syslog_wf 的值设置为 true）。

每类日志都写入到对应的 .log(.log.wf) 文件中，写满 256M 之后，把 .log(.log.wf) 文件改名（加上 .YYY YmmDDHHMMSS 后缀），然后再创建新的 .log(.log.wf) 文件。

通过 `ls -l` 查看 OceanBase 日志目录，可以看到类似如下的日志文件：

```
-rw-r--r-- 1 admin admin 18688919 10 月 9 02:08 election.log
-rw-r--r-- 1 admin admin 4998884 10 月 9 02:07 election.log.wf
-rw-r--r-- 1 admin admin 75158675 10 月 9 02:08 rootservice.log
-rw-r--r-- 1 admin admin 268437081 10 月 8 23:25 rootservice.log.20221008232523
-rw-r--r-- 1 admin admin 61688030 10 月 9 02:08 rootservice.log.wf
-rw-r--r-- 1 admin admin 227610855 10 月 8 23:25 rootservice.log.wf.20221008232523
-rw-r--r-- 1 admin admin 156856561 10 月 9 02:08 observer.log
-rw-r--r-- 1 admin admin 268435919 10 月 9 01:25 observer.log.20221009012502
-rw-r--r-- 1 admin admin 91282191 10 月 9 02:08 observer.log.wf
-rw-r--r-- 1 admin admin 158605686 10 月 9 01:25 observer.log.wf.20221009012502
```

2. 日志格式

按照日志错误级别不同，OceanBase 格式分为 2 种：

- DEBUG、TRACE、INFO 日志记录格式

```
[time] log_level [module_name] file_name:line_no [thread_id][coroutine_id][Ytraceid0-traceid1] [lt=last_log_print_time] [dc=dropped_log_count] log_data
```

- WARN、USER_ERROR、ERROR 日志记录格式

```
[time] log_level [module_name] function_name (file_name:line_no) [thread_id][coroutine_id][Ytraceid0-traceid1] [lt=last_log_print_time] [dc=dropped_log_count] log_data
```

2 种日志格式的不同之处是：WARN、USER_ERROR、ERROR 日志记录多了 `function_name` 字段。

这些日志字段中，需要重点介绍 3 个字段：

`trace_id`：格式为 `Ytraceid0-traceid1`，例如：`YB420ABA3C91-0005E98FC3148792`。

一条 SQL 在整个 OBServer 集群中对应唯一的 `trace_id`，通过 `trace_id` 可以找到一条 SQL 执行过程中在整个集群中打印的所有日志。

lt：`lt = last_log_print_time`，写日志方式不同，此字段值表示的含义也不同：

- 异步写日志：该字段值表示本条日志格式化消耗的时间，格式化指的是：源码中把本条日志所有字段拼接到一起得到字符串格式内容的过程。

- 同步写日志：该字段值表示上一条日志从格式化到写入日志文件成功消耗的时间。根据这个写入时间和上一条日志的内容长度，能够反映出写上一条日志时的磁盘 IO 负载。

为什么记录的是上一条日志，而不是本条日志写入过程消耗的时间？

因为在格式化本条日志的时候，还不知道本条日志成功写入日志文件需要消耗多长时间，记录本条日志消耗的时间就只能留给后来者（也就是本条日志的下一条日志）去做了。

dc: dc = dropped_log_count，表示自上一条日志成功写入日志文件以来，到当前这条日志为止，中间有多少条日志（包含所有级别的日志）写入失败了，只有异步日志记录中有这个字段。

异步写入日志失败的原因有以下几种：

- 为日志记录分配内存失败。
- 格式化日志失败（格式化的定义参照前面 lt 字段异步写日志中的定义）。
- 每秒写入的日志量超过最大限制了。
- 日志内容等待加入日志队列超时，队列中最多能加入的日志数量由 MAX_BUFFER_ITEM_CNT 常量控制，固定值为 524288。

3. 分析日志

如果执行某条 SQL 出现了错误，OceanBase 会给出错误码和错误信息，有时候这个错误信息并不明确，想要找到更明确的信息，可以按照 2 个步骤进行。

3.1 根据错误码找到 trace_id

因为 trace_id 在一个集群中唯一标识一条 SQL，所以第一步是从错误日志中找到 trace_id，按照以下顺序进行：

- 先根据错误码去 observer.log.wf 文件找 trace_id。
- 如果 observer.log.wf 中没找到，再去 rootservice.log.wf 文件找。
- 如果 rootservice.log.wf 中没找到，再去 election.log.wf 文件找。

为什么要按上面的文件顺序找 trace_id ？

- 选举过程并不是只涉及到选举模块，还会涉及到总控服务模块（RS）、其它模块，所以选举过程中，在 observer.log.wf、rootservice.log.wf、election.log.wf 中都会产生日志记录。
- 总控服务提供的功能，最终也要由集群中的 OBServer 执行，整个过程除了会涉及总控服务模块，还会涉及其它模块。
- 鉴于以上 2 点，不管执行的是什么 SQL，都要由最终干活的角色去执行，而 observer.log.wf 中记录的就是最终干活的角色产生的日志，这里最有可能记录着错误的源头，为此先从 observer.log.wf 入手。

接下来以 OceanBase 创建资源池为例，介绍从日志文件中查找错误信息的过程：

```
obclient [(oceanbase)]> CREATE RESOURCE POOL mini_pool_t4 unit=mini_1,unit_num=1;
ERROR 4624 (HY000): machine resource 'z1' is not enough to hold a new unit
```

创建资源池报错，错误码为 4624，先根据错误码从 observer.log.wf 文件中过滤错误日志，过滤字符串的格式为：ret=-错误码（注意错误码前面的负号）。

```
[admin@localhost log]$ grep "ret=-4624" observer.log.wf
[2022-10-
09 06:39:25.317545] WARN [SQL.ENG] execute (ob_resource_executor.cpp:42) [3414][2255][
YB420ABA3C91-
0005E98FC5948785] [lt=63] [dc=0] rpc proxy create resource_pool failed(ret=-4624)
[2022-10-
09 06:39:25.317883] WARN [SQL] open_cmd (ob_result_set.cpp:84) [3414][2255][YB420ABA3C
91-0005E98FC5948785] [lt=21] [dc=0] execute cmd failed(ret=-4624)
[2022-10-
09 06:39:25.317994] WARN [SQL] sync_open (ob_result_set.cpp:133) [3414][2255][YB420ABA
3C91-0005E98FC5948785] [lt=96] [dc=0] fail to exec execute()(ret=-4624)
[2022-10-
09 06:39:25.353385] WARN [SERVER] response_result (ob_sync_cmd_driver.cpp:60) [3414][2
255][YB420ABA3C91-0005E98FC5948785] [lt=14] [dc=0] close result set fail(cret=-4624)
[2022-10-
09 06:39:25.353471] WARN [SERVER] test_and_save_retry_state (ob_query_retry_ctrl.cpp:1
17) [3414][2255][YB420ABA3C91-
0005E98FC5948785] [lt=29] [dc=0] ddl, and errno is not OB_EAGAIN or OB_SNAPSHOT_DISCARD
ED or OB_ERR_PARALLEL_DDL_CONFLICT, do not need retry(client_ret=-4624, err=-
4624, retry_type_=0)
[2022-10-
09 06:39:25.353593] WARN [SERVER] response_result (ob_sync_cmd_driver.cpp:64) [3414][2
255][YB420ABA3C91-
0005E98FC5948785] [lt=21] [dc=0] result set open failed, check if need retry(ret=-
4624, cli_ret=-4624, retry_ctrl_.need_retry()=0)
[2022-10-
09 06:39:25.353969] WARN [SERVER] do_process (obmp_query.cpp:674) [3414][2255][YB420AB
A3C91-0005E98FC5948785] [lt=9] [dc=0] execute query fail(ret=-
4624, timeout_timestamp=1665298548583953)
[2022-10-
09 06:39:25.354347] WARN [SERVER] process (obmp_query.cpp:291) [3414][2255][YB420ABA3C
91-
0005E98FC5948785] [lt=18] [dc=0] fail execute sql(sql_id="", sql=CREATE RESOURCE POOL m
ini_pool_t4 unit=mini_1,unit_num=1, sessid=3221633143, ret=-
4624, ret="OB_MACHINE_RESOURCE_NOT_ENOUGH", need_disconnect=false)
```

上面过滤出来的错误日志中，YB420ABA3C91-0005E98FC5948785 就是 trace_id。

从 observer.log.wf 文件中，一般都能找到 trace_id，如果万一没找到，再依次从 rootservice.log.wf、election.log.wf 中找。

如果从上面 3 个日志文件中都没找到，那就是碰上特殊情况了，可以用报错中的错误信息依次从 observer.log.wf、rootservice.log.wf、election.log.wf 中找，例如：

```
[admin@localhost log]$ grep "machine resource 'z1' is not enough to hold a new unit" ob
server.log.wf
```

```
[2022-10-
09 08:17:02.170951] WARN log_user_error_and_warn (ob_rpc_proxy.cpp:300) [13567][2307][
YB420ABA3C91-
0005E98FC5C48785] [lt=7] [dc=0] machine resource 'z1' is not enough to hold a new unit
```

3.2 根据 trace_id 查找错误日志

从 3.1 根据错误码找到 trace_id 小节可以看到，根据错误码或者错误信息也能过滤出很多错误日志，为什么还要多此一举，再根据 trace_id 查找错误日志？

因为通过错误码或错误信息过滤出来的日志，有可能把非常关键的日志记录给过滤掉了，而通过 trace_id 能够拿到更完整的日志。

根据 trace_id 查找错误日志，也一样先从 observer.log.wf 中开始找，如果找到了能够确定错误原因的信息，就此结束，否则再接着到 rootservice.log.wf 、 election.log.wf 中找。

这一步的流程就是通过 grep trace_id 从各个 .log.wf 文件中找到错误日志，不展开举例了。

4. 存在的问题

实际使用过程中，有时候并不能通过 Oceanbase 的错误日志找到明确的错误原因，这种情况下要找到错误原因，就只能靠我们的经验了。

根据前面介绍的步骤，从错误日志中找到蛛丝马迹，然后按照猜测-验证流程，一点一点排查。

例如：OBServer 集群用以下 SQL 进行 BOOTSTRAP 选举，报 Timeout 错误，最终就不能通过错误日志直接定位到原因：

```
obclient [(none)]> ALTER SYSTEM BOOTSTRAP ZONE 'z1' SERVER 'xx.xx.xx.xx:2882',ZONE 'z2'
SERVER 'xx.xx.xx.xx:2882',ZONE 'z3' SERVER 'xx.xx.xx.xx:2882';
ERROR 4012 (HY000): Timeout
```

通过多轮猜测-验证流程确定错误原因之后，反过来查看错误日志，发现最接近实际情况的日志如下：

```
[2022-10-
09 09:05:32.052665] WARN [BOOTSTRAP] execute_bootstrap (ob_bootstrap.cpp:759) [16840][
446][YB420ABA3C91-0005EA9633379D17] [lt=21] [dc=0] failed to wait all rs online(ret=-
4622)
```

错误日志中的 failed to wait all rs online ，说明 BOOTSTRAP 操作正在等待所有 RootService 服务上线。

这个问题是由于集群中 3 个 OBServer 实例启动时，-z 参数都是 z1 ，而 BOOTSTRAP 指定的 RootService 服务器的 zone 却是 z1、z2、z3 。

3 个 RootService 服务器都启动了，端口也是正常监听状态，OceanBase 却没有识别出来，错误日志中也就没能准确的记录错误信息了。

5. 两点期待

在日志方面，对于即将正式出品的 OB 4.0 版本有两点期待：

期待 1: 如果 OceanBase 能够准确识别错误产生的原因，并记录到日志中，对于我们日常运维将会非常有帮助。

期待 2: .log.wf 文件中的错误日志记录，目前不能体现触发打印日志的方法之间的调用层级关系，不方便直接找到最后一个被调用的方法产生的错误日志。

如果能够在每条错误日志中记录触发打印当前日志的方法的调用层级，我们运维过程中就能够快速找到产生错误的方法，从而提升定位错误原因的效率。

调用层级类似下面这种形式（process、do_process 等方法前面的序号）：

```
[admin@localhost log]$ grep "YB420ABA3C91-0005E98FC5948785" observer.log.wf
[2022-10-
09 06:39:25.317429] WARN  3.2.1.1.1 log_user_error_and_warn (ob_rpc_proxy.cpp:300) [341
4][2255][YB420ABA3C91-
0005E98FC5948785] [lt=5] [dc=0] machine resource 'z1' is not enough to hold a new unit
[2022-10-
09 06:39:25.317545] WARN  [SQL.ENG] 3.2.1.1 execute (ob_resource_executor.cpp:42) [3414]
[2255][YB420ABA3C91-
0005E98FC5948785] [lt=63] [dc=0] rpc proxy create resource_pool failed(ret=-4624)
[2022-10-
09 06:39:25.317883] WARN  [SQL] 3.2.1 open_cmd (ob_result_set.cpp:84) [3414][2255][YB42
0ABA3C91-0005E98FC5948785] [lt=21] [dc=0] execute cmd failed(ret=-4624)
[2022-10-
09 06:39:25.317994] WARN  [SQL] 3.2 sync_open (ob_result_set.cpp:133) [3414][2255][YB42
0ABA3C91-0005E98FC5948785] [lt=96] [dc=0] fail to exec execute()(ret=-4624)
[2022-10-
09 06:39:25.353385] WARN  [SERVER] 3 response_result (ob_sync_cmd_driver.cpp:60) [3414]
[2255][YB420ABA3C91-0005E98FC5948785] [lt=14] [dc=0] close result set fail(cret=-4624)
[2022-10-
09 06:39:25.353471] WARN  [SERVER] 3.1 test_and_save_retry_state (ob_query_retry_ctrl.c
pp:117) [3414][2255][YB420ABA3C91-
0005E98FC5948785] [lt=29] [dc=0] ddl, and errno is not OB_EAGAIN or OB_SNAPSHOT_DISCARD
ED or OB_ERR_PARALLEL_DDL_CONFLICT, do not need retry(client_ret=-4624, err=-
4624, retry_type_=0)
[2022-10-
09 06:39:25.353593] WARN  [SERVER] 3 response_result (ob_sync_cmd_driver.cpp:64) [3414]
[2255][YB420ABA3C91-
0005E98FC5948785] [lt=21] [dc=0] result set open failed, check if need retry(ret=-
4624, cli_ret=-4624, retry_ctrl_.need_retry())=0)
[2022-10-
09 06:39:25.353969] WARN  [SERVER] 2 do_process (obmp_query.cpp:674) [3414][2255][YB420
ABA3C91-0005E98FC5948785] [lt=9] [dc=0] execute query fail(ret=-
4624, timeout_timestamp=1665298548583953)
```

```
[2022-10-09 06:39:25.354347] WARN [SERVER] 1 process (obmp_query.cpp:291) [3414][2255][YB420ABA3C91-0005E98FC5948785] [lt=18] [dc=0] fail execute sql(sql_id="", sql=CREATE RESOURCE POOL mini_pool_t4 unit=mini_1,unit_num=1, sessid=3221633143, ret=-4624, ret="OB_MACHINE_RESOURCE_NOT_ENOUGH", need_disconnect=false)
```

5.11 bgsave 导致 redis 定期卡顿案例一则

作者：任坤

1、背景

线上有套 redis 主从，版本 4.0，开发抱怨说经常会出现周期性卡顿。

应用日志显示每隔 10 多分钟出现一次，一个普通调用就需要执行 1s 左右，然后自动恢复，get/set 均受影响

	_ts	_content
>	2022-03-11 09:34:22	2022-03-11 09:34:22.278 [http-nio-5151-exec-9] [INFO] 1047ms - redis set use: 1047ms
>	2022-03-11 09:34:22	2022-03-11 09:34:22.277 [http-nio-5151-exec-5] [INFO] 1105ms - redis set use: 1105ms
>	2022-03-11 09:34:22	2022-03-11 09:34:22.277 [http-nio-5151-exec-2] [INFO] 217ms - redis set use: 217ms

2、诊断

查看 redis qps 和 cpu 监控，均未发现有用线索。

登录 redis 查看 slowlog，也没有吻合时间点的慢查询。

evicted_keys 指标一直是 0，expired_keys 数量虽然很多，但是一直没有明显波动，不太可能是驱逐过期键导致。

经组里同事提醒注意到 latest_fork_usec 指标，执行一次接近 1s 左右，大约每 15 分钟触发一次 bgsave，和应用出现慢查询的频率大致吻合，现在初步认定是 redis 实例定期 bgsave 导致的应用卡顿。

在相当长的一段时间内，我一直认为 redis 的 bgsave 衍生出 1 个子进程并且采用 copy-on-write 机制，不会对 redis 本身有太多影响，顶多落盘时占用点 IO 资源而已。

潜在瓶颈点出现 fork()调用上

Under Linux, `fork()` is implemented using copy-on-write pages, so the only penalty that it incurs is the time and memory required to duplicate the parent's page tables, and to create a unique task structure for the child.

如果父进程的页表比较大, `fork()` 耗时就会相应延长, 且 `redis` 采用了单工作进程模型, `fork()` 执行期间会阻塞所有用户请求。

当前 `redis` 实例的 RSS 已经达到了 16G

```
top - 07:46:06 up 996 days, 16:51, 1 user, load average: 0.00, 0.00, 0.02
Tasks: 162 total, 1 running, 161 sleeping, 0 stopped, 0 zombie
Cpu(s): 0.5%us, 0.9%sy, 0.0%ni, 98.6%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st
Mem: 32878868k total, 23987712k used, 8891156k free, 394488k buffers
Swap: 4194300k total, 8180k used, 4186120k free, 5719452k cached

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
 8844 root        20   0 22.4g 16g 1388 S   0.0  51.6 48693:18 redis-server
```

页表大小 33M

```
cat /proc/8844/status | grep -i pte
VmPTE: 33840 kB
```

采用 `strace` 跟踪一下 `fork()` 耗时, 在 `glibc` 里 `fork` 的调用实际上映射到了更底层的 `clone` 系统调用, 因指定 `-e trace=clone`

```
# strace -p 20324 -e trace=clone -T
...
clone(child_stack=0, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0x7f409d771a10) = 30793 <1.013945>
...
```

于此对应的 `redis` 监控指标为

```
latest_fork_usec:1014778
```

两者吻合, 由此可以确认是 `redis` 定期 `bgsave` 引发的。最简单的方法是禁用 `bgsave`, 但这种行为有很大的风险, 一旦主库被误杀且在主从切换前又被迅速拉起, 会导致 `redis` 数据全部丢失。

查看 `redis` 的内存利用率, 存在很严重的内存碎片,

```
used_memory_human 8.7G
.....
used_memory_rss_human 16.1G
```

这套 `redis` 马上就要迁移, 新环境实例的 RSS 只有 8.8G, `latest_fork_usec` 指标也下降达到了 0.25s 左右, 和开发确认后可以满足应用需求, 迁移后应用的定期卡顿现象有了很明显的缓解。

`redis 4.0` 引入了碎片自动回收功能, 由参数 `activedefrag` 调控, 默认关闭。

迁移后对老 `redis` 开启了 `activedefrag`(其余参数保持默认值), 最终 `used_memory_rss_human` 固定在 11G 左右, `latest_fork_usec` 在 0.76s 左右。

新环境以后也可能会遭遇严重内存碎片, 届时要么开启 `activedefrag`, 要么在维护时间段重启实例, 后者效果显然更好一点。

3、小结

我们线上 redis 主从都开启了 bgsave，之前一直忽视了 bgsave/fork() 可能引发的性能波动，最好的解决办法就是控制单个 redis 的内存上限，如果业务端无法瘦身可以考虑 redis cluster 或者设置 maxmemory。

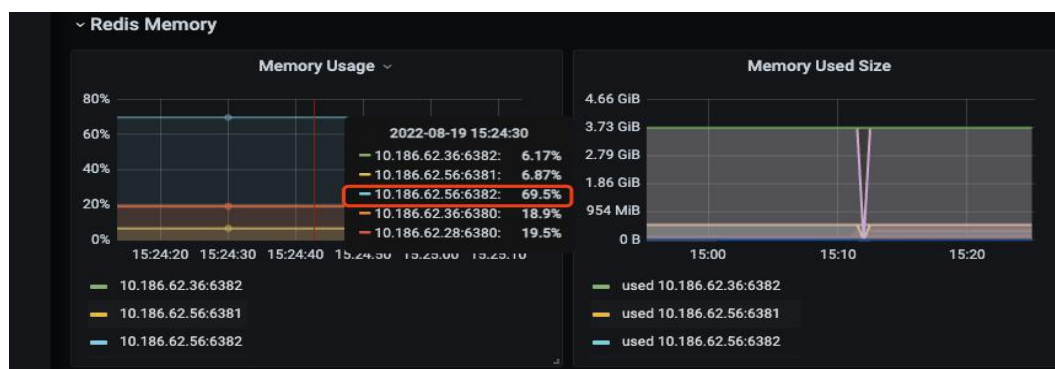
以后如果遇到了 redis 定期卡顿现象，可以优先从 latest_fork_usec 监控指标入手。

5.12 Redis Cluster 分片内存异常使用不均问题诊断

作者：任仲禹

背景

问题发生背景为某生产 Redis 集群（版本 Redis 5.0.10，架构为 30 片以上），该集群中某一个分片内存使用率异常高（内存占用达 70% 以上，其它片内存相对使用较低），我们模拟生产环境如下监控图所示：

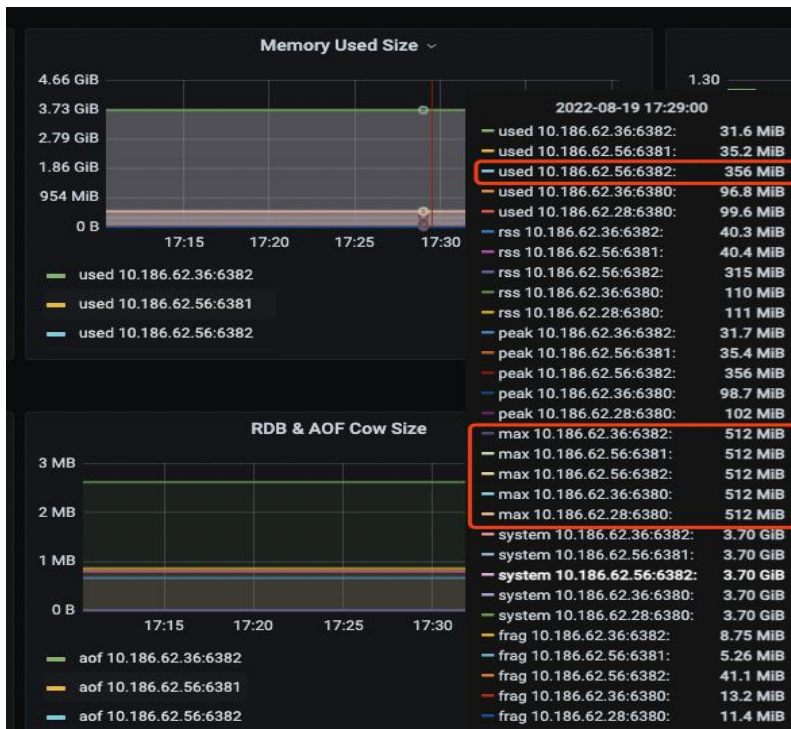


相信看文章标题大家都已知道问题结论，我这里想跟大家分享的是排查这种问题的方法。

诊断

内存使用分布监控

- 查看内存使用分布发现，该异常分片实例内存 Redis 使用为 356M 左右，单个 redis 最大可用内存 512M
- 其它正常分片 redis 内存使用为 100M 以内



异常与正常实例内存使用对比

- 观测到异常实例的数据量（info keyspace）相对还少一点
- 但异常实例 数据对象占用内存为其它正常实例 2 倍

正常实例

```
redis-cli -p 6380 -h 10.186.62.28 info keyspace ##数据量
# Keyspace
db0:keys=637147,expires=0,avg_ttl=0
redis-cli -p 6380 -h 10.186.62.28 info memory |grep -w used_memory ##内存使用
used_memory:104917416
```

异常实例

```
redis-cli -p 6382 -h 10.186.62.5 info keyspace ##数据量
# Keyspace
db0:keys=191433,expires=0,avg_ttl=0
redis-cli -p 6382 -h 10.186.62.56 info memory |grep -w used_memory ## 内存使用
used_memory:373672656
```

碎片率使用情况

- 异常实例内存碎片正常，排除碎片过多情况

```
redis-cli -p 6382 -h 10.186.62.56 info memory |grep mem_fragmentation_ratio
mem_fragmentation_ratio:0.89 ## 碎片率小于 1
```

Bigkeys 扫描分析

- 前面分析未果，尝试通过 bigkeys 分析扫描（为避免影响业务操作，建议业务低峰进行）

- 扫描结果如下（截取关键部分）

```
# redis-cli -p 6382 -h 10.186.62.56 --bigkeys

# Scanning the entire keyspace to find biggest keys as well as
# average sizes per key type. You can use -i 0.1 to sleep 0.1 sec
# per 100 SCAN commands (not usually needed).

[00.00%] Biggest string found so far '"key:{06S}:000061157249"' with 3 bytes
[00.03%] Biggest string found so far '"key3691"' with 4 bytes
[40.93%] Biggest string found so far '"bigkkkkk:0"' with 102400000 bytes
[51.33%] Biggest string found so far '"bigk:0"' with 204800000 bytes

----- summary -----

Sampled 191433 keys in the keyspace!
Total key length in bytes is 4161149 (avg len 21.74)

Biggest string found '"bigk:0"' has 204800000 bytes

0 lists with 0 items (00.00% of keys, avg size 0.00)
0 hashes with 0 fields (00.00% of keys, avg size 0.00)
191433 strings with 307777256 bytes (100.00% of keys, avg size 1607.75)
0 streams with 0 entries (00.00% of keys, avg size 0.00)
0 sets with 0 members (00.00% of keys, avg size 0.00)
0 zsets with 0 members (00.00% of keys, avg size 0.00)
```

- 其中获悉结果如下
 - 存在 2 个异常 key
 1. 如最大的 key 为 “bigk:0”，占用 200M 左右
 2. 大键“bigkkkkk:0”，占用 100M 左右
 - 其他 string key 相对较小，平均占用 10 字节以内，如 'key:xx' 等
- 注意，本文因为是自己模拟的测试环境相对简单，实质生产环境稍微复杂点，可能有不同类型的如 hash、set 类型的键，这些键通过 --bigkeys 分析工具后无法得到内存占用大小，而只能知道元素/成员个数，所以还需要通过其他命令获得内存占用大小：
 - 对异常 key 进行内存分析，结果如下，2 个异常 key 占用空间约 300M 左右，跟监控中内存异常占用高现象吻合

```
10.186.62.56:6382> memory usage bigkkkkk:0
(integer) 117440568
10.186.62.56:6382>
10.186.62.56:6382> memory usage bigk:0
(integer) 234881072
```

结论

- 通过上述分析流程，我们知道当发现 Redis Cluster 集群中内存分布不均时，分析 bigkeys 不失为一种快速有效的排查方法，但是需要注意在低峰期执行

- `redis-cli -p {port} -h {host} --bigkeys`

- BTW，如果大家需要模拟 Redis 的大键、大量数据、或阻塞可以通过一些好用的 `debug / mem` 命令

```
# 制造 10 条以 renzy:id: 为前缀，大小为 1024 字节的 key
```

```
127.0.0.1:9999> debug populate 10 renzy:id: 1024
```

```
OK
```

```
127.0.0.1:9999> keys renzy:id*
```

```
1) "renzy:id::8"
```

```
2) "renzy:id::2"
```

```
3) "renzy:id::4"
```

```
.....
```

```
## 制造阻塞
```

```
127.0.0.1:9999> debug sleep 2 //阻塞 2 秒
```

```
OK
```

```
或
```

```
# redis-cli -p 9999 keys \* > /dev/null
```

```
//（如果数据量大的话直接 keys *即可）
```